

С++ своими руками

От первой строки до четырёх живых консольных проектов — с разбором каждой строки

Серик Мурадов

Содержание

1	Вступление. Зачем эта книга	1
1.1	О чём она	1
1.2	Кому она	2
1.3	Почему C++ в 2026 году	3
1.4	Что мы построим — детальнее	4
1.4.1	Проект 1: текстовая RPG-игра demo-rpg/	4
1.4.2	Проект 2: мини-shell demo-shell/	5
1.4.3	Проект 3: мини-СУБД demo-db/	5
1.4.4	Проект 4: TCP-мессенджер demo-chat/	6
1.5	Что НЕ войдёт (и почему)	7
1.6	Какие у меня соглашения	7
1.7	Как читать	8
1.8	Что дальше	8
2	Глава 1. Что такое компьютер	9
2.1	Зачем это в книге про C++	9
2.2	Что такое компьютер на самом деле	9
2.2.1	Процессор (CPU)	10
2.2.2	Оперативная память (RAM)	11
2.2.3	Кэш процессора	11
2.2.4	Диск (SSD/HDD)	12
2.2.5	Операционная система	12
2.3	Что такое процесс	13
2.4	Что такое поток	14
2.5	Что увидите в C++	15
2.6	На пальцах: жизнь программы Hello	15
2.7	Маленькое упражнение	16
2.8	Что дальше	16
3	Глава 2. Биты, байты, числа, текст	18
3.1	Бит — самая маленькая единица	18
3.2	Байт — группа из 8 бит	19
3.3	Двоичная система счисления	19
3.4	Шестнадцатеричная система	20
3.5	Отрицательные числа: дополнительный код	21
3.6	Дробные числа: IEEE 754	22
3.7	Текст: ASCII	23
3.7.1	Чего не хватало в ASCII	23

3.7.2	Юникод и UTF-8	24
3.7.3	Что это значит для C++	25
3.8	Размеры и единицы	25
3.9	Что увидите в C++	25
3.10	Маленькое упражнение	26
3.11	Что дальше	26
4	Глава 3. Что такое программа и язык программирования	27
4.1	Программа — это последовательность инструкций	27
4.2	Машинный код	28
4.3	Ассемблер	28
4.4	Языки высокого уровня	29
4.4.1	В и С	30
4.4.2	C++	30
4.4.3	Другие популярные языки	31
4.5	Компиляция vs интерпретация	32
4.5.1	Компиляция	32
4.5.2	Интерпретация	32
4.5.3	JIT-компиляция (между двумя мирами)	33
4.5.4	Где здесь C++	33
4.6	Как программа становится исполняемой — общий контур	34
4.7	Линковка чуть подробнее	34
4.8	Стандартная библиотека	36
4.9	Что увидите в C++	36
4.10	Маленькое упражнение	37
4.11	Что дальше	37
5	Глава 4. Алгоритмическое мышление	38
5.1	Что такое алгоритм	38
5.2	Алгоритм в быту: заварить чай	39
5.3	Вход, обработка, выход	39
5.4	Переменные	40
5.5	Операции и выражения	40
5.6	Ветвление (условие)	41
5.7	Цикл	42
5.7.1	Бесконечный цикл	43
5.8	Функция	44
5.8.1	Функция, вызывающая функцию	44
5.9	Декомпозиция как главный навык	45
5.10	Псевдокод и реальный код	46
5.11	Состояние программы	46
5.12	Концепции, которые превратятся в C++	47
5.13	Алгоритм сложнее: найти максимум	47
5.14	Маленькое упражнение	48
5.15	Что дальше	49
6	Глава 5. Терминал, файлы, папки, права	50
6.1	Что такое терминал	50

6.2	Где он живёт	51
6.3	Приглашение к вводу (prompt)	51
6.4	Где я нахожусь — pwd	52
6.5	Что в папке — ls	52
6.6	Перемещение — cd	53
6.7	Пути — абсолютные и относительные	54
6.8	Создать папку — mkdir	55
6.9	Создать или открыть файл	55
6.10	Прочитать файл — cat	55
6.11	Копировать, перемещать, удалять — cp, mv, rm	56
6.12	Маски (wildcards)	57
6.13	Права доступа	57
6.13.1	chmod	58
6.13.2	chown	58
6.14	Переменные окружения	59
6.15	Перенаправление и пайпы	59
6.15.1	Стандартные потоки	59
6.15.2	Запись stdout в файл — >	60
6.15.3	Чтение stdin из файла — <	60
6.15.4	Пайпы — 	60
6.16	Запуск программы и ./	61
6.17	История команд	61
6.18	Автодополнение	61
6.19	Очистить экран	62
6.20	Прервать команду	62
6.21	Полезный набор команд	62
6.22	Маленькое упражнение	63
6.23	Что дальше	64
7	Глава 6. Инструменты и первая программа	65
7.1	Что нам нужно	65
7.2	macOS	66
7.3	Linux	66
7.4	Windows	67
7.5	Редактор	68
7.6	Структура проекта	68
7.7	Первая программа — hello.cpp	69
7.7.1	Строка 1: #include <iostream>	69
7.7.2	Строка 2: пустая	70
7.7.3	Строка 3: int main() {	70
7.7.4	Строка 4: std::cout << "Hello, world!" << std::endl;	70
7.7.5	Строка 5: return 0;	71
7.7.6	Строка 6: }	72
7.8	Компиляция	72
7.9	Запуск	73
7.10	Что произошло — большая картинка	73
7.11	Что если ошибка	74
7.12	Запоминания на сегодня	75

7.13	Маленькое упражнение	75
7.14	Что дальше	75
8	Глава 7. Как программа становится исполняемым файлом	77
8.1	Подопытная программа	77
8.2	Стадия 1: препроцессор	79
8.2.1	Посмотрим своими глазами	79
8.2.2	Макросы	80
8.2.3	Условная компиляция	81
8.2.4	Include guard детальнее	82
8.2.5	Что выучить про препроцессор	83
8.3	Стадия 2: собственно компиляция	83
8.3.1	Манглинг имён	85
8.3.2	Что выучить про компиляцию	85
8.4	Стадия 3: ассемблирование	86
8.4.1	Внутри объектного файла	86
8.4.2	Смотрим на символы	86
8.4.3	Утилита objdump	87
8.5	Стадия 4: линковка	88
8.5.1	А стандартная библиотека?	88
8.5.2	Сообщения об ошибках линковки	89
8.5.3	Статическая и динамическая линковка	89
8.5.4	Что выучить про линковку	91
8.6	Memory layout: что где лежит в памяти процесса	91
8.6.1	.text (код)	91
8.6.2	.rodata (константы)	92
8.6.3	.data (инициализированные глобальные)	92
8.6.4	.bss (неинициализированные глобальные)	92
8.6.5	Куча (heap)	92
8.6.6	Мmap-области	93
8.6.7	Стек (stack)	93
8.6.8	Картинка целиком	93
8.6.9	Утилита size	94
8.6.10	Стек: размер и переполнение	94
8.7	Полная картина: что происходит при сборке	95
8.8	Один важный нюанс: единицы трансляции	95
8.9	Маленькое упражнение	96
8.10	Что дальше	97
9	Глава 8. Переменные и типы	98
9.1	Почему в C++ типы такие важные	98
9.2	Целые числа: семейство int	99
9.2.1	Проверяем sizeof	99
9.2.2	signed и unsigned	100
9.2.3	Когда использовать unsigned	101
9.2.4	Целочисленные типы фиксированного размера	102
9.2.5	Литералы целых чисел	103
9.3	Дробные числа: float и double	103

9.3.1	Литералы дробных чисел	104
9.3.2	Точность — самое больное место	104
9.4	bool	105
9.5	char и символы	106
9.5.1	Escape-последовательности	106
9.5.2	char, signed char, unsigned char	107
9.6	const — переменная, которую нельзя менять	107
9.7	constexpr — константа времени компиляции (C++11)	108
9.8	Способы инициализации	109
9.8.1	Старый способ: с равно	109
9.8.2	С круглыми скобками	109
9.8.3	Единообразная инициализация (C++11): с фигурными скобками	109
9.8.4	Default initialization vs value initialization	110
9.9	auto — компилятор сам выводит тип (C++11)	111
9.9.1	Когда auto хороший выбор	111
9.9.2	Когда auto плохой выбор	111
9.9.3	auto и const/&	112
9.10	Имена переменных: правила и стиль	112
9.11	Полный собранный пример	113
9.12	Преобразования типов: коротко	114
9.13	Главное — что увидите в коде дальше	115
9.14	Маленькое упражнение	116
9.15	Что дальше	117
10	Глава 9. Управляющие конструкции	118
10.1	if — простое ветвление	118
10.2	if/else	119
10.3	Логические операции в условиях	120
10.3.1	Ловушка номер один: = vs ==	121
10.3.2	Short-circuit evaluation	121
10.3.3	Скобки и приоритет	122
10.4	Дополнительный конструктор: if с инициализацией (C++17)	122
10.5	Тернарный оператор	123
10.6	switch — выбор из множества значений	124
10.6.1	Ограничения switch	125
10.6.2	Fall-through — главная ловушка switch	125
10.6.3	default	126
10.7	while — цикл с проверкой в начале	127
10.7.1	Бесконечный цикл нарочно	128
10.8	do-while — цикл с проверкой в конце	129
10.9	for — классический цикл	129
10.9.1	Все три части — опциональны	130
10.9.2	Несколько переменных в for	130
10.10	Range-based for — главная новинка C++11	131
10.10.1	По ссылке vs по значению	131
10.10.2	Если нужен индекс — классический for	132
10.11	break и continue	133
10.12	goto — реабилитация чёрной овцы	134

10.13	Циклы с итераторами (предварительно)	136
10.14	Что выучить про управляющие конструкции	136
10.15	Пара рекомендаций	137
10.16	Пример: «угадай число»	137
10.17	Маленькое упражнение	139
10.18	Что дальше	140
11	Глава 10. Функции	141
11.1	Зачем нужны функции	141
11.2	Анатомия функции	141
11.2.1	Несколько параметров	142
11.2.2	void — функция без возврата	142
11.2.3	Функция без параметров	143
11.3	Объявление и определение	143
11.3.1	Разделение между файлами	145
11.4	Передача параметров: три способа	146
11.4.1	Способ 1: по значению	146
11.4.2	Способ 2: по ссылке	147
11.4.3	Ключевые свойства ссылок	148
11.4.4	По ссылке для эффективности	148
11.4.5	Способ 3: по указателю	149
11.4.6	Указатель vs ссылка	149
11.4.7	Когда что использовать	150
11.4.8	Сводная таблица	150
11.4.9	Правило передачи параметров	150
11.5	Возврат из функции	151
11.5.1	Несколько возвращаемых значений	152
11.6	Перегрузка функций	153
11.7	Параметры по умолчанию	154
11.8	inline	155
11.9	Рекурсия	156
11.9.1	Хвостовая рекурсия	157
11.10	Forward declaration	158
11.11	main — особая функция	158
11.12	Лямбды — короткое упоминание	159
11.13	Указатели на функции	159
11.14	Чистые функции и побочные эффекты	160
11.15	Большой пример: банковская симуляция	160
11.16	Маленькое упражнение	163
11.17	Что дальше	163
12	Глава 11. Память — стек, куча, RAII	164
12.1	Освежим картинку из главы 7	164
12.2	Стек: подробности	165
12.2.1	Стек растёт «вниз»	166
12.2.2	Стек ограничен	166
12.2.3	Преимущества стека	166
12.2.4	Недостатки стека	167

12.3	Куча: подробности	167
12.3.1	new — выделение	167
12.3.2	delete — возврат	168
12.3.3	new[] и delete[] — массивы	168
12.3.4	Размер можно вычислять в рантайме	169
12.4	Утечки памяти	169
12.4.1	Когда утечки нестрашны	170
12.4.2	Пять способов утечь память	170
12.4.3	Висячие указатели	171
12.4.4	Двойное освобождение	171
12.4.5	Использование после освобождения (UAF)	172
12.5	Проблема: слишком много ручной работы	172
12.6	RAII — главный паттерн C++	172
12.6.1	Конструктор и деструктор	172
12.6.2	Почему RAII работает с исключениями	174
12.7	Стандартные RAII-обёртки	174
12.7.1	Пример: вектор вместо ручного массива	175
12.7.2	Пример: файл вместо fopen	175
12.8	smart pointers — предварительно	176
12.9	Стек или куча — что выбрать	176
12.9.1	Конкретные примеры	177
12.10	Инструменты для борьбы с проблемами памяти	177
12.10.1	AddressSanitizer (ASan)	177
12.10.2	Valgrind	178
12.10.3	LeakSanitizer (LSan)	178
12.11	Большой пример: классический ресурс	178
12.11.1	C-стиль (без RAII)	178
12.11.2	C++-стиль (с RAII)	179
12.11.3	Вариант со стандартной библиотекой	180
12.12	Когда нужно писать свой деструктор	181
12.13	Главные правила этой главы	181
12.14	Маленькое упражнение	182
12.15	Что дальше	182
13	Глава 12. Строки и потоки ввода-вывода	184
13.1	Старая школа: C-strings	184
13.2	std::string — основа основ	185
13.2.1	Создание и базовые операции	185
13.2.2	Конкатенация	186
13.2.3	Индексация и обход	186
13.2.4	Сравнение	187
13.2.5	Подстроки	188
13.2.6	Поиск	188
13.2.7	Замена и вставка	188
13.2.8	Преобразование к C-строке	188
13.2.9	Преобразования с числами	189
13.2.10	Что лежит внутри std::string	189
13.2.11	Итог по строкам	189

13.3	<code>std::cout</code> — вывод	190
13.3.1	<code>endl</code> против <code>\n</code>	190
13.3.2	<code>std::cerr</code> — поток ошибок	191
13.3.3	Форматирование вывода	191
13.4	<code>std::cin</code> — ввод	192
13.4.1	Несколько значений подряд	193
13.4.2	Чтение строки целиком: <code>getline</code>	193
13.4.3	Главная ловушка <code>cin</code> : смешивание » и <code>getline</code>	194
13.5	Файловые потоки	194
13.5.1	Чтение из файла	195
13.5.2	Запись в файл	195
13.5.3	Бинарные данные	196
13.5.4	Полностью прочитать файл	197
13.6	<code>std::stringstream</code> — строка как поток	197
13.6.1	Форматированная сборка строк	198
13.7	Сводный пример: разобрать CSV	198
13.8	Пара тонкостей	200
13.8.1	Чтение чисел и точка в качестве разделителя	200
13.8.2	Юникод и <code>std::string</code>	201
13.8.3	<code>sync_with_stdio</code>	201
13.9	Главные правила главы	201
13.10	Маленькое упражнение	202
13.11	Что дальше	202
14	Глава 13. Контейнеры STL	203
14.1	Зачем контейнеры	203
14.2	<code>std::vector</code> — главный контейнер	204
14.2.1	Создание	204
14.2.2	Доступ к элементам	204
14.2.3	Добавление и удаление	205
14.2.4	Размер и ёмкость	205
14.2.5	<code>reserve</code> и оптимизация	206
14.2.6	Инвалидация итераторов и указателей	206
14.2.7	Почему <code>vector</code> обычно быстрее списка	207
14.2.8	Удаление-стирание идиома (<code>erase-remove</code>)	208
14.3	<code>std::array</code> — массив фиксированного размера	208
14.4	<code>std::deque</code> — двусторонняя очередь	209
14.5	<code>std::list</code> — двусвязный список	209
14.6	<code>std::map</code> — упорядоченный словарь	210
14.6.1	Ловушка <code>[]</code> на <code>map</code>	211
14.6.2	Вставка с проверкой существования	211
14.7	<code>std::unordered_map</code> — хеш-таблица	212
14.7.1	Ключи в <code>unordered_map</code>	212
14.7.2	Что внутри <code>unordered_map</code>	212
14.8	<code>std::set</code> и <code>std::unordered_set</code>	213
14.9	Итераторы — кратко	213
14.10	<code>std::pair</code> и <code>std::tuple</code>	214
14.11	Сравнительная таблица	215

14.12	Решающее дерево: какой контейнер брать	215
14.13	Распространённые операции	216
14.13.1	Найти элемент	216
14.13.2	Сортировка	216
14.13.3	Сумма	217
14.13.4	Подсчёт	217
14.13.5	Уникальные значения	217
14.14	Большой пример: подсчёт частоты слов	217
14.15	Главные правила главы	219
14.16	Маленькое упражнение	219
14.17	Что дальше	220
15	Глава 14. Алгоритмы и итераторы	221
15.1	Зачем алгоритмы — снова про переиспользование	221
15.2	Итераторы — обобщённый указатель	222
15.2.1	Что такое <code>begin()</code> и <code>end()</code>	222
15.3	Категории итераторов	223
15.3.1	Input iterator	223
15.3.2	Output iterator	223
15.3.3	Forward iterator	223
15.3.4	Bidirectional iterator	223
15.3.5	Random access iterator	223
15.3.6	Иерархия	224
15.4	Алгоритмы без модификации	224
15.4.1	<code>find</code> — найти первый	224
15.4.2	<code>find_if</code> — найти с предикатом	224
15.4.3	<code>count</code> и <code>count_if</code>	225
15.4.4	<code>all_of</code> , <code>any_of</code> , <code>none_of</code>	225
15.4.5	<code>min_element</code> и <code>max_element</code>	225
15.4.6	<code>equal</code> — сравнить два диапазона	225
15.5	Алгоритмы с модификацией	226
15.5.1	<code>fill</code> — заполнить значением	226
15.5.2	<code>iota</code> — заполнить последовательно	226
15.5.3	<code>transform</code> — применить функцию	226
15.5.4	<code>copy</code> и <code>copy_if</code>	227
15.5.5	<code>remove</code> и <code>remove_if</code>	227
15.5.6	<code>reverse</code> и <code>rotate</code>	228
15.5.7	<code>sort</code> и связанные	228
15.5.8	<code>unique</code>	229
15.6	Численные алгоритмы	229
15.6.1	<code>accumulate</code>	229
15.6.2	<code>inner_product</code>	230
15.6.3	<code>partial_sum</code>	230
15.6.4	<code>adjacent_difference</code>	231
15.7	Перестановки	231
15.8	Бинарный поиск	231
15.9	Лямбды и предикаты — короткое введение	232
15.9.1	Функтор — старый стиль	232

15.10	Алгоритмы vs методы контейнеров	233
15.11	Удалить элемент из map во время прохода	233
15.12	Свой for_each — попытка номер один	234
15.13	Производительность	234
15.13.1	Алгоритмы STL часто оптимизированнее ручных	234
15.13.2	Алгоритмы вызывают функцию для каждого элемента	234
15.13.3	Когда STL медленнее ручного	235
15.14	Большой пример: статистика оценок	235
15.15	Главные правила	238
15.16	Маленькое упражнение	238
15.17	Часть I закрыта	238
15.18	Что дальше	239
16	Глава 15. Дизайн игры и скелет проекта	240
16.1	Что мы строим	240
16.2	Архитектура: какие классы будут	241
16.2.1	Связи между классами	242
16.3	Файловая структура	242
16.4	Создаём папки	243
16.5	Makefile	244
16.5.1	Простой пример	244
16.5.2	-MMD — магия авто-зависимостей	245
16.5.3	Наш Makefile	245
16.6	Скелет кода	250
16.6.1	include/game.h	250
16.6.2	src/game.cpp	251
16.6.3	src/main.cpp	253
16.6.4	tests/test_smoke.cpp	254
16.7	Сборка и проверка	254
16.7.1	Release-сборка	255
16.8	Тонкости и расширения	256
16.8.1	Поддержка отладчика	256
16.8.2	AddressSanitizer	256
16.8.3	Зачем не CMake	256
16.8.4	Зачем не header-only	256
16.9	Главные правила главы	257
16.10	Маленькое упражнение	257
16.11	Что дальше	258
17	Глава 16. Классы — Player, Location, World	259
17.1	struct и class	259
17.2	Простой класс Player	260
17.2.1	Объявление	260
17.2.2	Поля	261
17.2.3	Методы доступа (getters)	261
17.2.4	Модифицирующие методы	262
17.2.5	Конструктор	262
17.2.6	Определение	263

17.2.7	Что такое this	266
17.2.8	Использование Player	266
17.3	Локация: класс посложнее	267
17.3.1	Объявление	267
17.3.2	Определение	268
17.3.3	Использование	269
17.4	Перегрузка операторов — короткий взгляд	269
17.5	Конструктор по умолчанию	270
17.5.1	= delete	271
17.6	Копирование: rule of three	271
17.6.1	Когда rule of zero ломается	272
17.6.2	Rule of five (C++11)	273
17.6.3	Запрет копирования	274
17.7	World: контейнер локаций	274
17.7.1	Объявление	274
17.7.2	find возвращает указатель	275
17.7.3	Заводская функция	275
17.7.4	emplace вместо insert+move	276
17.7.5	Default-конструктор через = default	277
17.8	Game: всё вместе	277
17.8.1	Объявление	277
17.8.2	Конструктор	278
17.8.3	Команды	278
17.9	Сборка и запуск	280
17.10	Что освоено в этой главе	281
17.11	Маленькое упражнение	281
17.12	Что дальше	282
18	Глава 17. Наследование и виртуальные функции	283
18.1	Зачем наследование	283
18.2	Базовый синтаксис наследования	284
18.2.1	public, protected, private — два смысла	285
18.3	Конструктор базы	286
18.4	Виртуальные функции	286
18.4.1	override — обязательная привычка	288
18.4.2	final — запрет дальнейшего переопределения	288
18.5	Чисто виртуальные функции и абстрактные классы	289
18.6	Виртуальный деструктор — обязателен!	290
18.7	vtable — как это работает	291
18.8	Слайсинг — главная ловушка	292
18.9	dynamic_cast — приведение базы к потомку	293
18.10	Множественное наследование — кратко	294
18.11	Item-иерархия в проекте	295
18.12	Тестируем полиморфизм	298
18.13	Несколько тонкостей	299
18.13.1	Виртуальный вызов в конструкторе/деструкторе	299
18.13.2	Вызов метода базы из потомка	300
18.13.3	protected и инвариант класса	300

18.14	Альтернативы наследованию	301
18.15	Главные правила главы	301
18.16	Маленькое упражнение	302
18.17	Что дальше	302
19	Глава 18. Умные указатели — <code>unique_ptr</code>, <code>shared_ptr</code>, <code>weak_ptr</code>	303
19.1	Зачем нужны умные указатели	303
19.2	<code>unique_ptr</code> — основной рабочий	304
19.2.1	Создание	304
19.2.2	Запрет копирования	305
19.2.3	Перемещение разрешено	305
19.2.4	<code>make_unique</code> — фабричная функция	306
19.2.5	Кастомные <code>deleters</code>	306
19.2.6	<code>unique_ptr</code> для массивов	307
19.3	<code>shared_ptr</code> — совладение	307
19.3.1	<code>make_shared</code>	308
19.3.2	Цена <code>shared_ptr</code>	308
19.3.3	Циклы — главная ловушка <code>shared_ptr</code>	308
19.4	<code>weak_ptr</code> — наблюдатель без владения	309
19.5	Когда что	310
19.6	<code>Inventory</code> — наш первый <code>shared_ptr</code> ... ах нет, <code>unique_ptr</code>	310
19.6.1	<code>add</code> — принимаем владение	311
19.6.2	<code>take</code> — отдаём владение	311
19.6.3	<code>find</code> — наблюдение без владения	312
19.6.4	Лямбда с захватом по ссылке	312
19.7	Применение в <code>Location</code> и <code>Player</code>	313
19.8	Команды <code>take</code> / <code>drop</code> / <code>use</code>	314
19.9	Запуск	315
19.10	Сырой указатель — когда уместен	316
19.11	Тонкость: передать <code>unique_ptr</code> нельзя по значению, можно только через <code>move</code>	316
19.12	<code>shared_ptr</code> — куда мы его не применили	316
19.13	<code>weak_ptr</code> — куда мы его не применили	317
19.14	Главные правила главы	317
19.15	Маленькое упражнение	317
19.16	Что дальше	318
20	Глава 19. Move-семантика на бою	319
20.1	Зачем <code>move</code>	319
20.2	<code>lvalue</code> и <code>rvalue</code>	320
20.3	Главное различие на практике	321
20.4	<code>Rvalue</code> -ссылки: <code>T&&</code>	321
20.4.1	Что внутри <code>rvalue</code> -ссылки	321
20.5	<code>Move</code> -конструктор и <code>move</code> -присваивание	322
20.5.1	поexcept на <code>move</code> -конструкторе	323
20.5.2	<code>Move</code> -присваивание	323
20.6	<code>std::move</code> — что это на самом деле	324
20.7	Когда <code>move</code> бесплатен	325
20.8	Rule of five детально	325

20.8.1	Когда компилятор сам генерирует move	326
20.9	RVO и NRVO	326
20.9.1	Когда RVO не работает	327
20.10	Perfect forwarding и std::forward	328
20.11	std::move vs std::forward	328
20.12	Что после move	329
20.13	Демонстрация	329
20.14	Где move применяется в нашем коде	331
20.15	Главные правила главы	332
20.16	Маленькое упражнение	333
20.17	Что дальше	333
21	Глава 20. Шаблоны — EventBus	334
21.1	Зачем шаблоны	334
21.2	typename vs class	335
21.3	Где живут шаблоны: только в заголовках	336
21.4	Шаблоны классов	337
21.5	Дедукция типа	338
21.6	Специализация	339
21.6.1	Частичная специализация	339
21.7	Variadic templates	340
21.7.1	Variadic в нашем make_unique	341
21.8	SFINAE — кратко	341
21.9	Ошибки шаблонов	342
21.10	EventBus	343
21.10.1	События	344
21.10.2	Тест	344
21.11	Концепции и применения	345
21.12	Интеграция в проект (предварительно)	346
21.13	Сторонние шаблонные паттерны	346
21.13.1	Тип-безопасное хранение разнотипных данных	346
21.13.2	Типовые функции	347
21.13.3	Generic algorithms	347
21.14	Шаблоны и компилятор	348
21.15	Главные правила главы	348
21.16	Маленькое упражнение	348
21.17	Что дальше	349
22	Глава 21. Лямбды и std::function	350
22.1	Лямбда — что это	350
22.2	Анатомия захвата	351
22.2.1	[] — без захвата	351
22.2.2	[x] — захват по значению	351
22.2.3	[&x] — захват по ссылке	352
22.2.4	[=] — захват всего по значению	352
22.2.5	[&] — захват всего по ссылке	352
22.2.6	[this] — захват указателя на текущий объект	353
22.2.7	[x, &y] — комбинированный	353

22.2.8	[x = ...] — захват с инициализатором (C++14)	353
22.3	Что лямбда на самом деле	353
22.4	Производительность	354
22.5	std::function — type erasure	355
22.5.1	Цена type erasure	355
22.5.2	Когда использовать что	355
22.6	Лямбды как параметры функций	356
22.7	std::bind — старый стиль (для контекста)	356
22.8	NPC и поведения — намёк на будущее	357
22.9	CommandRegistry — лямбды на работе	357
22.9.1	Регистрация команд	359
22.9.2	Парсинг	360
22.9.3	help автоматический	360
22.9.4	Что выиграли	360
22.10	Запуск	361
22.11	Главные правила главы	361
22.12	Маленькое упражнение	362
22.13	Что дальше	362
23	Глава 22. Исключения и exception safety	363
23.1	Зачем исключения	363
23.2	throw, try, catch	364
23.2.1	Ловите по ссылке, не по значению	365
23.2.2	Несколько catch	365
23.2.3	Rethrow	366
23.3	std::exception и иерархия	366
23.4	Свои типы исключений	367
23.4.1	Базовый тип runtime_error	368
23.5	Применяем: валидация в Player	368
23.6	Главный try/catch в main	369
23.7	Stack unwinding под капотом	370
23.8	ноэкхепт — обещание не бросать	370
23.9	Exception safety — три уровня гарантий	371
23.9.1	Уровень 1: basic guarantee — слабая	371
23.9.2	Уровень 2: strong guarantee — сильная	371
23.9.3	Уровень 3: nothrow guarantee — без бросков	372
23.9.4	Какая гарантия предпочтительна	372
23.10	Copy-and-swap идиома детально	372
23.11	Цена исключений	373
23.12	Почему отключают исключения	374
23.13	Когда что выбирать: исключение vs код ошибки vs assert	374
23.14	Главные правила главы	374
23.15	Маленькое упражнение	375
23.16	Что дальше	375
24	Глава 23. Парсер скриптов локаций	376
24.1	Зачем парсер	376
24.2	Выбор формата	377

24.3	Пример мира	377
24.4	Архитектура парсера	379
24.5	Заголовок	379
24.6	Реализация: вспомогательные функции	380
24.7	Промежуточное состояние	381
24.8	Парсинг строки предмета	381
24.9	Главный цикл парсера	382
24.9.1	name и desc	384
24.9.2	exit	385
24.9.3	item	385
24.9.4	Конец файла	385
24.10	Подключение к Game	385
24.11	Проверка	386
24.12	Уроки про парсинг	387
24.12.1	Состояния парсера	387
24.12.2	Сообщения об ошибках	387
24.12.3	Иммутабельность данных	387
24.12.4	Tokenizer vs парсер	387
24.12.5	Robust vs strict парсер	388
24.13	std::variant — упоминание (C++17)	388
24.14	Главные правила главы	388
24.15	Маленькое упражнение	389
24.16	Что дальше	389
25	Глава 24. Сохранение и загрузка игры	390
25.1	Что сохранять	390
25.2	Текстовый или бинарный формат	390
25.3	Формат сейва	391
25.4	Зачем версия	392
25.5	Зачем контрольная сумма	392
25.6	API менеджера сохранений	393
25.7	Реализация write	394
25.7.1	Сериализация предмета	395
25.7.2	Запись в файл	395
25.7.3	Эскейпинг	396
25.8	Реализация read	396
25.8.1	Двухпроходный парсер	397
25.8.2	Replace player	398
25.9	Команды в Game	398
25.10	Тест	399
25.11	Поломка checksum	400
25.12	Что осталось	401
25.13	Бинарный формат — кратко	401
25.14	Главные правила главы	402
25.15	Маленькое упражнение	402
25.16	Что дальше	402
26	Глава 25. Финал RPG	404

26.1	Зачем юнит-тесты	404
26.2	GoogleTest vs свой простой framework	405
26.3	Свой test-framework	405
26.4	Юнит-тесты модулей	407
26.4.1	Парсер и save/load	408
26.4.2	main и запуск	410
26.5	Сборка на нескольких платформах	410
26.6	Что мы построили — обзор	412
26.7	Что покрыли в C++	412
26.7.1	Часть I — основы:	413
26.7.2	Часть II — продвинутое:	413
26.8	Что осталось «за кадром»	413
26.8.1	Бой	414
26.8.2	NPC и диалоги	414
26.8.3	Экипировка	414
26.8.4	Квесты	414
26.8.5	Множество персонажей	414
26.8.6	События с EventBus	414
26.8.7	Графика (terminal)	414
26.8.8	Графика (графическая)	415
26.8.9	Сетевая игра	415
26.9	Если бы я писал это профессионально	415
26.10	Итог Части II	415
26.11	Главные правила главы	416
26.12	Маленькое упражнение	416
26.13	Что дальше	417
27	Глава 26. Процессы — fork, exec, wait	418
27.1	Зачем shell	418
27.2	Где не сработает	419
27.3	Что такое процесс — снова	419
27.4	Системный вызов fork	420
27.4.1	Что значит «копия»	420
27.4.2	Что наследуется от родителя	421
27.4.3	Где работа после fork разделяется	421
27.5	exec — заменить образ	421
27.5.1	_exit vs exit	422
27.5.2	Каноническая пара fork + exec	422
27.6	wait и waitpid — собрать статус	422
27.7	Зомби и orphans	423
27.8	Минимальный shell	424
27.8.1	main и REPL	426
27.9	Сборка и проверка	427
27.10	Что происходит под капотом	428
27.11	Тонкости	429
27.11.1	Почему стандартные потоки наследуются	429
27.11.2	Среда (environment)	429
27.11.3	Прерывание ребёнка	429

27.11.4	Что с zsh / bash подобными фичами	429
27.12	fork без exes	429
27.13	Главные правила главы	430
27.14	Маленькое упражнение	430
27.15	Что дальше	431
28	Глава 27. Файловые дескрипторы и pipes	432
28.1	Что такое файловый дескриптор	432
28.1.1	Открыть файл	433
28.2	fd наследуются через fork и exes	433
28.3	pipe() — создать пайп	434
28.3.1	EOF на пайпе	435
28.4	dup2 — перенаправить fd	435
28.5	A B — пайплайн на две команды	436
28.6	A B C — длинный пайплайн	437
28.7	Реализация в demo-shell	438
28.7.1	parse_pipeline	438
28.7.2	run_pipeline	439
28.8	Тест	441
28.9	Тонкости пайпов	441
28.9.1	Зомби пайплайн	441
28.9.2	Размер буфера	442
28.9.3	SIGPIPE	442
28.9.4	Когда пайп — overkill	442
28.9.5	Внутри одной программы — fork+pipe	442
28.10	Главные правила главы	443
28.11	Маленькое упражнение	443
28.12	Что дальше	444
29	Глава 28. Сигналы	445
29.1	Что такое сигнал	445
29.2	Распространённые сигналы	446
29.3	Как послать сигнал	447
29.4	Старый API: signal()	447
29.5	Современный API: sigaction()	448
29.6	Что можно делать в обработчике	448
29.7	SA_RESTART и системные вызовы	450
29.8	Игнорирование сигнала	450
29.9	SIGCHLD и зомби	450
29.10	SIGPIPE	451
29.11	Применение в demo-shell	451
29.12	Что в итоге	453
29.13	Тонкости и углы	453
29.13.1	Заранее заблокированные сигналы	453
29.13.2	Group leader и tcsetpgrp	454
29.13.3	SIGCHLD в реальном shell	454
29.13.4	sigaction с siginfo	454
29.14	Главные правила главы	454

29.15	Маленькое упражнение	455
29.16	Что дальше	455
30	Глава 29. Парсер командной строки	456
30.1	Что должен парсер	456
30.2	Этапы парсинга	457
30.3	Tokenizer	457
30.3.1	Незакрытые кавычки	461
30.3.2	Тестирование tokenize	461
30.4	Parser	461
30.5	Применение редиректов	463
30.5.1	Порядок важен	464
30.6	Редиректы и пайпы	465
30.7	Чего парсер пока не умеет	466
30.7.1	Последовательность команд: ;, &&, 	466
30.7.2	Подстановка переменных: \$HOME	466
30.7.3	Globbering: *, ?, [abc]	467
30.7.4	< /dev/null, heredoc <<EOF	467
30.7.5	& фон	467
30.8	Запуск	467
30.9	Архитектура парсера в целом	468
30.10	Главные правила главы	469
30.11	Маленькое упражнение	469
30.12	Что дальше	470
31	Глава 30. Builtins и история	471
31.1	Зачем builtins	471
31.2	API: try_builtin	472
31.2.1	Почему так	472
31.3	Builtins реализации	472
31.3.1	cd	472
31.3.2	pwd	473
31.3.3	export	474
31.3.4	echo	475
31.3.5	exit	475
31.4	История команд	475
31.4.1	В памяти	475
31.4.2	Загрузка/сохранение	476
31.4.3	Builtin history	476
31.4.4	В main	477
31.4.5	Альтернативный подход	477
31.5	Навигация стрелками — кратко	478
31.6	Job control — кратко	478
31.7	Что пользователь видит	478
31.8	Builtins vs внешние — производительность	480
31.9	Главные правила главы	480
31.10	Маленькое упражнение	480
31.11	Что дальше	481

32	Глава 31. Свои утилиты — cat, wc, grep	482
32.1	Зачем писать свои	482
32.2	Структура	483
32.3	myscat — простейший фильтр	483
32.3.1	Проверка	486
32.3.2	Почему read/write, а не std::ifstream	486
32.4	mywc — счётчик	486
32.4.1	main	488
32.4.2	Проверка	489
32.5	mygrep — фильтр по шаблону	489
32.5.1	Компиляция regex	491
32.5.2	Проверка	491
32.6	Полный пайплайн через myshell	491
32.7	Что мы не реализовали	492
32.8	mmap для скорости	492
32.9	Архитектура «маленьких программ»	493
32.10	Главные правила главы	494
32.11	Маленькое упражнение	494
32.12	Часть III закрыта	494
32.13	Что дальше	495
33	Глава 32. Бинарный формат данных	496
33.1	Текст vs бинарь	496
33.2	Что такое struct в памяти	497
33.3	alignof, alignas, offsetof	498
33.4	Packed-структуры	498
33.5	Проблема №1: raw struct dump	499
33.5.1	1. Padding бит — это мусор	499
33.5.2	2. Размер зависит от платформы	500
33.5.3	3. Endianness	500
33.5.4	4. Алайнмент зависит от ABI	500
33.5.5	Вывод	500
33.6	Правильный подход: explicit serialization	500
33.7	Sign vs unsigned, signed numbers	502
33.8	htonl и подобные	502
33.9	Пример: всё вместе	503
33.10	Hexdump для отладки	504
33.11	Versioning формата	504
33.12	Что увидите в следующих главах	504
33.13	Главные правила главы	505
33.14	Маленькое упражнение	505
33.15	Что дальше	506
34	Глава 33. Страничный файл и LRU-кэш	507
34.1	Зачем страницы	507
34.2	PAGE_SIZE = 4096	508
34.3	Адресация страниц	508
34.4	pread / pwrite	508

34.5	Кэш — зачем	509
34.6	LRU как стратегия	510
34.6.1	Реализация LRU	510
34.7	Dirty pages	511
34.8	fsync — durability	511
34.9	Класс PageManager	512
34.9.1	Открытие файла	513
34.9.2	Загрузка страницы	514
34.9.3	Выделение новой страницы	514
34.10	Демо	515
34.11	Cache size: сколько страниц держать	516
34.12	Тонкости и неприятности	516
34.12.1	Page tearing	516
34.12.2	Write amplification	516
34.12.3	O_DIRECT	517
34.13	Главные правила главы	517
34.14	Маленькое упражнение	517
34.15	Что дальше	518
35	Глава 34. B+tree	519
35.1	Зачем B+tree	519
35.2	B-tree vs B+tree	520
35.3	Структура узла	520
35.4	MAX_KEYS — порядок	520
35.5	Поиск	521
35.6	Вставка	522
35.6.1	Split leaf	523
35.6.2	Split internal	524
35.6.3	Insert in parent	524
35.7	Range scan	526
35.8	Удаление — кратко	526
35.9	Демо	527
35.10	Дисковое дерево — что нового	527
35.11	Performance estimates	528
35.12	Тонкости	528
35.12.1	Биение по середине	528
35.12.2	Concurrency	528
35.12.3	Variable-length keys	529
35.12.4	Prefix compression	529
35.13	Главные правила главы	529
35.14	Маленькое упражнение	529
35.15	Что дальше	530
36	Глава 35. Write-Ahead Log	531
36.1	Проблема, которую решает WAL	531
36.2	ACID — короткое напоминание	531
36.3	Идея WAL	532
36.4	Формат записи WAL	533

36.5	Алгоритм replay	533
36.6	Truncation после checkpoint	533
36.7	Group commit	534
36.8	Класс WAL	534
36.8.1	log_insert	535
36.8.2	replay	535
36.8.3	truncate	536
36.9	Демо: имитация краха	537
36.9.1	Запуск	538
36.10	Ещё уровни сложности	539
36.10.1	LSN	539
36.10.2	Physical vs logical WAL	539
36.10.3	Checkpoint protocol	539
36.10.4	Full page writes	539
36.10.5	WAL archiving	540
36.11	Главные правила главы	540
36.12	Маленькое упражнение	540
36.13	Что дальше	541
37	Глава 36. Мини-SQL	542
37.1	Зачем SQL	542
37.2	Грамматика мини-SQL	543
37.3	Архитектура	543
37.4	Tokenizer	543
37.4.1	Список keywords	545
37.5	Parser	545
37.5.1	Главная функция parse	546
37.5.2	parse_create	546
37.5.3	parse_select	547
37.6	Recursive descent в общем	548
37.7	AST как одна структура	548
37.8	Executor	549
37.8.1	CREATE TABLE	549
37.8.2	INSERT	550
37.8.3	SELECT	550
37.9	Демо	552
37.10	Архитектура «парсер-исполнитель»	553
37.11	Что не получится по нашей грамматике	553
37.12	Главные правила главы	554
37.13	Маленькое упражнение	554
37.14	Что дальше	555
38	Глава 37. Индексы и план запроса	556
38.1	Primary vs secondary index	556
38.1.1	Цена индексов	557
38.2	Структура secondary index в коде	557
38.2.1	Обновление индексов на INSERT	558
38.3	Уникальность ключей в индексе	559

38.4	Query planner	559
38.5	Стратегии execute	560
38.5.1	run_primary_eq	561
38.5.2	run_secondary	561
38.5.3	EXPLAIN	562
38.6	Демо	562
38.7	Сложные query planners	563
38.7.1	Cost-based optimization	563
38.7.2	Joins	564
38.7.3	Composite indexes	564
38.7.4	Index-only scan	564
38.8	Главные правила главы	564
38.9	Маленькое упражнение	565
38.10	Что дальше	565
39	Глава 38. REPL-клиент	566
39.1	Что такое REPL	566
39.2	Multi-line ввод	567
39.2.1	Ограничения	568
39.3	Meta-команды	568
39.4	История	569
39.5	Comments	570
39.6	Запуск	571
39.7	Параметры командной строки	572
39.8	Стрелки вверх/вниз	572
39.9	Раскраска	573
39.10	Архитектура	573
39.11	Главные правила главы	574
39.12	Маленькое упражнение	574
39.13	Что дальше	575
40	Глава 39. Бенчмарки и финал Части IV	576
40.1	Бенчмарк	576
40.1.1	Insert: 2.1k ops/s	577
40.1.2	Find by id: 5.6k ops/s	577
40.1.3	Find by value (без индекса): 64 ops/s	577
40.1.4	Find by value (с индексом): 2.8k ops/s	577
40.1.5	Что замедляет нашу мини-СУБД	577
40.1.6	Сравнение с release-сборкой	578
40.2	Что мы построили — обзор	578
40.3	Что НЕ реализовали	579
40.3.1	Delete и Update	579
40.3.2	Транзакции	579
40.3.3	JOIN	579
40.3.4	Aggregates	579
40.3.5	Concurrent access	580
40.3.6	Schema persistence	580
40.3.7	Replication	580

40.3.8	Triggers / Stored procedures	580
40.3.9	Foreign keys, constraints	580
40.3.10	Distributed	580
40.4	Сравнение идейно: наш ☒ SQLite ☒ PostgreSQL	581
40.5	Чему мы научились	581
40.6	Главные правила всей Части IV	582
40.7	Маленькое упражнение	582
40.8	Часть IV закрыта	582
40.9	Что дальше	583
41	Глава 40. BSD-сокеты с нуля	584
41.1	Что такое socket	584
41.2	TCP vs UDP	584
41.3	API	585
41.4	Жизненный цикл сервера	585
41.5	Жизненный цикл клиента	586
41.6	socket()	586
41.7	sockaddr_in для IPv4	586
41.7.1	sockaddr и его потомки	587
41.8	bind()	587
41.9	listen()	588
41.10	accept()	588
41.11	recv() / send()	588
41.12	close()	589
41.13	getaddrinfo для клиента	589
41.14	inet_ntop для печати	590
41.15	Echo-server — полная сборка	590
41.16	Client	592
41.17	Запуск	593
41.18	Что наш сервер НЕ умеет	593
41.19	SO_LINGER и другие настройки	594
41.20	TIME_WAIT и почему важен SO_REUSEADDR	594
41.21	Главные правила главы	594
41.22	Маленькое упражнение	595
41.23	Что дальше	595
42	Глава 41. Многопоточность — std::thread, mutex, condition_variable	596
42.1	Что такое поток	596
42.2	std::thread	597
42.2.1	Что захватывает std::thread	597
42.3	Race condition	598
42.4	std::mutex	598
42.4.1	lock_guard — RAII	599
42.4.2	unique_lock	599
42.5	Atomicity vs locking	599
42.6	condition_variable	600
42.7	Deadlock	601
42.7.1	Профилактика	601

42.8	Многопоточный chat-server	602
42.8.1	broadcast	602
42.8.2	handle_client	603
42.8.3	main	604
42.9	Запуск	604
42.10	Архитектура «thread-per-connection»	605
42.11	Главные правила главы	605
42.12	Маленькое упражнение	606
42.13	Что дальше	606
43	Глава 42. std::atomic и memory ordering	607
43.1	Зачем atomic — наглядно	607
43.2	std::atomic	608
43.3	compare_exchange (CAS)	608
43.4	Memory ordering — главная страшилка	609
43.5	Пять режимов memory_order	610
43.5.1	memory_order_relaxed	610
43.5.2	memory_order_release / memory_order_acquire	610
43.5.3	memory_order_acq_rel	610
43.5.4	memory_order_seq_cst — sequentially consistent	611
43.6	Сравнение	611
43.7	Пример с release/acquire	611
43.8	Lock-free очередь SPSC	612
43.9	ABA-проблема	613
43.10	std::memory_order_consume	614
43.11	Когда использовать atomic, когда mutex	614
43.12	Lock-free vs wait-free	614
43.13	Главные правила главы	615
43.14	Маленькое упражнение	615
43.15	Что дальше	615
44	Глава 43. Реактор — select/poll/epoll/kqueue	617
44.1	C10K problem	617
44.2	Blocking vs non-blocking	618
44.3	select() — оригинал	618
44.4	poll() — улучшенный select	619
44.5	epoll (Linux) и kqueue (BSD/macOS)	619
44.6	Edge-triggered vs level-triggered	620
44.7	Реактор-цикл	620
44.7.1	Per-client send buffer	621
44.8	set_nonblocking	622
44.9	Полная реализация — reactor_chat_server.cpp	622
44.10	Тестирование	625
44.11	Сравнение моделей	625
44.12	Cross-platform wrapper	626
44.13	Async/await и coroutines	626
44.14	Главные правила главы	627
44.15	Маленькое упражнение	627

44.16	Что дальше	628
45	Глава 44. Свой бинарный протокол	629
45.1	Зачем фрейминг	629
45.2	Формат фрейма	630
45.3	Encoder	630
45.4	Decoder и аккумулятор	631
45.4.1	Шаблон использования	632
45.5	Альтернатива — кольцевой буфер	632
45.6	Версионирование	633
45.7	Демо	633
45.8	Тонкости в production	634
45.8.1	Frame compression	634
45.8.2	TLS / шифрование на уровне фрейма	634
45.8.3	Heartbeat / keepalive	634
45.8.4	Big-endian для сетевых протоколов	634
45.8.5	Schema evolution: protobuf, FlatBuffers, MessagePack	635
45.9	Демо использования в чате	635
45.10	Главные правила главы	636
45.11	Маленькое упражнение	636
45.12	Что дальше	637
46	Глава 45. Многокомнатный чат	638
46.1	Что добавляется	638
46.2	Структуры данных	638
46.3	Команды-обработчики	639
46.4	leave_room — сложнее, чем кажется	640
46.5	broadcast_room	641
46.6	Mutex vs shared_mutex	641
46.7	Multi-room vs one-room-per-client	642
46.8	Слабые места реализации	642
46.8.1	Race на disconnect и broadcast	642
46.8.2	Очередь send_buf переполняется	643
46.8.3	Per-room mutex (если бы было multi-threading)	643
46.9	Полная реализация	643
46.10	Что в production-чатах	644
46.10.1	Persistence	644
46.10.2	Federation / distributed	644
46.10.3	Identity	644
46.10.4	Voice/video	644
46.10.5	End-to-end encryption	644
46.10.6	Anti-spam, anti-abuse	644
46.10.7	Push notifications	644
46.10.8	Search	645
46.11	Главные правила главы	645
46.12	Маленькое упражнение	645
46.13	Что дальше	646

47	Глава 46. Шифрование XOR pre-key + история на диск	647
47.1	XOR pre-key — учебный пример	647
47.1.1	Почему XOR pre-key сломан	648
47.2	Что делает настоящая криптография	649
47.3	TLS — стандарт для сетевого шифрования	649
47.3.1	Diffie-Hellman вкратце	649
47.3.2	TLS в коде	650
47.4	История на диск	650
47.4.1	Формат	650
47.4.2	History класс	651
47.4.3	Реализация append	651
47.4.4	Реализация recent	652
47.5	Демо	653
47.6	Интеграция в chat-server	653
47.7	End-to-end encryption — обзор	654
47.8	Главные правила главы	654
47.9	Маленькое упражнение	655
47.10	Что дальше	655
48	Глава 47. Сборка и деплой	656
48.1	Static vs dynamic — снова	656
48.2	g++ -static — что не так с glibc	656
48.3	musl — альтернатива	657
48.4	Docker для C++	657
48.4.1	Многоступенчатый build	657
48.4.2	Сборка и запуск	658
48.4.3	Базовые образы для C++	659
48.5	Cross-compilation	659
48.6	Что нужно для production-депоя	659
48.6.1	Logging	659
48.6.2	Метрики	660
48.6.3	Конфигурация	660
48.6.4	Health checks	660
48.6.5	Graceful shutdown	660
48.6.6	Authentication / authorization	660
48.6.7	Database вместо файлов	660
48.6.8	Horizontal scaling	661
48.7	Часть V — итог	661
48.8	Что не реализовано	661
48.9	Что выучили концептуально	661
48.10	Часть V закрыта	662
48.11	Главные правила всей Части V	662
48.12	Маленькое упражнение	662
48.13	Что дальше	663
49	Глава 48. C++17 — optional, variant, structured bindings, if constexpr	664
49.1	std::optional	664
49.1.1	Когда optional хорош	665

49.1.2	Когда optional плох	665
49.2	std::variant	666
49.2.1	variant vs наследование	666
49.3	std::any	667
49.4	Structured bindings	667
49.4.1	С map	668
49.4.2	С insert / emplace	668
49.4.3	С struct	669
49.4.4	Ограничения	669
49.5	if constexpr	669
49.5.1	typeid и type traits	670
49.6	Inline variables	671
49.7	std::filesystem (упоминание)	671
49.8	Когда переходить на C++17	671
49.9	Где переписать наш код	671
49.9.1	parser save (RPG, глава 24)	671
49.9.2	B+tree (DB, глава 34)	672
49.9.3	parser SQL (DB, глава 36)	672
49.10	Полное демо	673
49.11	Главные правила главы	673
49.12	Маленькое упражнение	673
49.13	Что дальше	674
50	Глава 49. std::filesystem	675
50.1	std::filesystem::path	675
50.1.1	Разбор пути	676
50.2	Файловые операции	676
50.2.1	Текущая папка	677
50.2.2	Symlinks и absolute paths	677
50.3	Итерация по каталогу	677
50.4	Демо	677
50.5	Где в наших проектах	679
50.5.1	demo-shell	679
50.5.2	demo-db	680
50.5.3	demo-chat	680
50.5.4	Обход проектной директории	680
50.6	Ошибки	680
50.7	Производительность	681
50.8	Windows nuances	681
50.9	Fallback под C++11	681
50.10	Главные правила главы	681
50.11	Маленькое упражнение	682
50.12	Что дальше	682
51	Глава 50. std::string_view и параллельные алгоритмы	683
51.1	std::string_view — строка, которой не владеют	683
51.1.1	Что умеет string_view	684
51.1.2	Главная ловушка: view не продлевает жизнь	685

51.1.3	Параметр функции: <code>string_view</code> вместо <code>const std::string&</code>	685
51.2	Демо: токенизатор без копий	686
51.2.1	Где в наших проектах	687
51.3	Параллельные алгоритмы	687
51.3.1	Когда это выгодно — и когда нет	688
51.3.2	Честная оговорка про поддержку	689
51.4	Главные правила главы	689
51.5	Маленькое упражнение	689
51.6	Что дальше	690
52	Глава 51. Куда дальше — обзор C++20/23 и маршрут роста	691
52.1	Концепты — конец SFINAE-ужасов	691
52.2	Ranges — конвейеры вместо «итератор, итератор»	692
52.3	Корутины — функции, умеющие приостанавливаться	693
52.4	Модули — прощание с <code>#include</code>	694
52.5	Несколько штрихов C++23	694
52.6	Куда расти после этой книги	695

Глава 1

Вступление. Зачем эта книга

1.1 О чём она

C++ — язык, на котором написана половина того, чем мы каждый день пользуемся: ядра баз данных (PostgreSQL частично, ClickHouse, RocksDB, FoundationDB), браузеры (Chrome, Firefox, Safari), графические движки (Unreal, Unity-native), пакеты типа Photoshop и Blender, торговые системы биржи, прошивки самолётов и автомобилей. На нём же написаны компиляторы JavaScript (V8) и Python (CPython). И при всей этой важности — это, наверное, самый непонятный язык из массово используемых. Я сам, когда первый раз открыл «C++ Primer» Липпмана, на двадцатой странице утонул в `cin >> x;` и захлопнул книгу: половина символов мне ничего не говорила, а автор как будто считал само собой разумеющимся, что я знаю, чем `int*` отличается от `int&`.

Эта книга писалась именно так, как мне самому когда-то нужна была. Я не делаю вид, что читатель — выпускник МГУ с кафедры вычмата. Книга рассчитана и на тех, кто уже знает другой язык (Python, PHP, JS), и на тех, у кого C++ — вообще первый язык программирования. Для вторых первые пять глав (Часть 0a) — это разгон с нуля: что такое компьютер, что такое программа, как устроены биты и байты, что такое алгоритм, как пользоваться терминалом. Без этого даже самый простой C++-код кажется магией. Если у вас уже есть опыт другого языка — пробежите эти главы по диагонали, основное всё равно полезно.

А дальше — за руку, со всеми объяснениями. Если в коде стоит звёздочка, я расскажу, что она означает. Если стоит амперсанд, расскажу, что он означает. Если на одной строке встретятся `&&`, я расскажу, что это вообще не два амперсанда, а отдельная штука. Я не считаю, что у читателя «и так должно быть понятно».

Главное отличие от стандартных C++-учебников — мы не пишем тридцать «hello world» с разными вариациями. Мы делаем **четыре полноценных консольных проекта** по нарастанию сложности. К концу книги у вас в репозитории будут собраны и работают:

1. **Текстовая RPG-игра** в стиле Zork — с локациями, инвентарём, NPC, боями и сохранениями.
2. **Свой мини-shell** — bash-подобная оболочка с pipes, redirections, builtin-командами и историей.
3. **Мини-СУБД** — key-value база данных с B+tree индексом, журналом изменений и собственным мини-SQL.
4. **TCP-мессенджер** — сервер и клиент с многокомнатным чатом, серверная часть на event loop.

Это не игрушки на пятнадцать минут. Это работающие проекты, на которых видно, что C++ — это не «странный язык со звёздочками», а инструмент промышленного уровня. И каждое новое языковое средство я ввожу тогда, когда оно реально нужно для нашего следующего шага в проекте, а не «впрок».

Книга — это не «C++ за выходные». Её читают вечерами по часу, и через пару месяцев получают **понимание**, как устроен язык и что им делать. Каждая глава самодостаточна: вы можете отложить книгу на неделю и вернуться, не потеряв нить. Каждый кусок кода в книге собирается своей командой компиляции, и команда тоже разобрана.

1.2 Кому она

Полным новичкам, для которых C++ — это первый язык программирования. Чаще всего так: подросток заинтересовался играми, услышал что Minecraft был на Java, а его движок Bedrock — на C++, и решил «начну с серьёзного». Или взрослый человек после математической специальности захотел научиться писать программы и не нашёл, с чего начать. Для них Часть 0a — это вход с улицы: без знания терминала, без знания, что такое файл, без знания, как считать в двоичной системе. Я предполагаю только одно: вы умеете пользоваться компьютером на бытовом уровне (открывать программы, искать файлы в файловом менеджере), и у вас есть желание разобраться. Дальше — за руку.

Тем, кто закончил основы программирования на любом другом языке и хочет понять C++. Не «выучить пять команд, чтобы пройти курс в вузе», а понять — как программа на C++ устроена, во что превращается, какие у неё бывают проблемы и как их избегать. Я считаю это самым полезным навыком в индустрии: даже если вы не будете писать на C++ ежедневно, понимание того, что под капотом, делает вас лучше как разработчика на любом другом языке. Когда вы знаете, что строка в Python — это объект с шапкой и указателем на буфер на куче, и что копирование строк не бесплатно, вы пишете лучший Python.

Школьникам и студентам, которым на учебе впахнули C++ и сказали «разбирайтесь». В стандартных учебниках авторы либо начинают с обсуждения «парадигм программирования» (вода), либо сразу прыгают в STL без объяснения, что такое память. Я начинаю с того, что у вас на экране — текстовый редактор, в нём вы пишете символы, и эти символы превращаются в исполняемый файл вот таким способом. Без воды.

Программистам с других языков, которым нужно «прочитать чужой C++-код и не сойти с ума». Особенно если вы пришли из Go, JavaScript или Python — у вас часто кружится голова от move-семантики, шаблонов и `const`. Я тщательно разбираю эти моменты, потому что сам когда-то на них спотыкался.

Кому НЕ подойдёт эта книга:

- если вам нужен `embedded` или микроконтроллеры — я касаюсь `systems-programming`, но не делаю фокус на работе с регистрами и прерываниями;
- если вы хотите быстро «выучить только то, что спросят на собеседовании» — здесь много контекста и историй вокруг, что замедляет «зазубривание»;
- если вам нужна графика (OpenGL, Vulkan, DirectX) — в этой книге только консольные проекты, графика — отдельная огромная тема.

1.3 Почему C++ в 2026 году

«Зачем учить C++, если есть Rust / Go / современные языки?» — вопрос, который вам обязательно зададут. У меня на него четыре ответа.

Первый — **зрелость экосистемы**. C++ существует с 1985 года. За сорок лет на нём написано столько кода, что выкинуть его и переписать на Rust невозможно физически. Это значит: вакансий на поддержку и развитие этого кода десятки тысяч, и они никуда не денутся ещё лет двадцать минимум. Если вы хотите работать в системном программировании, базах, играх, финтехе — вам надо знать C++.

Второй — **производительность**. Когда вы пишете на C++ грамотно, программа выполняется со скоростью железа. Не «как Java с `jit`-ом», не «как Go с GC-паузами», а реально так быстро, насколько способен процессор. Это критично там, где счёт идёт на наносекунды: трейдинг, рендеринг, аудио-обработка, низкоуровневая графика.

Третий — **контроль**. В C++ нет ничего, что бы выполнялось «само». Ни сборщика мусора, ни автоматических аллокаций, ни магических промисов. Каждое выделение памяти — это ваш код. Каждое копирование — ваш выбор. Этот контроль одновременно и сила, и боль: сила, потому что вы понимаете, что происходит; боль, потому что любая ошибка — ваша. И именно из-за этой боли C++ хорошо учит думать.

Четвёртый — **универсальность**. Один и тот же C++-код собирается под x86-64 Linux, ARM Apple Silicon, Windows, Android NDK, iOS, Raspberry Pi, микроконтроллеры (с оговорками), даже под WebAssembly через emscripten. Ни один другой язык такой переносимости не даёт.

Стоит ли C++ называть «устаревшим»? Стандарт C++23 принят в 2024 году, C++26 на подходе. Язык активно развивается: ranges, concepts, modules, coroutines — это всё современные средства, которые на голову выше того, что было даже в C++11. Просто C++ обязан тащить совместимость с кодом сорокалетней давности — отсюда репутация «динозавра».

1.4 Что мы построим — детальнее

Раз уж проекты сквозные, расскажу подробнее, что вы получите.

1.4.1 Проект 1: текстовая RPG-игра `demo-rpg/`

Игра в духе Zork (Infocom, 1980) и «Колобка» (если вы помните 90-е на советских ПК). Никакой графики — только текст в терминале:

```
> look
Лесная поляна. На западе виднеется тропа, ведущая к деревне.
На востоке — тёмный лес. На земле лежит ржавый меч.
> take меч
Вы подняли ржавый меч.
> go east
Тёмный лес. Ветви скрипят над головой.
Здесь стоит гоблин с дубиной.
> attack гоблин
Вы ударили гоблина мечом. Урон: 8. У гоблина осталось 4 HP.
Гоблин ударил вас дубиной. Урон: 3. У вас осталось 17 HP.
> attack гоблин
Гоблин повержен. Получено 15 опыта.
Из тела выпало: золото (3 монеты), кусок мяса.
> save my_game
Игра сохранена в data/saves/my_game.sav
```

В коде там будет всё, что нужно для понимания ООП в C++: иерархия классов (Entity → Player → Enemy → NPC), отдельная иерархия предметов (Item → Weapon/Armor/Consumable), полиморфные эффекты, шаблонная шина событий, лямбды для AI поведений, умные указатели для управления памятью, парсер сценария игры из текстовых файлов, сохранения

в бинарном и текстовом формате. Игру можно расширять — добавлять новые локации просто редактируя `data/world.txt`, без перекомпиляции.

1.4.2 Проект 2: мини-shell `demo-shell/`

Bash в миниатюре. Поддерживает:

```
$ ls | grep .cpp | wc -l
12
$ echo "hello" > out.txt
$ cat out.txt
hello
$ sleep 5 &
[1] 4823
$ jobs
[1] Running   sleep 5
$ cd ..
```

Внутри: парсер командной строки, который умеет в кавычки и escape-последовательности; пайплайны через `pipe()` и `dup2()`; редиректы `>`, `>>`, `<`, `2>&1`; фоновые задачи через `&`; список `jobs`; история команд в `~/.cpp_shell_history`; навигация стрелками. Плюс три «карманные» утилиты, написанные с нуля: `cat`, `wc`, `grep` — чтобы было что запускать в `pipe`-ах.

Этот проект учит работе с операционной системой через POSIX API: процессы (`fork/exec/wait`), файловые дескрипторы, сигналы, `terminal raw mode`. С Windows тут чуть сложнее — нативные API другие; мы обсудим в самой главе.

1.4.3 Проект 3: мини-СУБД `demo-db/`

Своя key-value база данных с настоящим, не игрушечным движком. По функциям:

```
db> CREATE TABLE users (id INT PRIMARY KEY, name TEXT, age INT);
OK (0.001s)
db> INSERT INTO users VALUES (1, 'Alice', 30);
db> INSERT INTO users VALUES (2, 'Bob', 25);
db> SELECT * FROM users WHERE age > 20;
id | name  | age
---|-----|----
1  | Alice | 30
2  | Bob   | 25
(2 rows, 0.0003s)
db> CREATE INDEX idx_age ON users(age);
```

```
db> EXPLAIN SELECT * FROM users WHERE age = 30;
```

```
Index scan: idx_age
```

```
db> .save
```

```
Database flushed to disk.
```

Внутри: страничный файл (страница 4 KiB, как у настоящих БД), LRU page cache, B+tree индекс с разбиением и слиянием узлов, write-ahead log для атомарности транзакций, парсер мини-SQL (CREATE/INSERT/SELECT/UPDATE/DELETE с WHERE и индексами), REPL с историей команд. Это **реальный** учебный движок БД, не «hello world».

Зачем нужно? Чтобы вы поняли, как устроены настоящие СУБД изнутри. После этого проекта вы по-другому будете писать SQL: вы будете чувствовать, какие запросы быстрые, а какие медленные, и почему.

1.4.4 Проект 4: TCP-мессенджер demo-chat/

Сетевой чат. Сервер слушает порт, клиенты подключаются и общаются в комнатах:

```
$ ./chat-server --port 9001
```

```
[server] listening on :9001
```

```
[server] client connected: 127.0.0.1:54021 (id=1)
```

```
[server] client connected: 127.0.0.1:54033 (id=2)
```

```
[server] [room=general] <user_1> hello!
```

```
[server] [room=general] <user_2> hi there
```

```
# В другом терминале:
```

```
$ ./chat-client --server 127.0.0.1:9001 --nick alice
```

```
* Connected. Type /help for commands.
```

```
/join general
```

```
* Joined room "general"
```

```
hello!
```

```
<user_2> hi there
```

```
/leave
```

```
* Left room "general"
```

```
/quit
```

Внутри: сокеты, многопоточность, atomic-операции, реактор на select/poll/epoll/kqueue, свой бинарный протокол с фреймированием, шифрование XOR pre-key (учебное), история чата на диске. Эта часть — для тех, кто хочет понять, как устроены онлайн-игры, мессенджеры, прокси-серверы и вообще любой network-софт.

1.5 Что НЕ войдёт (и почему)

Книга и так длинная. Поэтому некоторые темы я сознательно вынес «за скобки»:

- **GUI** (Qt, wxWidgets, Win32, GTK) — отдельная огромная тема, требует отдельной книги. Мы строго в консоли.
- **Compiler internals** — мы будем разбирать, как программа компилируется, но без углубления в внутренности LLVM/GCC.
- **Шаблонная метапрограммация на полную глубину** — введение в шаблоны будет; SFINAE, type traits на уровне «что это и где работает», но без <type_traits> от и до. Это материал для второго курса, не первого.
- **Coroutines/concepts/ranges из C++20** — обзорно в последней главе, без глубокого погружения. Книга и так на C++11 как базе.
- **Cross-platform GUI / iOS / Android** — за рамками. В сетевом проекте мы делаем POSIX-ориентированный код с короткими отступлениями про WinSock.
- **Real-time audio/video** — не моя область, не буду делать вид, что эксперт.

1.6 Какие у меня соглашения

Несколько технических моментов, чтобы потом не возвращаться.

Шрифт кода. Весь код будет в моноширинном блоке:

```
int main() {  
    return 0;  
}
```

Если что-то надо подсветить в строке текста — оно тоже в моноширинном виде, как `int` или `std::vector`. Имена файлов — `main.cpp`. Команды терминала отделены долларом:

```
$ g++ -std=c++11 main.cpp -o hello  
$ ./hello
```

Платформы. Я писал и тестировал всё на macOS (Apple Silicon, clang) и Ubuntu Linux 22.04 (gcc). Если будут особенности Windows — отмечу отдельно. Если для какой-то главы Windows нативно не подходит — честно скажу.

Стандарт. Везде, где не указано иное, код собирается командой:

```
$ g++ -std=c++11 -Wall -Wextra -pedantic file.cpp -o file
```

Если строка кода требует C++14 или C++17, я об этом скажу в тексте и в комментарии к коду.

Версии компиляторов. `clang` ≥ 11 , `gcc` ≥ 7 . На более старых может что-то не собраться, но это редкость. На современных Apple Silicon — Apple clang 14+ из коробки.

Структура папок. Все четыре проекта живут в корне репозитория книги: `demo-rpg/`, `demo-shell/`, `demo-db/`, `demo-chat/`. Само текстовое содержимое книги — в `docs/tutorial/00-intro.md`, `01-toolchain.md` и так далее. Сборка PDF из markdown — `build/build-pdf.sh`.

1.7 Как читать

По порядку. Я не делал «книгу-справочник, где можно прыгать по главам». В главе 12 я считаю, что вы уже понимаете, что такое класс и конструктор (глава 11), и не объясняю заново. Если что-то непонятно — лучше вернуться назад, чем читать дальше.

Не пропускайте код. В книге сотни примеров, и я часто говорю «соберите этот файл, запустите, посмотрите вывод». Это не оборот речи — если не запустить, теряется половина смысла. Программирование учится только пальцами.

Не торопитесь. Глава за вечер — идеальный темп. Если за час прочитали три главы — скорее всего, не задержались на коде. Вернитесь.

Делайте свои изменения в коде проектов. Я предлагаю упражнения в конце глав, но самое главное — играть с кодом. Сломайте что-нибудь и посмотрите, что вышло. Это лучший способ понять язык.

1.8 Что дальше

Если вы вообще не знакомы с программированием, дальше идёт **Часть 0a** — пять подготовительных глав. С первой по пятую: устройство компьютера, биты и байты, что такое программа, как думают алгоритмически, как пользоваться терминалом. Без этого фундамента глава про C++-компилятор покажется магией. Если опыт другого языка у вас есть — пробежите по диагонали, но не пропускайте полностью: контекст «зачем C++ устроен именно так» строится именно в этой части.

Глава 6 — про инструменты. Поставим компилятор, разберём первый «hello world» и посмотрим, что происходит между моментом, когда вы нажимаете Enter в терминале, и моментом, когда программа выводит «Hello, world». На это уйдёт целая глава, потому что событий там — десятки, и каждое стоит понять.

Поехали.

Глава 2

Глава 1. Что такое компьютер

Я начну с самого начала — буквально с того, как устроена железка, на которой вы сейчас читаете эту книгу. Если вы уже хорошо понимаете, что такое процессор, оперативная память и операционная система — можете пробежать главу по диагонали. Но даже опытные разработчики иногда путаются в простых вещах вроде «чем отличается ОЗУ от диска», и это потом мешает понимать C++. Так что не лишним будет освежить.

2.1 Зачем это в книге про C++

Большинство современных языков (Python, JavaScript, Java) скрывают железо от программиста. Вы пишете `x = 5`, и думать о том, где именно в физической памяти лежит эта пятёрка, — необязательно. Сам язык за вас об этом заботится: запускает «сборщик мусора», убирает ненужные значения, выделяет память.

C++ — другой. В C++ вы постоянно сталкиваетесь с понятиями вроде «адрес в памяти», «стек», «куча», «размер типа», «cache miss». Без понимания, что это значит на железе, код покажется набором заклинаний. Не «эта программа работает», а «эта программа работает, потому что компилятор разместил эти три переменные подряд в кэш-линии». Если разница пока непонятна — она прояснится по ходу книги. А чтобы было от чего отталкиваться, начнём с устройства железки.

2.2 Что такое компьютер на самом деле

Компьютер — это машина, которая умеет делать одну простую вещь: брать числа, выполнять над ними арифметику или логику, и записывать результат обратно. Всё остальное — текст, картинки, видео, музыка, игры — это надстройка над этой простой механикой. Текст — это последовательность чисел, каждое из которых обозначает букву. картинка — это сетка чисел,

каждое из которых обозначает цвет точки. Видео — это последовательность таких сеток. И так далее.

Внутри компьютера живут пять ключевых компонентов. Расскажу про каждый.

2.2.1 Процессор (CPU)

Центральный процессор — это собственно «мозг». Кусок кремния размером с ноготь, на котором миллиарды (буквально миллиарды) транзисторов. У современного процессора Apple M3 — около 25 миллиардов транзисторов; у Intel Core i9 — около 12-15 миллиардов; у простенького ARM-процессора для микроконтроллера может быть и несколько миллионов.

Что процессор умеет? Очень мало. По сути — выполнять «инструкции». Каждая инструкция — это маленькое действие: «сложи два числа», «сравни два числа и запомни результат сравнения», «прочитай число из памяти», «запиши число в память», «прыгни в другое место программы». Всё. Никаких «нарисуй картинку» или «открой файл» процессор сам не умеет — это уже надстройки.

Список инструкций, которые понимает процессор, называется его **архитектурой** (или «инструкционным набором»). Самые известные сейчас две: **x86-64** (она же AMD64) — используется в большинстве настольных компьютеров и серверов; и **ARM64** (она же AArch64) — используется в телефонах, планшетах, новых Mac на Apple Silicon. Между ними есть фундаментальные различия (например, x86-64 умеет складывать сразу пять чисел одной инструкцией, ARM64 — за две). Но для нас важно одно: программа, скомпилированная под x86-64, не запустится на ARM64 и наоборот. Нужна перекомпиляция.

Процессор не выполняет инструкции одну за одной с одной скоростью. Современный процессор работает на частоте 3-5 ГГц — это значит «3-5 миллиардов тактов в секунду». На каждом такте процессор может начать новую инструкцию (или несколько — современные процессоры умеют параллелизм внутри одного ядра, об этом потом). Получается, простая инструкция типа сложения выполняется за доли наносекунды. Но обращение в память — куда медленнее, чем сложение. Это ключевой факт, к которому мы будем часто возвращаться.

У процессора есть ещё одна важная штука — **регистры**. Это маленькие «карманы» прямо внутри процессора, в которых лежат числа, с которыми он сейчас работает. На x86-64 регистров общего назначения 16 штук, по 8 байт каждый. На ARM64 — 31 штука. Когда инструкция говорит «сложи два числа», она имеет в виду «сложи числа в этих двух регистрах». Если число лежит не в регистре, а в памяти, его сначала надо в регистр прочитать.

Регистры — это самая быстрая память в системе. Доступ к регистру занимает один такт. К сравнению, доступ к оперативной памяти — сотни тактов. К диску — миллионы. Запомните этот порядок, он часто всплывёт.

2.2.2 Оперативная память (RAM)

RAM (Random Access Memory) — это рабочая память. Большой массив байтов, к каждому из которых можно обратиться по адресу. У современных компьютеров — 8, 16, 32, 64 ГБ. ГБ — гигабайт, миллиард байт (точнее, 2^{30} , но это нюансы).

Слово «random access» — «произвольный доступ» — означает: к любому байту можно обратиться напрямую по адресу, без чтения соседних. Это отличает RAM от ленты — ленту приходится перематывать.

Когда вы запускаете программу — операционная система берёт её код с диска и кладёт в RAM, потому что процессор может читать инструкции и данные только из RAM (не с диска напрямую). Когда программа создаёт переменную — переменная появляется в RAM. Когда программа складывает два числа — она берёт их из RAM, кладёт в регистры процессора, складывает, и результат пишет обратно в RAM.

Память называется **оперативной** или **энергозависимой**, потому что при выключении компьютера её содержимое исчезает. Всё, что вы хотите сохранить — должно быть записано на диск.

Адрес в памяти — это число. На современных 64-битных системах адрес — 64-битное число (8 байт), то есть теоретически можно адресовать $2^{64} \approx 18$ квинтиллионов байт. На практике используется только часть (обычно 48 битов = 256 ТБ). Но факт: каждый байт в памяти имеет свой адрес, и этот адрес — обычное число. Когда мы дойдём до указателей в C++, окажется, что **указатель — это просто число**, которое значит «адрес в памяти». Это сильно поможет потом не путаться.

2.2.3 Кэш процессора

Я сказал выше: обращение в RAM — сотни тактов. Это очень долго по меркам процессора. Если бы каждое чтение действительно занимало 200 тактов — программа летала бы со скоростью улитки.

Поэтому между процессором и RAM встроили **кэш** — маленькая, но очень быстрая память прямо на чипе процессора. Когда процессор обращается к памяти по адресу X, он на самом деле сначала смотрит в кэш — есть ли там копия данных по этому адресу? Если есть — берёт оттуда (1-3 такта). Если нет — обращается в RAM (~200 тактов) и заодно копирует данные в кэш, чтобы следующее обращение было быстрым.

Кэшей обычно несколько уровней: - **L1** — самый быстрый и маленький (32-128 КБ), отдельный для каждого ядра. - **L2** — побольше и помедленнее (256 КБ — 2 МБ), тоже на ядро. - **L3** — общий для всех ядер (4-64 МБ).

Кэш работает «страницами»: когда процессор читает один байт, в кэш загружается целая «кэш-линия» — обычно 64 байта. Это важно! Если ваши данные лежат в памяти подряд — следующее чтение, скорее всего, попадёт в кэш. Если разбросаны по памяти — каждое чтение требует похода в RAM. Поэтому в C++ часто говорят «вектор быстрее списка», «структура данных, дружелюбная к кэшу, важнее теоретической сложности алгоритма».

В этой книге я не буду углубляться в оптимизацию под кэш — это материал отдельной книги. Но знать, что кэш существует, важно, потому что многие решения языка C++ (например, `std::vector` vs `std::list`) объясняются именно через кэш.

2.2.4 Диск (SSD/HDD)

Диск — это место, где данные хранятся при выключенном компьютере. Современные ноутбуки используют SSD (Solid State Drive) — флеш-память, без движущихся частей. Старые компьютеры — HDD (Hard Disk Drive), магнитные диски с механическими головками.

Диск медленный по сравнению с RAM. SSD — в 100-1000 раз медленнее RAM. HDD — в 10000-100000 раз. Поэтому файл с диска сначала читается в RAM, и дальше программа работает с этой копией.

Диск важен для нас в C++ потому, что сохранения игры, файлы базы данных, исходники программ — всё это на диске. В Части III (mini-shell) и Части IV (мини-СУБД) мы будем плотно работать с файлами.

2.2.5 Операционная система

Между вашей программой и железом стоит **операционная система** (ОС). У вас, скорее всего, либо macOS (на Mac), либо Windows, либо Linux (если вы продвинутый пользователь). На телефоне — iOS или Android.

Операционная система выполняет несколько ключевых задач:

1. **Управление процессами.** Процесс — это запущенная программа. На вашем компьютере прямо сейчас работают сотни процессов параллельно: браузер, мессенджер, фоновый монитор обновлений, и так далее. ОС переключает процессор между ними сотни раз в секунду, создавая иллюзию параллельности. Каждому процессу ОС выделяет своё «адресное пространство» — то есть свою куски памяти, отдельные от других. Если бы процессы делили память без изоляции — баг в одном уронил бы все.
2. **Управление памятью.** Когда программа просит «дай мне 100 байт», она обращается на самом деле к ОС. ОС находит свободный кусок памяти и отдаёт. Когда программа закрывается, ОС автоматически возвращает себе всю её память.

3. **Доступ к железу.** Программа не общается с диском напрямую — она просит ОС «прочитай мне такой-то файл». ОС сама общается с диском и возвращает данные. То же с сетью, принтером, экраном.
4. **Файловая система.** Диск — это просто массив байтов. Чтобы хранить там осмысленно «файлы» и «папки», ОС поверх диска организует файловую систему: APFS на Mac, NTFS на Windows, ext4 на Linux. Программа работает с «файлом по пути /Users/me/photo.jpg», а под капотом ОС находит, на каких именно секторах диска лежат данные.
5. **API для программ.** ОС предоставляет программам стандартные функции: открыть файл, прочитать байты, нарисовать окно, услышать клавишу. Это называется **системные вызовы**. На macOS/Linux большинство системных вызовов соответствуют стандарту POSIX. На Windows — другой стандарт (Win32 API). Это причина того, что Windows и macOS-программы не совместимы напрямую.

В C++ системные вызовы делаются через стандартную библиотеку (которая под капотом обращается к ОС). Например, когда вы пишете `std::cout << "hello"`, под капотом происходит системный вызов `write()` (на Unix) или `WriteFile()` (на Windows). Когда вы открываете файл — `open()` или `CreateFile()`. И так далее.

В Части III книги мы будем работать с системными вызовами напрямую, минуя стандартную библиотеку — чтобы написать свой shell. Это даст вам понимание, что под капотом у каждой программы — десятки таких вызовов.

2.3 Что такое процесс

Раз уж я упомянул процессы — расскажу подробнее. Процесс — самое важное понятие, без которого C++ не понять.

Когда вы дважды щёлкаете по программе (или запускаете из терминала) — ОС создаёт новый процесс. Это значит:

1. **Выделяется адресное пространство.** ОС резервирует кусок виртуальной памяти, в котором будет жить процесс. На 64-битной системе размер виртуального пространства — теоретически 256 ТБ, на практике это просто «много». Программа видит непрерывную область адресов от 0 до огромного числа, и думает, что вся память — её. Под капотом ОС динамически отображает виртуальные адреса на физическую RAM по мере надобности.
2. **В адресное пространство загружается код программы.** Бинарный файл с диска копируется в память. Точнее — отображается, через механизм `memory-mapping`; реально данные подгружаются по мере чтения.

3. **Создаётся стек.** Стек — это особая область памяти, где живут локальные переменные функций. О стеке мы будем говорить много, особенно в главе 11.

4. **Запускается `main`.** ОС находит в коде программы функцию `main` и передаёт ей управление. Дальше — работа уже программы.

Процесс изолирован от других процессов. Память процесса А не видна процессу Б. Если процесс А упал — он умирает, а процесс Б продолжает работать. Это огромное достижение операционных систем 70-80-х годов; до этого падение одной программы могло уронить всю систему.

У процесса есть набор «ресурсов» которые ОС ему выделила: - адресное пространство (память); - открытые файлы (через «файловые дескрипторы» — числа-номера); - сетевые соединения (тоже через файловые дескрипторы — в Unix всё файл); - сигналы, которые ему можно посылать (SIGTERM, SIGKILL, SIGINT).

Когда процесс закрывается — ОС автоматически освобождает все его ресурсы. Открытые файлы закрываются, память возвращается в общий пул, сетевые соединения разрываются. Этим часто пользуются: «утечка памяти — не страшно, всё равно через минуту процесс закроется и ОС всё уберёт». Это правда, но привычка такая плохая — на длинных процессах (серверы, демоны) утечки становятся проблемой.

В Части III книги мы будем создавать процессы из C++-кода (системный вызов `fork`), что бывает полезно для shell, серверов и параллельной обработки данных.

2.4 Что такое поток

Один процесс может состоять из нескольких **потоков** (threads). Поток — это «исполняющаяся последовательность инструкций». В простом случае процесс состоит из одного потока (главного), и он по очереди выполняет инструкции в программе. Когда нужен параллелизм — программа создаёт дополнительные потоки.

Чем поток отличается от процесса:

- Потоки одного процесса **делят память**. То есть переменная, созданная в одном потоке, видна другому потоку. Это и быстро (не надо ничего передавать), и опасно (легко получить гонки данных).
- Потоки одного процесса делят также открытые файлы, сетевые соединения, и так далее.
- Каждый поток имеет свой стек (отдельные локальные переменные).
- ОС переключает процессор между потоками точно так же, как между процессами.

Когда ноутбук пишет «8 ядер», это значит — 8 потоков могут реально выполняться одновременно, физически параллельно. На 8-ядерном процессоре десятки тысяч потоков

«крутятся», но реально активных в каждый момент времени — только 8 (плюс ещё «гипертрединг» удваивает это число — но грубо говоря, $8 = 8$ параллельных).

Многопоточность мы будем глубоко изучать в Части V (TCP-чат), где сервер должен обслуживать множество клиентов одновременно.

2.5 Что увидите в C++

Из всего сказанного выше в C++ напрямую отражены вот эти понятия:

- **Указатель** — переменная, в которой лежит адрес в памяти. Чем мощнее, чем боль — будем долго осваивать в главе 11.
- **Размер типа** — каждый тип в C++ имеет фиксированный размер в байтах (`int` обычно 4 байта, `double` 8 байт). Размер зависит от архитектуры процессора. Об этом — в главе 8.
- **Стек и куча** — две основные области памяти процесса. Стек — для локальных переменных, куча — для динамически выделяемой памяти. Глава 11.
- **`new` и `delete`** — обращение к ОС за памятью и возврат памяти ОС. Глава 11.
- **Файлы** — глава 12.
- **Потоки** (`std::thread`) — глава 41.
- **Процессы** (`fork`, `exec`) — глава 26.

Каждое из этих понятий — обёртка над тем, что я описал в этой главе. Когда вы будете встречать `int*` или `delete x`, вспоминайте: за этим стоит совершенно конкретное действие — выделение или возврат куска физической памяти.

2.6 На пальцах: жизнь программы Hello

Чтобы свести вместе всё, что я говорил, представим, что вы скомпилировали программу `hello`, которая выводит «Hello, world», и запустили её. Что произошло:

1. Вы написали `./hello` в терминале и нажали Enter.
2. Терминал — это тоже процесс. Он попросил у ОС создать новый процесс для `hello` (системный вызов `fork`, потом `exec`).
3. ОС нашла на диске файл `hello`, прочитала его заголовок (метаданные о том, как он устроен), выделила в памяти место для кода программы и скопировала туда содержимое файла. На самом деле скопировала не сразу, а отметила в таблице страниц «когда программа запросит этот участок памяти — подгружу данные с диска» (lazy loading через memory-mapping).
4. ОС выделила для процесса начальный стек (обычно 1-8 МБ виртуально, но физически пока только малая часть).

5. ОС нашла в программе функцию `main` и передала ей управление, то есть установила в регистр программного счётчика (RIP на x86-64) адрес первой инструкции `main`.
6. Процессор начал выполнять инструкции одну за одной.
7. Дошёл до инструкции, которая отвечает за `std::cout << "Hello, world"`. На самом деле это много инструкций; конечная цель — вызвать системный вызов `write` с указателем на строку и её длиной.
8. Процесс перешёл в режим ядра, ОС нашла файловый дескриптор 1 (стандартный вывод, обычно — терминал), записала туда байты «Hello, world».
9. Терминал увидел новые байты и отрисовал их на экране.
10. Программа вернулась из `main`, выполнила `return 0`. ОС увидела, что процесс хочет закрыться, освободила его память, файлы, и пометила процесс как завершённый.
11. Терминал увидел, что дочерний процесс умер, и снова показал приглашение к вводу команды.

Между шагами 1 и 11 проходит, наверное, миллисекунды. И за каждой из них стоит то, что я описал в этой главе.

2.7 Маленькое упражнение

Никакого C++ ещё нет, но небольшая разминка. Откройте задачник: попробуйте мысленно (или на бумаге) ответить на эти вопросы.

1. Если на вашем компьютере 16 ГБ RAM, и вы запустили программу, которая просит у ОС 32 ГБ памяти — что произойдёт?
2. Почему программа на C++, скомпилированная под Mac M3, не запустится на Windows-ноутбуке с Intel — даже если функционал у программы одинаковый?
3. Что произойдёт, если в коде есть «утечка памяти» — программа выделяет память и не освобождает её — и эта программа крутится месяцами?
4. Чем процесс отличается от потока? Назовите хотя бы три отличия.

Ответы на эти вопросы я не даю — попробуйте сами, и если затрудняетесь, вернитесь к нужному разделу. Это не контрольная; это способ проверить, что в голове осталось.

2.8 Что дальше

Глава 2 — про биты и байты. Раз компьютер всё хранит в виде чисел, надо понять, как именно. Что такое двоичная система? Почему именно она? Как с её помощью кодируются отрицательные числа, дробные числа, символы алфавита, текст на русском и казахском? Без этого знания нельзя понять, почему `int` в C++ — это «32-битное целое со знаком», и что значит

«переполнение signed int — это undefined behavior». Эту главу старайтесь не пропускать, даже если кажется, что знаете.

Глава 3

Глава 2. Биты, байты, числа, текст

В прошлой главе я несколько раз сказал «всё в компьютере хранится в виде чисел». Сейчас распакуем эту фразу. Откуда «всё»? И что это за «числа», если внутри процессора — миллиарды транзисторов, а не маленьких счётчиков?

Эта глава может казаться скучной — двоичная арифметика, ASCII-таблица, всё это вы могли проходить в школе. Но я прошу не пропускать. Без чёткого понимания, что такое «32-битное целое со знаком» и почему `char` в C++ занимает один байт, многое в дальнейшем будет казаться загадкой. Лучше потратить час сейчас, чем спотыкаться весь курс.

3.1 Бит — самая маленькая единица

Внутри процессора и памяти всё — это электрические сигналы. Сигнал может быть либо «включён» (есть напряжение), либо «выключен» (нет напряжения). Промежуточных состояний нет. Это техническое ограничение: транзисторы быстро и надёжно работают в режиме «открыт/закрыт». Сделать «полуоткрыт» — теоретически можно, но шумы и помехи в реальных схемах сразу превратят такое значение в одно из двух. Поэтому: только два состояния.

Эти два состояния принято обозначать **0** и **1**. И называется одна такая «ячейка» — **бит** (от англ. `binary digit` — двоичная цифра).

Один бит — это очень мало. Им можно различить две вещи: либо «нет», либо «да». Например, «лампочка горит» / «лампочка не горит». «Дверь открыта» / «закрыта». Реальные программы оперируют куда более сложными значениями (числа от 0 до миллиарда, текст, картинки), поэтому биты группируют.

3.2 Байт — группа из 8 бит

Стандартная группа — **8 битов**. Эти 8 битов вместе называются **байтом** (англ. byte). Один байт может принимать $2^8 = 256$ различных значений.

Почему именно 8? Историческая случайность. В 60-х годах разные компьютеры использовали разные размеры: на одних было 6 битов, на других 7, на третьих 9. Но в проекте System/360 (IBM, 1964) был выбран размер 8, потому что: (а) этого хватало для кодирования одной буквы латиницы плюс контрольных символов; (б) 8 удобно делится на 4, 2, 1 (можно использовать как два полу-байта по 4 бита). System/360 стал чрезвычайно популярен, и стандарт «1 байт = 8 бит» прижился. С тех пор все архитектуры — Intel, AMD, ARM, RISC-V — используют 8-битный байт.

В C++ строго гарантировано, что `char` — это один байт. Что байт содержит как минимум 8 битов. Технически стандарт C++ говорит «байт = `CHAR_BIT` битов, и `CHAR_BIT` ≥ 8 »; на всех современных платформах `CHAR_BIT` == 8. Можно считать это законом природы.

3.3 Двоичная система счисления

Раз 8 битов = 256 значений, как именно представить число от 0 до 255 этими восемью битами?

Так же, как мы представляем число в десятичной системе, только основание — 2 вместо 10. В десятичной системе число 247 разбирается так:

$$\begin{aligned} &2 \ 4 \ 7 \\ &2 \times 100 + 4 \times 10 + 7 \times 1 \\ &2 \times 10^2 + 4 \times 10^1 + 7 \times 10^0 \end{aligned}$$

То есть каждая позиция — это степень 10. Справа единицы (10^0), потом десятки (10^1), потом сотни (10^2).

В двоичной системе ровно то же самое, только степень 2:

$$\begin{aligned} &1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \\ &1 \cdot 128 + 1 \cdot 64 + 1 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 \\ &= 128 + 64 + 32 + 16 + 4 + 2 + 1 \\ &= 247 \end{aligned}$$

То есть восемь двоичных цифр 11110111 — это то же самое, что десятичное 247. Правый бит — единицы (2^0), потом двойки (2^1), потом четвёрки (2^2), потом восьмёрки (2^3), и так далее. Это называется **позиционная запись**.

Восемь битов могут представить числа от 00000000 (= 0) до 11111111 (= 255). Это 256 значений всего.

Если вам надо больше — берёте больше битов. Два байта (16 бит) = 65536 значений. Четыре байта (32 бита) = ~4.3 миллиарда. Восемь байт (64 бита) = ~18 квинтиллионов.

В C++ есть несколько целочисленных типов, отличающихся именно размером:

Тип	Размер обычно	Диапазон (если без знака)
char	1 байт	0..255
short	2 байта	0..65535
int	4 байта	0..4 294 967 295
long long	8 байт	0..18 446 744 073 709 551 615

Это «обычные» размеры; стандарт C++ их строго не фиксирует, и мы об этом подробно поговорим в главе 8.

3.4 Шестнадцатеричная система

Двоичные числа неудобно читать человеку. 11110111 — это 8 символов, а тут ещё ни одной интересной цифры. Поэтому программисты часто используют **шестнадцатеричную систему** — основание 16.

В шестнадцатеричной системе цифр 16 штук: - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 — как обычно; - A (= 10), B (= 11), C (= 12), D (= 13), E (= 14), F (= 15).

То есть после 9 идёт A, а после F — снова 10 (но это уже «1 шестнадцать + 0 единиц», что в десятичной системе = 16).

Зачем именно 16? Потому что $16 = 2^4$. То есть **одна шестнадцатеричная цифра == четыре бита**. Очень удобно: восьмибитный байт записывается двумя хекс-цифрами. Двухбайтное число — четырьмя. Четырёхбайтное — восемью.

Преобразование байта в хекс делается так: первые 4 бита считаем как одну цифру, последние 4 — как другую.

```

11110111
↓↓↓↓ ↓↓↓↓
1111 0111
  ↓    ↓
 15    7
  ↓    ↓
  F    7

```

То есть 11110111 в двоичной системе = F7 в шестнадцатеричной = 247 в десятичной. Все три записи означают одно и то же число.

В C++ литералы можно писать в любой из этих систем: - десятичный: 247; - шестнадцатеричный: 0xF7 (префикс 0x — особый признак); - двоичный: 0b11110111 (префикс 0b, появился в C++14); - восьмеричный: 0367 (префикс 0 — но не один ноль, а с цифрой; этим почти не пользуются).

0xF7, 0b11110111 и 247 — это три разных способа записать одно и то же число. Компилятор после препроцессинга даже не знает, какая запись была использована. Хранится-то всегда одинаково.

3.5 Отрицательные числа: дополнительный код

Хорошо, до 255 в одном байте мы дошли. А как закодировать минус пять?

Можно было бы сказать «один из битов — знак: 0 = плюс, 1 = минус, остальные 7 битов — модуль числа». Это называется **прямой код**, и он действительно использовался на первых компьютерах. Но у этого подхода две проблемы:

1. Появляется два нуля: $+0 = 00000000$ и $-0 = 10000000$. Это путает и арифметику, и сравнения.
2. Сложение становится сложным: если в $+5$ и -3 бит знака разный, надо сначала разобраться, что вычитать.

Поэтому современные компьютеры используют другую систему — **дополнительный код** (two's complement). Правило простое:

- Положительные числа кодируются как обычно.
- Чтобы получить $-x$ — берётся обычный x , инвертируется (все биты меняются: 0↔1), и добавляется 1.

Пример. Возьмём 8-битный байт.

$+5 = 00000101$ — обычное двоичное представление.

Чтобы получить -5 : 1. Инвертируем все биты: 11111010. 2. Добавляем 1: 11111011.

Итог: -5 в 8-битном дополнительном коде = 11111011 = 0xFB.

Почему так работает арифметика? Возьмём $5 + (-5)$:

```

00000101  (+5)
+ 11111011  (-5)
-----
100000000

```

Получилось 9 битов: 100000000. Но мы складываем в 8-битном байте, поэтому старший бит просто отбрасывается (переполнение). Остаётся 00000000 = 0. То есть $5 + (-5) = 0$. Работает.

Главное преимущество дополнительного кода: сложение положительных и отрицательных чисел не требует никакой особой логики. Процессор просто складывает биты, и результат правильный. Поэтому железо такого процессора — простое и быстрое.

В дополнительном коде в 8-битном байте можно представить числа **от -128 до +127** включительно. Не -127 до +127, как могло бы показаться: дело в том, что -0 в этой системе совпадает с +0 (нет двух нулей), и освободившееся значение 10000000 используется для самого отрицательного числа -128.

Это объясняет, почему в C++ диапазон `signed char` — это -128..+127, а `unsigned char` — это 0..255. Один и тот же байт; разный способ интерпретации битов.

То же для остальных типов: - 16-битный `short`: -32768..+32767 (signed) или 0..65535 (unsigned). - 32-битный `int`: -2147483648..+2147483647 (signed) или 0..4294967295 (unsigned). - 64-битный `long long`: огромные числа в обе стороны.

«Если у меня переменная типа `int` и я в неё кладу значение больше 2147483647 — что произойдёт?» Хороший вопрос. В C++ для unsigned-типов есть строго определённое правило: происходит **арифметика по модулю**, то есть значение «обернётся» через 0. Для signed-типов это **неопределённое поведение** (undefined behavior) — стандарт говорит, что программа может сделать что угодно. На практике обычно происходит то же оборачивание, но компилятор имеет право оптимизировать ваш код на основании предположения «переполнения не будет», и в редких случаях вы получите неожиданный результат. Поэтому правило: **с signed-типами на грани переполнения работать опасно, лучше явно использовать unsigned**.

3.6 Дробные числа: IEEE 754

Целые числа разобрались. А как кодировать 3.14?

Подход называется **числа с плавающей точкой** (floating point). Идея — кодировать число в виде «мантисса $\times 2^{\text{экспонента}}$ ». Например, $3.14 \approx 1.57 \times 2^1$. Мантисса 1.57 хранится приближённо, экспонента 1 — точно.

Стандарт IEEE 754 определяет два размера: - **float** (32 бита): 1 бит на знак, 8 битов на экспоненту, 23 бита на мантиссу. Примерно 7 значимых десятичных цифр точности. - **double** (64 бита): 1 бит на знак, 11 битов на экспоненту, 52 бита на мантиссу. Примерно 15-16 значимых цифр.

Главная неприятность плавающей точки: не каждое десятичное число можно точно представить. Например, 0.1 — это **бесконечная двоичная дробь** (как 1/3 в десятичной). При записи в `float` или `double` она округляется. Поэтому:

```
double x = 0.1 + 0.2;  
// x не равно ровно 0.3, а немного больше: 0.30000000000000004
```

Это не баг, это так устроена IEEE 754. Поэтому сравнивать дробные числа на равенство (`x == 0.3`) почти никогда нельзя — надо проверять «отличается меньше чем на эpsilon». Мы вернёмся к этому в главе 8.

3.7 Текст: ASCII

Хорошо, числа закодировали. А буквы?

Идея простая: каждой букве присвоили число. Латинская A = 65. B = 66. И так далее. Цифра 0 (символ) = 48. Пробел = 32. Перевод строки = 10. И так далее.

Эта таблица соответствий называется **ASCII** (American Standard Code for Information Interchange, принята в 1963). Она использует значения от 0 до 127, то есть помещается в 7 битов (один байт с запасом). В ASCII закодированы: - управляющие символы (0-31): NULL, BACKSPACE, TAB, NEWLINE и так далее; - пунктуация и цифры (32-63): пробел, !, ", ..., 0-9, ..., ?; - латинские заглавные (65-90): A-Z; - больше пунктуации (91-96): [, \,], ^, _, `; - латинские строчные (97-122): a-z; - ещё пунктуация (123-126): {, |, }, ~; - DEL (127).

В C++ строковый литерал "A" — это байт со значением 65. Сравнение символов — это просто сравнение чисел: 'a' < 'b' истинно, потому что 97 < 98.

3.7.1 Чего не хватало в ASCII

ASCII покрывает английский. А русский? Казахский? Китайский?

В 80-90-х годах разные страны делали «расширения ASCII», использующие старший бит (тот самый 128-й). Получались разные **кодировки**:

- **KOI8-R** (Россия, Linux 90-х): русские буквы во второй половине таблицы.
- **CP1251 (Windows-1251)**: русские буквы, но в другом порядке.
- **CP866 (DOS)**: русские буквы плюс псевдографика.
- **ISO 8859-1, 8859-2, 8859-5, ...**: западноевропейские, восточноевропейские, кириллические языки соответственно.

Каждая страна — своя кодировка. Если открыть русский текст в KOI8-R как CP1251 — получите кракозябры. Если в CP866 как KOI8-R — получите другие кракозябры. Это было адом до конца 90-х.

И ещё хуже: даже в одной стране разные системы использовали разные кодировки. Windows — CP1251, Mac — MacCyrillic, Linux — KOI8-R или ISO 8859-5. Открыть документ из одной системы в другой — лотерея.

С китайским/японским/корейским вообще беда: символов десятки тысяч, в один байт никак не помещается. Использовали двухбайтные кодировки (Shift-JIS для японского, GB2312 для китайского, Big5 для традиционного китайского), и они тоже не дружили друг с другом.

3.7.2 Юникод и UTF-8

В начале 90-х придумали **Юникод** — единую таблицу для всех языков мира. Сейчас в Юникоде около 150 тысяч символов: латиница, кириллица, китайские иероглифы, иврит, арабский, индийские письменности, эмодзи (☹ — это тоже символ юникода), даже клингонский язык (из Star Trek) и музыкальные ноты.

Каждый символ юникода имеет номер — **code point**. Латинская А имеет code point U+0041 (= 65 в десятичной, это совместимо с ASCII). Русская А — U+0410. Казахская ә — U+04D8. Эмодзи ☹ — U+1F600.

Но как именно записать code point в файл? У вас уже два-три байта на символ, простая «таблица» не работает. Придумали несколько способов:

- **UTF-32**: каждый символ — ровно 4 байта. Очень просто, но раздуто (для текста на латинице — в 4 раза больше места).
- **UTF-16**: каждый символ — 2 или 4 байта. Используется в Windows и Java внутренне.
- **UTF-8**: каждый символ — от 1 до 4 байт. Самый популярный в вебе и Linux.

UTF-8 устроен хитро: - Символы 0-127 (ASCII) — кодируются одним байтом, как в ASCII. Полная совместимость со старым латинским текстом. - Символы 128-2047 — два байта. - Символы 2048-65535 (включая всю кириллицу, греческий, арабский) — три байта. - Символы 65536+ (китайский, японский, эмодзи) — четыре байта.

Преимущество: английский текст в UTF-8 занимает столько же места, что и в ASCII (он и есть ASCII). Русский — раздувается в 2 раза по сравнению с CP1251 (3 байта против 2 в CP1251). Но зато один формат на всё, никаких кракозябр.

В современной разработке **используйте всегда UTF-8**. Все мои файлы в этой книге — UTF-8. Все исходники C++, которые вы напишете — пусть будут UTF-8. Это де-факто стандарт.

3.7.3 Что это значит для C++

В C++ есть два разных уровня работы с текстом.

Старый уровень — char как байт. `std::string` — это последовательность `char`-ов, то есть последовательность байтов. Длина строки "Hello" — 5 байтов / 5 «символов» (всё совпадает, потому что ASCII). Длина строки "Привет" — **12 байтов**, хотя «символов» там 6. Потому что каждая кириллическая буква в UTF-8 занимает 2 байта.

Это значит: если у вас в коде написано `if (str.size() == 6) ...` для русской строки «Привет» — условие не работает. Длина по байтам — 12.

Современный уровень — `std::u8string` (с C++20). Тип специально для UTF-8 строк. Но в C++11/14/17 — обычно работают на байтовом уровне через `std::string`.

В книге, особенно в проекте RPG, мы будем явно поддерживать русские названия предметов и локаций. Поэтому будет несколько разделов про правильную работу с UTF-8 — например, как корректно посчитать «количество символов» в русском названии.

3.8 Размеры и единицы

Раз мы говорим про память, нужно зафиксировать единицы:

- **1 байт** = 8 бит. Это единица минимальная для адресации. У каждого байта в памяти свой адрес.
- **1 КБ** (килобайт) = **1024 байт** в традиции вычислительной техники (а в системе СИ — 1000, но обычно используется 1024). Это 2^{10} .
- **1 МБ** (мегабайт) = 1024 КБ = $1024 \times 1024 = 1\,048\,576$ байт. Это 2^{20} .
- **1 ГБ** (гигабайт) = 1024 МБ = $2^{30} \approx 1\,073\,741\,824$ байт.
- **1 ТБ** (терабайт) = 1024 ГБ = 2^{40} .

Иногда (особенно на коробках жёстких дисков) производители используют десятичные килобайты: 1 КБ = 1000 байт. Поэтому «диск 1 ТБ» на самом деле часто 931 ГиБ в традиции 1024. Чтобы избежать путаницы, в IT встречаются обозначения KiB (kibibyte), MiB (mebibyte), GiB (gibibyte) — это явно «по 1024». Но в речи обычно «килобайт» = 1024 байт, потому что так удобнее в программировании.

Когда говорят «у меня 16 ГБ оперативки», обычно имеют в виду $16 \text{ ГиБ} = 16 \times 2^{30} = \sim 17.2$ миллиарда байт.

3.9 Что увидите в C++

Когда дойдём до главы 8 «Переменные и типы», эти знания пригодятся:

- `unsigned char` — это байт. Значения 0..255.
- `signed char` — это байт, интерпретированный как число со знаком. Значения -128..+127.
- `char` — это байт, может быть `signed` или `unsigned` (зависит от компилятора и платформы). Для текста используется неявно.
- `int` обычно 4-байтное целое со знаком, диапазон -2 миллиарда..+2 миллиарда.
- `unsigned int` — то же 4 байта, но 0..4 миллиарда.
- `float` — 4 байта, IEEE 754, ~7 десятичных цифр точности.
- `double` — 8 байт, IEEE 754, ~15-16 цифр точности.
- `bool` — формально 1 байт (хотя нужен только 1 бит). Значения `true` (1) и `false` (0).

И ещё: операторы вроде `&` (битовое И), `|` (битовое ИЛИ), `^` (исключающее ИЛИ), `~` (инверсия), `<</>>` (сдвиги) — работают именно на двоичном представлении. То есть `5 & 3` — это «по битам И числа 101 и 011», результат `001 = 1`. К этому вернёмся в главе 8.

3.10 Маленькое упражнение

Без программирования, на бумажке:

1. Преобразуйте число 100 в двоичную систему. Сколько битов понадобится?
2. Преобразуйте `0xSAFE` в десятичную систему. Сколько байт оно занимает?
3. Если число 200 положить в `signed char` (диапазон -128..+127) — что произойдёт?
Подсказка: вспомните дополнительный код.
4. Сколько байт занимает строка "привет" в UTF-8? А строка "hello"?
5. Какое максимальное значение можно представить в 64-битном беззнаковом целом?
Дайте порядок (10 в какой степени).

Если на какой-то вопрос затрудняетесь — вернитесь к соответствующему разделу. Эти штуки реально нужны для C++; их потом сложнее доучивать на ходу.

3.11 Что дальше

Глава 3 — про то, что такое программа и язык программирования. Почему одну программу можно «запустить» сразу (Python: `python file.py`), а другую надо сначала «скомпилировать» (C++: `g++ file.cpp`, потом `./a.out`)? В чём отличие машинного кода от ассемблера? Откуда вообще взялся C++? Это короткая глава, но без неё не понять, почему в C++ так много возни вокруг компилятора.

Глава 4

Глава 3. Что такое программа и язык программирования

В прошлой главе мы выяснили, что внутри компьютера всё — это числа. Процессор умеет с ними работать: складывать, сравнивать, прыгать. А что заставляет процессор делать это в нужной последовательности? **Программа**. Чем программа отличается от просто данных, что значит «язык программирования», почему одни программы компилируются, а другие интерпретируются — об этом и будет глава. Без C++ ни одной строки.

4.1 Программа — это последовательность инструкций

В первой главе я говорил, что процессор умеет выполнять «инструкции» — маленькие действия типа «сложи два числа», «сравни», «прыгни». Программа — это просто **список таких инструкций**. Лежит этот список в памяти, процессор берёт оттуда инструкцию за инструкцией и выполняет.

Хитрый момент: сама программа тоже хранится как набор чисел. Каждой инструкции соответствует своё число (или последовательность чисел). Когда процессор «читает следующую инструкцию» — он на самом деле читает несколько байт из памяти и интерпретирует их как «инструкция такая-то». Точно так же, как программа читает несколько байт и интерпретирует их как число или как букву.

Эта «двойная сущность» — байтов одновременно как данных и как инструкций — называется **архитектурой фон Неймана**, по имени математика Джона фон Неймана, который её предложил в 1945 году. До этого первые компьютеры (ENIAC) хранили программу в коммутационных проводах: чтобы запустить другую программу — приходилось буквально перетягивать провода в нужные гнезда. Фон Нейман предложил: пусть программа лежит в той

же памяти, что и данные. Это позволило компьютерам стать универсальными машинами, на которых можно запустить любую программу без перепайки.

Между прочим, именно из-за этой архитектуры существуют такие штуки, как вирусы и эксплойты: данные, которые программа читает (например, из сети), иногда удаётся подсунуть так, что они начнут выполняться как код. Это огромная тема информационной безопасности; в книге мы её касаемся вскользь, но имейте в виду.

4.2 Машинный код

Числа, в которые превращаются инструкции процессора, называются **машинным кодом** (machine code). Это самый «низкий» уровень, ниже него уже только электрические сигналы.

Покажу пример. Возьмём инструкцию x86-64: «сложить число 5 с содержимым регистра `eax`». В машинном коде она выглядит так:

```
83 c0 05
```

Три байта: `0x83`, `0xc0`, `0x05`. Первый байт `0x83` — код операции (опкод) «прибавь маленькое константное число к регистру». Второй байт `0xc0` — указание, к какому именно регистру (`eax`). Третий байт `0x05` — собственно константа 5.

Если процессор увидит эти три байта подряд — он распознает: «ага, надо прибавить 5 к `eax`», и выполнит.

На x86-64 инструкции бывают разной длины — от 1 до 15 байт. На ARM64 проще: все инструкции ровно 4 байта. Это одна из принципиальных разниц «архитектур с переменной длиной инструкций» (CISC, к ним относится x86) и «архитектур с фиксированной длиной» (RISC, к ним относится ARM).

Так вот, в принципе, программу можно писать прямо в машинном коде: набирать байты в hex-редакторе. И в 50-х так и делали — потому что других вариантов не было. Это было адом: один неверный байт ломал всю программу, никаких имён переменных, ничего. Один человек писал программу неделями. Поэтому очень быстро придумали следующий слой.

4.3 Ассемблер

Ассемблер (assembly language) — это «человеческое» представление машинного кода. Вместо `83 c0 05` пишут:

```
add eax, 5
```

Читается: «прибавь 5 к регистру `eax`». Каждая инструкция машинного кода имеет своё мнемоническое имя (для сложения — `add`, для умножения — `mul`, для прыжка — `jmp`, и так далее). Программист пишет на ассемблере, а специальная программа — **ассемблер** (или транслятор) — превращает текст в машинный код.

Сам по себе ассемблер мало что меняет в сложности: вы по-прежнему пишете каждую инструкцию вручную, программа состоит из десятков тысяч таких строк. Но хотя бы можно читать и редактировать. И сейчас, когда нужно выжать максимум скорости (например, на горячих участках в видеокодеках или криптографических библиотеках), часть кода всё ещё пишется на ассемблере — потому что компилятор C++ при всей своей умности не всегда сгенерирует самый оптимальный код.

В обычной разработке ассемблер вы видеть будете редко. Но иногда полезно — в этой книге, в главе 7, мы посмотрим на ассемблерный листинг нашего `hello.cpp`, чтобы понять, во что превращается C++-код.

Один маленький, но важный нюанс: **ассемблер привязан к архитектуре процессора**. Ассемблер для x86-64 — это один синтаксис, для ARM64 — совсем другой. Программа на ассемблере, написанная под Intel, не работает на Mac M3. Это была одна из причин появления следующего слоя.

4.4 Языки высокого уровня

В середине 50-х годов прозвучала революционная идея: пусть программист пишет не «инструкция за инструкцией», а формулу — а специальная программа сама превращает формулу в нужные инструкции.

Первым успешным языком такого рода стал **Fortran** (1957, IBM). Программа на Fortran выглядела так:

```
X = (A + B) * C
```

Эта одна строка — то, что на ассемблере заняло бы 4-5 строк (загрузить A в регистр, прибавить B, перемножить с C, записать в память). Программа, которая превращает Fortran-текст в машинный код — называется **компилятор**.

Fortran ускорил написание программ в десятки раз. Очень быстро появились другие языки:

- **COBOL** (1959) — для бизнес-приложений, до сих пор работает в банках на серверах IBM.
- **Lisp** (1958) — для научных вычислений и (позже) искусственного интеллекта.
- **ALGOL** (1958) — академический язык, повлиявший на синтаксис почти всего, что появилось позже.

- **BASIC** (1964) — для обучения; на нём в 80-х учились программировать миллионы школьников.

И вот теперь добираемся до нашей родословной.

4.4.1 В и С

В 1969 году Кен Томпсон в Bell Labs писал операционную систему Unix. Он начал на ассемблере, но быстро понял: с ассемблером работать неудобно. Особенно когда хочется переносить Unix на разные машины (а Unix как раз и задумывалась переносимой между разными моделями). Томпсон сделал свой язык **В** — упрощённую версию более старого языка BCPL.

В 1972 году коллега Томпсона Деннис Ритчи доработал В и переименовал в **С**. Главные отличия от В: явные типы данных (int, char, и так далее), структуры, лучшее соответствие железу. На С перенесли Unix — то есть Unix-ядро было написано не на ассемблере, а на С, с очень небольшими вкраплениями ассемблера. Это позволило переносить Unix на новые архитектуры с минимальными усилиями.

С оказался невероятно успешным. К концу 70-х он стал стандартом системного программирования. На С написаны: ядра Linux, macOS, Windows (частично), компиляторы (включая компиляторы самих C/C++), интерпретатор Python (CPython), движок V8 (на C++, но C-стиль), драйверы устройств, протоколы интернета.

Главные черты С: - **Близок к железу**: каждая С-конструкция превращается в небольшое число машинных инструкций. Никакой «магии». - **Без сборщика мусора**: программист сам управляет памятью. - **Минимальная стандартная библиотека**: всё, что хочется делать, надо писать либо самому, либо подключать внешние библиотеки. - **Указатели**: явное указание адресов в памяти.

С по-прежнему живой и активно используется. Стандарты: C89, C99, C11, C17, C23.

4.4.2 C++

В 1979 году Бьярне Страуструп, тоже в Bell Labs, начал работу над расширением С, которое назвал «C with Classes». В 1983 году переименовал в **C++** (плюс-плюс — отсылка к оператору инкремента в С, мол «следующая версия»). Первая официальная версия — 1985 год.

Зачем понадобилось С расширять? Главные идеи:

- **Классы и объекты** (объектно-ориентированное программирование) — для крупных программ удобно описывать сущности с их данными и методами.
- **Шаблоны** (generics) — писать код, который работает с любым типом.

- **Стандартная библиотека:** контейнеры (массивы, списки, хеш-таблицы) поставляются вместе с языком.
- **Исключения** — для обработки ошибок.
- **RAII** — автоматическое освобождение ресурсов через деструкторы.

При этом C++ сохранил всю мощь C: указатели, ручное управление памятью, скорость на уровне ассемблера. Главный девиз: «не платить за то, что не используешь» (You don't pay for what you don't use). Если вам не нужны исключения — компилятор не будет генерировать никакого оверхеда на их поддержку.

Стандарты C++: - **C++98** (1998) — первый официальный стандарт ISO. Очень долго был основным. - **C++03** (2003) — мелкие исправления. - **C++11** (2011) — революция. Move-семантика, лямбды, range-based for, smart pointers, auto, и так далее. Наша книжная база. - **C++14** (2014) — мелкие улучшения, make_unique, generic lambdas. - **C++17** (2017) — optional, variant, filesystem, structured bindings, parallel algorithms. - **C++20** (2020) — concepts, ranges, coroutines, modules — снова большая революция. - **C++23** (2024) — мелкие штрихи к C++20, expected, <print>.

В этой книге мы пишем на C++11 (с пометками про C++14/17 в финальных главах). Это «золотое сечение»: достаточно современно, чтобы код был приятным, и достаточно консервативно, чтобы собиралось у всех.

4.4.3 Другие популярные языки

Чтобы поместить C++ в общую картину:

- **Java** (1995) — компилируется в **байт-код**, который выполняется виртуальной машиной JVM. Кросс-платформенность из коробки. Управление памятью — автоматическое (garbage collector). Используется в банках, Android, серверах.
- **C#** (2000) — почти как Java, только от Microsoft. Используется в Windows-разработке, Unity для игр.
- **Python** (1991) — **интерпретируемый**, динамически типизированный. Просто пишется, медленно выполняется. Для скриптов, анализа данных, AI.
- **JavaScript** (1995) — изначально для браузеров, теперь везде. JIT-компилируется V8/SpiderMonkey/JavaScriptCore.
- **Go** (2009) — Google, типа «упрощённый C++ с автоматическим управлением памятью», для серверов.
- **Rust** (2015) — «C++ без выстрелов в ногу», с проверкой памяти на этапе компиляции. Будущий конкурент C++.
- **Swift** (2014) — Apple, для iOS/macOS, типа «современный, безопасный C++».

C++ в этой компании — мощный, быстрый, опасный (без защиты от ошибок), универсальный. Его учат, потому что (а) на нём написано слишком много существующего кода, чтобы он умер; (б) когда нужна максимальная скорость — C++ её даёт; (в) много концепций C++ влияет на другие языки (если выучите C++, потом легче в Rust, Swift, даже Python станет понятнее).

4.5 Компиляция vs интерпретация

Я уже несколько раз бросил это слово — компилятор, интерпретатор. Сейчас точно объясню разницу.

4.5.1 Компиляция

Компилируемый язык — программа сначала **полностью переводится** в машинный код, и только потом запускается. Цикл выглядит так:

```
file.cpp → [компилятор] → file (исполняемый) → запуск
|
| текст                машинный код
```

Когда вы пишете `g++ hello.cpp -o hello`, компилятор берёт текст, анализирует его, генерирует машинные инструкции, сохраняет в файл `hello`. Дальше вы запускаете `./hello`, и операционная система берёт уже готовый машинный код и отдаёт процессору на выполнение.

Плюсы: - **Быстро на этапе выполнения**: процессор работает с готовым машинным кодом, никаких дополнительных слоёв. - **Контроль ошибок на этапе сборки**: компилятор находит много ошибок (опечатки, неправильные типы) до того, как программа запустится. - **Не нужен компилятор у пользователя**: вы раздаёте исполняемый файл, он запускается без всего.

Минусы: - **Долгий цикл разработки**: правишь файл → перекомпилируешь → запускаешь. На больших проектах перекомпиляция занимает минуты. - **Привязка к платформе**: скомпилированный под Linux x86-64 — не запустится на macOS ARM64. Нужно компилировать заново на каждой целевой платформе.

Компилируемые языки: C, C++, Rust, Go, Swift, Fortran, Pascal.

4.5.2 Интерпретация

Интерпретируемый язык — программа **выполняется построчно** специальной программой-интерпретатором. Никакого «полного перевода в машинный код» не происходит:

```
file.py → [интерпретатор python] → выполнение
```

Когда вы пишете `python script.py`, интерпретатор Python берёт текст, читает первую строку, понимает её, исполняет (используя свои внутренние функции, написанные на C/C++!), читает вторую, исполняет, и так далее. Результат вашей программы — это последовательность действий интерпретатора, а не результат работы машинного кода вашей программы (которого нет).

Плюсы: - **Быстрый цикл разработки:** правишь файл → сразу запускаешь. Никакой пересборки. - **Кросс-платформенность бесплатно:** один файл работает везде, где есть интерпретатор Python (или Ruby, или PHP). - **Простой синтаксис:** не нужно объявлять типы заранее, можно лепить переменные на лету.

Минусы: - **Медленно:** интерпретатор делает много работы на каждое действие (разбор синтаксиса, поиск переменных в словаре, проверки типов). Программы на Python обычно в 10-100 раз медленнее аналогов на C++. - **Меньше ошибок на этапе разработки:** опечатку в имени переменной интерпретатор увидит только тогда, когда выполнение до неё дойдёт. - **Нужен интерпретатор у пользователя:** чтобы запустить ваш Python-скрипт, у пользователя должен стоять Python.

Интерпретируемые языки: классический Python, Ruby, Perl, Shell-скрипты, JavaScript (в браузерах исторически), PHP.

4.5.3 JIT-компиляция (между двумя мирами)

Современные интерпретаторы (V8 для JavaScript, PyPy для Python, JVM для Java) используют JIT (Just-In-Time) компиляцию. Идея такая: интерпретатор замечает, что какая-то функция вызывается часто (горячий код), и налету компилирует её в машинный код, а потом следующие вызовы идут уже по машинному коду. Это даёт скорость, близкую к компилируемым языкам, при сохранении гибкости интерпретации.

JIT — это сложная штука, в книге мы её не рассматриваем. Но знать, что JavaScript в Chrome или Java в банковских системах работает быстрее, чем «честный интерпретируемый Python» — полезно.

4.5.4 Где здесь C++

C++ — **строго компилируемый**. Никакого интерпретатора нет (точнее, есть экспериментальные `cling/Cling`, но это редкость и для специальных задач). Поэтому в C++-разработке цикл всегда такой:

1. Пишете в редакторе `file.cpp`.
2. Сохраняете.
3. Запускаете `g++ file.cpp -o file`.

4. Если есть ошибки — компилятор их выводит. Возвращаетесь к шагу 1.
5. Если ошибок нет — `./file` запускает программу.
6. Если результат не тот — возвращаетесь к шагу 1.

Этот цикл на больших проектах становится длинным. Поэтому появилось много инструментов: системы сборки (которые перекомпилируют только изменившиеся файлы), кэшеры компиляции (ccache, sccache), удалённая сборка (distcc). К ним мы вернёмся ближе к проекту 1.

4.6 Как программа становится исполняемой — общий контур

Раз C++ — компилируемый, у программы на C++ есть несколько стадий жизни от текста до запуска. Я уже намекал на это в первой главе про hello world, но соберу здесь:

1. **Препроцессинг** — текстовая обработка `#include`, `#define`, `#ifdef`. На выходе — «развёрнутый» исходник, обычно намного длиннее (тысячи строк).
2. **Компиляция (в узком смысле)** — превращение C++-текста в **ассемблерный код** для целевой архитектуры.
3. **Ассемблирование** — превращение ассемблерного текста в **объектный файл** (машинный код плюс список «дырок», где должны быть подставлены адреса других функций).
4. **Линковка** — соединение нескольких объектных файлов и стандартной библиотеки в единый исполняемый файл, с подстановкой всех «дырок».

В команде `g++ hello.cpp -o hello` все четыре стадии происходят за один вызов. Но вы можете попросить остановиться после каждой: `- g++ -E hello.cpp -o hello.i` — только препроцессинг. `- g++ -S hello.cpp -o hello.s` — препроцессинг + компиляция (получите ассемблер). `- g++ -c hello.cpp -o hello.o` — препроцессинг + компиляция + ассемблирование (получите объектный файл). `- g++ hello.cpp -o hello` — все четыре стадии (получите исполняемый файл).

В главе 7 мы пройдемся по этим стадиям подробно — что именно происходит, как выглядят результаты, что такое объектные файлы и символы.

4.7 Линковка чуть подробнее

Я упомянул «линковку» — что это значит, разберу сейчас по чуть-чуть. Эта тема — главная боль начинающих C++-разработчиков, потому что «ошибки линковки» отличаются от «ошибок компиляции», и их сообщения часто непонятные.

Пример. Допустим, ваша программа состоит из двух файлов:

`math.cpp` — где вы написали функцию сложения:

```
int add(int a, int b) {  
    return a + b;  
}
```

main.cpp — где вы её используете:

```
int add(int a, int b);  
  
int main() {  
    return add(2, 3);  
}
```

Когда вы компилируете main.cpp, компилятор видит вызов add(2, 3). Тут он знает, что add — это какая-то функция, но не знает, где она находится физически. Поэтому в объектном файле main.o он оставляет «дырку»: «здесь должен быть переход на функцию add». Имя функции add сохраняется как **символ**.

Когда вы компилируете math.cpp, в объектный файл math.o попадает машинный код функции add, и заодно — пометка «здесь определён символ add по такому-то адресу».

Дальше работает **линкер** (ld или эквивалент). Он берёт main.o и math.o, видит в main.o «дырку» с именем add, и в math.o — определение add, и соединяет их. Получается единый исполняемый файл.

Если компилируете отдельно, то g++ за раз делает только компиляцию (через флаг -c):

```
$ g++ -std=c++11 -c math.cpp -o math.o  
$ g++ -std=c++11 -c main.cpp -o main.o  
$ g++ math.o main.o -o program  
$ ./program
```

Чаще, конечно, пишут одной командой, и g++ сам всё разруливает:

```
$ g++ -std=c++11 main.cpp math.cpp -o program
```

Что может пойти не так:

- **undefined reference to add** — линкер не нашёл определения. Возможно, вы забыли скомпилировать math.cpp. Или объявили функцию, но не написали её тело.
- **multiple definition of add** — две версии функции с одним именем. Линкер не знает, какую выбрать.
- **cannot find -lXYZ** — линкер не нашёл библиотеку XYZ. Возможно, забыли установить её, или не указали где искать.

Эти ошибки появляются в самом конце сборки, после того, как все файлы успешно скомпилировались по отдельности. Поэтому новичкам всегда непонятно: «у меня же всё компилировалось, что не так?». Дело в том, что компилятор и линкер — разные программы, и у них разная информация.

К линковке мы вернёмся в главе 7 и потом регулярно — каждый раз, когда наша программа будет состоять больше чем из одного файла.

4.8 Стандартная библиотека

Когда вы пишете `#include <iostream>`, вы подключаете **стандартную библиотеку C++**. Это огромный набор готовых классов и функций, который поставляется вместе с компилятором. Туда входят:

- Контейнеры: `std::vector`, `std::string`, `std::map`, и так далее.
- Алгоритмы: `std::sort`, `std::find`, `std::accumulate`.
- Ввод-вывод: `std::cout`, `std::cin`, `std::ifstream`.
- Потоки: `std::thread`, `std::mutex`.
- Время: `<chrono>`.
- Регулярные выражения: `<regex>`.
- Файловая система (с C++17): `std::filesystem`.

Стандартная библиотека — большая и мощная. Половина книги посвящена её освоению. В коде её ещё называют **STL** (Standard Template Library), хотя STL — это её более узкая часть (контейнеры, итераторы, алгоритмы); полное название — **C++ Standard Library**.

На разных платформах библиотека реализована по-разному, хотя интерфейс один и тот же: - На Linux обычно используется **libstdc++** (от GCC). - На macOS — **libc++** (от LLVM/Clang). - На Windows — **MSVC STL** (от Microsoft).

Все три реализации выполняют одно и то же поведение, описанное стандартом C++. Различия — в деталях реализации и в производительности. Для нашей книги это неважно: код будет одинаково работать со всеми тремя.

4.9 Что увидите в C++

К следующей главе оставим в голове:

- C++ — компилируемый язык. Файл `.cpp` сам не запускается — нужна сборка через компилятор.
- За компиляцию отвечает компилятор (`g++`, `clang++`, `MSVC cl`).

- Программа на C++ часто состоит из нескольких файлов. Каждый компилируется отдельно (получается .o), потом всё соединяется линкером в один исполняемый.
- Машинный код, который генерирует компилятор, привязан к архитектуре процессора. Для x86-64 и ARM64 — разный.
- В C++ есть огромная стандартная библиотека, которая подключается через `#include`.
- Существует много стандартов C++: 98, 11, 14, 17, 20, 23. Каждый добавляет новые возможности. Наша база — 11.

4.10 Маленькое упражнение

На бумажке:

1. Можно ли запустить программу `hello.exe`, скомпилированную под Windows x86-64, на Mac M3 (ARM64)?
2. Можно ли запустить программу `hello.exe`, скомпилированную под Windows x86-64, на Intel Mac с macOS?
3. У вас есть исходник `script.py` на Python. Что нужно, чтобы его запустить?
4. У вас есть исходник `main.cpp` на C++. Что нужно, чтобы его запустить?
5. В чём принципиальная разница между компилятором и интерпретатором?

Ответ к 1 — нет (разные архитектуры). К 2 — нет (формат исполняемого файла Windows .exe не понимает macOS, хотя архитектура та же; есть Wine для эмуляции). К 3 — установленный Python. К 4 — установленный компилятор. К 5 — компилятор превращает текст в машинный код целиком, интерпретатор выполняет текст построчно во время работы.

4.11 Что дальше

Глава 4 — про алгоритмическое мышление. Что такое алгоритм? Как решать задачу «вычислить максимум из трёх чисел» в виде последовательности шагов? Что такое переменная не в смысле «`int x`», а в смысле общей идеи? Условие, цикл, функция — концептуально. Всё на псевдокоде по-русски, без C++. Без этой главы погружение в синтаксис C++ будет «как делать?» вместо «что делать?». Сначала научимся «что», потом «как».

Глава 5

Глава 4. Алгоритмическое мышление

К этой главе вы уже знаете, что компьютер выполняет инструкции, что языки программирования бывают компилируемые и интерпретируемые, что есть машинный код и есть высокоуровневый. Теперь — самое важное для будущего программиста: **как думать алгоритмически**. Это даже важнее, чем знать конкретный язык. Хороший программист отличается от плохого не тем, что лучше помнит синтаксис C++, а тем, что умеет разложить сложную задачу на простые шаги.

В этой главе по-прежнему ни строчки C++. Только русский язык и формальные «шаги». Когда мы доберёмся до главы 8 — там уже все эти концепции попадут в C++-синтаксис, и переход будет мягким.

5.1 Что такое алгоритм

Алгоритм — это **последовательность чётко описанных шагов**, которая решает какую-то задачу. Не «решает любую задачу когда-нибудь как-то», а вот конкретно эту задачу за конечное время.

Слово «алгоритм» — арабского происхождения, искажение имени персидского математика IX века Аль-Хорезми. Он написал учебник по арифметике, где описывал процедуры вычислений шаг за шагом. С тех пор любую такую процедуру и называют алгоритмом.

Алгоритм — это не обязательно про компьютеры. Рецепт борща — алгоритм. Инструкция по сборке Lego — алгоритм. Маршрут «как доехать из дома до работы» — алгоритм. Просто компьютер выполняет алгоритмы по другим правилам: формально, без права на «и так понятно», без интуиции.

5.2 Алгоритм в быту: заварить чай

Самый простой пример. Как заварить чай? Кажется очевидным, но попробуем расписать формально.

Версия 1 (неформальная): 1. Поставить чайник. 2. Подождать пока закипит. 3. Положить в чашку чайный пакетик. 4. Залить кипятком. 5. Подождать минуты три. 6. Выпить.

Ок, для человека сойдёт. Но если бы это писали для робота, возникли бы вопросы. «Поставить чайник» — это куда? Налить в него воду — забыли. Включить — забыли. «Подождать пока закипит» — как робот узнает, что закипел? У него есть датчик?

Версия 2 (для робота): 1. Налить в чайник 200 мл воды. 2. Закрыть чайник крышкой. 3. Включить чайник. 4. **Повторять:** проверить, выключился ли чайник автоматически. Если ещё нет — подождать 5 секунд. Если выключился — перейти к шагу 5. 5. Взять чашку. 6. Положить в чашку чайный пакетик. 7. Залить в чашку 200 мл воды из чайника. 8. **Повторять:** засечь время, прошло ли 3 минуты. Если нет — подождать 10 секунд. Если да — перейти к шагу 9. 9. Вынуть пакетик из чашки. 10. Поставить чашку на стол.

Гораздо подробнее. Но именно так нужно думать про алгоритм. Каждый шаг — простое, однозначное действие. «Повторять» — стандартный приём цикла. Шаг должен быть выполнен без интерпретации; если есть «и так понятно» — это плохой шаг.

5.3 Вход, обработка, выход

Алгоритмы в программировании почти всегда устроены по одной схеме:

ВХОД → ОБРАБОТКА → ВЫХОД

- **Вход** — данные, которые алгоритм получает. Например: список оценок ученика.
- **Обработка** — что алгоритм с ними делает. Например: складывает все оценки и делит на их количество.
- **Выход** — результат. Например: средний балл.

Возьмём конкретную задачу. «**Посчитать средний балл ученика**»:

- **Вход:** список оценок (например, [5, 4, 5, 3, 4, 5]).
- **Обработка:** сложить все оценки, разделить на их количество.
- **Выход:** одно число (средний балл).

Алгоритм: 1. Принять на вход список оценок. 2. Установить переменную *сумма* = 0. 3. Установить переменную *количество* = 0. 4. **Для каждой** оценки в списке: - прибавить оценку к *сумма*; - увеличить *количество* на 1. 5. Вычислить *средний* = *сумма* / *количество*. 6. Вернуть *средний* как результат.

Шесть шагов, любому человеку понятно. Для компьютера тоже подойдёт: каждый шаг чёткий, никаких неоднозначностей.

5.4 Переменные

В алгоритме чая мы говорили «200 мл воды», «3 минуты». В алгоритме среднего балла — «оценки», «сумма», «количество». Эти штуки — **переменные**: именованные «коробки», в которых хранятся значения.

Представить можно буквально: на столе стоит коробка с надписью «сумма», в коробке лежит число (например, 26). Когда мы говорим «прибавить оценку к сумме», мы: 1. Открываем коробку «сумма», достаём оттуда число (26). 2. Берём текущую оценку (например, 4). 3. Складываем: $26 + 4 = 30$. 4. Кладём 30 обратно в коробку «сумма».

После всех шагов в коробке «сумма» лежит общая сумма всех оценок. Коробка одна, но содержимое менялось.

У переменной есть: - **Имя** — как она называется (например, *сумма*, *количество*, *средний*). - **Значение** — что в ней лежит сейчас (число, текст, и так далее). - **Тип** — какого рода значения она хранит (целое число, дробное число, текст).

В C++ переменные пишутся синтаксисом `int сумма = 0;`. `int` — тип (целое число), *сумма* — имя, `= 0` — начальное значение. Когда мы дойдём до главы 8, увидите этот синтаксис; пока думайте про коробки.

Важная мысль: **компьютер не знает, что значит «сумма»**. Для него это просто этикетка. Можно назвать переменную как угодно — *s*, *total*, *sum*, *сумма*, *kazinaki* — компьютер не возражает. Но программа предназначена и для людей, которые её читают потом. Хорошие имена переменных = код, который легко читать. Это будет одно из важнейших правил.

5.5 Операции и выражения

Над переменными можно выполнять **операции**. Самые простые — арифметические:

- Сложение: $5 + 3 = 8$.
- Вычитание: $5 - 3 = 2$.
- Умножение: $5 * 3 = 15$ (звёздочка используется вместо знака умножения, потому что $\times$ нет на клавиатуре).
- Деление: $6 / 3 = 2$. Тут тонкость: если делить целые числа, можно получить дробь, а можно — целое отбрасывая дробную часть. В разных языках по-разному.
- Остаток от деления: $7 \% 3 = 1$ (после 6 идёт остаток 1). Это очень полезная операция.

И операции сравнения: - Равно: $5 == 5 \rightarrow$ истина. $5 == 6 \rightarrow$ ложь. (Знак $==$ с двумя равенствами, потому что одно равенство означает «положить значение», а сравнение — «проверить равенство». Это одна из основных ловушек начинающих в C++.) - Не равно: $5 != 6 \rightarrow$ истина. - Больше: $5 > 3 \rightarrow$ истина. - Меньше: $5 < 3 \rightarrow$ ложь. - Больше-или-равно: $5 >= 5 \rightarrow$ истина. - Меньше-или-равно: $5 <= 4 \rightarrow$ ложь.

Результат сравнения — это **логическое значение** (истина или ложь, в программировании ещё называют true/false или 1/0).

И логические операции над сравнениями: - И (and): (возраст > 18) И (возраст < 65) — истина, если оба условия выполнены. - Или (or): (день = «суббота») ИЛИ (день = «воскресенье») — истина, если хотя бы одно выполнено. - Не (not): НЕ (возраст < 18) — истина, если возраст не меньше 18 (то есть 18 и старше).

Из переменных, операций и значений собираются **выражения**:

- сумма + 5 — выражение, его значение зависит от того, что лежит в *сумма*.
- (возраст $>= 18$) И (гражданство = «РК») — выражение, результат которого истина или ложь.
- (длина * ширина) - (отступ * 2) — выражение, использующее три переменные и две операции.

Главный приём: **результат выражения можно положить в переменную.**

площадь = длина * ширина

Читается: «вычислить длина * ширина, и положить результат в коробку *площадь*». Если *длина* = 5 и *ширина* = 3, то после этой строки в коробке *площадь* лежит 15.

5.6 Ветвление (условие)

В алгоритме чая был такой шаг: «проверить, выключился ли чайник. Если ещё нет — подождать. Если выключился — перейти дальше». Это **ветвление**: алгоритм решает, что делать дальше, в зависимости от условия.

Общий вид:

```
ЕСЛИ <условие> ТО
    <действие 1>
ИНАЧЕ
    <действие 2>
```

Пример. Алгоритм «можно ли войти в кинотеатр на фильм 18+»:

```
ЕСЛИ возраст  $>= 18$  ТО
```

«Можно проходить»

ИНАЧЕ

«Извините, фильм только для взрослых»

Можно делать несколько проверок подряд:

ЕСЛИ возраст < 7 ТО

«На детский билет»

ИНАЧЕ ЕСЛИ возраст < 18 ТО

«На подростковый билет»

ИНАЧЕ ЕСЛИ возраст < 65 ТО

«На взрослый билет»

ИНАЧЕ

«На пенсионный билет»

Главное: проверки идут **сверху вниз**, и срабатывает **первая подходящая**. Если возраст 5, сработает первый блок и остальные не проверяются. Если 30 — сработает третий. Это важно: порядок проверок имеет значение.

Иногда ветвление бывает без альтернативы:

ЕСЛИ температура > 38 ТО

позвонить врачу

Если температура нормальная — ничего не делаем, идём дальше.

5.7 Цикл

Если в чае был «**повторять** проверку каждые 5 секунд» — это цикл. Цикл повторяет какие-то действия до тех пор, пока условие истинно.

Общий вид:

ПОКА <условие>:

<действия>

Пример. Алгоритм «налить в стакан 100 мл воды чайной ложкой» (одна ложка = 5 мл):

налито = 0

ПОКА налито < 100:

зачерпнуть ложкой воду

вылить в стакан

налито = налито + 5

Сначала *налито* = 0. Заходим в цикл, проверяем условие: $0 < 100$ — истина. Выполняем тело: чашка воды, перелив, *налито* теперь 5. Цикл повторяется: $5 < 100$ — истина. И так далее. Когда *налито* станет 100, условие $100 < 100$ будет ложью — цикл прервётся, и алгоритм продолжится дальше.

Бывает и другой вид цикла — **для каждого элемента**:

для КАЖДОЙ оценки В списке оценок:

<действия с оценкой>

Это, по сути, перебор всех элементов какой-то коллекции (списка, массива). В C++ это `for (auto оценка : оценки)`, мы доберёмся до этого в главе 9.

И ещё один — **повторить N раз**:

ПОВТОРИТЬ 10 РАЗ:

<действия>

Это просто синоним «цикл от 1 до 10».

5.7.1 Бесконечный цикл

Опасность циклов: если условие никогда не станет ложью — цикл будет крутиться вечно. Например:

`x = 5`

`ПОКА x > 0:`

`написать x`

Здесь `x` всегда 5, никогда не уменьшается. Программа будет бесконечно печатать «5, 5, 5, 5, ...» и никогда не остановится. Это **бесконечный цикл**, и это баг.

Иногда бесконечный цикл — это **сознательное решение**. Например, программа-сервер крутится в цикле, ожидая запросов:

`ПОКА истина:`

`ждать запрос`

`обработать запрос`

`отправить ответ`

Этот цикл специально вечный — потому что серверу всегда есть, что делать. Прервётся он только при завершении программы.

5.8 Функция

Допустим, нам нужно посчитать средний балл сразу для пяти учеников. Можно тупо повторить алгоритм пять раз, копируя его. Но это плохо: длинно, легко сделать опечатку, если найдём ошибку — придётся править в пяти местах.

Решение — **функция**. Это «алгоритм, упакованный под имя, который можно вызывать сколько угодно раз, передавая разные входные данные».

Описание функции:

ФУНКЦИЯ `средний_балл` (список оценок):

```
сумма = 0
количество = 0
ДЛЯ КАЖДОЙ оценки В списке оценок:
    сумма = сумма + оценка
    количество = количество + 1
ВЕРНУТЬ сумма / количество
```

Здесь: - **средний_балл** — имя функции. - (**список оценок**) — параметр функции, то, что мы ей передаём. - **ВЕРНУТЬ ...** — что функция отдаёт обратно.

Использование:

```
оценки_Ивана = [5, 4, 5, 3, 4, 5]
балл_Ивана = средний_балл(оценки_Ивана)
```

```
оценки_Маши = [5, 5, 5, 5, 4]
балл_Маши = средний_балл(оценки_Маши)
```

Функция написана один раз, используется много раз. Это **переиспользование**. Главная идея в программировании: «не повторяйся».

Функции — основа всего. Большая программа на C++ — это сотни функций, которые друг друга вызывают. Главная функция `main`, которую мы видели в `hello.cpp` — тоже функция, просто особая (с неё начинается выполнение).

5.8.1 Функция, вызывающая функцию

Функции можно собирать в иерархию. Например, мы хотим написать «средний балл нескольких учеников»:

ФУНКЦИЯ `средний_балл` (список оценок):

```
сумма = 0
```

```
количество = 0
ДЛЯ КАЖДОЙ оценки В списке оценок:
    сумма = сумма + оценка
    количество = количество + 1
ВЕРНУТЬ сумма / количество
```

```
ФУНКЦИЯ средний_класса (список_учеников):
    общая_сумма = 0
    количество = 0
    ДЛЯ КАЖДОГО ученика В список_учеников:
        балл_ученика = средний_балл(ученик.оценки)
        общая_сумма = общая_сумма + балл_ученика
        количество = количество + 1
    ВЕРНУТЬ общая_сумма / количество
```

Видим: `средний_класса` использует `средний_балл` внутри себя. Каждая функция занимается своим делом, не зная подробностей других. Это **декомпозиция** — разбиение большой задачи на маленькие.

5.9 Декомпозиция как главный навык

Когда вы видите большую задачу — типа «написать игру», «написать сервер чата», «написать базу данных» — она кажется неподъёмной. Хитрость в том, чтобы разбить её на маленькие задачи, которые сами по себе ясны.

Возьмём «написать RPG-игру» (наш Проект 1): - Игроком нужно как-то управлять → подзадача «парсер команд». - В игре есть локации → подзадача «модель локации». - Локации связаны между собой → подзадача «карта». - В локациях есть предметы → подзадача «предметы». - Игрок может брать предметы → подзадача «инвентарь». - В локациях есть враги → подзадача «враги». - Игрок может сражаться → подзадача «бой».

И так далее. Каждая подзадача — это уже что-то конкретное, что можно решить. Если подзадача всё ещё кажется большой — её можно ещё разбить. Например, «бой»: - Подсчёт урона → подзадача. - Очки здоровья → подзадача. - Очередность ходов → подзадача. - Что случается при смерти врага → подзадача.

В итоге задача «написать RPG» превращается в десятки маленьких задач, и каждая решается за час-два. Эта способность — «дробить» — главное, что отделяет программиста от непрограммиста.

5.10 Псевдокод и реальный код

Всё, что я писал выше — это **псевдокод**. Это формальный способ описать алгоритм, но не на каком-то конкретном языке программирования, а на смеси русского и математической нотации. Псевдокодом удобно описывать алгоритмы в учебниках, на собеседованиях, в дизайн-документах — потому что он не привязан к языку и понятен любому.

Реальный код на C++ для функции «средний балл» выглядит так:

```
double средний_балл(const std::vector<int>& оценки) {  
    int сумма = 0;  
    int количество = 0;  
    for (int оценка : оценки) {  
        сумма = сумма + оценка;  
        количество = количество + 1;  
    }  
    return static_cast<double>(сумма) / количество;  
}
```

Здесь больше деталей: `double` — это тип результата (число с плавающей точкой), `const std::vector<int>&` — параметр (список целых чисел, передаваемый по ссылке без копирования), `static_cast<double>` — преобразование типа. Эти детали мы будем разбирать главами 8-14.

Но логика — та же самая, что в псевдокоде. Сумма, цикл, деление. Когда вы написали псевдокод — вы решили задачу. Перевод в C++ — это в основном про синтаксис и нюансы языка, не про мышление.

Поэтому **сначала думайте на бумажке**, потом пишите код. Многие новички сразу садятся за клавиатуру и пытаются решать задачу прямо в редакторе — потом теряются в синтаксических ошибках и забывают, что вообще решали. Хорошая привычка: на сложную задачу сначала писать псевдокод (10-20 минут), потом переводить в реальный код (быстро).

5.11 Состояние программы

Ещё одно важное понятие: **состояние программы**. Это полный «снимок» всех переменных в какой-то момент времени.

Например, на середине выполнения нашего алгоритма расчёта среднего балла: - *оценки* = [5, 4, 5, 3, 4, 5]; - *сумма* = 12 (после первых трёх оценок); - *количество* = 3; - мы сейчас на четвёртом шаге цикла.

Это и есть состояние. Программа — это последовательность переходов между состояниями. На каждом шаге что-то меняется.

В крупной программе состояний десятки и сотни переменных, и от их сочетания зависит, что будет дальше. Программисту приходится постоянно держать в голове «а в каком состоянии сейчас программа?», чтобы понять, что произойдёт. Это сложно — поэтому хорошие программисты стараются делать так, чтобы состояний было меньше, и они были предсказуемее. К этому ведут такие подходы как «функциональное программирование» (минимум изменяемого состояния), «иммутабельные объекты» (объекты, которые нельзя изменить после создания) и тому подобное.

В С++ ничего этого нет «из коробки» — язык позволяет менять состояние в любой момент. Поэтому хорошие программисты на С++ сами навязывают себе дисциплину. Об этом ещё много раз поговорим.

5.12 Концепции, которые превратятся в С++

Вот что из этой главы мы потом будем видеть на С++:

Псевдокод	С++
коробка с именем <i>x</i>	<code>int x;</code>
положить значение	<code>x = 5;</code>
взять значение и прибавить	<code>y = x + 3;</code>
ЕСЛИ ... ТО ... ИНАЧЕ	<code>if (...) { ... } else { ... }</code>
ПОКА ...:	<code>while (...) { ... }</code>
ДЛЯ КАЖДОГО ... В ...:	<code>for (auto x : container) { ... }</code>
ПОВТОРИТЬ N РАЗ:	<code>for (int i = 0; i < N; ++i) { ... }</code>
ФУНКЦИЯ ИМЯ (параметры):	<code>тип имя(параметры) { ... }</code>
ВЕРНУТЬ	<code>return ...;</code>

В главах 8-10 этот словарь будет наполняться деталями. Но основа уже понятна.

5.13 Алгоритм сложнее: найти максимум

В завершение разберём алгоритм посложнее. **Дан список чисел, нужно найти максимум.**

Псевдокод:

ФУНКЦИЯ максимум (список чисел):

 ЕСЛИ список пуст ТО

ВЕРНУТЬ ошибку «список пуст»

максимум = первый элемент списка

ДЛЯ КАЖДОГО числа В списке (кроме первого):

ЕСЛИ число > максимум ТО

максимум = число

ВЕРНУТЬ максимум

Разбор. Начинаем с проверки: если список пуст — максимума нет. Эту ситуацию надо как-то обработать. Если бы мы её пропустили — на пустом списке программа поломалась бы.

Дальше идея: берём первый элемент как «текущий максимум». Перебираем все остальные элементы. Если встретили число больше — обновляем максимум. В конце возвращаем то, что в *максимум*.

Проверим на примере [3, 7, 2, 9, 4]: 1. *максимум* = 3 (первый элемент). 2. Очередное число 7. $7 > 3$? Да. *максимум* = 7. 3. Очередное число 2. $2 > 7$? Нет. *максимум* = 7. 4. Очередное число 9. $9 > 7$? Да. *максимум* = 9. 5. Очередное число 4. $4 > 9$? Нет. *максимум* = 9. 6. Вернуть 9. ✓

Работает. Это и есть алгоритмическое мышление: сформулировать стратегию решения, перевести её в шаги, проверить на примере.

5.14 Маленькое упражнение

На бумажке (или в Word) опишите псевдокодом алгоритмы:

1. Дан список чисел. Сосчитать, сколько в нём чётных. (Чётное — то, у которого остаток от деления на 2 равен 0.)
2. Дано целое число N . Вычислить $N! = 1 \times 2 \times 3 \times \dots \times N$.
3. Даны два слова. Проверить, не получено ли одно из другого перестановкой букв (анаграмма). Подсказка: один из способов — отсортировать буквы каждого слова и сравнить.
4. Даны два списка имён. Найти имена, которые есть в обоих списках.

Не нужно никакого кода. Просто запишите шаги. На каждое — 5-10 минут. Если запутаетесь — попробуйте на маленьком примере вручную, шаг за шагом, и потом обобщите.

Если задачи дались легко — поздравляю, у вас уже есть алгоритмическое мышление. Если тяжело — это нормально, это навык, он развивается с практикой. Делайте больше задачек.

5.15 Что дальше

Глава 5 — про терминал и файловую систему. Раз будем компилировать C++-программы, нужно уметь обращаться с командной строкой: где я сейчас нахожусь, как переместиться в другую папку, как создать файл, удалить, скопировать, увидеть содержимое. И ещё про права доступа — что значит «executable» и почему `./hello` запускается, а `hello.txt` нет. Это последняя глава Части 0a; после неё начнём ставить компилятор и писать C++.

Глава 6

Глава 5. Терминал, файлы, папки, права

К этой главе у вас в голове должны лежать: что такое компьютер (глава 1), как кодируются числа и текст (глава 2), что такое программа и компиляция (глава 3), и как думать алгоритмически (глава 4). Последняя глава Части 0a — про **терминал**, командную строку. Без него C++-разработка невозможна: вы будете жить в терминале. Если вы привыкли работать только мышкой и боитесь чёрного окна — после этой главы перестанете.

6.1 Что такое терминал

Терминал (он же «консоль», «командная строка», «shell») — это **текстовый интерфейс к компьютеру**. Вместо того чтобы тыкать мышкой в иконки, вы пишете текстовые команды, нажимаете Enter, и компьютер выполняет.

Зачем нужен текстовый интерфейс, если есть удобный графический? Несколько причин.

Скорость. Опытный программист в терминале делает за секунду то, на что в графическом интерфейсе ушли бы 10-20 секунд кликов. Например, скопировать все .cpp файлы из одной папки в другую: команда `cp *.cpp /target/folder/` — секунда. Через Finder/Explorer: открыть две папки, выделить нужные файлы (а ещё мышкой выделить только .cpp без других — отдельная морока), нажать Cmd+C, перейти в другую папку, Cmd+V. Минут пять.

Автоматизация. Команды можно объединять в скрипты. Один раз написал «делай это, потом это, потом это» — и в следующий раз запускаешь скрипт одной командой. Мышкой такое не записать.

Воспроизводимость. Если я в книге напишу «открой Finder, нажми сюда, потом сюда» — у вас может быть другая версия macOS, и кнопки могут стоять иначе. А команда `ls -la` одинакова и сегодня, и десять лет назад, на любой системе.

Удалённая работа. Серверы (где хостится ваш сайт, например) обычно не имеют графической оболочки — только командная строка через SSH. Если хотите работать с серверами — терминал обязателен.

Программирование требует. Все компиляторы C++, все системы сборки, все системы контроля версий (git) — это инструменты командной строки. Графические обёртки есть, но они скрывают то, что внутри, и мешают понять, что происходит.

Не пугайтесь чёрного окна. Сначала медленно, потом быстро.

6.2 Где он живёт

macOS. Программа называется **Terminal** (или **iTerm2**, более продвинутая бесплатная альтернатива; рекомендую поставить через `brew install --cask iterm2`). Открыть: `Cmd+Space`, набрать «terminal», Enter.

Linux. На Ubuntu это **GNOME Terminal**, на других дистрибутивах могут быть **Konsole**, **xterm**, **alacritty**. Обычно открывается сочетанием `Ctrl+Alt+T` или через меню приложений.

Windows. Тут несколько вариантов. **PowerShell** или **Windows Terminal** (новое приложение от Microsoft, рекомендую) — это «нативные» терминалы Windows. Но их синтаксис команд отличается от Unix-овского (macOS/Linux). Для нашей книги проще установить **WSL2** (Windows Subsystem for Linux) — это полноценный Linux внутри Windows, и тогда команды будут такие же, как в этой главе. В главе 6 мы про это поговорим подробнее. Пока, если вы на Windows и читаете эту главу, я рекомендую: либо примите, что некоторые команды у вас будут отличаться, либо поставьте WSL2 (это 10 минут установки).

6.3 Приглашение к вводу (prompt)

Когда вы открываете терминал, видите примерно такое:

```
serik@MacBook-Pro ~ %
```

Или такое:

```
serik@ubuntu:~$
```

Это **приглашение** (prompt). Оно состоит из: - `serik` — имя пользователя (вашей учётной записи в системе). - `MacBook-Pro` или `ubuntu` — имя компьютера. - `~` — текущая папка (тильда =

домашняя папка пользователя, об этом ниже). - % или \$ — символ конца приглашения. После него ждётся ваша команда.

Когда я в книге пишу:

```
$ ls
```

Знак \$ — это и есть приглашение. Команда — `ls`. Сам \$ вы не вводите.

После того, как набрали команду и нажали Enter — терминал передаёт её специальной программе, которая называется **shell** (по-русски «оболочка»). Самые популярные шеллы: - **bash** — Bourne Again SHell, стандартный во многих Linux-дистрибутивах. Старый, но мощный. - **zsh** — Z Shell. С macOS Catalina (2019) это шелл по умолчанию на Mac. Совместим с bash, плюс много улучшений. - **fish** — Friendly Interactive Shell. Удобный, с автодополнением, но синтаксис чуть отличается.

В этой книге я пишу под bash/zsh; команды работают одинаково.

6.4 Где я нахожусь — `pwd`

Каждый раз, когда вы в терминале, вы находитесь **в какой-то папке**. Все команды выполняются в контексте этой папки. Команда `pwd` (print working directory) показывает, где вы сейчас:

```
$ pwd
/Users/serik
```

На macOS домашняя папка — `/Users/<имя>`. На Linux — `/home/<имя>`. На Windows (WSL) — что-то вроде `/home/serik`, но «настоящие» Windows-файлы можно найти под `/mnt/c/Users/<имя>`.

6.5 Что в папке — `ls`

Команда `ls` (list) показывает содержимое текущей папки.

```
$ ls
Desktop Documents Downloads Music Pictures
```

Без аргументов выводит просто имена. С флагом `-l` («длинный формат») — много деталей:

```
$ ls -l
total 0
drwx-----+  5 serik  staff   160 Apr 10 14:22 Desktop
drwx-----+  3 serik  staff    96 Mar 22 09:30 Documents
```

```
drwx-----+  8 serik  staff   256 May 11 11:00 Downloads
...
```

Расшифровка одной строки: - drwx-----+ — права доступа (об этом ниже). - 5 — количество ссылок на файл (для папок — что-то техническое, не важно). - serik — владелец. - staff — группа владельца. - 160 — размер в байтах. - Apr 10 14:22 — дата изменения. - Desktop — имя.

С флагом -a (all) показываются **скрытые файлы** — те, чьи имена начинаются с точки:

```
$ ls -a
.  .bash_profile  .gitconfig  Desktop  Downloads
.. .DS_Store      .ssh        Documents Music
```

. — текущая папка (всегда есть). .. — родительская папка. .bash_profile, .gitconfig — конфиги, которые системные программы прячут от глаз, потому что они не для постоянного редактирования.

Можно комбинировать флаги: ls -la или ls -al — длинный формат и включая скрытые.

6.6 Перемещение — cd

Команда cd (change directory) переходит в другую папку:

```
$ pwd
/Users/serik
$ cd Documents
$ pwd
/Users/serik/Documents
$ ls
report.pdf  notes.txt
```

cd <папка> — перейти в подпапку. Чтобы вернуться обратно:

```
$ cd ..
$ pwd
/Users/serik
```

.. — это «на уровень выше». Можно поднимать на несколько уровней: cd ../.. — на два уровня вверх.

cd без аргументов — вернуться в домашнюю папку:

```
$ cd
$ pwd
/Users/serik
```

Эквивалент — `cd ~`. Тильда `~` означает «домашняя папка».

Можно прыгать сразу в любое место по абсолютному пути:

```
$ cd /Users/serik/Desktop/cpp-from-scratch
$ pwd
/Users/serik/Desktop/cpp-from-scratch
```

6.7 Пути — абсолютные и относительные

Это важная концепция, на ней спотыкаются почти все новички.

Абсолютный путь — начинается с корня файловой системы и однозначно указывает на файл/папку. На Unix это что-то вроде `/Users/serik/Desktop/photo.jpg` — начинается со слеша.

Относительный путь — указывает на файл/папку относительно текущей. Например, если вы сейчас в `/Users/serik`, то `Desktop/photo.jpg` — это относительный путь, и он указывает на тот же файл, что абсолютный путь `/Users/serik/Desktop/photo.jpg`.

Специальные относительные пути: `- .` — текущая папка («здесь»). `- ..` — родительская папка («на уровень выше»). `- ~` — домашняя папка.

Примеры. Вы в `/Users/serik/Desktop/cpp-from-scratch/docs/tutorial`:

Путь	На что указывает
<code>00-intro.md</code>	<code>/Users/serik/Desktop/cpp-from-scratch/docs/tutorial/00-intro.md</code>
<code>./00-intro.md</code>	то же самое (явно «здесь»)
<code>../tutorial-original/file.md</code>	соседняя папка
<code>../../README.md</code>	<code>/Users/serik/Desktop/cpp-from-scratch/README.md</code>
<code>~/Downloads/photo.jpg</code>	<code>/Users/serik/Downloads/photo.jpg</code>
<code>/etc/hosts</code>	<code>/etc/hosts</code> (абсолютный)

Относительные пути удобны, когда вы работаете в одном проекте: писать `src/main.cpp` короче, чем `/Users/serik/Desktop/myproject/src/main.cpp`. Абсолютные пути нужны, когда хочется указать на что-то «глобальное» — например, `/usr/bin/g++` (системный компилятор) или `/var/log/system.log` (системный лог).

6.8 Создать папку — `mkdir`

```
$ cd ~  
$ mkdir my-project  
$ ls  
my-project Desktop Documents ...
```

`mkdir <имя>` создаёт папку. Если папка уже есть — выдаёт ошибку.

С флагом `-p` создаются все промежуточные папки:

```
$ mkdir -p projects/2026/cpp/lesson-01  
$ ls projects/2026/cpp/  
lesson-01
```

Если флага `-p` нет, и какой-то промежуточной папки нет — `mkdir` выдаст ошибку.

6.9 Создать или открыть файл

Чтобы создать пустой файл, есть команда `touch`:

```
$ touch hello.cpp  
$ ls  
hello.cpp
```

Если файла не было — создаётся пустой. Если был — обновляется его «время последнего изменения», содержимое не трогается. Это редко нужно в реальной работе, но команда полезная.

Чаще файлы создаются через редактор. Я писал в главе 1 (или, в новой нумерации, в главе 6, которую вы скоро прочтёте), что лучший редактор для нас — VS Code. Чтобы открыть файл в VS Code из терминала:

```
$ code hello.cpp
```

Если файла нет — VS Code откроется с пустым новым файлом этого имени. Сохраните (Cmd+S) — файл создастся.

6.10 Прочитать файл — `cat`

`cat` (concatenate) выводит содержимое файла в терминал:

```
$ cat hello.cpp
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Для маленьких файлов хорошо. Для огромных — терминал захлебнётся (потому что весь файл напечатается за раз).

Если файл большой, лучше использовать `less`:

```
$ less BIG.txt
```

Откроется «листалка» — можно листать стрелками или PgUp/PgDn, поиск через `/`, выход — `q`. Это удобнее, чем `cat`, для логов и больших текстовых данных.

И ещё две полезные: `head <файл>` — выводит первые 10 строк (можно задать другое число: `head -n 5`). `tail <файл>` — выводит последние 10 строк.

Особо полезно: `tail -f <файл>` — выводит новые строки по мере их добавления в файл. Используется для слежения за логами в реальном времени.

6.11 Копировать, перемещать, удалять — `cp`, `mv`, `rm`

```
$ cp source.cpp backup.cpp
```

Копирует. Был `source.cpp`, теперь есть `source.cpp` и `backup.cpp` с тем же содержимым.

```
$ mv old-name.cpp new-name.cpp
```

Переименовывает (или перемещает в другую папку). Не оставляет копии.

```
$ rm file.txt
```

Удаляет. Будьте осторожны: в терминале **нет корзины**. Удалённое — удалено навсегда. Особенно опасно `rm -rf <папка>` — удаляет папку со всем содержимым рекурсивно, без подтверждения. Команда `rm -rf /` на Unix-системе с правами `root` в худшем случае стирает всё. Никогда не запускайте таких вещей бездумно.

С флагом `-r` (recursive) можно копировать или удалять папки:

```
$ cp -r my-project my-project-backup
$ rm -r old-project
```

Без `-r` `cp` или `rm` папку не возьмут — нужно явно сказать «да, я хочу с подпапками».

С флагом `-i` (interactive) команды спрашивают подтверждение перед каждым действием:

```
$ rm -i file.txt
remove file.txt? y
```

Полезная страховка для опасных операций.

6.12 Маски (wildcards)

Вместо имени файла можно использовать **маску** — шаблон с подстановочными символами.

Самый частый — `*` (звёздочка). Она означает «любое количество любых символов».

```
$ ls *.cpp           # все .cpp файлы в текущей папке
$ rm *.tmp           # удалить все .tmp файлы
$ cp src/*.cpp build/ # скопировать все .cpp из src в build
```

Менее частый — `?`. Это «ровно один любой символ»:

```
$ ls file?.txt
file1.txt file2.txt fileX.txt
```

Маски разворачиваются **самим шеллом до запуска команды**. То есть когда вы пишете `rm *.tmp`, шелл сначала находит все `.tmp` файлы (например, `a.tmp` и `b.tmp`), и фактически выполняется `rm a.tmp b.tmp`. Сама программа `rm` про звёздочку не знает. Это важно помнить — иногда у вас не сработает что-то по маске именно потому, что шелл подставил пустой список.

6.13 Права доступа

В выводе `ls -l` мы видели:

```
-rw-r--r--  1 serik  staff  1024  May 11 21:00  notes.txt
drwxr-xr-x  3 serik  staff    96  May 11 21:00  my-project
```

Первый «столбец» из 10 символов — это **права доступа**. Разберём.

Первый символ — тип объекта: `-` — обычный файл; `d` — папка (directory); `l` — символическая ссылка; `-b`, `c`, `s`, `p` — специальные файлы (устройства, сокеты).

Следующие 9 символов — три группы по 3: - Первая тройка — права **владельца** файла. - Вторая — права **группы** владельца. - Третья — права **всех остальных**.

В каждой тройке три буквы: - **r** — read (можно читать). - **w** — write (можно изменять). - **x** — execute (можно выполнить; для папки — войти внутрь).

Если права нет — стоит **-**.

Примеры: - **-rw-r--r--**: обычный файл. Владелец читает и пишет, остальные только читают. - **-rwxr-xr-x**: обычный файл, исполняемый. Все могут запустить, владелец может ещё изменить. - **drwxr-xr-x**: папка. Все могут войти и читать, владелец может ещё изменять содержимое. - **-rw-----**: обычный файл, только владелец может читать и писать. Другие — никак. Так выглядят файлы с паролями.

Сразу важный момент. **Чтобы запустить программу — у файла должны быть права x**. Когда вы компилируете C++-программу через `g++ hello.cpp -o hello`, компилятор автоматически проставляет **x** на выходном файле. Поэтому `./hello` срабатывает. А если бы прав **x** не было — была бы ошибка «Permission denied».

6.13.1 chmod

Для изменения прав используется `chmod` (change mode).

```
$ chmod +x my-script.sh      # добавить право выполнения всем
$ chmod -w file.txt          # снять право записи у всех
$ chmod 644 notes.txt        # установить точно: 6 для владельца, 4 для группы, 4 для остальных
```

Цифры в `chmod 644` — это сложение прав: - 4 = read, - 2 = write, - 1 = execute.

Сумма даёт права тройки: - 7 = 4+2+1 = **rwx** (всё); - 6 = 4+2 = **rw-** (читать и писать); - 5 = 4+1 = **r-x** (читать и выполнять); - 4 = **r--** (только читать); - 0 = **---** (ничего).

`chmod 755 file` — владельцу **rwx**, остальным **r-x**. Обычно так выставляют для скриптов и программ.

`chmod 644 file` — владельцу **rw-**, остальным **r--**. Обычно для обычных файлов.

6.13.2 chown

Меняет владельца файла:

```
$ sudo chown serik myfile.txt
```

Это редко нужно в обычной разработке, но на серверах часто встречается.

6.14 Переменные окружения

В терминале есть **переменные окружения** — что-то вроде глобальных переменных, которые читают программы. Самая важная — PATH.

PATH — это список папок, в которых шелл ищет программы. Когда вы пишете `g++ file.cpp` — шелл по очереди проверяет: есть ли в `/usr/local/bin/g++`? Если нет, в `/usr/bin/g++`? И так далее. Список папок — это и есть PATH.

Посмотреть его содержимое:

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

Папки разделены двоеточием. Команда `echo` просто выводит то, что ей дают. Знак `$` перед именем переменной означает «возьми её значение».

Другие важные переменные: - HOME — путь к домашней папке (`/Users/serik` или `/home/serik`). - USER — имя текущего пользователя. - SHELL — путь к используемому шеллу. - LANG — текущая локаль (например, `ru_RU.UTF-8` или `en_US.UTF-8`).

Установить свою переменную:

```
$ MY_VAR=hello
$ echo $MY_VAR
hello
```

Но эта переменная видна **только в текущем терминале**. Когда закроете и откроете заново — её не будет. Чтобы сделать постоянной, надо добавить в файл `~/.bashrc` (для `bash`) или `~/.zshrc` (для `zsh`) строку `export MY_VAR=hello`.

В разработке часто пользуются переменными окружения для конфигурации программ. Например, программа может читать `DATABASE_URL` и подключаться к БД по этому адресу. Это удобно для разных окружений (`dev/staging/prod`) и для секретов (не хранить пароли в коде).

6.15 Перенаправление и пайпы

Это одна из самых мощных штук в Unix-философии. Расскажу основы; в проекте Mini-shell (Часть III) мы будем строить такое сами.

6.15.1 Стандартные потоки

У каждой программы есть три «стандартных канала»: - **stdin** (standard input) — стандартный ввод. Откуда программа читает данные. По умолчанию — клавиатура. - **stdout** (standard output)

— стандартный вывод. Куда программа пишет результат. По умолчанию — терминал. - **stderr** (standard error) — стандартный поток ошибок. Куда программа пишет сообщения об ошибках. По умолчанию — тоже терминал.

Эти каналы можно **перенаправлять**.

6.15.2 Запись stdout в файл — >

```
$ ls > files.txt
```

Вместо вывода на экран `ls` пишет результат в файл `files.txt`. Если файл был — перезаписывается. Чтобы добавить в конец, не перезаписывая:

```
$ ls >> files.txt
```

>> дописывает к существующему файлу.

6.15.3 Чтение stdin из файла — <

```
$ sort < unsorted.txt
```

Команда `sort` (которая обычно ждёт данные с клавиатуры или из аргументов) теперь читает из файла `unsorted.txt`.

6.15.4 Пайпы — |

Самое мощное — соединять вывод одной программы со входом другой:

```
$ ls | sort
```

Вывод `ls` идёт во вход `sort`. Получается отсортированный список файлов.

Можно цеплять несколько подряд:

```
$ cat names.txt | grep "Иван" | sort | uniq -c
```

Что произойдёт: 1. `cat names.txt` — вывести содержимое файла. 2. `grep "Иван"` — отфильтровать строки, содержащие «Иван». 3. `sort` — отсортировать оставшиеся. 4. `uniq -c` — подсчитать дубликаты.

Каждая программа делает одну простую вещь, но соединяя их в цепочку — получаем сложную обработку. Это и есть **Unix-философия**: маленькие инструменты, делающие одно дело хорошо, и комбинирование их через пайпы.

6.16 Запуск программы и ./

Вспомним из главы 6 (или из первой версии главы 1): чтобы запустить программу из текущей папки, надо написать `./hello`, а не просто `hello`.

Почему?

Когда вы пишете `hello`, шелл ищет программу с этим именем **в папках из PATH**. Текущую папку — **не** проверяет. Это сделано из соображений безопасности: представьте, вы скачали архив с какой-то программы, распаковали в `/tmp`, перешли туда, и набрали `ls` — вместо системного `ls` шелл мог бы запустить злонамеренный `ls`, лежащий в архиве. Поэтому текущая папка в PATH не входит (на Unix; на Windows входит, и поэтому Windows исторически уязвимее).

`./hello` явно говорит: запустить программу `hello` из текущей папки (`.`). Точка тут — та же самая, что в `cd .` или `ls .` — означает «здесь».

Если хотите запустить программу из другой папки, можно указать путь:

```
$ /Users/serik/Desktop/cpp-from-scratch/demo-rpg/game
```

Или относительный путь:

```
$ ../demo-rpg/game
```

Если вы регулярно используете какую-то программу, её путь добавляют в PATH, и тогда можно вызывать просто по имени.

6.17 История команд

Шелл запоминает все команды, которые вы вводили. Чтобы вернуться к предыдущей — стрелка вверх. Ещё раз — позапрошлая. Стрелка вниз — обратно к новым.

Поиск по истории: `Ctrl+R`, потом начинаете печатать — шелл находит последнюю команду, начинающуюся с этого. Очень удобно.

История хранится в файле `~/.bash_history` или `~/.zsh_history`. Можно её отредактировать руками или экспортировать.

6.18 Автодополнение

Нажмите `Tab` — шелл попытается дополнить незаконченное слово. Например, набираете:

```
$ cd Desk<Tab>
```

И шелл сам допишет до `cd Desktop/`. Если вариантов несколько — `Tab` покажет их все. Это в десятки раз ускоряет работу — не приходится точно набирать длинные имена.

6.19 Очистить экран

Когда терминал засыпан выводом — `Ctrl+L` (или команда `clear`) очищает экран. Полезно, чтобы перевести дух.

6.20 Прервать команду

Запустили долгую команду и поняли, что не то? `Ctrl+C` — прервать (программа получает сигнал `SIGINT`).

`Ctrl+Z` — приостановить (программа замораживается). Чтобы продолжить в фоне — `bg`. Чтобы продолжить на переднем плане — `fg`. Эту функциональность мы будем плотно изучать в проекте `Mini-shell`.

6.21 Полезный набор команд

Сохраню сюда «карманный справочник», к которому буду отсылать в следующих главах:

Команда	Что делает
<code>pwd</code>	где я сейчас
<code>ls</code>	что в этой папке
<code>ls -la</code>	то же, с деталями и скрытыми файлами
<code>cd <папка></code>	перейти в папку
<code>cd ..</code>	подняться на уровень выше
<code>cd ~</code> или <code>cd</code>	в домашнюю папку
<code>mkdir <имя></code>	создать папку
<code>mkdir -p <путь></code>	создать все промежуточные
<code>touch <имя></code>	создать пустой файл
<code>cat <файл></code>	показать файл
<code>less <файл></code>	пролистать большой файл
<code>head <файл></code>	первые 10 строк
<code>tail <файл></code>	последние 10 строк
<code>tail -f <файл></code>	следить за новыми строками
<code>cp <откуда> <куда></code>	копировать файл
<code>cp -r <откуда> <куда></code>	копировать папку рекурсивно

Команда	Что делает
<code>mv <откуда> <куда></code>	переместить или переименовать
<code>rm <файл></code>	удалить файл
<code>rm -r <папка></code>	удалить папку с содержимым
<code>chmod +x <файл></code>	сделать исполняемым
<code>echo <текст></code>	напечатать в stdout
<code>echo \$VAR</code>	значение переменной
<code>export VAR=value</code>	установить переменную
<code>grep <текст> <файл></code>	найти строки, содержащие текст
<code>wc -l <файл></code>	сосчитать строки
<code>find <где> -name <маска></code>	поиск файлов
<code>man <команда></code>	справка по команде

Команда `man` — самая полезная. `man ls`, `man chmod`, `man find` — открывает встроенную справку. Выход — `q`. Туда можно лезть всегда, когда не помните флаги.

6.22 Маленькое упражнение

Откройте терминал и сделайте по шагам:

1. Перейдите в домашнюю папку (`cd ~`).
2. Создайте папку `cpp-practice` (`mkdir cpp-practice`).
3. Зайдите в неё (`cd cpp-practice`).
4. Создайте подпапки `src`, `include`, `build` одной командой (`mkdir src include build`).
5. Создайте пустой файл `src/main.cpp` (`touch src/main.cpp`).
6. Посмотрите структуру (`ls -la` или, если установлен, `tree`).
7. Запишите в `src/main.cpp` одно слово через `echo`: `echo "Hello" > src/main.cpp`.
8. Покажите содержимое: `cat src/main.cpp`.
9. Скопируйте файл: `cp src/main.cpp src/backup.cpp`.
10. Удалите бэкап: `rm src/backup.cpp`.
11. Поднимитесь на уровень выше: `cd ...`.
12. Удалите всю папку `cpp-practice` (только если уверены, что не нужно): `rm -r cpp-practice`.

Если всё прошло без ошибок — освоились. Если запутались — повторите спокойнее. После этой главы должно стать комфортно жить в терминале.

6.23 Что дальше

Часть 0а закончена. Теперь у вас есть фундамент: понимание устройства компьютера, битов и текста, программы и компиляции, алгоритмического мышления, командной строки. На этом фундаменте можно строить C++.

Глава 6 — установка компилятора (clang/gcc) и первая программа «Hello, world». Эта глава уже была написана в первой версии книги; вернётесь к ней и увидите: всё, что там написано про «команды терминала», «папки», «права», теперь укладывается в голову без вопросов. Вы готовы к C++.

Глава 7

Глава 6. Инструменты и первая программа

В этой главе мы поставим C++-компилятор, выберем редактор, напишем самую простую программу, соберём её и запустим. И главное — на каждом шаге я объясню, что именно происходит. Не «вот команда, запустите её», а «вот команда, и теперь покажу, что за этой командой стоит на самом деле».

7.1 Что нам нужно

Для C++-разработки нужны три вещи:

1. **Компилятор** — программа, которая превращает текст вашей программы (.cpp-файл) в исполняемый файл, который умеет запускать процессор.
2. **Текстовый редактор** — где вы пишете этот текст. Любой подойдёт; современные редакторы дополнительно подсказывают синтаксис и подсвечивают ошибки.
3. **Терминал** — программа, в которой вы пишете команды операционной системе. Команды вроде «скомпилируй мне вот этот файл» или «запусти вот этот исполняемый файл». Терминал есть в любой ОС, просто называется по-разному.

Иногда к этому добавляют **отладчик** (программа, которая позволяет остановить вашу программу в любой точке и посмотреть значения переменных) и **систему сборки** (программа, которая знает, какие файлы и в каком порядке компилировать в большом проекте). Их мы поставим, но не в этой главе.

Идём по платформам.

7.2 macOS

На современном Mac (любом, начиная с 2010-х) компилятор называется **clang**. Apple встроила его в систему — вернее, в инструмент Xcode Command Line Tools.

Откройте терминал (Cmd+Space → «Terminal» → Enter) и введите:

```
$ clang++ --version
```

Если в ответ вы видите что-то вроде `Apple clang version 14.0.3 (clang-1403.0.22.14.1)` — компилятор у вас уже есть, переходим к редактору.

Если же терминал предложит установить инструменты разработчика (появится окошко с большой кнопкой «Install») — нажмите её. Через несколько минут установка закончится, и команда `clang++ --version` начнёт работать. На современных Mac с Apple Silicon Xcode Command Line Tools идут как ~700 МБ — это нормально.

Полную Xcode IDE ставить **не нужно** — она огромная (десятки гигабайт), и для нашей книги избыточна. Хватит командной строки.

Альтернатива clang — **gcc** (про него поговорим в Linux-разделе). Поставить его на macOS можно через Homebrew (`brew install gcc`), но смысла особого нет: Apple clang делает всё то же самое. Я буду писать команды как с `g++`, так и с `clang++` — на macOS можете использовать любой, результат будет одинаков. По умолчанию даже команда `g++` на macOS вызывает clang (Apple подсунула символическую ссылку).

7.3 Linux

На Ubuntu/Debian и их производных:

```
$ sudo apt update
$ sudo apt install build-essential
```

Пакет `build-essential` содержит **gcc/g++** (GNU Compiler Collection — компилятор разрабатывается с 1987 года, считается «эталоном» в Linux-мире) плюс инструменты вроде `make` и `libc6-dev` (заголовки стандартной библиотеки). Проверка:

```
$ g++ --version
g++ (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
```

На Fedora/RHEL/CentOS:

```
$ sudo dnf install gcc-c++ make
```

На Arch:

```
$ sudo pacman -S base-devel
```

Если хотите clang — поставьте отдельно: `sudo apt install clang` или эквивалент. Иметь оба не вредно — на больших проектах удобно собирать обоими по очереди, потому что они выдают разные предупреждения, и баги ловятся лучше.

7.4 Windows

С Windows ситуация чуть хитрее, потому что нативный «виндовый» компилятор — Microsoft Visual C++ из Visual Studio — отличается от Linux-овского g++ многими мелочами (синтаксис командной строки, заголовочные файлы, флаги). Я в книге везде пишу команды для g++/clang++, потому что они одинаковые на macOS и Linux.

У вас три варианта.

Вариант А — WSL2 (Windows Subsystem for Linux), рекомендую. Microsoft встроила в Windows 10/11 полноценный Linux. Откройте PowerShell от администратора и наберите:

```
> wsl --install
```

Перезагрузитесь. После перезагрузки в меню «Пуск» появится Ubuntu — запустите, придумайте имя пользователя и пароль, и у вас теперь Linux внутри Windows. Далее следуйте инструкциям из секции «Linux» (`sudo apt install build-essential`). Все команды из книги будут работать «как на Linux».

Вариант В — MSYS2. Это среда, которая ставит на Windows полноценный GNU-toolchain (g++, make, всё как в Linux), но без виртуализации. Скачать: <https://www.msys2.org/>. После установки откройте «MSYS2 MINGW64» и наберите:

```
$ pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-clang
```

Далее команда g++ будет работать. Минус: пути с обратными слешами и другие мелочи Windows иногда вылезают.

Вариант С — Visual Studio Community. Полная IDE от Microsoft, бесплатная для индивидуальной разработки. При установке выберите workload «Desktop development with C++». После этого в меню появится «Developer Command Prompt» — это терминал, где работает команда cl (это виндовый компилятор; синтаксис другой, но первые программы соберёт). Подойдёт, но мои команды надо будет адаптировать.

Я в книге дальше пишу команды для g++/clang++. Если вы на Windows и выбрали вариант С — открывайте дополнительную вкладку с документацией Microsoft по cl.exe и сопоставляйте флаги. Если выбрали А или В — у вас всё будет совпадать с macOS/Linux.

7.5 Редактор

C++ можно писать в Notepad. Я серьёзно — буква в букву это совпадает с тем, как писали в 90-х. Но мы живём в 2026, и есть выбор получше.

VS Code — бесплатный, лёгкий, отлично работает с C++ через расширение «C/C++» от Microsoft и/или «clangd» (более современная альтернатива). Установка: <https://code.visualstudio.com/>, потом в боковой панели Extensions поставьте «C/C++» или «clangd». На macOS/Linux/Windows работает одинаково.

Sublime Text или **Vim/Neovim** — для тех, кто предпочитает классику. Vim требует настройки (plugin clangd), но после неё — летает.

CLion от JetBrains — полноценная IDE с самой умной автодополнялкой и встроенным отладчиком, но платная (студенческая лицензия бесплатно). Если вы любите среду JetBrains (PHPStorm, IntelliJ, PyCharm) — поставьте, разница только в языке.

Я в книге исхожу из того, что у вас VS Code. Это самый распространённый вариант в 2026 году, бесплатно, и достаточно мощно. Все скриншоты — оттуда. Если у вас другой редактор — большой разницы нет, в книге почти не будет «нажмите Ctrl+Shift+P».

7.6 Структура проекта

Создайте где-нибудь на диске папку для нашей книги. Я кладу свои учебные проекты в ~/code/cpp-book/ на macOS (~ — это домашняя папка пользователя). Откройте терминал и создайте папку:

```
$ mkdir -p ~/code/cpp-book/ch01
$ cd ~/code/cpp-book/ch01
```

Команда `mkdir` (make directory) создаёт папку. Флаг `-p` означает «и все промежуточные тоже, если их нет». Команда `cd` (change directory) переходит в эту папку.

Откройте эту папку в VS Code:

```
$ code .
```

Точка `.` означает «текущая папка». Если команды `code` нет — в VS Code откройте Command Palette (Cmd+Shift+P на mac, Ctrl+Shift+P на остальных) и выполните «Shell Command: Install 'code' command in PATH».

7.7 Первая программа — `hello.cpp`

В открытом VS Code создайте файл `hello.cpp` (правой кнопкой по дереву файлов слева → New File). Расширение `.cpp` — стандартное для исходников C++. Бывают ещё `.cxx`, `.cc`, `.C` — это всё C++; конкретно `.cpp` — самое распространённое.

Напишите туда:

```
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Шесть строк (с пустой между ними). Сейчас разберём каждый символ. Серьёзно — каждый.

7.7.1 Строка 1: `#include <iostream>`

Это директива препроцессора. **Препроцессор** — это первая стадия компиляции; он читает исходник, обрабатывает строчки, начинающиеся с `#`, и подменяет их на что-то другое. В нашем случае — берёт содержимое файла `iostream` и вставляет его прямо сюда, где написано `#include`.

Файл `iostream` — это часть стандартной библиотеки C++. Внутри него — описания того, как работать с вводом и выводом (input/output stream). Конкретно нам сейчас нужно `std::cout` — объект, который представляет «стандартный вывод» (то, куда пишет ваша программа: обычно это терминал). Объект `std::cout` объявлен где-то внутри `<iostream>`, поэтому без `#include <iostream>` компилятор не будет знать, что такое `std::cout`, и выдаст ошибку.

Угловые скобки `<...>` говорят: ищи этот файл в системных папках стандартной библиотеки (на macOS это что-то вроде `/Library/Developer/CommandLineTools/usr/include/c++/v1/`, а на Linux — `/usr/include/c++/11/`). Если бы мы написали `#include "iostream"` в кавычках — препроцессор сначала смотрел бы в папке нашего исходника, а только потом в системных. Это правило используется, когда вы подключаете *свои* заголовочные файлы (не стандартные).

`iostream` без расширения — это особенность стандартной библиотеки C++. Когда-то (давно) подключали `<iostream.h>`. В современном C++ — без `.h`. Под капотом файл всё равно есть, просто называется чуть иначе и лежит в специальной папке.

Заметьте: в конце строки **нет точки с запятой**. Препроцессорные директивы — особый случай; они обрабатываются ещё до того, как начинает работать собственно компилятор. Точка с запятой им не нужна, и если её поставить — будет ошибка.

7.7.2 Строка 2: пустая

Просто разделитель. Компилятор её игнорирует. Я люблю отделять блок `#include` от кода пустой строкой — это визуально структурирует файл. Можно не делать.

7.7.3 Строка 3: `int main() {`

Это начало функции `main`. Функция `main` — особенная: это точка входа в программу. Когда вы запускаете исполняемый файл, операционная система ищет в нём функцию с именем `main` и вызывает её. Если такой функции нет — программа не соберётся.

Разберём по частям.

`int` — это **тип возвращаемого значения**. Функция `main` возвращает целое число (`int = integer`). Это число — код возврата программы. По умолчанию `0` означает «всё прошло хорошо», ненулевое значение — «что-то сломалось». Этим кодом возврата пользуются shell-скрипты: они проверяют, успешно ли отработала ваша программа, чтобы решить, продолжать или нет.

`main` — имя функции. Это **зарезервированное имя** в C++ — нельзя сделать `main` чем-то другим (например, переменной).

`()` — пустые круглые скобки. Это **список параметров функции**. Сейчас он пуст; на самом деле `main` в C++ может принимать аргументы командной строки (`int main(int argc, char* argv[])`), но мы пока без них.

`{` — открывающая фигурная скобка. Это начало **тела функции**. Всё, что идёт от `{` до соответствующей `}`, выполняется при вызове функции. В C++ блоки кода всегда выделяются фигурными скобками — никаких отступов как в Python.

7.7.4 Строка 4: `std::cout << "Hello, world!" << std::endl;`

Это **выражение-инструкция** — основной строительный кирпичик любой C++ программы. Разбираем по кускам.

Четыре пробела в начале строки. Это отступ. Чисто визуально — чтобы было видно, что строка принадлежит телу функции `main`. Компилятор отступы игнорирует; они для человека. В команде обычно договариваются: 4 пробела или 2 пробела или `tab`. В книге я использую 4 пробела. Большинство C++-стандартов в отрасли — тоже 4 пробела (Google C++ Style Guide — 2 пробела, но в нашей книге держимся 4).

`std::cout` — обращение к объекту `cout` из пространства имён `std`. **Пространство имён** (namespace) — это что-то вроде «папки» для имён, чтобы две библиотеки могли использовать одинаковые названия и не конфликтовать. Вся стандартная библиотека C++ живёт в

пространстве `std`. Чтобы добраться до объекта внутри, пишут `имя_пространства::имя_объекта`. Двоеточия — две штуки подряд (`::`) — называются *оператором разрешения области видимости*.

Объект `cout` — это «стандартный поток вывода». Он наследник класса `std::ostream`. Технически это объект, у которого есть состояние (например, текущая позиция, флаги форматирования), и который умеет принимать данные и отправлять их в терминал.

Оператор `<<` — это **оператор вставки в поток**. Читается так: «отправь в `cout` то, что справа». То есть `std::cout << "Hello, world!"` означает «отправь строку "Hello, world!" в стандартный вывод». Само сочетание `<<` для битового сдвига влево используется (это тот же символ), но в контексте `std::cout` оно перегружено как «вставить в поток». Это перегрузка операторов — мощная вещь C++, доберёмся до неё в главе 11.

"Hello, world!" — это **строковый литерал**. Кавычки двойные. Тип у него — `const char[14]` (массив из 14 символов: 13 видимых + один невидимый «нулевой символ» `\0`, который маркирует конец строки в стиле C). Дальше эта штука неявно сконвертируется в `const char*` (указатель на первый символ массива), и `<<` этим займётся.

Внутри строкового литерала можно использовать **escape-последовательности**: `\n` — перевод строки, `\t` — табуляция, `\"` — кавычка внутри строки, `\\` — обратный слеш. Мы их пока не используем.

Второй `<<` и `std::endl` — снова оператор вставки, но теперь мы вставляем не строку, а специальный объект `std::endl`. Это «end line» — он делает две вещи: вставляет символ перевода строки (`'\n'`) и сбрасывает буфер вывода (то есть форсирует, чтобы текст реально появился в терминале, а не остался лежать в буфере).

В большинстве случаев вместо `std::endl` можно писать `"\n"` — это быстрее, потому что не сбрасывает буфер. Но для нашей первой программы `std::endl` нагляднее.

Точка с запятой ; в конце — обязательна. В C++ почти каждая инструкция оканчивается точкой с запятой. Если её забыть, компилятор выдаст ошибку. Привыкайте сразу.

Итого: эта строка отправляет в стандартный вывод текст «Hello, world!» и перевод строки.

7.7.5 Строка 5: `return 0;`

Возврат из функции `main` со значением 0. Как мы говорили, `main` возвращает целое число — код возврата программы. 0 — «успех». `return` — ключевое слово, которое останавливает выполнение функции и отдаёт значение наружу. Точка с запятой в конце — снова обязательна.

В C++ для функции `main` есть особое правило: если вы забудете `return 0;`, компилятор автоматически добавит его сам. То есть для `main` `return 0;` опционален. Но я всегда пишу явно — так понятнее.

7.7.6 Строка 6: }

Закрывающая фигурная скобка. Конец тела функции `main`. На этом файл и заканчивается.

7.8 Компиляция

Сохранили файл (`Cmd+S` на `mac`, `Ctrl+S` на остальных). Теперь в терминале:

```
$ g++ -std=c++11 -Wall -Wextra hello.cpp -o hello
```

Если у вас вместо `g++` есть только `clang++` — пишите `clang++`, всё то же самое:

```
$ clang++ -std=c++11 -Wall -Wextra hello.cpp -o hello
```

Разбираем по флагам.

g++ или **clang++** — собственно компилятор. Можно ещё `c++` (это символическая ссылка, обычно на системный C++-компилятор).

-std=c++11 — какой стандарт языка использовать. Без этого флага компилятор может взять стандарт по умолчанию (у `gcc` 11+ это C++17, у `clang` старше — C++98). Мы фиксируем `-std=c++11`, чтобы код собирался у всех читателей одинаково. Иногда увидите `-std=c++14`, `-std=c++17`, `-std=c++20`, `-std=c++23` — это более новые стандарты.

-Wall — включить большую часть стандартных предупреждений. `w` = warning, `all` — все. Предупреждения это не ошибки — компилятор просто говорит: «ты тут что-то странное написал, может быть, опечатка?» Хорошая практика — всегда собирать с `-Wall`.

-Wextra — включить ещё больше предупреждений, которые не вошли в `-Wall`. Тоже очень полезно.

-pedantic — иногда добавляю. Это «педантичный» режим: компилятор требует строгого соответствия выбранному стандарту, без расширений GNU/clang. Для книги полезно: если код собирается с `-pedantic`, значит, он стандартный.

hello.cpp — имя файла с исходным кодом. Можно несколько: `g++ a.cpp b.cpp c.cpp`.

-o hello — имя выходного файла. `-o` от output. Без этого флага компилятор по умолчанию назовёт результат `a.out` (наследие 70-х, когда так называли исполняемые файлы на Unix). Я предпочитаю явно говорить, как назвать.

Если всё хорошо — никакого вывода не будет. Это normal: «no news is good news» в Unix-философии. Зато в папке появится файл `hello` (или `hello.exe` на Windows). Проверим:

```
$ ls -lh hello
-rwxr-xr-x  1 user  staff   33K hello
```

Файл размером ~33 KB. Видите x в правах доступа (rwxr-xr-x) — это «executable», его можно запустить.

7.9 Запуск

```
$ ./hello
Hello, world!
```

Точка-слеш в начале (./) — это **обязательно**. В Unix-системах, когда вы пишете hello без ./, shell ищет программу с таким именем в специальных папках (/usr/bin, /bin, /usr/local/bin, и так далее — это переменная \$PATH). Текущую папку он *не* проверяет — из соображений безопасности (чтобы вы случайно не запустили вредоносный ls в каком-нибудь скачанном архиве, подменивший системный). Чтобы явно сказать «запусти программу из текущей папки», используют ./ — точка означает текущую папку, слеш — разделитель.

Программа выполнилась, вывела «Hello, world!», и закончилась. Если хотите увидеть код возврата:

```
$ echo $?
0
```

Команда echo \$? показывает код возврата последней запущенной программы. У нас 0, потому что мы написали return 0;. Поменяйте на return 42;, пересоберите, запустите, проверьте — увидите 42.

7.10 Что произошло — большая картинка

Между моментом, когда вы написали g++ hello.cpp -o hello, и моментом, когда появился исполняемый файл, прошло четыре стадии компиляции. Я расскажу о них коротко сейчас, а детально — в следующей главе.

Стадия 1: препроцессор. Компилятор берёт hello.cpp и обрабатывает все #include. То есть он буквально берёт содержимое файла iostream (а это десятки тысяч строк, потому что внутри ещё много #include) и подставляет его в место #include <iostream>. На выходе — один огромный «развёрнутый» исходник. Хотите увидеть? g++ -E hello.cpp (E от «preprocess»). Не пугайтесь, вылетит много текста.

Стадия 2: компилятор. Из «развёрнутого» исходника генерируется *ассемблерный код* — текст на низкоуровневом языке процессора. Хотите увидеть? g++ -S hello.cpp -o hello.s. Потом откройте hello.s — увидите инструкции вроде movq, callq, retq. Это уже почти машинный код, только в текстовом виде.

Стадия 3: ассемблер. Из ассемблерного текста делается *объектный файл* — бинарный файл, содержащий машинный код, но ещё не «соединённый» с другим кодом. `g++ -c hello.cpp -o hello.o`. Файл `hello.o` — это машинный код вашей функции `main`, но с «дырками» там, где она ссылается на чужие функции (например, на `std::cout::operator<<`).

Стадия 4: линкер. Линкер берёт все объектные файлы и подставляет в «дырки» реальные адреса. Например, наша программа ссылается на `std::cout`, который реализован в стандартной библиотеке (на macOS — `libc++`, на Linux — `libstdc++`). Линкер находит библиотеку, привязывает её, и получается готовый исполняемый файл.

Когда вы пишете `g++ hello.cpp -o hello` без флагов `-E`, `-S`, `-c` — компилятор молча проходит все четыре стадии и выдаёт готовый файл. Под капотом это четыре отдельных программы (`cpp`, `cc1plus`, `as`, `ld`), и `g++` их вызывает по очереди. Можете в этом убедиться: `g++ -v hello.cpp -o hello` покажет всё, что происходит, с подробными командами.

Глава 2 будет именно про это: компиляция по шагам, объектные файлы, символы, линковка. Сейчас же запомните: исходник `.cpp` не запускается напрямую. Он сначала превращается в исполняемый файл, и уже исполняемый файл запускает процессор.

7.11 Что если ошибка

Сломаем нашу программу нарочно — уберите точку с запятой в конце пятой строки:

```
    return 0
}
```

Соберите:

```
$ g++ -std=c++11 hello.cpp -o hello
hello.cpp:6:1: error: expected ';' before '}' token
    6 | }
      | ^
```

Компилятор честно говорит: «На строке 6 в начале я ждал точку с запятой перед `}`, но не нашёл». Сообщения об ошибках в современном `gcc/clang` очень подробные — внимательно читайте их, не пугайтесь объёма. Обычно ошибка ровно там, где компилятор её показал.

Несколько типичных ошибок, с которыми вы столкнётесь в первые дни:

error: 'cout' was not declared in this scope — забыли `#include <iostream>` или забыли `std::` перед `cout`.

undefined reference to 'main' — забыли написать функцию `main` (или назвали её по-другому).

expected ';' before ... — где-то выше потеряли точку с запятой.

'std' has not been declared — старый компилятор, который не понимает `std::`. Поставьте новый.

no such file or directory: 'iostream' — у вас не установлена стандартная библиотека C++ (бывает на минималистичных Linux-сборках). Поставьте `libstdc++-dev` или эквивалент.

7.12 Запоминания на сегодня

К следующей главе хорошо бы выучить наизусть:

- Каждый исполняемый файл — это результат компиляции одного или нескольких `.cpp` файлов плюс линковки с библиотеками.
- Функция `main` — точка входа в программу. Возвращает `int`.
- Каждый блок кода обрамляется фигурными скобками `{...}`.
- Каждая инструкция оканчивается точкой с запятой `;`.
- `#include <...>` подключает заголовочный файл.
- Пространство имён `std` — это вся стандартная библиотека. Доступ через `std::имя_объекта`.
- `std::cout << ...` — вывод. `<<` — оператор вставки в поток.
- `std::endl` — перевод строки + сброс буфера.
- Команда сборки: `g++ -std=c++11 -Wall -Wextra файл.cpp -o имя`.
- Запуск программы в текущей папке: `./имя`.

7.13 Маленькое упражнение

Чтобы рука запомнила синтаксис, напишите программу, которая выводит:

Меня зовут Ваше Имя.

Я учу C++.

Это моя первая программа.

Можете использовать `std::endl` или `"\n"` для переводов строки — попробуйте оба варианта. Откройте файл `intro.cpp`, напишите код, соберите, запустите. Если соберётся с первого раза — это очень хороший знак. Если нет — внимательно прочтите ошибку и попробуйте найти, где забыли точку с запятой или скобку. Это первый и самый частый источник ошибок в C++.

7.14 Что дальше

Глава 2 — про то, как программа превращается в исполняемый файл. Мы пройдемся по всем четырём стадиям компиляции, посмотрим, что лежит внутри объектного файла, что такое символы и почему «undefined reference» — это самая частая ошибка C++-новичка. Это не

«теория, которую можно пропустить» — без этого знания невозможно разобраться, когда программа из десятка файлов перестаёт собираться. Поэтому потратим на это целую главу.

Глава 8

Глава 7. Как программа становится исполняемым файлом

В главе 6 мы написали `hello.cpp`, скомпилировали и запустили. И вскользь упомянули, что компиляция идёт в четыре стадии: препроцессинг → компиляция → ассемблирование → линковка. В этой главе разбираем каждую стадию подробно. Что именно происходит, какие файлы появляются на каждом шаге, как выглядят их «потроха», что такое объектный файл, что такое символ, в чём разница между статической и динамической линковкой, и как программа лежит в памяти процесса.

Эта тема — фундамент, на котором всё дальнейшее построено. Без понимания, что компиляция многоступенчатая, и что «ошибка компиляции» отличается от «ошибки линковки», вы будете тонуть в сообщениях `gcc/clang` ближе к проектам. Поэтому глава длинная, но я постараюсь обойтись без лишней теории — будем смотреть на реальные файлы и команды.

8.1 Подопытная программа

Чтобы было что разбирать, заведём чуть более интересный пример, чем простой `hello`. Сделаем программу из двух файлов: главного `main.cpp` и вспомогательного `math.cpp` с функцией сложения.

Создайте папку и три файла.

`math.h`:

```
#ifndef MATH_H
#define MATH_H
```

```
int add(int a, int b);
```

```
#endif
```

math.cpp:

```
#include "math.h"
```

```
int add(int a, int b) {  
    return a + b;  
}
```

main.cpp:

```
#include <iostream>
```

```
#include "math.h"
```

```
int main() {  
    int x = add(2, 3);  
    std::cout << "2 + 3 = " << x << std::endl;  
    return 0;  
}
```

Три файла. Я их буду использовать на протяжении всей главы. Кратко скажу, что тут происходит — детали разберём по ходу.

В `math.h` лежит **объявление** функции `add` — обещание, что где-то есть функция с таким именем, принимающая два `int` и возвращающая `int`. Сам код функции тут не написан.

`#ifndef MATH_H ... #define MATH_H ... #endif` — это **защита от двойного включения** (include guard). Если этот заголовочный файл случайно попадёт в перевод дважды (например, его подключают два других файла, которые в свою очередь подключаются обоим), компилятор увидит, что `MATH_H` уже определён, и пропустит содержимое второй раз. Без этой защиты получим ошибку «redefinition of add». Подробнее про препроцессор — ниже.

В `math.cpp` лежит **определение** функции — собственно её код. Тут она `return a + b`.

В `main.cpp` функция `add` используется. Чтобы компилятор знал, что такая функция существует и что она принимает/возвращает, в начале файла подключаем `math.h` (через `#include "math.h"`). А определение функции живёт в `math.cpp` и попадёт в программу через линковку.

Это классическое разделение в C/C++: **заголовочный файл (.h)** — объявления, чтобы все знали, что есть такая функция; **исходный файл (.cpp)** — определения, реальные тела функций.

Соберём программу одной командой и убедимся, что работает:

```
$ g++ -std=c++11 main.cpp math.cpp -o program
$ ./program
2 + 3 = 5
```

Прекрасно. Теперь — препарируем процесс.

8.2 Стадия 1: препроцессор

Препроцессор — это первая стадия компиляции. Программа, отвечающая за неё, исторически называется `cpp` (C Pre-Processor; не путать с C++!). Что она делает?

Препроцессор работает с текстом. Он не понимает C++. Он не знает, что такое функция, что такое класс, что такое тип `int`. Для него файл `main.cpp` — это просто последовательность символов. Он ищет строчки, начинающиеся с `#`, и применяет к ним свои правила.

Самое частое — `#include`. Когда препроцессор видит `#include <iostream>`, он буквально берёт содержимое файла `iostream` из стандартной библиотеки и **вставляет его прямо в это место**. Текстуально, без анализа.

Также препроцессор обрабатывает: - `#define X Y` — макросы. После этой строки везде в файле `X` будет заменяться на `Y`. - `#ifdef`, `#ifndef`, `#endif` — условная компиляция. Куски кода включаются или исключаются в зависимости от того, определён ли тот или иной макрос. - `#pragma` — указания компилятору (нестандартные расширения). - Комментарии (хоть и не через `#`) — препроцессор их удаляет.

После работы препроцессора на выходе — один большой «развёрнутый» файл, в котором все `#include` подставлены, макросы развёрнуты, комментарии удалены.

8.2.1 Посмотрим своими глазами

GCC и Clang позволяют остановиться после препроцессора, флаг `-E`:

```
$ g++ -std=c++11 -E main.cpp -o main.i
$ wc -l main.i
40826 main.i
```

40 тысяч строк! Из наших 9 строк! Что произошло?

Откройте `main.i` любым редактором. Сверху увидите содержимое `iostream`, потом всех других файлов, которые `iostream` тянет за собой (а тянет он много — около сорока тысяч строк). А внизу, среди этой простыни, найдёте знакомое:

```
int add(int a, int b);

int main() {
    int x = add(2, 3);
    std::cout << "2 + 3 = " << x << std::endl;
    return 0;
}
```

Это и есть наш код. Заметили: `#include <iostream>` исчез — на его месте теперь буквально содержимое `iostream`. `#include "math.h"` тоже исчез — на его месте `int add(int a, int b);`.

8.2.2 Макросы

Препроцессор может ещё определять и подставлять **макросы**. Пример:

main.cpp:

```
#include <iostream>

#define PI 3.14159
#define SQUARE(x) ((x) * (x))

int main() {
    std::cout << "Pi = " << PI << std::endl;
    std::cout << "5 squared = " << SQUARE(5) << std::endl;
    return 0;
}
```

После препроцессора код превращается в:

```
int main() {
    std::cout << "Pi = " << 3.14159 << std::endl;
    std::cout << "5 squared = " << ((5) * (5)) << std::endl;
    return 0;
}
```

Каждое упоминание `PI` буквально заменилось на `3.14159`. Каждое упоминание `SQUARE(x)` развернулось в выражение `((x) * (x))`.

В C макросы были основным способом «не повторяться». В C++ макросы остаются, но их использование сократилось — вместо них есть `constexpr`-константы, `inline`-функции, шаблоны. Они почти всегда лучше макросов, потому что:

- Макрос — это текстовая замена; в нём нет понятия «тип». Если написать `SQUARE("hello")` — препроцессор просто подставит и оставит компилятору разбираться. Компилятор скажет «нельзя умножить строку на строку», но указание ошибки будет на сгенерированной строке, а не на той, где вы написали `SQUARE`.
- Макросы не видны отладчику.
- Макросы не уважают границы пространств имён.

Тем не менее макросы остаются: `include guard` (`#ifndef MATH_H` в нашем `math.h`) — это макрос. Условная компиляция (`#ifdef __APPLE__` — «выполнять этот блок только на macOS») — макросы. Платформенно-зависимый код часто полагается на макросы.

8.2.3 Условная компиляция

Допустим, мы хотим, чтобы наш код по-разному работал на Linux и на macOS:

```
#include <iostream>

int main() {
#ifdef __APPLE__
    std::cout << "Running on macOS\n";
#elif defined(__linux__)
    std::cout << "Running on Linux\n";
#elif defined(_WIN32)
    std::cout << "Running on Windows\n";
#else
    std::cout << "Running on something else\n";
#endif
    return 0;
}
```

Эти `__APPLE__`, `__linux__`, `_WIN32` — макросы, которые компилятор сам определяет в зависимости от того, на какой системе он работает. Препроцессор смотрит, что определено, и оставляет в коде только подходящий блок.

После препроцессора на macOS у вас в файле будет:

```
int main() {
    std::cout << "Running on macOS\n";
    return 0;
}
```

Никаких других веток нет — препроцессор их выкинул. Это **условная компиляция**. На больших проектах ей пользуются часто, чтобы единый код собирался под разные платформы.

8.2.4 Include guard детальнее

Раз про `#ifndef MATH_H` уже несколько раз говорили, разберём окончательно.

Допустим, у нас три файла:

a.h:

```
struct Point { int x; int y; };
```

b.h:

```
#include "a.h"
void draw(Point p);
```

main.cpp:

```
#include "a.h"
#include "b.h"

int main() {
    Point p{1, 2};
    draw(p);
    return 0;
}
```

После препроцессора main.cpp превратится в:

```
struct Point { int x; int y; }; // из a.h, через прямой include
struct Point { int x; int y; }; // из a.h, через b.h
void draw(Point p);           // из b.h
                               // (main.cpp дальше)

int main() {
    Point p{1, 2};
    draw(p);
    return 0;
}
```

Point объявлен дважды! Компилятор ругнётся: «redefinition of struct Point». Это и есть проблема двойного включения.

Решается стандартной защитой:

a.h:

```
#ifndef A_H
#define A_H

struct Point { int x; int y; };

#endif
```

Теперь, когда a.h подключается первый раз, A_H не определён, препроцессор заходит внутрь, определяет A_H и оставляет объявление Point. Когда a.h подключается второй раз (через b.h), A_H уже определён — препроцессор пропускает весь блок до #endif. Point объявлен ровно один раз.

В современном коде защиту иногда заменяют на #pragma once:

```
#pragma once

struct Point { int x; int y; };
```

Это нестандартное расширение, но его поддерживают все основные компиляторы. Удобнее (одна строка вместо трёх), но менее портabelно.

8.2.5 Что выучить про препроцессор

- Препроцессор работает с текстом, до компилятора.
- #include <header> — буквально подставляет содержимое.
- #include "header.h" — то же, но в первую очередь ищет в папке исходника.
- #define создаёт макрос. Старайтесь избегать макросов; для констант используйте const-expr, для функций — inline или шаблоны.
- Каждый .h-файл должен иметь защиту от двойного включения (#ifndef/#define/#endif или #pragma once).
- Платформенно-зависимый код пишут через #ifdef.

8.3 Стадия 2: собственно компиляция

После препроцессора мы получили развёрнутый текст. Теперь **компилятор** (в узком смысле — программа cscpp внутри g++) превращает его в **ассемблерный код** для целевой архитектуры.

Здесь происходит самое интересное:

1. **Лексический анализ.** Текст разбивается на «токены»: int, main, (,), {, и так далее.

2. **Синтаксический анализ.** Из токенов строится **AST** (abstract syntax tree) — дерево, отражающее структуру программы. Узел «функция main» с дочерним узлом «тело», в теле — узел «вызов функции add», и так далее.
3. **Семантический анализ.** Компилятор проверяет, что всё имеет смысл: типы совместимы, имена существуют, функции вызываются с правильным числом аргументов.
4. **Оптимизация.** Компилятор смотрит на код и переписывает его так, чтобы он делал то же самое, но эффективнее. Например, `int x = 2 + 3`; он сразу заменит на `int x = 5`; — это называется **constant folding**.
5. **Генерация кода.** AST превращается в ассемблерные инструкции под нужную архитектуру.

Если на любой из стадий компилятор найдёт проблему — это **ошибка компиляции**. Например, «add is not declared in this scope» — на стадии семантического анализа.

Остановимся после компилятора:

```
$ g++ -std=c++11 -S main.cpp -o main.s
$ wc -l main.s
80 main.s
```

Откройте `main.s` — это ассемблер для вашей архитектуры. На Mac M3 (ARM64) выглядит примерно так:

```
_main:
    sub sp, sp, #48
    stp x29, x30, [sp, #32]
    add x29, sp, #32
    mov w8, #2
    str w8, [sp, #8]
    mov w8, #3
    str w8, [sp, #4]
    ldr w0, [sp, #8]
    ldr w1, [sp, #4]
    bl _Z3addii
    ...
```

Под x86-64 (Intel/AMD) другой синтаксис, но смысл тот же. Каждая строка — одна инструкция.

Что тут особенно интересно: строчка `bl _Z3addii`. `bl` — это «branch with link», то есть вызов функции на ARM64. А `_Z3addii` — это имя функции `add`, преобразованное компилятором.

«Зачем имя так искорёжено?», — справедливый вопрос. Это называется **name mangling**.

8.3.1 Манглинг имён

В C функции называются буквально так, как вы их написали: `add` — это `add`. Это удобно, но непригодно для C++, где функции могут быть **перегружены**:

```
int add(int a, int b);
double add(double a, double b);
int add(int a, int b, int c);
```

Три разные функции, все называются `add`. Как линкер их отличит? Никак, если имена одинаковые.

Поэтому компилятор C++ «кодирует» в имя функции типы её параметров. На GCC/Clang используется схема Itanium ABI: `_Z3addii` = функция `add` с двумя `int` (`i` = `int`). `_Z3adddd` = функция `add` с двумя `double` (`d` = `double`). `_Z3addiii` = функция `add` с тремя `int`.

Линкер видит эти разные имена и спокойно различает.

На MSVC (Microsoft) используется другая схема манглинга, со своими правилами.

Чтобы посмотреть «человеческое» имя из «искорёженного», есть утилита `c++filt`:

```
$ echo "_Z3addii" | c++filt
add(int, int)
```

Очень полезно, когда вы видите в сообщении об ошибке «`undefined reference to _Z6lambda...`».

Иногда нужно отключить манглинг — например, при экспорте функций для C-кода. Для этого пишут `extern "C"`:

```
extern "C" int my_function(int x);
```

В этом случае компилятор имени `my_function` не трогает — линкер видит её как `my_function`. Это используется при создании библиотек, чтобы их могли использовать программы на других языках.

8.3.2 Что выучить про компиляцию

- Компилятор разбирает текст, проверяет смысл, и генерирует ассемблер.
- Ошибки компиляции — несовместимые типы, неизвестные имена, синтаксические ошибки.
- C++-имена кодируются (`mangle`) для поддержки перегрузки.
- Узнать «реальное» имя — через `c++filt`.

8.4 Стадия 3: ассемблирование

После компилятора у нас текстовый ассемблерный файл. Теперь его надо превратить в **машинный код**. Этим занимается **ассемблер** (программа `as`).

Ассемблер — простая программа. Она берёт ассемблерный текст и для каждой инструкции находит её бинарный опкод. На выходе — **объектный файл**.

```
$ g++ -std=c++11 -c main.cpp -o main.o
$ file main.o
main.o: Mach-O 64-bit object x86_64
# или, на ARM64 Mac:
main.o: Mach-O 64-bit object arm64
# на Linux:
main.o: ELF 64-bit LSB relocatable, x86-64, ...
```

Это уже бинарный файл. Прочитать как текст не получится — увидите кракозябры. Но есть утилиты для просмотра.

8.4.1 Внутри объектного файла

Объектный файл — это **не** просто массив машинных инструкций. Это структурированный формат, в котором есть:

1. **Секция кода (.text)** — собственно машинный код функций.
2. **Секция данных (.data)** — глобальные и статические переменные с начальным значением.
3. **Секция BSS (.bss)** — глобальные и статические переменные без начального значения (обнуляются автоматически при загрузке).
4. **Секция read-only (.rodata)** — константы, строковые литералы.
5. **Таблица символов** — список всех функций и переменных, которые объявлены в этом файле.
6. **Таблица релокаций** — список «дырок», где нужно подставить адреса других функций/переменных при линковке.

На разных платформах форматы разные: - **Linux** — ELF (Executable and Linkable Format). - **macOS** — Mach-O. - **Windows** — COFF/PE (Portable Executable).

Все они идейно похожи: секции, символы, релокации.

8.4.2 Смотрим на символы

Утилита `nm` показывает символы внутри объектного файла:

```
$ nm main.o
                 U __ZSt4cout
                 U __ZSt4endl
                 U __Z3addii
0000000000000000 T _main
                 U __ZNSolsEi
                 U __ZNSolsEPKc
```

Буква перед именем (U, T, и так далее) — это тип символа: - T (text) — символ определён в этом файле, в секции кода. У нас `_main` — определён. - U (undefined) — символ нужен, но не определён здесь. Например, `__Z3addii` — это `add(int, int)`, и наш `main.o` ждёт, что она будет где-то определена. - D (data) — определён в секции данных (глобальная переменная с инициализацией). - B (bss) — определён в секции BSS (глобальная переменная без инициализации). - R (rodata) — определён в read-only секции (константа).

Теперь сделаем `math.o`:

```
$ g++ -std=c++11 -c math.cpp -o math.o
$ nm math.o
0000000000000000 T __Z3addii
```

Видим: `__Z3addii` (то есть `add(int, int)`) **определён** в `math.o` (T). А в `main.o` он был **не определён** (U).

Это и есть основа линковки. У одного объектного файла есть «дырка» — символ, нужный, но не определённый. У другого этот же символ есть в готовом виде. Линкер их соединит.

8.4.3 Утилита `objdump`

`objdump` показывает содержимое объектного файла подробнее. Например:

```
$ objdump -d main.o | head -30
main.o: file format mach-o-x86-64

Disassembly of section __TEXT,__text:

_main:
    0:  push    %rbp
    1:  mov     %rsp, %rbp
    4:  sub     $0x20, %rsp
    8:  movl    $0x0, -0x4(%rbp)
    f:  movl    $0x2, %edi
```

```
14:  movl    $0x3, %esi
19:  callq   0x1e <_main+0x1e>
1e:  ...
```

Это уже почти то же самое, что в `.s`-файле, только с числовыми смещениями. `callq 0x1e` — это вызов функции по смещению `0x1e`. Сейчас оно ссылается само на себя (компилятор не знал куда указать). Линкер потом подставит правильный адрес.

8.5 Стадия 4: линковка

Самая интересная стадия. У нас есть два объектных файла: `main.o` и `math.o`. Линкер их соединит, разрешит все «дырки», и получит финальный исполняемый файл.

```
$ g++ main.o math.o -o program
$ ./program
2 + 3 = 5
```

Или, что то же самое, одной командой:

```
$ g++ main.cpp math.cpp -o program
```

Под капотом происходит:

1. Линкер читает все объектные файлы.
2. Собирает таблицу всех символов: какие определены где.
3. Для каждого символа `U` в каком-то файле ищет, где этот символ `T` (определён) — в других объектных файлах или в библиотеках.
4. Соединяет секции: все `.text` склеиваются в одну, все `.data` — в одну, и так далее.
5. Корректирует все ссылки: где была «дырка» с именем — подставляет реальный адрес.
6. Получается готовый исполняемый файл.

8.5.1 А стандартная библиотека?

Замечу: в `main.o` были «undefined» символы вроде `__ZSt4cout` (это `std::cout`). Откуда они придут?

Из **стандартной библиотеки**. Когда вы вызываете `g++`, он автоматически добавляет в линковку стандартную библиотеку C++ (`libstdc++` на Linux, `libc++` на macOS). Туда зашит код `std::cout`, `std::endl`, всех контейнеров, и так далее.

Если бы вы делали линковку вручную через `ld` (а не через `g++` как обёртку), вам пришлось бы явно указывать эти библиотеки. Поэтому-то и используют `g++` как обёртку — он знает, какие библиотеки нужно подцепить.

8.5.2 Сообщения об ошибках линковки

Самые частые ошибки этой стадии:

undefined reference to <symbol>

Линкер нашёл «дырку», но не нашёл, чем её заполнить. Возможные причины: - Забыли скомпилировать нужный .cpp файл. Например, написали `g++ main.cpp -o program` без `math.cpp` — `add` неоткуда взять. - Объявили функцию, но не написали её тело. - Не подключили нужную внешнюю библиотеку (флаг `-l`). - Перепутали манглинг (например, `extern "C"` где надо/не надо).

multiple definition of <symbol>

Два или больше определений одной функции. Линкер не знает, какое выбрать. Часто случается, когда определение функции случайно попадает в заголовочный файл, который подключается из нескольких .cpp-файлов.

Пример. Если бы мы в `math.h` написали не объявление, а определение:

```
// math.h
#ifndef MATH_H
#define MATH_H

int add(int a, int b) {    // ОШИБКА — определение в заголовке!
    return a + b;
}

#endif
```

И этот заголовок подключился бы в `main.cpp` и `math.cpp` — после препроцессинга в обоих файлах оказался бы код `add`. После компиляции — два определения в двух .о. Линкер: «multiple definition».

Решение: в заголовках — только объявления. Определения — в .cpp. (Исключение: шаблоны и `inline`-функции должны быть полностью в заголовках, об этом в главе 20.)

8.5.3 Статическая и динамическая линковка

Линковка бывает двух видов.

Статическая линковка. Все нужные функции из библиотек **копируются прямо в исполняемый файл**. Получается «жирный» бинарь, который работает сам по себе: библиотеки больше не нужны.

Плюсы: - Можно запустить на любом компьютере (нужной версии ОС/архитектуры) без необходимости иметь установленные библиотеки. - Не зависит от версий библиотек в системе.

Минусы: - Большой размер бинарника. - Если в библиотеке нашли уязвимость — нужно пересобрать программу, чтобы обновить.

Динамическая линковка. Линкер только записывает в исполняемый файл «здесь надо подгрузить такую-то библиотеку, и оттуда взять такую-то функцию». Сама библиотека лежит отдельным файлом и подгружается операционной системой при запуске программы.

Файлы динамических библиотек: - *.so (Shared Object) на Linux. - *.dylib на macOS. - *.dll (Dynamic Link Library) на Windows.

Плюсы: - Маленький размер бинарника. - Можно обновить библиотеку в системе — и все программы, использующие её, автоматически получают свежую версию. - Память экономится: если 50 программ используют libc — она в памяти один раз, не 50.

Минусы: - Нужно, чтобы библиотеки требуемых версий были в системе. - Знаменитый «DLL hell» — программа жаждет версию X.Y, а в системе X.Z, и не работает.

По умолчанию компиляторы используют **динамическую линковку для стандартной библиотеки и системных библиотек, и статическую для ваших объектных файлов**. То есть main.o + math.o слипаются в один бинарник, а libstdc++ подгружается во время запуска.

Узнать, какие динамические библиотеки нужны программе:

```
$ ldd ./program # на Linux
linux-vdso.so.1 (0x00007fff...)
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
/lib64/ld-linux-x86-64.so.2 (0x00007f...)

$ otool -L ./program # на macOS
./program:
    /usr/lib/libc++.1.dylib
    /usr/lib/libSystem.B.dylib
```

Чтобы собрать статически (всё внутрь бинарника):

```
$ g++ -static main.cpp math.cpp -o program
```

После этого `ldd program` покажет «not a dynamic executable». Размер бинаря вырастет в десятки раз (с ~30 KB до ~2-5 MB), потому что вся стандартная библиотека теперь внутри. На macOS полностью статическая линковка не поддерживается (Apple использует `dyld` и не разрешает статически линковать `libc`) — там можно только частично.

Статическую сборку часто используют для Docker-контейнеров минимального размера (образ `scratch` + статический бинарь), для подачи бинаря на сервер без установки зависимостей, для `embedded`-устройств.

8.5.4 Что выучить про линковку

- Линковка склеивает несколько `.o` файлов и библиотек в один исполняемый файл.
- «Undefined reference» — линкер не нашёл нужный символ. Проверьте, что нужный `.cpp` тоже компилируется и подключается.
- «Multiple definition» — символ определён в нескольких местах. В заголовках только объявления, не определения.
- Динамическая линковка — библиотеки подгружаются при запуске; статическая — копируются в бинарник.
- `ldd / otool -L` показывают, от каких библиотек зависит бинарник.

8.6 Memory layout: что где лежит в памяти процесса

Программа собрана, бинарник готов. Запускаем — что происходит в памяти?

Операционная система создаёт **процесс** (вспомните главу 1) и **отображает виртуальное адресное пространство процесса** на физическую память. В этом адресном пространстве у программы есть несколько ключевых областей.

Если идти от низких адресов к высоким (типичная картина на Linux x86-64):

8.6.1 `.text` (код)

В самом низу — секция `.text`. Это машинный код вашей программы: функции `main`, `add`, всё, что вы написали. Эта секция помечена как **read-only** и **executable**: процессор может её исполнять, но никто не может в неё писать. Если программа попытается записать в код — получит `segfault` (нарушение защиты памяти).

Это защита от того, чтобы программа случайно (или вирусно) не модифицировала собственный код во время работы.

8.6.2 .rodata (константы)

Чуть выше — секция **.rodata** (read-only data). Здесь лежат: - Строковые литералы: "Hello, world!", "2 + 3 = ". - const-константы с фиксированными значениями. - Виртуальные таблицы классов (vtable) — про них в главе 17.

Эта секция тоже read-only. Если вы возьмёте указатель на строковый литерал и попытаетесь через него что-то записать — segfault.

```
char* p = (char*)"hello";  
p[0] = 'H'; // segfault!
```

8.6.3 .data (инициализированные глобальные)

.data — глобальные и static-переменные с **ненулевым** начальным значением.

```
int counter = 5; // окажется в .data  
static int max_size = 100; // тоже в .data
```

Эта секция read-write. Программа может изменять эти переменные во время работы.

8.6.4 .bss (неинициализированные глобальные)

.bss — глобальные и static-переменные **без** начального значения (или с явным = 0). Они автоматически зануляются при загрузке.

```
int errors; // окажется в .bss, значение 0  
int counters[1000]; // .bss, все нули
```

Почему отдельная секция? Потому что .bss занимает в бинарнике ровно ноль байт — там просто записано «у меня нужно столько-то нулей». Заполнение нулями делает уже ОС, при загрузке. Экономия места на диске.

8.6.5 Куча (heap)

Дальше идёт **куча** (heap). Это область, из которой программа **динамически выделяет** память во время работы. Когда вы пишете `new int` или `malloc(100)`, ОС выделяет вам кусок из кучи. Когда вы вернёте через `delete` или `free`, кусок возвращается.

Куча растёт вверх (от низких адресов к высоким) по мере роста потребностей программы. Размер ограничен либо настройками ОС, либо физической памятью.

Куча — это про **долгоживущие** объекты. Например, дерево комнат в нашем RPG-проекте будет жить в куче, потому что оно должно существовать на протяжении всей игры, а не «пока выполняется одна функция».

8.6.6 Мmap-области

Где-то посередине между кучей и стеком (на современных ОС) живут **отображённые в память** регионы. Сюда подгружаются динамические библиотеки (`libstdc++.so`, `libc.so`). Сюда же попадают большие выделения через `mmap` (например, когда вы запрашиваете у ОС 100 MB одним куском).

Это нюанс — для понимания общей картины можно пропустить.

8.6.7 Стек (stack)

В самой верхней части адресного пространства — **стек**. Растёт вниз, в сторону кучи.

Стек — это **локальные переменные функций**, аргументы вызовов, адреса возврата. Когда вы вызываете функцию, на стек кладётся: - адрес, куда вернуться после функции (return address); - значения аргументов; - место под локальные переменные.

Когда функция возвращается — этот «кадр стека» (stack frame) убирается, и стек уменьшается.

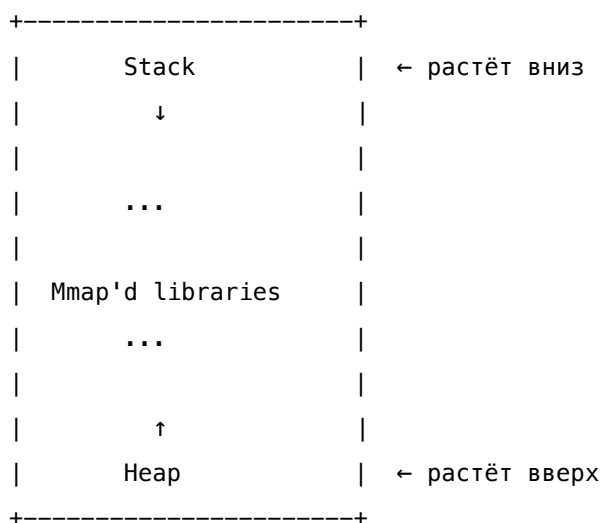
Стек — это про **короткоживущие** объекты. Локальная переменная `int x = 5`; внутри `main` — на стеке. Когда `main` закончится — `x` исчезнет автоматически.

```
void foo() {
    int local = 42;    // на стеке
}                    // local исчезает здесь
```

8.6.8 Картинка целиком

Адреса в типичном Linux-процессе x86-64 (упрощённо):

Высокие адреса



```

|      .bss      |
+-----+
|      .data     |
+-----+
|      .rodata   |
+-----+
|      .text     |
+-----+

```

Низкие адреса

Между стеком и кучей — большое пустое пространство, в которое они оба могут расти. Если стек дойдёт до кучи (или наоборот) — программа упадёт с переполнением.

8.6.9 Утилита `size`

Чтобы увидеть размеры секций бинарника:

```

$ size program
__TEXT __DATA __OBJC  others  dec hex
8192   4096    0  4295106560  4295119248  1000ff850

```

На Linux:

```

$ size program
   text    data     bss      dec       hex filename
  1820     608       24    2452     994 program

```

`text` — размер кода (`.text` + `.rodata`). `data` — инициализированных переменных. `bss` — неинициализированных (но в файле они не лежат, только размер).

8.6.10 Стек: размер и переполнение

Размер стека по умолчанию — на Linux обычно **8 MB**, на macOS — **8 MB** для главного потока и **512 KB** для дополнительных потоков, на Windows — **1 MB**. Можно изменить:

```
$ ulimit -s 16384 # установить 16 MB
```

Стек кончается при глубокой рекурсии:

```

void recur() {
    int big_array[10000]; // 40 KB на каждом вызове
    recur();
}

```

Эта функция вызывает себя бесконечно, каждый раз кладя на стек 40 КБ. Через 200 вызовов стек кончится → **stack overflow** → программа упадёт.

Куча кончается сложнее — обычно вы упрётесь в физическую память раньше, чем в ограничение на размер кучи. На 64-битной системе виртуального пространства хватит на петабайты, а физического — на 16-64 GB обычно.

В Главе 11 «Память — стек, куча, RAII» эти штуки разберём подробно с C++-примерами.

8.7 Полная картина: что происходит при сборке

Соберём весь процесс в одну схему. Команда `g++ main.cpp math.cpp -o program` выполняет:

1. **Препроцессор**: каждый `.cpp` файл проходит через `cpp`. Подставляются `include`'ы, разворачиваются макросы. На выходе — два развёрнутых файла (промежуточно, в памяти).
2. **Компилятор**: каждый развёрнутый файл проходит через `cc1plus`. Получается ассемблер. (Промежуточно.)
3. **Ассемблер**: каждый ассемблерный файл проходит через `as`. Получаются `main.o` и `math.o` (могут быть и не сохранены физически, если делаем одной командой).
4. **Линкер**: все объектные файлы плюс стандартная библиотека → исполняемый файл `program`.

Чтобы увидеть это в подробностях, есть флаг `-v` («verbose»):

```
$ g++ -v main.cpp math.cpp -o program
... много вывода ...
```

Увидите все запускаемые программы с их аргументами. Полезно, когда что-то не собирается и непонятно почему.

8.8 Один важный нюанс: единицы трансляции

В C++ есть понятие **translation unit** (единица трансляции). Это **один .cpp-файл вместе со всеми include'ами**, развёрнутый препроцессором.

Каждый `.cpp`-файл — отдельная `translation unit`. Они компилируются независимо. Это значит: - Изменения в одной TU не требуют пересборки остальных. Если поправили `math.cpp` — `main.cpp` пересобирать не надо, только новый `math.o` слинковать с старым `main.o`. - Декларации, которые видны только внутри одной TU (через `static` или `unnamed namespace`), не видны другим.

Это основа **incremental builds**: на больших проектах системы сборки (Make, CMake, Bazel, ...) пересобирают только те TU, чьи зависимости изменились. На моём ноутбуке Lirotech компилируется с нуля 8 минут; инкрементно — 5 секунд.

Подробнее — в главе 15 (Дизайн проекта RPG) с Makefile.

8.9 Маленькое упражнение

Заведите папку, создайте три файла из начала главы (math.h, math.cpp, main.cpp). Дальше:

1. Скомпилируйте каждый .cpp отдельно (флаг -c):

```
$ g++ -std=c++11 -c main.cpp
$ g++ -std=c++11 -c math.cpp
```

2. Посмотрите символы в каждом:

```
$ nm main.o
$ nm math.o
```

Найдите символ add в обоих. В одном он должен быть U (нужен), в другом T (определён).

3. Слинкуйте:

```
$ g++ main.o math.o -o program
```

4. Запустите.

Теперь нарочно сломайте линковку:

5. Скомпилируйте только main.cpp:

```
$ g++ -std=c++11 main.cpp -o broken
```

Что увидите? Изучите сообщение об ошибке.

6. В main.cpp замените add(2, 3) на multiply(2, 3). Скомпилируйте только main.cpp:

```
$ g++ -std=c++11 -c main.cpp
```

Что произойдёт и почему? Это ошибка компиляции или линковки?

7. Посмотрите ассемблер:

```
$ g++ -std=c++11 -S main.cpp -o main.s
$ less main.s
```

Найдите место вызова add. Обратите внимание на манглинг имени.

8. (Опционально) Посмотрите вывод препроцессора:

```
$ g++ -std=c++11 -E main.cpp | wc -l
```

Сколько строк? Откройте `g++ -E main.cpp -o main.i` и просмотрите. Найдите в конце своё `int main()`.

Эти упражнения покажут «вживую» все понятия главы. Если что-то не получается — перечитайте соответствующий раздел.

8.10 Что дальше

Часть 0 (подготовка) и Часть 0a (программирование с нуля) и Часть 0b (инструменты + компиляция) пройдены. Если вы дочитали досюда — вы знаете больше про сборку программ, чем 80% разработчиков на C++ с 3-летним стажем.

Глава 8 — **Часть I, Язык**. Начнём с самого фундамента C++: переменные и типы. Что такое `int`, какого он размера, чем отличаются `signed` и `unsigned`, как работают плавающие числа, что такое `auto` и почему в C++11 это полезно, какие бывают способы инициализации (`= 5`, `(5)`, `{5}`) и какой когда применять. Глава будет длинная — типы в C++ это правда сложно — но без этой базы дальше никак.

Глава 9

Глава 8. Переменные и типы

Часть 0 пройдена — у вас в голове устройство компьютера, биты, алгоритмическое мышление, терминал, модель компиляции. Теперь начинаем собственно C++ — Часть I, чистый язык, без проекта. Семь глав, в которых разберём базовые конструкции, чтобы к Части II (RPG-игра) подойти подготовленными.

Первая глава Части I — про **типы и переменные**. В Python вы пишете `x = 5` и не задумываетесь, что это число какого-то размера. В C++ всё иначе: при объявлении переменной вы обязаны сказать, какого она типа. И типов целых чисел — пять штук, разных размеров. И ещё типы дробных. И ещё `bool`, `char`. И ещё способы их инициализации — три разных. И ещё `const`, `constexpr`, `auto`. Глава длинная, но через неё надо честно пройти, иначе вся арифметика дальше будет «магией».

9.1 Почему в C++ типы такие важные

В Python `x = 5`; `x = "hello"` — нормально. Переменная меняет тип на лету. В C++ так нельзя. После `int x = 5`; переменная `x` навсегда `int`. Попытка `x = "hello"`; — ошибка компиляции.

Это нужно по двум причинам.

Производительность. Когда компилятор знает, что `x` — это 4-байтное целое, он генерирует машинный код, специально работающий с 4-байтными целыми. Никаких проверок «а вдруг это строка?», никаких словарей типов в рантайме. Поэтому C++ так быстр.

Безопасность. Компилятор находит много ошибок ещё до запуска. Если функция ждёт `int`, а вы пытаетесь передать строку — компилятор ругнётся. В Python такая ошибка вылезла бы только при запуске, и то — если до этого места выполнение бы дошло.

Цена — больше работы при написании кода и больше синтаксиса. Но это того стоит. Особенно когда программа дорастает до сотен тысяч строк.

9.2 Целые числа: семейство `int`

Начнём с самого частого. **Целое число** в C++ можно объявить пятью способами:

```
char    c = 1;    // 1 байт
short   s = 1;    // 2 байта
int     i = 1;    // 4 байта (обычно)
long    l = 1;    // 4 или 8 — зависит!
long long ll = 1; // 8 байт
```

Главный сюрприз: **размер этих типов не зафиксирован стандартом C++**. Стандарт гарантирует только некоторые минимумы и порядок:

- `char` — ровно 1 байт, как минимум 8 бит (де-факто на всех платформах = 8 бит).
- `short` — как минимум 16 бит.
- `int` — как минимум 16 бит.
- `long` — как минимум 32 бит.
- `long long` — как минимум 64 бит.

И ещё гарантия: $\text{sizeof(char)} \leq \text{sizeof(short)} \leq \text{sizeof(int)} \leq \text{sizeof(long)} \leq \text{sizeof(long long)}$.

На практике на современных 64-битных системах:

Тип	Linux/macOS x86-64	Windows x86-64
<code>char</code>	1	1
<code>short</code>	2	2
<code>int</code>	4	4
<code>long</code>	8	4
<code>long long</code>	8	8

Видите ловушку? **`long` под Linux/macOS — 8 байт, а под Windows — 4!** Поэтому полагаться на размер `long` нельзя. Если вам нужно ровно 8 байт — используйте `long long`. Если ровно 4 — `int`.

9.2.1 Проверяем `sizeof`

`sizeof` — оператор C++, возвращающий размер типа в байтах. Используется так:


```
#include <iostream>

int main() {
    std::cout << "char:      " << sizeof(char) << "\n";
    std::cout << "short:     " << sizeof(short) << "\n";
    std::cout << "int:        " << sizeof(int) << "\n";
    std::cout << "long:       " << sizeof(long) << "\n";
    std::cout << "long long: " << sizeof(long long) << "\n";
    return 0;
}
```

Соберите и запустите. На моём Mac M3 вывод:

```
char:      1
short:     2
int:       4
long:      8
long long: 8
```

`sizeof` — это **оператор времени компиляции**: компилятор вычисляет размер на этапе сборки и подставляет уже число. Поэтому `sizeof(int)` стоит ровно столько же, сколько просто литерал 4 — никакой работы в рантайме.

Можно применять `sizeof` и к переменной: `sizeof(x)` или просто `sizeof x` (без скобок). Тип переменной известен компилятору, размер он подставит.

9.2.2 signed и unsigned

По умолчанию все целочисленные типы — **со знаком** (signed). То есть могут хранить и положительные, и отрицательные значения. Можно явно сказать «без знака»:

```
unsigned int ui = 1;
unsigned long ul = 1;
unsigned char uc = 200;
```

Размер тот же, но интерпретация другая (вспомните главу 2 — дополнительный код).

Тип	Размер	Минимум	Максимум
signed char	1	-128	127
unsigned char	1	0	255
signed short	2	-32 768	32 767
unsigned short	2	0	65 535

Тип	Размер	Минимум	Максимум
signed int	4	-2 147 483 648	2 147 483 647
unsigned int	4	0	4 294 967 295
signed long long	8	-9.2×10^{18}	9.2×10^{18}
unsigned long long	8	0	1.8×10^{19}

Можно сокращать: вместо `signed int` пишут `int`, вместо `unsigned int` — `unsigned`. Это эквивалентно. Но я рекомендую писать `unsigned int` или `unsigned long`, потому что одно слово `unsigned` без типа выглядит загадочно.

9.2.3 Когда использовать `unsigned`

Естественное чувство: «если число не может быть отрицательным — берём `unsigned`». Возраст, количество чего-то, размер массива.

В C++ совет: **берите `signed`, даже если число всегда неотрицательное**. Парадокс? Объясню.

Главная причина — **arithmetic underflow с `unsigned` работает «странно»**. Возьмём пример:

```
unsigned int i = 0;
i = i - 1;
std::cout << i;    // напечатает 4294967295!
```

Что произошло? $0 - 1$ для `unsigned` даёт $2^{32} - 1$ (обернулось через ноль). Это, между прочим, **строго определённое поведение** для `unsigned`, не `undefined behavior`. Стандарт говорит: «арифметика `unsigned` — по модулю 2^N , где N — число битов».

Но из-за этого вы получаете «странные» баги. Самый известный — цикл назад:

```
std::vector<int> v = {1, 2, 3, 4, 5};

// Идём по вектору в обратном порядке, типичный новичковый код:
for (unsigned i = v.size() - 1; i >= 0; --i) {
    std::cout << v[i];
}
```

Цикл должен вывести 5 4 3 2 1 и остановиться. Но условие `i >= 0` для `unsigned i` всегда истинно! `i` уменьшается, на нуле она ещё валидная, на следующей итерации `--i` оборачивает в 4294967295, и цикл продолжается. Программа залезает за границы вектора и крашится (или, хуже, выдаёт мусор).

С `int` такой проблемы нет:

```
for (int i = v.size() - 1; i >= 0; --i) {  
    std::cout << v[i];  
}
```

Когда `i` станет `-1`, условие `>= 0` нарушится, цикл завершится.

Поэтому правило: **используйте `int` или `long (signed)` для индексов и счётчиков, даже если они логически не отрицательные.**

Когда оправдан `unsigned`: - Битовые операции (`AND`, `OR`, `XOR`, сдвиги) — там знак не имеет смысла.

- Когда вам нужны именно полные 64 бита диапазона (`signed long long` даёт только 63). - Когда вы реально хотите `arithmetic mod 2^N` (например, в хэширующих функциях).

9.2.4 Целочисленные типы фиксированного размера

Раз размеры основных типов не зафиксированы, иногда надо именно «8 битов» или именно «32 бита». На этот случай в стандартной библиотеке есть типы из заголовочного файла `<stdint.h>`:

```
#include <stdint.h>  
  
int8_t    a = 1;    // ровно 1 байт со знаком (-128..127)  
uint8_t   b = 1;    // ровно 1 байт без знака (0..255)  
int16_t   c = 1;    // ровно 2 байта со знаком  
uint16_t  d = 1;    // ровно 2 байта без знака  
int32_t   e = 1;    // ровно 4 байта со знаком  
uint32_t  f = 1;    // ровно 4 байта без знака  
int64_t   g = 1;    // ровно 8 байт со знаком  
uint64_t  h = 1;    // ровно 8 байт без знака
```

Эти типы гарантированно одного размера на всех платформах. Удобно для бинарных форматов файлов и сетевых протоколов, где важна точная раскладка байтов.

Внутри это псевдонимы (`typedef` или `using`). На Linux/macOS `int32_t` — это обычно `int`. На Windows — тоже `int`. Но писать `int32_t` яснее: ты сразу видишь «здесь нужны ровно 4 байта».

В этой книге я буду стараться использовать `int` для обычных счётчиков (это коротко, и так все пишут), и `int32_t`/`uint64_t`/etc там, где размер важен (например, форматы файлов в проекте мини-СУБД).

9.2.5 Литералы целых чисел

Когда вы пишете в коде число 42 — это **литерал**. Его тип по умолчанию — самый маленький из (int, long, long long), который вмещает значение. Для большинства чисел — int.

Можно явно указать тип литерала **суффиксом**:

```
42          // int
42L         // long
42LL        // long long
42U         // unsigned int
42UL        // unsigned long
42ULL       // unsigned long long
```

Регистр суффикса роли не играет: 42ll, 42LL, 42lL — всё одно. Но писать большими лучше — маленькая l визуально путается с цифрой 1.

Кроме десятичной формы, литералы можно писать:

```
42          // десятичный
0x2A        // шестнадцатеричный (0x префикс)
0b101010    // двоичный (0b префикс, появился в C++14)
052         // восьмеричный (один ноль в начале)
```

Все четыре записи — одно и то же число 42. Восьмеричной формой почти никто не пользуется — в практике она встречается только в правах доступа Unix (вспомните `chmod 755`).

С C++14 разрешены **разделители разрядов**:

```
1'000'000    // миллион, читается легче чем 1000000
0xFF'FF      // 65535
0b1010'1100   // байт
```

Апостроф между цифрами — просто визуальное разделение, на значение не влияет. Очень удобно для длинных чисел. Но это C++14, и в нашей C++11-базе не работает.

9.3 Дробные числа: float и double

Для дробных чисел в C++ есть три типа:

```
float        f = 3.14f;    // 4 байта
double       d = 3.14;     // 8 байт
long double  ld = 3.14L;   // зависит — 8, 12 или 16
```

В прошлой главе про IEEE 754 я уже упоминал детали. Здесь повторю важное:

- `float` — 4 байта, **примерно 7 значащих десятичных цифр** точности. Диапазон: $\pm 10^{\pm 3}$...
- `double` — 8 байт, **примерно 15-16 значащих цифр**. Диапазон: $\pm 10^{\pm 3}$...
- `long double` — на x86-64 Linux обычно 80-битное число с заполнением до 16 байт. На macOS ARM64 — то же, что `double` (Apple не реализовала extended precision на ARM). На Windows — то же, что `double`.

Если коротко: **используйте `double`** для всех расчётов. Это разумный дефолт. `float` берите только когда у вас огромный массив чисел и память жмёт (4 vs 8 байт даст экономию вдвое), или когда работаете с библиотекой, которая ждёт именно `float` (GPU, OpenGL, аудио). `long double` — экзотика, не нужная в 99% задач.

9.3.1 Литералы дробных чисел

```
3.14      // double по умолчанию
3.14f     // float
3.14L     // long double
1.0       // double, потому что 1.0 это не целое
1e6       // 1 миллион, double (экспоненциальная форма)
1.5e-3    // 0.0015, double
1.5E-3    // то же — регистр не важен
```

`f`-суффикс для `float` обязателен, если вы хотите именно `float`. Если написать `float f = 3.14;` — компилятор предупредит «implicit conversion from double to float», потому что `3.14` без `f` — это `double`, и оно «теряет точность» при присвоении в `float`.

9.3.2 Точность — самое больное место

Двоичная плавающая точка не может точно представить большинство десятичных дробей. Классический пример:

```
#include <iostream>

int main() {
    double x = 0.1 + 0.2;
    std::cout << x << "\n";           // 0.3 — но это печать с округлением!
    std::cout.precision(20);
    std::cout << x << "\n";           // 0.30000000000000004441

    if (x == 0.3) std::cout << "equal\n";
    else          std::cout << "NOT equal\n"; // эта ветка!
```

```
    return 0;
}
```

$0.1 + 0.2$ не равно ровно 0.3 . Поэтому **сравнивать дробные числа на равенство почти никогда нельзя**. Вместо `if (x == 0.3)` пишут:

```
#include <cmath>

if (std::abs(x - 0.3) < 1e-9) {    // отличается меньше чем на  $10^{-9}$ 
    // считаем равными
}
```

Или, для совсем серьёзных случаев, есть `std::numeric_limits<double>::epsilon()` — но это уже тема главы про библиотеку.

И ещё: **никогда не храните деньги в `double`**. Кажется логично — деньги это числа, что не так? А то, что у вас на счёте $100.10\$$. Программа на сервере умножает баланс на процент: $100.10 * 1.05 = 105.105$. Округляете до двух знаков — 105.11 . Но на самом деле в `double` лежало не 100.10 , а 100.09999999999999 . Умножение даёт 105.10499999999999 . Округление — 105.10 . Копейка пропала. Бухгалтерия в шоке.

Для денег используйте либо `int64_t` в копейках/центах (баланс хранится как 10010 копеек), либо специальные библиотеки `decimal-arithmetic`. В нашей книге денег не будет, но имейте в виду на будущее.

9.4 bool

Тип для логических значений:

```
bool a = true;
bool b = false;
bool c = (5 > 3);    // c = true
```

Два значения — `true` и `false`. Размер `sizeof(bool)` обычно 1 байт (хотя нужен только 1 бит — но процессоры не умеют адресовать отдельные биты).

В C++ `bool` неявно конвертируется к `int`: `true` → 1, `false` → 0. И наоборот: любое ненулевое число → `true`, ноль → `false`.

```
int n = 5;
if (n) {                // эквивалентно if (n != 0)
    std::cout << "non-zero\n";
}
```

```
bool b = 42;           // b = true, потому что 42 != 0
int x = true;          // x = 1
```

Эта возможность — приятное наследие C, но иногда мешает. Современный C++ пишет явно:

```
if (n != 0) { ... }    // лучше, чем if (n)
if (ptr != nullptr) { ... } // лучше, чем if (ptr)
```

В компиляторах есть флаги, ругающиеся на неявные конверсии (`-Wconversion`), и я их рекомендую включать на новых проектах.

9.5 char и символы

`char` — целочисленный тип размером 1 байт, традиционно используемый для хранения одного символа ASCII.

```
char c = 'A';
std::cout << c;           // напечатает A
std::cout << (int)c;      // напечатает 65 (код ASCII буквы A)
```

`'A'` — это литерал символа, **одинарные кавычки**, ровно один символ внутри. Его тип — `char`, значение — числовой код символа в ASCII (или Latin-1, или в чём угодно, что компилятор выбрал; для буквы A это всегда 65).

Двойные кавычки — это уже **строка**: `"A"` — это не `char`, а `const char[2]` (массив из двух элементов: `'A'` и невидимого `'\0'` в конце). Принципиально другой тип.

9.5.1 Escape-последовательности

В символьном или строковом литерале нельзя написать буквальный перевод строки или кавычку. Для них используют escape-последовательности:

```
'\n'    // перевод строки (newline)
'\t'    // табуляция
'\r'    // возврат каретки (на Windows используется в паре \r\n)
'\\'    // обратный слеш
'\''    // одинарная кавычка
'\"'    // двойная кавычка (внутри строкового литерала)
'\0'    // нулевой символ (терминатор C-строк)
'\xFF'  // байт со значением 0xFF (255)
'\377'  // байт со значением 0377 в восьмеричной (= 255)
```

9.5.2 char, signed char, unsigned char

Один из неприятных уголков C++: тип `char` может быть как `signed`, так и `unsigned` — **зависит от компилятора и платформы**. На большинстве — `signed` (диапазон -128..127), но на ARM-Linux исторически `unsigned` (0..255).

В реальности это редко имеет значение для текста (ASCII только 0-127, попадает в обе интерпретации). Но если работаете с байтами как с числами:

```
char c = 0xFF;
int n = c;           // 0xFF → -1 (если char signed) или 255 (если unsigned)
```

Чтобы было определённо, пишете явно:

```
signed char sc = 0xFF;    // -1
unsigned char uc = 0xFF;  // 255
```

В большинстве случаев `char` для текста, `unsigned char` или `uint8_t` для байтов.

9.6 const — переменная, которую нельзя менять

Иногда нужно сказать «вот значение, я его задаю один раз и потом не меняю». Это **константа**. В C++ ставится модификатор `const`:

```
const int max_size = 100;
const double pi = 3.14159;

max_size = 200;    // ОШИБКА компиляции: assignment of read-only variable
```

`const`-переменная инициализируется при создании, и больше её значение менять нельзя. Попытка — ошибка компиляции.

Зачем это нужно?

- **Документация для читателя.** Видя `const int max_size`, читатель сразу понимает: это константа, она никогда не меняется.
- **Защита от случайных правок.** Если коллега полез в код и попытался изменить — компилятор остановит.
- **Возможности оптимизации.** Зная, что значение не меняется, компилятор может его «подставить» прямо в код, не читая из памяти.
- **Корректность функций.** Параметр функции, помеченный `const`, гарантирует: внутри функции его не меняют. Об этом подробно в главе 10.

Запомните железно: в современном C++ всё, что не должно изменяться — должно быть `const`. Это лучшая защита от багов.

9.7 constexpr — константа времени компиляции (C++11)

`const` говорит «не менять». А `constexpr` (новинка C++11) говорит «это значение известно на этапе компиляции».

```
constexpr int max_size = 100;
constexpr double pi = 3.14159;
constexpr int doubled = max_size * 2; // 200, посчитано на компиляции
```

Разница с `const`:

```
int x;
std::cin >> x;
const int a = x; // OK: a — константа, но её значение известно только в рантайме
constexpr int b = x; // ОШИБКА: b должна быть известна на компиляции, а x — нет
constexpr int c = 100; // OK: 100 — литерал, известен на компиляции
```

То есть `constexpr` строже, чем `const`: значение должно быть вычислимо компилятором.

Зачем? Для использования в местах, где нужно именно «значение компилятора». Например, размер массива:

```
const int n = 10;
int array1[n]; // OK в C++ (хотя не в строгом стандарте)

int m;
std::cin >> m;
const int p = m;
int array2[p]; // НЕ OK: размер массива должен быть известен на компиляции
               // (хотя GCC через VLA это разрешает как расширение)
```

`constexpr` решает эту проблему явно: «вот это точно вычислимо на компиляции, не сомневайся».

В C++14+ можно делать `constexpr`-функции, в C++20 — даже выделять память на куче на компиляции. Но в C++11 `constexpr` довольно ограничен — простые арифметические выражения с литералами и другими `constexpr`-переменными.

Правило: для констант предпочитайте `constexpr`. Если нельзя — `const`. Если хотите изменяемость — обычная переменная.

9.8 Способы инициализации

В C++11 есть **три способа** проинициализировать переменную, и они немного отличаются. Это очень частый источник путаницы.

9.8.1 Старый способ: с равно

```
int x = 5;
double pi = 3.14;
char c = 'A';
```

Знакомый по другим языкам. Работает почти всегда.

9.8.2 С круглыми скобками

```
int x(5);
double pi(3.14);
char c('A');
```

Это «конструкторный синтаксис». Для встроенных типов делает то же, что =. Но для классов — вызывает соответствующий конструктор.

Подвох — **most vexing parse**:

```
int x();    // НЕ объявление int x = 0! Это ОБЪЯВЛЕНИЕ ФУНКЦИИ x, возвращающей int
```

Компилятор интерпретирует `int x()` как объявление функции без параметров, возвращающей `int`. Из-за этой ловушки многие предпочитают другие синтаксисы.

9.8.3 Единообразная инициализация (C++11): с фигурными скобками

```
int x{5};
double pi{3.14};
char c{'A'};
```

Этот синтаксис **появился в C++11** и считается рекомендуемым. Преимущества:

1. **Работает везде одинаково**: для встроенных типов, классов, массивов, стандартных контейнеров.

```
int x{5};
std::vector<int> v{1, 2, 3, 4};    // вектор из 4 элементов
```

2. **Защищает от «narrowing» — сужающих преобразований.** Если вы пытаетесь засунуть значение, которое не поместится — будет ошибка компиляции:

```
int x = 3.14;    // OK, x = 3 (молча отбросило 0.14)
int y(3.14);    // OK, y = 3 (тоже молча)
int z{3.14};    // ОШИБКА: narrowing conversion from double to int
```

Эта проверка спасла моих коллег от десятков багов. Серьёзно.

3. **Не страдает от most vexing parse:**

```
int x{};    // x = 0 (value initialization), однозначно
```

В книге я буду использовать {} там, где это естественно, и = для простых случаев. Это слегка непоследовательно, но соответствует общепринятой практике.

9.8.4 Default initialization vs value initialization

Что произойдёт, если объявить переменную **без значения**?

```
int x;    // !!! не инициализирован
```

Для **локальной переменной встроенного типа** значение **не определено**. Внутри переменной — мусор, что было в памяти до этого. Чтение мусора — UB (undefined behavior).

```
int x;
std::cout << x;    // UB — может напечатать что угодно, или упасть
```

Чтобы инициализировать нулём:

```
int x{};    // x = 0, value initialization
int x = 0;    // тоже OK, но явно
```

Для **глобальных и static-переменных** правила другие: они **автоматически** инициализируются нулями (точнее — value initialization). Это потому, что глобалы лежат в .bss-секции, которая зануляется при загрузке (вспомните главу 7).

```
int g;    // глобальная — автоматически 0
static int s;    // static — автоматически 0

int main() {
    int local;    // НЕ инициализирована
    std::cout << g << " " << s << " " << local;    // 0 0 ???
}
```

Правило: всегда инициализируйте локальные переменные. Не пишите `int x;` без значения. Привычка эта спасёт вас от сотен невоспроизводимых багов.

9.9 auto — компилятор сам выводит тип (C++11)

В C++11 появилось ключевое слово `auto`. Оно говорит: «компилятор, сам определи тип из правой части».

```
auto x = 5;           // компилятор видит 5 → int → x это int
auto y = 3.14;        // double
auto s = "hello";     // const char*
auto v = std::vector<int>{1, 2, 3}; // std::vector<int>
```

Зачем? Без `auto` иногда приходится писать длиннющие типы:

```
std::map<std::string, std::vector<int>>::iterator it = m.begin();
```

С `auto`:

```
auto it = m.begin();
```

Куда читабельнее.

9.9.1 Когда auto хороший выбор

- Длинные типы (итераторы, типы из шаблонов).
- Когда тип очевиден из правой части:

```
auto x = std::vector<int>(100); // ясно, что это vector<int>
```

- При работе с лямбдами (тип лямбды нельзя записать без `auto`).

9.9.2 Когда auto плохой выбор

- Когда читатель не понимает, какой тип получится. Например:

```
auto x = some_func(); // что такое x?
```

Если из имени функции не ясно — лучше написать тип явно.

- Когда `auto` скрывает важную деталь. Например:

```
for (auto x : container) { ... }
```

Здесь `x` — копия! Если контейнер хранит большие объекты, копии дорогие.

```
for (const auto& x : container) { ... }
```

А вот это — константная ссылка, без копирования. Гораздо лучше для больших объектов.

9.9.3 auto и const/&

auto берёт тип «как есть», но **отбрасывает const и &**:

```
const int& ref = ...;
auto a = ref;           // a — это int (без const, без &)
const auto& b = ref;    // b — это const int& (как и ref)
```

Поэтому привычное `const auto&` — это «возьми тип, и сделай ссылкой на const». Самый частый паттерн для циклов:

```
for (const auto& item : items) {
    // item — это const-ссылка на элемент контейнера
    // никаких копий, и нельзя случайно изменить
}
```

Запомните этот паттерн — он будет повторяться постоянно.

9.10 Имена переменных: правила и стиль

Правила компилятора: - Имя — буквы, цифры и подчёркивание. - Начинаться с буквы или подчёркивания (но **не с цифры**). - Регистр важен: `count` и `Count` — разные переменные. - Нельзя использовать ключевые слова (`int`, `if`, `while`, и т. д.). - Имена, начинающиеся с подчёркивания + заглавной буквы (`_Foo`) или с двух подчёркиваний (`__bar`), зарезервированы для компилятора и стандартной библиотеки. **Не используйте их**. - В C++ разрешены **юникод-имена**: `cpp int сумма = 0; // работает (если компилятор настроен на UTF-8)` Но я не рекомендую: смешение языков в коде раздражает, и не все системы понимают utf-8 source.

Стиль — это уже вопрос команды. Самые популярные:

- **snake_case** (`max_size`, `current_value`) — преобладает в стандартной библиотеке C++ и в стиле Google.
- **camelCase** (`maxSize`, `currentValue`) — преобладает в Java, popular в C++.
- **PascalCase** (`MaxSize`) — обычно для имён типов (классов, структур), не для переменных.

Я в книге использую **snake_case** для переменных, **PascalCase** для типов:

```
int player_health;
double damage_multiplier;
```

```
class Player;  
struct Inventory;
```

Это близко к стандартной библиотеке (`std::vector`, `std::string`) и читаемо.

Хорошие имена: - Говорят, что переменная значит: `player_health`, `damage_per_round`, `is_alive`.
- Не сокращены до неузнаваемости: `cnt` плохо, `count` лучше, `enemy_count` ещё лучше. - Не повторяют тип в имени: `int_count` плохо (и так видно по типу). - Короткие имена — только для коротких циклов: `i`, `j`, `k` для счётчиков, `it` для итератора, `n` для размера.

Плохие имена: - `temp`, `tmp`, `data`, `value` — не говорят ничего. - `x1`, `x2`, `x3` — порядок появления, не смысл. - `theBestVariableEverInTheWorld` — слишком длинное.

9.11 Полный собранный пример

Соберём всё в работающую программу:

```
#include <iostream>  
#include <cstdlib>  
  
int main() {  
    // Целочисленные типы  
    int count = 42;  
    long long big_number = 1000000000000LL;           // в C++14 для читаемости: 1'000'000'000'000LL  
    unsigned int positive_only = 100;  
    uint8_t one_byte = 255;  
  
    // Дробные  
    double pi = 3.14159265358979323846;  
    float weight_kg = 75.5f;  
  
    // Логический  
    bool game_over = false;  
  
    // Символ и строка (строки разберём в главе 12)  
    char first_letter = 'A';  
    const char* message = "Hello, C++!";  
  
    // Константа времени компиляции  
    constexpr int max_players = 8;
```

```

// auto и инициализация фигурными скобками
auto health = 100;      // int
auto damage = 12.5;     // double
auto enabled = true;    // bool

int explicit_init{0};   // value initialization

// Выводим (тут пока в одну строку, форматирование – в главе 12)
std::cout << "count = " << count << "\n";
std::cout << "big_number = " << big_number << "\n";
std::cout << "pi = " << pi << "\n";
std::cout << "weight_kg = " << weight_kg << "\n";
std::cout << "game_over = " << game_over << "\n";
std::cout << "first_letter = " << first_letter
          << " (code " << (int)first_letter << ") \n";
std::cout << "max_players = " << max_players << "\n";

return 0;
}

```

Сохраните как types_demo.cpp, соберите и запустите:

```

$ g++ -std=c++11 -Wall -Wextra types_demo.cpp -o types_demo
$ ./types_demo

```

Должно вывести что-то осмысленное. Если убрать LL у big_number — на 32-битных платформах будет переполнение и -Wall ругнётся.

9.12 Преобразования типов: коротко

Между разными типами C++ умеет автоматически конвертировать. Правила:

```

int i = 5;
double d = i;      // OK: int → double (расширение)

double pi = 3.14;
int j = pi;        // OK, но компилятор может предупредить – теряем 0.14

unsigned int u = 5;

```

```
int s = u;           // OK
unsigned int u2 = -1; // Ой: -1 как unsigned = 4294967295. Предупреждение от -Wsign-conversion.

bool b = 1;          // OK: 1 → true
int n = false;        // OK: false → 0
```

Преобразования бывают: - **Расширяющие** (widening): меньший тип → больший. Безопасно. - **Сужающие** (narrowing): больший тип → меньший. Может терять данные. - **Со знаком ☒ без знака**: интерпретация битов меняется, но биты те же. Опасно.

Через {} инициализацию (C++11) сужающие конверсии запрещены — это ошибка компиляции. Через = или () — только предупреждение, и то с подходящими флагами.

Явное преобразование пишут через `static_cast`:

```
double pi = 3.14;
int n = static_cast<int>(pi); // явно говорим: знаем что теряем дробную часть
```

Есть ещё `reinterpret_cast` (опасный, переинтерпретация битов) и `const_cast` (удаление `const`). К ним вернёмся в нужных местах книги.

9.13 Главное — что увидите в коде дальше

Этой главы хватит, чтобы понимать почти любую переменную, которая будет встречаться. На примерах:

```
int health = 100;           // 4-байтное целое
double damage = 12.5;       // 8-байтное дробное
bool alive = true;          // логический
char first_letter = 'A';    // 1-байтный символ
const int max_players = 8;  // неизменяемая константа
constexpr double pi = 3.14159; // константа времени компиляции
auto count = 42;            // компилятор сам определит → int
auto it = container.begin(); // компилятор сам определит → итератор
const auto& item = container[0]; // const-ссылка, без копии
int32_t precise_size = 100; // ровно 4 байта, для протоколов
uint8_t byte_value = 0xFF;  // ровно 1 байт, для байтовых данных
```

К каждой из этих форм будете возвращаться постоянно.

9.14 Маленькое упражнение

Соберите эти программы и проверьте, что вы правильно угадали поведение:

1. Узнайте размеры типов на вашем компьютере:

```
std::cout << sizeof(int) << " " << sizeof(long) << " " << sizeof(long long);
```

На Linux/macOS x86-64 будет 4 8 8. На Windows x86-64 — 4 4 8.

2. Покажите, что unsigned оборачивается:

```
unsigned int x = 0;
std::cout << x - 1 << "\n"; // напечатает 4294967295
```

3. Узнайте, signed или unsigned у вас char:

```
char c = -1;
if (c < 0) std::cout << "char is signed\n";
else      std::cout << "char is unsigned\n";
```

4. Покажите неточность плавающей точки:

```
std::cout.precision(20);
double a = 0.1 + 0.2;
std::cout << a;
```

5. (Самое полезное) Запустите программу с предупреждениями:

```
int main() {
    int x = 3.14;           // -Wfloat-conversion ругнётся
    int y{3.14};           // ошибка narrowing
    unsigned u = -1;       // -Wsign-conversion ругнётся
    int z;
    std::cout << z;         // -Wuninitialized ругнётся
    return 0;
}
```

Команда сборки: `g++ -std=c++11 -Wall -Wextra -Wpedantic -Wconversion test.cpp -o test`. Внимательно почитайте каждое предупреждение. Включите эти флаги во все проекты — они помогают.

9.15 Что дальше

Глава 9 — управляющие конструкции. `if/else`, `switch`, циклы `for`, `while`, `do-while`, `range-based for` из C++11, `break/continue`, и почему `goto` — это не зло, а просто инструмент с ограниченной областью применения. Дальше каждая следующая глава будет всё ближе к настоящей программе — после блока про основы языка перейдём к RPG-проекту, где все эти штуки соединятся в реальный код.

Глава 10

Глава 9. Управляющие конструкции

В главе 4 мы обсуждали ветвление и циклы концептуально, на псевдокоде. В главе 8 разобрались с переменными. Теперь склеиваем — пишем настоящие управляющие конструкции на C++. Тут много синтаксиса, но материал в целом простой: после Python/PHP/JS большая часть будет узнаваемой. Главное — пара ловушек, на которых спотыкаются новички, и одна-две особенности C++, которых нет в других языках.

10.1 if — простое ветвление

Базовая форма:

```
if (условие) {  
    действия;  
}
```

Если условие истинно — выполняются действия в {...}. Иначе — пропускаются.

```
int health = 50;  
if (health < 30) {  
    std::cout << "Внимание: мало HP!\n";  
}
```

Скобки (...) вокруг условия — **обязательны**. В отличие от Python или Ruby, в C++ нельзя написать `if health < 30 {...}`.

Фигурные скобки {...} вокруг тела — формально необязательны, если внутри одно действие:

```
if (health < 30)  
    std::cout << "Мало HP!\n";
```

Но я **настоятельно рекомендую всегда писать {...}**. Без скобок легко вляпаться:

```
if (health < 30)
    health = health + 10;
    std::cout << "Heal!\n";    // эта строка выполнится ВСЕГДА!
```

Отступы вводят в заблуждение. Без {...} к if относится только первая строка, второй std::cout выполняется всегда. Внутри if он не входит, потому что нет фигурных скобок. Я лично потерял несколько часов на таких багах в начале карьеры. С скобками такой ошибки не сделать:

```
if (health < 30) {
    health = health + 10;
    std::cout << "Heal!\n";
}
```

Дешёвый страховочный полис. Не экономьте две строки.

10.2 if/else

Альтернативная ветка пишется через else:

```
if (health > 0) {
    std::cout << "Жив\n";
} else {
    std::cout << "Мёртв\n";
}
```

else if для нескольких условий подряд:

```
if (health <= 0) {
    std::cout << "Мёртв\n";
} else if (health < 30) {
    std::cout << "Тяжёлое ранение\n";
} else if (health < 70) {
    std::cout << "Ранен\n";
} else {
    std::cout << "В порядке\n";
}
```

Проверки идут **сверху вниз**. Срабатывает первая истинная. Остальные не проверяются — важно, если в условиях есть вычисления.

Кстати, `else if` в C++ — это не отдельная конструкция, а просто `else` со вложенным `if`. То есть код выше эквивалентен:

```
if (health <= 0) {  
    ...  
} else {  
    if (health < 30) {  
        ...  
    } else {  
        if (...) { ... }  
    }  
}
```

Но первая форма читается лучше, поэтому все так и пишут.

10.3 Логические операции в условиях

Условие — это любое выражение, которое можно интерпретировать как `bool`. Самые частые операции:

```
==    // равно  
!=    // не равно  
<     // меньше  
<=    // меньше или равно  
>     // больше  
>=    // больше или равно  
&&    // логическое И  
||    // логическое ИЛИ  
!     // логическое НЕ
```

Примеры:

```
if (age >= 18 && age < 65) {  
    std::cout << "Работоспособный возраст\n";  
}  
  
if (day == "суббота" || day == "воскресенье") {  
    std::cout << "Выходной\n";  
}  
  
if (!alive) {
```

```
std::cout << "He жив\n"; // эквивалентно alive == false
}
```

10.3.1 Ловушка номер один: = vs ==

Это самый частый баг новичка в C++ (и в C, и в Java, и в JS).

= — это **присваивание**. `x = 5` — положить 5 в переменную `x`.

== — это **сравнение**. `x == 5` — проверить, равен ли `x` пятёрке.

В условии нужно сравнение. Если случайно написать одно равно:

```
if (health = 0) { // ОЙ!
    ...
}
```

— то компилятор не выдаст ошибку. Он подумает: «Ага, программист хочет положить 0 в `health`, а потом проверить, что присвоено». В присваивании результат — само значение, то есть 0. Ноль интерпретируется как `false`. Условие никогда не сработает. И ещё в `health` уляглись 0 — побочный эффект.

Очень коварный баг. Современные компиляторы это ловят:

```
warning: using the result of an assignment as a condition without parentheses
```

— но только с флагами `-Wall`. Включайте флаги.

Есть приём защиты — **сравнение задом наперёд**, когда константа слева:

```
if (0 == health) { // если случайно один знак — ошибка компиляции
    ...
}
```

Если написать `0 = health` — ошибка: нельзя присваивать константе. Но эта запись неестественная (читается «если ноль равен `health`» — задом наперёд). В современном C++ за неё не голосуют. Просто будьте внимательны и используйте флаги.

10.3.2 Short-circuit evaluation

Очень важная особенность: операторы `&&` и `||` **сокращают вычисления**.

Для `A && B`: если `A` ложно — `B` **не вычисляется**. Результат всё равно ложь.

Для `A || B`: если `A` истинно — `B` не вычисляется. Результат всё равно истина.

Зачем это полезно? Защита от ошибок:

```
if (ptr != nullptr && ptr->value > 0) {  
    ...  
}
```

Здесь, если `ptr` нулевой — мы **не** обращаемся к `ptr->value` (что было бы UB и крашем программы). Сначала проверили, что `ptr` не пуст, и только потом разыменовали.

То же самое с делением:

```
if (b != 0 && a / b > 5) {  
    ...  
}
```

Деление на ноль не произойдёт, потому что `b != 0` ложь $\rightarrow$ `a / b` не вычисляется.

Помните этот порядок — он спасает кучу багов.

10.3.3 Скобки и приоритет

Когда операторов несколько, могут возникать вопросы порядка:

```
if (a == 5 || b == 5 && c == 5) {  
    ...  
}
```

Что тут? `(a == 5) || ((b == 5) && (c == 5))`? Или `((a == 5) || (b == 5)) && (c == 5)`?

В C++ `&&` имеет **приоритет выше** `||`. То есть `&&` «связывает крепче». Расшифровка — первая.

Я **очень рекомендую** писать скобки явно:

```
if ((a == 5) || (b == 5 && c == 5)) {  
    ...  
}
```

Скобки лишние? Лучше лишние, чем непонятные. Особенно когда условие сложное.

10.4 Дополнительный конструктор: `if` с инициализацией (C++17)

С C++17 можно объявлять переменную прямо в условии:

```
if (int x = compute(); x > 0) {  
    use(x);  
}  
  
// x здесь уже не виден
```

Это C++17, и в нашей C++11-базе **не работает**. Но удобная штука, упоминаю на будущее.

В C++11 эквивалент:

```
{
    int x = compute();
    if (x > 0) {
        use(x);
    }
}
// x здесь не виден
```

10.5 Тернарный оператор

Краткая форма if/else для простого выбора между двумя значениями:

```
условие ? значение_если_истина : значение_если_ложь
```

Пример:

```
int abs_value = (x < 0) ? -x : x;
```

Эквивалент:

```
int abs_value;
if (x < 0) {
    abs_value = -x;
} else {
    abs_value = x;
}
```

Тернарник полезен в выражениях, где хочется выбрать значение:

```
std::cout << "Урон: " << (is_critical ? damage * 2 : damage) << "\n";
```

С тернарником без () вокруг — может быть некрасиво:

```
int x = a > b ? a : b;    // максимум, читается ок
```

Но злоупотреблять не стоит. Когда условие сложное или выбираемых значений много — лучше обычный if/else. Тернарник без скобок легко спутать с другими операторами:

```
int y = x > 0 ? 1 : x < 0 ? -1 : 0;    // SIGNUM, но читать тяжело
```

Лучше переписать в if/else if/else — будет понятнее.

10.6 switch — выбор из множества значений

Когда у вас много альтернатив на одну и ту же переменную:

```
char direction;
std::cin >> direction;

switch (direction) {
    case 'N':
        std::cout << "Идешь на север\n";
        break;
    case 'S':
        std::cout << "Идешь на юг\n";
        break;
    case 'E':
        std::cout << "Идешь на восток\n";
        break;
    case 'W':
        std::cout << "Идешь на запад\n";
        break;
    default:
        std::cout << "Неизвестное направление\n";
        break;
}
```

Эквивалент через if/else if:

```
if (direction == 'N') {
    ...
} else if (direction == 'S') {
    ...
} else if (...) {
    ...
} else {
    ...
}
```

switch — другая форма того же. Когда вариантов много и все они проверяют одну переменную — switch обычно читается лучше. Плюс, компилятор иногда генерирует **более быстрый код** через таблицу переходов (jump table) вместо последовательных проверок. Не всегда — это оптимизация, и её делает компилятор по своему усмотрению.

10.6.1 Ограничения switch

В switch можно проверять **только целочисленные и перечислимые** типы (int, char, enum, и т. д.). **Нельзя** — double, std::string, объекты классов. Поэтому такое **не сработает**:

```
std::string cmd;
std::cin >> cmd;

switch (cmd) {                                // ОШИБКА компиляции
    case "look": ...
    case "take": ...
}
```

Для строк используют if/else if.

Сами case-значения должны быть **constexpr-выражениями**:

```
const int MAX = 10;
constexpr int MIN = 1;

switch (x) {
    case MAX: ...    // ОК, MAX это const int с известным значением
    case MIN: ...    // ОК, constexpr
    case y:   ...    // ОШИБКА, y не известно на компиляции
}
```

10.6.2 Fall-through — главная ловушка switch

Заметили break после каждого case? Если его убрать — выполнение «просачивается» вниз к следующему case. Это называется fall-through, и оно унаследовано из C.

```
switch (x) {
    case 1:
        std::cout << "one ";
        // нет break – ВНИМАНИЕ
    case 2:
        std::cout << "two ";
        break;
    case 3:
        std::cout << "three ";
        break;
}
```

При `x == 1` напечатается `one two`! Потому что после `case 1` нет `break`, выполнение продолжается в `case 2`.

Чаще всего `fall-through` — это **баг** (программист забыл `break`). Иногда — фича, когда несколько `case` должны делать одно:

```
switch (cmd) {
    case 'h':
    case 'H':
    case '?':
        show_help();
        break;
    case 'q':
    case 'Q':
        quit();
        break;
}
```

Здесь `fall-through` сознательный — обрабатываем верхний и нижний регистр одной веткой. В C++17 для явности добавили атрибут `[[fallthrough]]`:

```
case 1:
    do_one();
    [[fallthrough]]; // явно говорим: «знаю что нет break, это специально»
case 2:
    do_two();
    break;
```

В C++11 такого нет — обходимся комментарием `// fall-through`. Современные компиляторы могут выдавать предупреждение про `fall-through` (`-Wimplicit-fallthrough`), и принимают комментарии в качестве «глушилки».

10.6.3 default

`default` — что делать, если ни один `case` не подошёл. Опционально, но **очень полезно**. Если все случаи перечислены — поставьте `default` с сообщением об ошибке или `assert`:

```
switch (state) {
    case State::Idle:    ...; break;
    case State::Active: ...; break;
    case State::Done:   ...; break;
    default:
```

```
std::cerr << "Unknown state\n";
std::abort();
}
```

Так, если в будущем добавите `State::Error`, но забудете `case` — программа упадёт с понятной ошибкой, а не «странно поведёт себя». Современные компиляторы с `-Wswitch` ругаются, если для `enum` пропущен `case`.

10.7 while — цикл с проверкой в начале

```
while (условие) {
    действия;
}
```

Сначала проверяется условие. Если истинно — выполняются действия, потом снова проверка, и так далее. Когда условие становится ложным — выход.

Если условие изначально ложно — тело не выполнится ни разу.

```
int health = 100;
while (health > 0) {
    health = health - 10;
    std::cout << "HP: " << health << "\n";
}
std::cout << "Игрок умер\n";
```

Циклы — главный источник багов. Самые частые проблемы:

Бесконечный цикл. Условие никогда не становится ложным. Например:

```
int i = 0;
while (i < 10) {
    std::cout << i << "\n";
    // забыли увеличить i — цикл вечный
}
```

Программа печатает `0 0 0 0 ...` бесконечно. Прервите через `Ctrl+C`.

Off-by-one error (ошибка на единицу). Цикл выполняется на один раз больше или меньше, чем надо. Классика — `<` vs `<=`:

```
for (int i = 0; i < 10; ++i) {    // 10 итераций: i = 0, 1, ..., 9
    ...
}
```

```
}  
for (int i = 0; i <= 10; ++i) {    // 11 итераций: i = 0, 1, ..., 10  
    ...  
}
```

Программисты привыкли к первой форме ($< n$), потому что она даёт ровно n итераций и в индексной нотации совпадает с длиной массива.

Изменение неправильной переменной. Особенно во вложенных циклах:

```
int i = 0;  
while (i < 10) {  
    int j = 0;  
    while (j < 5) {  
        std::cout << i << ", " << j;  
        i = i + 1;    // Ой! Меняем i во внутреннем цикле  
        j = j + 1;  
    }  
}
```

Внешний цикл сходит с ума, потому что i меняется внутри. Часто такие баги — результат копипасты.

10.7.1 Бесконечный цикл нарочно

Иногда хочется именно вечный цикл (внутри будет `break` по какому-то условию):

```
while (true) {  
    auto cmd = read_command();  
    if (cmd == "quit") break;  
    execute(cmd);  
}
```

Это идиома REPL (read-eval-print loop). Сервер тоже обычно крутится в `while (true)`.

Эквивалентная запись — `for (;;) :`

```
for (;;) {  
    ...  
}
```

Это тот же бесконечный цикл, just другой синтаксис. Используется реже, но встречается в чужом коде.

10.8 do-while — цикл с проверкой в конце

```
do {  
    действия;  
} while (условие);
```

В отличие от `while`, тело выполняется **минимум один раз** — сначала действие, потом проверка.

```
int answer;  
do {  
    std::cout << "Введите число 1-10: ";  
    std::cin >> answer;  
} while (answer < 1 || answer > 10);
```

Программа просит ввести число, пока не введут что-то в диапазоне. Сначала всегда спрашивает, потом проверяет.

Точка с запятой **после** `while (...)` обязательна — `do-while` это особая конструкция.

`do-while` нужен редко. Большинство случаев лучше переписываются в обычный `while`. Я в своей практике пишу его, может быть, раз в год — когда хочется именно «выполнить и проверить».

10.9 for — классический цикл

Самая частая форма цикла в C/C++:

```
for (инициализация; условие; шаг) {  
    действия;  
}
```

Эквивалент через `while`:

```
инициализация;  
while (условие) {  
    действия;  
    шаг;  
}
```

Классический пример — перебор от 0 до N-1:

```
for (int i = 0; i < 10; ++i) {  
    std::cout << i << "\n";  
}
```

Разбор по частям: - `int i = 0` — выполняется один раз, перед циклом. - `i < 10` — проверяется перед каждой итерацией. Если истина — выполняется тело. - `++i` — выполняется после каждой итерации тела. Увеличивает `i` на 1.

`++i` — это **инкремент**, увеличение на 1. Чуть короче и быстрее, чем `i = i + 1` (для встроенных типов разницы практически нет, для итераторов класса — лучше `++i`).

Есть ещё `i++` — постфиксный инкремент. На встроенных типах в условии цикла **разницы нет**. Но в общем случае: - `++i` — увеличить и вернуть **новое** значение. - `i++` — запомнить старое значение, увеличить, вернуть **старое**.

Префиксный `++i` для итераторов классов чуть быстрее (не делает копию), поэтому в `for`-циклах я пишу `++i`. Это вошло в привычку у всего C++-сообщества.

10.9.1 Все три части — опциональны

Можно опустить любую часть:

```
int i = 0;
for (; i < 10; ++i) {           // инициализации нет
    ...
}

for (int i = 0; ; ++i) {        // условия нет — бесконечный цикл
    if (...) break;
    ...
}

for (int i = 0; i < 10;) {      // шага нет — должен меняться внутри
    ...
    i = i + 2;
}

for (;;) {                      // вечный цикл
    ...
}
```

Чаще всего я пишу классическую полную форму. Опущения — редкая экзотика.

10.9.2 Несколько переменных в `for`

В части инициализации можно объявить несколько переменных:

```
for (int i = 0, j = 10; i < j; ++i, --j) {  
    std::cout << i << " " << j << "\n";  
}
```

Печатает 0 10, 1 9, 2 8, 3 7, 4 6. Останавливается, когда *i* догоняет *j*.

В шаге запятая — это **оператор запятая**, выполняющий несколько выражений по очереди. Используется редко, но иногда удобно для одновременного движения двух переменных.

10.10 Range-based for — главная новинка C++11

Классический `for` хорош, но если нужно просто пройти по всем элементам контейнера — много букв. В C++11 добавили краткую форму:

```
std::vector<int> v = {10, 20, 30, 40, 50};  
  
for (int x : v) {  
    std::cout << x << "\n";  
}
```

Читается «для каждого *x* из *v*». Без индексов, без счётчиков. Эквивалент классического:

```
for (size_t i = 0; i < v.size(); ++i) {  
    int x = v[i];  
    std::cout << x << "\n";  
}
```

Range-based `for` работает для любого, у кого есть `begin()` и `end()` (методы или функции). Это все стандартные контейнеры (`vector`, `array`, `map`, `set`, `string`), а также массивы:

```
int arr[] = {1, 2, 3, 4, 5};  
for (int x : arr) {  
    std::cout << x << "\n";  
}
```

10.10.1 По ссылке vs по значению

В записи `for (int x : v)` *x* — это **копия** элемента. Менять *x* бесполезно — оригинал не изменится:

```
std::vector<int> v = {1, 2, 3};  
for (int x : v) {  
    x = x * 2;    // меняет копию, не оригинал
```



```
}  
// v всё ещё {1, 2, 3}
```

Чтобы менять оригинал — берём по ссылке (&):

```
for (int& x : v) {  
    x = x * 2;  
}  
// v теперь {2, 4, 6}
```

Если контейнер хранит большие объекты (std::string, классы), копировать дорого — берите хотя бы константную ссылку:

```
std::vector<std::string> names = {"Alice", "Bob", "Charlie"};  
  
// Плохо: каждый name копируется (стоит выделения памяти и копирования содержимого)  
for (std::string name : names) {  
    std::cout << name << "\n";  
}  
  
// Хорошо: константная ссылка, без копий  
for (const std::string& name : names) {  
    std::cout << name << "\n";  
}
```

С auto запись становится ещё короче:

```
for (const auto& name : names) {  
    std::cout << name << "\n";  
}
```

Это идиома, которую вы будете писать постоянно. Запоминайте: **for (const auto& x : container)**.

10.10.2 Если нужен индекс — классический for

Range-based for не даёт индекс. Если он нужен (например, чтобы напечатать «элемент номер 3»):

```
for (size_t i = 0; i < v.size(); ++i) {  
    std::cout << i << ": " << v[i] << "\n";  
}
```

(`size_t` — стандартный беззнаковый тип для размеров и индексов; обычно эквивалент `unsigned long`. Можно использовать `int`, но `-Wsign-compare` тогда ругнётся; см. главу 8 — лучше `signed` для индексов всё-таки, но `size_t` — общепринятая практика для итерирования.)

В C++20 появилось `std::views::enumerate`, дающее одновременно индекс и значение. До тех пор — классический `for` с индексом.

10.11 `break` и `continue`

`break` — **выход** из ближайшего цикла или `switch`. Мы видели его в `switch`. В цикле:

```
for (int i = 0; i < 100; ++i) {
    if (found_what_we_want(i)) {
        break;    // выходим из цикла
    }
}
```

`continue` — **переход к следующей итерации** того же цикла. Текущая итерация прерывается:

```
for (int i = 0; i < 10; ++i) {
    if (i % 2 == 0) {
        continue;    // пропускаем чётные
    }
    std::cout << i << "\n";    // печатает только нечётные
}
```

Что важно: `break` и `continue` влияют **только на ближайший цикл**. Если их вызвать из вложенного цикла — выйдут только из внутреннего:

```
for (int i = 0; i < 10; ++i) {
    for (int j = 0; j < 10; ++j) {
        if (found) {
            break;    // выходит ТОЛЬКО из внутреннего for
        }
    }
    // сюда попадаем после break — продолжается внешний цикл
}
```

Это иногда раздражает — хочется выйти сразу из двух циклов. Варианты решения:

1. Флаг:

```
bool done = false;
for (int i = 0; i < 10 && !done; ++i) {
    for (int j = 0; j < 10; ++j) {
        if (found) {
            done = true;
            break;
        }
    }
}
```

Работает, но проверка `&& !done` каждую итерацию — некрасиво.

2. Выделить в функцию + return:

```
auto search = []() -> std::pair<int,int> {
    for (int i = 0; i < 10; ++i) {
        for (int j = 0; j < 10; ++j) {
            if (found) return {i, j};
        }
    }
    return {-1, -1};
};
```

Иногда самый чистый вариант. Лямбды разберём в главе 21.

3. goto. Seriously. Об этом ниже.

10.12 goto — реабилитация чёрной овцы

В каждой книге по программированию пишут «никогда не используйте goto». Это идёт из эссе Дейкстры 1968 года «Goto Considered Harmful». Эссе было верным **в контексте того времени** — программы на ассемблере и Fortran с goto превращались в спагетти, в которых ничего нельзя было понять.

В современном структурированном коде у goto остались два-три **законных** применения, и я о них расскажу.

Синтаксис goto простой:

```
goto метка;
...
```

```
метка:
    действие;
```

goto перепрыгивает на строку с меткой. Всё. Прыгать можно только внутри одной функции, нельзя пересекать инициализации переменных (компилятор не разрешит).

Применение 1: выход из вложенных циклов.

```
for (int i = 0; i < 10; ++i) {
    for (int j = 0; j < 10; ++j) {
        if (found) goto done;
    }
}
done:
    std::cout << "Found at " << i << "," << j << "\n";
```

Чисто и без лишних флагов. В C++ этот паттерн считается приемлемым.

Применение 2: освобождение ресурсов на ошибке (C-стиль).

```
int* a = malloc(...);
int* b = malloc(...);
int* c = malloc(...);

if (something_failed) {
    goto cleanup;
}

// ... использование ...

cleanup:
    free(c);
    free(b);
    free(a);
```

В C это типичный паттерн «единственная точка очистки». В C++ для этого есть **гораздо лучшие средства** — RAII (мы разберём в главе 11), деструкторы, smart pointers. Поэтому в C++ этот паттерн почти не нужен. Но если читаете старый C-код или общаетесь с C-API — будете видеть.

Что нельзя делать с goto:

- Прыгать вперёд через инициализацию переменной (компилятор не даст).

- Прыгать в другую функцию (только внутри одной).
- Использовать как «удобный if» — `if (...) goto err;` в обычном потоке. Это создаёт путаницу.

В нашей книге `goto` встретится максимум один-два раза — в местах, где он реально лучше альтернатив.

10.13 Циклы с итераторами (предварительно)

В C++ есть концепция **итераторов** — обобщённого способа перебирать элементы. Это основа STL. Подробно — в главе 14, но для полноты картины упомяну:

```
std::vector<int> v = {10, 20, 30};

for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << "\n";    // *it — это значение по итератору
}
```

`v.begin()` — итератор на первый элемент. `v.end()` — итератор «за последний» (sentinel). `++it` — перейти к следующему. `*it` — получить значение.

В C++11 такой цикл превращается в краткий range-based for, и писать его руками не надо. Но иногда классический for-с-итератором нужен — например, когда хочется иметь итератор для модификации контейнера прямо во время прохода. К этому вернёмся.

10.14 Что выучить про управляющие конструкции

Конструкция	Когда брать
if/else	Простое ветвление, 1-3 ветки.
else if-цепочка	Несколько альтернатив. Если их много и они на одну переменную — рассмотрите switch.
Тернарный ?:	В выражениях, выбор простого значения. Не больше одного на строку.
switch	Много веток на одну целочисленную/enum переменную. Не забывайте break!
while	Цикл с предусловием. Когда количество итераций неизвестно.
do-while	Цикл с пост-условием. Когда нужно гарантировать хотя бы один проход.

Конструкция	Когда брать
for (классический)	Цикл с известным числом итераций (от и до). Когда нужен индекс.
Range-based for	Перебор всех элементов контейнера. Самый частый цикл в C++11+.
break	Выйти из ближайшего цикла или switch.
continue	Перейти к следующей итерации.
goto	Выход из вложенных циклов. В редких случаях.

10.15 Пара рекомендаций

1. **Всегда фигурные скобки** `{...}` **после** `if/else/for/while`, даже если тело — одна строка.
2. **Всегда break в case** (или явный `[[fallthrough]]`/комментарий).
3. **default в switch** — даже если думаете, что не нужен. Будущие изменения добавят варианты.
4. **Range-based for с const auto&** для перебора. Запомните идиому.
5. **Включите флаги** `-Wall -Wextra -Wpedantic` — компилятор найдёт половину ловушек этой главы.

10.16 Пример: «угадай число»

Соберём что-то живое из всего изученного. Простая игра: компьютер загадывает число от 1 до 100, игрок угадывает. Программа подсказывает «больше» или «меньше».

```
#include <iostream>
#include <cstdlib>    // для rand и srand
#include <ctime>      // для time

int main() {
    // Инициализируем генератор случайных чисел текущим временем
    std::srand(static_cast<unsigned>(std::time(nullptr)));

    constexpr int min_value = 1;
    constexpr int max_value = 100;
    constexpr int max_attempts = 10;
```

```
int target = std::rand() % (max_value - min_value + 1) + min_value;

std::cout << "Угадай число от " << min_value << " до " << max_value << "\n";
std::cout << "У тебя " << max_attempts << " попыток.\n";

bool won = false;

for (int attempt = 1; attempt <= max_attempts; ++attempt) {
    std::cout << "\nПопытка " << attempt << "/" << max_attempts << ": ";

    int guess;
    std::cin >> guess;

    if (guess < min_value || guess > max_value) {
        std::cout << "Число вне диапазона. Попробуй ещё.\n";
        continue;
    }

    if (guess == target) {
        std::cout << "Угадал! Это " << target << ".\n";
        won = true;
        break;
    } else if (guess < target) {
        std::cout << "Загаданное больше.\n";
    } else {
        std::cout << "Загаданное меньше.\n";
    }
}

if (!won) {
    std::cout << "\nПопытки кончились. Это было " << target << ".\n";
}

return 0;
}
```

Соберите:

```
$ g++ -std=c++11 -Wall -Wextra guess.cpp -o guess
$ ./guess
```

Что тут использовано: - `constexpr` для констант (глава 8). - `static_cast` для явной конверсии (глава 8). - `for` с условием и шагом. - `if/else if/else` для трёх веток сравнения. - `continue` для пропуска неверного ввода. - `break` для выхода при выигрыше. - Логическое НЕ (`!won`).

Поиграйте, попробуйте сломать (введите букву вместо числа — что будет?), потом исправьте.

10.17 Маленькое упражнение

1. Напишите программу, которая печатает таблицу умножения 10×10 :

```
1  2  3 ...
2  4  6 ...
3  6  9 ...
```

Подсказка: вложенный `for`.

2. Напишите программу, которая читает числа из `stdin` до нуля и считает сумму:

```
$ ./sum
5
3
7
0
Сумма: 15
```

Подсказка: `while (true) { ... if (x == 0) break; }`.

3. Напишите программу, которая находит первое простое число больше `N` (`N` — ввод пользователя).
4. Напишите программу-калькулятор: на вход — два числа и операция (`+` `-` `*` `/`), на выход — результат. Используйте `switch` по операции.
5. (Сложнее) Напишите программу, которая для каждого числа от 1 до 100:

- если делится на 3 — печатает «Fizz»;
- если на 5 — «Buzz»;
- если и на 3, и на 5 — «FizzBuzz»;
- иначе — само число.

Это классическая задача FizzBuzz на собеседованиях.

Если не получается с первого раза — нормально. Если получается с третьего — отлично. Если получилось всё с первого — впечатляет. В любом случае запускайте и проверяйте каждую программу.

10.18 Что дальше

Глава 10 — про функции. Объявление и определение, передача параметров (по значению, по ссылке, по указателю), перегрузка, значения по умолчанию, `inline`, рекурсия и `stack overflow`. После этой главы у вас будет 90% инструментов для написания простых программ — без классов, но уже с собственными модулями. А ещё через две главы (память и RAII) подойдём к тому, чтобы стартовать первый проект.

Глава 11

Глава 10. Функции

С функциями вы уже сталкивались — `main`, `std::cout`, и пара случайных штук. Эта глава — про то, как функции писать самим. Это огромная тема, потому что в C++ передача аргументов в функцию устроена сильно иначе, чем в Python или Java. У вас будет три способа передачи (по значению, по ссылке, по указателю), и каждый со своим смыслом. Я буду долго на этом останавливаться — это центральная вещь языка, и без её понимания дальше будет хаос.

11.1 Зачем нужны функции

В главе 4 мы уже обсудили зачем — переиспользование, декомпозиция. Повторю кратко: функция позволяет один раз написать какой-то кусок логики и потом вызывать его сколько угодно раз. Это бережёт код от копи-пасты, делает структуру программы понятной и упрощает поиск ошибок (если ошибка в функции — правка одна, не пятнадцать).

Когда программа разрастается до тысяч строк — без функций жить нельзя. Хорошее правило: **если кусок кода вы скопировали два раза — пора вынести в функцию.**

11.2 Анатомия функции

Вот простейшая функция:

```
int square(int x) {  
    return x * x;  
}
```

Разбираем по частям.

- **int** в самом начале — **тип возвращаемого значения** (return type). Функция отдаёт обратно `int`. Если функция ничего не возвращает — пишут `void` (про это ниже).

- **square** — **имя функции**. Те же правила, что для переменных (буквы, цифры, подчёркивание; не начинается с цифры; не зарезервированное слово).
- **(int x)** — **список параметров**. У нашей функции один параметр типа `int` по имени `x`. Параметров может быть много, или ноль.
- **{ ... }** — **тело функции**. Код, который выполняется при вызове.
- **return x * x;** — возвращает значение из функции. Управление сразу переходит к месту вызова.

Вызов:

```
int result = square(5);    // result = 25
```

Программа доходит до строки `int result = square(5);`, перепрыгивает в тело `square`, выполняет его (умножает 5 на 5, возвращает 25), потом возвращается обратно, и `result` получает значение 25.

11.2.1 Несколько параметров

```
int max(int a, int b) {  
    if (a > b) return a;  
    else      return b;  
}
```

Можно иметь два, три, четыре параметра. У каждого свой тип и своё имя:

```
double distance(int x1, int y1, int x2, int y2) {  
    double dx = x2 - x1;  
    double dy = y2 - y1;  
    return std::sqrt(dx * dx + dy * dy);  
}
```

(Здесь использован `std::sqrt` из `<cmath>` — квадратный корень.)

11.2.2 void — функция без возврата

Если функция не должна ничего возвращать — указывайте `void`:

```
void print_hello() {  
    std::cout << "Hello!\n";  
}
```

Вызов:

```
print_hello();    // ничего не присваиваем
```

В void-функции можно писать `return;` (без значения) — это выход. Если функция дошла до конца тела без `return` — она тоже завершается, всё нормально:

```
void greet(const std::string& name) {  
    if (name.empty()) {  
        return;    // выход досрочно  
    }  
    std::cout << "Hello, " << name << "!\n";  
}
```

`return` в void-функции с значением — ошибка компиляции.

11.2.3 Функция без параметров

Если параметров нет — пишут пустые скобки:

```
int get_random() {  
    return std::rand();  
}
```

В C++ `()` означает «параметров нет», в отличие от C, где `()` означает «параметры неизвестны»; в C для «нет параметров» пишут `(void)`. В C++ обе формы работают, но `(...)` без `void` — стандарт.

11.3 Объявление и определение

В главе 7 я мельком говорил: **объявление** функции — это обещание её существования, **определение** — это собственно её код.

Объявление:

```
int square(int x);
```

(Точка с запятой в конце, тела нет.)

Определение:

```
int square(int x) {  
    return x * x;  
}
```

(Полное, с телом.)

Зачем разделять? Чтобы можно было использовать функцию **до того, как она написана**. Компилятор работает сверху вниз: дойдя до строки `square(5)`, он должен уже знать, что такое `square`. Если функция определена ниже — компилятор не найдёт.

Поэтому обычно файл выглядит так:

```
#include <iostream>

// объявления вверху
int square(int x);
int max(int a, int b);

// определения ниже
int main() {
    std::cout << square(5) << "\n";
    std::cout << max(3, 7) << "\n";
    return 0;
}

int square(int x) {
    return x * x;
}

int max(int a, int b) {
    return a > b ? a : b;
}
```

Альтернатива — определить функции **до** `main`:

```
int square(int x) {
    return x * x;
}

int max(int a, int b) {
    return a > b ? a : b;
}

int main() {
    std::cout << square(5) << "\n";
    std::cout << max(3, 7) << "\n";
}
```

```
    return 0;  
}
```

Если функция определена до использования — отдельное объявление не нужно. Каждое определение **само по себе** служит объявлением.

11.3.1 Разделение между файлами

В программе из нескольких файлов схема такая. Допустим, у нас функция `square` нужна и в `main.cpp`, и в `tests.cpp`.

`math.h`:

```
#ifndef MATH_H  
#define MATH_H  
  
int square(int x);    // объявление  
  
#endif
```

`math.cpp`:

```
#include "math.h"  
  
int square(int x) {    // определение  
    return x * x;  
}
```

`main.cpp`:

```
#include "math.h"  
  
int main() {  
    return square(5);  
}
```

`tests.cpp`:

```
#include "math.h"  
  
void run_tests() {  
    if (square(3) != 9) ...  
}
```

Когда `main.cpp` и `tests.cpp` компилируются, они оба видят объявление `square` (через `#include "math.h"`). Компилятор не возражает — он не требует тела на этапе компиляции. Тело берётся при **линковке**, из `math.cpp`. Линкер находит реальную `square` в `math.o` и подставляет.

Это разделение «объявление в `.h`, определение в `.cpp`» — стандартный паттерн в C++. Заголовочный файл — публичный интерфейс модуля. Исходный файл — реализация.

11.4 Передача параметров: три способа

Вот теперь главное место главы. Когда мы пишем:

```
int square(int x) {
    return x * x;
}

int main() {
    int n = 5;
    int r = square(n);
}
```

Что происходит с `n` при вызове `square(n)`? Передаётся **сама переменная** `n`? Или её копия? Или ссылка на неё?

В C++ есть **три** способа передачи параметров. Каждый со своим синтаксисом, своим смыслом, своей ценой.

11.4.1 Способ 1: по значению

Самый простой и самый частый. Параметр — обычная переменная.

```
void increment(int x) {
    x = x + 1;
    std::cout << "Внутри функции x = " << x << "\n";
}

int main() {
    int a = 5;
    increment(a);
    std::cout << "После вызова a = " << a << "\n";
}
```

Вывод:

Внутри функции $x = 6$

После вызова $a = 5$

а не изменилась! Хотя мы написали $x = x + 1$ внутри функции — менялось значение x , а x — это **копия** значения a , отдельная переменная. Внутри функции получили копию, поработали с ней, копия исчезла при возврате.

Это **передача по значению** (pass by value). Главные свойства:

- Параметр — это **новая переменная**, инициализированная значением аргумента.
- Изменения параметра внутри функции **не видны снаружи**.
- Происходит **копирование**. Для `int` — копия одного слова, мгновенно. Для большого `std::vector<std::string>` — копия всего содержимого, может быть мегабайтами.

Передача по значению — основная по умолчанию для «маленьких» типов: `int`, `double`, `bool`, `char`, маленькие структуры.

11.4.2 Способ 2: по ссылке

Если хотим, чтобы функция меняла оригинал — берём параметр **по ссылке**:

```
void increment(int& x) {           // обратите внимание: int& x, а не int x
    x = x + 1;
}

int main() {
    int a = 5;
    increment(a);
    std::cout << "a = " << a << "\n";    // a = 6 !
}
```

Знак `&` после типа параметра — **это ссылка**. `int& x` означает « x — это другое имя для переменной, которая будет передана при вызове». Внутри функции x и снаружи a — это **одна и та же переменная**, доступная под двумя именами. Никаких копий.

Когда вы пишете $x = x + 1$ — меняется именно a .

Ссылка — это **псевдоним**. Аналогии: - В русском языке у Петра Ильича Чайковского можно сказать «Чайковский» или «Петя» — это одно лицо. Ссылка — это «Петя», обычное имя — «Чайковский». - В файловой системе бывают символические ссылки — два пути, ведущие к одному файлу.

11.4.3 Ключевые свойства ссылок

1. **Ссылка должна быть инициализирована при создании.** Нельзя `int& r;` без значения — ошибка.

```
int x = 5;
int& r = x;    // ОК: r это псевдоним x
int& r2;       // ОШИБКА: ссылка должна быть инициализирована
```

2. **Ссылка нельзя «перенаправить» на другую переменную.** После создания она навсегда указывает на ту же.

```
int x = 5;
int y = 10;
int& r = x;    // r это x
r = y;         // НЕ перенаправление! Это присваивание: x теперь равен 10
```

В отличие от указателей (ниже) — ссылка после создания «приклеена» намертво.

3. **К ссылке относятся как к самой переменной.** `r * 2` — это `x * 2`. `++r` — это `++x`. И так далее.

11.4.4 По ссылке для эффективности

Главная причина передавать **большие** объекты по ссылке — **не делать копию**:

```
void print(const std::vector<int>& v) {    // const ссылка — нет копии, нет модификации
    for (int x : v) {
        std::cout << x << "\n";
    }
}
```

`const std::vector<int>&` — это «константная ссылка на вектор». Без копирования (вектор может быть мегабайтным!), и при этом гарантия: внутри функции его не изменят.

Правило: если параметр — большой тип (`std::string`, `std::vector`, класс), и функция его не меняет — передавайте `const T&`. Не `T` (будет копия), не `T&` (без `const` — функция может «случайно» поменять).

Маленькие типы (`int`, `double`, `bool`) — по значению. Копия дешёвая, и компилятор всё равно положит копию в регистр.

11.4.5 Способ 3: по указателю

Указатель — это переменная, в которой лежит **адрес** другой переменной. Это уже не псевдоним, а явная «координата» другой переменной в памяти.

```
void increment(int* p) {           // int* — указатель на int
    *p = *p + 1;                   // *p — значение по адресу p
}

int main() {
    int a = 5;
    increment(&a);                 // &a — адрес a
    std::cout << "a = " << a << "\n"; // a = 6
}
```

Тут много нового. Разбираем.

- **int*** в типе параметра — это «указатель на int». То есть «адрес какой-то ячейки памяти, в которой лежит int».
- **&a** при вызове — оператор «**взять адрес**». Возвращает адрес переменной a. Тип — int*.
- ***p** внутри функции — оператор «**разыменовать**». Возвращает значение по адресу p. То есть a.
- ***p = *p + 1** — взять значение по адресу p, прибавить 1, и положить обратно по тому же адресу. То есть меняем a.

11.4.6 Указатель vs ссылка

Указатели и ссылки делают **похожую** работу — позволяют функции работать с оригиналом, а не с копией. Отличия:

1. **Указатель — это число (адрес)**. Ссылка — псевдоним. Указатель занимает 8 байт (на 64-битной системе), ссылка — формально ноль (компилятор её обычно реализует как тот же указатель, но это деталь).
2. **Указатель может быть nullptr** — «никуда не указывающий». Ссылку нельзя инициализировать «ничем».
3. **Указатель можно перенаправлять** — `p = &b;` меняет, на что указывает. Ссылку — нельзя.
4. Для вызова с указателем нужен **&** у вызывающего и ***** для разыменования — синтаксис заметнее. Со ссылкой — вызов выглядит как обычный.

5. **Указатели — наследие С.** Они везде в С-коде и в системных API. Ссылки — изобретение С++, в С их нет.

11.4.7 Когда что использовать

Если **обязательно** есть что передать (значение точно есть) и функция должна его менять или эффективно прочитать — **ссылка**. Кратко, безопасно.

Если параметр **может отсутствовать** (тогда передаём `nullptr`) или нужно по С-API — **указатель**.

```
// "Дай мне строку, я её обработаю":
void process(const std::string& s);

// "Дай мне строку, ИЛИ ничего":
void process(const std::string* s) {
    if (s == nullptr) {
        std::cout << "Нет строки\n";
    } else {
        // обработка
    }
}
```

В современном С++17+ для «опционального» параметра обычно используют `std::optional`, но до этого далеко. На С++11 — либо указатель, либо специальные значения.

11.4.8 Сводная таблица

Способ	Синтаксис	Можно менять оригинал?	Копирование?
По значению	<code>void f(int x)</code>	Нет	Да
По ссылке	<code>void f(int& x)</code>	Да	Нет
По const-ссылке	<code>void f(const int& x)</code>	Нет	Нет
По указателю	<code>void f(int* p)</code>	Да	Нет
По указателю на const	<code>void f(const int* p)</code>	Нет	Нет

11.4.9 Правило передачи параметров

Я в начале книги обещал, что буду давать «правила без иффов». Вот оно:

1. **Маленькие типы** (`int`, `double`, `bool`, `char`, маленькие структуры до ~16 байт) — **по значению**.

2. **Большие типы** для чтения — **const T&** (по const-ссылке).
3. **Большие типы для модификации** — **T&** (по ссылке). Думайте дважды; часто лучше принять копию и вернуть результат.
4. **Указатели** — только когда нужен nullptr или для C-API.

Этим правилом обходитесь в 95% случаев.

11.5 Возврат из функции

С возвратом обычно проще, чем с параметрами. По умолчанию — возврат по значению, копия:

```
std::string make_greeting(const std::string& name) {  
    return "Hello, " + name + "!";  
}  
  
int main() {  
    std::string g = make_greeting("Alice");  
}
```

Здесь возвращается копия строки. На больших объектах это могло бы быть дорого, но **компилятор почти всегда оптимизирует** через RVO/NRVO (return value optimization) — копии не делается, объект сразу строится в нужном месте. Об этом подробно в главе 19 (move semantics).

Можно возвращать ссылку:

```
int& get_element(std::vector<int>& v, int index) {  
    return v[index];  
}  
  
int main() {  
    std::vector<int> nums = {1, 2, 3};  
    get_element(nums, 1) = 99;    // меняет nums[1] на 99  
}
```

Опасный приём! Если вернуть ссылку на **локальную переменную** функции — после возврата эта переменная исчезает, ссылка становится «висячей» (dangling), и обращение к ней — UB:

```
int& bad() {  
    int local = 42;  
    return local;    // ОПАСНО: local исчезает, ссылка указывает в никуда  
}
```

```
int x = bad();    // x получает мусор или программа крашится
```

Современные компиляторы обычно ругаются на такое (`-Wreturn-local-addr`), но не всегда ловят. Поэтому правило: **никогда не возвращайте ссылку или указатель на локальную переменную**.

Возвращать ссылку безопасно, когда: - Вы возвращаете ссылку на параметр. - Вы возвращаете ссылку на `static`-переменную. - Вы возвращаете ссылку на член класса (об этом в главе 16).

11.5.1 Несколько возвращаемых значений

В C++ функция возвращает **только одно** значение. Если хочется два — есть несколько вариантов.

1. Через ссылочные параметры:

```
void divmod(int a, int b, int& quotient, int& remainder) {
    quotient = a / b;
    remainder = a % b;
}

int main() {
    int q, r;
    divmod(17, 5, q, r);    // q = 3, r = 2
}
```

2. Через `std::pair`:

```
#include <utility>

std::pair<int, int> divmod(int a, int b) {
    return std::make_pair(a / b, a % b);
}

int main() {
    auto result = divmod(17, 5);
    std::cout << result.first << " " << result.second;
}
```

`std::pair<int, int>` — стандартная пара. `result.first` — первое значение, `result.second` — второе.

3. Через `std::tuple` (C++11) для трёх и больше:

```
#include <tuple>

std::tuple<int, int, int> get_rgb() {
    return std::make_tuple(255, 128, 64);
}

int main() {
    int r, g, b;
    std::tie(r, g, b) = get_rgb();
}
```

В C++17 появились structured bindings — `auto [q, r] = divmod(17, 5);` — но это не наша база.

4. Через свою структуру — часто чище всех:

```
struct DivResult {
    int quotient;
    int remainder;
};

DivResult divmod(int a, int b) {
    return {a / b, a % b};
}

int main() {
    DivResult r = divmod(17, 5);
    std::cout << r.quotient;
}
```

Я в книге буду использовать в основном именованные структуры — они яснее.

11.6 Перегрузка функций

В C++ можно иметь **несколько функций с одинаковым именем**, если у них разные параметры:

```
int max(int a, int b) {
    return a > b ? a : b;
}
```

```
double max(double a, double b) {  
    return a > b ? a : b;  
}  
  
int max(int a, int b, int c) {  
    return max(max(a, b), c);  
}
```

Три функции с именем `max`. Компилятор по типам аргументов выбирает нужную:

```
max(3, 5);           // int max(int, int)  
max(3.14, 2.71);     // double max(double, double)  
max(1, 2, 3);        // int max(int, int, int)
```

Это **перегрузка** (overloading). Помните манглинг имён из главы 7? Именно для этого компилятор кодирует типы в имена символов — чтобы при линковке три функции `max` были разными `_Z3maxii`, `_Z3maxdd`, `_Z3maxiii`.

Перегрузка различает функции по: - Количеству параметров. - Типам параметров. - `const`-у на параметрах (но обычно не имеет смысла).

Перегрузка **не** различает функции по: - Типу возвращаемого значения. **Это важно!** Нельзя `int f(int); double f(int);` — компилятор не поймёт, какую из них вы хотите.

Перегрузка хороша, когда логически функции делают «одно и то же» с разными типами. Например, `max`, `min`, `print`, `to_string`. Плоха, когда вы пишете перегрузку, чтобы скрыть разную логику под одним именем — путает читателя.

11.7 Параметры по умолчанию

Можно задать значение **по умолчанию** для параметра:

```
void greet(const std::string& name, const std::string& greeting = "Hello") {  
    std::cout << greeting << ", " << name << "!\n";  
}  
  
int main() {  
    greet("Alice");           // Hello, Alice!  
    greet("Bob", "Welcome");  // Welcome, Bob!  
}
```

Параметры по умолчанию указываются **с конца**. Нельзя `void f(int a = 5, int b)` — компилятор не поймёт, как вызывать `f(7)` (это `b`? или `a`?).

Если функция объявлена и определена отдельно — параметры по умолчанию **только в объявлении**, не в определении:

```
// math.h
int sum(int a, int b, int c = 0);

// math.cpp
int sum(int a, int b, int c) {    // здесь = 0 НЕ повторять
    return a + b + c;
}
```

Параметры по умолчанию хороши, когда у функции «обычный» случай и редкие особые. Например, в большинстве вызовов вы хотите стандартное приветствие — `greet("Bob")` короче, чем `greet("Bob", "Hello")`. Плохо, когда параметров много и непонятно, какие по умолчанию — лучше отдельные функции или структура с параметрами.

11.8 inline

Когда функция очень короткая (одна-две строки), у вызова есть **накладные расходы**: положить аргументы в регистры/на стек, перепрыгнуть, выполнить, вернуться. Для крохотной функции это может быть дольше самого тела.

Решение — **inline-функция**. Это намёк компилятору: «не делай вызов, подставь тело прямо в место вызова».

```
inline int square(int x) {
    return x * x;
}

int main() {
    int r = square(5);
    // компилятор подставит это как:
    // int r = 5 * 5;
}
```

Это называется **инлайнинг** (inlining). После него — никакого вызова функции, просто выражение.

Тонкость: `inline` в современном C++ — **слабая** рекомендация. Компилятор сам решает, инлайнить или нет, и без `inline` тоже может. Но `inline` важен по другой причине — **он разрешает определение в заголовочном файле**:

```
// math.h
#ifndef MATH_H
#define MATH_H

inline int square(int x) {
    return x * x;
}

#endif
```

Без `inline` это привело бы к ошибке `multiple definition` при линковке — потому что заголовок включается в несколько `.cpp`, и компилируется в каждый. С `inline` линкер знает: «это определение, оно может быть в нескольких файлах одинаковое, выбери одно».

В простых случаях — пишите функцию в `.cpp`, не в `.h`. Если выяснилось, что для шаблона или производительности нужно в `.h` — добавьте `inline`.

11.9 Рекурсия

Функция может вызывать **сама себя**. Это **рекурсия**:

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

`factorial(5)` вызывает `factorial(4)`, который вызывает `factorial(3)`, который вызывает `factorial(2)`, который вызывает `factorial(1)`. Последний возвращает 1, дальше — $2 * 1$, $3 * 2$, $4 * 3$, $5 * 24 = 120$.

Главное в рекурсии — **base case** (базовый случай). Условие выхода, при котором функция возвращается без рекурсивного вызова. У нас это `if (n <= 1) return 1;`. Без `base case` рекурсия бесконечная.

Каждый рекурсивный вызов кладёт на стек новый кадр (как мы обсуждали в главе 7). Размер стека ограничен (обычно 8 MB на Linux/macOS, 1 MB на Windows). Когда рекурсия слишком глубокая — **stack overflow**, программа падает.

```
int infinite_recursion(int x) {  
    return infinite_recursion(x);    // никогда не остановится  
}
```

На моей машине это падает где-то на 200 000-й итерации.

Рекурсия — мощный инструмент. Многие задачи естественно рекурсивные: - Обход дерева: посетить корень, потом рекурсивно обход левого поддерева, потом правого. - Алгоритм поиска в файловой системе: открыть папку, для каждого элемента — если файл, обработать; если папка, рекурсивно зайти. - Алгоритмы делящие задачу пополам — quicksort, mergesort, binary search.

Но рекурсии не везде стоит верить. Иногда итеративная версия (через цикл) быстрее и не рискует переполнить стек:

```
int factorial(int n) {  
    int result = 1;  
    for (int i = 2; i <= n; ++i) {  
        result *= i;  
    }  
    return result;  
}
```

Это эквивалентно рекурсивной версии, но без стека. Для факториала такая форма обычно лучше.

11.9.1 Хвостовая рекурсия

Один особый вид рекурсии — **хвостовая**: рекурсивный вызов — последняя операция в функции. Например:

```
int sum_recursive(int n, int acc) {  
    if (n == 0) return acc;  
    return sum_recursive(n - 1, acc + n);  
}  
  
// sum_recursive(100, 0) даёт сумму 1+2+...+100 = 5050
```

В хвостовой рекурсии нет «работы» после рекурсивного вызова. Поэтому теоретически компилятор может заменить такой вызов прыжком — без расхода стека. Это называется **tail call optimization (TCO)**. C++-компиляторы её обычно делают (с включёнными оптимизациями), но не гарантируют. Языки типа Scheme, Haskell — гарантируют. В C++ для гарантированной безстековой рекурсии нужно переписать в цикл.

11.10 Forward declaration

Иногда две функции должны вызывать друг друга:

```
bool is_even(int n);    // forward declaration

bool is_odd(int n) {
    if (n == 0) return false;
    return is_even(n - 1);
}

bool is_even(int n) {
    if (n == 0) return true;
    return is_odd(n - 1);
}
```

`is_odd` использует `is_even`, который определён ниже. Без объявления вверху компилятор бы выдал ошибку «`is_even not declared`». Объявление-«обещание» снимает проблему.

(Алгоритм этот — учебный, в реальной жизни `n % 2 == 0` куда проще. Зато он показывает паттерн взаимной рекурсии.)

11.11 main — особая функция

`main` — точка входа в программу. Особенности:

- **Возвращает `int`.** Это код выхода: 0 — успех, ненулевой — ошибка.
- **Опускать `return 0`;** разрешено — компилятор подставит сам. Я предпочитаю писать явно.
- **Может принимать аргументы командной строки:**

```
int main(int argc, char* argv[]) {
    std::cout << "Количество аргументов: " << argc << "\n";
    for (int i = 0; i < argc; ++i) {
        std::cout << "  argv[" << i << "] = " << argv[i] << "\n";
    }
}
```

```
    }  
    return 0;  
}
```

`argc` (argument count) — сколько аргументов передано (включая имя программы). `argv` (argument vector) — массив строк (на самом деле — массив `char*`-указателей).

```
$ ./program hello world 42
```

Количество аргументов: 4

```
    argv[0] = ./program  
    argv[1] = hello  
    argv[2] = world  
    argv[3] = 42
```

Мы будем это использовать в проектах — например, `./chat-server --port 9001` или `./grep "pattern" file.txt`.

11.12 Лямбды — короткое упоминание

В C++11 появились **лямбды** — функции, объявляемые «прямо на месте»:

```
auto add = [](int a, int b) { return a + b; };  
std::cout << add(3, 4);    // 7
```

Это огромная тема, мы её разберём в главе 21 на проекте RPG. Сейчас просто знайте — такая штука есть.

11.13 Указатели на функции

Можно сохранить функцию в переменную и вызвать через неё:

```
int add(int a, int b) { return a + b; }  
int sub(int a, int b) { return a - b; }  
  
int main() {  
    int (*op)(int, int);    // указатель на функцию, принимающую (int, int) и возвращающую int  
    op = add;  
    std::cout << op(5, 3) << "\n";    // 8  
    op = sub;  
    std::cout << op(5, 3) << "\n";    // 2  
}
```

Синтаксис уродливый. Обычно вместо него используют `std::function` (про неё в главе 21) или `auto`:

```
auto op = add;    // тип выводится
op(5, 3);
```

Указатели на функции редко нужны в современном C++. Но они есть, и в чужом коде встречаются.

11.14 Чистые функции и побочные эффекты

Функция **чистая** (pure), если: - Её результат зависит **только от параметров**. - Она **не меняет** ничего снаружи (никаких глобалов, никакого ввода-вывода, ничего).

```
int square(int x) { return x * x; }    // чистая
```

Чистая, потому что: возвращает $x*x$ (зависит только от x), ничего не меняет.

Не чистая:

```
int counter = 0;

int increment_and_return() {
    counter = counter + 1;    // меняет глобал
    return counter;
}
```

Вызов два раза подряд даёт разные результаты — функция «помнит» состояние.

Чистые функции хороши: - **Тестируются легко** — нет внешних зависимостей. - **Параллелятся** — нет общего состояния, безопасно запускать в разных потоках. - **Кэшируются** — если результат уже посчитан, можно его запомнить. - **Читаются** — поведение полностью описано сигнатурой.

В C++ принудить функцию быть чистой нельзя (нет такого ключевого слова), но писать **по возможности чисто** — хороший стиль. Особенно для «вычислительных» функций. С побочными эффектами лучше работать в маленьких островках; остальной код пусть будет чистым.

11.15 Большой пример: банковская симуляция

Соберём всё в осмысленную программу. Сделаем подобие банка: счета и операции над ними.

```
#include <iostream>
#include <string>

struct Account {
    std::string owner;
    double balance;
};

// Чистая функция: вычисляет процент
double calculate_interest(double balance, double rate) {
    return balance * rate;
}

// Изменяет счёт (передача по ссылке)
void deposit(Account& account, double amount) {
    account.balance = account.balance + amount;
}

// Возвращает true если перевод прошёл, false если денег не хватило
bool transfer(Account& from, Account& to, double amount) {
    if (from.balance < amount) {
        return false;
    }
    from.balance = from.balance - amount;
    to.balance = to.balance + amount;
    return true;
}

// Только читает (const-ссылка)
void print(const Account& account) {
    std::cout << "Счёт " << account.owner << ": " << account.balance << "\n";
}

// Перегрузки – для разных типов суммы
void print(const Account& account, double amount) {
    std::cout << "Счёт " << account.owner << ": " << account.balance
        << " (сумма операции: " << amount << ")\n";
}
```

```
int main() {
    Account alice{"Alice", 1000.0};
    Account bob{"Bob", 500.0};

    print(alice);
    print(bob);

    std::cout << "\nДепозит 200 на счёт Alice\n";
    deposit(alice, 200);
    print(alice);

    std::cout << "\nПеревод 300 от Alice к Bob\n";
    if (transfer(alice, bob, 300)) {
        std::cout << "OK\n";
    } else {
        std::cout << "Недостаточно средств\n";
    }
    print(alice);
    print(bob);

    std::cout << "\nПеревод 999999 от Alice к Bob\n";
    if (transfer(alice, bob, 999999)) {
        std::cout << "OK\n";
    } else {
        std::cout << "Недостаточно средств\n";
    }
    print(alice);

    constexpr double annual_rate = 0.05;
    double interest = calculate_interest(alice.balance, annual_rate);
    std::cout << "\nГодовые Alice по ставке " << annual_rate
                << ": " << interest << "\n";

    return 0;
}
```

Соберите:

```
$ g++ -std=c++11 -Wall -Wextra bank.cpp -o bank
$ ./bank
```

Тут использовано: - `struct` — для определения типа `Account` (узнаем подробнее в главе 16). - Передача по значению (`double amount`). - Передача по ссылке (`Account& from`). - Передача по const-ссылке (`const Account& account`). - Перегрузка (две функции `print`). - Чистая функция (`calculate_interest`). - Возврат `bool` для индикации успеха.

11.16 Маленькое упражнение

1. Напишите функцию `int gcd(int a, int b)` — наибольший общий делитель. Алгоритм Евклида: `gcd(a, 0) = a`, `gcd(a, b) = gcd(b, a % b)`. Рекурсивно.
2. Перепишите `gcd` итеративно (без рекурсии).
3. Напишите функцию `void swap(int& a, int& b)` — обмен значениями. Проверьте, что работает.
4. Напишите функцию `int sum(int n)`, которая возвращает $1 + 2 + \dots + n$. Сделайте две версии: через цикл и через формулу $n * (n + 1) / 2$. Сравните: какая быстрее? Какая понятнее?
5. Напишите функцию `bool is_prime(int n)` — простое ли число. Подсказка: проверять делимость до `sqrt(n)`.
6. Напишите перегрузку `print` для `int`, `double`, `std::string`. Каждая выводит свой тип с подписью «integer: X», «double: Y», «string: Z».
7. Напишите функцию `int read_int()`, которая запрашивает у пользователя число с проверкой ввода (если не число — спросить снова).

11.17 Что дальше

Глава 11 — про **память**: что лежит на стеке, что на куче, `new` и `delete`, утечки памяти, и главный принцип C++ — RAII (Resource Acquisition Is Initialization). После этой главы вы поймёте, **почему** в C++ всё так сложно с указателями и ссылками, и **зачем** придумали `smart pointers` (про них уже на проекте RPG, глава 18). Это самая концептуально тяжёлая глава Части I, но и самая полезная. Готовьтесь.

Глава 12

Глава 11. Память — стек, куча, RAII

Это самая важная глава Части I. После неё вы поймёте, почему C++ устроен именно так — со ссылками, указателями, конструкторами, деструкторами. Поймёте, откуда берутся «утечки памяти», «висячие указатели», «двойное освобождение» — и как современный C++ их предотвращает. Без этого знания дальше можно писать только маленькие игрушки; на больших программах вы будете наткаться на загадочные крэши и недоумевать.

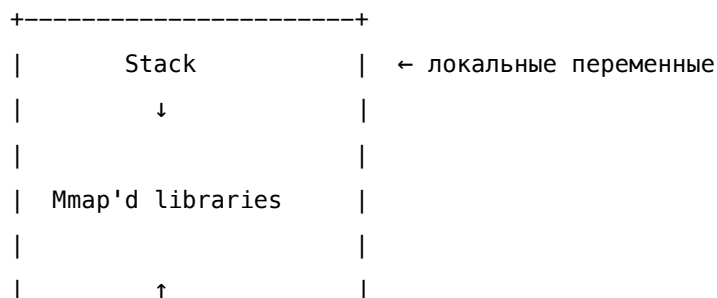
Главное на эту главу: **в C++ программист сам отвечает за память**. В Python, Java, JavaScript есть сборщик мусора — он сам находит ненужные объекты и удаляет. В C++ — нет. Если вы взяли память — вы её и возвращаете. Если забыли — утечка. Если вернули дважды — крэш. Если используете после возврата — UB.

Этой ручной работой можно было бы заняться лично, но за десятилетия C++ выработал паттерн **RAII**, который перекладывает работу на компилятор. RAII — главное изобретение C++, отличающее его от C. Об этом и поговорим.

12.1 Освежим картинку из главы 7

В главе 7 я показывал layout памяти процесса:

Высокие адреса



	Heap		← динамически выделяемая
+-----+			
	.bss		← глобальные неинициализированные
	.data		← глобальные инициализированные
	.rodata		← константы
	.text		← код
+-----+			

Низкие адреса

Из всех областей в этой главе нас интересуют две — **стек** и **куча**. Глобальные данные (.data, .bss) — это редкость в хорошем C++-коде, серьёзный разговор о них не нужен. Код (.text, .rodata) — read-only, программа в них не пишет.

Стек и куча — два главных места жизни данных. Различия в одной фразе: **стек** — **автоматически**, **куча** — **вручную**.

12.2 Стек: подробности

Каждая функция при вызове **получает кусок стека** — называется **stack frame** (кадр стека). В этом кадре лежат: - Параметры функции. - Локальные переменные. - Адрес возврата (куда прыгать после return). - Сохранённые регистры (если нужно).

Когда функция возвращается — её кадр **исчезает**. Локальные переменные, которые в нём лежали, перестают существовать. Если у переменной был деструктор (об этом ниже) — он вызывается прямо в момент исчезновения кадра.

Пример:

```
void foo() {
    int x = 5;
    int y = 10;
    // x и y лежат в кадре foo
    // адреса близкие: &x и &y отличаются на несколько байт
}
// после возврата кадр исчезает, x и y больше нет

int main() {
    foo();    // создаётся кадр foo, потом исчезает
    foo();    // снова создаётся новый кадр на тех же адресах, потом исчезает
}
```

Если бы внутри `foo` была переменная, которую мы хотели использовать после `foo` — её бы не было. Кадр уничтожается.

12.2.1 Стек растёт «вниз»

По соглашению почти всех современных архитектур стек **растёт от высоких адресов к низким**. Когда вы вызываете функцию, указатель стека (`rsp` на x86-64, `sp` на ARM64) **уменьшается** на размер кадра. Когда функция возвращается — увеличивается обратно.

Это особенность исторически такая. С точки зрения программы это неважно — стек просто «есть».

12.2.2 Стек ограничен

Размер стека по умолчанию: - **Linux**: 8 MB (можно изменить через `ulimit -s`). - **macOS**: 8 MB для главного потока, 512 KB для дополнительных. - **Windows**: 1 MB.

Если кадры функций займут больше — **stack overflow**, программа упадёт. Это бывает в двух случаях.

1. Слишком глубокая рекурсия (про это в главе 10).

```
int recurse(int n) {
    return recurse(n + 1);    // бесконечная рекурсия
}
// упадёт где-то на 100–200 тысячах вызовов
```

2. Огромные локальные переменные.

```
void bad() {
    int big_array[10'000'000]; // 40 MB на стеке!
    // ...
}
```

40 MB не помещается ни в 8 MB, ни в 1 MB. Программа упадёт ещё до выполнения тела функции. Решение — переместить такой массив в **кучу** (об этом ниже).

12.2.3 Преимущества стека

- **Бесплатное создание и уничтожение.** Никаких системных вызовов, никакого поиска свободного места. Просто перемещение указателя стека на N байт. Это **одна инструкция процессора**.
- **Автоматическое управление.** Программист не думает, когда освобождать — это делается само при выходе из функции.

- **Кэш-дружелюбность.** Локальные переменные функции близки в памяти, скорее всего попадают в одну кэш-линию.
- **Безопасность.** Нет утечек, нет двойного освобождения. Стек сам знает свою жизнь.

12.2.4 Недостатки стека

- **Размер ограничен.** Большие массивы туда нельзя.
- **Размер фиксирован на этапе компиляции.** Нельзя сказать «создай мне массив размером N, где N — переменная». В стандартном C++ — нельзя. (В C99 разрешены VLA — массивы переменной длины, и GCC поддерживает их как расширение для C++. Но в стандартном C++ — нет.)
- **Жизнь привязана к функции.** Если нужно, чтобы объект пережил функцию — на стеке нельзя.

12.3 Куча: подробности

Куча — большая область памяти, из которой программа может выделять кусочки **во время работы**. Размер выделения определяется в рантайме. Жизнь куска **не привязана** к функции — он живёт, пока вы его явно не вернёте.

В C++ есть два уровня работы с кучей: - **Низкий:** оператор `new`, оператор `delete`. Аналоги `malloc/free` из C. - **Высокий:** smart pointers (`std::unique_ptr`, `std::shared_ptr`) и стандартные контейнеры (`std::vector`, `std::string`, ...). Они сами зовут `new/delete` внутри, программисту думать не надо.

В современном коде используют высокий уровень. Низкий — для понимания, как это работает, и для редких случаев. Я разберу низкий уровень детально (это нужно понимать), а потом покажу высокий и объясню, почему он лучше.

12.3.1 new — выделение

Чтобы выделить из кучи память для одного объекта:

```
int* p = new int;           // выделить место для int, p — адрес
*p = 42;                    // положить 42
std::cout << *p << "\n";   // напечатать 42
```

Что произошло:

1. Оператор `new` пошёл к ОС (через библиотеку: `malloc` → `mmap/brk`) и попросил «дай мне 4 байта» (размер `int`).
2. ОС нашла свободный кусок в куче и вернула его адрес.

3. Этот адрес положили в `p` (тип `int*`).
4. Теперь `*p` — это значение по этому адресу. Записали 42.

С инициализацией значения сразу при создании:

```
int* p = new int(42);      // выделить и положить 42
int* q = new int{42};     // то же, через uniform initialization (C++11)
```

Для классов это вызывает соответствующий конструктор — об этом в главе 16.

12.3.2 delete — возврат

Когда объект больше не нужен — обязательно верните память:

```
delete p;
```

Это говорит: «я закончил с памятью по адресу `p`, верни её в общий пул». ОС/менеджер памяти помечает кусок как свободный, и его смогут переиспользовать следующие `new`.

После `delete` указатель **становится опасным**. Адрес тот же, но память уже не «наша». Использовать `*p` после `delete p` — UB:

```
int* p = new int(42);
delete p;
*p = 0;      // UB! Память уже не наша
delete p;    // UB! Двойное освобождение
```

Хорошая привычка — после `delete` присвоить указатель `nullptr`:

```
delete p;
p = nullptr;    // теперь *p вызовет краш сразу, а не UB
delete p;      // delete nullptr — безопасно, ничего не делает
```

12.3.3 new[] и delete[] — массивы

Чтобы выделить массив объектов:

```
int* arr = new int[100];    // массив из 100 int
arr[0] = 1;
arr[99] = 100;

delete[] arr;              // вернуть, ВНИМАНИЕ — delete[], не delete
```

Никогда не путайте `delete` и `delete[]`: - `new int` → `delete` (одиночный). - `new int[N]` → `delete[]` (массив).

Если перепутать — UB. Самое противное: на простых типах (`int`, `float`) это часто «работает», но протекает память. На сложных типах с деструкторами (`std::string[]`) — краш.

12.3.4 Размер можно вычислять в рантайме

В отличие от стека, размер на куче **может быть переменным**:

```
int n;
std::cin >> n;
int* arr = new int[n];    // OK! n может быть любым

for (int i = 0; i < n; ++i) {
    arr[i] = i * i;
}

delete[] arr;
```

Это то, чего не сделать на стеке стандартным C++. Куча — единственный способ.

12.4 Утечки памяти

Утечка — это когда вы выделили память через `new`, но **никогда не освободили** через `delete`. Память остаётся занятой, но программа её больше не видит — потеряна.

```
void leak() {
    int* p = new int(42);
    // забыли delete
}

int main() {
    for (int i = 0; i < 1'000'000; ++i) {
        leak();
    }
    // программа потребляет ~4 MB лишней памяти
}
```

Один пропуск `delete` в коде, который выполняется миллион раз — миллион утечек. Программа толстеет, начинает занимать гигабайты, ОС начинает свапиться, всё тормозит, в худшем случае — OOM-killer убивает программу.

12.4.1 Когда утечки нестрашны

Парадокс: иногда утечки **не имеют значения**.

- Если программа **выполняется секунду и закрывается** — ОС всё равно освободит её память при выходе. Утечка в `main` в маленькой утилите — никому не вредит.
- Если программа выделила что-то и удерживает до самого конца (а конец близок) — тоже не страшно.

Опасны утечки в **долгоживущих** процессах: серверы, демоны, мобильные приложения. Там каждая утечка остаётся навсегда, и через месяц работы — гигабайты лишнего.

12.4.2 Пять способов утечь память

Я набрал из практики пять типичных:

1. Забыл `delete`.

```
void process() {  
    int* p = new int(42);  
    use(p);  
    // нет delete  
}
```

2. Исключение между `new` и `delete`.

```
void process() {  
    int* p = new int(42);  
    might_throw();    // исключение!  
    delete p;        // не выполнится  
}
```

3. Ранний `return`.

```
int* find_or_create() {  
    int* p = new int(42);  
    if (some_condition) {  
        return p;    // вернули, вызывающий должен delete  
    }  
    if (other_condition) {  
        return nullptr; // Ой: p потеряно  
    }  
    return p;  
}
```

4. Перезапись указателя.

```
int* p = new int(42);
p = new int(43);      // адрес 42-го объекта потерян, утечка
delete p;             // удаляет только 43-й
```

5. Циклы между `shared_ptr` — про это в главе 18. Сейчас просто упомяну: даже с smart pointers можно утечь, если структура объектов закольцованная.

12.4.3 Висячие указатели

Обратная проблема: `delete`-нули объект, но указатель остался. Использование такого указателя — UB.

```
int* p = new int(42);
int* q = p;      // q тоже указывает туда

delete p;        // память освобождена
p = nullptr;

*q = 0;          // q — висячий указатель! UB!
```

Сложнее с возвратом из функции:

```
int* make_int() {
    int x = 42;
    return &x;    // !!! &x — адрес локальной переменной
}

int main() {
    int* p = make_int(); // p висячий: x уничтожен
    *p = 0;             // UB
}
```

Современные компиляторы это ловят (`-Wreturn-local-addr`), но не всегда.

12.4.4 Двойное освобождение

```
int* p = new int(42);
delete p;
delete p;      // UB: второе освобождение
```


Что произойдёт — зависит от рантайма. На glibc вы получите красивое сообщение «double free or corruption». На других — UB, может быть SEGFAULT, может быть тихая катастрофа.

12.4.5 Использование после освобождения (UAF)

```
int* p = new int(42);
delete p;
std::cout << *p;    // UB: use-after-free
```

Это золотая жила багов безопасности — большинство уязвимостей в браузерах и ядрах ОС в 2010-2020-х годах связаны с UAF.

12.5 Проблема: слишком много ручной работы

Получается, программисту нужно: - Помнить о каждом new — где он, кто делает delete. - Освобождать на всех путях выхода из функции (включая исключения). - Не использовать указатель после delete. - Не освобождать дважды.

На больших программах это нереально. Поэтому в C — постоянная борьба с утечками. В C++ нашли решение.

12.6 RAII — главный паттерн C++

RAII — Resource Acquisition Is Initialization, «получение ресурса есть инициализация». Идея, кажущаяся простой, но меняющая всё:

Привязать жизнь ресурса к жизни объекта на стеке.

Когда объект создаётся (на стеке) — он **захватывает** ресурс. Когда объект уничтожается (выходит из области видимости) — он **освобождает** ресурс.

Стек уничтожает объекты автоматически. Значит, ресурс автоматически освобождается. Никаких ручных delete. Никаких ошибок «забыл освободить».

Это работает для любого ресурса, не только памяти: файлов, мьютексов, сокетов, окон, чего угодно.

12.6.1 Конструктор и деструктор

Чтобы это работало, у объекта должно быть два специальных метода:

- **Конструктор** — выполняется при создании объекта.
- **Деструктор** — выполняется при уничтожении.

Подробно про эти штуки в главе 16 на проекте RPG. Сейчас — лёгкий обзор.

Простой пример. Класс, который сам управляет одним динамически выделенным `int`:

```
class IntHolder {
public:
    // Конструктор: создаём объект, выделяем int
    IntHolder(int value) {
        ptr_ = new int(value);
        std::cout << "Created at " << ptr_ << "\n";
    }

    // Деструктор: освобождаем
    ~IntHolder() {
        std::cout << "Destroying at " << ptr_ << "\n";
        delete ptr_;
    }

    int get() const { return *ptr_; }
    void set(int v) { *ptr_ = v; }

private:
    int* ptr_;
};

int main() {
    IntHolder a(42);
    IntHolder b(100);
    std::cout << a.get() << " " << b.get() << "\n";
} // здесь автоматически вызываются деструкторы a и b
```

Вывод:

```
Created at 0x600003a04020
Created at 0x600003a04030
42 100
Destroying at 0x600003a04030
Destroying at 0x600003a04020
```

Заметили: я не написал ни одного `delete` в `main`. Память освобождается **сама**, потому что деструктор `IntHolder` это делает, и деструктор вызывается автоматически при выходе из `main`.

Деструкторы вызываются в **обратном порядке создания** — это гарантия языка, и она важна, когда объекты зависят друг от друга.

Это и есть RAII в чистом виде. Объект на стеке владеет ресурсом на куче, и его деструктор разруливает уборку.

12.6.2 Почему RAII работает с исключениями

Главная сила RAII — она устойчива к исключениям.

```
void might_throw();

void process() {
    IntHolder x(42);
    might_throw();    // если выкинет исключение — деструктор x всё равно вызовется!
}
```

При возникновении исключения C++ выполняет **stack unwinding** (раскрутка стека): по очереди вызываются деструкторы всех объектов в текущем кадре, потом в кадре функции выше, и так далее до места, где исключение поймают. Утечек нет.

Сравните с прямой работой через new/delete:

```
void process() {
    int* p = new int(42);
    might_throw();    // утечка! delete не выполнится
    delete p;
}
```

Утечка. Можно обернуть в try/catch, но громоздко. RAII делает это автоматически.

12.7 Стандартные RAII-обёртки

В стандартной библиотеке десятки RAII-объектов. Самые частые:

- **std::vector<T>** — массив, который сам управляет памятью. Растёт по мере добавления, сам освобождает в деструкторе.
- **std::string** — строка, аналогично.
- **std::ifstream** / **std::ofstream** — файлы, открываемые в конструкторе и закрываемые в деструкторе.
- **std::lock_guard** — берёт мьютекс в конструкторе, отпускает в деструкторе (про потоки в главе 41).

- `std::unique_ptr<T>` — умный указатель, владеющий объектом и удаляющий его в деструкторе.
- `std::shared_ptr<T>` — умный указатель с подсчётом ссылок (`std::shared_ptr`).

С этими штуками вы **почти никогда не пишете new/delete сами**. Они делают это за вас.

12.7.1 Пример: вектор вместо ручного массива

Так делать **не** надо:

```
int* arr = new int[1000];  
// ...использование...  
delete[] arr;    // надо помнить!
```

Так — надо:

```
std::vector<int> arr(1000);  
// ...использование...  
// arr автоматически освободится
```

Короче, безопаснее, и нет потенциальной утечки.

12.7.2 Пример: файл вместо fopen

Так делать **не** надо (C-стиль):

```
FILE* f = fopen("data.txt", "r");  
if (f == nullptr) return;  
// ...использование...  
fclose(f);    // надо помнить!
```

Так — надо:

```
#include <fstream>  
  
std::ifstream f("data.txt");  
if (!f.is_open()) return;  
// ...использование...  
// f автоматически закроется
```

Меньше кода, нет вероятности забыть `fclose`.

12.8 smart pointers — предварительно

Раз стандартные контейнеры обёртки идут с готовой RAII-логикой, может, и для голого указателя кто-то сделал? Да, и это **smart pointers**.

В C++11 их три:

- `std::unique_ptr<T>` — единоличный владелец. Только один указатель на объект; при уничтожении — удаляет.
- `std::shared_ptr<T>` — совладельцы. Объект живёт, пока есть хоть один `shared_ptr` на него; уничтожается, когда последний пропадает.
- `std::weak_ptr<T>` — наблюдатель, не владеет. Нужен против циклических ссылок `shared_ptr`.

Они подробно разбираются в главе 18 на проекте RPG. Сейчас — лёгкое знакомство.

```
#include <memory>

void use_unique() {
    std::unique_ptr<int> p(new int(42));
    std::cout << *p << "\n";
    // p автоматически удалит int
}

void use_shared() {
    std::shared_ptr<int> p(new int(42));
    auto q = p;           // оба указывают на тот же объект
    std::cout << *p << " " << *q << "\n";
    // объект удалится, когда оба p и q уничтожатся
}
```

В современном C++ почти **весь** код, имеющий дело с памятью на куче — это контейнеры и smart pointers. Ручной `new/delete` — крайне редкая экзотика, обычно только в реализации низкоуровневых структур данных или интерфейсах к C-API.

В C++14 добавили `std::make_unique`, в C++11 его нет — поэтому в нашей C++11-базе будем писать `std::unique_ptr<T>(new T(...))` или сделаем свой шим. Подробнее в главе 18.

12.9 Стек или куча — что выбрать

После всех этих описаний — простое правило:

По умолчанию — стек. Куча — только если есть конкретная причина.

Причины брать кучу:

1. **Размер заранее неизвестен.** Массив n элементов, где n ввёл пользователь.
2. **Объект должен пережить функцию.** Создаём в одной функции, отдаём наружу.
3. **Объект слишком большой для стека.** Например, видеобуфер 10 MB.
4. **Полиморфизм через базовый класс.** Передаём `Item*`, реальный объект — `Weapon` или `Armor`. Об этом в главе 17.

Во всех остальных случаях — стек. Локальная переменная, маленький массив, маленькая структура — на стеке. Это и быстрее, и безопаснее.

12.9.1 Конкретные примеры

```
// Стек:
int x = 42;
double pi = 3.14;
std::vector<int> v = {1, 2, 3};
std::string name = "Alice";
Player player(100, "Hero");

// Куча (через RAII обёртки):
std::vector<int> big_array(1'000'000); // вектор сам хранит данные на куче
std::unique_ptr<Player> hero(new Player(100, "Hero")); // когда нужен полиморфизм
auto enemies = std::vector<std::unique_ptr<Enemy>>{}; // массив указателей на разные типы врагов
```

В каждой строке стек хранит **управляющую часть** (указатель + размер для вектора, указатель для `unique_ptr`). А содержимое — на куче. Деструкторы стеков-управляющих частей освобождают кучу. Это и есть RAII в действии.

12.10 Инструменты для борьбы с проблемами памяти

Несмотря на RAII, иногда что-то протекает или указывает не туда. Есть инструменты, которые помогают находить эти баги.

12.10.1 AddressSanitizer (ASan)

Встроен в clang и gcc. Скомпилируйте программу с `-fsanitize=address`:

```
$ g++ -std=c++11 -fsanitize=address -g program.cpp -o program
$ ./program
```

ASan вставит проверки в каждое обращение к памяти. Если будет: - Использование после освобождения, - Двойное освобождение, - Выход за границы массива, - Утечка,

— программа упадёт с красивым отчётом, какая строка кода виновата.

ASan чуть замедляет программу (раза в 2), но для отладки незаменим. **Всегда** запускайте тесты под ASan.

12.10.2 Valgrind

Раньше (до ASan) самым популярным был **Valgrind** — внешняя программа, эмулирующая выполнение и проверяющая все обращения к памяти. Сейчас на Linux:

```
$ valgrind --leak-check=full ./program
```

Точнее ASan, но в 50 раз медленнее. На macOS работает плохо. Раньше — стандарт; сейчас ASan чаще.

12.10.3 LeakSanitizer (LSan)

Часть ASan. Включается флагом `-fsanitize=leak`. Просто проверяет утечки, не остальные ошибки. Полезно для CI.

12.11 Большой пример: классический ресурс

Соберём пример, который ясно показывает разницу C-стиля и C++-стиля. Класс «лог-файл»: открывает файл, пишет в него, закрывает.

12.11.1 C-стиль (без RAII)

```
class LogFile_BAD {
public:
    LogFile_BAD(const std::string& filename) {
        f_ = std::fopen(filename.c_str(), "w");
    }

    void close() {
        if (f_ != nullptr) {
```

```

        std::fclose(f_);
        f_ = nullptr;
    }
}

void write(const std::string& message) {
    if (f_ != nullptr) {
        std::fprintf(f_, "%s\n", message.c_str());
    }
}

private:
    FILE* f_;
};

void use_bad() {
    LogFile_BAD log("test.log");
    log.write("hello");
    log.close();           // программист ОБЯЗАН вспомнить close
}

```

Проблема: что если программист забудет `close()`? Или между `write` и `close` будет ранний `return`, или исключение? Файл останется открытым, дескриптор утечёт. На длинной программе можно исчерпать лимит ОС на открытые файлы.

12.11.2 C++-стиль (с RAII)

```

class LogFile {
public:
    LogFile(const std::string& filename) {
        f_ = std::fopen(filename.c_str(), "w");
        if (f_ == nullptr) {
            throw std::runtime_error("cannot open " + filename);
        }
    }

    ~LogFile() {
        if (f_ != nullptr) {

```



```

        std::fclose(f_);
    }
}

void write(const std::string& message) {
    std::fprintf(f_, "%s\n", message.c_str());
}

// Запрещаем копирование (об этом в главе 16)
LogFile(const LogFile&) = delete;
LogFile& operator=(const LogFile&) = delete;

private:
    FILE* f_ = nullptr;
};

void use_good() {
    LogFile log("test.log");
    log.write("hello");
    // close не нужен! Деструктор сделает это сам.
}

```

Этот код **физически не может** забыть закрыть файл. Что бы ни случилось — исключение, ранний return, нормальный выход — деструктор сработает и fclose будет вызван. Это и есть сила RAII.

Конструкции = delete запрещают копирование объекта. Иначе можно было бы сделать копию LogFile, и при уничтожении обоих копий случился бы двойной fclose. Об этом подробно в главе 16 (rule of five).

12.11.3 Вариант со стандартной библиотекой

Ещё короче — std::ofstream уже RAII:

```

#include <fstream>

void use_modern() {
    std::ofstream log("test.log");
    log << "hello\n";
}

```

```
// ничего больше не надо  
}
```

Стандартная библиотека уже использует RAII для большинства ресурсов. Поэтому **в реальной работе вы редко пишете деструкторы сами** — обычно завернуть в готовый `vector/string/fstream/unique_ptr` достаточно.

12.12 Когда нужно писать свой деструктор

Когда стандартные обёртки не подходят. Например:

- Работа с C-API, где нужно вызвать специфическую очистку (`SDL_FreeSurface`, `curl_easy_cleanup`, и так далее).
- Свой контейнер с особой логикой.
- Объект, владеющий несколькими ресурсами сразу.

Правило **rule of five** — если у класса есть нетривиальный деструктор, ему **скорее всего** нужны:

- Конструктор копирования, - Оператор присваивания при копировании, - Конструктор перемещения, - Оператор присваивания при перемещении.

— или явно запретить все эти операции через `= delete`.

Об этом в главе 16. Сейчас просто запомните, что свой деструктор тянет за собой много работы — поэтому **избегайте писать свой деструктор**, если можно обойтись стандартной обёрткой.

12.13 Главные правила этой главы

1. **По умолчанию — переменная на стеке.** Только если нужна гибкость или объект должен пережить функцию — куча.
2. **На куче — через RAII-обёртки.** `std::vector`, `std::string`, `std::unique_ptr`, `std::shared_ptr`. **Не** через сырой `new/delete`.
3. **Каждый ресурс — в одном объекте.** Если класс владеет несколькими ресурсами — оберните каждый в свою RAII-обёртку, не пишите общий деструктор.
4. **`new` без `delete` или со «свободным» указателем — звоночек.** Используйте `std::unique_ptr` или поместите в контейнер.
5. **Запускайте тесты под ASan.** Многие баги памяти он находит мгновенно.
6. **Помните про двойной `delete` и UAF.** Они выглядят страшнее, чем «легко исправимые опечатки» — это часто эксплуатируемые уязвимости.

12.14 Маленькое упражнение

1. Напишите класс `IntArray`, который владеет динамическим массивом `int`:

- конструктор `IntArray(size_t n)` выделяет `new int[n]`.
- деструктор `~IntArray()` делает `delete[]`.
- метод `at(size_t i)` возвращает `int&` на элемент.

Запустите программу, создающую `IntArray`, заполняющую его, и убедитесь, что деструктор зовётся.

2. Скомпилируйте программу из пункта 1 с `-fsanitize=address`. Запустите. Должно работать без жалоб.
3. Намеренно сломайте: уберите `delete[]` из деструктора. Скомпилируйте под `ASan`, запустите. `ASan` должен сказать про утечку.
4. Намеренно сломайте: вернитесь к рабочей версии, но в `main` после уничтожения `IntArray` попытайтесь обратиться к освобождённой памяти через сохранённый указатель. `ASan` должен сказать про `use-after-free`.
5. Напишите программу:

```
void deep(int n) {  
    int x = n;  
    deep(n + 1);  
    (void)x; // чтобы компилятор не выкинул  
}
```

Запустите `deep(0)`. Программа должна упасть через секунды. Если есть `ulimit -s 1024` — упадёт быстрее.

6. (Опционально, важно). Перепишите упражнение 1 через `std::unique_ptr<int[]>` (использует `delete[]` под капотом):

```
std::unique_ptr<int[]> arr(new int[100]);
```

Никакого пользовательского деструктора. Память по-прежнему освобождается. Это и есть современный C++.

12.15 Что дальше

Глава 12 — про строки (`std::string`) и потоки ввода-вывода (`std::cin/std::cout/std::ifstream`). После 11-й главы, на которой мы дотащили концептуальную базу, эти штуки уже не загадка

— `std::string` это RAII-обёртка над `char*` в куче, `std::ofstream` — RAII над `FILE*`. Дальше изученное только закрепляется.

Часть I на этой главе перевалила за середину: остались главы 12 (строки и I/O), 13 (контейнеры), 14 (алгоритмы). После них стартует Часть II — RPG-проект, и весь язык начнёт работать вместе.

Глава 13

Глава 12. Строки и потоки ВВОДА-ВЫВОДА

Программы редко обходятся без текста. Прочитать имя пользователя, вывести список товаров, открыть файл с настройками, разобрать команду «attack goblin» — всё это строки. В этой главе разберём, как с ними работать в C++.

C++ — язык с историей. Поэтому строки тут двух видов: **C-style** (наследие C, `char*` с нулевым байтом в конце) и **C++-style** (`std::string`, RAII-обёртка с современным интерфейсом). Я расскажу про обе, но писать в реальном коде будем `std::string`. Глава 11 уже дала нужный фундамент: `std::string` — это просто RAII-объект, который владеет куском памяти на куче.

13.1 Старая школа: C-strings

В C, до появления `std::string`, строкой назывался **массив `char` с нулевым байтом в конце**:

```
char str[] = "Hello";  
// в памяти: 'H' 'e' 'l' 'l' 'o' '\0'  
// размер: 6 байт (5 видимых + терминатор)
```

Терминатор `'\0'` нужен, чтобы все функции знали, где строка кончается. `strlen("Hello")` пробегает по символам до первого нуля и возвращает 5. Нет терминатора — выйдет за границы массива в чужую память, UB.

В C++ литералы `"Hello"` имеют тип `const char[6]` (массив из 6 элементов), который легко конвертируется в `const char*` (указатель на первый символ):

```
const char* msg = "Hello";
std::cout << msg << "\n";    // OK
std::cout << msg[1] << "\n"; // 'e'
```

C-строки до сих пор используются: - В системных API (`open(const char*, ...)`, `printf(const char*, ...)`). - В литералах в коде — "Hello" это C-строка, и компилятор её хранит в `.rodata`. - В производительности — иногда нужно избежать аллокации `std::string`.

Но писать **новый** код на C-строках почти никогда не стоит. Слишком много ловушек:

- **Нельзя просто склеить.** "hello" + " world" в C — UB (это сложение двух указателей).
- **Размер не известен из типа.** `char[]` теряет размер при передаче в функцию (там это уже `char*`).
- **Терминатор легко забыть.** Все функции типа `strcat`, `strcpy` пишут в чужую память, если буфер маловат.

Поэтому весь современный C++-код использует `std::string`. C-строку оставляем для интерфейса с C-API.

13.2 `std::string` — основа основ

`std::string` — это объект, который **владеет** куском памяти с текстом. Управляет им через RAII: выделяет в конструкторе, освобождает в деструкторе. Никаких ручных `delete`. Можно копировать, передавать в функции, возвращать — всё работает корректно.

Подключается через заголовок `<string>`:

```
#include <string>
```

(Обычно подтянется автоматически через `<iostream>`, но лучше явно.)

13.2.1 Создание и базовые операции

```
std::string a;                // пустая строка
std::string b = "Hello";      // из C-строки
std::string c("World");       // через конструктор
std::string d{"!"};           // uniform initialization
std::string e(5, 'x');         // "xxxxx" — 5 одинаковых символов
std::string f = b;             // копия b

// Размер
std::cout << b.size() << "\n"; // 5
```

```
std::cout << b.length() << "\n"; // 5 (синоним size)
std::cout << b.empty() << "\n"; // 0 (false)
std::cout << a.empty() << "\n"; // 1 (true)
```

`size()` возвращает количество **байтов**. Для ASCII совпадает с количеством символов. Для UTF-8 — нет (русская буква занимает 2 байта; мы говорили в главе 2).

13.2.2 Конкатенация

Главная сила `std::string` — её можно склеивать оператором `+`:

```
std::string first = "Hello";
std::string second = "World";
std::string greeting = first + ", " + second + "!"; // "Hello, World!"
```

И дописывать в существующую через `+=`:

```
std::string s = "Hello";
s += ", "; // "Hello, "
s += "World"; // "Hello, World"
s += '!'; // "Hello, World!" (даже одиночный char можно добавить)
```

Под капотом конкатенация — это **аллокация новой памяти, копирование старой строки, копирование добавляемой части, освобождение старой**. Не бесплатная операция, но обычно достаточно быстрая.

Если строите длинную строку через много `+=`, `std::string` иногда выделяет больше, чем нужно, чтобы следующие `+=` не вызывали аллокацию. Это похоже на `std::vector`. Для критичной производительности можно явно зарезервировать через `s.reserve(1000)` — выделить место под 1000 символов сразу.

13.2.3 Индексация и обход

```
std::string s = "Hello";
char c = s[0]; // 'H'
s[0] = 'J'; // s теперь "Jello"

// Обход через range-based for (C++11)
for (char ch : s) {
    std::cout << ch << "\n";
}
```

```
// Обход с индексами
for (size_t i = 0; i < s.size(); ++i) {
    std::cout << i << ": " << s[i] << "\n";
}
```

`s[i]` **не проверяет границы**. `s[100]` на 5-символьной строке — UB. Если нужна проверка с исключением — `s.at(i)`:

```
try {
    char c = s.at(100);    // выкинет std::out_of_range
} catch (const std::out_of_range& e) {
    std::cout << "out of range\n";
}
```

В цикле проверки замедлили бы код, поэтому почти всегда используют `[]`, а правильность границ обеспечивают сами.

13.2.4 Сравнение

```
std::string a = "apple";
std::string b = "banana";
std::string c = "apple";

if (a == c) ...    // true
if (a != b) ...    // true
if (a < b) ...     // true (лексикографически по байтам)
```

`==`, `!=`, `<`, `>`, `<=`, `>=` работают как ожидаешь — сравнение байт за байтом. Регистр учитывается, заглавные < строчных (по таблице ASCII).

В C-стиле сравнивать через `==` нельзя:

```
const char* a = "apple";
const char* b = "apple";
if (a == b) ...    // ОШИБКА в смысле: сравниваются указатели, не содержимое!
```

Это сравнение указателей. Чтобы сравнить содержимое, нужен `strcmp(a, b) == 0`. Ещё один повод использовать `std::string`.

13.2.5 Подстроки

```
std::string s = "Hello, World!";

std::string sub = s.substr(7, 5);    // "World" — с позиции 7, длиной 5
std::string sub2 = s.substr(7);     // "World!" — с позиции 7 до конца
```

13.2.6 Поиск

```
std::string s = "Hello, World, World!";

size_t pos = s.find("World");        // 7 — позиция первого вхождения
size_t pos2 = s.find("World", 8);    // 14 — поиск с позиции 8

if (s.find("Foo") == std::string::npos) {
    std::cout << "не найдено\n";
}
```

`std::string::npos` — специальное значение (обычно максимум `size_t`), возвращаемое при «не найдено». Запомните проверку `== npos`.

13.2.7 Замена и вставка

```
std::string s = "Hello, World!";
s.replace(7, 5, "C++");             // Hello, C++!
s.insert(0, "Greetings: ");         // Greetings: Hello, C++!
s.erase(0, 11);                     // Hello, C++!
```

Не особенно частая операция, но иногда нужна.

13.2.8 Преобразование к C-строке

Когда нужно передать в C-API, который ждёт `const char*`:

```
std::string s = "test.txt";
FILE* f = std::fopen(s.c_str(), "r");
```

`s.c_str()` возвращает `const char*` на содержимое строки, с гарантированным `'\0'` на конце. Указатель **валиден, пока строка не изменена** — после `s += "x"` он может стать невалидным. Не сохраняйте `c_str()` надолго.

13.2.9 Преобразования с числами

```
#include <string>

// Число → строка
std::string s1 = std::to_string(42);           // "42"
std::string s2 = std::to_string(3.14);         // "3.140000" (увы, форматирование кучее)

// Строка → число
int n = std::stoi("42");                       // 42
double d = std::stod("3.14");                  // 3.14
long ll = std::stoll("1234567890");            // 1234567890

// Если строка не парсится — выкинет исключение
try {
    int bad = std::stoi("not a number");
} catch (const std::invalid_argument& e) {
    std::cout << "не число\n";
}
```

`std::to_string` для дробных даёт «6 цифр после точки» — не всегда хорошо. Для приличного форматирования используют `std::stringstream` (об этом ниже).

13.2.10 Что лежит внутри `std::string`

Под капотом `std::string` обычно содержит три поля: - Указатель на буфер с символами (на куче). - Текущая длина. - Текущая ёмкость (`capacity`) — сколько символов помещается без новой аллокации.

То есть это **указатель + два размера** ≈ 24 байта на 64-битной системе. Сама строка лежит отдельно, в куче.

Современные реализации хорошо оптимизированы: для **коротких** строк (~15 символов на GCC libstdc++, ~22 на clang libc++) данные хранятся **внутри** самой `std::string`, без аллокации. Это называется **SSO** — Small String Optimization. Поэтому `std::string s = "hi"` обычно вообще не выделяет память на куче. А `std::string s = "очень длинная строка..."` — выделит.

13.2.11 Итог по строкам

- Используйте `std::string` везде.
- C-строки — только для интерфейса с C-API.

- `s.size()` — количество **байтов**, не символов (для UTF-8 будут отличаться).
- `s[i]` без проверки, `s.at(i)` с исключением.
- `==`, `<` сравнивают содержимое, а не указатели.
- Поиск через `find`, проверка `==` `npos`.
- `c_str()` — для передачи в C-API; не сохранять надолго.
- `to_string`, `stoi`, `stod` — конверсии с числами.

13.3 `std::cout` — вывод

С `std::cout` мы знакомы с главы 6. Здесь — детали.

`std::cout` — это объект класса `std::ostream`. Оператор `<<` — это **сдвиг влево**, но для потоков перегружен как «вставить в поток». Возвращает сам поток, поэтому можно цеплять:

```
std::cout << "Имя: " << name << ", возраст: " << age << "\n";
```

Это эквивалентно:

```
(((((std::cout << "Имя: ") << name) << ", возраст: ") << age) << "\n";
```

Каждый `<<` принимает поток и значение, кладёт значение в поток, возвращает поток.

`<<` поддерживает все встроенные типы плюс `std::string`, `const char*`. Для своих типов перегрузку можно написать самому — об этом в главе 16.

13.3.1 `endl` против `\n`

Что лучше:

```
std::cout << "Hello" << std::endl;  
std::cout << "Hello\n";
```

Оба выводят «Hello» с переводом строки. Разница: - `std::endl` дополнительно **сбрасывает буфер** — заставляет немедленно записать содержимое в терминал/файл. - `"\n"` — просто символ перевода строки, ничего не сбрасывает.

Когда программа выводит много, флушинг каждой строки — заметная трата. Для производительности предпочитайте `"\n"`. `std::endl` оставляйте для случаев, когда нужно немедленно показать пользователю (например, прогресс).

Современный код часто пишет `"\n"` везде, и я тоже буду так.

13.3.2 std::cerr — поток ошибок

std::cerr — отдельный поток для ошибок. По умолчанию связан с тем же терминалом, что и std::cout, но **не буферизирован**. Каждая запись сразу выводится.

```
std::cerr << "Error: cannot open file\n";
```

Зачем разделять? В Unix-философии: - std::cout — основной вывод, его можно перенаправить (./program > result.txt). - std::cerr — диагностика, она остаётся в терминале (даже после перенаправления).

```
$ ./program > out.txt
Error: cannot open file      # это std::cerr — на экране
$ cat out.txt
обычные выводы программы   # это std::cout — в файле
```

Это полезно: пользователь видит ошибки сразу, даже если стандартный вывод куда-то перенаправлен.

std::clog — ещё один поток ошибок, **буферизированный**. Используется редко.

13.3.3 Форматирование вывода

По умолчанию числа выводятся «как есть»:

```
std::cout << 3.14159265 << "\n";    // 3.14159
std::cout << 1234567 << "\n";        // 1234567
std::cout << true << "\n";           // 1
```

Чтобы управлять форматом — манипуляторы из <iomanip>:

```
#include <iomanip>

// Число знаков после запятой:
std::cout << std::fixed << std::setprecision(2) << 3.14159 << "\n";    // 3.14

// Ширина поля (выравнивание):
std::cout << std::setw(10) << 42 << "\n";          // "          42" — справа
std::cout << std::left << std::setw(10) << 42 << "\n"; // "42          " — слева

// Заполнитель:
std::cout << std::setfill('0') << std::setw(4) << 42 << "\n";    // "0042"

// Системы счисления:
```

```
std::cout << std::hex << 255 << "\n";    // ff
std::cout << std::oct << 8 << "\n";      // 10
std::cout << std::dec << 255 << "\n";    // 255 (десятичная — стандартная)

// Логические как true/false:
std::cout << std::boolalpha << true << "\n";    // true
std::cout << std::noboolalpha << true << "\n";  // 1
```

Большинство манипуляторов **остаются в силе** на следующие выводы — это «состояние потока». Если установили `std::hex` — все числа дальше будут в hex до явного `std::dec`. Это иногда подвох. Принято возвращать поток в дефолт после нестандартного формата.

Чёткое управление шириной полей — для красивых таблиц:

```
std::cout << std::left
    << std::setw(15) << "Name"
    << std::setw(5) << "Age"
    << "\n";

for (const auto& person : people) {
    std::cout << std::left
        << std::setw(15) << person.name
        << std::setw(5) << person.age
        << "\n";
}
```

Получится:

Name	Age
Alice	30
Bob	25
Charlie	42

В современном C++20 появилась `std::format` (как Python f-strings), но в C++11 её нет.

13.4 std::cin — ввод

`std::cin` — стандартный ввод. По умолчанию читает с клавиатуры.

```
int age;
std::cout << "Сколько вам лет? ";
```

```
std::cin >> age;
std::cout << "Вы ввели " << age << "\n";
```

Оператор >> («извлечь из потока»): - Пропускает ведущие пробелы и переводы строк. - Читает до первого пробела/перевода строки. - Преобразует прочитанное в нужный тип.

Если ввели нечто, что не парсится как `int` — поток переходит в **failure state**, переменная остаётся неинициализированной (или со старым значением). После `cin >> age` хорошо проверить:

```
int age;
if (std::cin >> age) {
    std::cout << "OK, возраст " << age << "\n";
} else {
    std::cout << "Не число\n";
    std::cin.clear();    // сбросить failure
    std::cin.ignore(1000, '\n'); // пропустить плохие символы до конца строки
}
```

Эта проверка работает потому, что `std::cin >> age` возвращает сам поток, который в булевом контексте даёт `true` если всё ОК, `false` если ошибка.

13.4.1 Несколько значений подряд

```
int a, b, c;
std::cin >> a >> b >> c;    // три числа, разделённых пробелами или переводами строк
```

Если ввести «3 5 7» и Enter — `a=3`, `b=5`, `c=7`. Так же сработает «3\n5\n7» — пробельные символы любые.

13.4.2 Чтение строки целиком: `getline`

`std::cin >> s` для строки прочитает до первого пробела. Если хочется прочитать **всю строку** (включая пробелы):

```
std::string name;
std::getline(std::cin, name);
```

`getline` читает до перевода строки (не включая) и сохраняет в `name`. Идеально для имён, фраз, путей.

13.4.3 Главная ловушка `cin`: смешивание » и `getline`

Самый частый баг новичка в I/O. Код такой:

```
int age;
std::string name;

std::cin >> age;           // вводят 25 + Enter
std::getline(std::cin, name); // вводят "John"

std::cout << age << " " << name;
```

Ожидаешь: 25 John. На самом деле: 25 (имя пустое!).

Почему? Когда вы вводите 25 Enter, в буфере остаётся 25\n. `cin >> age` читает 25 и **оставляет** \n в буфере. Дальше `getline` сразу видит \n — и считает строку пустой.

Решение — после `>>` явно пропустить остаток строки:

```
std::cin >> age;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, name);
```

`std::cin.ignore(N, ch)` пропускает до N символов или до встречи `ch`. С большим N (`max`) — пропустит до конца строки.

Громоздко, но необходимо. Альтернатива — **всегда** читать через `getline` и парсить руками:

```
std::string line;
std::getline(std::cin, line);
int age = std::stoi(line);

std::string name;
std::getline(std::cin, name);
```

Этот стиль безопаснее. В книге я буду чередовать в зависимости от ситуации.

13.5 Файловые потоки

Файлы открываются через `std::ifstream` (для чтения) и `std::ofstream` (для записи). Обе — RAII-обёртки, как мы видели в главе 11.

13.5.1 Чтение из файла

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream file("data.txt");
    if (!file.is_open()) {
        std::cerr << "Не удалось открыть файл\n";
        return 1;
    }

    std::string line;
    while (std::getline(file, line)) {
        std::cout << line << "\n";
    }

    return 0;
    // файл закроется автоматически в деструкторе
}
```

Проверка `if (!file.is_open())` — открыть могли не удать (нет файла, нет прав). `while (std::getline(file, line))` — стандартный паттерн чтения построчно: цикл продолжается, пока `getline` успешно читает; на конце файла `getline` ставит failure state, цикл завершается.

Также можно читать через `>>` — числа, отдельные слова:

```
std::ifstream file("numbers.txt");
int n;
while (file >> n) {
    std::cout << n * 2 << "\n";
}
```

13.5.2 Запись в файл

```
#include <fstream>

int main() {
    std::ofstream file("output.txt");
```



```
if (!file.is_open()) {  
    return 1;  
}  
  
file << "Hello, file!\n";  
file << "Number: " << 42 << "\n";  
  
return 0;  
}
```

По умолчанию `std::ofstream` открывает файл **на перезапись**: если он был — содержимое затирается. Чтобы дописать в конец:

```
std::ofstream file("log.txt", std::ios::app);  
file << "новая запись\n";
```

`std::ios::app` — флаг «append». Файл открывается так, что каждая запись добавляется в конец.

Другие флаги: - `std::ios::in` — для чтения (`ifstream` по умолчанию это и делает). - `std::ios::out` — для записи (`ofstream` по умолчанию). - `std::ios::app` — append. - `std::ios::ate` — открыть и сразу перейти в конец (для записи или чтения). - `std::ios::binary` — бинарный режим. По умолчанию текстовый — на Windows `\n` превращается в `\r\n` при записи. Бинарный — без преобразований. - `std::ios::trunc` — очистить файл при открытии (по умолчанию для `ofstream`).

Флаги можно комбинировать через `|`:

```
std::ofstream file("data.bin", std::ios::out | std::ios::binary);
```

13.5.3 Бинарные данные

Когда нужно записать «сырые» байты:

```
int values[] = {1, 2, 3, 4, 5};  
std::ofstream out("data.bin", std::ios::binary);  
out.write(reinterpret_cast<const char*>(values), sizeof(values));
```

`write(const char*, size_t)` пишет заданное количество байтов. `reinterpret_cast<const char*>` — это говорит компилятору «трактуй этот указатель как `const char*`». Об этом ужасе в проекте мини-СУБД (глава 32).

Аналогично `read`:

```
int loaded[5];
std::ifstream in("data.bin", std::ios::binary);
in.read(reinterpret_cast<char*>(loaded), sizeof(loaded));
```

13.5.4 Полностью прочитать файл

Иногда хочется прочитать весь файл в строку:

```
#include <fstream>
#include <sstream>
#include <string>

std::string read_file(const std::string& filename) {
    std::ifstream in(filename);
    if (!in.is_open()) return "";

    std::stringstream buffer;
    buffer << in.rdbuf();
    return buffer.str();
}
```

`in.rdbuf()` — это «внутренний буфер» потока. Перенаправляя его в `stringstream`, мы копируем всё содержимое. Далее `buffer.str()` возвращает результат.

13.6 `std::stringstream` — строка как поток

Иногда нужны возможности потока для работы со строками. Например, разобрать строку в числа:

```
#include <sstream>

std::string line = "1 2 3 4 5";
std::stringstream ss(line);

int n;
while (ss >> n) {
    std::cout << n * 2 << "\n";
}
```

`std::stringstream` — поток, работающий с памятью, а не с файлом. Поддерживает `<<` и `>>`.
Полезно для: - Разбора строк (как пример выше). - Сборки строк с форматированием.

13.6.1 Форматированная сборка строк

`std::to_string` для дробных мало настраивается. С `stringstream` точнее:

```
#include <sstream>
#include <iomanip>

std::stringstream ss;
ss << std::fixed << std::setprecision(2) << 3.14159;
std::string s = ss.str();    // "3.14"
```

Это идиома, которую вы будете использовать постоянно для красивого форматирования.

Класс отдельно для записи — `std::ostringstream`. Для чтения — `std::istringstream`.
`std::stringstream` — оба сразу.

13.7 Сводный пример: разобрать CSV

Соберём осмысленный пример. Файл `people.csv`:

```
Alice,30,Engineer
Bob,25,Designer
Charlie,42,Manager
```

Прочитаем его и выведем красивой таблицей:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <iomanip>

struct Person {
    std::string name;
    int age;
    std::string job;
};
```

```
std::vector<Person> read_csv(const std::string& filename) {
    std::vector<Person> people;
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Не могу открыть " << filename << "\n";
        return people;
    }

    std::string line;
    while (std::getline(file, line)) {
        std::stringstream ss(line);
        Person p;

        std::getline(ss, p.name, ',');

        std::string age_str;
        std::getline(ss, age_str, ',');
        p.age = std::stoi(age_str);

        std::getline(ss, p.job);

        people.push_back(p);
    }

    return people;
}

void print_table(const std::vector<Person>& people) {
    std::cout << std::left
        << std::setw(15) << "Name"
        << std::setw(5) << "Age"
        << "Job\n";
    std::cout << std::string(30, '-') << "\n";

    for (const auto& p : people) {
        std::cout << std::left
            << std::setw(15) << p.name
            << std::setw(5) << p.age
```

```

        << p.job << "\n";
    }
}

int main() {
    auto people = read_csv("people.csv");
    print_table(people);
    return 0;
}

```

Создайте файл `people.csv` с указанным содержимым, соберите программу:

```

$ g++ -std=c++11 -Wall -Wextra csv.cpp -o csv
$ ./csv

```

Name	Age	Job
Alice	30	Engineer
Bob	25	Designer
Charlie	42	Manager

Что использовано: - `std::ifstream` для чтения файла. - `std::getline` с **разделителем** (третий параметр) — читает до указанного символа. - `std::stringstream` для разбора одной строки на поля. - `std::stoi` для конвертации строки в число. - Форматирование таблицы через `std::setw` и `std::left`. - Заполнение разделителя через `std::string(30, '-') (30 минусов)`.

13.8 Пара тонкостей

13.8.1 Чтение чисел и точка в качестве разделителя

Если у вас файл с дробными числами в локали с **запятой** в качестве разделителя (русская локаль):

```

3,14
2,71

```

Стандартный `>>` на дробное число ждёт **точку**. Поэтому он прочитает 3 и остановится. Чтобы поток понимал запятую — нужно настроить локаль:

```
std::cin.imbue(std::locale("ru_RU.UTF-8"));
```

В книжных примерах я везде использую точку. На реальных проектах с локалями отдельно разбираться.

13.8.2 Юникод и `std::string`

Что будет если прочитать русский текст?

```
std::string s;  
std::cin >> s;    // вводим "Привет"  
std::cout << s;   // печатается "Привет"
```

Работает! `std::string` хранит байты, и поток ввода/вывода тоже работает с байтами. Никаких преобразований. Если терминал на UTF-8 (на современных macOS/Linux так), всё проходит через без потерь.

Но: `s.size()` будет 12 байтов (русские буквы по 2 в UTF-8), не 6 символов. И `s[0]` будет первым байтом многобайтного символа, не первой буквой. Для правильной работы с символами нужно конвертировать в `std::u32string` (UTF-32) или использовать библиотеку типа ICU. В нашей книге обходимся байтами — для большинства задач достаточно.

13.8.3 `sync_with_stdio`

В C из тех же файлов можно писать через `printf/scanf`. По умолчанию C++ потоки **синхронизируются** с C-потоками, чтобы можно было смешивать `printf` и `std::cout`. Это замедляет ввод-вывод.

Если в вашей программе только C++-потоки и нужно ускорить (актуально для соревновательного программирования):

```
std::ios::sync_with_stdio(false);  
std::cin.tie(nullptr);
```

В обычных проектах не нужно.

13.9 Главные правила главы

1. **Всегда `std::string`, не C-строки.** Исключение — только при работе с C-API.
2. **"\n" вместо `std::endl`** для скорости.
3. **`std::cerr` для ошибок, `std::cout` для основного вывода.** Не смешивайте.
4. **После `cin >> x` пропускайте остаток строки**, если потом `getline`.
5. **Проверяйте, что файл открылся** через `is_open()`.
6. **Файловые потоки сами закрываются** в деструкторе — не нужен явный `close()`, можно и не нужно.
7. **`std::stringstream` — мощная штука** для парсинга и форматирования.

13.10 Маленькое упражнение

1. Напишите программу, которая просит ввести строку и выводит её задом наперёд. Подсказка: `std::reverse` или ручной цикл.
2. Напишите программу, которая читает файл `numbers.txt` (по одному числу на строке) и выводит их сумму.
3. Напишите программу-калькулятор, читающую выражение вида `5 + 3` (число, оператор, число). Используйте `std::stringstream`.
4. Напишите программу, читающую строки до пустой строки (или `Ctrl+D`), и выводящую их обратно отсортированными.
5. Прочитайте файл (`std::ifstream`) и подсчитайте: сколько строк, сколько слов, сколько символов. По сути, это утилита `wc`.
6. (Сложнее) Напишите программу, которая принимает аргументом командной строки имя файла, открывает его как CSV, и выводит первые 10 строк красивой таблицей.

13.11 Что дальше

Главы 13-14 — про **STL контейнеры** (`vector`, `array`, `list`, `map`, `set`, `unordered_map`, `unordered_set`) и **алгоритмы** (`sort`, `find`, `transform`, `accumulate`). Это последние главы Части I — после них стартуем RPG-проект.

`std::vector` уже мелькал в предыдущих главах. Дальше посмотрим внимательно, как он работает изнутри (вместе с `capacity` vs `size` и инвалидацией итераторов), какие ещё контейнеры есть, и когда какой брать. Тема большая, но фундаментально не новая — много RAII, который мы уже понимаем.

Глава 14

Глава 13. Контейнеры STL

Стандартная библиотека C++ даёт целый зоопарк готовых контейнеров. Это огромное преимущество языка — почти любую задачу хранения данных закрывает что-то стандартное, и писать своё не надо. В этой главе разберём основные: `vector`, `array`, `list`, `map`, `unordered_map`, `set`, `unordered_set`. Расскажу, как они устроены изнутри, что у каких быстро/медленно, и в каком случае какой выбирать.

Эта глава очень практическая. Контейнеры — то, чем вы будете пользоваться **постоянно** в любом коде на C++. Без свободного владения хотя бы `vector` далеко не уехать.

14.1 Зачем контейнеры

В Python вы пишете `[1, 2, 3]` — это список. `{1, 2, 3}` — множество. `{"a": 1}` — словарь. Под капотом всё это разные структуры данных, но интерфейс однообразный и удобный.

В C++ исторически был только сырой массив (`int arr[10]`), который мы рассмотрели в главе 11. У него есть проблемы: размер на этапе компиляции, не растёт, нет удобных операций. Поэтому в стандартную библиотеку добавили контейнеры — каждый со своей структурой данных и интерфейсом.

Все контейнеры являются **шаблонами** (templates) — вы указываете, какой тип они будут хранить:

```
std::vector<int> numbers;           // вектор целых
std::vector<std::string> names;     // вектор строк
std::vector<std::vector<int>> grid; // вектор векторов (двумерная таблица)
```

Про шаблоны подробно — в главе 20. Сейчас достаточно знать: `Container<T>` — это контейнер, хранящий элементы типа `T`.

Все контейнеры — **РАП-объекты**. Они владеют своими данными, копируются и удаляются корректно, никаких ручных delete не нужно.

14.2 std::vector — главный контейнер

Если бы пришлось выбрать **один** контейнер на всю программу — выбрали бы vector. На моих проектах он покрывает 80% всех случаев хранения данных.

Это **динамический массив**: элементы лежат в памяти подряд, размер может расти. Подключается через <vector>:

```
#include <vector>
```

14.2.1 Создание

```
std::vector<int> a;           // пустой
std::vector<int> b(10);       // 10 элементов, все 0
std::vector<int> c(10, 42);   // 10 элементов, все 42
std::vector<int> d{1, 2, 3, 4, 5}; // initializer list — 5 указанных элементов
std::vector<int> e = {1, 2, 3}; // тоже initializer list
std::vector<int> f(c);        // копия c

std::cout << d.size(); // 5
```

std::vector<int> v(10) и std::vector<int> v{10} — **разные!** Первое — «10 нулей», второе — «один элемент 10» (инициализация-списком). Это та самая ловушка () vs {} из главы 8. Будьте внимательны.

14.2.2 Доступ к элементам

```
std::vector<int> v = {10, 20, 30, 40};

int x = v[0];           // 10 (без проверки границ)
int y = v.at(1);        // 20 (с исключением при выходе за границу)

int first = v.front();  // 10
int last  = v.back();   // 40
```

v[i] быстрее, чем v.at(i) (нет проверки), но опаснее. В новом коде с непроверенными индексами я обычно использую [] плюс ручной assert(i < v.size()) в отладке.

14.2.3 Добавление и удаление

```
std::vector<int> v;  
  
v.push_back(1);    // [1]  
v.push_back(2);    // [1, 2]  
v.push_back(3);    // [1, 2, 3]  
  
v.pop_back();      // [1, 2]  
  
v.insert(v.begin() + 1, 99); // [1, 99, 2] — вставить перед позицией 1  
v.erase(v.begin());      // [99, 2] — удалить первый  
  
v.clear();          // []
```

`push_back` — **главная** операция, добавление в конец. **Амортизированно $O(1)$** — обычно мгновенно, иногда долго (при росте `capacity`, об этом ниже).

`insert` и `erase` в середине — **$O(n)$** : нужно сдвинуть все элементы, которые идут после. Не используйте `insert` для частых вставок не в конец.

14.2.4 Размер и ёмкость

Здесь начинается самое интересное. У `vector` есть **два** размера:

- **`size()`** — сколько элементов реально хранится.
- **`capacity()`** — сколько элементов может поместиться без новой аллокации памяти.

```
std::vector<int> v;  
std::cout << v.size() << " " << v.capacity() << "\n"; // 0 0  
  
v.push_back(1);  
std::cout << v.size() << " " << v.capacity() << "\n"; // 1 1  
  
v.push_back(2);  
std::cout << v.size() << " " << v.capacity() << "\n"; // 2 2  
  
v.push_back(3);  
std::cout << v.size() << " " << v.capacity() << "\n"; // 3 4  
  
v.push_back(4);
```

```
std::cout << v.size() << " " << v.capacity() << "\n"; // 4 4

v.push_back(5);

std::cout << v.size() << " " << v.capacity() << "\n"; // 5 8
```

Размер растёт по 1. Ёмкость — **скачками**: $0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$. На GCC (libstdc++) и clang (libc++) это умножение на 2, на MSVC — на 1.5. Каждое «уплотнение» — это:

1. Выделить новую память (вдвое больше).
2. Скопировать (или переместить — об этом в главе 19) старые элементы.
3. Уничтожить старые.
4. Освободить старую память.

Это медленно. Но **амортизированная** сложность всё равно $O(1)$: если рост экспоненциальный, то операций «скопировать N элементов» происходит редко (логарифм от размера), и общее время на N вставок — $O(N)$.

14.2.5 reserve и оптимизация

Если **заранее известно**, сколько элементов будет — лучше зарезервировать:

```
std::vector<int> v;
v.reserve(1000);           // capacity = 1000 заранее
for (int i = 0; i < 1000; ++i) {
    v.push_back(i);        // ни одной реаллокации
}
```

reserve(N) гарантирует: первые N push_back пройдут без перевыделения памяти. Хорошая привычка, когда размер можно прикинуть.

shrink_to_fit() — обратное: убрать запас, сделать capacity() == size(). Используется редко, обычно когда хотите вернуть ОС лишнюю память.

14.2.6 Инвалидация итераторов и указателей

Серьёзная ловушка vector. После любой операции, которая может вызвать **перевыделение** памяти (push_back, insert, reserve, ...), все указатели и итераторы на элементы **становятся невалидными**.

```
std::vector<int> v = {1, 2, 3};

int* p = &v[0];           // p указывает на первый элемент
v.push_back(4);
```

```
v.push_back(5);
v.push_back(6);           // одно из этих push_back вызвало реаллокацию

*p = 99;                   // UB! p указывает на старую освобождённую память
```

То же с итераторами. То же с range-based for:

```
std::vector<int> v = {1, 2, 3};
for (int& x : v) {
    if (x == 2) {
        v.push_back(99);    // UB! Меняем контейнер во время итерации
    }
}
```

Правило: пока вы итерируете по вектору, **не меняйте его** — не добавляйте, не удаляйте, не вызывайте reserve. Если нужно — итерируйте по индексам и заранее запоминайте размер:

```
size_t n = v.size();
for (size_t i = 0; i < n; ++i) {
    if (v[i] == 2) {
        v.push_back(99);    // OK, n зафиксировано
    }
}
```

Или собирайте «что добавить» в отдельный список, и добавляйте после цикла.

14.2.7 Почему vector обычно быстрее списка

Элементы vector лежат в памяти **подряд**. Это значит: - При доступе к `v[i]` адрес вычисляется как `&v[0] + i * sizeof(T)` — одна арифметическая операция. - При проходе подряд процессор подгружает соседние элементы в кэш сразу. Это очень быстро. - Никаких «прыжков» по адресам, никаких разыменований указателей.

Сравните с **связным списком**: каждый элемент — отдельный узел на куче, с указателем на следующий. Чтобы перейти к следующему, нужно прочитать указатель — это обращение в RAM (кэш-промах) → ~200 тактов.

В среднем, проход по `vector<int>` из миллиона элементов — миллисекунды. По `list<int>` — десятки миллисекунд. Разница в 10-50 раз. Это про **кэш-дружелюбность**, о которой я говорил в главе 1.

Поэтому современный C++ совет: **используйте vector по умолчанию**. Список бери только если есть очень особые причины.

14.2.8 Удаление-стирание идиома (erase-remove)

Удалить **все** элементы, удовлетворяющие условию:

```
std::vector<int> v = {1, 2, 3, 4, 5, 6};
// убрать все чётные

v.erase(
    std::remove_if(v.begin(), v.end(),
        [](int x) { return x % 2 == 0; }),
    v.end()
);
// v теперь {1, 3, 5}
```

Это идиома **erase-remove**. `std::remove_if` сдвигает «оставляемые» элементы в начало и возвращает итератор «за концом нужного». `erase` удаляет хвост. Лямбды и алгоритмы — главы 14 и 21, но идиому стоит знать.

В C++20 есть прямой `std::erase_if(v, predicate)`, но в C++11 — только составной приём.

14.3 std::array — массив фиксированного размера

`std::array<T, N>` — обёртка над сырым массивом, но с интерфейсом вектора. Размер **фиксирован на компиляции**, элементы лежат на стеке.

```
#include <array>

std::array<int, 5> a = {1, 2, 3, 4, 5};

std::cout << a.size(); // 5 (всегда константа)
std::cout << a[0];     // 1
a.front();
a.back();

for (int x : a) {
    std::cout << x;
}
```

Когда брать `array`: - Размер заранее известен и не меняется (например, RGB-цвет = `array<unsigned char, 3>`). - Хотим то же удобство, что у `vector`, но без аллокации на куче.

Когда `vector`: - Размер меняется. - Размер известен только в рантайме. - Размер очень большой (не уместится на стеке).

Размер `std::array` — это часть его типа. `std::array<int, 5>` и `std::array<int, 10>` — **разные типы**. Поэтому функция, принимающая `array`, должна знать размер на компиляции. Передача массивов разных размеров — тема шаблонов (глава 20).

14.4 `std::deque` — двусторонняя очередь

`deque` (double-ended queue, читается «дэк») — как `vector`, но эффективно добавляет и удаляет с **обоих концов**:

```
#include <deque>

std::deque<int> d;
d.push_back(1);
d.push_back(2);
d.push_front(0);    // эффективно, в отличие от vector::insert(begin(), ...)
// d = [0, 1, 2]
```

Внутри — список «блоков» (массивов небольшого размера). Поэтому индексация $O(1)$, но элементы **не подряд в памяти** — кэш-эффективность хуже, чем у `vector`.

Когда брать: реальная очередь FIFO, иногда буферы. На практике — редко. `vector` хватает в 90% случаев.

14.5 `std::list` — двусвязный список

`std::list` — двусвязный список (у каждого узла указатели на следующий и предыдущий).

```
#include <list>

std::list<int> l = {1, 2, 3, 4, 5};
l.push_back(6);
l.push_front(0);

// нет индексации! l[2] не компилируется
auto it = l.begin();
std::advance(it, 2);    // перейти на 3-ю позицию
std::cout << *it;      // 3
```

Преимущества: - Вставка/удаление в **середине** — $O(1)$, если у вас уже есть итератор на нужное место. - Итераторы **не инвалидируются** при добавлении/удалении (кроме удалённого).

Недостатки: - Доступ по индексу — $O(N)$, потому что нужно пройти от начала. - Каждый узел — отдельная аллокация на куче (24 байта overhead на узел плюс выравнивание). Огромный overhead для маленьких элементов. - Плохая кэш-эффективность.

Когда брать list: - Очень частые вставки/удаления в середину. - Стабильные итераторы важны.

В моей практике list нужен раз в год. Для большинства случаев — vector.

14.6 std::map — упорядоченный словарь

std::map<K, V> — словарь, отображающий ключи K в значения V. Реализован как **красно-чёрное дерево** (балансированное двоичное дерево поиска).

```
#include <map>

std::map<std::string, int> ages;
ages["Alice"] = 30;
ages["Bob"] = 25;
ages["Charlie"] = 42;

std::cout << ages["Alice"];    // 30
std::cout << ages.size();      // 3

// Проверка наличия
if (ages.count("Alice") > 0) ...           // C++11 способ
if (ages.find("Alice") != ages.end()) ...  // более общий способ

// Удаление
ages.erase("Bob");

// Обход — отсортированно по ключу
for (const auto& pair : ages) {
    std::cout << pair.first << " = " << pair.second << "\n";
}
// "Alice = 30"
// "Charlie = 42"
```

Главные свойства: - **Все операции $O(\log N)$** — find, insert, erase. Это не «быстро», но прилично.
 - **Элементы отсортированы по ключу.** При обходе вы получите их по возрастанию.
 - **Стабильные итераторы:** операции с другими элементами не инвалидируют итератор на элемент.

14.6.1 Ловушка [] на map

map[key] имеет **побочный эффект**:

```
std::map<std::string, int> ages;
ages["Alice"] = 30;
std::cout << ages["Bob"];    // 0, и Bob появился в карте с значением 0!
std::cout << ages.size();    // 2 !
```

Если ключа нет, [] **создаёт** его со значением по умолчанию (для int — 0, для std::string — пустая). Это иногда удобно (счётчики), иногда — опасно.

Безопасный доступ:

```
auto it = ages.find("Bob");
if (it != ages.end()) {
    std::cout << it->second;
} else {
    std::cout << "не найден";
}
```

it->first — ключ, it->second — значение.

Или через at, который кидает исключение:

```
try {
    int x = ages.at("Bob");
} catch (const std::out_of_range&) {
    std::cout << "не найден";
}
```

14.6.2 Вставка с проверкой существования

```
auto result = ages.insert({"Alice", 50});
if (result.second) {
    std::cout << "Добавлен\n";
} else {
```



```
std::cout << "Alice уже была, значение " << result.first->second << "\n";  
}
```

insert возвращает пару: итератор на элемент и bool «была ли вставка».

14.7 std::unordered_map — хеш-таблица

std::unordered_map<K, V> — словарь на основе **хеш-таблицы**. Интерфейс тот же, что у map, но другие свойства:

```
#include <unordered_map>  
  
std::unordered_map<std::string, int> ages;  
ages["Alice"] = 30;  
ages["Bob"] = 25;  
  
std::cout << ages["Alice"]; // 30
```

Главные свойства: - **Поиск/вставка/удаление** — **O(1)** в среднем. **O(N)** в худшем случае (когда хеши плохо распределены, и все элементы попали в одно «ведро»). - **Элементы НЕ отсортированы**. Обход — в произвольном порядке. - **Перехеширование** при росте — инвалидация итераторов (как vector при push_back).

Когда брать unordered_map: - Не нужно отсортированное хранение. - Нужны быстрые операции.

Когда map: - Нужен отсортированный обход. - Нужно «найти первый элемент с ключом $\geq X$ » — есть lower_bound. - Хочется стабильное поведение в худшем случае (map гарантированно $O(\log N)$, unordered_map может стать $O(N)$).

В большинстве случаев берите unordered_map — он быстрее.

14.7.1 Ключи в unordered_map

Чтобы тип K мог быть ключом unordered_map, для него должна быть определена **функция хеширования**. Для стандартных типов (int, std::string, double) она уже есть. Для своих классов придётся определять — std::hash<MyClass>::operator(). Об этом коротко в главе 20.

14.7.2 Что внутри unordered_map

Это массив «**вёдер**» (buckets). Хеш-функция превращает ключ в число; число делится по модулю на количество вёдер, получается индекс. В этом ведре лежит список (обычно односвязный) всех элементов с тем же индексом.

Если все элементы попадают в одно ведро — поиск становится $O(N)$, потому что нужно пройти по списку. Хорошая хеш-функция распределяет равномерно, чтобы в каждом ведре было ~ 1 элемент.

14.8 `std::set` и `std::unordered_set`

Те же, что `map/unordered_map`, но **без значения** — только ключи. То есть **множество**.

```
#include <set>
#include <unordered_set>

std::set<std::string> taken_names;
taken_names.insert("Alice");
taken_names.insert("Bob");

if (taken_names.count("Charlie") == 0) {
    std::cout << "Charlie свободно\n";
}

std::unordered_set<int> visited;
visited.insert(5);
visited.insert(10);
```

Свойства: - `set` — отсортированное, $O(\log N)$, на дереве. - `unordered_set` — без сортировки, $O(1)$ в среднем, на хеш-таблице.

Когда брать: проверка «есть ли», подсчёт уникальных, удаление дубликатов.

Простой алгоритм «убрать дубликаты из вектора»:

```
std::vector<int> v = {1, 2, 2, 3, 3, 3, 4};
std::set<int> unique_set(v.begin(), v.end());
std::vector<int> unique(unique_set.begin(), unique_set.end());
// unique = {1, 2, 3, 4}, отсортированный
```

Через `unordered_set` — было бы быстрее, но порядок утратился бы.

14.9 Итераторы — кратко

Все контейнеры поддерживают **итераторы** — обобщённый способ обхода. Это будем подробно в главе 14, но базово:

```
std::vector<int> v = {10, 20, 30};

for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << "\n";
}
```

- `v.begin()` — итератор на первый элемент.
- `v.end()` — итератор «за последний», `sentinel`.
- `++it` — следующий элемент.
- `*it` — значение по итератору.

`range-based for` под капотом — это `for` с итераторами. Тот же код:

```
for (int x : v) {
    std::cout << x << "\n";
}
```

— компилируется в эквивалент классического `for (auto it = v.begin(); it != v.end(); ++it)`. Поэтому `range-for` работает для всего, у чего есть `begin()` и `end()`.

В главе 14 разберём, как использовать итераторы с алгоритмами.

14.10 `std::pair` и `std::tuple`

Малые «контейнеры» для группировки разнородных значений.

`std::pair<A, B>` — пара:

```
std::pair<std::string, int> entry{"Alice", 30};
std::cout << entry.first << ": " << entry.second;
```

Используется внутри `map` (элементы `map` — это `pair<key, value>`).

`std::tuple<A, B, C, ...>` — кортеж из нескольких значений (любого количества):

```
#include <tuple>

std::tuple<std::string, int, double> t{"Alice", 30, 5.7};
std::cout << std::get<0>(t) << " " << std::get<1>(t) << " " << std::get<2>(t);
```

В C++17 появились `structured bindings` (`auto [name, age, height] = t;`), но в C++11 — через `std::get<N>(t)` или `std::tie(name, age, height) = t`.

`Tuple` используется нечасто; обычно лучше определить именованную `struct`.

14.11 Сравнительная таблица

Контейнер	Доступ по индексу	Вставка в конец	Вставка в начало	Вставка в середину	Удаление в середине	Память
vector	$O(1)$	$O(1)$ аморт.	$O(N)$	$O(N)$	$O(N)$	компактно, на куче
array	$O(1)$	—	—	—	$O(N)$	компактно, обычно на стеке
deque	$O(1)$	$O(1)$	$O(1)$	$O(N)$	$O(N)$	блочко, на куче
list	—	$O(1)$	$O(1)$	$O(1)$	$O(N)$	разрозненно, с итератором на куче
map	—	—	—	$O(\log N)$	$O(\log N)$	дерево на куче
unordered_map	—	—	—	$O(1)$	$O(1)$	хеш-среднеесреднее на куче
set	—	—	—	$O(\log N)$	$O(\log N)$	дерево на куче
unordered_set	—	—	—	$O(1)$	$O(1)$	хеш-среднеесреднее на куче

14.12 Решающее дерево: какой контейнер брать

Нужна последовательность с порядком?

└─ Да: упорядочена и фиксирована? → array

| упорядочена, размер меняется? → vector

- | часто вставляем и в начало, и в конец? → deque
- | очень частые вставки/удаления в середину? → list
- └ Нет: нужны пары ключ-значение?
 - | Да: нужен порядок ключей? → map
 - | только быстрый доступ? → unordered_map
 - └ Нет (только уникальные элементы):
 - нужен порядок? → set
 - только быстрый доступ? → unordered_set

По умолчанию — vector или unordered_map. Они покрывают 90% случаев и в среднем самые быстрые благодаря кэш-эффективности и хешированию соответственно.

14.13 Распространённые операции

14.13.1 Найти элемент

```
std::vector<int> v = {1, 2, 3, 4, 5};

auto it = std::find(v.begin(), v.end(), 3);
if (it != v.end()) {
    std::cout << "Found at index " << (it - v.begin()) << "\n";
}
```

std::find из <algorithm> — работает с любым контейнером.

В map / set — встроенный find быстрее, потому что использует структуру (дерево или хеш):

```
auto it = ages.find("Alice");
```

14.13.2 Сортировка

```
std::vector<int> v = {5, 2, 4, 1, 3};
std::sort(v.begin(), v.end());
// v = {1, 2, 3, 4, 5}

// По убыванию:
std::sort(v.begin(), v.end(), std::greater<int>());
// v = {5, 4, 3, 2, 1}
```

std::sort — алгоритм из главы 14. Работает для контейнеров с произвольным доступом (vector, deque, array). Не работает для list — у списка свой sort метод.

14.13.3 Сумма

```
std::vector<int> v = {1, 2, 3, 4, 5};  
int total = std::accumulate(v.begin(), v.end(), 0);  
// 15
```

Третий аргумент — начальное значение. Можно использовать для произвольной операции (не только сложение), но это уже глава 14.

14.13.4 Подсчёт

```
std::vector<int> v = {1, 2, 2, 3, 2, 4};  
int count = std::count(v.begin(), v.end(), 2);  
// 3
```

14.13.5 Уникальные значения

```
std::vector<int> v = {1, 2, 2, 3, 3, 3, 4};  
std::sort(v.begin(), v.end());  
v.erase(std::unique(v.begin(), v.end()), v.end());  
// v = {1, 2, 3, 4}
```

`std::unique` работает только на **отсортированных** — поэтому сначала `sort`.

14.14 Большой пример: подсчёт частоты слов

Соберём программу, читающую текст и выводящую самые частые слова:

```
#include <iostream>  
#include <fstream>  
#include <sstream>  
#include <vector>  
#include <unordered_map>  
#include <string>  
#include <algorithm>  
  
int main(int argc, char* argv[]) {  
    if (argc < 2) {  
        std::cerr << "Usage: " << argv[0] << " <file>\n";  
        return 1;  
    }
```

```
}

std::ifstream file(argv[1]);
if (!file.is_open()) {
    std::cerr << "Cannot open " << argv[1] << "\n";
    return 1;
}

// Считаем частоту
std::unordered_map<std::string, int> freq;
std::string word;
while (file >> word) {
    // Приведём к нижнему регистру (только ASCII – упрощение)
    for (char& c : word) {
        c = std::tolower(static_cast<unsigned char>(c));
    }
    // Уберём знаки препинания (упрощённо)
    word.erase(
        std::remove_if(word.begin(), word.end(),
            [](char c) { return !std::isalpha(static_cast<unsigned char>(c)); }),
        word.end()
    );
    if (!word.empty()) {
        freq[word] = freq[word] + 1;
    }
}

// Переложим в вектор и отсортируем по убыванию частоты
std::vector<std::pair<std::string, int>> pairs(freq.begin(), freq.end());
std::sort(pairs.begin(), pairs.end(),
    [](const std::pair<std::string, int>& a,
        const std::pair<std::string, int>& b) {
        return a.second > b.second;
    });

// Вывод первых 10
int shown = 0;
for (const auto& p : pairs) {
```

```

        std::cout << p.first << ": " << p.second << "\n";
        if (++shown >= 10) break;
    }

    return 0;
}

```

Использовано: - `std::unordered_map<std::string, int>` для подсчёта (быстро, без сортировки).
 - `std::vector<std::pair<...>>` для переноса в сортируемый формат. - `std::sort` с лямбдой-компаратором (глава 14 + 21). - `std::remove_if` для очистки от пунктуации. - `range-based for` с `const auto&`. - `argv` для имени файла.

Сборка:

```

$ g++ -std=c++11 -Wall -Wextra freq.cpp -o freq
$ ./freq book.txt

```

14.15 Главные правила главы

1. По умолчанию — `std::vector` для последовательностей, `std::unordered_map` для словарей.
2. `reserve` перед циклом `push_back`, если знаете размер.
3. Не модифицируйте контейнер во время `range-based for` — UB.
4. `map::[]` создаёт элемент, если его не было. Используйте `find` или `at` для безопасного чтения.
5. `list` нужен очень редко. Не давайте в это «теории алгоритмов».
6. Помните про инвалидацию итераторов при изменении контейнера.
7. `const T&` в `range-for` для больших типов, без копий.

14.16 Маленькое упражнение

1. Прочитайте N чисел из `stdin` в `std::vector<int>`. Найдите минимум, максимум, среднее.
2. Прочитайте слова и выведите их по алфавиту, без дубликатов.
3. Имея `std::vector<int>`, удалите все отрицательные.
4. Имея `std::map<std::string, int>` оценок учеников, найдите ученика с максимальной оценкой.
5. Прочитайте имена из файла. Выведите частоту каждого имени, отсортированную по убыванию.

6. (Сложнее) У вас граф, представленный как `std::unordered_map<int, std::vector<int>>` (соседи каждой вершины). Реализуйте BFS — поиск кратчайшего пути от вершины 0 до вершины N.

14.17 Что дальше

Глава 14 — алгоритмы STL (`sort`, `find`, `transform`, `accumulate`, ...) и итераторы подробно. Многие из них мы уже видели мельком; теперь они получают своё место. После 14-й главы Часть I закрыта, и мы готовы к **Части II — RPG-проекту**, где все эти инструменты заработают вместе на реальной программе.

Глава 15

Глава 14. Алгоритмы и итераторы

В прошлой главе мы освоили контейнеры — где хранить данные. Эта глава — про то, что с ними **делать**: искать, сортировать, преобразовывать, сворачивать. Всё это уже написано в стандартной библиотеке через `<algorithm>` и `<numeric>`. Самим алгоритмы писать почти никогда не нужно — `std::sort` от Google написан умнее, чем любая реализация на коленке за вечер. Ваша работа — научиться этими готовыми штуками пользоваться.

После этой главы Часть I закрывается, и мы стартуем RPG-проект.

15.1 Зачем алгоритмы — снова про переиспользование

Допустим, у вас есть `std::vector<int>` и нужно найти, есть ли там значение 42. Можно написать вручную:

```
bool found = false;
for (size_t i = 0; i < v.size(); ++i) {
    if (v[i] == 42) {
        found = true;
        break;
    }
}
```

Или короче:

```
auto it = std::find(v.begin(), v.end(), 42);
bool found = (it != v.end());
```

Второе короче, читается яснее, и не зависит от того, `vector` у вас или `list` или `array`. `std::find` работает с **любым** контейнером, у которого есть итераторы. Это и есть главная идея STL — **алгоритмы не привязаны к контейнерам**.

15.2 Итераторы — обобщённый указатель

В прошлой главе я обещал детали по итераторам. Итак.

Итератор — это объект, который ведёт себя как указатель на элемент контейнера: - `*it` — получить значение. - `++it` — перейти к следующему. - `it1 == it2 / it1 != it2` — сравнить.

В разных контейнерах итераторы устроены по-разному: - В `vector` — это просто `T*` (или почти). - В `list` — указатель на узел. - В `map` — указатель на узел дерева.

Но интерфейс одинаков. Алгоритм типа `std::find` пишется в общем виде через шаблоны:

```
template <typename Iterator, typename Value>
Iterator find(Iterator begin, Iterator end, const Value& value) {
    for (; begin != end; ++begin) {
        if (*begin == value) return begin;
    }
    return end;
}
```

(Шаблоны мы подробно разберём в главе 20, но идею вы уже видите.)

Алгоритм не знает, что за контейнер. Знает только, что итератор умеет `++`, `*` и `==`. Поэтому один и тот же `std::find` работает с `vector`, `list`, `map`, и так далее.

15.2.1 Что такое `begin()` и `end()`

`v.begin()` — итератор на **первый** элемент.

`v.end()` — итератор **за последний** (sentinel, «маркер конца»). Это **не последний элемент**, а позиция «после последнего». Разыменовывать `end()` — UB.

Размер контейнера: `end() - begin()` (для случайно-доступных итераторов).

Пустой контейнер: `begin() == end()`.

«Полуоткрытый интервал» `[begin, end)` — стандарт для всех алгоритмов STL. Принимайте как данность.

15.3 Категории итераторов

Не все итераторы одинаково мощные. STL делит их на **категории**, и каждый алгоритм требует определённой категории.

15.3.1 Input iterator

Самый слабый. Можно только **читать**, и только **один раз**: - *it — прочитать. - ++it — перейти к следующему. - Сравнивать.

Пример: итератор `std::istream_iterator` (читает из потока).

Со слабым итератором работают алгоритмы типа `find`, `count` — они один раз проходят и не возвращаются.

15.3.2 Output iterator

Можно только **писать**: - *it = value — записать. - ++it — перейти.

Пример: `std::ostream_iterator` (пишет в поток).

С слабым output итератором работают `copy`, `transform` (пишущие куда-то).

15.3.3 Forward iterator

Можно читать **много раз**, перебирать в одну сторону.

Пример: итераторы `std::forward_list`.

15.3.4 Bidirectional iterator

Прямой плюс возможность идти **назад** (`--it`).

Пример: итераторы `std::list`, `std::map`, `std::set`.

15.3.5 Random access iterator

Самый мощный. Можно прыгать на N позиций (`it + N`, `it - N`), вычислять разность, индексировать.

Пример: итераторы `std::vector`, `std::deque`, `std::array`, сырые указатели.

Со случайно-доступными работает `std::sort`, потому что ему нужно прыгать туда-сюда.

15.3.6 Иерархия

Random access → Bidirectional → Forward → Input/Output
 (vector) (list) (flist) (потoki)

Каждая следующая категория — расширение предыдущей. Random access умеет всё, что Forward, плюс прыжки.

Если алгоритм требует Random access, а у вас Forward — не сработает. Поэтому `std::sort` нельзя применить к `std::list`. У `list` есть свой метод `l.sort()` для этого случая.

В большинстве кода вы не задумываетесь о категориях. Знаете только: «vector и array — Random access, list и map — Bidirectional, остальное реже».

15.4 Алгоритмы без модификации

Не меняют контейнер, только читают.

15.4.1 find — найти первый

```
std::vector<int> v = {1, 2, 3, 4, 5};

auto it = std::find(v.begin(), v.end(), 3);
if (it != v.end()) {
    int index = it - v.begin(); // для random access
    std::cout << "Found at index " << index << "\n";
}
```

15.4.2 find_if — найти с предикатом

«Предикат» — это функция или лямбда, возвращающая `bool`. Найти первый элемент, удовлетворяющий условию:

```
auto it = std::find_if(v.begin(), v.end(),
                      [](int x) { return x > 3; });
if (it != v.end()) {
    std::cout << "Первый > 3: " << *it << "\n"; // 4
}
```

`[](int x) { return x > 3; }` — это **лямбда**, безымянная функция. Глава 21 — детально. Сейчас можно читать как «функция, принимающая `int` и возвращающая, больше ли он трёх».

15.4.3 count и count_if

```
std::vector<int> v = {1, 2, 2, 3, 2, 4};

int n = std::count(v.begin(), v.end(), 2);    // 3 – сколько двоек
int m = std::count_if(v.begin(), v.end(),
                     [](int x) { return x > 2; });    // 2 – сколько > 2
```

15.4.4 all_of, any_of, none_of

Проверки «все», «хоть один», «ни один»:

```
bool all_positive = std::all_of(v.begin(), v.end(),
                               [](int x) { return x > 0; });
bool has_negative = std::any_of(v.begin(), v.end(),
                                [](int x) { return x < 0; });
bool no_zeros     = std::none_of(v.begin(), v.end(),
                                 [](int x) { return x == 0; });
```

15.4.5 min_element и max_element

Найти минимальный и максимальный:

```
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};

auto min_it = std::min_element(v.begin(), v.end());
auto max_it = std::max_element(v.begin(), v.end());

std::cout << "Min: " << *min_it << "\n";    // 1
std::cout << "Max: " << *max_it << "\n";    // 9
```

std::min/std::max — это просто между двумя значениями:

```
int x = std::min(5, 7);    // 5
int y = std::max({5, 3, 9, 2});    // 9 – из списка (C++11)
```

15.4.6 equal — сравнить два диапазона

```
std::vector<int> a = {1, 2, 3};
std::vector<int> b = {1, 2, 3};
```

```
bool same = std::equal(a.begin(), a.end(), b.begin()); // true
```

15.5 Алгоритмы с модификацией

Меняют содержимое контейнера.

15.5.1 fill — заполнить значением

```
std::vector<int> v(10);  
std::fill(v.begin(), v.end(), 42);  
// v = {42, 42, 42, 42, 42, 42, 42, 42, 42, 42}
```

15.5.2 iota — заполнить последовательно

```
#include <numeric>  
  
std::vector<int> v(5);  
std::iota(v.begin(), v.end(), 10);  
// v = {10, 11, 12, 13, 14}
```

«iota» — старое слово, означает «единица» (греческая буква йота). Алгоритм заполняет начиная с указанного значения с шагом 1.

15.5.3 transform — применить функцию

Применить функцию к каждому элементу:

```
std::vector<int> v = {1, 2, 3, 4};  
std::vector<int> doubled(v.size());  
  
std::transform(v.begin(), v.end(),  
               doubled.begin(),  
               [](int x) { return x * 2; });  
// doubled = {2, 4, 6, 8}
```

Параметры: начало входа, конец входа, начало выхода, функция.

Можно писать в **тот же** контейнер:

```
std::transform(v.begin(), v.end(),
              v.begin(),
              [](int x) { return x * 2; });
// v = {2, 4, 6, 8}
```

15.5.4 copy и copy_if

Копировать элементы:

```
std::vector<int> src = {1, 2, 3, 4, 5};
std::vector<int> dst(5);

std::copy(src.begin(), src.end(), dst.begin());
// dst = {1, 2, 3, 4, 5}
```

С предикатом — только подходящие:

```
std::vector<int> src = {1, 2, 3, 4, 5, 6};
std::vector<int> evens;

std::copy_if(src.begin(), src.end(),
             std::back_inserter(evens),
             [](int x) { return x % 2 == 0; });
// evens = {2, 4, 6}
```

`std::back_inserter(evens)` — особый итератор, который при записи вызывает `evens.push_back()`. Если бы мы написали `evens.begin()`, было бы UB — мы бы писали в пустой вектор.

15.5.5 remove и remove_if

Хитрая семантика: эти алгоритмы **не удаляют** элементы (контейнер они не знают). Они **сдвигают** «оставляемые» в начало и возвращают итератор «нового конца». Хвост остаётся в неопределённом состоянии.

```
std::vector<int> v = {1, 2, 3, 4, 5};
auto new_end = std::remove(v.begin(), v.end(), 3);
// v = {1, 2, 4, 5, ?}   (5 элементов всё ещё, последний — мусор)
// new_end указывает на позицию «после 5»

v.erase(new_end, v.end());
// v = {1, 2, 4, 5}
```


Эта пара `remove + erase` — идиома **erase-remove**, мы её видели в главе 13. Запомните как одно действие.

С предикатом — `remove_if`:

```
v.erase(
    std::remove_if(v.begin(), v.end(),
        [](int x) { return x < 0; }),
    v.end()
);
```

15.5.6 reverse и rotate

```
std::vector<int> v = {1, 2, 3, 4, 5};

std::reverse(v.begin(), v.end());
// v = {5, 4, 3, 2, 1}

std::rotate(v.begin(), v.begin() + 2, v.end());
// «провернуть» так, чтобы элемент с позиции 2 стал началом
```

15.5.7 sort и связанные

Главный алгоритм сортировки. Использует introsort (комбинацию quicksort, heapsort и insertion sort), $O(N \log N)$ в среднем и в худшем случае.

```
std::vector<int> v = {5, 2, 4, 1, 3};

std::sort(v.begin(), v.end());
// v = {1, 2, 3, 4, 5}

// По убыванию через компаратор:
std::sort(v.begin(), v.end(),
    [](int a, int b) { return a > b; });
// v = {5, 4, 3, 2, 1}

// Сортировка строк по длине:
std::vector<std::string> words = {"apple", "hi", "world", "a"};
std::sort(words.begin(), words.end(),
    [](const std::string& a, const std::string& b) {
```

```

        return a.size() < b.size();
    });
// words = {"a", "hi", "apple", "world"}

```

Компаратор — это **двухаргументная функция**, возвращающая `true`, если первый «меньше» второго.

`std::stable_sort` — то же, но сохраняет относительный порядок равных элементов. Чуть медленнее, но иногда важно.

`std::partial_sort` — отсортировать только первые `N`.

`std::nth_element` — поставить `N`-й элемент на нужное место (как если бы массив был отсортирован). Быстрее, чем полная сортировка, если нужен только медиана или `N`-й.

15.5.8 `unique`

Убрать **подряд идущие** дубликаты:

```

std::vector<int> v = {1, 1, 2, 2, 3, 1, 1};

auto new_end = std::unique(v.begin(), v.end());
v.erase(new_end, v.end());
// v = {1, 2, 3, 1} ← одиночка 1 в конце осталась!

```

Чтобы убрать **все** дубликаты, сначала отсортируйте:

```

std::vector<int> v = {3, 1, 2, 1, 3, 2};
std::sort(v.begin(), v.end());
// v = {1, 1, 2, 2, 3, 3}
v.erase(std::unique(v.begin(), v.end()), v.end());
// v = {1, 2, 3}

```

15.6 Численные алгоритмы

Из заголовка `<numeric>`:

15.6.1 `accumulate`

Свернуть в одно значение:

```

#include <numeric>

```

```
std::vector<int> v = {1, 2, 3, 4, 5};
int sum = std::accumulate(v.begin(), v.end(), 0);
// 15
```

Третий аргумент — начальное значение. Тип результата равен типу начального значения. Поэтому осторожно:

```
std::vector<double> v = {1.5, 2.5, 3.5};
auto bad = std::accumulate(v.begin(), v.end(), 0);    // int! результат 6, не 7.5
auto good = std::accumulate(v.begin(), v.end(), 0.0); // double! 7.5
```

Можно задать **операцию**:

```
// Произведение
int product = std::accumulate(v.begin(), v.end(), 1,
                             std::multiplies<int>());
// 120 (= 1*2*3*4*5)

// Через лямбду
int sum_of_squares = std::accumulate(v.begin(), v.end(), 0,
                                     [](int acc, int x) {
                                         return acc + x * x;
                                     });
// 55 (= 1 + 4 + 9 + 16 + 25)
```

В общем `accumulate(first, last, init, op)` делает: начать с `init`, для каждого элемента `e` сделать `result = op(result, e)`. Это **fold**, тот самый из функциональных языков.

15.6.2 inner_product

Скалярное произведение двух последовательностей:

```
std::vector<int> a = {1, 2, 3};
std::vector<int> b = {4, 5, 6};
int dot = std::inner_product(a.begin(), a.end(), b.begin(), 0);
// 32 (= 1*4 + 2*5 + 3*6)
```

15.6.3 partial_sum

Префиксная сумма:

```
std::vector<int> v = {1, 2, 3, 4, 5};
std::vector<int> sums(v.size());
std::partial_sum(v.begin(), v.end(), sums.begin());
// sums = {1, 3, 6, 10, 15}
```

15.6.4 adjacent_difference

Обратная — разности соседних:

```
std::vector<int> sums = {1, 3, 6, 10, 15};
std::vector<int> diffs(sums.size());
std::adjacent_difference(sums.begin(), sums.end(), diffs.begin());
// diffs = {1, 2, 3, 4, 5}
```

15.7 Перестановки

```
std::vector<int> v = {1, 2, 3};

do {
    for (int x : v) std::cout << x;
    std::cout << "\n";
} while (std::next_permutation(v.begin(), v.end()));
```

Выведет все 6 перестановок. Удобно для перебора всех вариантов.

15.8 Бинарный поиск

На **отсортированных** диапазонах:

```
std::vector<int> v = {1, 3, 5, 7, 9, 11};

bool present = std::binary_search(v.begin(), v.end(), 7); // true

auto it = std::lower_bound(v.begin(), v.end(), 6); // первый >= 6 → итератор на 7
auto it2 = std::upper_bound(v.begin(), v.end(), 7); // первый > 7 → итератор на 9
```

$O(\log N)$. На несортированных данных результат непредсказуем.

15.9 Лямбды и предикаты — короткое введение

Я уже много раз писал лямбды. Скажу о них минимум, чтобы освоиться. Подробно — в главе 21.

Лямбда — это безымянная функция, объявленная «на месте»:

```
auto is_even = [](int x) { return x % 2 == 0; };
std::cout << is_even(4);    // 1
std::cout << is_even(5);    // 0
```

Синтаксис: [capture](параметры) { тело }. Возвращаемый тип компилятор выводит сам.

[] — пустой **захват** (capture). Лямбда может «захватить» переменные из окружения:

```
int threshold = 10;
auto big_enough = [threshold](int x) { return x > threshold; };

std::vector<int> v = {3, 12, 7, 15, 1};
int count = std::count_if(v.begin(), v.end(), big_enough);
// 2 (12 и 15)
```

[threshold] — копировать threshold в лямбду по значению. [&threshold] — ссылка на оригинал (если изменится — лямбда увидит). [=] — копировать все используемые переменные. [&] — все по ссылке.

Это короткие безымянные функции, идеально подходящие для алгоритмов. Подробнее — глава 21.

15.9.1 Функтор — старый стиль

До C++11 вместо лямбд использовали **функторы** — классы с перегруженным operator():

```
struct IsBigger {
    int threshold;
    IsBigger(int t) : threshold(t) {}
    bool operator()(int x) const { return x > threshold; }
};

IsBigger pred(10);
int count = std::count_if(v.begin(), v.end(), pred);
```

В современном коде функторы редкость, лямбды короче. Но в чужом коде встречаются.

15.10 Алгоритмы vs методы контейнеров

Иногда один и тот же алгоритм есть и как свободная функция, и как метод контейнера. **Какой брать?**

Правило: **если у контейнера есть свой метод — используйте его**. Он знает структуру и работает быстрее.

Примеры:

Алгоритм	Свободная функция (медленнее)	Метод контейнера (быстрее)
Сортировка list	не работает	<code>l.sort()</code>
Поиск в map	<code>std::find</code> — $O(N)$	<code>m.find(key)</code> — $O(\log N)$
Поиск в un- ordered_map	<code>std::find</code> — $O(N)$	<code>m.find(key)</code> — $O(1)$
Поиск в set	<code>std::find</code> — $O(N)$	<code>s.find(value)</code> — $O(\log N)$

`std::find` для vector/array — нормально, $O(N)$ всё равно. Для map/set — глупо, у них свой `find` использует дерево.

15.11 Удалить элемент из map во время прохода

Хитрая операция:

```
std::map<std::string, int> m = {...};

// Так нельзя — итератор инвалидируется при erase
for (auto it = m.begin(); it != m.end(); ++it) {
    if (it->second < 0) {
        m.erase(it);    // UB: it после этого мёртв
    }
}

// Правильно — `erase` возвращает следующий итератор
for (auto it = m.begin(); it != m.end(); ) {
    if (it->second < 0) {
        it = m.erase(it);    // двигаемся через возврат
    } else {
        ++it;
    }
}
```

```
}  
}
```

Для `vector` — лучше через `erase-remove` идиому.

15.12 Свой `for_each` — попытка номер один

Зная итераторы и лямбды, можем написать свой простой `for_each`:

```
template <typename Iterator, typename Function>  
void my_for_each(Iterator first, Iterator last, Function f) {  
    for (; first != last; ++first) {  
        f(*first);  
    }  
}  
  
int main() {  
    std::vector<int> v = {1, 2, 3, 4};  
    my_for_each(v.begin(), v.end(),  
                [](int x) { std::cout << x << " "; });  
    // 1 2 3 4  
}
```

Шаблон обобщает тип итератора и функции. Алгоритмы STL устроены примерно так же, только сложнее (есть проверки, оптимизации, специализации).

15.13 Производительность

Несколько практических замечаний.

15.13.1 Алгоритмы STL часто оптимизированнее ручных

Реализация `std::sort` в современных компиляторах — десятки тысяч строк, с микро-оптимизациями под архитектуру. Ваш ручной `quicksort` скорее всего будет в 2-5 раз медленнее.

Поэтому: **используйте STL для очевидно стандартных задач**. Не переписывайте `sort` или `find_if`, если нет особых причин.

15.13.2 Алгоритмы вызывают функцию для каждого элемента

В цикле:

```
for (int x : v) {  
    process(x);  
}
```

process вызывается на каждый элемент — нет оверхеда. Через `std::for_each`:

```
std::for_each(v.begin(), v.end(), [](int x) { process(x); });
```

Лямбда тоже вызывается на каждый элемент, плюс лямбда создаётся как объект. Компилятор обычно инлайнит, и оверхед нулевой. Но в общем — не страшно.

15.13.3 Когда STL медленнее ручного

Очень редко. Бывает в случаях: - Хочется ранний выход из алгоритма, которого нет в STL. - Особая структура данных, где STL не знает. - Нужно несколько преобразований за один проход (STL делает их за несколько).

В 99% случаев STL — не узкое место.

15.14 Большой пример: статистика оценок

Собирающий пример. У нас оценки учеников; нужно посчитать среднее, медиану, отклонение, найти лучших, отсортировать.

```
#include <iostream>  
#include <vector>  
#include <algorithm>  
#include <numeric>  
#include <cmath>  
#include <string>  
  
struct Student {  
    std::string name;  
    int score;  
};  
  
double average(const std::vector<int>& scores) {  
    if (scores.empty()) return 0.0;  
    double sum = std::accumulate(scores.begin(), scores.end(), 0.0);  
    return sum / scores.size();  
}
```



```
double median(std::vector<int> scores) {    // копия — модифицируем
    if (scores.empty()) return 0.0;
    std::sort(scores.begin(), scores.end());
    size_t n = scores.size();
    if (n % 2 == 1) {
        return scores[n / 2];
    } else {
        return (scores[n / 2 - 1] + scores[n / 2]) / 2.0;
    }
}

double stddev(const std::vector<int>& scores) {
    if (scores.size() < 2) return 0.0;
    double mean = average(scores);
    double sum_sq = std::accumulate(scores.begin(), scores.end(), 0.0,
                                     [mean](double acc, int x) {
                                         double diff = x - mean;
                                         return acc + diff * diff;
                                     });
    return std::sqrt(sum_sq / (scores.size() - 1));
}

int main() {
    std::vector<Student> students = {
        {"Alice", 85},
        {"Bob", 72},
        {"Charlie", 90},
        {"Dave", 65},
        {"Eve", 78},
        {"Frank", 95},
        {"Grace", 58},
    };

    // Выдернем оценки в отдельный вектор
    std::vector<int> scores(students.size());
    std::transform(students.begin(), students.end(),
                   scores.begin(),
```

```

        [](const Student& s) { return s.score; });

std::cout << "Учеников: " << students.size() << "\n";
std::cout << "Среднее: " << average(scores) << "\n";
std::cout << "Медиана: " << median(scores) << "\n";
std::cout << "Ст. отклонение: " << stddev(scores) << "\n";

auto max_it = std::max_element(scores.begin(), scores.end());
auto min_it = std::min_element(scores.begin(), scores.end());
std::cout << "Лучшая оценка: " << *max_it << "\n";
std::cout << "Худшая: " << *min_it << "\n";

// Сколько прошло порог 70
int passed = std::count_if(scores.begin(), scores.end(),
                           [](int s) { return s >= 70; });
std::cout << "Прошли (>=70): " << passed << "\n";

// Отсортированный по убыванию рейтинг
std::sort(students.begin(), students.end(),
          [](const Student& a, const Student& b) {
              return a.score > b.score;
          });

std::cout << "\nРейтинг:\n";
int rank = 1;
for (const auto& s : students) {
    std::cout << rank++ << ". " << s.name << " (" << s.score << ")\n";
}

return 0;
}

```

Соберите:

```

$ g++ -std=c++11 -Wall -Wextra stats.cpp -o stats
$ ./stats

```

Использовано: `accumulate` (с лямбдой для квадратов отклонений), `transform` (выдернуть поле из структур), `min_element`/`max_element`, `count_if`, `sort` с лямбдой-компаратором.

Это **идиоматичный** современный C++. Никаких ручных циклов для поиска max/min, для подсчёта, для сортировки. Все стандартное.

15.15 Главные правила

1. Для известной задачи — берите алгоритм STL. Не пишите свой sort или find.
2. Для map/set/unordered_* — find метод контейнера, не std::find.
3. erase-remove для удаления из вектора по предикату.
4. accumulate с типом начального значения — 0 для int, 0.0 для double.
5. Лямбды с захватом по ссылке ([&]) — только когда понимаете, что переменная переживёт лямбду.
6. На отсортированных — бинарный поиск (binary_search, lower_bound).
7. Помните инвалидацию итераторов при erase — используйте возвращаемое значение.

15.16 Маленькое упражнение

1. Дан std::vector<int>. Найдите сумму всех **положительных** элементов. Используйте accumulate с лямбдой.
2. Дан std::vector<std::string>. Отсортируйте по длине, при равных длинах — лексикографически.
3. Дан std::vector<int>. Используя transform, постройте новый вектор, где каждый элемент — квадрат исходного.
4. Дан std::vector<int>. Используя partial_sum, постройте префиксные суммы. Используя их, ответьте на запрос «сумма от индекса L до R включительно» за O(1).
5. Дан std::vector<std::pair<std::string, int>> — оценки. Найдите имя ученика с максимальной оценкой через max_element с компаратором.
6. (Сложнее) Дан std::vector<int>. Сделайте функцию, которая удаляет все дубликаты, но **сохраняет первоначальный порядок** (то есть sort + unique не подходит).

15.17 Часть I закрыта

На этом Часть I заканчивается. Перечислю кратко, что у вас теперь есть:

- **Глава 8:** типы и переменные. int, double, bool, char, const, constexpr, auto.
- **Глава 9:** управляющие конструкции. if/else, switch, while, for, range-based for.
- **Глава 10:** функции. Объявление/определение, передача по значению/ссылке/указателю, перегрузка, рекурсия.

- **Глава 11:** память. Стек/куча, new/delete, RAII — главный паттерн C++.
- **Глава 12:** строки и I/O. `std::string`, `cin/cout`/файловые потоки, `stringstream`.
- **Глава 13:** контейнеры. `vector`, `array`, `list`, `map`, `unordered_map`, `set`.
- **Глава 14:** алгоритмы и итераторы. `find`, `sort`, `transform`, `accumulate`, лямбды.

Этого хватит, чтобы писать осмысленные программы. Но это всё ещё «низший уровень» C++ — мы пока не делали свои классы, не пользовались наследованием, не работали с умными указателями. Это всё в Части II.

15.18 Что дальше

Часть II — RPG-проект `demo-rpg/`. Мы начинаем писать большую программу: текстовая игра с локациями, инвентарём, NPC, боями, сохранениями. Все знания Части I заработают вместе, плюс мы введём: - Классы и объекты (глава 16). - Наследование и виртуальные функции (глава 17). - Smart pointers `unique_ptr/shared_ptr/weak_ptr` (глава 18). - Move-семантику и `rvalue`-ссылки (глава 19). - Шаблоны функций и классов (глава 20). - Лямбды и `std::function` (глава 21) — после кратких намёков. - Исключения и `exception safety` (глава 22). - Парсер скриптов локаций (глава 23). - Сохранение/загрузку (глава 24).

Сначала — глава 15: дизайн игры и скелет проекта. Мы разработаем архитектуру, создадим файловую структуру `demo-rpg/`, напомним `Makefile` с автогенерацией зависимостей, и подготовим почву для следующих глав. Это будет существенно отличаться от глав «учебных примеров»: реальная разработка с реальными решениями.

Глава 16

Глава 15. Дизайн игры и скелет проекта

С этой главы начинается **Часть II — RPG-проект**. До сих пор у нас были «учебные программы»: коротенькие файлы, демонстрирующие одну концепцию. Теперь мы строим полноценное приложение — `demo-rpg/`. Игра текстовая, в стиле *Zork*. Десяток локаций, инвентарь, NPC, бои, сохранения. К 25-й главе всё это будет работать и собираться одной командой.

В этой главе — **только подготовка**. Дизайн игры, архитектура классов, файловая структура, сборка, минимальный скелет, который компилируется. Никакого кода игровой логики ещё нет; это начнётся с главы 16. Но фундамент мы зложим прочный.

16.1 Что мы строим

Игрок управляет персонажем, путешествующим по миру. Все действия — текстовыми командами:

```
=== RPG-демо ===
```

```
Введите 'help' для списка команд.
```

```
> look
```

```
Лесная поляна. На западе видна тропа к деревне.
```

```
На востоке — тёмный лес.
```

```
> go east
```

```
Тёмный лес. Здесь стоит гоблин.
```

```
> attack гоблин
```

```
Вы ударили гоблина мечом. Урон 8. У гоблина 4 HP.
```

Гоблин ударил вас дубиной. Урон 3. У вас 17 HP.

```
> attack гоблин
```

Гоблин повержен. Получено 15 опыта.

```
> inventory
```

Инвентарь:

- ржавый меч
- кожаная броня

```
> save game1
```

Игра сохранена в data/saves/game1.sav

```
> quit
```

Что должно быть в финальной версии:

- **Мир** из ~15 локаций. Загружается из текстового файла, не зашит в код.
- **Команды:** look, go <direction>, take <item>, drop <item>, inventory/i, examine <thing>, attack <enemy>, talk <npc>, save <slot>, load <slot>, help, quit.
- **Инвентарь:** предметы можно брать, носить, использовать. Разные типы — оружие, броня, расходники.
- **NPC:** с ними можно говорить, иногда давать или брать предметы.
- **Враги:** бой пошаговый. У игрока и врага HP; кто первым опустился до 0 — проиграл.
- **Квесты:** например, «принеси меч кузнецу — получишь золото».
- **Сохранения:** в файл, по слотам. Можно загрузить в любой момент.

Это не пародия на RPG, а серьёзный игровой движок. Расширять его (новые локации, предметы, квесты) можно правя только текстовые файлы — без перекомпиляции.

16.2 Архитектура: какие классы будут

С прицелом на дальнейшие главы. Здесь сразу зафиксируем «карту» проекта.

Класс	Файлы	Что хранит	Глава
Game	game.{h,cpp}	Главный цикл, владеет миром и игроком.	15 (скелет)
World	world.{h,cpp}	Карта локаций.	16
Location	location.{h,cpp}	Описание комнаты, выходы, предметы, NPC.	16
Player	player.{h,cpp}	Состояние игрока: HP, инвентарь, опыт.	16
Entity (база)	entity.{h,cpp}	Общая база для всего живого.	17
Enemy : Entity	enemy.{h,cpp}	Враги: HP, урон, AI.	17

Класс	Файлы	Что хранит	Глава
NPC : Entity	npc.{h,cpp}	Дружественный персонаж с диалогами.	17
Item (база)	item.{h,cpp}	Базовый предмет.	17
Weapon : Item	weapon.{h,cpp}	Оружие: модификатор урона.	17
Armor : Item	armor.{h,cpp}	Броня: модификатор защиты.	17
Consumable : Item	consumable.{h,cpp}	Расходники: эффект при использовании.	17
Inventory	inventory.{h,cpp}	Управление предметами игрока.	18
EventBus<T>	event_bus.h	Шаблонная шина игровых событий.	20
CommandRegistry	commands.{h,cpp}	Парсер и диспетчер команд.	21
WorldParser	world_parser.{h,cpp}	Чтение мира из data/world.txt.	23
SaveManager	save_manager.{h,cpp}	Сохранения.	24

Может показаться, что классов много. Но каждый отвечает за **одно** дело, и связи между ними понятные. Лучше десять простых классов, чем один «БогОбъект» на тысячу строк.

16.2.1 Связи между классами

Грубо: Game владеет World и Player. World владеет коллекцией Location-ов. Location хранит список Item*-ов и Enemy*/NPC*-ов. Player имеет свой Inventory. Команды (CommandRegistry) при выполнении меняют состояние мира и игрока.

Что нового и страшного для свежее-прошедших Часть I:

- **Item, Weapon, Armor, Consumable** — иерархия классов с **полиморфизмом**. Локация хранит `unique_ptr<Item>`, а внутри может быть конкретный тип. Это глава 17 + 18.
- **EventBus** — шаблон. Глава 20.
- **CommandRegistry** — словарь имени команды на лямбду-обработчик. Глава 21.

Сейчас всего этого нет — у нас только Game с зашитыми двумя командами. По мере глав будем достраивать.

16.3 Файловая структура

Как организовать файлы — большая тема в C++-проектах. Я пользуюсь стандартной для типичных Open Source-проектов структурой:

```
demo-rpg/
├─ Makefile          — сборка
├─ include/          — заголовочные файлы (публичный интерфейс)
│  └─ game.h
│  └─ world.h        (появится в гл. 16)
│  └─ location.h
│  └─ ...
├─ src/              — реализации (.cpp)
│  └─ main.cpp        (вход в программу)
│  └─ game.cpp
│  └─ world.cpp
│  └─ ...
├─ tests/            — тесты (каждый .cpp — отдельная программа)
│  └─ test_smoke.cpp
├─ data/              — данные игры (мир, сейвы)
│  └─ world.txt        (появится в гл. 23)
│  └─ saves/
└─ build/             — собранное (НЕ коммитим в git!)
   └─ rpg              (исполняемый файл игры)
   └─ *.o, *.d         (промежуточные)
   └─ tests/*
```

Несколько правил:

1. **include/ для интерфейсов, src/ для реализаций.** Чёткое разделение. В include/ ничего исполняемого — только декларации (которые попадают через #include в код пользователя).
2. **build/ — артефакты сборки.** В git не коммитится (добавляется в .gitignore). Можно удалить и пересобрать.
3. **data/ — статические данные,** не код. Карта мира, диалоги. Можно править без перекомпиляции.
4. **tests/ — отдельный мир.** Тесты не входят в rpg-бинарь; они собираются отдельными исполняемыми файлами.

Если присмотреться, такая структура — почти стандарт. PostgreSQL, LLVM, SQLite, redis — все так устроены, с вариациями.

16.4 Создаём папки

В корне проекта:


```
$ cd cpp-from-scratch/demo-rpg
$ mkdir -p include src tests data/saves build
```

Файл .gitignore (если работаете под git):

```
build/
*.o
*.d
data/saves/*
!data/saves/.gitkeep
```

Это говорит git’у: не коммитить артефакты сборки, не коммитить пользовательские сейвы (только пустую папку с .gitkeep).

16.5 Makefile

Дальше — сборка. На больших проектах редко пишут команды компиляции руками; обычно используют **систему сборки**. Самая старая и универсальная — make со своими Makefile.

Современные альтернативы — CMake, Bazel, Meson, Ninja. Они мощнее, но make достаточно для нашего проекта, и хорошо иллюстрирует, что система сборки делает под капотом.

Идея make: вы описываете **правила** вида «чтобы сделать X, нужно Y и Z, и команда <cmd>». Make сам строит граф зависимостей, видит, что изменилось, и пересобирает только нужное.

16.5.1 Простой пример

```
build/rpg: src/main.cpp src/game.cpp
    g++ -std=c++11 -Iinclude src/main.cpp src/game.cpp -o build/rpg
```

Прочитайте как: «build/rpg зависит от src/main.cpp и src/game.cpp. Если они новее, чем build/rpg, выполнить команду».

Команда **должна** начинаться с Tab (не пробелов!). Это историческая особенность make, плохо стареющая, но изменить уже нельзя.

С такой простой версией каждое изменение пересобирает **всё**, включая файлы, которые не трогали. На больших проектах — больно.

Лучше: компилировать каждый .cpp в свой .o, потом линковать .o-файлы.

```
build/rpg: build/main.o build/game.o
    g++ build/main.o build/game.o -o build/rpg
```

```
build/main.o: src/main.cpp
    g++ -std=c++11 -Iinclude -c src/main.cpp -o build/main.o

build/game.o: src/game.cpp
    g++ -std=c++11 -Iinclude -c src/game.cpp -o build/game.o
```

Теперь, изменив `src/game.cpp`, мы пересобираем только `game.o` и линкуем. Уже лучше.

Но есть проблема: **зависимости от заголовков**. Если изменился `include/game.h`, нужно пересобрать `main.o` и `game.o` (потому что они включают `game.h`). Make об этом не знает! Он видит зависимости только от того, что мы написали явно. Если мы написали `build/game.o: src/game.cpp` — изменения в `game.h` make пропустит. Будет ошибка несоответствия (старая версия компилировалась с одним интерфейсом, новая — с другим).

16.5.2 -MMD — магия авто-зависимостей

Компиляторы умеют сами генерировать списки зависимостей. Флаг `-MMD` говорит: «при компиляции `.cpp` создавай рядом `.d`-файл со списком `.h`, которые входят».

Пример. Файл `src/game.cpp` подключает `include/game.h` и `<iostream>`. После `g++ -MMD -c src/game.cpp -o build/game.o` рядом появится `build/game.d`:

```
build/game.o: src/game.cpp include/game.h
```

Это **готовое правило make**! Если мы его подключим через `include` в наш Makefile — make узнает все зависимости автоматически.

Это и есть то, ради чего флаг `-MMD` стоит в любом современном проекте. Без него вы либо пишете зависимости вручную (и забываете), либо `make clean` перед каждой сборкой (медленно).

16.5.3 Наш Makefile

Собранный (с пояснениями ниже):

```
CXX      := g++
CXXSTD   := -std=c++11
WARNINGS := -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion
DEBUG    := -g -O0
RELEASE  := -O2 -DNDEBUG

MODE     ?= debug
ifeq ($(MODE),release)
```

```

    OPT := $(RELEASE)
else
    OPT := $(DEBUG)
endif

CXXFLAGS := $(CXXSTD) $(WARNINGS) $(OPT) -Iinclude -MMD -MP
LDFLAGS :=

SRCS      := $(filter-out src/main.cpp, $(wildcard src/*.cpp))
OBSJS     := $(patsubst src/%.cpp, build/%.o, $(SRCS))
DEPS      := $(OBSJS:.o=.d)

all: build/rpg

build/rpg: build/main.o $(OBSJS)
    @mkdir -p build
    $(CXX) $(CXXFLAGS) $^ -o $@ $(LDFLAGS)

build/main.o: src/main.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) -c $< -o $@

build/%.o: src/%.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
    rm -rf build

-include $(DEPS)

.PHONY: all clean

```

Разберём по частям.

Переменные

```

CXX      := g++
CXXSTD   := -std=c++11

```

`:=` — это **немедленное присваивание** (в отличие от `=`, ленивое). Хорошая практика — всегда `:=` для переменных-констант.

```
WARNINGS := -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion
```

Список флагов предупреждений. Расшифровка: - `-Wall` — большая часть стандартных предупреждений. - `-Wextra` — дополнительные. - `-Wpedantic` — строгое соответствие стандарту C++11. - `-Wshadow` — предупреждать о затенении переменных (когда локальная переменная скрывает внешнюю с тем же именем — типичный баг). - `-Wconversion` — неявные сужающие конверсии (`int` → `char`, `double` → `int` и т. д.). - `-Wsign-conversion` — конверсии `signed` ↔ `unsigned`.

Эти флаги ловят кучу багов. С ними новичкам приходится переписывать код несколько раз, пока компилятор не успокоится — но код потом качественнее.

```
DEBUG      := -g -O0
RELEASE    := -O2 -DNDEBUG
```

Два режима. **Debug**: `-g` (включить символы отладки, чтобы можно было через `gdb/lldb`), `-O0` (никаких оптимизаций, чтобы код и переменные соответствовали исходнику). **Release**: `-O2` (умеренная оптимизация), `-DNDEBUG` (определить макрос `NDEBUG`, который отключает `assert` и подобные проверки).

```
MODE      ?= debug
ifeq ($(MODE), release)
    OPT := $(RELEASE)
else
    OPT := $(DEBUG)
endif
```

Переменная `MODE` — режим сборки. Если не задана пользователем, по умолчанию `debug`. Через `make MODE=release` собираем в `release`. `?` — присваивание «если не задано».

```
CXXFLAGS := $(CXXSTD) $(WARNINGS) $(OPT) -Iinclude -MMD -MP
```

Финальный набор флагов: - стандарт C++11; - предупреждения; - режим (`debug/release`); - `-Iinclude` — добавить `include/` в пути поиска заголовков; - `-MMD` — генерировать `.d`-файлы зависимостей; - `-MP` — добавлять «пустые цели» для заголовков (защита от ошибок, когда удалят файл).

Списки файлов

```
SRCS      := $(filter-out src/main.cpp, $(wildcard src/*.cpp))
OBJS      := $(patsubst src/%.cpp, build/%.o, $(SRCS))
DEPS      := $(OBJS:.o=.d)
```

- `$(wildcard src/*.cpp)` — все `.cpp` в `src/`.
- `$(filter-out X, list)` — убрать `X` из списка. Мы убираем `main.cpp`, потому что он будет отдельно (в правиле для `rpg`).
- `$(patsubst src/%.cpp, build/%.o, ...)` — преобразование путей: `src/foo.cpp` → `build/foo.o`.
- `$(OBJSEX:.o=.d)` — то же, заменяя `.o` на `.d`.

После этих манипуляций: - `SRCS = src/game.cpp src/world.cpp ...` (всё кроме `main`). - `OBJS = build/game.o build/world.o ...` - `DEPS = build/game.d build/world.d ...`

Удобство: при добавлении нового `.cpp` ничего не надо править в `Makefile`.

Правила сборки

```
build/rpg: build/main.o $(OBJS)
    @mkdir -p build
    $(CXX) $(CXXFLAGS) $^ -o $@ $(LDFLAGS)
```

`build/rpg` зависит от `build/main.o` и всех остальных `OBJS`. Если что-то новое — выполняется команда.

В команде: - `@mkdir -p build` — `@` означает «не выводить команду в терминал». `-p` для `mkdir` мы знаем из главы 5. - `$^` — все зависимости (`build/main.o build/game.o ...`). - `$@` — имя цели (`build/rpg`).

Это **автоматические переменные** `make`. Удобство — не повторять имена.

```
build/%.o: src/%.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) -c $< -o $@
```

Шаблонное правило (pattern rule): для любого `build/X.o`, который соответствует `src/X.cpp`, выполнить команду. `$<` — первая зависимость (то есть конкретный `.cpp`).

Так одно правило обрабатывает все исходники: `game.cpp`, `world.cpp`, `player.cpp` и так далее.

`-include` — подключение зависимостей

```
-include $(DEPS)
```

Это **подключение файлов** в `Makefile`. Дефис в начале означает «не ошибка, если файла нет» (в первый запуск `.d`-файлов ещё нет — это нормально).

После первой сборки в `build/` появятся `game.o`, `main.o` со списками зависимостей. На следующих запусках `make` их прочитает и знает, что и от чего зависит.

.PHONY

```
.PHONY: all clean
```

Объявляет цели «фиктивными» — они не создают файла с именем `all` или `clean`. Без этого `make clean` мог бы перестать работать, если бы в каталоге случайно создался файл `clean`.

Запуск

```
$ make
g++ -std=c++11 -Wall -Wextra ... -c src/main.cpp -o build/main.o
g++ -std=c++11 -Wall -Wextra ... -c src/game.cpp -o build/game.o
g++ ... build/main.o build/game.o -o build/rpg
```

Все исходники компилируются и линкуются. Запустить:

```
$ ./build/rpg
```

При следующем `make` без изменений:

```
$ make
make: Nothing to be done for 'all'.
```

Make видит, что ничего не поменялось, и не делает работу заново.

Изменим `src/game.cpp` (без изменения `game.h`):

```
$ touch src/game.cpp
$ make
g++ ... -c src/game.cpp -o build/game.o
g++ ... build/main.o build/game.o -o build/rpg
```

Только `game.o` пересоберётся. Линковка — да, потому что `game.o` изменился. `main.o` не трогается.

Изменим `include/game.h`:

```
$ touch include/game.h
$ make
g++ ... -c src/main.cpp -o build/main.o
g++ ... -c src/game.cpp -o build/game.o
g++ ... build/main.o build/game.o -o build/rpg
```

Оба .cpp пересобрались, потому что оба зависят от game.h (через #include). Make узнал об этом из .d-файлов.

Это и есть **инкрементальная сборка**. Без -MMD пришлось бы перечислять зависимости руками — на десятке файлов уже невозможно.

16.6 Скелет кода

Готов Makefile. Теперь — минимальный код. Цель: компилируется и запускается, печатает приглашение, понимает help/look/quit. Игровой логики нет.

16.6.1 include/game.h

```
#ifndef RPG_GAME_H
#define RPG_GAME_H

#include <string>

namespace rpg {

class Game {
public:
    Game();
    int run();

private:
    bool step();
    void execute(const std::string& cmd);

    bool running_;
};

} // namespace rpg

#endif
```

Что нового:

Пространство имён `namespace rpg { ... }`. Все наши классы будут в нём. Это защита от коллизии имён: если кто-то подключит наш `Game` рядом с другим `Game` из чужой библиотеки, конфликта не будет — наш `rpg::Game`, его `other::Game`.

Class declaration:

```
class Game {
public:
    Game();           // конструктор
    int run();        // публичный метод

private:
    bool step();      // приватные методы
    void execute(const std::string& cmd);

    bool running_;    // приватное поле
};
```

`public:` — что видно снаружи. `private:` — только внутри класса.

Конвенция: **поля класса с подчёркиванием в конце** (`running_`). Это отличает их от локальных переменных и параметров.

`const std::string&` для параметра `cmd` — то, что мы изучили в главе 10 (по `const`-ссылке, без копии).

Глубокий разбор синтаксиса класса — глава 16. Сейчас — увидели, что есть, и идём дальше.

16.6.2 src/game.cpp

```
#include "game.h"

#include <iostream>
#include <string>

namespace rpg {

Game::Game() : running_(true) {
}

int Game::run() {
    std::cout << "=== RPG-демо ===\n";
```



```
std::cout << "Введите 'help' для списка команд.\n\n";

while (running_) {
    if (!step()) {
        break;
    }
}

std::cout << "До свидания.\n";
return 0;
}

bool Game::step() {
    std::cout << "> ";

    std::string line;
    if (!std::getline(std::cin, line)) {
        // EOF (Ctrl+D) – корректный выход.
        std::cout << "\n";
        return false;
    }

    if (line.empty()) {
        return true;
    }

    execute(line);
    return running_;
}

void Game::execute(const std::string& cmd) {
    if (cmd == "help") {
        std::cout << "Доступные команды:\n";
        std::cout << "  help    – этот список\n";
        std::cout << "  look   – осмотреться (пока заглушка)\n";
        std::cout << "  quit   – выход\n";
    } else if (cmd == "look") {
        std::cout << "Вы стоите на пустой поляне. Здесь пока ничего нет.\n";
    }
}
```

```

    } else if (cmd == "quit" || cmd == "q" || cmd == "exit") {
        running_ = false;
    } else {
        std::cout << "Неизвестная команда: '" << cmd << "'. Введите 'help'.\n";
    }
}

} // namespace rpg

```

Имена методов класса записываются как `Game::run` (член-функция). Двоеточие — оператор разрешения области видимости, тот же, что в `std::cout`.

`Game::Game() : running_(true) {}` — **конструктор с инициализационным списком**. После двоеточия — инициализация полей. Эквивалент:

```

Game::Game() {
    running_ = true;
}

```

Но через инициализационный список **правильнее** — поле сразу создаётся с нужным значением, а не сначала по умолчанию, потом перезаписывается. Для `bool` разницы практически нет, для классов — большая. Подробно в главе 16.

16.6.3 src/main.cpp

```

#include "game.h"

int main() {
    rpg::Game game;
    return game.run();
}

```

Создаём `Game` на стеке, вызываем `run()`, возвращаем код.

Заметили — `rpg::Game`, не просто `Game`. Без префикса `rpg::` компилятор не нашёл бы класс. Альтернатива:

```

using namespace rpg;

int main() {
    Game game;
}

```

```
    return game.run();  
}
```

`using namespace` втаскивает имена пространства в текущую область. Удобно, но опасно: на больших проектах можно случайно «втащить» имя, конфликтующее с чем-то ещё. Поэтому я в `main.cpp` обхожусь без `using namespace`.

16.6.4 tests/test_smoke.cpp

Совсем маленький smoke-тест:

```
#include "game.h"  
#include <iostream>  
#include <sstream>  
  
int main() {  
    std::istringstream empty_input("");  
    std::cin.rdbuf(empty_input.rdbuf());  
  
    rpg::Game game;  
    int rc = game.run();  
  
    if (rc != 0) {  
        std::cerr << "FAIL\n";  
        return 1;  
    }  
  
    std::cout << "OK\n";  
    return 0;  
}
```

«Smoke test» — простейшая проверка, что код вообще запускается. Здесь мы подменяем `std::cin` на пустой поток через `rdbuf`, чтобы `getline` сразу вернул `false` и игра завершилась.

Полноценные тесты с `GoogleTest` появятся в главе 25.

16.7 Сборка и проверка

```
$ cd demo-rpg  
$ make
```

```
g++ -std=c++11 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion \
    -g -O0 -Iinclude -MMD -MP -c src/main.cpp -o build/main.o
g++ ... -c src/game.cpp -o build/game.o
g++ ... build/main.o build/game.o -o build/rpg
Built: build/rpg (debug)
```

```
$ ./build/rpg
```

```
=== RPG-демо ===
```

```
Введите 'help' для списка команд.
```

```
> help
```

```
Доступные команды:
```

```
help    — этот список
look    — осмотреться (пока заглушка)
quit    — выход
```

```
> look
```

```
Вы стоите на пустой поляне. Здесь пока ничего нет.
```

```
> quit
```

```
До свидания.
```

Работает. Запустим тест:

```
$ make tests
```

```
$ ./build/tests/test_smoke
```

```
=== RPG-демо ===
```

```
...
```

```
OK
```

Скелет готов. Все следующие главы — наполнение его настоящей игрой.

16.7.1 Release-сборка

Сейчас собралось в debug-режиме. Соберём в release:

```
$ make clean
```

```
$ make MODE=release
```

```
g++ -std=c++11 ... -O2 -DNDEBUG ...
```

```
Built: build/rpg (release)
```

```
$ ls -lh build/rpg
```

```
-rwxr-xr-x 1 user staff 48K ... build/rpg
```

Размер 48 KB (в debug было ~100 KB). С -O2 компилятор оптимизирует код — программа быстрее, бинарь меньше (debug-символы выкинуты).

16.8 Тонкости и расширения

Несколько вещей, которые могут пригодиться позже.

16.8.1 Поддержка отладчика

В debug-режиме файлы скомпилированы с -g, можно открыть в отладчике:

```
$ gdb ./build/rpg
(gdb) break Game::run
(gdb) run
(gdb) next
(gdb) print running_
```

Или lldb на macOS (то же).

16.8.2 AddressSanitizer

Чтобы собрать с ASan для поиска багов памяти, добавьте в Makefile (или вызовите):

```
$ make CXXFLAGS="$(make CXXFLAGS) -fsanitize=address -fno-omit-frame-pointer"
```

В реальных проектах делают отдельный режим MODE=asan для удобства. Я добавлю это в проекте на середине Части II.

16.8.3 Зачем не CMake

CMake — современный стандарт. Я не использую его в книге специально: - Makefile проще для обучения (~50 строк, всё видно). - CMake требует знания «третьего языка» (CMakeLists.txt). - На нашем масштабе проекта (~30 файлов) разница незаметна.

В реальных коммерческих проектах — конечно, CMake. Когда дойдёте до глав 26+ (mini-shell и далее), стоит уже посмотреть на CMake — синтаксис похожий, идея та же.

16.8.4 Зачем не header-only

Иногда видишь библиотеки, где все классы написаны прямо в .h-файлах, без .cpp (Catch2, например). Это называется header-only.

Удобно для пользователя — одна `#include`-строка и поехали. Но плохо для **времени компиляции**: каждый файл, который включает заголовок, компилирует все его реализации. На больших проектах — десятки секунд лишних.

В нашем проекте — классический подход. Объявления в `.h`, реализации в `.cpp`. Компилятор работает быстрее, организация чище.

16.9 Главные правила главы

1. **Файловая структура** `include/src/tests/data/build/` — стандарт, придерживайтесь.
2. **-MMD -MP для авто-зависимостей**. Никаких ручных списков `.h` в `Makefile`.
3. **Жёсткие флаги предупреждений** с самого начала. Лучше две недели потратить на исправление, чем потом неделю искать редкий баг.
4. **Заголовок класса с `include guard` или `#pragma once`**. Без них — `multiple definition`.
5. **Класс в `namespace`** для проекта. Защита от коллизий имён.
6. **Поля приватные**, доступ через методы. Подробно в главе 16.
7. **Один класс — один файл**. `Game` → `game.h/game.cpp`. Облегчает навигацию.

16.10 Маленькое упражнение

1. Соберите проект (`make`), запустите (`./build/rpg`). Поиграйтесь с тремя командами.
2. Добавьте новую команду: `name <текст>` — установить имя игрока, и `whoami` — показать текущее имя. Имя храните в новом поле `Game`. Подсказка: для парсинга используйте `std::stringstream` (глава 12).
3. Соберите в `release` (`make MODE=release`). Сравните размер бинарника.
4. Сделайте `make clean`, потом `make`. Засеките время.
5. Добавьте к флагам предупреждений `-Werror` (превращает все предупреждения в ошибки). Соберите. Если будут срабатывания — исправляйте код, а не глушите флаги.
6. Откройте `build/game.d` и `build/main.d`. Посмотрите, что там за зависимости. Убедитесь, что после изменения `game.h` пересобираются оба `.o`.
7. Создайте `src/world.cpp` с пустой функцией:

```
namespace rpg {  
void hello_world() {}  
}
```

Запустите `make`. Должен сам подхватить новый файл без правки `Makefile`.

16.11 Что дальше

Глава 16 — наполняем скелет настоящими классами. Начнём с `Player` и `Location`. Введём наследование, конструкторы, деструкторы, инициализаторы, инкапсуляцию. После этой главы у нас уже будет работающий игрок, который может «осмотреть локацию» с конкретными деталями. Потом, глава за главой, добавим инвентарь, врагов, бои, NPC, сохранения.

Часть II — самая длинная и интересная: настоящий код, настоящие решения, и весь язык в работе.

Глава 17

Глава 16. Классы — Player, Location, World

В прошлой главе мы собрали скелет проекта. Класс `Game` там уже был, но мы прошли по нему по верхам. Эта глава — глубокий разбор **классов** в C++ на примере трёх настоящих сущностей игры: `Player`, `Location`, `World`. К концу главы у игрока будут параметры (НР, имя), в мире — три локации, и можно ходить из одной в другую.

Главы 16-19 — самые «тяжёлые» в Части II. Это про объектно-ориентированный C++: классы, наследование, виртуальные функции, `smart pointers`, `move`-семантика. Если эти главы пройдут плотно — дальше будет легко.

17.1 `struct` и `class`

В C++ есть два ключевых слова для определения пользовательских типов: `struct` и `class`. Технически они почти не отличаются. Единственная разница:

- В `struct` поля и методы по умолчанию **публичные**.
- В `class` поля и методы по умолчанию **приватные**.

```
struct Point {  
    int x;  
    int y;  
};  
  
class Person {  
    std::string name;    // приватное по умолчанию  
    int age;             // тоже приватное
```



```
public:
    Person(std::string n, int a) : name(std::move(n)), age(a) {}
};
```

Конвенция: - **struct** — для простых наборов данных без поведения. Point, Color, Vector3, DivvResult. Поля торчат наружу, никакой инкапсуляции. - **class** — для всего с инкапсуляцией, инвариантами, нетривиальной логикой.

Эту конвенцию никто не запрещает нарушать, но я в книге придерживаюсь. И в стандартной библиотеке так же: `std::pair` — `struct`, `std::vector` — `class`.

17.2 Простой класс Player

Начнём с Player. У игрока должны быть: - Имя. - Текущее здоровье (hp). - Максимальное здоровье (max_hp). - Методы: получить урон, вылечиться, проверить, жив ли.

17.2.1 Объявление

include/player.h:

```
#ifndef RPG_PLAYER_H
#define RPG_PLAYER_H

#include <string>

namespace rpg {

class Player {
public:
    Player(std::string name, int max_hp);

    const std::string& name() const;
    int hp() const;
    int max_hp() const;
    bool alive() const;

    void take_damage(int amount);
    void heal(int amount);

private:
```

```
    std::string name_;  
    int max_hp_;  
    int hp_;  
};  
  
} // namespace rpg  
  
#endif
```

Разберём по элементам.

17.2.2 Поля

```
private:  
    std::string name_;  
    int max_hp_;  
    int hp_;
```

Это **поля** (fields) — данные класса. У каждого объекта Player будут свои name_, max_hp_, hp_.

Соглашение об именовании: поля заканчиваются на подчёркивание. Это популярная конвенция (Google C++ Style Guide и многие другие). Зачем? Чтобы внутри метода легко различить:

```
void set_name(const std::string& name) {  
    name_ = name;    // присвоить полю значение параметра  
}
```

Без подчёркивания пришлось бы либо использовать `this->name = name;`, либо переименовать параметр. Подчёркивание убирает путаницу.

Поля **приватные**: снаружи к name_ напрямую обратиться нельзя. Это **инкапсуляция** — главная идея ООП. Класс отвечает за свои данные, никто извне их случайно не испортит.

17.2.3 Методы доступа (getters)

```
public:  
    const std::string& name() const;  
    int hp() const;  
    int max_hp() const;  
    bool alive() const;
```

Это **getters**. Они возвращают значение полей, никак их не меняя. Особенности:

`const std::string& name() const` — два `const` в одной строке, и они **разные**.

- Первый `const` — у возвращаемого типа: «возвращаю ссылку на неизменяемую строку». Так читатель не сможет через возвращённую ссылку поменять `name_`.
- Второй `const` (в конце) — это **const-метод**: «обещаю, что метод не меняет состояние объекта». Внутри `name()` нельзя написать `name_ = ...` — компилятор не позволит.

`const`-методы можно вызывать на **константных объектах**:

```
const Player p("Hero", 100);
std::cout << p.name();      // ОК: name() это const-метод
p.take_damage(5);           // ОШИБКА: take_damage не const
```

Это важно. Если у вас в коде где-то `const Player& p` (как параметр функции), на нём можно вызывать только `const`-методы.

Правило: все методы, не меняющие объект, помечайте `const`. Это документация для читателя и контракт для компилятора. Привычка `const-correctness` — отличает уверенного C++-программиста.

17.2.4 Модифицирующие методы

```
public:
    void take_damage(int amount);
    void heal(int amount);
```

Эти меняют состояние — не `const`.

17.2.5 Конструктор

```
public:
    Player(std::string name, int max_hp);
```

Конструктор — особый метод, имя которого совпадает с именем класса. Возвращаемого типа нет (даже `void` не пишут).

Вызывается **автоматически** при создании объекта:

```
Player hero("Alice", 100);
// неявно вызывается Player::Player("Alice", 100)
```

Параметры конструктора `name` (по значению `std::string`) и `max_hp` (по значению `int`) — это входные данные для инициализации.

17.2.6 Определение

src/player.cpp:

```
#include "player.h"

#include <utility>

namespace rpg {

Player::Player(std::string name, int max_hp)
    : name_(std::move(name)),
      max_hp_(max_hp),
      hp_(max_hp) {}

const std::string& Player::name() const {
    return name_;
}

int Player::hp() const {
    return hp_;
}

int Player::max_hp() const {
    return max_hp_;
}

bool Player::alive() const {
    return hp_ > 0;
}

void Player::take_damage(int amount) {
    if (amount < 0) amount = 0;
    hp_ -= amount;
    if (hp_ < 0) hp_ = 0;
}

void Player::heal(int amount) {
```

```

    if (amount < 0) amount = 0;
    hp_ += amount;
    if (hp_ > max_hp_) hp_ = max_hp_;
}

} // namespace rpg

```

Каждый метод определяется как `ClassName::method`. Это правильное использование квалифицированного имени.

Инициализационный список — самое важное

Конструктор содержит **инициализационный список** после двоеточия:

```

Player::Player(std::string name, int max_hp)
    : name_(std::move(name)),
      max_hp_(max_hp),
      hp_(max_hp) {
}

```

Каждое поле инициализируется в скобках. Это **правильный** способ. Альтернатива через присваивание в теле:

```

Player::Player(std::string name, int max_hp) {
    name_ = name;
    max_hp_ = max_hp;
    hp_ = max_hp;
}

```

— работает, но **хуже**. Что происходит при таком стиле:

1. Сначала каждое поле создаётся **по умолчанию**. `name_` становится пустой строкой, `max_hp_` и `hp_` — неинициализированными.
2. Потом в теле конструктора им **присваиваются** нужные значения. То есть `std::string::operator=` вызывается, что лишняя работа.

С инициализационным списком — поля сразу создаются с правильными значениями, без промежуточного «пустого» состояния. Это **быстрее и правильнее**.

Для `int` разница невелика. Для `std::string` или собственных классов с дорогим конструктором — заметная.

И есть случаи, когда инициализационный список **обязателен**: - Поля типа `const` (нельзя присвоить — только инициализировать). - Поля-ссылки (то же самое). - Поля без `default-конструктора` (классы, у которых нет конструктора без параметров).

Поэтому: **всегда инициализируйте поля в инициализационном списке, не в теле.**

Порядок в списке должен **совпадать с порядком объявления полей**. Если в классе:

```
private:
    std::string name_;
    int max_hp_;
    int hp_;
```

Инициализация **именно** в этом порядке. Иначе компилятор молча перенаправит (поля всё равно инициализируются в порядке объявления), и `-Wreorder` это поймает.

std::move в конструкторе

Почему `name_(std::move(name))`?

В прошлой главе видел такое выражение, не разбирали детально. Объясню кратко (полностью — в главе 19).

`std::move(name)` — это «преобразование к `rvalue`». Грубо: «эту переменную я больше использовать не буду, можно её ограбить».

Без `std::move`:

```
Player::Player(std::string name, int max_hp)
    : name_(name),          // КОПИЯ строки name
    ...
```

`name_(name)` — это **копия** строки. Аллокация на куче, копирование байтов. Если переданная строка большая — дорого.

С `std::move`:

```
    : name_(std::move(name)),
```

`name_(std::move(name))` — **перемещение**: `name_` забирает себе внутренний указатель и буфер `name`. Никакой копии. Источник (`name`) становится «пустым», но это нормально — мы её больше не используем.

В новом C++-коде вы будете часто видеть `std::move` в конструкторах для строк и других «дорогих» типов. Это идиома.

Тело конструктора

После инициализационного списка идёт обычное тело `{ }`. Здесь можно делать дополнительную работу — выводы, проверки, что угодно. Часто оно пустое.

В нашем `Player::Player` — пусто. Все поля инициализированы списком.

17.2.7 Что такое `this`

Внутри методов класса есть невидимый параметр **this** — указатель на текущий объект. Когда вы пишете внутри метода `hp_`, на самом деле подразумевается `this->hp_`.

```
int Player::hp() const {
    return hp_;          // эквивалент: return this->hp_;
}
```

Часто `this->` опускают для краткости (поэтому подчёркивание в имени поля помогает не путать с параметрами). Но иногда нужен явный `this`:

```
void Player::set_hp(int hp) {
    this->hp_ = hp;      // явно, чтобы не путать с параметром
}
```

Или когда нужно вернуть указатель/ссылку на сам объект:

```
Player& Player::heal(int amount) {
    hp_ += amount;
    if (hp_ > max_hp_) hp_ = max_hp_;
    return *this;        // вернуть ссылку на сам объект
}
```

Это позволяет цепочки: `player.heal(5).heal(10).heal(15)` (правда, в нашем коде мы возвращаем `void`).

17.2.8 Использование Player

В `Game::Game()` мы теперь создаём игрока:

```
Game::Game()
    : running_(true),
      world_(make_demo_world()),
      player_("Герой", 20),
      current_location_id_("glade") {
}
```

`player_("Герой", 20)` — вызов конструктора `Player`. Это и есть инициализационный список самого `Game`.

В `Game::run()`:

```
std::cout << "Привет, " << player_.name() << "!\n";
```

`player_.name()` — вызов `const`-метода. Возвращает `const std::string&`. Дальше `<<` для потока умеет работать со строкой.

17.3 Локация: класс посложнее

`Location` хранит: - ID (короткая строка-идентификатор). - Имя (для отображения). - Описание.
- Выходы — словарь «направление → ID соседней локации».

17.3.1 Объявление

`include/location.h:`

```
#ifndef RPG_LOCATION_H
#define RPG_LOCATION_H

#include <string>
#include <unordered_map>

namespace rpg {

class Location {
public:
    Location(std::string id, std::string name, std::string description);

    const std::string& id() const;
    const std::string& name() const;
    const std::string& description() const;

    void add_exit(const std::string& direction, const std::string& target_id);
    std::string exit(const std::string& direction) const;

    const std::unordered_map<std::string, std::string>& exits() const;

private:
```



```

    std::string id_;
    std::string name_;
    std::string description_;
    std::unordered_map<std::string, std::string> exits_;
};

} // namespace rpg

#endif

```

Очень похоже на Player, только полей побольше.

17.3.2 Определение

```

Location::Location(std::string id, std::string name, std::string description)
    : id_(std::move(id)),
      name_(std::move(name)),
      description_(std::move(description)) {
}

```

Конструктор инициализирует строки через `std::move`. `exits_` не упомянут — он автоматически создаётся пустым `unordered_map`-ом (default constructor).

```

void Location::add_exit(const std::string& direction, const std::string& target_id) {
    exits_[direction] = target_id;
}

std::string Location::exit(const std::string& direction) const {
    auto it = exits_.find(direction);
    if (it == exits_.end()) return {};
    return it->second;
}

```

`add_exit` — обычный setter. `exit` — поиск с возвратом пустой строки при отсутствии. Я предпочитаю возвращать пустую строку, а не выкидывать исключение — для нашей логики (не каждый ход надо проверять направления) это удобнее.

`return {};` — пустая инициализация для `std::string`. Эквивалент `return std::string();`.

Замечание: метод **называется** `exit`, но это не выход из программы. У нас в namespace `rpg`, имя не конфликтует с системным `exit()`. Так можно.

17.3.3 Использование

```
Location glade("glade", "Лесная поляна",
              "Залитая солнцем поляна. На западе тропа к деревне.");
glade.add_exit("west", "village");
glade.add_exit("east", "forest");
```

Создаём, заполняем выходами. Простой и понятный код.

17.4 Перегрузка операторов — короткий взгляд

В стандартной библиотеке вы видели:

```
std::string a = "hello";
std::string b = a + " world";           // оператор +
std::cout << a;                         // оператор <<
if (a == "hello") ...                  // оператор ==
```

Откуда эти операторы для `std::string`? Они **перегружены** в самом классе. C++ позволяет определить операторы для своих типов.

Подробный разбор — глава 20 (там вместе с шаблонами). Сейчас для контекста:

```
class Vector2 {
public:
    Vector2(double x, double y) : x_(x), y_(y) {}

    Vector2 operator+(const Vector2& other) const {
        return Vector2(x_ + other.x_, y_ + other.y_);
    }

private:
    double x_, y_;
};

int main() {
    Vector2 a(1, 2);
    Vector2 b(3, 4);
    Vector2 c = a + b;           // вызов a.operator+(b), даёт {4, 6}
}
```

В нашей игре операторов мы пока не делаем — обходимся методами `add_exit`, `exit`. Но потом, в иерархии предметов и в сохранениях, будем перегружать `<<` для потоков (как `std::cout << player`).

17.5 Конструктор по умолчанию

Если вы не пишете **никакого** конструктора, компилятор создаёт **конструктор по умолчанию** — без параметров. Он инициализирует поля их default-значениями.

```
class Empty {
    int x;
    std::string y;
};

Empty e;      // ОК: x неинициализирован (мусор), y пустая строка
```

Но как только вы написали **любой** свой конструктор — компилятор перестаёт автоматически создавать default. Поэтому:

```
class Player {
public:
    Player(std::string name, int max_hp);    // мы написали свой
};

Player p;    // ОШИБКА: нет конструктора без параметров
```

Если вам нужен ещё и default:

```
class Player {
public:
    Player();    // отдельно объявляем
    Player(std::string name, int max_hp);
};

Player::Player() : Player("Unnamed", 100) {}
// делегирование — вызов другого конструктора (C++11)
```

В нашем `Player` default-конструктор не нужен. В `World` — нужен:

```
class World {
public:
    World();    // явно объявляем
```

```

    // ...
};

World::World() = default;

```

= default — особый синтаксис: «компилятор, создай мне дефолтную реализацию». Полезно, когда хочется явно показать, что default-конструктор есть, но писать его руками нечего.

17.5.1 = delete

Антипод = default. Запрещает конструктор/метод:

```

class Singleton {
public:
    Singleton(const Singleton&) = delete;           // запрет копирования
    Singleton& operator=(const Singleton&) = delete; // запрет присваивания
};

```

Если кто-то попытается скопировать Singleton — ошибка компиляции. Полезно для классов, которые не должны копироваться (RAII-обёртки для уникальных ресурсов).

17.6 Копирование: rule of three

Что произойдёт, если мы скопируем объект?

```

Player a("Hero", 100);
Player b = a;           // копия

```

Компилятор автоматически генерирует **конструктор копирования**:

```

Player(const Player& other)
    : name_(other.name_),
      max_hp_(other.max_hp_),
      hp_(other.hp_) {}

```

Поле за полем. Для встроенных типов и std::string это работает корректно (std::string сам знает, как себя копировать).

Также генерируется **оператор присваивания при копировании**:

```

Player& operator=(const Player& other) {
    name_ = other.name_;

```

```

    max_hp_ = other.max_hp_;
    hp_ = other.hp_;
    return *this;
}

```

И **деструктор** (для Player он тривиальный — встроенные типы и `std::string` сами уничтожаются).

Это «**rule of zero**» (правило нуля): если ваш класс хранит только стандартные типы и контейнеры — **ничего не писать**, компилятор всё сгенерирует правильно.

17.6.1 Когда rule of zero ломается

Если поле — **сырой указатель** на динамическую память:

```

class IntHolder {
public:
    IntHolder(int v) : ptr_(new int(v)) {}
    ~IntHolder() { delete ptr_; }    // вот, освобождаем

private:
    int* ptr_;
};

IntHolder a(42);
IntHolder b = a;           // !!! компилятор скопирует ptr_ как есть
// теперь у a и b ОДИН указатель на тот же int
// при уничтожении обоих — двойной delete, UB!

```

Это типичный баг. Компилятор копирует **по умолчанию** поверхностно (shallow copy) — то есть копирует значения полей. Для указателя значение — адрес. Глубокого копирования (создать новый объект и скопировать содержимое) не делает.

Решение — «**rule of three**»: если у класса есть **нетривиальный деструктор** (нужен для управления ресурсом), скорее всего нужны и: - Конструктор копирования (правильно копировать ресурс). - Оператор присваивания (то же).

```

class IntHolder {
public:
    IntHolder(int v) : ptr_(new int(v)) {}

    ~IntHolder() { delete ptr_; }

```

```

    IntHolder(const IntHolder& other) : ptr_(new int(*other.ptr_)) {}

    IntHolder& operator=(const IntHolder& other) {
        if (this != &other) {                // защита от self-assignment
            int* new_ptr = new int(*other.ptr_);
            delete ptr_;
            ptr_ = new_ptr;
        }
        return *this;
    }

private:
    int* ptr_;
};

```

Целая программа из-за одного указателя. Поэтому в современном C++ **сырые указатели почти не используются** — берут `std::unique_ptr` (глава 18), у которого правильное копирование/перемещение из коробки.

17.6.2 Rule of five (C++11)

В C++11 добавились **move-семантика**: перемещение объекта без копирования. Это потребовало двух новых специальных функций: - Конструктор перемещения. - Оператор присваивания при перемещении.

Если у класса нетривиальный деструктор, скорее всего нужны **пять** функций — это **rule of five**:

1. Деструктор.
2. Конструктор копирования.
3. Оператор присваивания при копировании.
4. Конструктор перемещения (новинка).
5. Оператор присваивания при перемещении (новинка).

Подробно про move — в главе 19. Сейчас знайте, что в современном C++:

- **Rule of zero** — лучший выбор: всё стандартно, ничего писать не надо.
- **Rule of five** — если нужно управлять ресурсами руками.
- **Rule of three** — устаревшее, для C++03.

17.6.3 Запрет копирования

Иногда копирование нежелательно (например, наш `LogFile_BAD` из главы 11). Запрещаем явно:

```
class LogFile {
public:
    LogFile(const std::string& filename);
    ~LogFile();

    LogFile(const LogFile&) = delete;           // запрет копирования
    LogFile& operator=(const LogFile&) = delete;
};
```

Это удобно для RAII-классов: копировать файл (значит, иметь два владельца одного дескриптора) — глупость.

17.7 World: контейнер локаций

World хранит коллекцию Location-ов и предоставляет доступ по ID.

17.7.1 Объявление

include/world.h:

```
#ifndef RPG_WORLD_H
#define RPG_WORLD_H

#include "location.h"

#include <string>
#include <unordered_map>

namespace rpg {

class World {
public:
    World();

    void add(Location location);
    const Location* find(const std::string& id) const;
    std::size_t size() const;
};
```

```
private:
    std::unordered_map<std::string, Location> locations_;
};

World make_demo_world();

} // namespace rpg

#endif
```

Ключевое поле — `locations_` типа `unordered_map<string, Location>`. Карта строка-в-локацию. `World` владеет всеми локациями — они хранятся **по значению** внутри карты. При уничтожении `World` все локации уничтожатся (RAII!).

17.7.2 `find` возвращает указатель

```
const Location* World::find(const std::string& id) const;
```

Возвращает `const Location*`. Почему указатель, а не ссылка?

Потому что **может не найтись**. Ссылка не может быть «никакой» — она всегда на что-то указывает. Указатель может быть `nullptr`. В нашем случае:

```
const Location* loc = world_.find("unknown_id");
if (loc == nullptr) {
    // не найдено
}
```

Это вариант «**optional**». В C++17 есть `std::optional<T>`, но в C++11 — обходимся указателями.

17.7.3 Заводская функция

```
World make_demo_world();
```

Это **свободная функция** в `namespace rpg`, не метод класса. Создаёт демо-мир из трёх локаций. Возвращает `World` по значению.

В реализации:

```
World make_demo_world() {
    World w;
```



```
Location glade("glade", "Лесная поляна", "...");
glade.add_exit("west", "village");
glade.add_exit("east", "forest");

Location village("village", "Деревня", "...");
village.add_exit("east", "glade");

Location forest("forest", "Тёмный лес", "...");
forest.add_exit("west", "glade");

w.add(std::move(glade));
w.add(std::move(village));
w.add(std::move(forest));

return w;
}
```

Создаём World, три Location, перемещаем их в World, возвращаем сам World. `std::move` тут потому, что после `add` локальные `glade`, `village`, `forest` не нужны — пусть World забирает их «внутренности» без копирования.

Возврат World по значению — тоже без копирования благодаря **RVO** (return value optimization), о которой говорили в главе 10. Компилятор просто строит World сразу в нужном месте.

17.7.4 `emplace` вместо `insert+move`

В `World::add`:

```
void World::add(Location location) {
    std::string id = location.id();
    locations_.emplace(std::move(id), std::move(location));
}
```

`emplace` — это «вставка с конструированием на месте». Эффективнее `insert`:

- `insert(std::make_pair(id, location))` сначала создаёт пару, потом копирует/перемещает её в карту.
- `emplace(id, location)` пересылает аргументы прямо в конструктор пары внутри карты. Никаких промежуточных копий.

Тонкость: `emplace` ожидает аргументы конструктора `std::pair<key, value>`, и таких пар бывает два вида (с разными порядками копирования/перемещения). На наших размерах локаций разница незаметна, но привычка `emplace` вместо `insert` полезна.

17.7.5 Default-конструктор через `= default`

```
World::World() = default;
```

Я мог бы написать `World::World() {}` — пустое тело. Но `= default` явно говорит «дефолтная реализация». Тонкие отличия (для агрегатной инициализации и других вещей) для нас сейчас неважны; стиль `c = default` — более явный.

17.8 Game: всё вместе

Обновлённый `Game` теперь владеет `Player` и `World`. Карта команд расширилась.

17.8.1 Объявление

```
class Game {
public:
    Game();
    int run();

private:
    bool step();
    void execute(const std::string& cmd);

    void cmd_help() const;
    void cmd_look() const;
    void cmd_go(const std::string& direction);
    void cmd_status() const;

    bool running_;
    World world_;
    Player player_;
    std::string current_location_id_;
};
```

Каждая команда — отдельный метод. `cmd_help`, `cmd_look`, `cmd_status` — не меняют состояние (`const`). `cmd_go` меняет (`current_location_id_`).

17.8.2 Конструктор

```
Game::Game()  
    : running_(true),  
      world_(make_demo_world()),  
      player_("Герой", 20),  
      current_location_id_("glade") {  
}
```

Все поля инициализируются в списке. `world_(make_demo_world())` — вызов заводской функции, она возвращает `World` по значению, `world_` инициализируется этим значением (через `move`-конструктор `World`, который компилятор сгенерировал нам автоматически — *rule of zero!*).

17.8.3 Команды

```
void Game::execute(const std::string& cmd) {  
    std::istringstream ss(cmd);  
    std::string verb;  
    ss >> verb;  
  
    if (verb == "help") {  
        cmd_help();  
    } else if (verb == "look" || verb == "l") {  
        cmd_look();  
    } else if (verb == "go") {  
        std::string dir;  
        ss >> dir;  
        if (dir.empty()) {  
            std::cout << "Куда идти?\n";  
        } else {  
            cmd_go(dir);  
        }  
    } else if (verb == "status" || verb == "stat") {  
        cmd_status();  
    } else if (verb == "quit" || verb == "q" || verb == "exit") {  
        running_ = false;  
    }  
}
```

```

    } else {
        std::cout << "Неизвестная команда\n";
    }
}

```

Распознавание команд — пока через if/else. В главе 21 заменим на словарь имя → лямбда. Сейчас простой код.

```

void Game::cmd_look() const {
    const Location* loc = world_.find(current_location_id_);
    if (!loc) {
        std::cout << "Где-то в пустоте.\n";
        return;
    }
    std::cout << "== " << loc->name() << " ==\n";
    std::cout << loc->description() << "\n";

    if (!loc->exits().empty()) {
        std::cout << "Выходы:";
        for (const auto& exit_pair : loc->exits()) {
            std::cout << " " << exit_pair.first;
        }
        std::cout << "\n";
    }
}

```

world_.find(current_location_id_) — получаем указатель. Проверяем на nullptr. Если есть — печатаем имя, описание, выходы.

for (const auto& exit_pair : loc->exits()) — стандартная идиома обхода карты по const-ссылке.

```

void Game::cmd_go(const std::string& direction) {
    const Location* loc = world_.find(current_location_id_);
    if (!loc) return;

    std::string target = loc->exit(direction);
    if (target.empty()) {
        std::cout << "Туда не пройти.\n";
        return;
    }
}

```

```
    current_location_id_ = target;
    cmd_look();
}
```

Получаем текущую локацию, ищем выход в нужном направлении, если есть — меняем `current_location_id_`. После — `cmd_look()` показывает новую локацию.

17.9 Сборка и запуск

```
$ make clean && make
g++ ... -c src/main.cpp -o build/main.o
g++ ... -c src/game.cpp -o build/game.o
g++ ... -c src/location.cpp -o build/location.o
g++ ... -c src/player.cpp -o build/player.o
g++ ... -c src/world.cpp -o build/world.o
g++ ... build/main.o build/game.o build/location.o build/player.o build/world.o -o build/rpg
Built: build/rpg (debug)
```

Заметили? `make` сам подхватил новые `.cpp` файлы — wildcard `src/*.cpp` в `Makefile` собрал их все. Никаких ручных правок.

```
$ ./build/rpg
=== RPG-демо ===
Привет, Герой! Введите 'help' для справки.

== Лесная поляна ==
Залитая солнцем поляна. На западе видна тропа к деревне, на востоке — тёмный лес.
Выходы: east west

> status
Герой: HP 20/20

> go east
== Тёмный лес ==
Ветви скрипят над головой. Здесь сумрачно и тихо. На запад уходит тропа.
Выходы: west

> go west
```

== Лесная поляна ==

Залитая солнцем поляна. На западе видна тропа к деревне, на востоке — тёмный лес.

Выходы: east west

> go north

Туда не пройти.

> quit

До свидания.

Игра. Не богатая, но игра.

17.10 Что освоено в этой главе

1. **class vs struct** — разница в умолчании доступа.
2. **Поля, методы, инкапсуляция** — private/public.
3. **Конвенция _ в конце поля** — отличает от параметров.
4. **Конструктор и инициализационный список** — правильный способ инициализации полей.
5. **std::move в конструкторе** — для дорогих параметров.
6. **this** — указатель на текущий объект.
7. **const-методы** — обещание «не мутирую состояние».
8. **= default и = delete** — явные специальные функции.
9. **Rule of zero / rule of three / rule of five** — иерархия правил.
10. **Возврат указателя из find** — паттерн для «может не найтись».
11. **emplace вместо insert** — без промежуточных копий.

17.11 Маленькое упражнение

1. Добавьте в Player поле `int xp_` (опыт). Метод `void gain_xp(int amount)` и `int xp() const`. В `cmd_status` показывайте опыт.
2. Добавьте в игру новую локацию — пещеру (`cave`), доступную с лесной поляны на юг. Из пещеры можно вернуться на север.
3. Добавьте команду `where` — выводит ID текущей локации (для дебага).
4. Сделайте `Player::heal` так, чтобы он отказывался лечить мёртвого игрока (если `hp_ == 0`, ничего не делать, выдать сообщение).

5. Перепишите `Location::exit` так, чтобы он бросал `std::out_of_range`, если направление не найдено. Сравните с текущей версией — какая удобнее для нашего `cmd_go`?
6. (Сложнее) Добавьте `Location::add_item(std::string item_name)` и `std::vector<std::string> items()`. В `cmd_look` показывайте список предметов: «На полу лежит: меч, золото».
7. (Сложнее) Прочитайте код `World::add`. Объясните, почему `std::string id = location.id();` копирует строку, а `std::move(id)` дальше уже её перемещает.

17.12 Что дальше

Глава 17 — **наследование и виртуальные функции**. Создадим иерархию `Entity` → `Player`/`Enemy`/`NPC` и параллельную `Item` → `Weapon`/`Armor`/`Consumable`. Это полиморфизм — главная идея ООП. Узнаем, что такое `vtable`, почему виртуальный деструктор обязателен, что такое `override` и `final`.

После 17-й главы у нас будут враги, оружие, броня. Бои добавятся в главе 18 (заодно введём `unique_ptr` для управления предметами).

Темп Части II: новый класс или две — каждую главу, постепенно. К 25-й главе соберём всё в живую игру.

Глава 18

Глава 17. Наследование и виртуальные функции

В этой главе — главная тема ООП в C++: **наследование** и **полиморфизм**. На примере нашего проекта построим иерархию предметов: базовый `Item`, и три потомка — `Weapon`, `Armor`, `Consumable`. Локация и инвентарь смогут хранить указатели на базу `Item*` — а внутри будут лежать конкретные оружия, брони, расходники. Один и тот же `item->describe()` будет работать по-разному в зависимости от реального типа объекта. Это и есть полиморфизм.

К концу главы вы поймёте: что такое `virtual`, зачем `override`, почему виртуальный деструктор обязателен, что такое `vtable`, в чём проблема «slicing». Тема концептуально тяжёлая, но если разберётесь — большая часть страшности C++ останется позади.

18.1 Зачем наследование

Предположим, нам в игре нужны три типа предметов: оружие (даёт урон), броня (даёт защиту), расходник (даёт эффект при использовании). У всех общее: - Имя. - Вес. - Возможность отобразить в инвентаре.

У каждого свои особенности: - `Weapon`: урон. - `Armor`: защита. - `Consumable`: что делает при использовании.

Можно было бы написать три отдельных класса без связи. Локация бы тогда хранила три коллекции:

```
class Location {  
    std::vector<Weapon> weapons_;  
    std::vector<Armor> armors_;
```



```
std::vector<Consumable> consumables_;  
};
```

Дальше команда `take <name>` искала бы предмет в трёх местах. Команда `look` показывала бы три цикла. Если завтра появится «волшебный артефакт» — четвёртая коллекция, четвёртый цикл. И повторение кода: имена, вес, описание — у всех одинаковые.

Решение — **общая база**. Создаём абстрактный класс `Item`, от него наследуем три конкретных. Локация хранит один список **указателей на базу**:

```
class Location {  
    std::vector<Item*> items_;  
};
```

Когда нужно «отобразить» предмет, мы вызываем `item->describe()` — компилятор сам выберет нужную реализацию в зависимости от реального типа. Это и есть полиморфизм.

18.2 Базовый синтаксис наследования

```
class Item {  
public:  
    Item(std::string name, int weight);  
  
    const std::string& name() const;  
    int weight() const;  
  
protected:  
    std::string name_;  
    int weight_;  
};  
  
class Weapon : public Item {  
public:  
    Weapon(std::string name, int weight, int damage);  
  
    int damage() const;  
  
private:  
    int damage_;  
};
```

`class Weapon : public Item` — **Weapon наследует от Item**. Это значит:

1. У `Weapon` есть **все поля и методы** `Item` плюс свои.
2. `Weapon` — это «тоже `Item`» в каком-то смысле: можно положить `Weapon*` туда, где ждут `Item*`.

Объявление `Weapon`:

```
Weapon sword("ржавый меч", 5, 8);

std::cout << sword.name();      // OK, унаследовано от Item
std::cout << sword.weight();    // OK, унаследовано
std::cout << sword.damage();    // OK, собственное
```

18.2.1 `public`, `protected`, `private` — два смысла

В заголовке `class Weapon : public Item` слово `public` определяет **тип наследования**:

- `public Item` — публичное наследование. `Weapon` — это `Item` снаружи. Самый частый вид.
- `protected Item` — наследование «для своих». Внешний код не знает о связи.
- `private Item` — наследование как «реализация через». `Weapon` использует `Item` внутри, но снаружи не виден.

В нашей книге будем использовать только **`public`** — самый частый и осмысленный.

Внутри класса слова `public/protected/private` определяют **доступ к членам**. С `protected` мы пока не встречались. Это «доступно классу и его потомкам, но не снаружи»:

```
class Item {
protected:
    std::string name_;
    // доступен снаружи: только через name()
    // доступен потомкам: напрямую как this->name_ или просто name_
};

class Weapon : public Item {
public:
    std::string describe() const {
        return name_ + " (оружие)"; // OK: protected доступно потомкам
    }
};
```

Если бы `name_` был `private` — `Weapon` не смог бы к нему обращаться напрямую. Пришлось бы через `name()` (геттер).

Когда что использовать: - `private` — для внутренних деталей класса, потомкам не нужно лезть. - `protected` — для общих полей/методов базы, к которым потомки хотят прямой доступ. - `public` — то, что доступно снаружи.

В нашем `Item` поля `name_` и `weight_` — `protected`, чтобы потомки могли их печатать в своём `describe`.

18.3 Конструктор базы

При создании `Weapon` сначала вызывается конструктор `Item`, потом — собственный конструктор `Weapon`. Вы обязаны указать, как звать конструктор базы:

```
Weapon::Weapon(std::string name, int weight, int damage)
    : Item(std::move(name), weight),    // вызов конструктора базы
      damage_(damage) {
}
```

В инициализационном списке **первой** идёт инициализация базы — синтаксис как для поля, но с именем класса. Перед собственными полями. Без этого:

```
Weapon::Weapon(std::string name, int weight, int damage)
    : damage_(damage) {                // забыли про Item
}
```

— компилятор попытается вызвать `Item()` (default-конструктор). Раз у `Item` его нет (мы написали свой с параметрами) — будет ошибка компиляции.

18.4 Виртуальные функции

Теперь центральная вещь. Допустим, у `Item` есть метод `describe()`, который возвращает описание:

```
class Item {
public:
    std::string describe() const {
        return name_ + " (предмет)";
    }
};
```

И у `Weapon` мы хотим, чтобы `describe()` показывал и урон:

```
class Weapon : public Item {
public:
    std::string describe() const {
        return name_ + " (оружие, урон " + std::to_string(damage_) + ")";
    }
};
```

Что произойдёт?

```
Weapon sword("меч", 5, 10);
std::cout << sword.describe();    // "меч (оружие, урон 10)" □

Item& item = sword;
std::cout << item.describe();      // "меч (предмет)" □
```

Через ссылку базы вызывается метод **базы**, а не потомка. Это **static dispatch** — компилятор смотрит на тип переменной (Item&), не на реальный объект (Weapon).

Это не то, что мы хотим. Хотим: «какой бы тип реально ни был, вызови подходящий метод». Для этого — **virtual**:

```
class Item {
public:
    virtual std::string describe() const {
        return name_ + " (предмет)";
    }
};

class Weapon : public Item {
public:
    std::string describe() const override {
        return name_ + " (оружие)";
    }
};
```

Ключевое слово **virtual** в базе говорит: «этот метод можно переопределить, и при вызове через ссылку/указатель базы выбирай реализацию по реальному типу».

Теперь:

```
Item& item = sword;
std::cout << item.describe();    // "меч (оружие)" □ dynamic dispatch
```

18.4.1 `override` — обязательная привычка

Слово `override` в потомке — **не магия, а проверка**. Оно говорит компилятору: «эта функция переопределяет виртуальную функцию базы». Если что-то не так — ошибка компиляции.

Без `override` опечатки молча создают новую функцию:

```
class Item {
public:
    virtual std::string describe() const { ... }
};

class Weapon : public Item {
public:
    std::string describe() const { ... }    // ОПЕЧАТКА: describe вместо describe
};
```

Без `override` компилятор подумает: «нормально, метод `describe` в `Weapon`, рядом с унаследованным `describe`». Тестирование укажет, что переопределение не сработало. Пол часа поисков, недоумение.

С `override`:

```
std::string describe() const override { ... }
// error: 'describe' marked 'override' but does not override
```

Компилятор сразу указывает на проблему.

Правило: ВСЕГДА ставьте `override` на переопределяющие методы. Это бесплатный страховочный полис.

18.4.2 `final` — запрет дальнейшего переопределения

Слово `final` запрещает потомкам переопределять метод:

```
class Item {
public:
    virtual std::string describe() const { ... }
};

class Weapon : public Item {
public:
    std::string describe() const override final { ... }
};
```

```
class MagicSword : public Weapon {
public:
    std::string describe() const override { ... }    // ОШИБКА: переопределение final
};
```

`final` можно применить и к классу — запретит наследование вообще:

```
class Weapon final : public Item {
    // от Weapon нельзя наследоваться
};
```

В нашем `Item.h` я как раз помечу `Weapon`, `Armor`, `Consumable` как `final` — на текущем этапе нам не нужно их расширять, и `final` помогает компилятору оптимизировать (статически разрешить виртуальные вызовы).

18.5 Чисто виртуальные функции и абстрактные классы

В `Item` есть метод `kind()` — возвращает тип («оружие», «броня», ...). У базового `Item` непонятно, что вернуть — это зависит от потомка. Делаем метод **чисто виртуальным**:

```
class Item {
public:
    virtual std::string kind() const = 0;    // = 0 значит «нет реализации»
};
```

`= 0` в конце — это синтаксис «pure virtual». Реализации нет; потомки **обязаны** её предоставить.

Класс, у которого есть хоть одна чисто виртуальная функция — **абстрактный**. Объекты абстрактного класса нельзя создать:

```
Item item("...", 5);    // ОШИБКА: cannot allocate object of abstract type
Item* p = new Item(...);    // тоже ошибка
```

Можно только указатели и ссылки:

```
Item* p = new Weapon(...);    // ОК
Item& r = some_weapon;        // ОК
```

Это удобно: запрещает «безсмысленные» объекты базы.

Конкретные потомки должны реализовать все чисто виртуальные методы:

```
class Weapon : public Item {
public:
    std::string kind() const override { return "оружие"; }    // обязательно
};
```

Если забыть — `Weapon` останется абстрактным, его тоже нельзя будет создать. Компилятор быстро выявит.

18.6 Виртуальный деструктор — обязателен!

Главный подвох наследования. Покажу на ошибке.

```
class Item {
public:
    Item(std::string n) : name_(n) {}
    ~Item() { std::cout << "Item dtor\n"; }    // HE virtual!
protected:
    std::string name_;
};

class Weapon : public Item {
public:
    Weapon(std::string n, int* big_array)
        : Item(n), buffer_(big_array) {}
    ~Weapon() {
        std::cout << "Weapon dtor\n";
        delete[] buffer_;
    }
private:
    int* buffer_;
};

int main() {
    Item* p = new Weapon("...", new int[1000]);
    delete p;    // ВНИМАНИЕ
}
```

Что произойдёт при `delete p`? `p` имеет тип `Item*`. Компилятор смотрит на тип указателя, видит — деструктор `Item`. Вызывает `~Item()`. `~Weapon()` **НЕ вызывается!**

Результат: `Weapon::buffer_` не освобождён, утечка памяти 4 КВ. И это ещё «милый» случай — если `Weapon` владел файлом или мьютексом, последствия серьёзнее.

Это **undefined behavior** в строгом смысле: стандарт C++ говорит, что `delete` через указатель базы без виртуального деструктора — UB. На практике обычно «работает» с утечкой; но в общем случае может быть что угодно.

Решение: деструктор базы должен быть **virtual**:

```
class Item {
public:
    virtual ~Item() = default;    // вот так
};
```

Тогда при `delete p` компилятор смотрит в **vtable** реального объекта, находит правильный деструктор (`~Weapon`), вызывает его. Внутри `~Weapon` потом автоматически вызывается `~Item` базы. Цепочка корректная, утечки нет.

`= default` — потому что для `Item` деструктор тривиальный (только освободить `std::string name_`, что и так само). `virtual ~Item() = default;` — идиома для «у меня виртуальный деструктор, но особой логики нет».

Правило: если класс задумывается как база для наследования, его деструктор обязан быть **virtual**. Без исключений.

Если класс **не** задумывается как база — деструктор пусть будет неvirtуальным или вовсе не объявлен. Виртуальность стоит лишние байты (vtable pointer внутри объекта).

18.7 vtable — как это работает

Под капотом виртуальные вызовы устроены так.

Каждый класс с хотя бы одной виртуальной функцией имеет **vtable** (virtual table) — массив указателей на методы. Где-то в `.rodata`-секции бинарника лежит:

```
vtable_Item:
    &Item::describe      // или &Weapon::describe, ...
    &Item::kind          // или &Weapon::kind, ...
    &Item::~~Item        // или &Weapon::~~Weapon, ...
    ...
```

Каждый объект класса с виртуальными функциями имеет **скрытый указатель** на свою vtable (обычно как первое поле). Размер объекта увеличивается на 8 байт.

`Item* p` → объект `Weapon` в памяти:


```

[vtable ptr: → vtable_Weapon]  <- скрытое поле
[name_: "меч"]                  <- из Item
[weight_: 5]                    <- из Item
[damage_: 10]                   <- из Weapon

```

Когда вы пишете `p->describe()`:

1. Компилятор знает, что `describe` — виртуальный.
2. Генерирует код: «возьми `vtable` из объекта по адресу `p`, найди в ней `describe`, вызови».
3. Поскольку у `Weapon` своя `vtable`, в ней `describe` указывает на `Weapon::describe`.

Это **дополнительный косвенный вызов** — чуть медленнее, чем прямой. Обычно — единицы наносекунд, не критично. Но если виртуальный метод вызывается миллионы раз за секунду — может стать узким местом. Тогда смотрят: правда ли нужен полиморфизм здесь?

18.8 Слайсинг — главная ловушка

```

Weapon sword("меч", 5, 10);

Item item = sword;          // !!!
std::cout << item.describe();

```

Это копирование `sword` в **новый объект** `item` типа `Item`. При копировании поля `Weapon` (`damage_`) **отсекаются** — у `Item` их нет. Это **slicing** (срезание).

Объект `item` — настоящий `Item`, не `Weapon`. Даже если методы виртуальные, `item.describe()` вызовет `Item::describe()`.

Чтобы избежать `slicing`:

- **Никогда не присваивайте объект потомка в переменную базы.** Только указатели или ссылки.
- Если нужно «как-то хранить разнотипные объекты» — `std::vector<Item*>` или `std::vector<std::unique_ptr<Item>>` (глава 18).

В нашем `Item` я сделал ещё страховку:

```

class Item {
public:
    Item(const Item&) = delete;
    Item& operator=(const Item&) = delete;
    Item(Item&&) = delete;

```

```
Item& operator=(Item&&) = delete;
};
```

Запрет копирования и перемещения. Если кто-то попытает `Item item = sword;` — ошибка компиляции. Безопасно: объект `Item` живёт ровно один раз, доступ к нему только через указатели/ссылки.

В реальной жизни иногда копирование нужно. Тогда определяют **виртуальный `clone()`-метод**:

```
class Item {
public:
    virtual std::unique_ptr<Item> clone() const = 0;
};

class Weapon : public Item {
public:
    std::unique_ptr<Item> clone() const override {
        return std::unique_ptr<Item>(new Weapon(*this));
    }
};
```

`item->clone()` создаёт копию реального типа. Это идиома **prototype**. У нас в книге она появится позже — для копирования предметов в инвентаре.

18.9 `dynamic_cast` — приведение базы к потомку

Иногда у вас на руках `Item*`, и нужно «если это оружие — узнать урон». В C++ для этого есть `dynamic_cast`:

```
Item* p = ...;
if (Weapon* w = dynamic_cast<Weapon*>(p)) {
    int dmg = w->damage();
    ...
}
```

`dynamic_cast<Weapon*>(p)`: - Если объект **реально** `Weapon` или его потомок — возвращает `Weapon*`.
- Иначе — `nullptr`.

То же для ссылок (но при «не тот тип» бросает `std::bad_cast`, ловите через `try/catch`):

```
try {  
    Weapon& w = dynamic_cast<Weapon>(some_item);  
    ...  
} catch (const std::bad_cast&) {  
    std::cout << "не оружие\n";  
}
```

`dynamic_cast` использует **RTTI** (runtime type information) — компилятор записывает в `vtable` имя класса, и `dynamic_cast` это проверяет. Требуется, чтобы у базы был **хотя бы один виртуальный метод** (иначе RTTI не работает; обычно работает, потому что виртуальный деструктор уже есть).

`dynamic_cast` — относительно медленная операция (несколько указательных переходов). Не злоупотребляйте — если в каждой второй функции нужен `dynamic_cast`, скорее всего, дизайн классов плохой. Подумайте: можно ли вместо `if (Weapon*)` сделать ещё один виртуальный метод?

В нашей игре `dynamic_cast` пригодится в команде «надеть» — отличить `Armor` от `Weapon` чтобы понять, в какой слот вставлять.

18.10 Множественное наследование — кратко

В C++ можно наследоваться **сразу от нескольких классов**:

```
class Swimmer { virtual void swim() = 0; };  
class Flyer { virtual void fly() = 0; };  
  
class Duck : public Swimmer, public Flyer {  
    void swim() override { ... }  
    void fly() override { ... }  
};
```

Это **множественное наследование** (multiple inheritance). Считается «острым» — много граблей (diamond problem, неоднозначность имён). Java и C# его не разрешают вовсе; вместо этого — наследование от одного класса + реализация многих интерфейсов.

В C++ совет: **избегайте множественного наследования**, кроме одного случая:

Множественное наследование от интерфейсов — классов с одними чисто виртуальными функциями, без полей. Это безопасно и похоже на интерфейсы Java/C#.

```
class Printable {
public:
    virtual ~Printable() = default;
    virtual std::string to_string() const = 0;
};

class Comparable {
public:
    virtual ~Comparable() = default;
    virtual int compare(const Comparable& other) const = 0;
};

class Item : public Printable, public Comparable {
    ...
};
```

В нашем коде множественного наследования не будет.

18.11 Item-иерархия в проекте

Соберём всё. include/item.h:

```
#ifndef RPG_ITEM_H
#define RPG_ITEM_H

#include <string>

namespace rpg {

class Item {
public:
    Item(std::string name, int weight);
    virtual ~Item() = default;

    Item(const Item&) = delete;
    Item& operator=(const Item&) = delete;
    Item(Item&&) = delete;
    Item& operator=(Item&&) = delete;
```

```
const std::string& name() const;
int weight() const;

virtual std::string kind() const = 0;
virtual std::string describe() const;

protected:
    std::string name_;
    int weight_;
};

class Weapon final : public Item {
public:
    Weapon(std::string name, int weight, int damage);

    int damage() const;
    std::string kind() const override;
    std::string describe() const override;

private:
    int damage_;
};

class Armor final : public Item {
public:
    Armor(std::string name, int weight, int defense);

    int defense() const;
    std::string kind() const override;
    std::string describe() const override;

private:
    int defense_;
};

class Consumable final : public Item {
public:
    Consumable(std::string name, int weight, int heal_amount);
```

```

    int heal_amount() const;
    std::string kind() const override;
    std::string describe() const override;

private:
    int heal_amount_;
};

} // namespace rpg

#endif

```

Разберём ключевые моменты:

База Item: - Виртуальный деструктор `virtual ~Item() = default;` — обязательно. - Запрет копирования/перемещения через `= delete` — защита от slicing. - Поля `protected`, чтобы потомки могли их печатать. - `kind()` чисто виртуальный — `Item` абстрактный, его нельзя создать. - `describe()` обычный виртуальный — с дефолтной реализацией.

Потомки: - `final` после имени класса — нельзя дальше наследоваться. - `override` на всех переопределяемых методах.

`src/item.cpp` (фрагменты):

```

// Реализация базы
Item::Item(std::string name, int weight)
    : name_(std::move(name)), weight_(weight) {}

const std::string& Item::name() const { return name_; }
int Item::weight() const { return weight_; }

std::string Item::describe() const {
    std::ostringstream ss;
    ss << name_ << " (" << kind() << ", вес " << weight_ << ")";
    return ss.str();
}

```

В `Item::describe()` мы вызываем `kind()` — виртуальный! Хотя сами мы в базе, во время выполнения через `vtable` будет вызван `kind()` реального объекта. Это нормально — внутри методов базы виртуальные вызовы тоже работают полиморфно.

```
// Реализация Weapon
Weapon::Weapon(std::string name, int weight, int damage)
    : Item(std::move(name), weight), damage_(damage) {
}

int Weapon::damage() const { return damage_; }

std::string Weapon::kind() const { return "оружие"; }

std::string Weapon::describe() const {
    std::ostringstream ss;
    ss << name_ << " (оружие, урон " << damage_ << ", вес " << weight_ << ")";
    return ss.str();
}
```

Weapon явно переопределяет describe() для добавления урона. Если бы не переопределили — работала бы Item::describe(), использующая kind() = "оружие". Тоже бы вышло осмысленно, но без урона.

18.12 Тестируем полиморфизм

tests/test_items.cpp:

```
#include "item.h"
#include <iostream>
#include <vector>

int main() {
    using namespace rpg;

    std::vector<Item*> items;
    items.push_back(new Weapon("ржавый меч", 5, 8));
    items.push_back(new Armor("кожаный нагрудник", 8, 4));
    items.push_back(new Consumable("зелье лечения", 1, 15));

    // Полиморфный обход
    for (Item* it : items) {
        std::cout << "- " << it->describe() << "\n";
    }
}
```

```

// dynamic_cast
for (Item* it : items) {
    if (Weapon* w = dynamic_cast<Weapon*>(it)) {
        std::cout << w->name() << " наносит " << w->damage() << "\n";
    }
}

for (Item* it : items) {
    delete it;
}

return 0;
}

```

Запуск:

- ржавый меч (оружие, урон 8, вес 5)
- кожаный нагрудник (броня, защита 4, вес 8)
- зелье лечения (расходник, лечит 15, вес 1)

ржавый меч наносит 8 урона

Видим: один и тот же `it->describe()` даёт разный вывод для разных реальных типов. Один и тот же `dynamic_cast<Weapon*>` срабатывает только на `Weapon`. Это полиморфизм в работе.

В цикле `delete it` мы освобождаем память. Без виртуального деструктора в `Item` это вызывало бы UB и утечку `damage_/defense_/heal_amount_`. С виртуальным — корректно вызывается `~Weapon/~Armor/~Consumable`, потом `~Item`, всё чисто.

В главе 18 заменим `Item*` и `new/delete` на `std::unique_ptr<Item>` — будет ещё короче и безопаснее.

18.13 Несколько тонкостей

18.13.1 Виртуальный вызов в конструкторе/деструкторе

В конструкторе базы виртуальный механизм **не работает**:

```

class Item {
public:
    Item() {
        std::cout << "creating " << kind(); // !!! вызовет Item::kind, не Weapon::kind
    }
}

```



```
    }  
    virtual std::string kind() const = 0;  
};
```

На момент конструктора `Item` объект ещё не «стал» `Weapon`. `Vtable` указывает на `vtable Item`. Для чисто виртуального метода (`= 0`) это вообще приведёт к **UB** или к ошибке линковки.

В деструкторе то же — `vtable` уже «откатилась» к базе.

Правило: не вызывайте виртуальные методы из конструктора/деструктора базы.

Если нужно похожее поведение — отдельный метод `init()`, который вызывается после конструктора.

18.13.2 Вызов метода базы из потомка

Иногда из переопределённого метода хочется вызвать базовую реализацию:

```
class Weapon : public Item {  
public:  
    std::string describe() const override {  
        return Item::describe() + ", урон " + std::to_string(damage_);  
    }  
};
```

`Item::describe()` — явный квалифицированный вызов. Обходит виртуальный механизм, вызывает именно версию `Item`.

Удобно, когда хочется «дополнить» базовое поведение, не дублируя его. У нас в книге пока не пользуемся, но идиома известная.

18.13.3 `protected` и инвариант класса

Делать поля `protected` — иногда соблазнительно, но риск.

Если поле `private`, доступ только через методы. Можете в методе валидировать (например, `take_damage` не пускает `hp` в отрицательные). Потомки тоже идут через метод.

Если поле `protected`, потомки могут менять напрямую. Инварианты класса могут нарушиться.

Рекомендация: лучше держать поля `private` и давать `protected` getters/setters, если потомкам нужно. Но это в идеале; в нашем `Item` поля `protected` для удобства (потомки печатают их в `describe`).

18.14 Альтернативы наследованию

Наследование не единственный способ переиспользования. Современный C++ часто советует **композицию вместо наследования**:

```
// Вместо наследования
class Magic : public Weapon { ... };

// Композиция
class Weapon {
    MagicEffect effect_; // оружие имеет эффект
};
```

«Оружие имеет эффект» — композиция (has-a). «Магическое оружие — это оружие» — наследование (is-a). Если is-a сомнительно, лучше композиция.

Наследование добавляет **связанность**: потомок зависит от базы. Менять базу больно — все потомки могут сломаться. Композиция связана слабее: меняете внутреннее устройство, наружу не видно.

В нашей игре Item → Weapon/Armor/Consumable — настоящий is-a. «Меч — это предмет». Полностью оправдано.

Но не делайте наследование «потому что в учебнике написано». Если можно композицией — обычно лучше композицией.

18.15 Главные правила главы

1. **Виртуальный деструктор** для любого класса, который задумывается как база. **Обязательно.**
2. **override** на каждом переопределении. Бесплатная защита от опечаток.
3. **final** на классах/методах, которые не будут расширять — даёт компилятору оптимизировать.
4. **Никогда не присваивайте объект потомка в переменную базы по значению** — slicing.
5. **Запрещайте копирование** для классов в иерархии — = delete. Доступ только через указатели/ссылки.
6. **dynamic_cast** для безопасного приведения — но злоупотребление — признак плохого дизайна.
7. **Виртуальные методы не работают в конструкторе/деструкторе базы** — не зовите их там.

8. **Композиция часто лучше наследования** — наследуйте только когда is-a реально и иерархия плоская.

18.16 Маленькое упражнение

1. Добавьте новый тип предмета — `KeyItem` (квестовый предмет, нельзя выбросить). Чисто наследник `Item` с переопределённым `kind()` = "квестовый" и описанием.
2. Создайте `Item* it = new Weapon(...)` без виртуального деструктора в `Item` (нарочно уберите `virtual ~Item()`). Запустите под `ASan` (`make CXXFLAGS=... с -fsanitize=address`). `ASan` должен указать утечку.
3. Попробуйте `Item item("test", 5);` — должна быть ошибка компиляции (абстрактный класс). Прочтите сообщение.
4. Уберите `override` у `Weapon::describe()` и переименуйте его в `describe()`. Что произойдёт при сборке? Что произойдёт при попытке вызвать `weapon.describe()`?
5. Добавьте в `Item` виртуальный метод `int value() const` — «цена предмета». В `Weapon` — `damage_ * 10`, в `Armor` — `defense_ * 15`, в `Consumable` — `heal_amount_ * 5`. Используйте в тесте.
6. (Сложнее) Добавьте `virtual std::unique_ptr<Item> clone() const` для копирования предметов. Реализуйте в потомках. Какие проблемы это создаёт?
7. (Сложнее) Перепишите `Item` так, чтобы `Weapon/Armor/Consumable` не наследовались, а использовали композицию: `Item` хранит `enum Type` и `union` для специфических данных. Сравните, какой подход проще для добавления нового типа.

18.17 Что дальше

Глава 18 — **умные указатели**: `std::unique_ptr`, `std::shared_ptr`, `std::weak_ptr`. Применим их к нашим `Item`: уберём ручной `new/delete`, заменим на `unique_ptr<Item>`. Добавим **Inventory** — игрок сможет таскать предметы. Локации тоже смогут содержать предметы — командой `take` будем их подбирать.

Это завершит «инфраструктуру предметов». После — бои, NPC, парсер мира из файла, сейвы. К 25-й главе соберётся всё.

Глава 19

Глава 18. Умные указатели — unique_ptr, shared_ptr, weak_ptr

В предыдущей главе мы построили иерархию `Item`. Тестировали полиморфизм через `std::vector<Item*>`, выделяя предметы через `new` и освобождая через `delete`. Это работало, но было хрупко: забудьте `delete` — утечка, забудьте про виртуальный деструктор — UB, выкиньте исключение между `new` и `delete` — снова утечка.

Эта глава — про **умные указатели** (smart pointers). Они владеют объектом на куче и сами заботятся о его освобождении в нужный момент. Никаких ручных `delete`. Никаких утечек. Это и есть «современный C++» в одном понятии.

Параллельно построим класс `Inventory`, и в игре наконец-то можно будет брать и бросать предметы.

19.1 Зачем нужны умные указатели

Вспомним проблему. У нас был такой код:

```
std::vector<Item*> items;
items.push_back(new Weapon("меч", 5, 10));
// ...

for (Item* it : items) {
    delete it;
}
items.clear();
```

Что может пойти не так:

1. **Забыли delete.** Память не освобождена.
2. **Между new и delete исключение.** Деструкторы не вызовутся для оставшихся.
3. **Возвращаем Item* из функции.** Кто его освобождает? Без документации непонятно.
4. **Два указателя на один объект.** Оба пытаются delete — UB.
5. **Использование после delete.** Висячий указатель — UB.

В главе 11 я ввёл RAII как решение этих проблем. Умные указатели — это RAII для куча-указателей. Они конкретные классы, которые **владеют** объектом на куче и в деструкторе вызывают delete.

Их три:

- `std::unique_ptr<T>` — единоличный владелец. Только один указатель на объект.
- `std::shared_ptr<T>` — совладельцы. Объект живёт, пока есть хотя бы один указатель.
- `std::weak_ptr<T>` — наблюдатель. Сам не владеет, но может проверить, жив ли объект.

Все три — в заголовке `<memory>`.

19.2 unique_ptr — основной рабочий

`std::unique_ptr<T>` — самый частый. Если вы выделяете объект на куче и есть **один** ответственный за его жизнь — это `unique_ptr`.

19.2.1 Создание

```
#include <memory>

std::unique_ptr<int> p(new int(42));
std::cout << *p << "\n";    // 42
```

`p` владеет `int`-ом со значением 42. Когда `p` выходит из области видимости (или уничтожается как-то иначе), его деструктор делает `delete` на хранимый указатель.

Полная безопасность: даже если будет исключение между `new int(42)` и концом функции — `p` всё равно уничтожится, `int` освободится.

Использование почти как обычный указатель:

```
*p = 100;           // разыменование
p->method();         // вызов метода (если T — класс)
T* raw = p.get();    // получить сырой указатель (без передачи владения)
bool ok = bool(p);   // проверка на nullptr — p != nullptr
```

19.2.2 Запрет копирования

Главное свойство: unique_ptr **нельзя копировать**.

```
std::unique_ptr<int> p(new int(42));  
std::unique_ptr<int> q = p;    // ОШИБКА компиляции
```

Почему? Если бы можно было — у нас стало бы два владельца, оба бы попытались delete. Двойное освобождение.

Поэтому копи-конструктор unique_ptr помечен = delete. Создатели стандартной библиотеки об этом подумали.

19.2.3 Перемещение разрешено

Что можно — **переместить** владение:

```
std::unique_ptr<int> p(new int(42));  
std::unique_ptr<int> q = std::move(p);    // q теперь владеет, p пустой  
  
if (!p) std::cout << "p пустой\n";    // выведется  
std::cout << *q << "\n";            // 42
```

std::move(p) говорит: «забери у p всё, что он держит». После этого p становится nullptr, владельцем становится q.

Это идиома **передачи владения**. В функциях:

```
std::unique_ptr<Item> make_sword() {  
    return std::unique_ptr<Item>(new Weapon("меч", 5, 10));  
}  
  
void use_item(std::unique_ptr<Item> item) {  
    // функция взяла владение  
    std::cout << item->describe();  
    // в конце item уничтожится, delete вызовется  
}  
  
int main() {  
    std::unique_ptr<Item> sword = make_sword();  
    // ...  
    use_item(std::move(sword));  
}
```

```
// sword теперь пустой
}
```

Возврат unique_ptr из функции — нормально (move выполняется автоматически). Передача в функцию — через std::move.

19.2.4 make_unique — фабричная функция

В C++14 добавили std::make_unique:

```
auto p = std::make_unique<Weapon>("меч", 5, 10);
// эквивалент: std::unique_ptr<Weapon>(new Weapon("меч", 5, 10))
```

Преимущество — нет голого new. И ещё чуть-чуть исключение-безопасности (между new и конструктором unique_ptr теоретически могло что-то выкинуться, с make_unique — нет).

В C++11 make_unique нет. У нас C++11-база. Поэтому пишем шим:

```
// include/util.h
template <typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

Поместил в namespace rpg, чтобы не конфликтовать с системным std::make_unique (хотя в C++11 его и нет — но в C++14+ есть, и наш код тогда должен работать).

Args&&... — variadic template, передаёт аргументы как они есть (perfect forwarding). Шаблоны и &&... — это глава 20. Сейчас просто принимайте как идиому: «rpg::make_unique<T>(args...) создаёт unique_ptr<T> с передачей args в конструктор T».

Использование:

```
auto sword = rpg::make_unique<Weapon>("меч", 5, 10);
```

Это и есть современный способ выделения на куче в нашем коде.

19.2.5 Кастомные deleters

unique_ptr обычно использует delete для освобождения. Иногда нужно другое — например, для C-API нужно вызвать fclose:

```
auto deleter = [](FILE* f) { if (f) std::fclose(f); };
std::unique_ptr<FILE, decltype(deleter)> file(std::fopen("test.txt", "r"), deleter);
// file сам закроет файл
```

Это редко нужно — обычно лучше написать свой RAII-класс. Но в системном коде встречается.

19.2.6 unique_ptr для массивов

Для массивов есть особая форма unique_ptr<T[]>:

```
std::unique_ptr<int[]> arr(new int[100]);
arr[0] = 1;
arr[99] = 100;
// в деструкторе автоматически delete[]
```

Использует delete[] (не delete). Поэтому отдельный синтаксис.

В современном C++ почти всегда вместо unique_ptr<T[]> берут std::vector<T>. Вектор тот же массив на куче, плюс знает свой размер. Используйте unique_ptr<T[]> редко, для совместимости со старым кодом.

19.3 shared_ptr — совладение

Иногда у объекта несколько владельцев. Пример: окно UI, на которое ссылаются и менеджер окон, и виджет внутри окна. Какой из них освобождает? Если кто-то один — другой может потом обратиться к освобождённой памяти.

Решение — **подсчёт ссылок**. Объект живёт, пока есть хоть один указатель на него. Уничтожается, когда последний пропадает.

```
std::shared_ptr<int> a(new int(42));
std::cout << a.use_count(); // 1

{
    std::shared_ptr<int> b = a; // КОПИРУЕТСЯ! не move
    std::cout << a.use_count(); // 2
    std::cout << b.use_count(); // 2
}
// здесь b уничтожается, счётчик становится 1

std::cout << a.use_count(); // 1
// в конце a уничтожится, счётчик = 0, объект delete'ится
```

В отличие от unique_ptr, **shared_ptr копируется** — это безопасно благодаря подсчёту.

19.3.1 make_shared

```
auto p = std::make_shared<Weapon>("меч", 5, 10);
```

Это C++11 — у нас работает.

make_shared ещё лучше make_unique: он делает **одну** аллокацию для объекта и для счётчика (внутри shared_ptr есть «control block»). Если выделять через shared_ptr<T>(new T(...)) — будут две аллокации (объект и control block отдельно). Маленькая, но реальная экономия.

19.3.2 Цена shared_ptr

shared_ptr не бесплатный:

1. **Размер:** 16 байт (два указателя — на объект и на control block), вместо 8 у unique_ptr.
2. **Счётчик:** каждое копирование увеличивает счётчик (атомарная операция!), каждое уничтожение уменьшает. Это медленнее простого копирования.
3. **Косвенный доступ:** чтобы добраться до объекта, нужно идти через control block.

Поэтому правило: **используйте unique_ptr по умолчанию. shared_ptr только когда совладение реально нужно.**

В моих проектах — shared_ptr нужен раза в десять реже, чем unique_ptr.

19.3.3 Циклы — главная ловушка shared_ptr

Допустим, у нас два класса, ссылающиеся друг на друга:

```
struct Node {
    std::shared_ptr<Node> next;
    ~Node() { std::cout << "destroyed\n"; }
};

int main() {
    auto a = std::make_shared<Node>();
    auto b = std::make_shared<Node>();
    a->next = b;
    b->next = a;    // цикл!
}
```

Что произойдёт в конце main?

1. b уничтожается, счётчик на b-объекте становится 1 (он держится из a->next).
2. a уничтожается, счётчик на a-объекте становится 1 (держится из b->next).

3. Никого не уничтожаем — счётчики не дошли до нуля.

Утечка. Деструкторы не вызовутся, "destroyed" не выведется.

Это серьёзная проблема. shared_ptr не умеет автоматически разрешать циклы (для этого нужен сборщик мусора). Решение — weak_ptr.

19.4 weak_ptr — наблюдатель без владения

std::weak_ptr<T> — указатель на объект, который не **держит** его. То есть не влияет на счётчик. Но может проверить, жив ли объект ещё.

```
std::shared_ptr<int> s = std::make_shared<int>(42);
std::weak_ptr<int> w = s;

std::cout << s.use_count();    // 1 (weak не считается)

if (auto locked = w.lock()) {   // попытка временно получить shared_ptr
    std::cout << *locked << "\n";    // 42
} else {
    std::cout << "объект уничтожен\n";
}

s.reset();    // уничтожаем

if (auto locked = w.lock()) {
    std::cout << *locked << "\n";
} else {
    std::cout << "объект уничтожен\n";    // эта ветка
}
```

w.lock() пытается превратить weak_ptr в shared_ptr. Если объект ещё жив — возвращает рабочий shared_ptr. Если уничтожен — возвращает пустой shared_ptr.

Это позволяет разорвать цикл:

```
struct Node {
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> back;    // обратная ссылка слабая
};

auto a = std::make_shared<Node>();
```

```

auto b = std::make_shared<Node>();
a->next = b;
b->back = a;    // weak — НЕ увеличивает счётчик a

// В конце оба уничтожаются нормально:
// b уничтожается → счётчик a остаётся 1 (только переменная a)
// a уничтожается → счётчик a = 0 → delete → a->next пустеет → b уже мёртв

```

В нашей игре weak_ptr нам пригодится для «NPC помнит, кто ему дал квест» — без weak это создавало бы цикл.

19.5 Когда что

Сводная таблица:

Указатель	Когда брать	Стоимость
T* (raw)	Только для «не владею, просто смотрю» — например, параметр функции.	бесплатно
unique_ptr<T>	Единственный владелец динамического объекта.	8 байт
shared_ptr<T>	Несколько владельцев.	16 байт + control block + атомарный счётчик
weak_ptr<T>	Наблюдатель в графе объектов с shared_ptr.	16 байт

По умолчанию — `unique_ptr`. Используйте `shared_ptr` только когда реально несколько владельцев. Используйте `weak_ptr` против циклов. Используйте сырой указатель когда не владеете.

19.6 Inventory — наш первый shared_ptr... ах нет, unique_ptr

В нашей игре каждым предметом владеет ровно **один** держатель: либо локация (предмет на земле), либо инвентарь игрока. При подъёме — передача владения. При выбрасывании — обратная.

Это **классический случай** для `unique_ptr<Item>`.

include/inventory.h:

```

class Inventory {
public:
    Inventory() = default;

    void add(std::unique_ptr<Item> item);
    std::unique_ptr<Item> take(const std::string& name);
    const Item* find(const std::string& name) const;
    bool empty() const;
    const std::vector<std::unique_ptr<Item>>& items() const;
    int total_weight() const;

private:
    std::vector<std::unique_ptr<Item>> items_;
};

```

Поле items_ — vector из unique_ptr<Item>. Каждый предмет хранится через указатель на базу (Item* под капотом), что даёт **полиморфизм**: в одном векторе лежат Weapon, Armor, Consumable. При уничтожении вектора каждый unique_ptr сделает delete на свой указатель, виртуальный деструктор позаботится о правильной очистке.

19.6.1 add — принимаем владение

```

void Inventory::add(std::unique_ptr<Item> item) {
    if (item) {
        items_.push_back(std::move(item));
    }
}

```

Параметр std::unique_ptr<Item> item — **по значению**. Чтобы передать — нужно std::move(...) снаружи:

```

inv.add(rpg::make_unique<Weapon>("меч", 5, 10)); // временный, move автоматический

```

Внутри add мы перемещаем item в вектор (push_back(std::move(item))). После этого локальный item пустеет.

19.6.2 take — отдаём владение

```

std::unique_ptr<Item> Inventory::take(const std::string& name) {
    auto it = std::find_if(items_.begin(), items_.end(),

```

```

        [&name](const std::unique_ptr<Item>& p) {
            return p && p->name() == name;
        });
    if (it == items_.end()) {
        return nullptr;
    }
    std::unique_ptr<Item> result = std::move(*it);
    items_.erase(it);
    return result;
}

```

Находим предмет по имени через `find_if` с лямбдой. Лямбда: «возьми указатель из вектора, проверь, что он не пустой и что имя предмета совпадает».

Если нашли — `std::move(*it)` забирает предмет из вектора (теперь там пустой `unique_ptr`), а `erase` удаляет эту пустышку. Возвращаем найденный.

Вызывающий получает `unique_ptr<Item>` — владение перешло к нему.

19.6.3 find — наблюдение без владения

```

const Item* Inventory::find(const std::string& name) const {
    auto it = std::find_if(items_.begin(), items_.end(),
        [&name](const std::unique_ptr<Item>& p) {
            return p && p->name() == name;
        });
    if (it == items_.end()) return nullptr;
    return it->get();
}

```

Возвращает **сырой** `const Item*`. Это «я не владею, я просто хочу посмотреть». Точно так же, как ссылка, но может быть `nullptr`.

Главное: **не вызывайте delete на этом указателе**. Владение остаётся у инвентаря.

`it->get()` — метод `unique_ptr`, возвращает сырой указатель без передачи владения.

19.6.4 Лямбда с захватом по ссылке

В лямбдах `find_if`:

```
[&name](const std::unique_ptr<Item>& p) { ... }
```

[&name] — захват по ссылке. Лямбда видит снаружи переменную name через ссылку. Это безопасно: лямбда живёт ровно столько же, сколько find_if (несколько миллисекунд), name не уничтожится за это время.

Если бы захват был по значению ([name]), копировалась бы строка. Для коротких имён ничего страшного, но привычка & для read-only — лучше.

19.7 Применение в Location и Player

Локация теперь хранит Inventory:

```
class Location {
public:
    // ...
    Inventory& items();
    const Inventory& items() const;
private:
    // ...
    Inventory items_;
};
```

Игрок тоже:

```
class Player {
public:
    // ...
    Inventory& inventory();
    const Inventory& inventory() const;
private:
    // ...
    Inventory inventory_;
};
```

И заполняем локации в make_demo_world:

```
glade.items().add(rpg::make_unique<Weapon>("ржавый меч", 5, 8));
village.items().add(rpg::make_unique<Consumable>("зелье лечения", 1, 15));
forest.items().add(rpg::make_unique<Armor>("кожаный нагрудник", 8, 4));
```

Каждая локация теперь имеет один предмет.

19.8 Команды take / drop / use

В Game добавились методы:

```
void Game::cmd_take(const std::string& item_name) {
    Location* loc = world_.find(current_location_id_);
    if (!loc) return;

    std::unique_ptr<Item> picked = loc->items().take(item_name);
    if (!picked) {
        std::cout << "Здесь нет '" << item_name << "'.\n";
        return;
    }
    std::cout << "Вы подняли: " << picked->name() << "\n";
    player_.inventory().add(std::move(picked));
}
```

Логика: вынимаем предмет из инвентаря локации (получаем unique_ptr), кладём в инвентарь игрока. **Между двумя строками владение временно у локальной переменной picked** — без проблем, потому что в конце мы её отдаём через std::move.

Заметили: World::find теперь возвращает Location* (без const) — потому что мы модифицируем содержимое локации. Добавили перегрузку:

```
Location* World::find(const std::string& id);           // для модификации
const Location* World::find(const std::string& id) const; // для чтения
```

cmd_drop — наоборот:

```
void Game::cmd_drop(const std::string& item_name) {
    std::unique_ptr<Item> dropped = player_.inventory().take(item_name);
    if (!dropped) {
        std::cout << "У вас нет '" << item_name << "'.\n";
        return;
    }
    Location* loc = world_.find(current_location_id_);
    // ...
    loc->items().add(std::move(dropped));
}
```

cmd_use — пример полиморфизма через dynamic_cast:

```

void Game::cmd_use(const std::string& item_name) {
    const Item* item = player_.inventory().find(item_name);
    if (!item) {
        std::cout << "У вас нет '" << item_name << "'.\n";
        return;
    }

    if (const Consumable* c = dynamic_cast<const Consumable*>(item)) {
        int heal = c->heal_amount();
        std::cout << "Вы выпили " << c->name() << " (+ " << heal << " HP).\n";
        player_.heal(heal);
        player_.inventory().take(item_name);    // удаляет
        cmd_status();
    } else {
        std::cout << "Это нельзя «использовать».\n";
    }
}

```

find даёт сырой const Item* — наблюдение. dynamic_cast<const Consumable*> проверяет, расходник ли. Если да — лечим и удаляем предмет. Если нет (оружие, броня) — отказ.

19.9 Запуск

```

$ make && echo -e "look\ntake ржавый меч\ninventory\nquit" | ./build/rpg
== Лесная поляна ==
...
На земле:
- ржавый меч (оружие, урон 8, вес 5)
> Вы подняли: ржавый меч
> Инвентарь (общий вес 5):
- ржавый меч (оружие, урон 8, вес 5)
> До свидания.

```

Игра умеет: - Брать предметы с земли (take). - Носить их в инвентаре (inventory). - Бросать обратно (drop). - Использовать расходники (use).

И всё это **без единого ручного delete**. Каждый предмет с момента создания через make_unique и до конца программы безопасно владеется одним unique_ptr. Передачи владения через std::move явны.

19.10 Сырой указатель — когда уместен

Может показаться: unique_ptr так хорош, что сырые указатели вообще не нужны. Это не так.

Сырой T* уместен, когда вы не владеете объектом. Например, как параметр функции:

```
void print_item(const Item* it) {    // не владею, просто смотрю
    if (it) std::cout << it->describe();
}
```

Или возврат «может не найтись» (как find):

```
const Item* find(const std::string& name) const;
```

Сырой указатель == **non-owning view**. Это его смысл в современном C++.

Если же функция **выделяет** объект и возвращает — берите unique_ptr. Если **принимает владение** — принимайте unique_ptr по значению. Без unique_ptr ваш интерфейс будет неясен: «кто освобождает? непонятно».

19.11 Тонкость: передать unique_ptr нельзя по значению, можно только через move

Часто новички пишут:

```
auto p = rpg::make_unique<Weapon>("меч", 5, 10);
inventory.add(p);    // ОШИБКА: unique_ptr нельзя копировать
```

Компилятор укажет на это. Решение:

```
inventory.add(std::move(p));    // ОК: передача владения
// после этого p пустой
```

Если же предмет создаётся «временным» — move происходит автоматически:

```
inventory.add(rpg::make_unique<Weapon>("меч", 5, 10));    // ОК без move
```

Временный объект — это rvalue, для него unique_ptr автоматически перемещается. Подробности — в главе 19.

19.12 shared_ptr — куда мы его не применили

В нашей игре shared_ptr не появился, и это правильно. Каждый предмет имеет одного владельца — unique_ptr достаточно.

Кейс, где может пригодиться `shared_ptr`: - **Шаблоны предметов**. Допустим, у нас 100 одинаковых «зелий лечения» в магазине, и игрок носит «ссылку» на шаблон. `shared_ptr<ItemTemplate>` — все 100 ссылок указывают на один шаблон. При удалении магазина шаблон жив, пока игрок носит зелье. - **NPC, на которого указывают разные квесты**. Если один и тот же NPC участвует в 5 квестах — `shared_ptr<NPC>` от каждого квеста.

В нашем коде до 25-й главы `shared_ptr` появится в одном-двух местах, не больше. `unique_ptr` — рабочая лошадка.

19.13 `weak_ptr` — куда мы его не применили

В нашей игре циклов нет (предмет не знает свой инвентарь, инвентарь знает предметы). Поэтому `weak_ptr` пока не нужен.

В большом проекте `weak_ptr` появляется в момент, когда вы заметили утечку через цикл `shared_ptr`. Тогда одно из направлений ссылок делаете `weak_ptr` — обычно «обратное» или «менее важное».

19.14 Главные правила главы

1. **Никогда не пишите `new T(...)` без `unique_ptr` или эквивалента.** Голый `new` в C++17+ — почти всегда баг.
2. **`unique_ptr<T>` по умолчанию.** Самый дешёвый и понятный.
3. **`make_unique<T>(args)` через шим в C++11, через стандарт в C++14+.**
4. **`shared_ptr<T>` только когда совладение реально нужно.** Цена — двойной размер, атомарный счётчик.
5. **`weak_ptr<T>` против циклов в `shared_ptr`-графе.**
6. **Сырой `T*` означает “не владею”.** Только для параметров и наблюдения.
7. **Передача `unique_ptr` через `std::move`, временные — автоматически.**
8. **`get()` даёт сырой указатель без передачи владения.** Используйте только когда нужно отдать в C-API.
9. **`reset()` на `unique_ptr` освобождает текущий и опционально берёт новый.**

19.15 Маленькое упражнение

1. Добавьте на одну локацию второй предмет (например, в лесу не только броня, но и палка-weapon). Проверьте, что `look` показывает оба, `take` берёт по имени.
2. Добавьте проверку «вес инвентаря не больше 30» — если превышен, `take` отказывает.

3. Команда `examine <item>` — показать описание предмета **в инвентаре**, не вынимая. Использовать `find` (тот, что возвращает `const Item*`).
4. Перепишите `Game::cmd_use` так, чтобы для оружия выдавало «Это оружие используется в бою через `wield`», для брони — аналогично. Используйте `dynamic_cast`.
5. (Сложнее) Добавьте поле `Item* equipped_weapon_` в `Player` — текущее оружие. Команда `wield <item>` берёт оружие из инвентаря и «надевает». Заметьте: `Item*` — сырой, потому что **владелец — инвентарь**. `equipped_weapon_` — лишь наблюдает.
6. (Сложнее) Сравните размеры под разные указатели:

```
std::cout << sizeof(Item*) << "\n";           // 8
std::cout << sizeof(std::unique_ptr<Item>) << "\n"; // 8
std::cout << sizeof(std::shared_ptr<Item>) << "\n"; // 16
std::cout << sizeof(std::weak_ptr<Item>) << "\n";  // 16
```

Объясните, почему `unique_ptr` остался 8.

7. Создайте циклический `shared_ptr` (две `Node`, каждая держит `shared_ptr` на другую). Запустите под ASan. Что покажет (или не покажет)?

19.16 Что дальше

Глава 19 — **move-семантика** глубоко. Что такое `rvalue`, `lvalue`, `rvalue`-ссылки. Что делает `std::move` на самом деле. Зачем нужны `move`-конструктор и `move`-присваивание. Когда компилятор сам инлайнит вызов и не делает копий (RVO/NRVO). После этой главы у вас не останется загадок вокруг производительности.

Глава 20 — шаблоны. Сделаем `EventBus<T>` для игровых событий. После — лямбды и `std::function` (21), исключения и `exception safety` (22). На главе 23 начнём парсер скриптов локаций — мир будет загружаться из текстового файла.

Глава 20

Глава 19. Move-семантика на бою

Move-семантика — одна из главных новинок C++11 и одна из самых страшно выглядящих. На самом деле идея простая: вместо копирования объекта **перемещать** его. Если объект тяжёлый — это огромная экономия. Если временный — это вообще без накладных расходов.

Мы уже несколько глав используем `std::move` и `&&` в сигнатурах, не объясняя глубоко. Эта глава — подробный разбор: что такое rvalue, что такое rvalue-ссылки, что **на самом деле** делает `std::move`, как пишут move-конструктор, что такое RVO и почему компилятор иногда вообще ничего не копирует.

После этой главы у вас в голове сложится цельная картина того, как объекты живут, копируются, перемещаются и умирают в C++.

20.1 Зачем move

Допустим, у нас функция, возвращающая большую коллекцию:

```
std::vector<int> generate(int n) {  
    std::vector<int> result(n);  
    for (int i = 0; i < n; ++i) result[i] = i * i;  
    return result;  
}  
  
int main() {  
    auto v = generate(1000000);  
}
```

Что происходит?

1. Внутри `generate` создаётся `result` — миллион `int`-ов на куче.
2. `return result;` — возврат.
3. В `main` создаётся `v` — копия?

В C++98 — **да**, это была копия. Миллион `int`-ов выделить заново и скопировать — несколько мегабайт памяти. Дорого.

В C++11 благодаря move-семантике (и RVO ещё раньше) — **нет**. Никакой копии. `v` получает «внутренности» `result` напрямую. Скорость — наносекунды.

Этот сдвиг изменил, как пишут на C++. Большие объекты можно свободно возвращать из функций, передавать в контейнеры. Раньше использовали указатели и аллокации руками, а в современном C++ — просто значения. И они быстрые.

20.2 lvalue и rvalue

Начнём с фундамента. В C++ есть две категории выражений (упрощённо):

lvalue — то, что имеет адрес в памяти, на что можно «указать». Имеет имя и/или живёт дольше одного выражения.

```
int x = 5;    // x — lvalue, у неё есть адрес &x
int& r = x;   // r — тоже lvalue (другое имя для x)
x = 10;       // ОК: можно присвоить
```

rvalue — временное значение. Живёт ровно одно выражение. Адреса нет (формально).

```
5           // rvalue (литерал)
x + 1       // rvalue (результат вычисления)
foo()       // rvalue (если foo возвращает по значению)
std::move(x) // rvalue (об этом ниже)
```

Можно попробовать:

```
&5;         // ОШИБКА: cannot take the address of an rvalue
&(x + 1);   // ОШИБКА: то же
&x;         // ОК: x это lvalue
```

«L» от «left side of assignment»: `x = 5;` — слева lvalue, справа rvalue. «R» — соответственно «right».

Это упрощённая модель. В современном C++ категорий выражений больше (lvalue, xvalue, prvalue, glvalue), но для нашего обсуждения хватит lvalue/rvalue.

20.3 Главное различие на практике

С rvalue можно делать вещи, которых нельзя с lvalue: - «Ограбить» — забрать у него внутренности. Раз он временный — никто не заметит.

С lvalue вообще-то тоже можно (объявлено как `std::move`), но это **программистское решение**: «я знаю, что эту переменную больше использовать не буду, можно её опустошить».

20.4 Rvalue-ссылки: T&&

Чтобы функция могла различать lvalue и rvalue, в C++11 добавили **rvalue-ссылки**:

```
void f(int& x) { std::cout << "lvalue\n"; }
void f(int&& x) { std::cout << "rvalue\n"; }

int main() {
    int a = 5;
    f(a);      // "lvalue"
    f(5);      // "rvalue"
    f(a + 1);  // "rvalue"
    f(std::move(a)); // "rvalue"
}
```

`int&` — обычная ссылка на lvalue. `int&&` — ссылка на rvalue.

Двойной амперсанд `&&` это **отдельный знак**, не два амперсанда. Логического И тут нет.

Обе ссылки — это «псевдоним переменной». Различие — в том, **какую категорию выражения** они принимают.

20.4.1 Что внутри rvalue-ссылки

Парадокс: переменная-rvalue-ссылка сама является **lvalue**:

```
void f(int&& x) {
    // x — это rvalue-ссылка, но сама x — lvalue
    int& r = x;    // OK: lvalue

    // чтобы передать дальше как rvalue:
    int&& r2 = std::move(x);
    // или
```

```
other_function(std::move(x));
}
```

Это путает. Запомните: rvalue-ссылка — это **способ принять rvalue**, но внутри функции переменная имеет имя и адрес, то есть lvalue.

20.5 Move-конструктор и move-присваивание

Раз можно перегружать функции по rvalue, можно перегрузить и **конструктор**. Это и есть move-конструктор:

```
class Buffer {
public:
    // Обычный (copy) конструктор: принимает lvalue
    Buffer(const Buffer& other);

    // Move-конструктор: принимает rvalue
    Buffer(Buffer&& other) noexcept;
};
```

Зачем? Сору-конструктор делает **глубокую копию** — выделяет новую память, копирует байты. Move-конструктор просто **забирает** указатель из источника:

```
class Buffer {
public:
    // сору: новая память + копирование
    Buffer(const Buffer& other)
        : data_(new int[other.size_]),
          size_(other.size_) {
        for (std::size_t i = 0; i < size_; ++i) {
            data_[i] = other.data_[i];
        }
    }

    // move: забираем чужие внутренности
    Buffer(Buffer&& other) noexcept
        : data_(other.data_),
          size_(other.size_) {
        other.data_ = nullptr;    // обнуляем источник
        other.size_ = 0;
    }
};
```

```

    }

    ~Buffer() {
        delete[] data_; // если data_ == nullptr, delete[] просто ничего не делает
    }

private:
    int* data_;
    std::size_t size_;
};

```

Сравним стоимость: - Сору: `new int[N] + memcpy(N*sizeof(int))`. На миллионе элементов — миллисекунды. - Move: четыре присваивания указателей и чисел. **Наносекунды.**

В сто-тысячу раз быстрее. Особенно для строк и контейнеров.

20.5.1 поехсепт на move-конструкторе

Заметили поехсепт в сигнатуре move-конструктора? Это обещание «не бросает исключений». Важно!

Если move-конструктор может бросить — `std::vector` при перевыделении (когда `push_back` исчерпал `capacity`) **откажется** его использовать и упадёт на сору. Стандарт говорит: контейнерам нужна **сильная exception guarantee**. Они полагаются на поехсепт move.

Правило: помечайте move-конструктор/присваивание поехсепт, если не делаете ничего, что может выкинуть. Обычно так и есть — простые присваивания указателей и чисел.

20.5.2 Move-присваивание

Тот же принцип для оператора `=`:

```

class Buffer {
public:
    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            delete[] data_;
            data_ = new int[other.size_];
            size_ = other.size_;
            for (std::size_t i = 0; i < size_; ++i) {
                data_[i] = other.data_[i];
            }
        }
    }
};

```



```

    }
    return *this;
}

Buffer& operator=(Buffer&& other) noexcept {
    if (this != &other) {
        delete[] data_;           // освобождаем своё
        data_ = other.data_;      // забираем чужое
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    return *this;
}
};

```

Move-присваивание сначала освобождает свои ресурсы, потом забирает чужие. И обнуляет источник.

`if (this != &other)` — проверка от **self-assignment** (`a = std::move(a);`). Без неё `delete[] data_` уничтожил бы то, что мы потом пытаемся забрать у того же объекта.

20.6 std::move — что это на самом деле

Я говорил «`std::move` — это перемещение». На самом деле — **не совсем**.

`std::move(x)` **ничего не перемещает**. Это просто **приведение типа** — оно превращает `x` в `rvalue`.

Грубо:

```

template <typename T>
typename std::remove_reference<T>::type&& move(T&& x) noexcept {
    return static_cast<typename std::remove_reference<T>::type&&>(x);
}

```

Это шаблон, который принимает что угодно и кастует к `rvalue`-ссылке. Никакого «перемещения» в этой функции нет. Имя `move` — обманчивое, лучше бы было `rvalue_cast`.

Перемещение делает потом — `move`-конструктор или `move`-присваивание, к которым приводит эта `rvalue`.

```
Buffer a(100);
Buffer b = std::move(a);
// std::move(a) превратило a в rvalue
// потом вызывается Buffer(Buffer&&) — move-конструктор
// внутри он опустошает a
```

Сам по себе `std::move` не делает ничего. Без move-конструктора (если вы его не определили) `std::move` бесполезен — компилятор откатится на `copy`.

20.7 Когда move бесплатен

Move-конструктор имеет смысл, когда у класса есть **дорогой ресурс**: динамическая память, файл, сокет, мьютекс.

Для классов с одними тривиальными полями (`int`, `double`, `bool`) `move` = `copy`. Никакой разницы.

Стандартные классы, у которых полезный `move`:

- `std::string` — для длинных строк.
- `std::vector` — внутренний массив переходит к новому владельцу.
- `std::unique_ptr` — указатель переходит (мы уже это делали).
- `std::shared_ptr` — указатель + контрол блок (но без атомарного увеличения счётчика).
- `std::map`, `std::unordered_map`, `std::set`, `std::list`, и так далее.

Для собственных классов с динамической памятью — пишите `move`.

20.8 Rule of five детально

В главе 11 и 16 я упоминал `rule of five`. Теперь полная картина.

Если ваш класс **управляет ресурсом** (нетривиальный деструктор), вам нужны **пять** специальных функций:

1. **Деструктор** — освобождает ресурс.
2. **Сору-конструктор** — глубокая копия (или запрет).
3. **Сору-присваивание** — глубокое копирование с предварительным освобождением.
4. **Move-конструктор** — забор ресурса с обнулением источника.
5. **Move-присваивание** — освобождение своего + забор + обнуление.

Это **rule of five**. C++11 разделил его на «обязательную часть» (1-3) и «опциональный плюс» (4-5).

Если все пять не нужны, явно их пишите как `= delete` или `= default`:

```
class Unique {
public:
    Unique() = default;
    ~Unique() = default;

    Unique(const Unique&) = delete;
    Unique& operator=(const Unique&) = delete;

    Unique(Unique&&) noexcept = default;
    Unique& operator=(Unique&&) noexcept = default;
};
```

«Я не копирую, перемещаю по умолчанию».

20.8.1 Когда компилятор сам генерирует move

По умолчанию (без `= default`) move-конструктор и move-присваивание **генерируются автоматически**, но только если:

- Вы не объявили **никакой** из специальных функций (деструктора, сору-конструктора, сору-присваивания).
- Все поля — движимые.

Если объявили деструктор — move **не** генерируется. Если объявили сору — тоже.

Это «правило zero»: для класса с только стандартными полями (`std::string`, `std::vector`, ...) — ничего не пишете, всё генерируется правильно.

20.9 RVO и NRVO

В коде:

```
Buffer make_buffer() {
    Buffer b(100);
    return b;
}

int main() {
    Buffer x = make_buffer();
}
```

Сколько раз вызывается конструктор?

В C++11 — **один** раз, благодаря RVO/NRVO.

RVO (return value optimization): компилятор замечает, что возвращаемый объект — это конкретный возвращаемый объект. Вместо «создать в `make_buffer`, потом скопировать в `main`» — он строит **x прямо в стеке `main`**. Никакого `make_buffer::b` отдельно нет. Всё в одном месте.

NRVO (named RVO) — то же, но для именованных переменных (`Buffer b` в нашем коде). Чуть сложнее реализовать; компилятор делает это эвристически.

С C++17 **RVO стало обязательным** — компилятор гарантированно не делает копию для `rvalue`. Для NRVO — пока на усмотрение компилятора (обычно делает).

Это значит: вы пишете `return some_local;` для большого объекта **без страха**. Компилятор сам оптимизирует.

20.9.1 Когда RVO не работает

NRVO ломается в нескольких случаях:

```
Buffer make_buffer(bool flag) {  
    Buffer a(100);  
    Buffer b(200);  
    return flag ? a : b;    // !! NRVO не работает  
}
```

Компилятор не знает заранее, какой возвращать. Не может построить «прямо в месте назначения». Тут будет `move` (если есть) или `copy`.

Ещё:

```
Buffer make_buffer() {  
    Buffer b(100);  
    return std::move(b);    // !!! НЕ ДЕЛАЙТЕ ТАК  
}
```

Многие новички думают «помогу компилятору с `move`». На самом деле — **испортите**. С `std::move` компилятор не может применить NRVO; вместо нуля копий получится `move`. `Move` дешевле, чем `copy`, но дороже, чем ничего.

Правило: не пишите `std::move` в `return` на локальную переменную. Компилятор сам разберётся.

20.10 Perfect forwarding и std::forward

Мы уже использовали std::forward в шиме make_unique:

```
template <typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

Что тут происходит?

Forwarding reference (universal reference) — T&& в шаблоне. Особенный зверь: он принимает и lvalue, и rvalue. Компилятор «свёртывает» ссылки:

- Вызвали make_unique<Weapon>(arg) где arg — lvalue (std::string& name):
 - T = std::string&, T&& = std::string& && = std::string& (свёртывание).
 - Параметр args принимает по lvalue-ссылке.
- Вызвали make_unique<Weapon>(std::move(arg)):
 - T = std::string, T&& = std::string&&.
 - Параметр принимает по rvalue-ссылке.

Внутри функции args — это переменная (всегда lvalue), и нужно **передать дальше с сохранением категории**. Для этого — std::forward<T>(args). Это похоже на std::move, но условный: - Если T — std::string& — вернёт lvalue-ссылку. - Если T — std::string — вернёт rvalue-ссылку.

Это и есть **perfect forwarding** — переслать аргументы дальше «как есть», без потери rvalue-ности или lvalue-ности.

Используется в шаблонах, особенно в фабричных функциях. В обычном коде редко нужен.

20.11 std::move vs std::forward

- **std::move** — безусловное приведение к rvalue. Используйте, когда **знаете**, что переменная — lvalue, и хотите её переместить.
- **std::forward<T>** — условное. Используется в шаблонах с forwarding reference. Сохраняет «кате́горию» аргумента.

Если в обычном коде хотите переместить — std::move. std::forward оставьте для шаблонов.

20.12 Что после move

Состояние объекта **после** перемещения: - Валидное, но **неопределённое**. Можно вызывать любые методы, которые не требуют конкретного содержимого. - Деструктор сработает корректно. - Присвоить новое значение — можно. - Использовать содержимое — нельзя (точнее, оно есть, но какое — неопределено).

Для стандартной библиотеки конкретно: - `std::string` после `move` — пустая или содержит что попало; вызвать `.size()` можно, но какое значение — неизвестно. - `std::vector` после `move` — обычно пустой. - `std::unique_ptr` после `move` — точно `nullptr`.

Иногда говорят «moved-from object is in valid but unspecified state». Запомните: после `std::move(x)` использовать `x` нельзя, только присвоить новое значение или уничтожить.

```
std::string s = "hello";
std::string t = std::move(s);

// s сейчас в moved-from состоянии
std::cout << s;           // OK, но что внутри — неизвестно
s.size();                 // OK, но размер непредсказуем
s = "new";                // OK, можно присвоить новое
```

20.13 Демонстрация

Соберём пример. Класс `Buffer` владеет массивом, печатает все вызовы:

```
class Buffer {
public:
    explicit Buffer(std::size_t n, const std::string& tag)
        : data_(new int[n]), size_(n), tag_(tag) {
        std::cout << "[" << tag_ << "]" ctor\n";
    }

    ~Buffer() {
        std::cout << "[" << tag_ << "]" dtor\n";
        delete[] data_;
    }

    Buffer(const Buffer& other)
        : data_(new int[other.size_]),
          size_(other.size_),
```

```

        tag_(other.tag_ + "_copy") {
            std::cout << "[" << tag_ << "]" copy ctor\n";
            for (std::size_t i = 0; i < size_; ++i) data_[i] = other.data_[i];
        }

Buffer(Buffer&& other) noexcept
    : data_(other.data_),
      size_(other.size_),
      tag_(std::move(other.tag_) + "_moved") {
    std::cout << "[" << tag_ << "]" move ctor\n";
    other.data_ = nullptr;
    other.size_ = 0;
}

private:
    int* data_;
    std::size_t size_;
    std::string tag_;
};

int main() {
    std::cout << "--- 1. Копирование ---\n";
    Buffer a(5, "a");
    Buffer b = a;    // copy

    std::cout << "\n--- 2. Перемещение ---\n";
    Buffer c = std::move(a);    // move

    std::cout << "\n--- 3. vector с резервом ---\n";
    std::vector<Buffer> v;
    v.reserve(2);
    v.push_back(Buffer(10, "x"));    // временный → move
    v.push_back(Buffer(20, "y"));
}

```

Вывод:

```

--- 1. Копирование ---
[a] ctor (size=5)

```

```
[a_copy] copy ctor from a
```

```
--- 2. Перемещение ---
```

```
[a_moved] move ctor
```

```
--- 3. Push в vector с резервом ---
```

```
[x] ctor (size=10)
```

```
[x_moved] move ctor
```

```
[] dtor
```

```
[y] ctor (size=20)
```

```
[y_moved] move ctor
```

```
[] dtor
```

```
[y_moved] dtor
```

```
[x_moved] dtor
```

```
[a_moved] dtor
```

```
[a_copy] dtor
```

```
[] dtor
```

Видно: - Копирование `Buffer b = a` вызывает `copy ctor` — `[a_copy] copy ctor from a`. - `Buffer c = std::move(a)` вызывает `move ctor` — `[a_moved] move ctor`. - `v.push_back(Buffer(10, "x"))` создаёт временный `x`, потом `vector` использует `move ctor` (потому что он `noexcept!`) — `[x_moved] move ctor`. После — деструктор пустого источника — `[] dtor` (тег пуст, потому что `tag_` был перемещён).

Этот тест входит в `demo-rpg/tests/test_move.cpp` — соберите и убедитесь сами.

20.14 Где move применяется в нашем коде

В `demo-rpg/` мы уже несколько раз использовали `std::move`:

1. В конструкторах для передачи параметров без копирования:

```
Player::Player(std::string name, int max_hp)
    : name_(std::move(name)), // забираем у параметра
    ...
```

`name` — параметр по значению. Внутри функции — `lvalue`. `std::move(name)` превращает в `rvalue`, что позволяет `name_` забрать байты строки.

Альтернатива — принимать параметр по `const &`:


```
Player::Player(const std::string& name, int max_hp)
    : name_(name),    // копия
```

Здесь нет move, потому что параметр — const-ссылка. Какая лучше?

- **std::string name + std::move:** одна копия при передаче в функцию (при Player("Alice", ...) — для строкового литерала; для Player(s, ...) с lvalue — копия s), один move.
- **const std::string& name + copy:** одна копия в любом случае (внутри).

Для коротких строк (SSO) разницы нет. Для длинных — оба варианта примерно одинаковы. Но **std::string name + move** позволяет вызывающему передать `std::move(s)`, и тогда копий не будет вообще. Так чуть гибче.

2. В World::add:

```
void World::add(Location location) {
    std::string id = location.id();
    locations_.emplace(std::move(id), std::move(location));
}
```

location принят по значению — он наш. `std::move(location)` отдаёт его в `emplace` без копии.

3. В Inventory::add:

```
void Inventory::add(std::unique_ptr<Item> item) {
    if (item) {
        items_.push_back(std::move(item));
    }
}
```

`unique_ptr` **нельзя** копировать. Только перемещать. Без `std::move` была бы ошибка компиляции.

20.15 Главные правила главы

1. **Rule of zero** — лучший выбор. Стандартные поля → ничего не пишете, всё генерируется правильно.
2. **Rule of five** — если управляете ресурсами руками. Реализуйте все пять или явно = delete/= default.
3. **поexcept на move** — обязательно, иначе контейнеры откатятся на сору.
4. **std::move** — это приведение типа, не само перемещение. Перемещает move-конструктор.
5. **Не пишите std::move в return** — мешает RVO.
6. **После std::move(x)** объект x в valid-but-unspecified состоянии. Не используйте.

7. `std::forward` — для шаблонов, обычный код обходится `std::move`.
8. Принимать параметр по значению + `std::move` — гибче, чем `const T&`. Особенно для конструкторов.

20.16 Маленькое упражнение

1. Скомпилируйте `test_move.cpp`. Сопоставьте вывод с тем, что я показал.
2. Закомментируйте `noexcept` в `move`-конструкторе `Buffer`. Запустите снова. В `push_back` изменится: теперь `vector` сделает `copy` вместо `move` (потому что не доверяет «бросающему» `move`). Увидите ли вы `copy ctor` в логе?
3. Уберите `reserve(2)` в `vector`. Что произойдёт при втором `push_back`? Пересмотр (`capacity` → 2, опять `move` старых элементов).
4. Замените `Buffer c = std::move(a)` на просто `Buffer c = a`. Будет `copy`. Сравните логи.
5. Добавьте операторы `= default` для `move`-конструктора (`Buffer(Buffer&&) noexcept = default;`). Что сгенерирует компилятор? Будет ли это правильным? (Подсказка: данные-член `data_` — сырой указатель, копирование которого не делает `move`. Дефолтный `move` просто скопирует адрес — но не обнулит источник. Это значит, что и `a`, и `c` имеют один указатель, и оба будут пытаться `delete[]`. Сломается.)
6. (Сложнее) Сделайте функцию `Buffer make_big_buffer(int n)`. Внутри один путь возврата — RVO сработает (вы увидите только `ctor` и `dtor`, без `move/copy`). Добавьте `if (n < 0) return Buffer(0, "small"); else return Buffer(n, "big");` — RVO может не сработать. Проверьте.
7. (Сложнее) Перепишите `Buffer c std::unique_ptr<int[]>` вместо сырого `int*`. Большая часть `rule of five` станет тривиальной — `unique_ptr` сам перемещается. Можно ли использовать `= default` для всех специальных функций?

20.17 Что дальше

Глава 20 — шаблоны (`templates`). Самая мощная и страшная часть C++. Сделаем `EventBus<T>` для шины игровых событий — типобезопасный механизм рассылки сообщений между подсистемами. Заодно узнаем `typename` vs `class`, частичную специализацию, `variadic templates`, и почему ошибки шаблонов так нечитаемы.

Часть II приближается к середине. После шаблонов (20), лямбд (21), исключений (22) дойдём до главы 23 — парсера скриптов локаций. Мир из текстового файла, не зашитый в C++. Это даст наглядное «программа делает то, что хочет пользователь».

Глава 21

Глава 20. Шаблоны — EventBus

В этой главе — **шаблоны** (templates), одна из самых мощных и одновременно самых страшных частей C++. Шаблоны позволяют писать код, работающий с **любым типом**, без потери производительности. STL без них не было бы — `std::vector<int>` и `std::vector<std::string>` это два разных типа, но кода в стандартной библиотеке для них один.

Мы построим `EventBus<T>` — шаблонную шину событий. Игровые подсистемы смогут публиковать события (`ItemPickedEvent`, `PlayerMovedEvent`), а подписчики получать колбэки. Это паттерн **observer**, реализованный обобщённо.

Главное обещание главы: после неё вы будете понимать большую часть кода стандартной библиотеки, плюс уметь писать свои простые шаблоны. Это шаг от «пользователя C++» к «продвинутому пользователю».

21.1 Зачем шаблоны

Вернёмся к функции `max`:

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

Работает для `int`. А для `double`? Нужна другая функция. Для `std::string`? Третья. Для какого-нибудь `Player`-а с оператором `<`? Четвёртая.

В Python с этим проще: `def max(a, b): return a if a > b else b`. Один код для всего.

В C++ можно перегрузкой:

```
int max(int a, int b) { return a > b ? a : b; }
double max(double a, double b) { return a > b ? a : b; }
std::string max(const std::string& a, const std::string& b) { return a > b ? a : b; }
```

Три функции, идентичный код. Если потребуется четвёртый тип — четвёртая функция. Это копи-паст.

Шаблон — это «функция-фабрика». Вы пишете один раз, компилятор сам создаёт версии для нужных типов:

```
template <typename T>
T max(T a, T b) {
    return a > b ? a : b;
}
```

template <typename T> объявляет шаблон. T — параметр-тип. Может быть int, double, чем угодно.

Использование:

```
int a = max(3, 7); // T выводится как int
double b = max(3.14, 2.71); // T выводится как double
std::string c = max(std::string("apple"), std::string("banana")); // T = std::string

// Можно явно указать:
int d = max<int>(3, 7);
```

Компилятор для каждого использования создаёт реальную функцию: max<int>, max<double>, и так далее. Это называется **инстанциация** шаблона.

После компиляции в бинарнике лежат именно сгенерированные конкретные функции. Сам шаблон — это «рецепт», не существующий в финальном коде.

21.2 typename vs class

В template <typename T> слово typename означает «следующий идентификатор — это тип-параметр». Можно также писать class:

```
template <typename T>
T max(T a, T b);

template <class T>
T max(T a, T b);
```

— оба варианта **полностью эквивалентны**. Различий нет.

Исторически: ключевое слово `class` для шаблонов появилось первым, и до C++ не было отдельного слова для «параметра-типа». Потом ввели `typename` как более понятное. Сейчас обычно пишут **`typename`** — оно лучше отражает смысл. Но в чужом коде увидите оба.

В шаблоне `T` может быть не только пользовательским типом, но и встроенным (`int`, `double`), даже указателем (`int*`). Главное — у него должны быть операции, которые шаблон использует. У нас `max` использует `operator>` — значит, у `T` должна быть эта операция. Иначе компилятор выдаст ошибку при инстанцииции.

21.3 Где живут шаблоны: только в заголовках

В главе 7 я говорил: объявления в `.h`, определения в `.cpp`. Для шаблонов это **не работает**.

Шаблоны должны быть **полностью видны** в момент инстанцииции. То есть тело шаблона должно быть в заголовочном файле. Если положить `template T max(T a, T b) { return ... }` в `.cpp` — другой файл, использующий `max<int>`, не увидит реализацию и выдаст ошибку линковки.

Поэтому шаблоны живут **полностью в .h-файле**:

```
// math_utils.h
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

template <typename T>
T max(T a, T b) {
    return a > b ? a : b;
}

#endif
```

Никакого `math_utils.cpp`. Реализация в заголовке.

Это имеет последствия:

- **Раздувание кода.** Каждый `.cpp`, использующий шаблон, перекомпилирует его тело. Это медленно.
- **Утечка реализации.** Все детали шаблона видны пользователю.
- **Реальная инстанцииция в нескольких translation units.** Линкер потом убирает дубликаты (через `inline`, шаблоны автоматически).

Альтернатива — **explicit instantiation**: явно говорим, какие версии шаблона нужны, генерируем их в одном .cpp. Сложновато; в этой книге не пользуемся.

21.4 Шаблоны классов

То же с классами:

```
template <typename T>
class Vec {
public:
    Vec() : data_(nullptr), size_(0), cap_(0) {}

    void push_back(const T& value) {
        if (size_ == cap_) grow();
        data_[size_++] = value;
    }

    T& operator[](std::size_t i) { return data_[i]; }

    std::size_t size() const { return size_; }

private:
    void grow() { /* ... */ }

    T* data_;
    std::size_t size_;
    std::size_t cap_;
};
```

Это сокращённый std::vector. Использование:

```
Vec<int> v;
v.push_back(1);
v.push_back(2);
std::cout << v[0] << "\n"; // 1
```

Vec<int> — это **тип**, сгенерированный из шаблона с T = int. У него есть свои методы, конструкторы, всё как у обычного класса.

Когда вы пишете методы шаблонного класса **вне** класса, синтаксис громоздкий:

```

template <typename T>
class Vec {
public:
    void push_back(const T& value);
};

// Реализация:
template <typename T>
void Vec<T>::push_back(const T& value) {
    // ...
}

```

Каждое определение метода требует префикса `template <typename T>` и `Vec<T>::`. Если параметров шаблона много — становится трудно читать. Многие пишут все методы прямо внутри класса (как у нашего `EventBus`), чтобы не повторять.

21.5 Дедукция типа

Когда вы вызываете `max(3, 7)`, компилятор сам выводит `T = int`. Это **template argument deduction**.

Дедукция работает по аргументам функции. Если параметр объявлен `T`, и вы передаёте `int` — `T = int`. Если параметр `const T&`, передаёте `int` — `T = int` всё равно (`const &` отбрасывается при дедукции).

Иногда дедукция не работает — например, для возвращаемого типа:

```

template <typename T>
T from_string(const std::string& s);

int i = from_string<int>("42");           // явно: T = int
int j = from_string("42");                // ОШИБКА: не из чего вывести T

```

Тогда указываем явно: `from_string<int>("42")`.

Для классов в C++11 дедукция не работала (`std::vector v{1, 2, 3}` — нужен был `std::vector<int> v{1, 2, 3}`). С C++17 добавили **CTAD** (class template argument deduction), но это не наша база.

21.6 Специализация

Иногда хочется для **одного конкретного** типа сделать особую реализацию. Это **специализация шаблона**.

Пример. У нас шаблон `print<T>`:

```
template <typename T>
void print(const T& value) {
    std::cout << value;
}
```

Для `bool` хотим выводить `"true"/"false"`, а не `1/0`:

```
template <> // явная специализация
void print<bool>(const bool& value) {
    std::cout << (value ? "true" : "false");
}
```

`template <>` (пустые скобки) — синтаксис **полной специализации**. Дальше `print<bool>` указывает, какой тип специализируем. Когда компилятор увидит `print(true)` — выберет специализацию, не общий шаблон.

Использование:

```
print(42); // "42"
print(true); // "true"
print("hello"); // "hello"
```

В стандартной библиотеке полно специализаций. Например, `std::vector<bool>` — особая реализация (упаковывает в биты), отличающаяся от `std::vector<T>` для других типов.

21.6.1 Частичная специализация

Только для классов (для функций нет — пишут перегрузки). Допустим, у нас шаблон `Wrapper<T, N>`, и для случая `N == 0` нужна особая реализация:

```
template <typename T, int N>
class Wrapper {
public:
    void show() { std::cout << "general " << N << "\n"; }
};

// Частичная специализация для N = 0
```



```
template <typename T>
class Wrapper<T, 0> {
public:
    void show() { std::cout << "specialized: zero\n"; }
};

Wrapper<int, 5> a; a.show(); // "general 5"
Wrapper<int, 0> b; b.show(); // "specialized: zero"
```

В нашей книге частичная специализация почти не появится — это глубокая тема. Просто знайте, что она есть.

21.7 Variadic templates

Шаблон может принимать **переменное число** параметров:

```
template <typename... Args>
void print_all(Args... args) {
    // ...
}
```

Args... — это «пакет параметров» (parameter pack). Принимает любое количество типов. Можно вызвать:

```
print_all(1, 2.5, "hello", true); // 4 параметра
print_all();                       // 0 параметров
```

Внутри функции пакет нужно «развернуть». Самый частый способ — рекурсия:

```
void print_all() {
    std::cout << "\n";
}

template <typename T, typename... Rest>
void print_all(T first, Rest... rest) {
    std::cout << first << " ";
    print_all(rest...); // рекурсивный вызов
}

int main() {
```

```
print_all(1, 2.5, "hello", true);
}
```

Каждый вызов «отрывает» первый элемент и идёт дальше. В конце — пустая база (terminator).

Вывод: 1 2.5 hello 1 (true печатается как 1).

В C++17 появилось **fold expression** — более короткий синтаксис, но он не наша база.

21.7.1 Variadic в нашем make_unique

Помните rpg::make_unique?

```
template <typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

Args... args — пакет аргументов любых типов. std::forward<Args>(args)... — развёртывание пакета. Каждый args пересылается через std::forward, который сохраняет lvalue/rvalue-ность.

Это **perfect forwarding** в действии. Можем создавать любые объекты с любыми параметрами конструктора:

```
auto p1 = make_unique<int>(42);
auto p2 = make_unique<std::string>(10, 'x'); // строка из 10 'x'
auto p3 = make_unique<Player>("Hero", 100);
```

И всё это **без копирования** параметров — они пересылаются как есть.

21.8 SFINAE — кратко

«Substitution Failure Is Not An Error». Один из главных механизмов шаблонной магии.

Идея: если при подстановке типа в шаблон возникает ошибка — компилятор не выдаёт error, а **молча игнорирует** этот шаблон. Это позволяет писать шаблоны, работающие только с определёнными типами.

Пример:

```
template <typename T>
typename std::enable_if<std::is_integral<T>::value, T>::type
square(T x) {
    return x * x;
```

```

}

square(5);           // OK
square(3.14);        // ОШИБКА: square отброшен, потому что double не integral

```

`std::enable_if` — шаблон-машина: если первый параметр истина, есть тип `::type`; иначе ошибка. SFINAE ловит «нет `::type`», молча отбрасывает шаблон.

Синтаксис страшный. В C++20 для этого появились **концепты** (concepts), которые гораздо чище. В C++11 — только SFINAE.

В нашей книге SFINAE не используем. Это упоминание для тех, кто будет читать чужой код.

21.9 Ошибки шаблонов

Если в шаблонном коде ошибка — компилятор выдаёт **очень длинные** сообщения. Иногда десятки строк на одну ошибку.

```

template <typename T>
T sum(const T& a, const T& b) {
    return a + b;
}

struct Foo {};

int main() {
    Foo a, b;
    sum(a, b);
}

```

Компилятор:

```

error: no match for 'operator+' (operand types are 'const Foo' and 'const Foo')
note: candidate: 'T sum(const T&, const T&) [with T = Foo]'
note: in instantiation of function template specialization 'sum<Foo>' requested here
...

```

Видны и шаблон, и место использования. На сложных шаблонах (`std::vector<std::map<int, std::vector<...>>>`) ошибка может занять страницу.

Совет: читайте сообщение **снизу вверх**. Снизу — место ошибки. Над ним — что попытался сделать компилятор. И там, где конкретно «не подошло» — оригинальная причина.

В C++20 концепты делают сообщения чище. Но C++11 — терпите.

21.10 EventBus

Соберём всё в нашем EventBus. Это шаблонный класс, который для каждого типа события держит свой список колбэков:

```
template <typename Event>
class EventBus {
public:
    using Handler = std::function<void(const Event&);>;

    void subscribe(Handler handler) {
        handlers_.push_back(std::move(handler));
    }

    void publish(const Event& event) const {
        for (const auto& h : handlers_) {
            h(event);
        }
    }

    std::size_t subscriber_count() const {
        return handlers_.size();
    }

private:
    std::vector<Handler> handlers_;
};
```

Разберём.

template <typename Event> — шаблон по типу события.

using Handler = std::function<void(const Event&);>; — псевдоним типа. Handler — это «функция, принимающая `const Event&` и не возвращающая ничего». `std::function` — тип-«стиратель» (type-erased wrapper), который умеет хранить любую callable: лямбду, функтор, указатель на функцию. Подробно — в главе 21.

using — современная альтернатива `typedef`:

```
typedef std::function<void(const Event&)> Handler;    // старо
using Handler = std::function<void(const Event&)>;    // ново, читается лучше
```

В шаблонных классах `using` особенно ценен, потому что работает с шаблонными параметрами без проблем.

subscribe добавляет обработчик в вектор. Перемещаем, чтобы избежать копии.

publish проходит по всем и вызывает. Метод `const` — публикация не меняет список подписчиков.

Простой, но работающий код. Около 25 строк.

21.10.1 События

Сами события — просто структуры:

```
// events.h
struct ItemPickedEvent {
    std::string player_name;
    std::string item_name;
};

struct PlayerMovedEvent {
    std::string player_name;
    std::string from_id;
    std::string to_id;
};
```

Никакой иерархии, ничего сложного. Каждый тип события — своя структура. `EventBus` параметризуется конкретным типом, и события только этого типа в неё попадают.

21.10.2 Тест

```
EventBus<ItemPickedEvent> pickup_bus;
EventBus<PlayerMovedEvent> move_bus;

// Подписчик-логгер
pickup_bus.subscribe([](const ItemPickedEvent& e) {
    std::cout << "[log] " << e.player_name << " поднял " << e.item_name << "\n";
});
```

```
// Второй подписчик — ачивка за первый предмет
bool first = true;
pickup_bus.subscribe([&first](const ItemPickedEvent& e) {
    if (first) {
        std::cout << "[ачивка] первый предмет!\n";
        first = false;
    }
});

move_bus.subscribe([](const PlayerMovedEvent& e) {
    std::cout << "[log] " << e.player_name << ": "
               << e.from_id << " -> " << e.to_id << "\n";
});

pickup_bus.publish({"Герой", "ржавый меч"});
pickup_bus.publish({"Герой", "зелье"});
move_bus.publish({"Герой", "glade", "village"});
```

Каждая лямбда — это объект `std::function<void(const Event&)>`. Лямбды и `std::function` — глава 21.

Вывод:

```
[log] Герой поднял ржавый меч
[ачивка] первый предмет!
[log] Герой поднял зелье
[log] Герой: glade -> village
```

Каждое `publish` вызвало всех подписчиков по очереди.

21.11 Концепции и применения

Что нам это даёт?

1. **Развязывание подсистем.** Бой не знает про логирование, но при ударе шлёт `DamageEvent`. Логгер подписан на `DamageEvent` и сам решает, что писать.
2. **Лёгкость расширения.** Новая фишка (например, «достижения за 100 убийств») — отдельный подписчик. Старый код не меняется.
3. **Тестирование.** Подписать тест-подписчика — проверить, что событие выстрелило.

Это паттерн **observer** или **pub-sub**. На больших играх — основа архитектуры. В нашей книге используется частично.

21.12 Интеграция в проект (предварительно)

Я не делаю интеграцию EventBus в Game в этой главе — добавим в финальной (25). Но архитектура такая:

```
class Game {
private:
    EventBus<ItemPickedEvent> pickup_bus_;
    EventBus<PlayerMovedEvent> move_bus_;
    // ...

    void cmd_take(const std::string& name) {
        // ... поднимаем предмет ...
        pickup_bus_.publish({player_.name(), item_name});
    }

    void cmd_go(const std::string& dir) {
        // ... меняем локацию ...
        move_bus_.publish({player_.name(), old_id, new_id});
    }
};
```

В начале программы — подписываемся:

```
Game game;
game.subscribe_pickup([](const auto& e) { ... });
```

Удобно? Очень. Но в нашей маленькой игре он не настолько критичен — оставим этот рефакторинг на потом.

21.13 Сторонние шаблонные паттерны

Несколько других мест, где шаблоны блистают.

21.13.1 Тип-безопасное хранение разнотипных данных

```

template <typename T>
class Holder {
public:
    Holder(T value) : value_(std::move(value)) {}
    T& get() { return value_; }
    const T& get() const { return value_; }
private:
    T value_;
};

Holder<int> a(42);
Holder<std::string> b("hi");

a.get() += 1;           // a содержит 43
b.get() += " world";    // b содержит "hi world"

```

21.13.2 Типовые функции

```

template <typename Container, typename Predicate>
auto count_if(const Container& c, Predicate p) -> std::size_t {
    std::size_t n = 0;
    for (const auto& x : c) {
        if (p(x)) ++n;
    }
    return n;
}

std::vector<int> v = {1, 2, 3, 4, 5};
auto evens = count_if(v, [](int x) { return x % 2 == 0; });

```

(Стандарт уже даёт `std::count_if`, но мы могли бы написать свой.)

21.13.3 Generic algorithms

Вся STL — шаблоны. `std::sort`, `std::find`, `std::accumulate` — все принимают любой контейнер и любые типы. Это даёт огромную переиспользуемость.

21.14 Шаблоны и компилятор

Под капотом шаблоны работают так:

1. **На этапе парсинга** компилятор проверяет синтаксис шаблона, но **не компилирует его в код**. Просто записывает «вот рецепт».
2. **При использовании** (`max<int>(3, 7)`) компилятор делает **инстанциацию** — берёт рецепт, подставляет `T = int`, и компилирует получившуюся конкретную функцию.
3. **Каждая инстанциация — новая функция**. `max<int>` и `max<double>` — два разных символа в объектном файле.
4. **Линкер убирает дубликаты**. Если два `.cpp`-файла оба используют `max<int>` — оба генерируют его, но линкер оставляет одну копию.

Поэтому в больших проектах с шаблонами: - **Долгая компиляция** — каждый файл компилирует все используемые шаблоны. - **Большой код** — много дубликатов до линковки. - **Сложные сообщения об ошибках** — раскрыт каждый уровень шаблона.

Современные компиляторы это знают и оптимизируют. Но факт остаётся: шаблоны не бесплатны на `compile-time`. На `run-time` же они часто **быстрее** обычных функций — компилятор знает все типы и может агрессивно инлайнить.

21.15 Главные правила главы

1. **Шаблоны для обобщённого кода**. Если есть `N` копий функции, отличающейся только типом — шаблон.
2. **Тело шаблона в .h-файле**. Реализации в `.cpp` не работают для шаблонов.
3. **typename или class** — выбирайте `typename`, оно лучше отражает смысл.
4. **Дедукция типа** работает для функций. Можно явно: `func<int>(arg)`.
5. **using лучше typedef** для псевдонимов типов.
6. **... для variadic** — переменное число параметров. Используется через рекурсию.
7. **Ошибки шаблонов** читайте снизу вверх.
8. **Шаблоны не бесплатны на compile-time** — длительная сборка на больших проектах.
9. **std::function для type-erased callable** — подробнее в следующей главе.

21.16 Маленькое упражнение

1. Напишите шаблон `min<T>(T a, T b)` по аналогии с `max`. Используйте для `int`, `double`, `std::string`.

2. Напишите шаблон `swap<T>(T& a, T& b)`. Сравните со `std::swap`.
3. Напишите шаблон `print_pair<A, B>(const std::pair<A, B>& p)`, выводящий "(first, second)".
4. Сделайте EventBus методом `clear()` — убрать всех подписчиков.
5. Добавьте к EventBus метод `subscribe_once(handler)` — обработчик удалится после первого вызова. Подсказка: можно через флаг внутри лямбды, или специальной обёртке.
6. (Сложнее) Сделайте шаблон `Stack<T>` — стек на массиве с операциями `push`, `pop`, `top`, `empty`, `size`. Используйте `std::vector<T>` внутри.
7. (Сложнее) Создайте шаблон `Function<R(Args...)>` — упрощённый аналог `std::function`. Поддерживает только хранение лямбд без захвата. Это сложно — подсказка: внутри используйте `R (*func_)(Args...)` (указатель на функцию).

21.17 Что дальше

Глава 21 — **лямбды и `std::function`**. Мы уже много раз использовали лямбды, теперь разберём детально: захваты (`[]`, `[=]`, `[&]`, `[this]`, `[var]`, `[&var]`), `mutable`, тип лямбды, `std::function` как тип-стиратель, performance trade-offs. После — исключения (22), парсер мира (23), сохранения (24), финал RPG (25).

Глава 22

Глава 21. Лямбды и `std::function`

В предыдущих главах мы уже много раз использовали лямбды, особо не вглядываясь. В `std::sort` с компаратором, в `std::count_if` с предикатом, в `Inventory::take` с предикатом поиска, в `EventBus<T>::subscribe` для обработчиков. Эта глава — детальный разбор: что такое лямбда под капотом, что значит каждый вид захвата, зачем `mutable`, что такое `std::function`, и где между ними разница.

Дополнительно построим `CommandRegistry` — реестр игровых команд, в котором имя команды отображается в обработчик-лямбду. Заменяем длинную цепочку `if/else if` в `Game::execute` на короткий `commands_.dispatch(verb, rest)`.

22.1 Лямбда — что это

Лямбда в C++ — это **анонимная функция**, объявленная прямо в коде, где она нужна.

```
auto add = [](int a, int b) { return a + b; };  
std::cout << add(3, 4);    // 7
```

`[] (int a, int b) { return a + b; }` — это и есть лямбда. Расшифровка по частям:

- `[]` — **захват** (capture). Какие переменные снаружи лямбда «видит».
- `(int a, int b)` — параметры, как у обычной функции.
- `{ return a + b; }` — тело.
- (опционально) Между `()` и `{ }` можно указать `-> int` — возвращаемый тип. Если опущен — компилятор выводит сам.

Тип лямбды компилятор генерирует автоматически. Программисту имя этого типа неизвестно. Поэтому используем `auto` или `std::function` для хранения.

```
auto plus = [](int a, int b) { return a + b; };           // auto – самое короткое
std::function<int(int, int)> plus2 = ...;                 // через std::function
```

22.2 Анатомия захвата

Самая сложная часть. Захват — что лямбда «запоминает» из окружающего кода.

```
int x = 10;
auto f = [x]() { return x + 1; };    // x захвачен по значению
std::cout << f();    // 11

x = 100;
std::cout << f();    // всё ещё 11! Лямбда хранит свою копию x
```

Возможные варианты:

22.2.1 [] — без захвата

Лямбда не видит ничего из окружения. Только параметры и глобалы.

```
auto inc = [](int x) { return x + 1; };
```

Самые «свободные» лямбды — могут конвертироваться в указатель на функцию, удобны как параметры в C-API.

22.2.2 [x] — захват по значению

Делает **копию** переменной x в момент создания лямбды. Изменения x снаружи не влияют.

```
int counter = 0;
auto f = [counter]() { return counter; };
counter = 100;
std::cout << f();    // 0 – захватили старое значение
```

Внутри лямбды захваченные по значению переменные **const** по умолчанию — изменять нельзя:

```
int n = 5;
auto f = [n]() { n = 10; };    // ОШИБКА
```

Чтобы разрешить менять — mutable:

```
auto f = [n]() mutable { n = 10; std::cout << n; };  
f();
```

Но изменение влияет только на копию внутри лямбды. Снаружи `n` не меняется.

22.2.3 `[&x]` — захват по ссылке

Лямбда «помнит» **ссылку** на `x`. Все изменения снаружи видны внутри, и наоборот.

```
int counter = 0;  
auto f = [&counter]() { counter++; };  
f();  
f();  
f();  
std::cout << counter;    // 3
```

Это часто используется для накопления состояния.

Опасность: ссылка живёт, пока лямбда живёт. Если лямбда переживёт `counter` — будет висячая ссылка → UB.

```
std::function<void()> bad() {  
    int x = 5;  
    return [&x]() { std::cout << x; };    // !!! x уничтожится, ссылка висячая  
}
```

Запоминайте: **захват по ссылке только когда лямбда живёт меньше, чем источник**. Часто это локальные функции и алгоритмы STL — там безопасно.

22.2.4 `[=]` — захват всего по значению

Все переменные, которые лямбда использует, копируются.

```
int x = 10, y = 20;  
auto f = [=]() { return x + y; };    // x и y скопированы
```

Удобно, но опасно — можно случайно «прихватить» что-то лишнее. В современных стилях `[=]` часто не рекомендуют. Лучше явно указать, что захватываете.

22.2.5 `[&]` — захват всего по ссылке

Все используемые переменные — по ссылке.

```
int x, y, z;
auto f = [&]() { x = 1; y = 2; z = 3; };
```

Удобно для писателей в окружение. Та же опасность с висячими ссылками.

22.2.6 [this] — захват указателя на текущий объект

Внутри методов класса можно захватить this:

```
class Game {
public:
    void register_commands() {
        commands_.add("look", [this](const std::string&) {
            cmd_look(); // вызов метода этого Game
        });
    }
};
```

[this] копирует **указатель** this. Через него лямбда может вызывать методы и обращаться к полям.

Опасность: если Game уничтожится, а лямбда останется — this будет висячим. У нас Game живёт всё время программы, проблем нет.

В C++17 появился [*this] — копирует **сам объект**. В C++11 нет.

22.2.7 [x, &y] — комбинированный

Можно смешивать:

```
int x, y;
auto f = [x, &y]() { ... }; // x по значению, y по ссылке
```

22.2.8 [x = ...] — захват с инициализатором (C++14)

```
auto f = [n = 10]() { return n; };
```

В C++11 нет. У нас не появится.

22.3 Что лямбда на самом деле

Лямбда — это **синтаксический сахар** для создания **функтора** (объекта-функции). Компилятор за вас генерирует класс примерно такого вида:

```
int x = 10;
auto f = [x](int y) { return x + y; };

// эквивалент:
class __lambda_42 {
    int x;
public:
    __lambda_42(int x_) : x(x_) {}
    int operator()(int y) const { return x + y; }
};

auto f = __lambda_42(x);
```

То есть лямбда — это объект некоторого анонимного класса с `operator()`. Захват — это поля класса. Параметры — параметры `operator()`. Тело — тело `operator()`.

mutable лямбда — это объект без `const` на `operator()`:

```
auto f = [n=10]() mutable { ++n; return n; };

// эквивалент:
class __lambda_42 {
    int n = 10;
public:
    int operator()() { return ++n; } // нет const
};
```

Это помогает понять любую лямбду: «это безымянный объект, вызываемый через `()`».

22.4 Производительность

Лямбды **дешёвые**. Они: - Не выделяют память на куче (объект на стеке). - Не используют виртуальные вызовы (вызов `operator()` инлайнится компилятором). - Без захватов превращаются в указатель на функцию.

В STL-алгоритмах (`std::sort`, `std::find_if`) использование лямбды vs обычной функции в скорости **не отличается**. Иногда даже быстрее — потому что компилятор видит код лямбды на месте и инлайнит.

22.5 std::function — type erasure

Лямбды все разного типа. Даже две одинаково выглядящие лямбды — два разных типа. Это создаёт проблему: как **хранить** лямбду в поле класса, если её тип не назвать?

auto работает только для **локальных** переменных. В классе как поле — нельзя.

Решение — **std::function<Signature>**. Это **type-erased** контейнер для любой callable.

```
#include <functional>

std::function<int(int, int)> op = [](int a, int b) { return a + b; };
std::cout << op(3, 4);    // 7

op = [](int a, int b) { return a * b; };
std::cout << op(3, 4);    // 12

op = std::plus<int>();    // функтор тоже ОК
std::cout << op(3, 4);    // 7

int my_func(int a, int b) { return a - b; }
op = my_func;            // указатель на функцию тоже ОК
```

std::function<int(int, int)> принимает **любой** callable с сигнатурой «принимает (int, int), возвращает int». Лямбды, функторы, указатели на функции — всё подходит.

22.5.1 Цена type erasure

std::function не бесплатный:

1. **Размер:** обычно 32-64 байта (внутри SBO — small buffer optimization, как у std::string).
2. **Аллокация для больших захватов:** если лямбда с захватом большего чем SBO размера, std::function выделит память на куче.
3. **Косвенный вызов:** внутри std::function хранится указатель на «виртуальную таблицу». Каждый вызов — это разыменование + вызов через указатель.

В критичных к производительности местах (горячий цикл, миллион вызовов) std::function чувствуется. В обычном коде (как наш EventBus или CommandRegistry) — не проблема.

22.5.2 Когда использовать что

- **Шаблонный параметр** — самый дешёвый, без type erasure. Подходит, когда тип лямбды известен на компиляции:


```
template <typename F>
void run(F f) { f(); }
```

- **std::function** — когда нужно **хранить** callable без знания его типа: поле класса, элемент в vector, возврат из функции с разными возможными лямбдами.
- **Указатель на функцию** — для C-API. Работает только без захватов.

В нашем EventBus<T> мы использовали std::function<void(const Event&)> — потому что нужно хранить разные обработчики в vector. Это типичный случай.

22.6 Лямбды как параметры функций

Два способа принять лямбду:

```
// 1. Шаблонный параметр — компилятор инлайнит, быстро
template <typename F>
void apply(F f) { f(); }

// 2. std::function — type-erased, проще читается
void apply(std::function<void()> f) { f(); }
```

STL использует первый подход — алгоритмы шаблонные.

Если функция большая и часто использует callback — std::function упрощает интерфейс. Если функция маленькая и часто вызывается — шаблон даёт лучшую производительность.

22.7 std::bind — старый стиль (для контекста)

До лямбд для «частичного применения» функции использовали std::bind:

```
#include <functional>

int sub(int a, int b) { return a - b; }

auto from_10 = std::bind(sub, 10, std::placeholders::_1);
std::cout << from_10(3); // 10 - 3 = 7
```

Сложный синтаксис, ограничения, странные placeholders. В современном C++ это **полностью** заменяется лямбдами:

```
auto from_10 = [](int x) { return sub(10, x); };
```

Лямбды короче, читаемее, эффективнее. std::bind в новом коде не пишите — но в старом увидите.

22.8 NPC и поведения — намёк на будущее

Главу можно назвать «поведения NPC через лямбды», и вот пример, к чему мы потом придём:

```
class NPC {
public:
    using Tick = std::function<void(NPC&, Game&)>;

    void on_tick(Tick t) { tick_ = std::move(t); }
    void update(Game& g) { if (tick_) tick_(*this, g); }

private:
    Tick tick_;
};

// Использование: NPC, который патрулирует туда-сюда
auto patroller = std::make_shared<NPC>();
patroller->on_tick([direction = 1](NPC& self, Game& g) mutable {
    // ходим вперёд-назад, переключая direction
    // ...
});
```

Каждый NPC имеет свой tick-обработчик. Один — патрулирует, другой — отвечает на приветствие, третий — продаёт товары. Все через лямбды с захватом своего состояния.

В нашей текущей RPG NPC ещё нет (добавятся за пределами книги, как упражнение). Но шаблон поведения вы уже понимаете.

22.9 CommandRegistry — лямбды на работе

Главная интеграция этой главы. У нас в Game::execute была длинная цепочка:

```
if (verb == "help") {
    cmd_help();
} else if (verb == "look" || verb == "l") {
```

```

    cmd_look();
} else if (verb == "go") {
    std::string dir; ss >> dir;
    if (dir.empty()) std::cout << "Куда идти?\n";
    else cmd_go(dir);
}
// ...ещё восемь веток...
else {
    std::cout << "Неизвестная команда\n";
}

```

Уродливо, длинно, и для каждой новой команды нужно добавлять `else if`. Перепишем через **словарь имя → лямбда**.

include/commands.h:

```

class CommandRegistry {
public:
    using Handler = std::function<void(const std::string&)>;

    void add(const std::string& name, Handler handler);
    void add_alias(const std::string& alias, const std::string& target);
    bool dispatch(const std::string& verb, const std::string& rest) const;
    std::vector<std::string> names() const;

private:
    std::unordered_map<std::string, Handler> handlers_;
    std::unordered_map<std::string, std::string> aliases_;
};

```

Реализация — простая:

```

bool CommandRegistry::dispatch(const std::string& verb, const std::string& rest) const {
    std::string resolved = verb;
    auto alias_it = aliases_.find(verb);
    if (alias_it != aliases_.end()) {
        resolved = alias_it->second;
    }

    auto it = handlers_.find(resolved);
    if (it == handlers_.end()) return false;
}

```

```

        it->second(rest);
        return true;
    }

```

Сначала разрешаем алиас (`l` → `look`), потом ищем обработчик, потом вызываем с аргументом.

22.9.1 Регистрация команд

В `Game::register_commands()`:

```

void Game::register_commands() {
    commands_.add("help",      [this](const std::string&) { cmd_help(); });
    commands_.add("look",      [this](const std::string&) { cmd_look(); });
    commands_.add("status",    [this](const std::string&) { cmd_status(); });
    commands_.add("inventory", [this](const std::string&) { cmd_inventory(); });
    commands_.add("go",        [this](const std::string& rest) {
        if (rest.empty()) std::cout << "Куда идти?\n";
        else cmd_go(rest);
    });
    commands_.add("take",      [this](const std::string& rest) {
        if (rest.empty()) std::cout << "Что взять?\n";
        else cmd_take(rest);
    });
    commands_.add("drop",      [this](const std::string& rest) { /* ... */ });
    commands_.add("use",       [this](const std::string& rest) { /* ... */ });
    commands_.add("quit",      [this](const std::string&) { running_ = false; });

    commands_.add_alias("l",   "look");
    commands_.add_alias("i",   "inventory");
    commands_.add_alias("inv", "inventory");
    commands_.add_alias("stat","status");
    commands_.add_alias("get", "take");
    commands_.add_alias("q",   "quit");
    commands_.add_alias("exit","quit");
}

```

Каждая команда — лямбда с захватом `this`. `[this]` копирует указатель, через него лямбда вызывает метод `cmd_xxx`. `CommandRegistry` ничего о `Game` не знает — он просто хранит обработчики.

В сигнатуре `Handler = std::function<void(const std::string&)>` — обработчик принимает «остаток строки» (всё после `verb`). Это позволяет вытащить аргумент:

```
// Ввод: "go east"
// verb = "go", rest = "east"
```

22.9.2 Парсинг

В `Game::step()`:

```
std::istringstream ss(line);
std::string verb;
ss >> verb;

std::string rest;
std::getline(ss, rest);
rest = trim_left(rest);

if (!commands_.dispatch(verb, rest)) {
    std::cout << "Неизвестная команда: '" << verb << "'.\n";
}
```

Извлекаем первое слово как `verb`, остальное как `rest` (через `getline` после `>>` забирает остаток строки), убираем ведущие пробелы. Вся логика — в `CommandRegistry`.

22.9.3 help автоматический

```
void Game::cmd_help() const {
    std::cout << "Команды:\n";
    for (const auto& name : commands_.names()) {
        std::cout << "  " << name << "\n";
    }
}
```

`commands_.names()` возвращает список всех зарегистрированных. `Help` автоматически показывает что есть. Добавьте новую команду — она появится в `help` без правок.

22.9.4 Что выиграли

- Цепочка `if/else` исчезла.
- Новая команда — одна строка `commands_.add(...)`.

- Алиасы централизованы.
- Help автогенерируется.
- Парсер изолирован от логики команд.

Это **стандартный паттерн** организации команд в REPL/CLI программах: словарь обработчиков плюс единая точка диспетчеризации. В нашей mini-shell (часть III) сделаем то же самое.

22.10 Запуск

```
$ make && echo -e "help\nl\ninv\nq" | ./build/rpg
```

Команды:

```
drop
go
help
inventory
look
quit
status
take
use
```

== Лесная поляна ==

...

Инвентарь пуст.

До свидания.

Работает: алиасы l, inv, q разрешаются в look, inventory, quit.

22.11 Главные правила главы

1. **Лямбда** — это объект анонимного класса с `operator()`. Не магия.
2. **Захват [var] копирует, [&var] ссылка**. Запоминайте опасность висячих ссылок.
3. **mutable** для изменения захваченных по значению.
4. **[this]** для лямбд в методах класса.
5. **auto для хранения лямбды**, `std::function<Signature>` для типостёртого хранения в полях/контейнерах.
6. **std::function не бесплатен** — стоимость аллокации для больших захватов и косвенного вызова.
7. **Шаблонный параметр** для callback в шаблонных функциях — быстрее, чем `std::function`.
8. **std::bind устарел** — заменяйте лямбдами.

22.12 Маленькое упражнение

1. Напишите лямбду `is_even`, принимающую `int` и возвращающую `bool`. Используйте в `std::count_if` на векторе.
2. Захватите счётчик по ссылке и сделайте лямбду, увеличивающую его на каждый вызов. Вызовите 5 раз. Проверьте, что счётчик стал 5.
3. Лямбда без захватов конвертируется в указатель на функцию:

```
void (*p)(int) = [](int x) { std::cout << x; };  
p(42);
```

Проверьте. С захватом не сработает — попробуйте.

4. Добавьте команду `where` в `CommandRegistry`, выводящую текущий ID локации.
5. Добавьте алиас `whereami` → `where`.
6. (Сложнее) Реализуйте функцию `times(int n, F func)`, вызывающую `func()` ровно `n` раз. Шаблонный параметр `F`.
7. (Сложнее) Сделайте простой `Function<int(int)>` — упрощённая копия `std::function` без аллокации (только для маленьких захватов). Подсказка: внутри буфер `char[32]` и указатель на «vtable» (`void (*invoke)(void*, int)`, и так далее).

22.13 Что дальше

Глава 22 — **исключения и exception safety**. Узнаем, как `try/catch/throw` работает на самом деле, что такое `stack unwinding` (мельком в главе 11), что такое слабая/сильная/`no-throw` гарантия безопасности при исключениях, идиома `copy-and-swap`, почему многие C++-проекты исключения отключают.

Затем — главы 23 (парсер мира из текстового файла), 24 (сохранения), 25 (финал RPG). После этого Часть II закрыта.

Глава 23

Глава 22. Исключения и exception safety

Исключения — механизм сигнализации об ошибках в C++. Они мощные и удобные, но имеют свою цену и кучу подводных камней. Эта глава — детальный разбор: как работают throw/try/catch, что такое stack unwinding, какие бывают гарантии безопасности при исключениях, и почему некоторые большие проекты (Google, ядро Chrome, LLVM) исключения **отключают**.

В коде добавим иерархию ошибок (GameError → ValidationError, WorldError), научимся ловить исключения в main, и проверим валидацию через них.

23.1 Зачем исключения

В функции что-то пошло не так. Файл не открылся. Память не выделилась. Параметр невалидный. Как сообщить об ошибке вызывающему?

Подход 1: возвращаемое значение.

```
int divide(int a, int b, int& result) {
    if (b == 0) return -1;    // код ошибки
    result = a / b;
    return 0;
}

int r;
if (divide(10, 0, r) != 0) {
```



```
std::cerr << "ошибка\n";
}
```

Проблемы: - Возвращаемое значение «занято» под код ошибки — реальный результат через out-параметр. - Каждый вызов нужно проверять. Забыли — баг. - Глубоко в стеке вызовов ошибка должна **«всплыть»** через все промежуточные функции — каждая проверяет, каждая возвращает.

Подход 2: глобальная переменная `errno` (C-стиль).

```
double d = std::strtod("not a number", nullptr);
if (errno == ERANGE) { ... }
```

Те же проблемы плюс **глобальное состояние**, плохо с потоками.

Подход 3: исключения.

```
int divide(int a, int b) {
    if (b == 0) throw std::invalid_argument("division by zero");
    return a / b;
}

try {
    int r = divide(10, 0);
} catch (const std::exception& e) {
    std::cerr << "ошибка: " << e.what() << "\n";
}
```

Преимущества: - **Возвращаемое значение свободно** — для результата. - **Нельзя случайно проигнорировать** — необработанное исключение валит программу. - **Автоматическое распространение** — пробивает все уровни стека.

Недостатки тоже есть, разберём.

23.2 throw, try, catch

Базовый синтаксис:

```
try {
    // код, который может бросить
    do_something();
} catch (const SpecificException& e) {
    // ловим конкретный тип
```

```

} catch (const std::exception& e) {
    // ловим базу всех стандартных
} catch (...) {
    // ловим всё, что осталось (плохой стиль, но иногда нужно)
}

```

Бросить исключение — throw expression:

```

throw std::runtime_error("что-то пошло не так");
throw std::invalid_argument("плохой параметр");
throw 42;      // да, можно бросить int — но не надо

```

throw ищет ближайший try в стеке вызовов, у которого есть подходящий catch. Если найден — управление прыгает туда, и **все локальные переменные** между throw и catch уничтожаются по обычным правилам (вызываются деструкторы). Это и есть **stack unwinding**.

Если ни одного подходящего catch нет — программа вызывает std::terminate, что обычно приводит к падению с сообщением вроде terminate called after throwing an instance of 'std::runtime_error'.

23.2.1 Ловите по ссылке, не по значению

```

catch (std::exception e) { ... }      // ПЛОХО: копия
catch (const std::exception& e) { ... } // ХОРОШО: ссылка

```

Если ловим по значению — исключение копируется. Хуже того: если оно полиморфное (бросили std::runtime_error, ловите как std::exception), при копировании происходит **slicing** (помните из главы 17). Теряются дополнительные данные.

Ссылка избегает копии и slicing. **Всегда ловите по const T&.**

23.2.2 Несколько catch

Можно ловить разные типы:

```

try {
    risky();
} catch (const std::invalid_argument& e) {
    // плохие параметры
} catch (const std::runtime_error& e) {
    // что-то сломалось в рантайме
} catch (const std::exception& e) {

```

```

    // все остальные стандартные
} catch (...) {
    // вообще всё (последняя линия защиты)
}

```

Проверяются **сверху вниз**, срабатывает первый подходящий. Полиморфизм: `const std::exception&` поймает любой потомок, поэтому ставьте конкретные типы **выше** общих, иначе общий перехватит всё.

`catch (...)` — без типа — ловит **любое** исключение. Удобно для логирования и продолжения работы, но информации никакой (даже `e.what()` нельзя). Использовать редко.

23.2.3 Rethrow

Внутри `catch` можно «передать дальше»:

```

try {
    risky();
} catch (const std::exception& e) {
    log("ошибка: ", e.what());
    throw;    // та же ошибка летит дальше
}

```

`throw;` без аргументов — повторно бросает текущее исключение. Можно и новое:

```

catch (const std::runtime_error& e) {
    throw std::runtime_error("обёрнуто: " + std::string(e.what()));
}

```

Это иногда нужно для добавления контекста.

23.3 std::exception и иерархия

В стандартной библиотеке есть **базовый класс** `std::exception` с методом `what()`:

```

class std::exception {
public:
    virtual const char* what() const noexcept;
    virtual ~exception();
};

```

От него наследуются:

- `std::logic_error` — программная ошибка (что-то, что **программист должен был избежать**):
 - `std::invalid_argument`
 - `std::out_of_range`
 - `std::length_error`
 - `std::domain_error`
- `std::runtime_error` — внешняя ошибка (что-то, что **программа не могла предвидеть**):
 - `std::range_error`
 - `std::overflow_error`
 - `std::underflow_error`
 - `std::system_error`
- `std::bad_alloc` — не выделилась память (бросается `new`).
- `std::bad_cast` — неудачный `dynamic_cast` к ссылке.

Эта иерархия даёт **тип-безопасное** ловление. Если ловите `std::out_of_range` — точно знаете, что это «индекс за границей». Если ловите `std::exception` — поймает что угодно стандартное.

Свои ошибки лучше **наследовать** от подходящих стандартных. Тогда любой код, ловящий `std::exception`, поймает и ваши.

23.4 Свои типы исключений

В нашем проекте сделаем три:

```
// include/errors.h
class GameError : public std::runtime_error {
public:
    explicit GameError(const std::string& msg)
        : std::runtime_error(msg) {}
};

class ValidationError : public GameError {
public:
    explicit ValidationError(const std::string& msg)
        : GameError("ValidationError: " + msg) {}
};

class WorldError : public GameError {
public:
    explicit WorldError(const std::string& msg)
```

```

: GameError("WorldError: " + msg) {}
};

```

- `GameError` — корень нашей иерархии. Кто-то ловит **все** наши ошибки одним `catch (const GameError&)`.
- `ValidationError` — невалидный ввод (пустое имя, отрицательный HP).
- `WorldError` — состояние мира неправильное (локация не найдена).

`explicit` у конструктора — запрещает неявную конверсию из `std::string`. Без него можно было бы случайно сделать `throw "some text"` (точнее, не совсем, но всё же).

23.4.1 Базовый тип `runtime_error`

Я наследую от `std::runtime_error`, потому что:

1. Это стандартный класс. Любой код, ловящий `std::exception` или `std::runtime_error`, поймает мою ошибку.
2. У него уже есть конструктор от `std::string` и реализация `what()`. Не надо переписывать.

Альтернатива — наследовать прямо от `std::exception` и самим реализовывать `what()`. Сложнее, без явных плюсов.

23.5 Применяем: валидация в `Player`

В `src/player.cpp`:

```

Player::Player(std::string name, int max_hp)
: name_(std::move(name)),
  max_hp_(max_hp),
  hp_(max_hp) {
    if (name_.empty()) {
        throw ValidationError("имя игрока не может быть пустым");
    }
    if (max_hp <= 0) {
        throw ValidationError("max_hp должен быть положительным");
    }
}

```

Важно: проверяем после инициализации. К моменту `throw` поля уже инициализированы. Это нормально — при выходе из конструктора через исключение поля корректно уничтожатся (их деструкторы вызовутся).

Что произойдёт, если `Player p("", 100):`

1. Конструктор вызывается.
2. `name_` инициализируется пустой строкой через `move`.
3. `max_hp_` и `hp_` = 100.
4. Проверка: `name_.empty()` истина.
5. `throw ValidationError(...)` — исключение полетело.
6. Stack unwinding: `name_` уничтожается (вызывается `~std::string()`), `max_hp_/hp_` — тривиально.
7. Объект `p` **никогда не создан**. Его как будто не было.
8. Исключение летит к ближайшему `catch`.

Это **критично важное свойство**: если конструктор бросил, объекта нет. Нельзя получить «полусоздан» объект. RAII работает безопасно.

23.6 Главный try/catch в main

Чтобы не падать с ужасным сообщением `terminate after throwing...`, оборачиваем `main`:

```
int main() {
    try {
        rpg::Game game;
        return game.run();
    } catch (const rpg::GameError& e) {
        std::cerr << "Игра не запустилась: " << e.what() << "\n";
        return 1;
    } catch (const std::exception& e) {
        std::cerr << "Неожиданная ошибка: " << e.what() << "\n";
        return 2;
    } catch (...) {
        std::cerr << "Неизвестная ошибка\n";
        return 3;
    }
}
```

Три уровня:

1. **Наши ошибки** — выводим как «ожидаемые», возвращаем 1.
2. **Стандартные ошибки** — как «неожиданные», возвращаем 2.
3. **Что-то совсем странное** — возвращаем 3.

Это **best practice**: ваш `main` должен быть единой точкой обработки всех исключений. Внутри программы — бросайте свободно; в `main` ловите и форматируйте для пользователя.

23.7 Stack unwinding под капотом

Когда `throw` срабатывает, среда выполнения C++ делает примерно следующее:

1. **Создаёт объект исключения** в специальной памяти (не на стеке вызвавшего, потому что стек будет очищаться).
2. **Идёт вверх по стеку**, проверяя для каждого кадра: есть ли в нём активный `try` с подходящим `catch`?
3. Для каждого пройденного кадра **вызывает деструкторы** локальных переменных. Это и есть `unwinding`.
4. Когда найден подходящий `catch` — управление переходит туда.

Этим механизмом пользуется **RAII** (глава 11). Все RAII-объекты в `stack frame` получают свой деструктор бесплатно, даже когда исключение разрушает выполнение.

```
void foo() {  
    std::ofstream f("output.txt");    // RAII: файл откроется  
    risky_operation();                // бросает исключение  
    f << "это не запишется";          // эта строка не выполнится  
    // ~f вызывается АВТОМАТИЧЕСКИ во время unwinding — файл корректно закроется  
}
```

В C-стиле пришлось бы оборачивать в `try/catch` и закрывать вручную. С C++-RAII — само.

23.8 noexcept — обещание не бросать

Функция, помеченная `noexcept`, обещает не бросать исключений:

```
void safe_thing() noexcept {  
    // не бросает  
}
```

Если **всё-таки бросит** — программа вызывает `std::terminate` (как при необработанном). То есть `noexcept` это **строгое** обещание; нарушение фатально.

Зачем `noexcept`:

- **Документация.** Программист видит «эта функция гарантированно не бросит».
- **Оптимизация.** Компилятор может не генерировать `stack-unwinding`-таблицу для этой функции — экономия места.

- **Move-семантика:** `std::vector` использует move-конструктор только если он noexcept (см. главу 19).
- **Деструкторы должны быть noexcept.** В C++11 деструкторы **неявно** noexcept, если не сказали иначе. Это правильно — бросать из деструктора почти всегда баг.

Ставьте noexcept на: - Деструкторы (хотя автоматически). - Move-конструкторы и move-присваивание. - Простые геттеры, не делающие ничего сложного.

Не ставьте noexcept на функции, которые могут позвать аллокацию памяти (`new`, `std::vector::push_back`) — они могут бросить `std::bad_alloc`.

23.9 Exception safety — три уровня гарантий

Когда функция бросает, что с **состоянием программы**? Этот вопрос изучили глубоко, выделили три уровня.

23.9.1 Уровень 1: basic guarantee — слабая

После исключения программа **в валидном состоянии**, нет утечек ресурсов, инвариантов не нарушено. Но конкретное содержимое может быть «частично применённым» — то есть какие-то изменения сделаны, какие-то нет.

Пример. Функция, добавляющая элементы в два контейнера:

```
void add_to_both(std::vector<int>& a, std::vector<int>& b, int v) {
    a.push_back(v);
    b.push_back(v);    // если эта строка бросит — a уже изменён, b нет
}
```

Basic guarantee: оба вектора целы, утечек нет, но программа **не может предположить**, что обе вставки сделаны.

23.9.2 Уровень 2: strong guarantee — сильная

Либо операция полностью успешна, либо состояние **точно как было**. «Транзакционность» — всё или ничего.

```
// Сильная гарантия:
void add_to_both(std::vector<int>& a, std::vector<int>& b, int v) {
    std::vector<int> a_new = a;    // копия
    a_new.push_back(v);

    std::vector<int> b_new = b;
```



```

b_new.push_back(v);

// Если до сюда дошли — операции прошли. Теперь атомарно меняем.
a.swap(a_new);    // swap у vector — noexcept
b.swap(b_new);
}

```

Если любая из вставок бросает — оригиналы `a` и `b` нетронуты. Это потому, что мы делали копию, и только в конце атомарно меняли.

Это **copy-and-swap** идиома — частый паттерн для сильной гарантии.

23.9.3 Уровень 3: **nothrow guarantee** — без бросков

Функция **гарантированно не бросает**. Например, деструкторы, простые геттеры, перемещения. Идеал.

```
void operator=(const T& other) noexcept { ... }
```

23.9.4 Какая гарантия предпочтительна

- Деструкторы — **nothrow** (иначе ад при stack unwinding — что делать с исключением во время unwinding?).
- Move-операции — **nothrow** (для оптимизации STL).
- Простые методы (getters, etc) — **nothrow** где возможно.
- Сложные методы — хотя бы **basic**, лучше **strong** через copy-and-swap.

В нашей RPG большая часть кода имеет basic guarantee. Это часто достаточно, особенно с RAII.

23.10 Copy-and-swap идиома детально

Классический пример: оператор присваивания с сильной гарантией.

```

class Buffer {
public:
    Buffer(const Buffer& other);    // copy ctor
    ~Buffer();

    // Простая copy-and-swap
    Buffer& operator=(Buffer other) { // принимаем по значению — компилятор сам копирует
        swap(*this, other);         // меняемся местами с временной
    }
}

```

```
        return *this;
    }

    friend void swap(Buffer& a, Buffer& b) noexcept {
        using std::swap;
        swap(a.data_, b.data_);
        swap(a.size_, b.size_);
    }

private:
    int* data_;
    std::size_t size_;
};
```

Что происходит: 1. Параметр `other` создаётся как **копия** аргумента. Если копирование бросит — `*this` нетронут. 2. `swap` (без `std::`, для нахождения через ADL — argument-dependent lookup) меняет внутренности. Он `noexcept`, не бросает. 3. В конце `other` уничтожается, забирая с собой **бывшие данные** `*this`.

Если бы кому-то пришло на ум присваивать самому себе (`a = a`), сначала бы скопировался `a` в `other`, потом `swap`, потом `other` уничтожился. Всё корректно.

Это **исключительно элегантный** паттерн. Использует RAII (`other` уничтожается сама), move-семантику (передача параметра), и одну операцию `swap` с гарантией `noexcept`. Сильная гарантия за дёшево.

23.11 Цена исключений

«Цена» — спорный вопрос. Зависит от ABI и оптимизаций.

На «happy path» (нет исключений): - Современные ABI (Itanium, Microsoft x64) реализуют «zero-cost exception handling» — никакого оверхеда, пока исключение не бросилось. Стек-таблицы лежат в памяти, но не выполняются. - Совсем нулевой? Не совсем. Компилятор не может агрессивно оптимизировать функции, которые могут бросать.

Когда исключение полетело: - Дорого. Поиск `catch`'а, unwinding, вызов деструкторов — десятки микросекунд. - Поэтому не используйте исключения для **обычного управления потоком** (например, выход из цикла). Только для **ошибочных** ситуаций.

В обычном коде overhead исключений не виден. В код горячих циклов и реал-тайм-систем — заметен.

23.12 Почему отключают исключения

Несколько громких проектов отключают `-fno-exceptions`:

- **Google C++ Style Guide** — отключает в основном legacy-причинам (старый код Google не знал исключений, плюс есть проблемы с интеграцией с C-кодом).
- **Embedded** — overhead неприемлем, плюс таблицы исключений жрут память.
- **Game engines** — стабильность fps критична, нельзя «зависать» на исключении.
- **LLVM/Clang** — отключены, ошибки через `llvm::Error`-тип.

При `-fno-exceptions`: `- throw` → ошибка компиляции (или `abort()`). - Любой код стандартной библиотеки, который может бросить — заменяется на `abort()` или возврат специального значения. - Используются альтернативные механизмы: коды ошибок, `std::expected` (C++23), `Result`-типы.

В нашей книге исключения используем — это идиоматический C++ для приложений. Но знайте, что в индустрии бывает по-разному.

23.13 Когда что выбирать: исключение vs код ошибки vs assert

Грубая модель:

- **assert** для ошибок программиста, которые **никогда не должны** случаться в правильной программе. Деление на ноль в коде, который должен это исключать. Пустой указатель там, где гарантированно не пустой. В release-сборке `assert` отключается.
- **Возврат `bool/std::optional`/код ошибки** для **ожидаемых** «безуспешных» исходов, которые не считаются ошибкой. Например, «не нашли элемент». Тут исключение — overkill.
- **Исключения для реальных ошибок**, которые **должен поймать вышестоящий код**. Файл не открылся, память не выделилась, параметр невалиден.

В нашей RPG: `- Inventory::take` возвращает `nullptr` при отсутствии — это не ошибка, а нормальная ситуация. `- Player(empty_name, ...)` бросает — это программистская ошибка. `- World::find` возвращает `nullptr` — не ошибка, а «нет такой».

23.14 Главные правила главы

1. **Ловите по `const T&`** — не по значению, не по указателю.
2. **Бросайте конкретные типы**, наследованные от `std::runtime_error` или `std::logic_error`.
3. **Главный `try/catch` в `main`** — единая точка форматирования ошибок.
4. **Деструкторы и `move`** — поexcept. Без исключений.

5. **assert для невозможных ситуаций**, исключение для реальных ошибок.
6. **copy-and-swap** для сильной гарантии оператора присваивания.
7. **Не используйте исключения для управления потоком** — они дорогие при бросании.
8. **-fno-exceptions** — реальная опция в некоторых проектах. Подумайте, как ваш код переживёт это.

23.15 Маленькое упражнение

1. Запустите `test_exceptions.cpp` (`make tests && ./build/tests/test_exceptions`). Изучите вывод.
2. Добавьте в `Inventory::take` бросок `WorldError`, если предмет не найден (вместо возврата `nullptr`). Что произойдёт с `cmd_take` в игре? Какая логика проще: возврат `nullptr` или `try/catch`?
3. Перепишите `Location::exit` так, чтобы возвращал `std::string` (сейчас и так), но при отсутствии направления **бросал** `WorldError`. Команда `cmd_go` должна это ловить и говорить «не пройти». Сравните с текущим кодом.
4. Сделайте функцию, которая нарочно бросает в деструкторе. Запустите. Что произойдёт?
5. Сделайте функцию `divide` с двумя `int`-параметрами. Бросайте `std::invalid_argument` при делении на 0. Поймайте в `main`. Выведите `e.what()`.
6. Прочитайте `errors.h`. Объясните, почему конструкторы `ValidationError/WorldError` принимают `const std::string&`, а не `std::string` по значению.
7. (Сложнее) Сделайте функцию, бросающую `std::runtime_error`. Поймайте, **залогируйте**, и **перебросьте** (`throw;`). Поймайте на следующем уровне. Это типичный паттерн добавления контекста к ошибке.

23.16 Что дальше

Глава 23 — **парсер скриптов локаций**. Мир игры будет загружаться из текстового файла (`data/world.txt`), а не зашиваться в C++. Это огромный шаг: можно править контент без перекомпиляции, добавлять локации руками, делиться мирами. Используем `std::ifstream` (глава 12), `std::stringstream` (12), `std::unordered_map` (13), и наши `Item`-классы (17, 18). Это будет реальная мини-задача парсинга — пример того, что вы сможете применять везде.

Затем — глава 24 (сохранения), глава 25 (финал RPG), и Часть II закрыта.

Глава 24

Глава 23. Парсер скриптов локаций

До сих пор наш мир из трёх локаций был **зашит в C++**: функция `make_demo_world()` явно создавала их вручную. Чтобы добавить пещеру, нужно было править `world.cpp`, пересобирать программу, запускать. Это работает для трёх локаций, но не масштабируется.

В этой главе мы построим **парсер**: мир будет описываться в текстовом файле `data/world.txt`, и программа будет его читать. Добавил локацию — сохранил файл — запустил, никаких пересборок. Это огромный шаг: контент отделён от кода.

Заодно — это первая в книге задача парсинга. Очень полезный навык. Любая программа, читающая конфиги, JSON, XML, CSV, файлы данных — это тот же парсинг. Освойте принцип на простом примере — лучше поймёте сложные форматы.

24.1 Зачем парсер

Несколько причин, по которым контент не зашивают в код:

1. **Скорость итерации.** Дизайнер игры (или вы сами) меняет локацию — сохранил файл — посмотрел в игре. Без перекомпиляции.
2. **Расширяемость пользователями.** Кто-то скачивает игру, открывает `world.txt`, добавляет свои локации. Получает свою версию.
3. **Локализация.** Тексты в файлах легко переводить, не лезя в C++-код.
4. **Разделение компетенций.** Программисту не приходится «писать тексты», дизайнеру — лезть в код.
5. **Скрипты vs данные.** Можно держать «движок» в C++, а «контент» в текстовых файлах.

Большие игры (Skyrim, World of Warcraft) хранят сотни тысяч объектов в файлах. Жёсткое разделение «движок против контента» — стандарт индустрии.

24.2 Выбор формата

Какой формат файла? Варианты:

JSON. Стандарт. Хорошо парсится, есть библиотеки. Но многословный и требует кавычек:

```
{
  "id": "glade",
  "name": "Лесная поляна",
  "description": "...",
  "exits": {"west": "village", "east": "forest"},
  "items": [
    {"type": "weapon", "name": "меч", "weight": 5, "damage": 8}
  ]
}
```

YAML. Менее многословно, но синтаксис со значимыми отступами тонкий.

TOML. Хорошо для конфигов, но плохо для иерархии.

Свой формат. Можно сделать максимально простой под свои нужды.

Для учебной цели свой формат лучше всего: меньше зависимостей, понятнее, что и как парсится. На реальном проекте — JSON или TOML через готовую библиотеку.

Наш формат — **линейно-ключевой**:

```
# Комментарий
```

```
location <id>
name <текст до конца строки>
desc <текст до конца строки>
exit <direction> <target_id>
item <type> <name> <weight> <extra>
```

Каждая строка — одна команда. Парсер читает построчно. Локация начинается с `location <id>`, дальше идут её свойства, пока не встретится новая `location` или конец файла.

24.3 Пример мира

data/world.txt:

```
# Описание мира RPG-демо.
# Имена предметов с подчёркиванием — превратится в пробел.
```

```
location glade
name Лесная поляна
desc Залитая солнцем поляна. На западе тропа, на востоке лес.
exit west village
exit east forest
exit south mountain
item weapon ржавый_меч 5 8
```

```
location village
name Деревня
desc Маленькая деревня из десятка домов.
exit east glade
item consumable зелье_лечения 1 15
item consumable хлеб 1 5
```

```
location forest
name Тёмный лес
desc Ветви скрипят над головой.
exit west glade
exit north cave
item armor кожаный_нагрудник 8 4
```

```
location cave
name Пещера
desc Сырая пещера. Капает вода.
exit south forest
item weapon боевой_топор 7 12
item armor стальной_щит 6 6
```

```
location mountain
name Горная тропка
desc Узкая тропка вдоль обрыва.
exit north glade
item consumable вода 1 3
```

Пять локаций, граф связан, есть и оружие, и броня, и расходники. Без C++.

24.4 Архитектура парсера

Парсер состоит из:

1. **Read line** — читаем строку файла.
2. **Trim** — убираем пробелы по краям.
3. **Skip** — пропускаем комментарии и пустые строки.
4. **Tokenize** — извлекаем keyword и аргументы.
5. **Dispatch** — выполняем нужное действие на основе keyword.

Состояние парсера — текущая «черновая» локация. Каждый name, desc, exit, item модифицирует её. Каждая новая location — сохраняет старую и начинает новую.

Это **state machine**: два состояния — «нет активной локации» и «активная локация». Переход — по ключу location.

24.5 Заголовок

include/world_parser.h:

```
#ifndef RPG_WORLD_PARSER_H
#define RPG_WORLD_PARSER_H

#include "world.h"

#include <iosfwd>
#include <string>

namespace rpg {

    // Парсер мира из текстового формата. Бросает WorldError при ошибках.
    World parse_world(std::istream& in);
    World load_world(const std::string& path);

} // namespace rpg

#endif
```

Две функции: - parse_world(istream&) — для тестов и для случаев, когда хотим читать не из файла. - load_world(path) — удобная обёртка, открывает файл и зовёт parse_world.

<iosfwd> — заголовок с **forward declarations** для потоков. Объявляет `std::istream` и подобные, но без полных определений. Достаточно для параметров. Полная <iostream> тяжёлая (десятки тысяч строк через <algorithm> и прочие транзитивные include). <iosfwd> лёгкий.

В `.cpp` мы подключаем уже полные <fstream> и <sstream> — там нужно работать с потоками реально.

24.6 Реализация: вспомогательные функции

src/world_parser.cpp:

```
namespace { // анонимное пространство — функции видны только в этом файле

std::string trim(const std::string& s) {
    auto b = s.find_first_not_of(" \t\r");
    if (b == std::string::npos) return {};
    auto e = s.find_last_not_of(" \t\r");
    return s.substr(b, e - b + 1);
}

std::string unescape_name(const std::string& s) {
    std::string r = s;
    std::replace(r.begin(), r.end(), '_', ' ');
    return r;
}

} // namespace
```

`trim` — стандартная штука: убрать пробелы/табуляции/\r (на Windows-файлах в конце строки) с обоих краёв.

`unescape_name` — наш способ обработки пробелов в именах. В файле пишем `ржавый_меч`, в игре отображаем `ржавый меч`.

Анонимное пространство (`namespace { ... }`) — для функций, видимых только в этом `.cpp`. Альтернатива `static`. Я в книге использую `namespace { ... }` — это идиоматично в современном C++.

24.7 Промежуточное состояние

Один блок в файле описывает локацию **постепенно** — сначала name, потом desc, потом несколько exit и item. Мы не можем создать Location сразу, нужно собирать.

Решение — **промежуточная структура**:

```
struct LocationDraft {
    std::string id;
    std::string name;
    std::string desc;
    std::vector<std::pair<std::string, std::string>> exits;
    std::vector<std::unique_ptr<Item>> items;
};

Location finalize(LocationDraft& draft) {
    Location loc(std::move(draft.id), std::move(draft.name), std::move(draft.desc));
    for (auto& e : draft.exits) {
        loc.add_exit(e.first, e.second);
    }
    for (auto& item : draft.items) {
        loc.items().add(std::move(item));
    }
    return loc;
}
```

LocationDraft — простой POD (plain old data, набор полей). Мы её наполняем по мере чтения, а в конце finalize превращает в настоящий Location. После этого можно создать новый draft для следующей локации.

Это типичный паттерн **builder**: собираем «чертёж», в конце «строим».

24.8 Парсинг строки предмета

```
std::unique_ptr<Item> parse_item_line(std::istream& ss, int line_no) {
    std::string type, name;
    int weight = 0, extra = 0;
    if (!(ss >> type >> name >> weight >> extra)) {
        throw WorldError("строка " + std::to_string(line_no)
            + ": неполная строка item");
    }
}
```

```

}
std::string pretty = unescape_name(name);

if (type == "weapon")    return make_unique<Weapon>(pretty, weight, extra);
if (type == "armor")    return make_unique<Armor>(pretty, weight, extra);
if (type == "consumable") return make_unique<Consumable>(pretty, weight, extra);

throw WorldError("строка " + std::to_string(line_no)
                 + ": неизвестный тип предмета '" + type + "'");
}

```

ss >> type >> name >> weight >> extra — извлекает четыре поля через `std::istream` (мы их подробно рассмотрели в главе 12). Если поток в failure state — что-то не извлеклось, кидаем `WorldError`.

Дальше по типу диспатчируем: создаём `Weapon`, `Armor` или `Consumable`. Если тип неизвестен — ошибка.

Каждое исключение содержит **номер строки** — это критично для диагностики. Без него «строка item плохая» — где её искать?

24.9 Главный цикл парсера

```

World parse_world(std::istream& in) {
    World world;
    LocationDraft draft;
    bool have_draft = false;

    auto commit_draft = [&]() {
        if (have_draft) {
            world.add(finalize(draft));
            have_draft = false;
            draft = LocationDraft{};
        }
    };

    std::string line;
    int line_no = 0;
    while (std::getline(in, line)) {

```

```
++line_no;
line = trim(line);
if (line.empty() || line[0] == '#') continue;

std::istringstream ss(line);
std::string keyword;
ss >> keyword;

if (keyword == "location") {
    commit_draft();
    ss >> draft.id;
    if (draft.id.empty()) {
        throw WorldError("строка " + std::to_string(line_no)
            + ": location без id");
    }
    have_draft = true;
    continue;
}

if (!have_draft) {
    throw WorldError("строка " + std::to_string(line_no)
        + ": '" + keyword + "' до 'location'");
}

if (keyword == "name") {
    std::string rest;
    std::getline(ss, rest);
    draft.name = trim(rest);
} else if (keyword == "desc") {
    std::string rest;
    std::getline(ss, rest);
    draft.desc = trim(rest);
} else if (keyword == "exit") {
    std::string dir, target;
    ss >> dir >> target;
    if (dir.empty() || target.empty()) {
        throw WorldError("строка " + std::to_string(line_no)
            + ": exit требует <direction> <target_id>");
    }
}
```

```

    }
    draft.exits.emplace_back(std::move(dir), std::move(target));
} else if (keyword == "item") {
    draft.items.push_back(parse_item_line(ss, line_no));
} else {
    throw WorldError("строка " + std::to_string(line_no)
                    + ": неизвестный ключ '" + keyword + "'");
}
}
}

commit_draft();
return world;
}

```

Разберём.

Лямбда `commit_draft` — фиксирует «черновую» локацию в мире. Зачем лямбда? Используется дважды: при встрече новой location и в конце файла. Не хочется дублировать код. Захват [&] — нужно менять draft, have_draft, world снаружи.

Цикл `while (std::getline(in, line))` — читаем строки до конца файла. `getline` возвращает поток в булевом контексте: true пока всё ОК, false при EOF/ошибке.

`if (line.empty() || line[0] == '#') continue;` — пропуск пустых строк и комментариев.

`std::istringstream ss(line)` — превращаем строку в поток для дальнейшего парсинга >>. Мы уже видели этот трюк в главе 12 и 13.

Дальше — диспатч по ключевому слову. Каждая ветка ожидает свои аргументы.

24.9.1 name и desc

```

if (keyword == "name") {
    std::string rest;
    std::getline(ss, rest);
    draft.name = trim(rest);
}

```

`getline(ss, rest)` берёт **весь остаток строки**, включая пробелы. Это нужно, потому что name Лесная поляна — поле из двух слов. Если бы мы делали `ss >> ss >> ...`, потеряли бы «поляна».

`trim(rest)` убирает ведущий пробел (тот, что между name и началом значения).

24.9.2 exit

```

} else if (keyword == "exit") {
    std::string dir, target;
    ss >> dir >> target;
    if (dir.empty() || target.empty()) {
        throw WorldError(...);
    }
    draft.exits.emplace_back(std::move(dir), std::move(target));
}

```

Два слова: направление и id цели. `emplace_back` конструирует `std::pair` прямо в векторе — без промежуточных копий (это глава 13).

24.9.3 item

```

} else if (keyword == "item") {
    draft.items.push_back(parse_item_line(ss, line_no));
}

```

Делегируем в `parse_item_line`. Внутри — четыре поля.

24.9.4 Конец файла

```

commit_draft();
return world;

```

Не забыть зафиксировать последний `draft`. До этого `commit_draft` вызывался только при встрече **новой** location; последний блок завершается концом файла.

24.10 Подключение к Game

Меняем `Game::Game()`:

```

Game::Game()
: running_(true),
  world_(load_world("data/world.txt")), // <--- было make_demo_world()
  player_("Герой", 20),
  current_location_id_("glade") {
    register_commands();
}

```

Одна строка изменилась. Вся остальная логика игры не знает, откуда мир. RAII работает, как раньше.

Если файл недоступен — `load_world` бросит `WorldError`, и наш `main-catch` покажет «Игра не запустилась: `WorldError`: не открылся файл мира». Графично.

24.11 Проверка

```
$ make && echo -e "look\ngo south\nlook\ngo north\ngo east\ngo north\nlook\nq" | ./build/rpg
== Лесная поляна ==
Залитая солнцем поляна. На западе видна тропа к деревне, на востоке — тёмный лес. На юг ведёт горная тропа.
Выходы: south east west
На земле:
  - ржавый меч (оружие, урон 8, вес 5)
  ...
== Горная тропка ==
Узкая тропка вьётся вдоль обрыва. Виден весь лес.
  ...
== Тёмный лес ==
Ветви скрипят над головой. Здесь сумрачно и тихо.
Выходы: north west
На земле:
  - кожаный нагрудник (броня, защита 4, вес 8)
== Пещера ==
Сырая пещера. Где-то в глубине капает вода.
Выходы: south
На земле:
  - боевой топор (оружие, урон 12, вес 7)
  - стальной щит (броня, защита 6, вес 6)
```

Пять локаций (вместо трёх в `make_demo_world`). Все из текстового файла. Пещера, тропка, лес и предметы появились **без правок C++-кода**.

Откройте `data/world.txt`, добавьте ещё одну локацию — пересоберите без `make clean` (только `Makefile` видит изменения). Если изменили только `world.txt` — даже `make` не нужно! Программа просто прочтёт новый файл при следующем запуске.

24.12 Уроки про парсинг

Несколько наблюдений из этой главы.

24.12.1 Состояния парсера

У нас всего два: «нет активного блока» и «активный блок». Переход — `keyword location`. Это **machine с очень малым количеством состояний**.

Для большего формата (например, вложенные блоки) понадобились бы: - Стек состояний (`current score`). - Или рекурсивный парсинг (каждое состояние — отдельная функция).

JSON-парсер, например, использует рекурсию.

24.12.2 Сообщения об ошибках

Каждая ошибка содержит номер строки. Это критично. Без номера пользователь не найдёт проблему.

Хороший парсер на ошибку: - Указывает, где (файл, строка, иногда колонка). - Говорит, что ожидалось. - Говорит, что получено.

Мы это делаем. Можно ещё добавить контекст (несколько строк вокруг), но для учебного достаточно.

24.12.3 Иммутабельность данных

Заметили: `Location` после конструктора **не меняется**. `name`, `desc`, `exits`, `items` устанавливаются один раз. Это упрощает логику и делает класс безопасным к ошибкам.

Если бы у `Location` были сеттеры (`set_name`, `set_desc`), можно было бы строить её итеративно. Но тогда повсюду риск: «а вдруг кто-то поменяет уже созданную локацию во время игры?». Мы избегаем этого через паттерн **builder** (`LocationDraft` → `finalize`).

Это **полу-immutable** дизайн. У `World` сеттер есть (`add`), но он используется только при загрузке.

24.12.4 Tokenizer vs парсер

В нашем парсере токенизация и парсинг **смешаны**: `std::istream` сама делит строку на слова, и мы прямо в логике диспатчим.

В **больших парсерах** (компилятор C++, JSON-движок) это разделяют: 1. **Lexer/tokenizer** превращает текст в поток токенов (`KEYWORD`, `IDENTIFIER`, `NUMBER`, ...). 2. **Parser** работает с токенами, строит AST.

Для нашей задачи разделение `overkill`. Но знайте, что в больших задачах оно есть.

24.12.5 Robust vs strict парсер

Наш парсер **строгий**: любая ошибка — `throw`. Запустился — мир загружен. Не запустился — игра не работает вообще.

Альтернатива — **прощающий** парсер: пропускать неизвестные ключи, не падать на одной плохой локации. Подходит для пользовательских моддинг-форматов: пользователь добавил кривую локацию — остальное всё равно загрузится.

Что лучше — зависит от задачи. Для разработки лучше строгий (баги видны сразу). Для пользователей — мягкий.

24.13 `std::variant` — упоминание (C++17)

В нашем коде у Item-иерархии **три типа**: `Weapon`/`Armor`/`Consumable`. Мы держим их через `std::unique_ptr<Item>` и проверяем через `dynamic_cast`. Это классический ОО-подход.

В C++17 появился `std::variant<Weapon, Armor, Consumable>` — он хранит **один из** перечисленных типов. Доступ через `std::visit` (компилятор сам выбирает нужный код в зависимости от того, что сейчас в `variant`'е).

Преимущества `std::variant`: - Нет виртуальных функций — быстрее. - Тип фиксирован на компиляции — компилятор может оптимизировать. - Нет наследования — проще структура.

Недостатки: - Закрытое множество типов — не добавить четвёртый тип без переделки. - Сложнее API (visitor pattern).

У нас C++11. `std::variant` нет. Альтернатива — **tagged union**: `struct { Type tag; union { Weapon w; Armor a; Consumable c; }; }`. Старо, многословно, но работает. В нашем коде проще остаться на наследовании.

24.14 Главные правила главы

1. **Контент в файлах, движок в коде.** Разделение.
2. **Builder паттерн** для постепенного сбора объекта.
3. **Каждая ошибка с номером строки.** Без этого диагностика плачет.
4. **Анонимные пространства** для функций видимых только в `.cpp`.
5. `std::istream` для парсинга одной строки.
6. `getline(ss, rest)` для получения остатка строки (включая пробелы).
7. **Builder-объект + finalize** для постепенной инициализации иммутабельных классов.

8. **Большие парсеры разделяют tokenizer и parser**, маленькие — нет.

24.15 Маленькое упражнение

1. Запустите игру (`make && ./build/rpg`). Попробуйте все 5 локаций.
2. Добавьте в `data/world.txt` локацию `tower` — высокая башня. Из деревни выход `north tower`, из башни — `south village`. Положите в неё `weapon` магический_посох 3 20. Запустите. Никаких изменений в C++ не делайте.
3. Намеренно сломайте `data/world.txt`: уберите `name` у одной из локаций. Запустите игру. Что произойдёт? Сообщение об ошибке понятное?
4. Добавьте в формат строку `start <id>` — указание стартовой локации. Сейчас зашито `glade` в `Game::Game()`. Сделайте, чтобы стартовая локация читалась из файла.
5. Добавьте поддержку `# inline` комментариев — комментарий **на конце строки**, например:

```
exit west village # дорога вниз
```

Подсказка: найти `#` в строке и обрезать до него.
6. (Сложнее) Сделайте парсер прощающим: при ошибке вместо `throw` логировать предупреждение и продолжать. Подсказка — заменить `throw` на `std::cerr << ...; continue;` или собирать ошибки в `std::vector<std::string>` для итогового отчёта.
7. (Сложнее) Сделайте формат поддерживающим **NPC** — пока без логики, просто запись имени и тек. Расширьте `Location`, чтобы хранил список NPC-имён.

24.16 Что дальше

Глава 24 — **сохранение и загрузка игры**. Текстовый формат vs бинарный, версии формата, обратная совместимость, контрольная сумма. Похоже на парсер мира, но в обе стороны: можем сохранить и можем загрузить.

Затем — глава 25, **финал RPG**: модульные тесты с `GoogleTest`, сборка под три платформы, итоги, что осталось «как упражнение». На этом Часть II закроется, и мы перейдём к проекту `mini-shell`.

Глава 25

Глава 24. Сохранение и загрузка игры

Текущее состояние игры — это **снимок** мира и игрока в момент времени. Чтобы можно было выключить программу и продолжить позже — нужно этот снимок куда-то сохранить и потом восстановить. В этой главе сделаем систему сохранений: формат файла, версионирование, контрольную сумму. Добавим команды `save <slot>` и `load <slot>`.

Тема очень практическая. Любая нетривиальная программа рано или поздно сталкивается с **сериализацией**: сохранением состояния. Игры, базы данных, мессенджеры, графические редакторы — все хранят что-то на диске.

25.1 Что сохранять

Состояние нашей игры состоит из:

- **Игрок**: имя, текущее `hp`, `max hp`, инвентарь (список предметов).
- **Текущая локация**: `id`.
- **Мир**: остаточные предметы по локациям (часть взяли — этих предметов больше нет).

Что **не** сохранять: - Имена локаций, описания, выходы — это статика из `world.txt`. - Список команд, обработчики — это код.

То есть сохраняется **разница** между «свежим миром из `world.txt`» и «текущим состоянием». Это уменьшает размер файла и облегчает миграции (если поменяем мир — старые сейвы продолжат работать в том, что не задела).

25.2 Текстовый или бинарный формат

Два подхода.

Бинарный: - Записываем байты значений напрямую (через `out.write(&value, sizeof(value))`). - Компактно, быстро. - Нечитаемо без декодера. - Зависит от `endianness`, размеров типов.

Текстовый: - Каждое значение — строка. - Больше места, медленнее. - Читается глазами. - Легко версионировать и отлаживать.

Для нашей RPG — **текстовый**. Размер сейва — килобайты, скорость неважна. Зато можно открыть в редакторе и посмотреть, что не так. В мини-СУБД (Часть IV) будет бинарный — там скорость и размер критичны.

25.3 Формат сейва

```
SAVE 1
name Герой
hp 20 20
loc mountain
inv weapon ржавый_меч 5 8
wloc cave weapon боевой_топор 7 12
wloc cave armor стальной_щит 6 6
wloc mountain consumable вода 1 3
wloc forest armor кожаный_нагрудник 8 4
wloc village consumable зелье_лечения 1 15
wloc village consumable хлеб 1 5
CHK 5e54e64c
```

Структура:

- **SAVE <version>** — первая строка, версия формата.
- **name <player_name>** — имя игрока (_ вместо пробелов).
- **hp <current> <max>** — здоровье.
- **loc <location_id>** — где игрок.
- **inv <type> <name> <weight> <extra>** — предмет в инвентаре. Несколько строк.
- **wloc <loc_id> <type> <name> <weight> <extra>** — предмет, оставшийся в локации.
- **CHK <8_hex_digits>** — контрольная сумма для проверки целостности.

Похоже на формат мира из главы 23, но с другими ключами. Один и тот же стиль.

25.4 Зачем версия

Программа развивается. Завтра вы добавите экипировку — у Player появится поле `equipped_weapon`. Старые сейвы не знают о нём.

Без версии: чтение крашится на отсутствии новых полей. Игрок теряет прогресс.

С версией: программа смотрит, какая версия в файле, и применяет соответствующую логику парсинга. Например: - v1 — без экипировки. - v2 — с экипировкой; для v1-сейвов считать «не экипирован».

В нашем коде:

```
namespace save {  
    constexpr int CURRENT_VERSION = 1;  
    // ...  
}
```

При загрузке проверяем:

```
if (ver != CURRENT_VERSION) {  
    throw GameError("несовместимая версия сейва: " + std::to_string(ver));  
}
```

Это **строгая** реакция. Альтернатива — **миграции**: при $v < \text{CURRENT_VERSION}$ вызывать функцию `migrate_v1_to_v2`, потом загружать как v2. Так делают серьёзные приложения. У нас пока строгое отклонение.

25.5 Зачем контрольная сумма

Файлы повреждаются. Прервалась запись (отключили питание), повредился диск, кто-то открыл в редакторе и неудачно правил. Загружать **повреждённый** сейв опасно — можно получить непредсказуемое состояние, краш, или хуже — испорченную игровую логику.

Контрольная сумма (checksum) — функция, дающая на основе содержимого короткое значение. Если содержимое изменилось — значение тоже. На загрузке сравниваем сохранённую сумму с пересчитанной по содержимому. Не сходится — отказ.

Простые алгоритмы: - **Сумма байтов** — очень простая, плохо ловит «компенсирующие» изменения. - **CRC32** — стандартное, точное обнаружение. - **Хеш-функции** (MD5, SHA-1) — криптографические, но избыточны для нашей задачи.

У нас — простой 32-битный «накатывающий хеш»:

```
std::uint32_t checksum(const std::string& text) {
    std::uint32_t sum = 0;
    for (char ch : text) {
        sum = sum * 31u + static_cast<unsigned char>(ch);
    }
    return sum;
}
```

Это **не криптография**. Злонамеренный пользователь подделает её за минуту. Защита только от **случайных** повреждений. Для нашей цели — достаточно.

31 — стандартный множитель в хешах строк (используется в Java String.hashCode). unsigned char — чтобы избежать UB при отрицательных значениях char (см. главу 8).

25.6 API менеджера сохранений

include/save_manager.h:

```
namespace rpg {
namespace save {

constexpr int CURRENT_VERSION = 1;

void write(const std::string& path,
           const Player& player,
           const World& world,
           const std::string& current_location_id);

void read(const std::string& path,
          Player& player,
          World& world,
          std::string& current_location_id);

} // namespace save
} // namespace rpg
```

Две функции: write и read. Оба бросают GameError при проблемах.

Заметьте: save — это **внутреннее пространство имён** внутри rpg. Так не «загрязняем» основной namespace. Использование: rpg::save::write(...).

25.7 Реализация write

```

void write(const std::string& path,
          const Player& player,
          const World& world,
          const std::string& current_location_id) {
    std::ostringstream body;
    body << "SAVE " << CURRENT_VERSION << "\n";
    body << "name " << escape(player.name()) << "\n";
    body << "hp " << player.hp() << " " << player.max_hp() << "\n";
    body << "loc " << current_location_id << "\n";

    for (const auto& item : player.inventory().items()) {
        body << "inv " << item_line(*item) << "\n";
    }
    for (const auto& kv : world.locations()) {
        const Location& loc = kv.second;
        for (const auto& item : loc.items().items()) {
            body << "wloc " << loc.id() << " " << item_line(*item) << "\n";
        }
    }

    std::string content = body.str();
    std::uint32_t sum = checksum(content);

    std::ofstream out(path);
    if (!out.is_open()) {
        throw GameError("не открыт для записи: " + path);
    }
    out << content << "CHK " << to_hex(sum) << "\n";
}

```

Алгоритм:

1. **Собираем тело в stringstream**, чтобы посчитать checksum.
2. Каждое поле — одна строка.
3. Инвентарь игрока через цикл.
4. Все локации мира через `world.locations()` (World теперь возвращает свой `unordered_map`).
5. **Считаем checksum от тела.**

6. Записываем в файл: тело + строка CHK <hex>.

Зачем сначала в stringstream, а потом сразу в файл? Чтобы: 1. Посчитать checksum от того же текста, что попадёт в файл. 2. Если запись частично прошла и оборвалась — можно повторить.

25.7.1 Сериализация предмета

```
std::string item_line(const Item& it) {
    std::ostringstream ss;
    if (auto w = dynamic_cast<const Weapon*>(&it)) {
        ss << "weapon " << escape(w->name()) << " " << w->weight() << " " << w->damage();
    } else if (auto a = dynamic_cast<const Armor*>(&it)) {
        ss << "armor " << escape(a->name()) << " " << a->weight() << " " << a->defense();
    } else if (auto c = dynamic_cast<const Consumable*>(&it)) {
        ss << "consumable " << escape(c->name()) << " " << c->weight() << " " << c->heal_amount();
    } else {
        throw GameError("неизвестный тип предмета при сохранении");
    }
    return ss.str();
}
```

Через `dynamic_cast` определяем реальный тип и пишем нужное «extra» поле (урон для оружия, защиту для брони, лечение для расходника).

Этот код **дублирует** логику парсера из главы 23 — там по типу мы создавали потомков, здесь по типу пишем. Это типичная неприятность сериализации: чтение и запись должны быть **синхронны**.

Альтернативно можно было бы добавить виртуальный метод `Item::serialize(std::ostream&)`, и `Weapon`/`Armor`/`Consumable` его переопределили бы. Это чище — логика типа лежит в самом классе. Но требует трогать `Item.h`. Для учебного — `dynamic_cast` проще.

25.7.2 Запись в файл

```
std::ofstream out(path);
if (!out.is_open()) {
    throw GameError("не открыт для записи: " + path);
}
```

Если файл не открылся — бросаем. Чаще всего причина — нет папки `data/saves/`.

25.7.3 Эскейпинг

`escape(name)` заменяет пробелы на `_`. На загрузке `unescape` обратно. Если бы оставить пробелы, поток `>>` разделил бы имя на части и парсинг сломался.

Альтернатива — заключать имена в кавычки и парсить их специально. Делать сложнее, у нас обходимся подчёркиваниями.

25.8 Реализация `read`

```
void read(const std::string& path,
         Player& player,
         World& world,
         std::string& current_location_id) {
    std::ifstream in(path);
    if (!in.is_open()) throw GameError("сейв не открыт: " + path);

    std::ostringstream buf;
    buf << in.rdbuf();
    std::string text = buf.str();
    // ...
}
```

Сначала читаем **весь файл в строку**. Зачем? Чтобы проверить checksum: нужен текст без строки CHK.

```
auto chk_pos = text.rfind("\nCHK ");
if (chk_pos == std::string::npos) {
    throw GameError("в сейве нет CHK — файл повреждён");
}

std::string body = text.substr(0, chk_pos + 1);
std::string chk_line = text.substr(chk_pos + 1);

std::istringstream chk_ss(chk_line);
std::string chk_word, chk_hex;
chk_ss >> chk_word >> chk_hex;

std::uint32_t expected = static_cast<std::uint32_t>(std::stoul(chk_hex, nullptr, 16));
std::uint32_t actual = checksum(body);
if (expected != actual) {
```

```
throw GameError("чек-сумма не сходится — сейв повреждён");
}
```

`rfind("\nCHK ")` — ищем строку CHK с конца. `text.substr(0, chk_pos + 1)` — всё до CHK (включая `\n`). `text.substr(chk_pos + 1)` — сама строка CHK.

Парсим hex через `std::stoul(s, nullptr, 16)`. 16 — основание системы.

Сравниваем суммы. Не сходятся — бросаем.

25.8.1 Двухпроходный парсер

Главная тонкость загрузки: **не применять состояние сразу**. Если посередине файла ошибка, у нас окажется наполовину загруженная игра — нелогичное состояние.

Решение — **двухпроходно**: сначала всё парсим во **временные структуры**, проверяем валидность, **потом** атомарно применяем.

```
std::string new_name = player.name();
int new_hp = 0, new_max = 1;
std::string new_loc;
std::vector<std::unique_ptr<Item>> player_items;
std::unordered_map<std::string, std::vector<std::unique_ptr<Item>>> location_items;

while (std::getline(lines, line)) {
    // парсим в new_name / new_hp / player_items / location_items
}

// Валидация
for (const auto& kv : location_items) {
    if (!world.find(kv.first)) {
        throw GameError("сейв ссылается на неизвестную локацию: " + kv.first);
    }
}

// Применение — теперь точно ничего не упадёт
player = Player(new_name, new_max);
player.set_hp(new_hp);
for (auto& it : player_items) {
    player.inventory().add(std::move(it));
}
```

```

for (const auto& kv : world.locations()) {
    Location* mloc = world.find(kv.first);
    if (mloc) mloc->items().clear();
}
for (auto& kv : location_items) {
    Location* mloc = world.find(kv.first);
    for (auto& it : kv.second) {
        mloc->items().add(std::move(it));
    }
}
current_location_id = new_loc;

```

Это **strong exception guarantee** (глава 22): если что-то бросит до «применения» — состояние программы не изменилось. Если применение пошло — гарантированно завершится (нет операций, которые могут бросить на этом этапе; перемещения `unique_ptr` это поexcept).

25.8.2 Replace player

```

player = Player(new_name, new_max);
player.set_hp(new_hp);

```

Move-assignment `player_ = ....` Поскольку `Player` move-assignable (rule of zero — все поля движимы), это работает. Старый `Player` уничтожается, новый занимает его место.

После пересоздания `player` — пустой инвентарь. Далее добавляем сохранённые предметы.

25.9 Команды в Game

```

void Game::cmd_save(const std::string& slot) {
    std::string path = "data/saves/" + slot + ".sav";
    try {
        save::write(path, player_, world_, current_location_id_);
        std::cout << "Сохранено в " << path << "\n";
    } catch (const GameError& e) {
        std::cout << "Ошибка сохранения: " << e.what() << "\n";
    }
}

void Game::cmd_load(const std::string& slot) {

```

```

std::string path = "data/saves/" + slot + ".sav";
try {
    World fresh = load_world("data/world.txt");
    save::read(path, player_, fresh, current_location_id_);
    world_ = std::move(fresh);
    std::cout << "Загружено из " << path << "\n";
    cmd_look();
} catch (const GameError& e) {
    std::cout << "Ошибка загрузки: " << e.what() << "\n";
}
}

```

cmd_save собирает путь, вызывает write. Исключения ловит и показывает по-человечески.

cmd_load сначала загружает **свежий мир** из world.txt (статика — имена локаций, описания, выходы), потом накладывает сейв на этот мир (динамика — игрок, оставшиеся предметы). Если всё ок — заменяет world_ на fresh.

Регистрация:

```

commands_.add("save", [this](const std::string& rest) {
    if (rest.empty()) std::cout << "Имя слота?\n";
    else cmd_save(rest);
});
commands_.add("load", [this](const std::string& rest) {
    if (rest.empty()) std::cout << "Имя слота?\n";
    else cmd_load(rest);
});

```

25.10 Тест

```

$ mkdir -p data/saves
$ make
$ echo -e "take ржавый меч\ngo south\nsave slot1\nq" | ./build/rpg
...
> Вы подняли: ржавый меч
== Горная тропка ==
...
> Сохранено в data/saves/slot1.sav

```

```
$ cat data/saves/slot1.sav
SAVE 1
name Герой
hp 20 20
loc mountain
inv weapon ржавый_меч 5 8
wloc cave weapon боевой_топор 7 12
wloc cave armor стальной_щит 6 6
wloc mountain consumable вода 1 3
wloc forest armor кожаный_нагрудник 8 4
wloc village consumable зелье_лечения 1 15
wloc village consumable хлеб 1 5
CHK 5e54e64c

$ echo -e "load slot1\nstatus\ninv\nq" | ./build/rpg
== Лесная поляна ==
...
> Загружено из data/saves/slot1.sav
== Горная тропка ==
> Герой: HP 20/20
> Инвентарь (общий вес 5):
  - ржавый меч (оружие, урон 8, вес 5)
```

Сохранение/загрузка работают.

Заметьте: после загрузки **меч в инвентаре** (мы его взяли до сейва), на **поляне его нет** (мы его подняли), мы **на горной тропке** (мы туда переместились). Состояние восстановлено в точности.

25.11 Полломка checksum

```
$ sed -i '' 's/CHK 5e54e64c/CHK 00000000/' data/saves/slot1.sav
$ echo "load slot1" | ./build/rpg
> Ошибка загрузки: GameError: чек-сумма не сходится — сейв повреждён
```

Изменили чек-сумму руками — load отказался. Это и есть страховка.

Удалите CHK строку совсем:

```
$ sed -i '' '/^CHK /d' data/saves/slot1.sav
$ echo "load slot1" | ./build/rpg
> Ошибка загрузки: GameError: в сейве нет CHK — файл повреждён
```

Опять отказ. Файл без чек-суммы — повреждён.

25.12 Что осталось

В реальной игре сделали бы ещё:

- **Метаданные сейва:** дата создания, имя игры, скриншот.
- **Несколько слотов** с автоматическими именами (slot1, slot2, ..., autosave).
- **Quicksave / quickload** на горячие клавиши.
- **Журнал изменений** (autosave каждые N минут).
- **Облачная синхронизация.**

Это всё надстройка над тем, что мы сделали. Базовая работа — write / read с версией и checksum — закрыта.

25.13 Бинарный формат — кратко

Для контраста: тот же сейв в бинарном виде мог выглядеть так:

```
out.write(reinterpret_cast<const char*>(&version), sizeof(version)); // 4 байта
out.write(reinterpret_cast<const char*>(&name_len), sizeof(name_len));
out.write(name.data(), name_len);
out.write(reinterpret_cast<const char*>(&hp), sizeof(hp));
out.write(reinterpret_cast<const char*>(&max_hp), sizeof(max_hp));
// ...
```

Преимущества: - Меньше места: int это 4 байта, не 1-10 символов как в тексте. - Быстрее запись и чтение. - Не нужен парсинг — читаем по байтам в нужные позиции.

Недостатки: - Нечитаемо без декодера. - Зависит от endianness (на разных архитектурах байты в int могут лежать по-разному). - Сложнее версионировать (добавил поле — старый файл неправильно интерпретируется). - Сложнее отлаживать.

В Части IV (мини-СУБД) мы будем плотно работать с бинарными форматами — там скорость и размер критичны. Для игры — текст лучше.

25.14 Главные правила главы

1. **Сохранение и парсер дополняют друг друга** — формат должен уметь сериализовать что парсер понимает.
2. **Версия в первой строке** — для обратной совместимости.
3. **Checksum для целостности** — простой хеш достаточен.
4. **Двухпроходный загрузчик** — сначала всё в локальные переменные, потом применяем (strong guarantee).
5. **Текст для отладки, бинарь для производительности.**
6. **Сохраняйте только разницу со статикой** — не дублируйте world.txt в сейве.
7. **Эскейпинг для значений с пробелами** — `_` → или кавычки.
8. **Метаданные** (дата, имя) полезны на больших проектах.

25.15 Маленькое упражнение

1. Запустите игру, поиграйте, сохраните в slot1. Откройте data/saves/slot1.sav в редакторе. Изучите формат.
2. Сделайте slot2, slot3. Все работают независимо.
3. Сломайте чек-сумму руками — проверьте, что load отказывает.
4. Добавьте поддержку **saves** — команда выводящая список существующих сейвов в data/saves/. Подсказка: `std::filesystem` в C++17 / для C++11 — `opendir` из `<dirent.h>` или просто пробег по предсказуемым именам.
5. Добавьте в сейв **дату создания** (число секунд от эпохи). Используйте `std::time(nullptr)`. На загрузке игнорируйте, но при списке сейвов выводите.
6. Добавьте версию 2: в неё дополнительное поле «опыт игрока» (поле в Player ещё не добавлено — добавьте). Сделайте код, который при `v=1` ставит опыт в 0, при `v=2` читает.
7. (Сложнее) Замените простую сумму на CRC32. Готовая реализация — `<zlib.h>` или таблица CRC32 руками. Сравните, как ловит специально подобранные повреждения.
8. (Сложнее) Перепишите сейв в бинарном формате. Сравните размер с текстовым.

25.16 Что дальше

Глава 25 — **финал RPG**. Закрепляем всё, что построили. Модульные тесты с GoogleTest (или свой простой test framework). Сборка на macOS/Linux/Windows. Итоги. Что осталось как упражнения для читателя. Это закрывает Часть II.

После — Часть III (mini-shell), Часть IV (мини-СУБД), Часть V (ТСР-чат), Часть VI (C++17 бонусы). Тут разворачивается остальная половина книги.

Глава 26

Глава 25. Финал RPG

Часть II подходит к концу. У нас в репозитории `demo-rpg/` — больше 1600 строк C++, полностью рабочий мини-движок текстовой RPG. Игрок ходит по миру, поднимает предметы, использует расходники, сохраняет и загружает игру. Мир — в текстовом файле, можно править без перекомпиляции.

В этой финальной главе мы: 1. Добавим **юнит-тесты** для основных классов — практика, которой пользуются на любых проектах. 2. Соберём итог: что построили в C++, какие приёмы освоили. 3. Поговорим про сборку на других платформах. 4. Обсудим, что осталось «за кадром» и можно добавить в качестве упражнений.

После этой главы Часть II закрыта. Дальше — три новых проекта (`mini-shell`, мини-СУБД, ТСП-чат) и обзорная Часть VI про C++17 как бонус.

26.1 Зачем юнит-тесты

Программа из 1600 строк уже достаточно большая, чтобы случайные изменения ломали неожиданные вещи. Вы правите `Inventory::take`, а тут ломается `cmd_drop`. Юнит-тесты — **первая линия защиты** от регрессий.

Хороший тест: - Покрывает один сценарий (одна функция, один аспект). - Запускается за миллисекунды. - Не зависит от других тестов. - Чётко говорит, что прошло и что упало.

Юнит-тесты пишутся **на ту же кодовую базу**, ловят баги до того, как они доберутся до пользователя, и заодно служат документацией: «вот так этот класс используется».

26.2 GoogleTest vs свой простой framework

Стандартный фреймворк тестирования C++ — **GoogleTest**. Он мощный (макросы EXPECT_EQ, фикстуры, параметризованные тесты), но требует подключения внешней зависимости.

Для нашего учебного проекта возьмём **свой минимальный test-runner**. Это: - Никаких внешних зависимостей. - Видна вся логика подсчёта. - Достаточно для нашего масштаба.

В реальном проекте на нескольких десятках тысяч строк — GoogleTest или Catch2. Идея та же, просто фреймворк удобнее.

26.3 Свой test-framework

tests/test_framework.h:

```
#ifndef RPG_TEST_FRAMEWORK_H
#define RPG_TEST_FRAMEWORK_H

#include <iostream>
#include <sstream>
#include <string>

namespace test {

struct Stats {
    int total = 0;
    int passed = 0;

    int report(const std::string& suite) const {
        std::cout << "[" << suite << "]" " << passed << "/" << total
                    << " прошло\n";
        return (passed == total) ? 0 : 1;
    }
};

} // namespace test

#define CHECK(stats, cond) \
    do { \
        (stats).total++; \
    }
```

```

    if (cond) {
        (stats).passed++;
    } else {
        std::cerr << "FAIL " << __FILE__ << ":" << __LINE__
            << " - " #cond << "\n";
    }
} while (0)

#define CHECK_EQ(stats, a, b)
do {
    (stats).total++;
    auto&& _a = (a);
    auto&& _b = (b);
    if (_a == _b) {
        (stats).passed++;
    } else {
        std::ostringstream _ss;
        _ss << "FAIL " << __FILE__ << ":" << __LINE__
            << " - " #a " == " #b " (got " << _a << " vs " << _b << ")";
        std::cerr << _ss.str() << "\n";
    }
} while (0)

#endif

```

Разбор макросов.

do { ... } while (0) — стандартный приём, чтобы макрос вёл себя как одна инструкция. Без этого `if (cond) CHECK(...)`; сломался бы — `do/while(0)` фиксирует «один блок, требующий ; после».

#cond — stringification: имя выражения превращается в строку. `CHECK(s, x > 5)` напечатает `"x > 5"`.

__FILE__ и __LINE__ — макросы препроцессора, подставляют имя файла и номер строки.

auto&& _a = (a); — захват выражения в локальную ссылку, чтобы вычислить **один раз** (выражение могло иметь побочные эффекты — например, `++i`).

Это не идеальный фреймворк (нет фикстур, параметризации), но достаточный.

26.4 Юнит-тесты модулей

tests/test_units.cpp. Каждая функция — отдельная группа тестов:

```
void test_player(test::Stats& s) {
    Player p("Alice", 50);
    CHECK_EQ(s, p.name(), std::string("Alice"));
    CHECK_EQ(s, p.hp(), 50);
    CHECK(s, p.alive());

    p.take_damage(20);
    CHECK_EQ(s, p.hp(), 30);
    p.heal(100);           // не выше max
    CHECK_EQ(s, p.hp(), 50);
    p.take_damage(1000);   // не ниже 0
    CHECK_EQ(s, p.hp(), 0);
    CHECK(s, !p.alive());
}
```

Проверяем: конструктор работает, ограничения работают (heal не превысит max, damage не загонит в минус), alive() правильно меняется.

```
void test_player_validation(test::Stats& s) {
    bool empty_caught = false;
    try { Player p("", 10); }
    catch (const ValidationError&) { empty_caught = true; }
    CHECK(s, empty_caught);

    bool neg_caught = false;
    try { Player p("ok", -1); }
    catch (const ValidationError&) { neg_caught = true; }
    CHECK(s, neg_caught);
}
```

Проверяем валидацию: пустое имя и отрицательный hp бросают исключение. Это **обратная сторона**: показываем, что баги (поломки) тоже работают как ожидаешь.

```
void test_inventory(test::Stats& s) {
    Inventory inv;
    CHECK(s, inv.empty());
}
```

```

    inv.add(make_unique<Weapon>("меч", 5, 8));
    inv.add(make_unique<Armor>("шлем", 3, 4));
    CHECK_EQ(s, inv.items().size(), static_cast<std::size_t>(2));
    CHECK_EQ(s, inv.total_weight(), 8);

    auto taken = inv.take("меч");
    CHECK(s, static_cast<bool>(taken));
    CHECK_EQ(s, taken->name(), std::string("меч"));
    CHECK_EQ(s, inv.items().size(), static_cast<std::size_t>(1));

    auto miss = inv.take("щит");
    CHECK(s, !miss);

    inv.clear();
    CHECK(s, inv.empty());
}

```

Полный жизненный цикл инвентаря: пустой → добавили → проверили → взяли → проверили → попытались взять несуществующее → проверили → очистили.

`static_cast<std::size_t>(2)` — для подавления `-Wsign-compare`: `size()` возвращает `size_t` (unsigned), а 2 — это `int` (signed). С строгими флагами компилятор требует, чтобы оба были одного знака.

26.4.1 Парсер и save/load

```

void test_world_parser(test::Stats& s) {
    std::string source =
        "location start\n"
        "name Старт\n"
        "desc Тут начало.\n"
        "exit east finish\n"
        "item weapon кинжал 2 4\n"
        "\n"
        "location finish\n"
        "name Финиш\n"
        "desc Конец.\n";
    std::istringstream in(source);
    World w = parse_world(in);
}

```

```

CHECK_EQ(s, w.size(), static_cast<std::size_t>(2));

const Location* start = w.find("start");
CHECK(s, start != nullptr);
CHECK_EQ(s, start->name(), std::string("Старт"));
CHECK_EQ(s, start->exit("east"), std::string("finish"));
CHECK_EQ(s, start->items().items().size(), static_cast<std::size_t>(1));
}

```

Парсер тестируем **на строке**, не на файле. Так быстрее и тест не зависит от внешних файлов. `parse_world` принимает `std::istream&` — через `std::istreamstream` подаём строку.

```

void test_save_load_roundtrip(test::Stats& s) {
    World w;
    Location loc("home", "Дом", "Тут уютно.");
    w.add(std::move(loc));

    Player p("Hero", 100);
    p.take_damage(30);
    p.inventory().add(make_unique<Consumable>("яблоко", 1, 5));

    std::string path = "/tmp/rpg_test_save.sav";
    save::write(path, p, w, "home");

    // Восстанавливаем в другой Player/World
    Player p2("Other", 50);
    World w2;
    Location loc2("home", "Дом", "Тут уютно.");
    w2.add(std::move(loc2));
    std::string cur;
    save::read(path, p2, w2, cur);

    CHECK_EQ(s, p2.name(), std::string("Hero"));
    CHECK_EQ(s, p2.hp(), 70);
    CHECK_EQ(s, p2.max_hp(), 100);
    CHECK_EQ(s, cur, std::string("home"));
    CHECK_EQ(s, p2.inventory().items().size(), static_cast<std::size_t>(1));
}

```

Roundtrip тест: записали → прочли → проверили совпадение. Если что-то сломали в сериализации — тест упадёт.

26.4.2 main и запуск

```
int main() {
    test::Stats s;
    test_player(s);
    test_player_validation(s);
    test_inventory(s);
    test_location(s);
    test_world(s);
    test_world_parser(s);
    test_world_parser_errors(s);
    test_save_load_roundtrip(s);
    return s.report("unit-tests");
}
```

Все тесты передают одну и ту же Stats. В конце — отчёт.

Сборка и запуск:

```
$ make tests
$ ./build/tests/test_units
[unit-tests] 37/37 прошло
$ echo $?
0
```

37 проверок, все прошли. Код возврата 0.

Если запустить через CI (Continuous Integration) — например, GitHub Actions — этот тест автоматически запускается на каждый push. Любая регрессия будет видна.

26.5 Сборка на нескольких платформах

Makefile мы заточили под GCC/Clang с Unix-инструментами. Это работает: - **macOS** — clang/g++ из Xcode Command Line Tools (глава 6). - **Linux** — gcc из build-essential. - **Windows + WSL2** — Linux-окружение в Windows.

Что **не работает** на нативном Windows (без WSL): - `mkdir -p` — на Windows это другое. - `rm -rf` — на `cmd.exe` нет `rm`. - Возможно проблемы с UTF-8 в исходниках (MSVC требует особых флагов).

Решений несколько:

1. Использовать CMake вместо Makefile. CMake генерирует Visual Studio проекты на Windows, Makefile на Unix, Ninja-файлы — везде.

```
cmake_minimum_required(VERSION 3.10)
project(rpg CXX)
set(CMAKE_CXX_STANDARD 11)

add_executable(rpg
    src/main.cpp
    src/game.cpp
    src/commands.cpp
    src/item.cpp
    src/inventory.cpp
    src/location.cpp
    src/player.cpp
    src/world.cpp
    src/world_parser.cpp
    src/save_manager.cpp
)

target_include_directories(rpg PRIVATE include)
target_compile_options(rpg PRIVATE -Wall -Wextra)
```

Команда:

```
$ cmake -B build
$ cmake --build build
```

Получаете build/rpg независимо от платформы.

2. WSL2 на Windows. Простейший вариант: пользователь на Windows ставит WSL2, дальше всё как на Linux.

3. Условные правила в Makefile. Можно усложнить наш Makefile, чтобы он автоопределял ОС:

```
ifeq ($(OS),Windows_NT)
    MKDIR := mkdir
    RM     := rmdir /S /Q
else
    MKDIR := mkdir -p
```



```
RM      := rm -rf
endif
```

— и так далее. Возможно, но рассыпает простоту.

Для учебной книги — **простой Makefile + рекомендация WSL на Windows**. На реальном проекте перешли бы на CMake.

26.6 Что мы построили — обзор

Структура demo-rpg/:

```
include/
├─ commands.h      — реестр команд
├─ errors.h        — иерархия исключений GameError/ValidationError/WorldError
├─ event_bus.h     — шаблонная шина событий
├─ events.h        — типы игровых событий
├─ game.h          — главный класс
├─ inventory.h     — инвентарь
├─ item.h          — иерархия предметов (abstract Item + Weapon/Armor/Consumable)
├─ location.h      — локация с выходами и предметами
├─ player.h        — игрок
├─ save_manager.h  — сохранение/загрузка
├─ util.h          — make_unique-шим
├─ world.h         — мир (коллекция локаций)
└─ world_parser.h  — парсер мира из text-файла
src/               — реализации (.cpp на каждый заголовок)
tests/            — юнит-тесты
data/
├─ world.txt       — описание мира
└─ saves/         — слоты сохранений
build/            — артефакты сборки (в .gitignore)
Makefile
```

13 заголовков, 11 реализаций, 6 тестовых файлов. Около 1700 строк C++.

26.7 Что покрыли в C++

Список языковых концепций, которые мы применили **в этом одном проекте**:

26.7.1 Часть I — основы:

- Переменные и типы (`int`, `std::string`, `bool`).
- Управляющие конструкции (`if/else`, `while`, `range-based for`).
- Функции с разными способами передачи параметров.
- Память на стеке и куче, RAII.
- Строки и потоки ввода-вывода.
- Контейнеры STL (`vector`, `unordered_map`).
- Алгоритмы (`std::find_if`, `std::sort`, `std::replace`).

26.7.2 Часть II — продвинутое:

- **Классы и инкапсуляция** — `Player`, `Location`, `World`, `Inventory`, `Item`.
- **Инициализационные списки** в конструкторах.
- **const-correctness** — `const`-методы, `const T&` для чтения.
- **Наследование и полиморфизм** — иерархия `Item` → `Weapon`/`Armor`/`Consumable`.
- **Виртуальные методы и `override/final`**.
- **Виртуальный деструктор** для базы.
- **Чистые виртуальные функции** (`= 0`).
- `= delete` для запрета копирования.
- **Smart pointers** — `std::unique_ptr<Item>` в инвентаре.
- **Move-семантика** — `std::move` для передачи владения.
- **Шаблоны** — `EventBus<T>`, `make_unique<T>`.
- **Variadic templates + perfect forwarding** в `make_unique`.
- **Лямбды** с захватами в `CommandRegistry`.
- `std::function` для `type-erased` обработчиков.
- **Исключения** — `throw/try/catch`, своя иерархия.
- **Strong exception guarantee** в `save::read`.
- **Парсинг текстовых файлов** через `istreamstream`.
- **Сериализация объектов** в файл и обратно.
- **Контрольные суммы** для проверки целостности.
- `namespace` для организации кода.

Этого хватит, чтобы вы могли читать почти любой C++-проект из открытых источников. Незнакомые куски будут — но «слов» в вашем словаре хватает.

26.8 Что осталось «за кадром»

Игру можно расширять. Идеи как **самостоятельные упражнения**:

26.8.1 Бой

В мире нет врагов. Добавьте: - Класс Enemy с hp, damage, name. - Команда attack <name> — игрок и враг наносят урон друг другу. Если враг умер — выпадает loot. Если игрок умер — game over. - В data/world.txt поддерживайте enemy <name> <hp> <damage> в локации.

26.8.2 NPC и диалоги

- Класс NPC с именем и репликами.
- Команда talk <npc> — выводит реплику.
- Расширенные диалоги через «деревья» (ID реплики → варианты ответов).

26.8.3 Экипировка

- В Player поля equipped_weapon, equipped_armor (как сырые указатели, наблюдают за предметами в инвентаре).
- Команды wield <item>, wear <item>.
- Урон/защита влияют на бой.

26.8.4 Квесты

- Класс Quest с условием и наградой.
- NPC даёт квест («принеси меч»).
- Игрок выполняет — получает награду.

26.8.5 Множество персонажей

- Возможность создать нескольких игроков, сохранения у каждого свои.

26.8.6 События с EventBus

- EventBus<DamageEvent> для логов всего урона.
- EventBus<ItemPickedEvent> для системы ачивок.
- Подписки регистрируются в начале игры.

26.8.7 Графика (terminal)

- ncurses на macOS/Linux, PDCurses на Windows — псевдо-окна и цвета.
- Карта мира в виде ascii-таблицы.
- Цвета по типам предметов.

26.8.8 Графика (графическая)

- SDL2 или SFML — спрайты, мышь, звук.
- Это выходит из «консольной книги», но возможно.

26.8.9 Сетевая игра

- Несколько игроков в одном мире через TCP.
- Конечно, после Части V (TCP-чат) у вас будут навыки.

Я не реализую эти расширения в книге — слишком много. Но дизайн позволяет добавлять, не ломая существующее. Это и есть **расширяемая архитектура**.

26.9 Если бы я писал это профессионально

Если этот RPG-движок надо было довести до production-качества:

1. **CMake** вместо Makefile.
2. **GoogleTest** или **Catch2** для тестов.
3. **GitHub Actions** для CI: проверка сборки на Linux/macOS/Windows, запуск тестов.
4. **clang-format** для единого стиля кода.
5. **clang-tidy** для статического анализа.
6. **AddressSanitizer + UBSanitizer** в режиме asan в Makefile.
7. **fuzzing** для парсеров (libFuzzer, AFL) — автоматическая подача мусорных строк, чтобы найти крэши.
8. **Coverage** — какой процент кода покрыт тестами.
9. **Документация** — Doxygen для API.
10. **Логирование** — spdlog или своё.

Эти штуки делают код **серьёзным**, но требуют дополнительной инфраструктуры. Для учебного — мы сделали разумный минимум.

26.10 Итог Части II

С главы 15 (дизайн) до главы 25 (финал) мы построили рабочую программу. Не «hello world», не «упражнение из учебника», а **проект, в который не стыдно показать собеседнику**. С тестами, документацией в коде, разделением модулей, обработкой ошибок, версионированием формата сохранений.

Если вы шли по главам последовательно, к этому моменту у вас должно быть ощущение «я могу написать C++-программу». Не «зазубрил синтаксис», а **могу разложить задачу, выбрать**

структуру данных, написать класс с правильной семантикой, обработать ошибки. Это и есть **навык**.

Дальше — три новых проекта в Частях III–V и обзорная Часть VI. Каждый проект — отдельная область C++:

- **mini-shell** (Часть III) — POSIX API: процессы, файловые дескрипторы, сигналы.
- **мини-СУБД** (Часть IV) — низкоуровневая работа с файлами, бинарные форматы, B+tree.
- **TCP-чат** (Часть V) — сокет, многопоточность, реактор-цикл.
- **C++17 как бонус** (Часть VI) — optional, variant, filesystem, string_view.

Каждая Часть — отдельный демо-проект в своей папке demo-shell/, demo-db/, demo-chat/. Структура и стиль будут похожи на RPG: include/, src/, tests/, Makefile.

26.11 Главные правила главы

1. **Юнит-тесты — первая линия защиты.** Не покрытие важно, а возможность быстро ловить регрессии.
2. **Каждый класс — отдельный тест-файл.** Не сваливайте всё в один.
3. **Roundtrip-тесты** (запиши + прочти + сравни) — золото для сериализации.
4. **CI запускает тесты автоматически.** Без этого тесты живут только пока их вспомнят.
5. **CMake для cross-platform.** Makefile — для учёбы.
6. **AddressSanitizer + UBSanitizer** в дебажной сборке.
7. **Расширяемая архитектура** = добавление фич не ломает существующее.
8. **Не зазубривайте, понимайте.** Цель — навык, а не словарь.

26.12 Маленькое упражнение

1. Запустите `make tests && ./build/tests/test_units`. Все 37 прошли?
2. Поломайте одну строку в `Player::heal` (например, уберите ограничение по `max_hp`). Запустите тесты — увидите fail.
3. Добавьте тест для команды `cmd_take`: создайте `Game`, имитируйте ввод (это сложно для CLI; можно через `Game::cmd_take(...)` напрямую через friend или public метод для теста).
4. Установите CMake (`brew install cmake` или `apt install cmake`). Напишите `CMakeLists.txt` по примеру выше. Соберите через `cmake -B build && cmake --build build`.
5. Реализуйте класс `Enemy` (имя, hp, damage). Добавьте поддержку в `world.txt`: `enemy гоблин 10 3` — кладёт врага в локацию. В `cmd_look` показывайте врагов.

6. Реализуйте команду `attack <enemy>` — пошаговый бой. Используйте `dynamic_cast` для определения «оружия» в инвентаре игрока.
7. (Сложнее) Добавьте `EventBus<DamageEvent>` в `Game`. Подпишите логгер в `register_commands`. В команде `attack` публикуйте события — увидите, как декаплинг работает.
8. (Сложнее) Прогоните игру под **AddressSanitizer**:

```
make CXXFLAGS="-std=c++11 -Wall -fsanitize=address -g"
```

Если ASan находит проблему — это серьёзный сигнал. Чистая программа не должна ничего давать.

26.13 Что дальше

Часть III — mini-shell demo-shell/ (главы 26–31): свой `bash`-подобный `shell` с пайпами, редиректами, `builtins`, историей. Это другая половина C++: системное программирование через POSIX API. Узнаем `fork`, `exec`, `wait`, `pipe`, `dup2`, сигналы, `terminal raw mode`.

Затем — **Часть IV (главы 32–39)**: мини-СУБД с `B+tree`, страничным файлом, WAL, мини-SQL.

Часть V (главы 40–47) — TCP-чат с реактор-циклом, `atomic`, многопоточностью.

Часть VI (главы 48–51) — C++17 как бонус.

Всё это — впереди. До встречи в Части III.

Глава 27

Глава 26. Процессы — fork, exec, wait

Начинается **Часть III — mini-shell**. Шесть глав, в которых построим свою bash-подобную оболочку. Это другая половина C++: не «прикладной код через классы и STL», а **системное программирование** через POSIX API. Никаких новых языковых средств — но новый мир системных вызовов.

Эта глава — про **процессы**. Что они такое (повтор из главы 1 с подробностями), как их создавать через fork, как заменять их образ через exec, как ждать завершения через wait. К концу главы у нас работает **минимальный shell**: вводите `ls /tmp`, программа запускает `ls`, ждёт, печатает результат.

27.1 Зачем shell

Shell (по-русски «оболочка») — это программа, которая: 1. Печатает приглашение (\$). 2. Читает строку от пользователя. 3. Запускает указанную команду. 4. Ждёт завершения. 5. Печатает результат и возвращается к шагу 1.

В Unix вы каждый день пользуетесь shell — `bash`, `zsh`, `fish`. Они мощные, с автодополнением, историей, скриптами. Мы пишем простой, но настоящий.

Зачем это в книге про C++? Несколько причин:

1. **Учим POSIX API.** Системные вызовы `fork`, `exec`, `wait`, `pipe`, `dup2`, `signal` — это фундамент Unix. Понимая их, вы понимаете, как работает любой Linux/macOS-софт.
2. **Близко к ОС.** Мы выходим из «прикладного слоя» (RPG-игра, нажимаем кнопки) в системный (работаем с ядром).
3. **Полезный навык.** Часто нужно «запустить внешнюю программу из моей» — серверы, билд-системы, тестовые runner'ы. Это всё `fork+exec`.

4. **C++ может работать на низком уровне.** Не только Boost и шаблоны, но и сырые системные вызовы.

27.2 Где не сработает

POSIX — это Unix-стандарт. Linux и macOS его поддерживают. **Нативный Windows — нет.** На Windows другой API: `CreateProcess`, `WaitForSingleObject`. Идея похожа, синтаксис другой.

Если вы на Windows — поставьте WSL2 (глава 6) и работайте оттуда. Всё, что я буду писать, будет работать. На нативном `cmd.exe` — не запустится.

27.3 Что такое процесс — снова

В главе 1 я уже говорил, что процесс — это «запущенная программа». Расширим.

Каждому процессу ОС выделяет:

- **PID** (process ID) — уникальный номер.
- **Адресное пространство** — виртуальная память: код, данные, стек, куча. Изолировано от других процессов.
- **Открытые файловые дескрипторы** — обычно как минимум три: `stdin` (0), `stdout` (1), `stderr` (2).
- **Текущая рабочая папка** — где он «находится» в файловой системе.
- **Переменные окружения** — наследуемые от родителя.
- **Идентификатор владельца** (UID).
- **Идентификатор родительского процесса** (PPID).

Процессы связаны в **дерево**. У каждого процесса есть родитель. У корня всего дерева (PID 1, обычно `init` или `systemd`) родителя нет. Когда вы запускаете программу из терминала, ваш `bash` — родитель этой программы.

```
PID 1: systemd
├─ PID 123: sshd
│   └─ PID 456: bash    (терминал на котором вы)
│       └─ PID 789: ls   (запустили ls — bash его родитель)
```

Когда процесс завершается, его статус возвращается **родителю**. Если родитель не забрал — процесс становится **зомби** (об этом ниже).

27.4 Системный вызов fork

В Unix есть `fork()` — функция, **дублирующая** текущий процесс. После `fork()` есть **два** одинаковых процесса: - **Родитель**: продолжает работать как раньше. - **Ребёнок**: копия родителя в момент `fork`.

Оба возвращаются из `fork()` с разным значением: - В родителе `fork()` возвращает **PID ребёнка** (положительное число). - В ребёнке `fork()` возвращает **0**. - При ошибке возвращает **-1**.

Это позволяет программе понять, где она сейчас:

```
#include <unistd.h>
#include <iostream>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        std::cerr << "fork failed\n";
        return 1;
    }

    if (pid == 0) {
        std::cout << "Я ребёнок, PID = " << getpid() << "\n";
    } else {
        std::cout << "Я родитель, PID ребёнка = " << pid << "\n";
    }

    return 0;
}
```

Запустите — увидите две строки. Они выводятся в **одной** программе, но в **двух процессах**.

`pid_t` — тип для PID, обычно `int`. Объявлен в `<sys/types.h>` (но `<unistd.h>` его подтягивает).

27.4.1 Что значит «копия»

`fork()` копирует **всё**: содержимое памяти, стек, открытые файлы. Сразу после `fork` оба процесса абсолютно одинаковы, кроме одной вещи — значения, возвращаемого `fork()`.

Полная копия процесса с 100 MB памяти — это дорого. Поэтому современные ОС используют **copy-on-write**: реальная копия не делается; ядро помечает страницы памяти как «общие,

копировать при попытке записи». Если оба процесса только читают — данные общие. Если кто-то пишет — страница копируется на лету.

Это значит, что `fork()` сам по себе **быстрый**, даже на больших процессах.

27.4.2 Что наследуется от родителя

- Память: всё содержимое.
- Открытые файловые дескрипторы.
- Текущая папка.
- Переменные окружения.
- Маски сигналов (что игнорировать).

Что **не** наследуется (вернее, обнуляется/новое): - PID (у ребёнка свой). - Время CPU (счётчик 0). - Pending signals (ожидające сигналы родителя — не передаются). - Замки на файлы (`flock`).

27.4.3 Где работа после fork разделяется

Сразу после `fork` процессы абсолютно одинаковы. Дальше каждый идёт своей дорогой:
- Родитель обычно **продолжает свою основную работу**. - Ребёнок обычно **делает специальную задачу** и завершается.

В нашем shell: - Родитель ждёт завершения ребёнка. - Ребёнок вызывает `exec` чтобы стать другой программой.

27.5 exec — заменить образ

`exec` сам по себе **не функция**, а семейство функций: `execv`, `execvp`, `execvre`, `execl`, `execle`, `execrp`. Они различаются способом передачи аргументов и поиска программы.

Мы используем **`execvp`** (`v` = vector аргументов, `p` = искать в `$PATH`):

```
#include <unistd.h>

char* argv[] = {"ls", "-l", "/tmp", nullptr};
execvp("ls", argv);
// если выполнилось — мы НЕ здесь, мы стали `ls`
```

`execvp(name, argv)` делает следующее: 1. Ищет программу `name` в `$PATH` (если есть слеш — ищет по этому пути напрямую). 2. **Загружает** её в текущий процесс, **заменяя** код, данные, стек. 3. Запускает с `main(argc, argv)`.

После `execvp` сама программа уже **другая**. Ваш C++-код не выполняется дальше. Это как «магически перевоплотиться».

Если `execvp` **вернулась** — значит, что-то не получилось (программу не нашли, прав нет, недостаточно памяти). Тогда обычно вызывают `_exit(N)` чтобы немедленно завершить ребёнка.

27.5.1 `_exit` vs `exit`

- `exit(N)` — стандартная C-функция: вызывает все деструкторы `global`-объектов, `atexit`-обработчики, потом завершает.
- `_exit(N)` — POSIX: завершает **немедленно**, без деструкторов и `cleanup`.

В ребёнке после неудачного `exec` лучше **`_exit`**: ребёнок — это копия родителя, его `global`-объекты ещё «работают» в родителе. Если ребёнок начнёт их разрушать — будет хаос.

Возврат 127 — конвенция: «команда не найдена».

27.5.2 Каноническая пара `fork` + `exec`

```
pid_t pid = fork();
if (pid < 0) {
    // ошибка
} else if (pid == 0) {
    // ребёнок
    execvp("ls", argv);
    _exit(127);    // exec failed
} else {
    // родитель
    // ...
}
```

Это **базовый шаблон Unix**. Один процесс хочет запустить другой? `Fork` (создаём пустую заготовку), `exec` (превращаем в нужную программу), `wait` (ждём результат).

Винда делает то же одним `CreateProcess(...)`. Идея та же, синтаксис другой.

27.6 `wait` и `waitpid` — собрать статус

Когда ребёнок завершается, его статус не теряется — он **ждёт**, пока родитель его заберёт. Если родитель не забирает — ребёнок остаётся **зомби** (записью в таблице процессов, занимающей PID и память для `status`).

Чтобы забрать статус, родитель вызывает **wait** или **waitpid**:

```
#include <sys/wait.h>
```

```
int status;  
pid_t finished = waitpid(pid, &status, 0);
```

- `waitpid(pid, ...)` — ждать конкретного ребёнка.
- `waitpid(-1, ...)` — ждать любого.
- `waitpid(0, ...)` — ждать любого в той же группе процессов.

Третий аргумент — флаги. `0` — блокировать до завершения. `WNOHANG` — не блокировать, проверить и вернуться.

Возвращает PID завершившегося (или -1 при ошибке, 0 если `WNOHANG` и никто не готов).

В `status` — кодированный статус. Распаковывается макросами:

- `WIFEXITED(status)` — true, если процесс завершился нормально (через `return` из `main` или `exit()`).
- `WEXITSTATUS(status)` — код возврата (если `WIFEXITED`).
- `WIFSIGNALED(status)` — true, если завершился из-за сигнала (например, `SIGSEGV`, `SIGKILL`).
- `WTERMSIG(status)` — номер сигнала (если `WIFSIGNALED`).
- `WIFSTOPPED(status)` — true, если приостановлен (через `SIGSTOP`).

Пример:

```
int status;  
waitpid(pid, &status, 0);  
  
if (WIFEXITED(status)) {  
    int code = WEXITSTATUS(status);  
    std::cout << "exit " << code << "\n";  
} else if (WIFSIGNALED(status)) {  
    std::cout << "killed by signal " << WTERMSIG(status) << "\n";  
}
```

27.7 Зомби и orphans

Два особых состояния процесса.

Зомби (zombie): процесс умер, но родитель не вызвал `wait`. Процесс присутствует в таблице, занимает PID и небольшой кусок памяти (для статуса). В `ps` помечается как `Z` или `<defunct>`.

Если зомби много — кончатся PID-ы (на Linux обычно лимит 32768). Поэтому каждый родитель **обязан** забирать статус всех своих детей.

Орфан (orphan): ребёнок жив, родитель умер раньше. Ребёнок «усыновляется» PID 1 (init/systemd). Тот забирает статус и не оставляет зомби.

В нашем shell зомби не появятся: на каждый fork есть waitpid.

27.8 Минимальный shell

Соединим всё.

include/exec_runner.h:

```
namespace shell {

int run_external(const std::vector<std::string>& argv);
std::vector<std::string> tokenize(const std::string& line);

} // namespace shell
```

src/exec_runner.cpp:

```
#include "exec_runner.h"

#include <cerrno>
#include <cstring>
#include <iostream>
#include <sstream>
#include <vector>

#include <sys/wait.h>
#include <unistd.h>

namespace shell {

std::vector<std::string> tokenize(const std::string& line) {
    std::vector<std::string> result;
    std::istringstream ss(line);
    std::string tok;
    while (ss >> tok) {
        result.push_back(tok);
    }
}
```

```
}
return result;
}

int run_external(const std::vector<std::string>& argv) {
    if (argv.empty()) return 0;

    std::vector<char*> c_argv;
    c_argv.reserve(argv.size() + 1);
    for (const auto& s : argv) {
        c_argv.push_back(const_cast<char*>(s.c_str()));
    }
    c_argv.push_back(nullptr);

    pid_t pid = fork();
    if (pid < 0) {
        std::cerr << "fork failed: " << std::strerror(errno) << "\n";
        return -1;
    }

    if (pid == 0) {
        execvp(c_argv[0], c_argv.data());
        std::cerr << "exec '" << argv[0] << "' failed: "
                    << std::strerror(errno) << "\n";
        _exit(127);
    }

    int status = 0;
    if (waitpid(pid, &status, 0) < 0) {
        std::cerr << "waitpid failed: " << std::strerror(errno) << "\n";
        return -1;
    }

    if (WIFEXITED(status)) return WEXITSTATUS(status);
    if (WIFSIGNALED(status)) {
        std::cerr << "[процесс прерван сигналом " << WTERMSIG(status) << "]\n";
        return 128 + WTERMSIG(status);
    }
}
```

```

    return -1;
}

} // namespace shell

```

Разбор.

tokenize разбивает строку по пробелам через `istringstream` (это глава 12). Не учитывает кавычки и `escape` — добавим в главе 29.

c_argv — массив `char*`, чтобы передать в `execvp`. Используем `const_cast<char*>(s.c_str())` — `execvp` объявлена для `char*`, не `const char*` (исторически — API из C 1970-х). На самом деле программа не меняет аргументы; `const_cast` тут безопасен.

`c_argv.push_back(nullptr)` — обязательно. `execvp` ожидает массив, **оканчивающийся `nullptr`**.

fork() — создаёт ребёнка. Проверяем три случая: ошибка (`< 0`), ребёнок (`== 0`), родитель (`> 0`).

В ребёнке — `execvp(c_argv[0], c_argv.data())`. Если вернулась — `exec` не сработал. Печатаем причину через `strerror(errno)` (это глобальная ошибка из C), и завершаем `_exit(127)`.

В родителе — `waitpid` блокирует до завершения ребёнка. Далее расшифровываем статус.

`128 + WTERMSIG(status)` — конвенция `bash`: сигнал 9 (SIGKILL) даёт код 137, сигнал 11 (SIGSEGV) — 139. Так пользователь различает «нормальное завершение с кодом X» и «убит сигналом Y».

27.8.1 main и REPL

src/main.cpp:

```

#include "exec_runner.h"
#include <iostream>
#include <string>

int main() {
    std::cout << "=== myshell ===\n";

    while (true) {
        std::cout << "myshell$ ";
        std::cout.flush();

        std::string line;
        if (!std::getline(std::cin, line)) {
            std::cout << "\n";

```

```
        break;
    }
    if (line.empty()) continue;

    auto tokens = shell::tokenize(line);
    if (tokens.empty()) continue;

    if (tokens[0] == "exit" || tokens[0] == "quit") break;

    int rc = shell::run_external(tokens);
    if (rc != 0) {
        std::cerr << "[exit " << rc << "]\n";
    }
}
return 0;
}
```

REPL (read-eval-print loop): 1. Приглашение и флаш. 2. Чтение строки. 3. Если пустая — повтор. 4. Если exit/quit — выход. 5. Иначе — запуск.

`std::cout.flush()` — обязательно, иначе приглашение зависнет в буфере. Без явного flush текст появляется только после переноса строки.

27.9 Сборка и проверка

```
$ cd demo-shell
$ make
g++ ... build/main.o build/exec_runner.o -o build/myshell

$ ./build/myshell
=== myshell ===
myshell$ echo hello world
hello world
myshell$ pwd
/Users/serik/Desktop/cpp-from-scratch/demo-shell
myshell$ ls Makefile
Makefile
myshell$ ls nonexistent
```



```
ls: nonexistent: No such file or directory
[exit 1]
myshell$ python3 -c "print(2+2)"
4
myshell$ exit
```

Работает. Запускает любую программу, видимую в `$PATH`. Печатает `exit`-код, если ненулевой.

Что **не работает** пока: - Пайпы `cmd1 | cmd2`. - Редиректы `> out.txt`. - Builtin-команды (наш `cd` будет менять папку только в ребёнке — а ребёнок умирает; нужны `builtins`). - Сигналы (`Ctrl+C` должно прерывать ребёнка, не `shell`). - История команд (стрелка вверх).

Всё это — следующие главы.

27.10 Что происходит под капотом

Сценарий «вы набрали `ls /tmp` в моём `shell`»:

1. `getline(std::cin, line)` — `shell` читает строку.
2. `tokenize` — `["ls", "/tmp"]`.
3. `run_external` вызывается с этим вектором.
4. `fork()` — ядро создаёт ребёнка. Сейчас у нас два процесса с одинаковой памятью.
5. **В ребёнке** — `execvp("ls", {"ls", "/tmp", nullptr})`. Ядро ищет `/bin/ls`, читает его исполняемый файл, **заменяет** код и данные процесса. Старая копия `myshell` в ребёнке **перестает существовать**. Дальше работает `ls`.
6. `ls` выполняется, печатает содержимое `/tmp` в свой `stdout`, который **наследовался от `shell`** и направлен в терминал.
7. `ls` **завершается** через `return 0` или `exit(0)`. Ядро записывает статус.
8. **В родителе** — `waitpid` пробуждается. Получает PID и статус.
9. `WIFEXITED(status) = true, WEXITSTATUS(status) = 0`.
10. `run_external` возвращает 0.
11. `main` идёт на следующую итерацию.

Десять шагов на одну команду. Каждый шаг — конкретный системный вызов или часть кода. Никакой магии.

27.11 Тонкости

27.11.1 Почему стандартные потоки наследуются

Файловые дескрипторы (включая stdin/stdout/stderr) **наследуются** при fork. После exec они **сохраняются** (если не отмечены флагом FD_CLOEXEC). Поэтому ls, запущенный из shell, пишет в тот же терминал. Если бы exec обнулil дескрипторы — ls ничего бы не вывел.

В главе 27 мы используем это поведение для пайпов: dup2 подменит fd 1 (stdout) ребёнка на пайп, и вывод пойдёт в следующую команду.

27.11.2 Среда (environment)

При fork копируются переменные окружения. execvp тоже их сохраняет (использует свою копию). Поэтому если в shell задана LANG=ru_RU.UTF-8 — у запущенной программы тоже она будет.

Изменить — setenv("KEY", "value", 1) перед exec. Это редко нужно, но иногда полезно («запусти эту программу с переменной A=1»).

27.11.3 Прерывание ребёнка

В нашем shell, если ребёнок зависает (например, cat без stdin), вы не можете его остановить. Ctrl+C посылает SIGINT — но **в нашу группу процессов**, включая родителя. Родитель тоже падает.

В настоящем shell сигналы разруливают так: shell в **своей** группе процессов, ребёнок в **отдельной**. Ctrl+C идёт ребёнку, родитель ловит и **игнорирует**.

Это глава 28 (сигналы).

27.11.4 Что с zsh / bash подобными фидами

Наш shell — **очень** простой: - Нет переменных (x=5; echo \$x). - Нет history. - Нет автодополнения. - Нет alias. - Нет встроенных команд (cd, export). - Нет skripts (bash script.sh).

Это всё надстройки. Базовый цикл fork+exec+wait у настоящего bash и у нашего myshell — одинаковый.

27.12 fork без exec

fork без exec — это **просто параллельный код**. Можно так:

```
pid_t pid = fork();
if (pid == 0) {
    // ребёнок — выполняет свою работу
    do_heavy_calculation();
    _exit(0);
}
// родитель — делает свою работу одновременно
do_other_work();
waitpid(pid, nullptr, 0);
```

Это **примитивный параллелизм**. Раньше так делали серверы — отдельный процесс на клиента. Сейчас обычно потоки (`std::thread`), потому что они дешевле. Но `fork` без `exec` тоже используется — например, в **prefork** моделях серверов (`apache`, `php-fpm`).

27.13 Главные правила главы

1. **fork дублирует процесс, exec заменяет образ.** Стандартный паттерн запуска внешней команды.
2. **В ребёнке после exec НЕ должно быть кода**, кроме обработки ошибки и `_exit`.
3. **Каждый fork требует waitpid** — иначе зомби.
4. **_exit в ребёнке** на error path, не `exit` — чтобы не запустить деструкторы родителя.
5. **POSIX = Linux/macOS. Windows = WSL2.**
6. **argv массив для exec кончается nullptr.** Забыли — UB.
7. **fflush или endl перед чтением ввода.** Иначе приглашение зависнет.
8. **Сигналы и Ctrl+C** — отдельная тема, глава 28.

27.14 Маленькое упражнение

1. Соберите `demo-shell` (`cd demo-shell && make`). Запустите. Поэкспериментируйте: `ls`, `pwd`, `date`, `whoami`, `echo hello`.
2. Попробуйте запустить несуществующую команду. Что увидите? Какой `exit`-код?
3. Запустите `sleep 10`. Что происходит? Можно ли прервать через `Ctrl+C`? Что с `shell` после?
4. Запустите `python3 -c "import sys; sys.exit(42)"`. Какой код вернётся в `[exit ...]`?
5. Сделайте программу из 10 строк: `fork`, в ребёнке `std::cout << "child"`, в родителе `std::cout << "parent"`, `waitpid`. Запустите. Какой порядок вывода? Стабильный или нет?

6. (Сложнее) Сделайте программу: fork дважды, получите трёх процессов (родитель + 2 ребёнка). Каждый печатает свой PID. Родитель ждёт обоих через два `waitpid(-1, ...)`. Запустите. Объясните, почему PID-ы каждый раз разные.
7. (Сложнее) Прочитайте `man 2 fork` и `man 3 execv`. Узнайте про различия `execv`, `execvp`, `execvpe`, `execcl`, `execle`. В каких случаях каждая лучше?

27.15 Что дальше

Глава 27 — **пайпы и редиректы**. `pipe()`, `dup2()`, и как `cmd1 | cmd2` работает на самом деле. Это самая интересная часть Unix — почему «маленькие программы, делающие одно дело» можно собирать в мощные цепочки.

Затем — сигналы (28), парсер командной строки с кавычками и редиректами (29), builtin-команды и история (30), свои утилиты `cat/wc/grep` (31). После этого Часть III закрывается, и мы переходим к Части IV — мини-СУБД.

Глава 28

Глава 27. Файловые дескрипторы и pipes

В прошлой главе мы научились запускать одну программу через `fork+exec+wait`. В этой — соединим программы в **пайплайн**: `ls | grep .cpp | wc -l`. Это самая красивая и мощная идея Unix: маленькие программы, каждая делающая одно дело, соединяются в цепочку через стандартный механизм.

Я долго подходил к этой главе — пайпы концептуально просты, но их реализация требует **дисциплины** в работе с файловыми дескрипторами. Один забытый `close` — и пайплайн зависнет навсегда. К концу главы вы напишете правильную реализацию, которая работает для произвольной длины цепочки.

28.1 Что такое файловый дескриптор

В Unix **всё — файл**. Точнее, всё доступно через единый интерфейс «открыть/прочитать/записать/закрыть». Дисковый файл, терминал, сетевое соединение, пайп, даже устройство (`/dev/null`, `/dev/random`) — всё через одну абстракцию.

Чтобы программа работала с файлом, ядро даёт ей **файловый дескриптор** (`fd`) — небольшое целое число, указатель на запись во внутренней таблице ядра.

У каждого процесса своя таблица `fd`. По умолчанию первые три занимают:

fd	Имя	Куда
0	stdin	Ввод
1	stdout	Вывод
2	stderr	Ошибки

Когда вы запустили программу из терминала, эти три fd связаны с терминалом. Программа пишет в fd 1 — текст появляется в окне.

Когда вы запустили через `./program > out.txt` — shell перенаправил fd 1 на файл `out.txt`. Программа сама не знает, что её stdout куда-то перенаправлен.

28.1.1 Открыть файл

```
#include <fcntl.h>
#include <unistd.h>

int fd = open("data.txt", O_RDONLY);
char buf[100];
ssize_t n = read(fd, buf, sizeof(buf));
close(fd);
```

`open` возвращает наименьший свободный fd. Обычно начинает с 3 (после 0/1/2).

`read(fd, buf, n)` читает до n байтов. Возвращает реально прочитанное число, 0 при EOF, -1 при ошибке.

`write(fd, buf, n)` — наоборот.

`close(fd)` — освобождает запись в таблице. После — fd снова свободен для нового `open`.

В C++ обычно пользуются `<fstream>` (глава 12), а не сырыми вызовами. Но для пайпов придётся работать на низком уровне.

28.2 fd наследуются через fork и exec

Главное: **при fork ВСЕ открытые fd дублируются в ребёнка**. Они указывают на ту же запись в ядре, что и у родителя. То есть если родитель писал в fd 5, и ребёнок пишет в fd 5 — оба пишут в **одно и то же** место.

При `exec` fd **сохраняются** (если на них не выставлен флаг `FD_CLOEXEC`). Поэтому новая программа в ребёнке видит те же 0/1/2 (и любые другие открытые).

Это и есть основа **пайпов и редиректов**. Shell открывает пайп **до fork**, дальше ребёнок наследует fd, и его stdout (или stdin) подменяется на пайп через `dup2`.

28.3 pipe() — создать пайп

pipe(fds) — системный вызов, создающий **пайп**. На входе — массив из двух int. На выходе: - fds[0] — fd для **чтения** из пайпа. - fds[1] — fd для **записи** в пайп.

Пайп — это буфер в ядре. Что записывается в fds[1], можно прочитать через fds[0]. Размер буфера обычно 64 KB.

```
#include <unistd.h>

int fds[2];
if (pipe(fds) < 0) {
    perror("pipe");
    return 1;
}

write(fds[1], "hello", 5);

char buf[10];
ssize_t n = read(fds[0], buf, sizeof(buf));
// buf = "hello", n = 5

close(fds[0]);
close(fds[1]);
```

В одном процессе пайп бесполезен. Но если **сначала pipe**, потом **fork** — у ребёнка тоже будут эти fd, и можно общаться:

```
int fds[2];
pipe(fds);

pid_t pid = fork();
if (pid == 0) {
    // Ребёнок пишет:
    close(fds[0]); // нам не нужен read end
    write(fds[1], "hi parent", 9);
    close(fds[1]);
    _exit(0);
} else {
    // Родитель читает:
    close(fds[1]); // нам не нужен write end
```

```
char buf[20];
ssize_t n = read(fds[0], buf, sizeof(buf));
std::cout << "received: " << std::string(buf, n) << "\n";
close(fds[0]);
waitpid(pid, nullptr, 0);
}
```

Этот код печатает «received: hi parent».

Зачем оба процесса закрывают **один** конец? Каждый fd — это **счётчик ссылок** в ядре. Пайп удалится только когда ВСЕ ссылки закрыты. Если ребёнок не закроет `fds[0]` — у пайпа двое читателей; неаккуратно.

28.3.1 EOF на пайпе

Когда **все** ссылки на write-конец закрыты, читатель получает **EOF** (read возвращает 0). Это сигнал «больше данных не будет».

Если хоть один write-конец открыт — read **блокируется**, ожидая данных. **Это самая частая причина зависшего пайплайна**: какой-то процесс забыл закрыть write-конец, и пайп никогда не сигналил EOF.

28.4 dup2 — перенаправить fd

Файловый дескриптор — это просто номер в таблице. Можно **переназначить** этот номер на другую запись через `dup2`:

```
dup2(src, dst);
```

После этого `dst` указывает на то же, на что `src`. Старая запись по `dst` закрывается (если была открыта).

Это критический инструмент. Пример: перенаправить `stdout` в файл.

```
int file = open("out.txt", O_WRONLY | O_CREAT, 0644);
dup2(file, STDOUT_FILENO); // теперь fd 1 = файл
close(file);                // оригинальный fd больше не нужен

write(1, "hello\n", 6);      // пишется в out.txt, не в терминал
// или через std::cout — то же самое, cout пишет в fd 1
std::cout << "world\n";
```


STDOUT_FILENO — макрос для 1. Аналогично STDIN_FILENO (0), STDERR_FILENO (2). Числа можно писать напрямую, но макросы яснее.

После `dup2(file, 1)`: - fd 1 теперь связан с `out.txt`. - Оригинальный `file` (например, 3) тоже связан с тем же. - Программа может писать в любой из них — попадёт в файл. - `close(file)` отзывает запись fd 3. Запись fd 1 остаётся.

Это **основа** редиректов. Shell перед `exec` ребёнка делает `dup2(file, STDOUT_FILENO)`, и ребёнок пишет в файл без своего ведома.

28.5 А | В — пайплайн на две команды

Соберём всё. Нужно реализовать `ls | grep .cpp`.

Алгоритм: 1. Создать пайп. 2. Fork первого ребёнка (`ls`): - `dup2(pipe_write, STDOUT_FILENO)` — его `stdout` идёт в пайп. - Закрывать `pipe_read` и `pipe_write` (после `dup` они не нужны). - `execvp("ls", ...)`. 3. Fork второго ребёнка (`grep`): - `dup2(pipe_read, STDIN_FILENO)` — его `stdin` идёт из пайпа. - Закрывать `pipe_read` и `pipe_write`. - `execvp("grep", ...)`. 4. Родитель: **закрывать оба конца пайпа** (важно!). Иначе `grep` не получит EOF. 5. Ждать обоих детей.

Псевдокод:

```
int fds[2];
pipe(fds);

pid_t p1 = fork();
if (p1 == 0) {
    dup2(fds[1], STDOUT_FILENO);
    close(fds[0]);
    close(fds[1]);
    execvp("ls", argv_ls);
    _exit(127);
}

pid_t p2 = fork();
if (p2 == 0) {
    dup2(fds[0], STDIN_FILENO);
    close(fds[0]);
    close(fds[1]);
    execvp("grep", argv_grep);
    _exit(127);
}
```

```

}

close(fds[0]);
close(fds[1]);

waitpid(p1, nullptr, 0);
waitpid(p2, nullptr, 0);

```

Главные тонкости:

1. **pipe до fork** — иначе fd не наследуется.
2. **В каждом ребёнке закрываем оба конца** после dup2. Один нужный уже подменён на 0 или 1, дублировать не нужно.
3. **Родитель закрывает оба конца** перед waitpid. Если оставить открытыми — grep будет ждать EOF от пайпа, который никто не закрыл.

28.6 А | В | С — длинный пайплайн

Обобщение на N команд:

- Создать **N-1 пайпов** (между каждой парой соседних команд).
- Fork N детей.
- Ребёнок i:
 - Если $i > 0$: dup2(pipes[i-1][0], STDIN_FILENO) — stdin из левого пайпа.
 - Если $i < N-1$: dup2(pipes[i][1], STDOUT_FILENO) — stdout в правый пайп.
 - Закрывать ВСЕ пайповые fd (они либо дублированы, либо чужие).
 - exec.
- Родитель: закрыть ВСЕ пайповые fd. Ждать всех детей.

```

int N = commands.size();
std::vector<std::array<int, 2>> pipes(N - 1);
for (int i = 0; i < N - 1; ++i) {
    pipe(pipes[i].data());
}

for (int i = 0; i < N; ++i) {
    pid_t pid = fork();
    if (pid == 0) {
        if (i > 0)    dup2(pipes[i - 1][0], STDIN_FILENO);
        if (i < N - 1) dup2(pipes[i][1], STDOUT_FILENO);
    }
}

```

```

        for (auto& p : pipes) {
            close(p[0]);
            close(p[1]);
        }

        execvp(...);
        _exit(127);
    }
    children.push_back(pid);
}

// родитель закрывает все
for (auto& p : pipes) {
    close(p[0]);
    close(p[1]);
}

// ждём всех
for (pid_t c : children) {
    waitpid(c, nullptr, 0);
}

```

Тот же алгоритм для двух команд, но обобщённый.

28.7 Реализация в demo-shell

28.7.1 parse_pipeline

```

Pipeline parse_pipeline(const std::string& line) {
    Pipeline p;
    auto tokens = tokenize(line);

    std::vector<std::string> current;
    for (auto& t : tokens) {
        if (t == "|") {
            if (!current.empty()) {
                p.commands.push_back(std::move(current));
                current.clear();
            }
        }
    }
    p.commands.push_back(std::move(current));
}

```

```

    }
    } else {
        current.push_back(std::move(t));
    }
}
if (!current.empty()) {
    p.commands.push_back(std::move(current));
}
return p;
}

```

Разбираем строку. Каждый `|` — разделитель команд. Остальные токены — аргументы текущей команды.

Ограничение: `|` должен быть **окружён пробелами**. `ls|grep` не сработает (наш `tokenize` это один токен). Настоящий shell умеет без пробелов; мы оставим как упражнение.

28.7.2 run_pipeline

```

int run_pipeline(const Pipeline& p) {
    if (p.empty()) return 0;
    const int n = static_cast<int>(p.commands.size());

    std::vector<std::array<int, 2>> pipes(n - 1);
    for (int i = 0; i < n - 1; ++i) {
        if (pipe(pipes[i].data()) < 0) {
            std::cerr << "pipe failed: " << std::strerror(errno) << "\n";
            close_all_pipes(pipes);
            return -1;
        }
    }

    std::vector<pid_t> children(n);
    for (int i = 0; i < n; ++i) {
        pid_t pid = fork();
        if (pid < 0) {
            std::cerr << "fork failed\n";
            close_all_pipes(pipes);
            return -1;
        }
    }
}

```

```

    }
    if (pid == 0) {
        if (i > 0) {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
        if (i < n - 1) {
            dup2(pipes[i][1], STDOUT_FILENO);
        }
        close_all_pipes(pipes);

        auto argv = to_c_argv(p.commands[i]);
        execvp(argv[0], argv.data());
        std::cerr << "exec '" << argv[0] << "' failed: "
                    << std::strerror(errno) << "\n";
        _exit(127);
    }
    children[i] = pid;
}

close_all_pipes(pipes);

int last_status = 0;
for (int i = 0; i < n; ++i) {
    int status = 0;
    waitpid(children[i], &status, 0);
    if (i == n - 1) {
        if (WIFEXITED(status)) last_status = WEXITSTATUS(status);
        else if (WIFSIGNALED(status)) last_status = 128 + WTERMSIG(status);
    }
}
return last_status;
}

```

`close_all_pipes` — утилита, проходящая по всем парам и закрывающая обе стороны:

```

void close_all_pipes(std::vector<std::array<int, 2>>& pipes) {
    for (auto& p : pipes) {
        if (p[0] >= 0) { close(p[0]); p[0] = -1; }
        if (p[1] >= 0) { close(p[1]); p[1] = -1; }
    }
}

```

```
}  
}
```

Обнуляем после закрытия — если случайно вызовется ещё раз, не закроем чужой fd.

Код возврата пайплайна — у последней команды. Это конвенция bash: `ls | grep foo` возвращает код `grep`, не `ls`. (В bash есть `pipefail` опция, меняющая это, но по умолчанию — последняя.)

28.8 Тест

```
$ make && ./build/myshell  
myshell$ ls -l src | wc -l  
3  
myshell$ ls -l | sort | head -3  
Makefile  
build  
include  
myshell$ echo hello world | tr a-z A-Z  
HELLO WORLD  
myshell$ ls | grep .cpp  
exec_runner.cpp  
main.cpp  
pipeline.cpp  
myshell$ exit
```

Работает: три команды последовательно (`ls | sort | head`), две команды (`echo | tr`, `ls | wc -l`, `ls | grep`).

28.9 Тонкости пайпов

Несколько ловушек, на которые легко напороться.

28.9.1 Зомби пайплайн

Если в ребёнке вы забыли `close_all_pipes` после `dup2` — у него осталось 2 лишних fd. После `exec` они тоже наследуются. Когда `grep` ждёт EOF — `pipe[0]` всё ещё открыт у `ls`-ребёнка (а тот скоро умрёт, но fd закроется... может, и не закроется, если что-то странное).

Более серьёзный случай: родитель забыл закрыть write-конец. У ребёнка-читателя есть открытый writer (родитель). EOF никогда не приходит. Программа висит.

Правило: после dup2 и форка — **закройте всё, что не нужно.**

28.9.2 Размер буфера

Буфер пайпа на Linux обычно 64 KB. Если писатель отправил больше, чем 64 KB, и читатель ещё не читал — write **блокирует** писателя. Поэтому пайплайн идёт **параллельно**: писатель пишет → буфер заполняется → читатель читает → буфер освобождается → писатель продолжает.

В нашем `ls | wc -l` — `ls` пишет, `wc -l` читает. Если бы `ls` производил 100 GB вывода, `wc` бы потреблял по 64 KB за раз; всё работало бы, не падая из-за памяти.

Это **streaming**. Огромное преимущество пайпов.

28.9.3 SIGPIPE

Что произойдёт, если писатель пишет в пайп, у которого **закрыт** read-конец? Например, `yес | head -1` — `head` берёт одну строку и закрывает stdin.

Ядро посылает писателю сигнал **SIGPIPE**. По умолчанию он завершает процесс. Поэтому `yес` после первого вывода завершается. Это нормально.

Если хотите ловить ошибку самостоятельно — игнорируйте SIGPIPE:

```
signal(SIGPIPE, SIG_IGN);
```

Тогда write вернёт `-1` с `errno == EPIPE`. Это глава 28 (сигналы).

28.9.4 Когда пайп — overkill

Пайп — это **межпроцессное** соединение. Тяжелее, чем «функция вызывает функцию». Если ваши «команды» — это код, который вы пишете сами, не нужно их разделять в процессы; пишите функции.

Пайп оправдан, когда: - Соединяются **разные программы** (вам не подвластные). - Нужна **изоляция** (один процесс упал — другой жив). - Хотите **параллелизм без потоков**.

В нашем shell — пайп оправдан, потому что мы соединяем чужие программы.

28.9.5 Внутри одной программы — fork+pipe

Если внутри одной программы нужен пайп между двумя кусками логики (вы пишете оба), можно тот же fork+pipe использовать без `exec`:

```
int fds[2]; pipe(fds);
pid_t pid = fork();
if (pid == 0) {
    close(fds[1]);
    // ребёнок читает из fds[0]
    char buf[1024];
    while (true) {
        ssize_t n = read(fds[0], buf, sizeof(buf));
        if (n <= 0) break;
        // обработать
    }
    _exit(0);
}
close(fds[0]);
// родитель пишет в fds[1]
write(fds[1], data, len);
close(fds[1]);
waitpid(pid, nullptr, 0);
```

Это межпроцессная очередь. Очень полезно для серверов, которые отделяют обработчики запросов в отдельные процессы.

28.10 Главные правила главы

1. **fd 0/1/2 — stdin/stdout/stderr.** Это везде в Unix.
2. **pipe ДО fork.** Иначе fd не наследуется.
3. **После dup2 закройте оригинальный fd.** Дублирующая запись больше не нужна.
4. **Закройте ВСЕ концы пайпа в каждом ребёнке** и в родителе. Один забытый — зависание.
5. **Пайп — это streaming.** Буфер 64 KB, всё параллельно.
6. **SIGPIPE** при записи в закрытый пайп. По умолчанию убивает писателя.
7. **Код возврата пайплайна — у последней команды** (bash-конвенция).
8. **fork+pipe без exec** — для своей внутренней коммуникации.

28.11 Маленькое упражнение

1. Соберите и попробуйте: `ls | wc, ls | grep .cpp | sort, echo hello | tr a-z A-Z.`

2. Попробуйте `cat << EOF | grep ...` — у нашего shell не поддерживается heredoc, но cat без аргументов читает stdin. Поведение?
3. Сломайте `close_all_pipes` — закомментируйте вызов в ребёнке. Соберите. Запустите `ls | wc`. Зависнет?
4. Попробуйте `ls /` с многими файлами в shell. Подумайте — пайп между ls и shell блокируется? Почему мы этого не видим в обычном ls?
5. (Сложнее) Добавьте поддержку | **без пробелов**: `ls|grep .cpp`. Подсказка — преобразовать токенайзер так, чтобы он распознавал | как отдельный символ-разделитель.
6. (Сложнее) Сделайте программу `ripedemo.cpp`: fork + pipe, родитель пишет 1 миллион строк, ребёнок читает и считает. Используйте время измерения через `<chrono>` (глава 13). Сколько строк в секунду?
7. (Очень сложно) Прочитайте `man 2 pipe2` — расширенная версия `pipe()`. Что такое `O_CLOEXEC` и для чего полезно при наших пайпах?

28.12 Что дальше

Глава 28 — **сигналы**. Что такое сигнал, как их посылать (`kill`), как ловить (`signal/sigaction`), что такое `async-signal-safe` функции. Реализуем Ctrl+C в нашем shell, чтобы прерывал ребёнка, не убивая shell.

Дальше — парсер с кавычками и редиректами (29), builtin-команды и история (30), свои утилиты `cat/wc/grep` (31). После этого Часть III закроется.

Глава 29

Глава 28. Сигналы

В предыдущих главах мы построили shell, который запускает программы и собирает их в пайплайны. Сейчас попробуйте `sleep 10` в shell, потом `Ctrl+C`. Что произойдёт? Скорее всего, **умрут оба** — и `sleep`, и сам shell. Это потому, что `Ctrl+C` — это **сигнал**, и он по умолчанию идёт всем процессам в группе. В этой главе разберёмся с сигналами и сделаем так, чтобы shell их игнорировал, а ребёнок — нет.

Сигналы — старейший механизм межпроцессного взаимодействия в Unix. Простой и мощный, но с подводными камнями (asynchronous-safe функции, race conditions). К концу главы вы поймёте, почему «программа со сложной логикой в обработчике» — это плохая идея, и почему `sigaction` лучше старого `signal`.

29.1 Что такое сигнал

Сигнал — это асинхронное уведомление от ядра процессу. «Тебе кто-то послал номер `N` — обработай как-нибудь».

Каждый сигнал имеет: - **Номер** (например, 2). - **Имя** (например, `SIGINT`). - **Действие по умолчанию** (например, «убить процесс»).

Когда сигнал приходит, ядро **прерывает** обычное выполнение процесса и запускает обработчик. Если обработчика нет — выполняется действие по умолчанию.

Сигнал — **асинхронный**: процесс не знает, когда он придёт. Может прийти в середине вычисления, посреди вызова `printf`, между двумя инструкциями процессора. Поэтому обработчики сигналов — очень тонкая штука.

29.2 Распространённые сигналы

Я перечислю самые часто встречающиеся.

Номер	Имя	По умолчанию	Когда
1	SIGHUP	Завершить	Терминал отключён (закрыли SSH-сессию)
2	SIGINT	Завершить	Ctrl+C в терминале
3	SIGQUIT	Завершить + core dump	Ctrl+\
9	SIGKILL	Завершить (НЕВОЗМОЖНО ПЕРЕХВАТИТЬ)	kill -9
11	SIGSEGV	Завершить + core dump	Обращение к невалидной памяти
13	SIGPIPE	Завершить	Запись в пайп без читателей
15	SIGTERM	Завершить	kill без аргумента — «вежливая просьба завершиться»
17	SIGCHLD	Игнорируется	Ребёнок изменил состояние (умер, остановлен)
19	SIGSTOP	Приостановить (НЕВОЗМОЖНО ПЕРЕХВАТИТЬ)	kill -STOP
18	SIGCONT	Продолжить остановленный	После SIGSTOP

Особый случай — **SIGKILL** и **SIGSTOP**: их **невозможно** перехватить или проигнорировать. Это страховка ОС от зависших процессов: какой-бы код не был в программе, ядро гарантированно её убьёт/остановит.

Все остальные сигналы можно: - **Игнорировать** (SIG_IGN). - **Установить свой обработчик**. - **Восстановить действие по умолчанию** (SIG_DFL).

29.3 Как послать сигнал

Команда `kill` посылает сигнал процессу:

```
$ kill -INT 1234      # послать SIGINT процессу 1234
$ kill -9 1234        # послать SIGKILL
$ kill 1234           # послать SIGTERM (15) по умолчанию
```

В C++:

```
#include <signal.h>
#include <sys/types.h>

int rc = kill(pid, SIGTERM);
```

Возвращает 0 при успехе, -1 при ошибке (нет такого `pid`, нет прав, ...).

Терминал автоматически посылает сигналы: - Ctrl+C → SIGINT всем процессам в **foreground group**. - Ctrl+\ → SIGQUIT. - Ctrl+Z → SIGTSTP (приостанавливает, можно вернуть через `fg`).

«Foreground group» — это процессная группа, которая сейчас «владеет» терминалом. Когда вы запускаете `sleep 10` в `bash`, `sleep` становится `foreground`; Ctrl+C идёт ему. В фоне (`sleep 10 &`) — Ctrl+C не доходит.

29.4 Старый API: `signal()`

`signal(SIGNUM, handler)` — установить обработчик. Простой, но **проблемный**:

```
#include <signal.h>
#include <iostream>

void handler(int sig) {
    std::cout << "got signal " << sig << "\n";
}

int main() {
    signal(SIGINT, handler);

    while (true) {
        pause();    // спать до сигнала
    }
}
```

Проблемы signal:

1. **Поведение различается** на разных Unix (BSD vs SysV). После срабатывания обработчика на SysV — signal восстанавливается в SIG_DFL; на BSD — остаётся. Современный POSIX следует BSD.
2. **Нет контроля флагов** — какие сигналы блокировать на время обработчика, рестартовать ли прерванные syscalls.
3. **Race condition** на установке — может между восстановлением SIG_DFL и повторной установкой прийти второй сигнал.

Поэтому в новом коде — sigaction.

29.5 Современный API: sigaction()

```
#include <signal.h>

void handler(int sig) {
    // ...
}

int main() {
    struct sigaction sa;
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, nullptr);
}
```

struct sigaction имеет три поля (упрощённо): - **sa_handler** — указатель на обработчик. Или SIG_IGN, или SIG_DFL. - **sa_mask** — какие сигналы блокировать на время выполнения обработчика. sigemptyset — пустая маска. - **sa_flags** — флаги (например, SA_RESTART).

sigaction детерминированный, переносимый, со всеми возможностями. Используйте его.

29.6 Что можно делать в обработчике

Главная ловушка сигналов: обработчик может **прервать что угодно**. Включая саму программу посреди printf, посреди malloc, посреди обновления связного списка.

Если обработчик вызовет `printf`, который внутри использует те же буферы, что и прерванный `printf` — **race condition**. Может быть UB, краш, испорченный вывод.

POSIX определяет **async-signal-safe функции** — функции, безопасные для вызова из обработчика. Список ограничен:

- `write`, `read`, `close`, `open` — низкоуровневый I/O.
- `_exit`, `kill`, `getpid`.
- `signal`, `sigaction`.
- Несколько других.

`printf`, `malloc`, `std::cout` — **НЕ async-signal-safe**. Использовать в обработчике — играть с огнём.

Что делают грамотные обработчики:

1. Устанавливают глобальный **флаг** (`volatile sig_atomic_t got_sigint = 1`).
2. Главная программа этот флаг проверяет и реагирует.

```
volatile sig_atomic_t got_sigint = 0;

void handler(int) {
    got_sigint = 1;    // только флаг, ничего больше
}

int main() {
    // установка...

    while (true) {
        if (got_sigint) {
            got_sigint = 0;
            std::cout << "interrupt detected\n";    // безопасно вне обработчика
        }
        // обычная работа
    }
}
```

`volatile sig_atomic_t` — тип целого числа, **атомарно** считываемого/записываемого. Стандарт C гарантирует это; в C++ — практически то же.

29.7 SA_RESTART и системные вызовы

Когда сигнал приходит во время **блокирующего системного вызова** (например, `read` ждёт ввода от пользователя), у вызова два варианта:

1. **Прерваться** — вернуть `-1` с `errno == EINTR`. Программа должна перезапустить вызов сама.
2. **Перезапуститься** — после обработки сигнала продолжить ждать.

С `signal()` (BSD) обычно перезапуск. С `sigaction()` без флагов — прерывание. С `SA_RESTART` — перезапуск.

```
sa.sa_flags = SA_RESTART;
```

Для `shell` мы хотим, чтобы `getline` не прерывался от `SIGCHLD` (ребёнок умер). Поставим `SA_RESTART`. Но для `waitpid` это другой случай — там цикл.

29.8 Игнорирование сигнала

Установка `SIG_IGN`:

```
sa.sa_handler = SIG_IGN;  
sigaction(SIGINT, &sa, nullptr);
```

Сигнал приходит, но **ничего не происходит**. Действие по умолчанию отменено, нет вызова обработчика. Просто игнор.

`SIG_IGN` сохраняется при `exec`. Это критично: если `shell` установит `SIG_IGN` для `SIGINT`, **ребёнок после `exec` тоже его игнорирует**. То есть `Ctrl+C` не будет работать на запущенные программы.

Поэтому в ребёнке мы **восстанавливаем** `SIG_DFL` перед `exec`.

29.9 SIGCHLD и зомби

Когда ребёнок умирает, родителю приходит **`SIGCHLD`**. По умолчанию игнорируется, но процесс остаётся зомби, пока родитель не сделает `waitpid`.

Способы:

1. **Синхронный `waitpid`** — как у нас. После запуска ребёнка ждём его. Зомби нет.
2. **Обработчик `SIGCHLD`:**

```
void sigchld_handler(int) {  
    while (waitpid(-1, nullptr, WNOHANG) > 0) {
```

```
// забираем всех завершившихся детей  
}  
}
```

Полезно для **фоновых задач** (cmd &). Дети умирают в произвольное время; обработчик их подбирает.

3. **signal(SIGCHLD, SIG_IGN)** — POSIX-расширение: ядро автоматически чистит за вами. Удобно, но прячет статус.

В нашем shell — синхронный waitpid. Никаких обработчиков SIGCHLD не нужно.

29.10 SIGPIPE

Запись в пайп, у которого нет читателей, приводит к **SIGPIPE**. По умолчанию — смерть процесса.

Это бывает: пайплайн `yes | head -1` — head берёт строку и закрывает stdin. yes пишет в закрытый пайп — SIGPIPE — yes умирает.

Если хотите ловить ошибку самостоятельно:

```
sa.sa_handler = SIG_IGN;  
sigaction(SIGPIPE, &sa, nullptr);
```

Теперь write в закрытый пайп вернёт `-1` с `errno == EPIPE`. Программа сможет обработать корректно.

В нашем shell игнорируем SIGPIPE — иначе одна неудачная программа в пайплайне могла бы убить shell.

29.11 Применение в demo-shell

include/signals.h:

```
namespace shell {  
  
void install_shell_signal_handlers();  
void restore_default_signal_handlers();  
  
}
```

src/signals.cpp:


```

#include "signals.h"
#include <csignal>

namespace shell {

void install_shell_signal_handlers() {
    struct sigaction sa;
    sa.sa_handler = SIG_IGN;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, nullptr);
    sigaction(SIGQUIT, &sa, nullptr);
    sigaction(SIGPIPE, &sa, nullptr);
}

void restore_default_signal_handlers() {
    struct sigaction sa;
    sa.sa_handler = SIG_DFL;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, nullptr);
    sigaction(SIGQUIT, &sa, nullptr);
    sigaction(SIGPIPE, &sa, nullptr);
}

}

```

Две функции. Первая — для shell: игнорировать SIGINT/SIGQUIT/SIGPIPE. Вторая — для ребёнка после fork, перед exec: вернуть всё в дефолт.

В main.cpp:

```

int main() {
    shell::install_shell_signal_handlers();
    // ... REPL ...
}

```

В pipeline.cpp, в ребёнке после fork:

```
if (pid == 0) {  
    // Ребёнок i.  
    restore_default_signal_handlers();  
    // ... dup2, exec ...  
}
```

29.12 Что в итоге

Сценарий: пользователь набирает `sleep 10`, нажимает Ctrl+C.

1. Терминал ловит Ctrl+C, посылает SIGINT всем процессам в **foreground group**. Это сейчас shell + ребёнок sleep.
2. **Shell**: SIGINT игнорируется (мы поставили SIG_IGN). Продолжает ждать waitpid.
3. **Sleep**: SIGINT с действием по умолчанию (мы восстановили SIG_DFL). Sleep умирает.
4. Shell waitpid пробуждается, видит WIFSIGNALED(status) с сигналом 2 (SIGINT).
5. Shell печатает «процесс прерван сигналом 2», возвращает 130 (= 128 + 2) как exit code.
6. REPL продолжается.

То, что нужно. Shell живой, sleep умер, пользователь видит сообщение.

Без наших правок: 1. SIGINT приходит и в shell, и в sleep. 2. Оба используют action по умолчанию (SIGINT убивает). 3. Оба умирают. Терминал возвращается к bash.

Ctrl+\, кстати, делает то же самое, но шумнее — он дампит core file (с настройками по умолчанию). Поэтому мы игнорируем и SIGQUIT.

29.13 Тонкости и углы

29.13.1 Заранее заблокированные сигналы

sigprocmask блокирует доставку сигналов. Они остаются в **очереди** и приходят, когда разблокируешь:

```
sigset_t mask;  
sigemptyset(&mask);  
sigaddset(&mask, SIGINT);  
  
sigprocmask(SIG_BLOCK, &mask, nullptr);  
// ... критическая секция ...  
sigprocmask(SIG_UNBLOCK, &mask, nullptr);  
// если SIGINT пришёл — сейчас сработает
```

Это нужно для атомарных операций. Например, обновление структуры данных, которую и обработчик может трогать.

29.13.2 Group leader и tcsetpgrp

Настоящий shell делает больше. Он создаёт для каждого ребёнка **свою группу процессов** (через `setpgid`), и делает её **foreground** через `tcsetpgrp`. Тогда терминал шлёт сигналы только этой группе, не shell.

Сложный код с возможностью race conditions. Мы обошлись простым «shell игнорирует, ребёнок реагирует». Если хотите job control (fg/bg/jobs) — нужно делать как bash.

29.13.3 SIGCHLD в реальном shell

Bash подписан на SIGCHLD, чтобы реагировать на завершение фоновых задач. В нашей версии — нет фоновых задач, нет необходимости.

29.13.4 sigaction с siginfo

struct sigaction имеет ещё поле sa_sigaction (вместо sa_handler) и флаг SA_SIGINFO:

```
sa.sa_flags = SA_SIGINFO;
sa.sa_sigaction = [](int sig, siginfo_t* info, void*) {
    // info содержит детали — кто послал, какой адрес и т.д.
};
```

Полезно для SIGSEGV (узнать, какой адрес вызвал сегфолт). Для наших целей не нужно.

29.14 Главные правила главы

1. **sigaction** вместо **signal** — портабельнее и контролируемое.
2. В обработчике — только флаг. `printf/malloc/std::cout` — не `async-signal-safe`.
3. Используйте `volatile sig_atomic_t` для флагов из обработчика.
4. **SIGKILL** и **SIGSTOP** — не перехватываются. Это страховка ОС.
5. **SIG_IGN** сохраняется при `exec`, `custom-handler` — нет (резется в `SIG_DFL`).
6. Восстанавливайте **SIG_DFL** в ребёнке перед `exec`, если родитель что-то менял.
7. **SIGCHLD** — для асинхронной обработки умерших детей. Если синхронно `waitpid` — не нужно.
8. **SIGPIPE** игнорируйте в **shell**, иначе одна сломанная программа в пайплайне убьёт shell.

29.15 Маленькое упражнение

1. Соберите обновлённый shell. Запустите `sleep 5` и сразу нажмите Ctrl+C. Должно умереть только `sleep`, `shell` — нет.
2. Запустите `cat` (без аргументов — он ждёт ввода) и нажмите Ctrl+C. Cat умирает? Shell живой? Почему `cat` не использует наш `SIG_IGN`, который был у `shell`?
3. Запустите `yes | head -1`. Где умирает `yes` — от `SIGPIPE`? У нас включен `SIG_IGN` для `SIGPIPE` в `shell` — но `yes` это **ребёнок** с восстановленными дефолтами. Запустите. Объясните вывод.
4. Снимите `restore_default_signal_handlers()` в `pipeline.cpp` (закомментируйте). Соберите. Запустите `sleep 5` и нажмите Ctrl+C. Что произойдёт? Объясните.
5. Напишите программу из 15 строк: устанавливает обработчик `SIGINT` через `sigaction`, обработчик ставит `got_signal = 1`. Главный цикл проверяет флаг и печатает. Запустите, нажимайте Ctrl+C. Сколько срабатываний поймали?
6. (Сложнее) Прочитайте `man 7 signal-safety`. Найдите функции из вашего любимого языка C — какие можно вызывать из обработчика, какие нет?
7. (Сложнее) Реализуйте таймер: программа печатает сообщение каждую секунду через `alarm(1)` + обработчик `SIGALRM`. После 10 повторов завершается. Что произойдёт, если внутри обработчика вызвать `printf`? Соберите с `-fsanitize=undefined` и проверьте.

29.16 Что дальше

Глава 29 — **парсер командной строки**. Что нужно для настоящего shell: - Кавычки: `echo "hello world"` (с пробелом внутри). - Escape: `echo hello\ world`. - Редиректы: `> file.txt, 2>&1, < input.txt`. - Точка с запятой и `&&/||` между командами. - Подстановка переменных `$HOME`.

Это всё парсер. Похож на парсер мира из главы 23, но сложнее. Дальше — builtin-команды (30), свои утилиты `cat/wc/grep` (31). Часть III закроется.

Глава 30

Глава 29. Парсер командной строки

Наш текущий shell разбирает строку через простой `tokenize` — `split` по пробелам с `|` как отдельный токен. Это работает для `ls -l | wc`, но **не работает** для:

- `echo "hello world"` — пробел внутри строки должен быть частью одного аргумента.
- `echo hello > out.txt` — `>` это редирект, а не аргумент.
- `cat file.txt | grep "foo bar"` — кавычки и пайпы вместе.
- `ls 2> /dev/null` — отдельный редирект для `stderr`.

В этой главе сделаем настоящий парсер: с двойными кавычками, одинарными кавычками, escape через `\`, и редиректами `>/>>/</2>/2>>/2>&1`. После этого shell сильно ближе к `bash`.

Парсинг **командной строки** — классическая задача. Идея та же, что в парсере мира (глава 23), но больше тонкостей.

30.1 Что должен парсер

Возьмём строку:

```
grep "hello world" file.txt > result.txt 2>&1
```

Что должен выдать парсер:

- Команда: `grep`.
- Аргументы: `["grep", "hello world", "file.txt"]`.
- Редиректы:
 - `>` → файл `result.txt`.
 - `2>&1` → `stderr` туда же, куда `stdout`.

Структура:

```
struct Redirect {
    enum Type { Input, Output, Append, ErrOutput, ErrAppend, ErrToOut };
    Type type;
    std::string target;
};

struct Command {
    std::vector<std::string> argv;
    std::vector<Redirect> redirects;
};

struct Pipeline {
    std::vector<Command> commands;
};
```

Pipeline — список Command. Command — argv + редиректы. Redirect — тип + файл.

Тип ErrToOut соответствует 2>&1 (нет файла — переключение fd).

30.2 Этапы парсинга

Парсинг делится на два этапа:

1. **Tokenization** — превращение строки в список **токенов**: слова, кавычечные строки, спец-символы (|, >, <, >>, 2>, ...).
2. **Parsing** — превращение списка токенов в дерево (Pipeline → Commands → argv/redirects).

Этапы независимы. Tokenizer не знает про команды и редиректы — он знает только про синтаксис строк, кавычек, спец-символов. Parser работает уже над токенами, знает про структуру команды.

Это разделение упрощает каждую часть. В больших парсерах (компиляторы) tokenizer и parser ещё более отделены, иногда tokenizer — генерируемая программа (lex/flex). У нас всё руками.

30.3 Tokenizer

Главные требования:

1. **Пробелы и табы** разделяют слова, если **не внутри кавычек**.
2. **Двойные кавычки** "... " обрамляют строку. Внутри пробелы — часть слова. Внутри \", \\, \\$, \ — escape.

3. **Одинарные кавычки** `'...'` обрамляют **буквально**: никакого `escape` внутри.
4. `\` вне кавычек — `escape` следующего символа.
5. `|`, `<`, `>`, `>>` — отдельные токены.
6. `2>`, `2>>`, `2>&1` — отдельные токены, но **только в начале слова** (чтобы `file2.txt` не парсилось как `file + 2> + .txt`).

Реализация — конечный автомат: для каждого символа смотрим, в каком состоянии (внутри кавычек или нет) и решаем, что делать.

```
std::vector<std::string> tokenize(const std::string& line) {
    std::vector<std::string> tokens;
    std::string current;
    bool in_dq = false;    // в двойных кавычках
    bool in_sq = false;    // в одинарных

    auto flush = [&]() {
        if (!current.empty()) {
            tokens.push_back(std::move(current));
            current.clear();
        }
    };

    for (std::size_t i = 0; i < line.size(); ++i) {
        char c = line[i];

        if (in_sq) {
            if (c == '\\') in_sq = false;
            else current += c;
            continue;
        }

        if (in_dq) {
            if (c == '"') {
                in_dq = false;
            } else if (c == '\\') && i + 1 < line.size() &&
                (line[i + 1] == '"' || line[i + 1] == '\\') ||
                line[i + 1] == '$' || line[i + 1] == '`') {
                current += line[++i];
            } else {

```

```
        current += c;
    }
    continue;
}

// Вне кавычек.
if (c == '\\') { in_sq = true; continue; }
if (c == '"') { in_dq = true; continue; }

if (c == '\\\' && i + 1 < line.size()) {
    current += line[++i];
    continue;
}

if (c == ' ' || c == '\\t') {
    flush();
    continue;
}

// 2>, 2>>, 2>&1 – только в начале слова.
if (current.empty() && c == '2' && i + 1 < line.size() && line[i + 1] == '>') {
    if (i + 3 < line.size() &&
        line[i + 2] == '&' && line[i + 3] == '1') {
        tokens.push_back("2>&1");
        i += 3;
        continue;
    }
    if (i + 2 < line.size() && line[i + 2] == '>') {
        tokens.push_back("2>>");
        i += 2;
        continue;
    }
    tokens.push_back("2>");
    i += 1;
    continue;
}

// |, <, >, >>.
```



```

    std::string spec;
    std::size_t consumed = match_special(line, i, spec);
    if (consumed > 0) {
        flush();
        tokens.push_back(spec);
        i += consumed - 1;
        continue;
    }

    current += c;
}

if (in_sq || in_dq) {
    throw std::runtime_error("незакрытая кавычка");
}

flush();
return tokens;
}

```

Разбор по блокам:

Внутри одинарных кавычек (`in_sq`) — только `'` закрывает, всё остальное буквально. `\'` внутри одинарных — нельзя. Если очень нужно — закрыть `'`, дописать `\'`, снова открыть `'`: `'don\'\'t' = don't`. Bash-стиль.

Внутри двойных кавычек (`in_dq`) — `"` закрывает, `\` экранирует следующий **только** для `", \, $, ``. Другие escape-последовательности (`\n`, `\t`) внутри `"..."` **не** интерпретируются — будут литеральные `\n`. Это особенность `bash`; стандартная.

Вне кавычек: `- \` экранирует следующий любой символ. `-` Пробел/таб разделяют. `-` Спец-символы становятся отдельными токенами.

Условие `current.empty()` для `2>` — критично. Без него `file2.txt` парсилось бы как `file + 2> + .txt`. С условием — `file2.txt` остаётся одним словом, потому что когда мы видим `2`, `current` уже содержит `file`.

`match_special(line, i, spec)` распознаёт `|, <, >, >>`:

```

std::size_t match_special(const std::string& s, std::size_t i, std::string& out) {
    char c = s[i];
    if (c == '|') { out = "|"; return 1; }
}

```

```

if (c == '<') { out = "<"; return 1; }
if (c == '>') {
    if (i + 1 < s.size() && s[i + 1] == '>') { out = ">>"; return 2; }
    out = ">"; return 1;
}
return 0;
}

```

Возвращает **количество поглощённых символов** (1 или 2). В вызывающей функции делаем `i += consumed - 1` (тоже + 1 в for-loop), и идём дальше.

30.3.1 Незакрытые кавычки

В конце:

```

if (in_sq || in_dq) {
    throw std::runtime_error("незакрытая кавычка");
}

```

Если строка кончилась, а кавычка не закрыта — синтаксическая ошибка. Bash в таком случае печатает дополнительный prompt > и ждёт продолжения. Мы — просто отказываем.

30.3.2 Тестирование tokenize

```

tokenize("echo hello")           // → {"echo", "hello"}
tokenize("echo \"hello world\"") // → {"echo", "hello world"}
tokenize("ls | wc")               // → {"ls", "|", "wc"}
tokenize("cmd > file.txt")        // → {"cmd", ">", "file.txt"}
tokenize("cmd 2>&1")              // → {"cmd", "2>&1"}
tokenize("ls 2> err.txt")         // → {"ls", "2>", "err.txt"}
tokenize("echo a\\ b")            // → {"echo", "a b"}      (escape пробела)
tokenize("echo 'one \"two\"'")    // → {"echo", "one \"two\""}

```

30.4 Parser

Теперь токены → Pipeline.

```

Pipeline parse_pipeline(const std::string& line) {
    Pipeline p;
    auto tokens = tokenize(line);
}

```

```

if (tokens.empty()) return p;

auto seg_start = tokens.begin();
for (auto it = tokens.begin(); it != tokens.end(); ++it) {
    if (*it == "|") {
        if (seg_start == it) {
            throw std::runtime_error("пустая команда перед '|'");
        }
        p.commands.push_back(parse_command(seg_start, it));
        seg_start = it + 1;
    }
}
if (seg_start != tokens.end()) {
    p.commands.push_back(parse_command(seg_start, tokens.end()));
} else {
    throw std::runtime_error("пустая команда после '|'");
}

for (const auto& cmd : p.commands) {
    if (cmd.argv.empty()) {
        throw std::runtime_error("команда без названия");
    }
}
return p;
}

```

Идём по токенам, разбиваем на сегменты по |. Каждый сегмент идёт в parse_command.

```

Command parse_command(std::vector<std::string>::iterator begin,
                     std::vector<std::string>::iterator end) {
    Command cmd;
    for (auto it = begin; it != end; ++it) {
        if (is_redirect_token(*it)) {
            Redirect r;
            r.type = redirect_type_for(*it);
            if (r.type == Redirect::ErrToOut) {
                // 2>&1 – без аргумента
            } else {
                if (it + 1 == end || is_redirect_token(*(it + 1)) || *(it + 1) == "|") {

```

```

        throw std::runtime_error("редирект '" + *it + "' без файла");
    }
    ++it;
    r.target = *it;
}
cmd.redirects.push_back(std::move(r));
} else {
    cmd.argv.push_back(*it);
}
}
return cmd;
}

```

Идём по токенам: - Если редирект — следующий токен это файл (кроме 2>&1). Сохраняем. - Иначе — слово, добавляем в argv.

Проверки: после редиректа должен быть файл, не другой редирект и не |.

30.5 Применение редиректов

После fork, перед exec, в ребёнке вызываем apply_redirects:

```

void apply_redirects(const std::vector<Redirect>& redirects) {
    for (const auto& r : redirects) {
        switch (r.type) {
            case Redirect::Input: {
                int fd = open(r.target.c_str(), O_RDONLY);
                if (fd < 0) {
                    std::cerr << r.target << ": " << std::strerror(errno) << "\n";
                    _exit(1);
                }
                dup2(fd, STDIN_FILENO);
                close(fd);
                break;
            }
            case Redirect::Output: {
                int fd = open(r.target.c_str(),
                            O_WRONLY | O_CREAT | O_TRUNC, 0644);
                // ... аналогично, dup2 в STDOUT_FILENO
            }
        }
    }
}

```

```

    }
    case Redirect::Append: {
        int fd = open(r.target.c_str(),
                     O_WRONLY | O_CREAT | O_APPEND, 0644);
        // ... dup2 в STDOUT_FILENO
    }
    case Redirect::ErrOutput: {
        // ... open + dup2 в STDERR_FILENO
    }
    case Redirect::ErrAppend: {
        // O_APPEND + dup2 в STDERR_FILENO
    }
    case Redirect::ErrToOut: {
        dup2(STDOUT_FILENO, STDERR_FILENO);
        break;
    }
}
}
}
}
}

```

Каждый редирект: 1. Открыть файл с правильными флагами (`O_RDONLY` для чтения, `O_WRONLY | O_CREAT | O_TRUNC` для перезаписи, `O_WRONLY | O_CREAT | O_APPEND` для дописывания). 2. `dup2` дескриптор на нужный `stdin/stdout/stderr`. 3. Закрывать исходный `fd`.

Третий аргумент `open` (0644) — режим доступа для нового файла. 644 в восьмеричной = `rw-r--r--` (владелец читает/пишет, остальные читают). Учили в главе 5.

`ErrToOut (2>&1)` — просто `dup2(STDOUT_FILENO, STDERR_FILENO)`. После этого `stderr` идёт туда же, куда `stdout`.

30.5.1 Порядок важен

```
cmd > out.txt 2>&1
```

Это: 1. Открыть `out.txt`, `dup2(fd, 1)` — `stdout` идёт в файл. 2. `dup2(1, 2)` — `stderr` тоже идёт куда `stdout` (то есть в файл).

Итог: обе потока в `out.txt`.

```
cmd 2>&1 > out.txt
```

Это: 1. `dup2(1, 2)` — `stderr` туда же, куда сейчас `stdout` (терминал!). 2. `open out.txt, dup2(fd, 1)` — `stdout` идёт в файл.

Итог: `stderr` остаётся на терминале (потому что в момент `2>&1` он направлен туда), `stdout` в файл.

Это **классический подвох `bash`**. Запомните: `> file 2>&1` — оба в файл. `2>&1 > file` — нет.

Наш парсер сохраняет порядок редиректов в `vector`, и `apply_redirects` применяет их по очереди. То же поведение, что в `bash`.

30.6 Редиректы и пайпы

Что если в одной команде и пайп, и редирект?

```
$ ls > file.txt | wc -l
```

Тут `ls` имеет два направления вывода: пайп в `wc` и файл. Что побеждает?

По стандарту `bash` — **последний** редирект побеждает. Для `ls > file.txt`: 1. Сначала `ls`'s `stdout` идёт в pipe (наш код `dup2` от pipe в `STDOUT`). 2. Потом `apply_redirects` открывает `file.txt` и `dup2` в `STDOUT`.

Итог: `stdout` `ls` идёт в `file.txt`. Pipe в `wc` остаётся пустым (read end ждёт EOF). `wc -l` напечатает 0.

В нашем коде так и происходит, потому что мы вызываем `apply_redirects` **после** `dup2` пайпов:

```
if (pid == 0) {
    restore_default_signal_handlers();

    if (i > 0) dup2(pipes[i - 1][0], STDIN_FILENO);
    if (i < n - 1) dup2(pipes[i][1], STDOUT_FILENO);
    close_all_pipes(pipes);

    apply_redirects(p.commands[i].redirects); // ← после пайп-dup2

    auto argv = to_c_argv(p.commands[i].argv);
    execvp(argv[0], argv.data());
    _exit(127);
}
```

30.7 Чего парсер пока не умеет

В книге это упомяну, но реализовывать не буду — оставлю как упражнения.

30.7.1 Последовательность команд: `;`, `&&`, `||`

```
$ cmd1 ; cmd2
$ cmd1 && cmd2      # cmd2 если cmd1 успешно
$ cmd1 || cmd2      # cmd2 если cmd1 ошибка
```

Это **отдельный уровень** парсинга. Pipeline становится «вершиной» в дереве:

```
Sequence
├─ Pipeline (cmd1 | cmd2)
├─ &&
├─ Pipeline (cmd3)
├─ ||
└─ Pipeline (cmd4)
```

Для реализации — отдельный enum типов соединителей, отдельная функция `parse_sequence`. Не сложно, просто ещё уровень.

30.7.2 Подстановка переменных: `$HOME`

```
$ echo $HOME
/Users/serik
$ echo "${HOME}/file"
/Users/serik/file
```

Tokenizer должен распознать `$VAR` (или `${VAR}`) и **подставить значение** из окружения. `getenv("HOME")` — функция для этого.

Особенность: **только внутри двойных кавычек и без кавычек**. Внутри одинарных — буквально.

```
$ echo "$HOME"      # /Users/serik
$ echo '$HOME'      # $HOME
```

Подстановка делается после `tokenize` или прямо во время него — обе схемы работают.

30.7.3 Globbing: *, ?, [abc]

```
$ ls *.cpp
foo.cpp bar.cpp
```

Это **wildcard expansion**. После tokenize, перед exec — каждое слово с */?/[] развёртывается в список файлов. Стандартная функция — glob() из <glob.h>.

В нашем shell — не сделаем. Если нужно: возьмите glob() и разверните слова с шаблонами.

30.7.4 < /dev/null, heredoc <<EOF

```
$ cmd < /dev/null
$ cmd << EOF
hello
EOF
```

< file уже работает у нас. <<EOF — heredoc — это особая фишка: содержимое до EOF передаётся как stdin. Парсер должен обрабатывать **многострочный ввод**, что усложняет REPL.

Не реализуем; для простых нужд echo X | cmd или временный файл достаточно.

30.7.5 & фон

```
$ sleep 10 &
[1] 1234
$ jobs
[1]  Running    sleep 10
```

& означает «запусти и не жди». Shell сразу возвращается к prompt. Чтобы это сделать: - В run_pipeline ребёнок не ждётся через waitpid сразу. - Shell ведёт список запущенных в фоне процессов. - SIGCHLD обрабатывается асинхронно, чтобы обнаружить умерших. - Команда jobs показывает список, fg N — вернуть на передний план.

Это **job control** — целая отдельная подсистема. У нас выходит за рамки. Глава 30 коснётся лишь его минимума.

30.8 Запуск

```
$ make
$ ./build/myshell
```



```
myshell$ echo "hello world"
hello world

myshell$ echo hello > /tmp/out.txt
myshell$ cat /tmp/out.txt
hello

myshell$ ls Makefile > /tmp/out.txt
myshell$ cat /tmp/out.txt
Makefile

myshell$ echo 'single quotes test'
single quotes test

myshell$ ls nonexistent 2> /tmp/err.txt
[exit 1]
myshell$ cat /tmp/err.txt
ls: nonexistent: No such file or directory

myshell$ echo "with pipe" | tr a-z A-Z
WITH PIPE

myshell$ ls 2>&1 | grep .cpp
exec_runner.cpp
main.cpp
pipeline.cpp
signals.cpp

myshell$ exit
```

Всё работает.

30.9 Архитектура парсера в целом

Если посмотреть на нашу систему сверху:

Строка пользователя

↓

```

tokenize ← состояние: вне/внутри кавычек, escape
    ↓
список токенов
    ↓
parse_pipeline ← разбивает по `|`, валидирует
    ↓
parse_command ← собирает argv + redirects
    ↓
Pipeline (commands + redirects)
    ↓
run_pipeline ← системные вызовы fork/dup2/exec
    ↓
результат

```

Это **классическая трёхступенчатая схема** компилятора (в миниатюре): 1. **Lexer** (наш tokenize) — текст → токены. 2. **Parser** (наш parse_pipeline) — токены → AST. 3. **Interpreter / Code generator** (наш run_pipeline) — AST → действие.

Те же три фазы в gcc, в python (CPython), в ваших любимых интерпретаторах.

30.10 Главные правила главы

1. **Разделяйте lexer и parser** — упрощает каждую часть.
2. **Tokenizer = state machine**. Состояния: внутри кавычек, escape.
3. **Спец-символы определяются позицией**. 2> только в начале слова.
4. **Парсер строит AST**, не выполняет действия.
5. **Применение редиректов — после пайп-dup2** в ребёнке.
6. **Порядок редиректов важен!** > file 2>&1 ≠ 2>&1 > file.
7. **Хорошие сообщения об ошибках** парсера полезны пользователям.
8. **Не реализуйте всё bash-фичи разом**. Сделайте core, остальное — по запросу.

30.11 Маленькое упражнение

1. Соберите shell. Попробуйте:
 - echo "Hello, World" (с запятой и пробелом).
 - echo it\'s (escape апострофа).
 - ls > /tmp/a.txt && cat /tmp/a.txt (sequencing — а, нет, у нас не работает! проверьте, и какая ошибка).

2. Перенаправьте только stderr на файл: `python3 -c "import sys; sys.stderr.write('error\n'); sys.stdout.write('out\n')"` `2> /tmp/err.txt`. Проверьте, что out в stdout, а error в /tmp/err.txt.
3. Сравните `cmd > /tmp/x 2>&1` и `cmd 2>&1 > /tmp/x`. Создайте программу errout (на C++), которая печатает в stdout И stderr. Запустите обе версии. Посмотрите, что получилось в файле и на экране.
4. (Сложнее) Реализуйте подстановку переменных \$VAR. Добавьте в tokenizer развёртывание перед добавлением в current. Внутри одинарных кавычек — не разворачивать.
5. (Сложнее) Реализуйте ; (последовательность). Парсер должен возвращать `std::vector<Pipeline>`. main выполняет их по очереди, **независимо от кодов возврата**.
6. (Сложнее) Реализуйте && и ||. Парсер строит дерево или плоский список с типами связей. Выполнение учитывает код возврата предыдущей команды.
7. (Очень сложно) Реализуйте & (фон). Нужен SIGCHLD-обработчик, список фоновых задач, команда jobs.

30.12 Что дальше

Глава 30 — **builtin-команды и история**. Команды `cd`, `export`, `pwd` нельзя реализовать через `fork+exec` (они должны менять состояние shell — текущую папку или окружение). Их делают «внутри» shell — встроенными. Заодно сделаем хранение истории команд (стрелка вверх не сработает без сырого `termios`, но запись в `~/.myshell_history` — можно).

Глава 31 — свои утилиты `cat`, `wc`, `grep` написанные с нуля. Поймём, как утилиты Unix устроены изнутри. После этого Часть III закроется.

Глава 31

Глава 30. Builtins и история

К этому моменту наш shell умеет запускать любую внешнюю программу: пайпы, редиректы, кавычки, корректная обработка сигналов. Но что произойдёт, если вы наберёте `cd /tmp?` Запустится `/bin/cd` (или вернётся «не найден»). И даже если бы `/bin/cd` существовал — он бы поменял **свою** папку, не нашего shell. После завершения ребёнка shell остаётся в той же папке. Эффекта ноль.

То же с `export PATH=...`: ребёнок ставит переменную и умирает, родителю по умолчанию ничего не передаётся.

Эти команды должны **выполняться в самом shell**, не в ребёнке. Они называются **builtins** — встроенные команды. В этой главе мы их добавим: `cd`, `pwd`, `export`, `unset`, `echo`, `history`. Плюс сохранение истории в `~/.myshell_history`.

31.1 Зачем builtins

Часть команд **не может** работать через `fork+exec`, потому что они должны **менять состояние самого shell**:

- **cd** — менять рабочую папку shell. Ребёнок не может изменить `cwd` родителя.
- **export/unset** — менять переменные окружения shell. Ребёнок только меняет свои.
- **exit** — завершать сам shell.
- **alias** — добавлять алиасы для будущих команд в этом же shell.
- **source/.** — выполнять скрипт в текущем процессе.

Другие команды **могут** быть и встроенными, и внешними. У `bash` есть встроенные `echo`, `pwd`, `kill` — для скорости (не нужен `fork+exec`). Внешние `/bin/echo` и `/bin/pwd` тоже существуют. Builtin побеждает, если команда есть в обеих формах.

Третий вариант — **builtin для удобства**: - **history** — показать сохранённые команды (которые знает только shell). - **jobs/bg/bg** — управление фоновыми задачами. - **help** — справка.

31.2 API: try_builtin

В нашем коде:

```
bool try_builtin(const std::vector<std::string>& argv, int& out_code);
```

Логика: 1. Если argv[0] — известная встроенная команда, выполнить её, положить код в out_code, вернуть true. 2. Иначе вернуть false — пусть вызывающий запускает через exes.

В main сначала пробуем builtin, потом — внешнюю:

```
if (p.commands.size() == 1 && p.commands[0].redirects.empty()) {
    int code = 0;
    if (shell::try_builtin(first, code)) {
        if (code != 0) std::cerr << "[exit " << code << "]\n";
        continue;
    }
}

int rc = shell::run_pipeline(p);
```

Условие: builtin работает только если **одна команда и без редиректов**. Иначе делаем fork (как обычно).

31.2.1 Почему так

Если в пайплайне (echo hi | cd /tmp) — cd в ребёнке. Эффекта на shell нет.

С редиректами (cd /tmp > log.txt) — теоретически можно: выполнить cd напрямую в shell, открыть файл и записать «что-то». Но bash тоже не делает builtins с редиректами в shell — он форкается. Это спорное место, мы упрощаем.

31.3 Builtins реализации

31.3.1 cd

```
int builtin_cd(const std::vector<std::string>& argv) {
    std::string target;
    if (argv.size() < 2) {
```

```

const char* home = std::getenv("HOME");
if (!home) {
    std::cerr << "cd: HOME не задана\n";
    return 1;
}
target = home;
} else {
    target = argv[1];
}
if (chdir(target.c_str()) < 0) {
    std::cerr << "cd: " << target << ": " << std::strerror(errno) << "\n";
    return 1;
}
return 0;
}

```

Без аргумента — в домашнюю папку (\$HOME). С аргументом — туда. `chdir(path)` — системный вызов, меняет `cwd` текущего процесса. Возвращает 0 при успехе, -1 при ошибке.

`std::getenv("HOME")` — функция из `<cstdlib>`, читает переменную окружения. Возвращает указатель или `nullptr`.

В `bash` `cd` — означает «предыдущая папка», `cd ~user` — домашняя другого пользователя, и так далее. Мы — минимум.

31.3.2 pwd

```

int builtin_pwd() {
    char buf[4096];
    if (!getcwd(buf, sizeof(buf))) {
        std::cerr << "pwd: " << std::strerror(errno) << "\n";
        return 1;
    }
    std::cout << buf << "\n";
    return 0;
}

```

`getcwd(buf, size)` — системный вызов, кладёт в `buf` текущую папку. Возвращает указатель на `buf` при успехе, `nullptr` при ошибке (например, буфер маленький — `errno == ERANGE`).

`PATH_MAX` на Linux обычно 4096 байт. 4 KiB достаточно для разумных глубин.

31.3.3 export

```
int builtin_export(const std::vector<std::string>& argv) {
    if (argv.size() < 2) {
        std::cerr << "export: укажите KEY=VALUE\n";
        return 1;
    }
    for (std::size_t i = 1; i < argv.size(); ++i) {
        const std::string& kv = argv[i];
        auto eq = kv.find('=');
        if (eq == std::string::npos) {
            std::cerr << "export: ожидался формат KEY=VALUE: " << kv << "\n";
            return 1;
        }
        std::string key = kv.substr(0, eq);
        std::string val = kv.substr(eq + 1);
        if (setenv(key.c_str(), val.c_str(), 1) < 0) {
            std::cerr << "export: " << std::strerror(errno) << "\n";
            return 1;
        }
    }
    return 0;
}
```

setenv(key, val, overwrite) — установить переменную окружения. Третий аргумент: если 0, не перезаписывать существующую; если 1, перезаписать. Мы перезаписываем.

В bash `export PATH=$PATH:/new/path` использует **подстановку** `$PATH`. Мы её не реализовали (см. главу 29). Так что у нас `export PATH=$PATH:/x` даст буквальную строку `$PATH:/x`. Чтобы это работало — реализовать подстановку в tokenizer.

unsetenv — обратная операция:

```
int builtin_unset(const std::vector<std::string>& argv) {
    for (std::size_t i = 1; i < argv.size(); ++i) {
        unsetenv(argv[i].c_str());
    }
    return 0;
}
```

31.3.4 echo

В bash echo — и builtin, и внешний (/bin/echo). Builtin быстрее. Реализуем:

```
int builtin_echo(const std::vector<std::string>& argv) {
    bool newline = true;
    std::size_t start = 1;
    if (argv.size() > 1 && argv[1] == "-n") {
        newline = false;
        start = 2;
    }
    for (std::size_t i = start; i < argv.size(); ++i) {
        if (i > start) std::cout << " ";
        std::cout << argv[i];
    }
    if (newline) std::cout << "\n";
    return 0;
}
```

Поддерживаем -n (без перевода строки на конце). Не поддерживаем -e (interpret escapes) — у нас уже tokenizer обрабатывает escape.

31.3.5 exit

exit мы обрабатываем до try_builtin — это особый случай, заканчивает цикл shell:

```
if (!first.empty() && (first[0] == "exit" || first[0] == "quit")) {
    break;
}
```

Можно было бы сделать builtin, но он бы вызвал std::exit(N) — это пропускает деструкторы и atexit. Лучше явное break из цикла.

31.4 История команд

Простая модель: список строк в памяти. Сохраняем в файл при выходе, загружаем при старте.

31.4.1 В памяти

```
namespace {
    std::vector<std::string> g_history;
```



```

}

void history_add(const std::string& line) {
    if (line.empty()) return;
    if (!g_history.empty() && g_history.back() == line) return;
    g_history.push_back(line);
}

```

if (g_history.back() == line) — не сохраняем дубликаты подряд. Полезно, если кто-то много раз нажимает Enter на одной команде.

31.4.2 Загрузка/сохранение

```

void history_load(const std::string& path) {
    std::ifstream in(path);
    if (!in.is_open()) return;
    std::string line;
    while (std::getline(in, line)) {
        if (!line.empty()) g_history.push_back(line);
    }
}

void history_save(const std::string& path) {
    std::ofstream out(path);
    if (!out.is_open()) return;
    for (const auto& line : g_history) {
        out << line << "\n";
    }
}

```

Простой текстовый формат: одна команда на строку. Если файл не открылся (нет прав, нет папки) — молча игнорируем. Это **не критическая** функция.

31.4.3 Builtin history

```

int builtin_history() {
    std::size_t i = 1;
    for (const auto& line : g_history) {
        std::cout << " " << i << " " << line << "\n";
    }
}

```

```
    ++i;  
}  
return 0;  
}
```

Печатает с номерами.

31.4.4 В main

```
namespace {  
    std::string history_path() {  
        const char* home = std::getenv("HOME");  
        if (!home) return "./.myshell_history";  
        return std::string(home) + "/.myshell_history";  
    }  
}  
  
int main() {  
    shell::install_shell_signal_handlers();  
    shell::history_load(history_path());  
  
    while (...) {  
        // ...  
        shell::history_add(line);  
        // ...  
    }  
  
    shell::history_save(history_path());  
    return 0;  
}
```

Загружаем при старте, добавляем после каждой команды, сохраняем при выходе.

31.4.5 Альтернативный подход

Большие shell'ы (bash, zsh) пишут историю **сразу** после команды, не на выходе. Это надёжнее: если shell упадёт — история уцелеет.

Реализация — открыть файл в append-режиме, дописать строку, закрыть. На каждую команду.

В нашем коде проще: при выходе. Если shell умирает аварийно — потеря.

31.5 Навигация стрелками — кратко

Если запустить `bash`, стрелка вверх возвращает предыдущую команду. У нас — нет. Стрелка вверх в `std::getline` даст коды `^[` (escape-последовательность).

Чтобы реализовать как `bash`, нужно:

1. **Поставить терминал в raw mode** через `tcsetattr(STDIN, TCSANOW, ...)` с отключённым `ICANON` и `ECHO`.
2. **Читать по одному символу** через `read(STDIN, &c, 1)`.
3. **Распознавать escape-последовательности**: `\033[A` — стрелка вверх, `\033[B` — вниз, `\033[C` — вправо, `\033[D` — влево.
4. **Управлять курсором** через эти же escape-последовательности (`\033[K` — стереть остаток строки, `\033[<n>D` — сдвинуть курсор влево на `n`).
5. **Восстановить терминал** при выходе.

Это **большая** работа. Готовое решение — **GNU readline** (или **libedit**). Подключаешь, и она делает всё: история, автодополнение, многострочный ввод, поиск `Ctrl+R`, и так далее.

В нашем учебном — без `readline`. Стрелочки не работают, но при наборе команды есть `Ctrl+U` (стереть всю строку — стандартный `shortcut` терминала). Для учебной цели достаточно.

31.6 Job control — кратко

Что делает реальный `shell` для фоновых задач (`cmd &`):

1. **setpgid** в ребёнке — поместить его в **свою** группу процессов.
2. **tcsetpgrp** не вызывать (фоновая задача не владеет терминалом).
3. **Не делать waitpid** сразу — ребёнок идёт в фон.
4. Когда команда `jobs` — печатать список фоновых.
5. `fg N` — `tcsetpgrp(terminal, pgid) + waitpid`.
6. **SIGCHLD** — асинхронно обновлять статус (живой/умер).

Это серьёзная инфраструктура. У нас оставим как упражнение. Если интересно — посмотрите исходники `mksh` или `dash`.

31.7 Что пользователь видит

```
$ ./build/myshell
=== myshell ===
Builtins: cd, pwd, export, unset, history, echo, exit.
```

```
myshell$ pwd
/Users/serik/Desktop/cpp-from-scratch/demo-shell
```

```
myshell$ cd /tmp
myshell$ pwd
/private/tmp
```

```
myshell$ export F00=bar
myshell$ env | grep F00
F00=bar
```

```
myshell$ cd
myshell$ pwd
/Users/serik
```

```
myshell$ history
 1  pwd
 2  cd /tmp
 3  pwd
 4  export F00=bar
 5  env | grep F00
 6  cd
 7  pwd
 8  history
```

```
myshell$ exit
$ cat ~/.myshell_history
pwd
cd /tmp
pwd
export F00=bar
env | grep F00
cd
pwd
history
exit
```

Builtins работают. История сохранилась в `~/.myshell_history`. На следующем запуске она загрузится.

Заметьте: после `cd /tmp` команда `pwd` показывает `/private/tmp`. Это симлинк macOS (`/tmp` — это `/private/tmp`). На Linux была бы просто `/tmp`. `chdir` следует симлинкам.

31.8 Builtins vs внешние — производительность

Запустить `echo hi` через builtin — ноль системных вызовов, кроме `write` для вывода. Через `exes` — `fork` (тысячи микросекунд), `exes` (загрузка `/bin/echo`, инициализация runtime), `wait`. Разница в **сотни раз**.

Поэтому популярные команды (`echo`, `printf`, `test`, `l`, `pwd`) **всегда** делают builtins в любом shell.

Поэтому же скрипты с тысячами `[ "$x" = "y" ]` работают быстро в `bash` — это builtin, не внешний `/usr/bin/[` (хотя последний существует ради совместимости).

31.9 Главные правила главы

1. **Команды, меняющие состояние shell** (`cd`, `export`, `unset`), обязаны быть builtins.
2. **Try-builtin до fork+exes** — порядок важен.
3. **Builtins в пайплайне** не должны быть «настоящими» (мы упрощаем — пайплайны через `exes`).
4. **История в текстовом файле** — простой формат, легко расширять.
5. **Дедупликация подряд идущих** — не сохраняем одинаковые команды.
6. **Сохраняйте сразу** после команды для устойчивости к крашу (наш код сохраняет на выходе — компромисс).
7. **Стрелки вверх/вниз** — отдельная инфраструктура с raw termios. Готовое решение — `read-line`.
8. **Job control (&/jobs/fg/bg)** — отдельная подсистема с группами процессов.

31.10 Маленькое упражнение

1. Соберите. Попробуйте `cd /tmp && pwd && ls`. У нас `&&` не работает — три команды на отдельных строках.
2. Добавьте builtin alias: `alias ll="ls -l"`. Парсер должен распознать `ll` как алиас и заменить на `ls -l` перед исполнением. Подсказка — глобальный `std::unordered_map<std::string, std::string>`.

3. Добавьте `cd` — переход в предыдущую папку. Подсказка — хранить предыдущую папку в глобальной переменной.
4. Реализуйте подстановку переменных в `tokenizer` (см. глава 29, упражнение 4). После этого `export PATH=$PATH:/new` будет работать.
5. Сделайте сохранение истории **сразу** после каждой команды (`append`-режим), не на выходе. Сравните производительность для большого количества команд.
6. Добавьте `history -c` — очистить историю.
7. (Сложнее) Подключите `libreadline`. Установите `brew install readline` или `apt install libreadline-dev`. Замените `std::getline` на `readline("myshell$ ")`. Получите стрелочки, `Ctrl+R`, автодополнение по командам в `$PATH`.
8. (Сложнее) Реализуйте `&` фон. Минимально: после команды с `&` — не вызывать `waitpid`, но сохранить `pid`; команда `jobs` печатает список; при выходе из `shell` завершить всех. `Job control` полный — целая глава отдельной книги.

31.11 Что дальше

Глава 31 — **свои утилиты** `cat`, `wc`, `grep`. Напишем три классические Unix-утилиты с нуля на C++. Это будет упражнение в **чтении файлов** на низком уровне, **обработке текста**, и **передаче через стандартные потоки**. После этого Часть III закроется, и мы перейдём к Части IV — мини-СУБД.

Глава 32

Глава 31. Свои утилиты — cat, wc, grep

Финальная глава Части III. Мы написали shell, который умеет запускать программы, соединять их пайпами, перенаправлять потоки. А теперь напишем **сами программы**: три классических Unix-утилиты с нуля. `myscat`, `mywc`, `mygrep`.

Это упражнение в **системном программировании** — низкоуровневое чтение файлов, обработка байтов, регулярные выражения. И **проверка концепций Unix**: ваш `myscat` читает из `stdin`, ваш `mygrep` пишет в `stdout`, и они соединяются в пайплайне через `myshell`. Полный цикл: написал shell → написал утилиты → запустил утилиты через свой shell.

После этой главы Часть III закрыта.

32.1 Зачем писать свои

Готовые `cat`, `wc`, `grep` уже есть. Зачем повторять?

1. **Понять, как они работают изнутри.** Эти программы 50 лет используются, но мало кто знает, что у них под капотом.
2. **Тренировка в низкоуровневом C++.** `read/write` напрямую, без `<iostream>` сверху. UTF-8 на байтовом уровне.
3. **Проверить наш shell.** Соединить свои бинари пайпами — увидеть, что архитектура «программы как фильтры» работает.
4. **Это маленькие, но полные программы.** Каждая решает понятную задачу, можно увидеть её целиком.

32.2 Структура

Каждая утилита — отдельный исполняемый файл с собственным main. Размещаем в demo-shell/utils/:

```
demo-shell/
├── src/          — код myshell
├── utils/
│   ├── mycat.cpp
│   ├── mywc.cpp
│   └── mygrep.cpp
└── Makefile      — собирает myshell и все утилиты
```

В Makefile добавим правило: для каждого utils/X.cpp собираем build/X. Через wildcard — без правок при добавлении новой утилиты.

```
UTIL_SRCS := $(wildcard utils/*.cpp)
UTILS      := $(patsubst utils/%.cpp, build/%, $(UTIL_SRCS))

all: build/myshell utils

utils: $(UTILS)

build/%: utils/%.cpp
    @mkdir -p build
    $(CXX) $(CXXFLAGS) $< -o $@
```

make соберёт build/myshell, build/mycat, build/mywc, build/mygrep. Каждая утилита — один .cpp файл, прямая компиляция без линковки с библиотеками проекта (они независимы).

32.3 mycat — простейший фильтр

cat — самая базовая Unix-утилита. Читает файлы (или stdin) и пишет в stdout. Имя — от **con-catenate**: при двух файлах сливает их.

Алгоритм: 1. Если аргументов нет — читать stdin. 2. Иначе для каждого аргумента — открыть, прочитать, записать. 3. — как имя файла означает stdin.

utils/mycat.cpp:

```
#include <cerrno>
#include <cstring>
```



```
#include <iostream>
#include <string>

#include <fcntl.h>
#include <unistd.h>

namespace {

int cat_fd(int fd, const std::string& name) {
    char buf[4096];
    while (true) {
        ssize_t n = read(fd, buf, sizeof(buf));
        if (n == 0) return 0;           // EOF
        if (n < 0) {
            if (errno == EINTR) continue; // прервано сигналом — повтор
            std::cerr << "mycat: " << name << ": " << std::strerror(errno) << "\n";
            return 1;
        }
        ssize_t written = 0;
        while (written < n) {
            ssize_t m = write(STDOUT_FILENO, buf + written, n - written);
            if (m < 0) {
                if (errno == EINTR) continue;
                std::cerr << "mycat: write: " << std::strerror(errno) << "\n";
                return 1;
            }
            written += m;
        }
    }
}

} // namespace

int main(int argc, char* argv[]) {
    if (argc < 2) {
        return cat_fd(STDIN_FILENO, "<stdin>");
    }
}
```

```

int rc = 0;
for (int i = 1; i < argc; ++i) {
    std::string name = argv[i];
    if (name == "-") {
        if (cat_fd(STDIN_FILENO, "<stdin>") != 0) rc = 1;
        continue;
    }
    int fd = open(name.c_str(), O_RDONLY);
    if (fd < 0) {
        std::cerr << "mycat: " << name << ": " << std::strerror(errno) << "\n";
        rc = 1;
        continue;
    }
    if (cat_fd(fd, name) != 0) rc = 1;
    close(fd);
}
return rc;
}

```

Разбор.

`cat_fd(int fd, name)` — основная работа. Читает из `fd`, пишет в `STDOUT_FILENO`.

Буфер 4 КВ — компромисс между «много syscall'ов на маленьком буфере» и «много памяти на большом». 4 КВ примерно равно размеру страницы памяти на большинстве систем; это «дешёвый» размер.

read возвращает: - 0 — EOF. Файл закончился. - > 0 — число прочитанных байт (может быть меньше запрошенного — это **short read**). - -1 — ошибка. `errno` уточняет.

EINTR — прерывание сигналом. Если `read` был прерван (например, программе пришёл `SIGCHLD`), мы повторяем. Без этого было бы случайное падение при появлении сигналов.

write тоже может вернуть short write: меньше, чем запрошено. Нужно повторять, пока всё не записано. В нашем цикле — внутренний цикл с накопителем `written`. Это **критично** для пайпов: если читатель медленный, буфер пайпа заполняется, и `write` может уйти в `short`.

При ошибке read одного файла — продолжаем со следующим. Не падаем сразу. Возвращаем `rc = 1` в конце, как делает обычный `cat`.

32.3.1 Проверка

```
$ ./build/mycat Makefile | head -3
# Makefile для демо-shell.
# Зеркало стиля демо-rpg/Makefile: -MMD для авто-зависимостей,
# wildcard для подхвата новых .cpp без правок.

$ echo "hello" | ./build/mycat
hello

$ echo "stdin part" | ./build/mycat - Makefile | head -5
stdin part
# Makefile для демо-shell.
...
```

Работает. С точки зрения системы это полноценный cat.

32.3.2 Почему read/write, а не std::ifstream

Мог бы быть код в стиле:

```
std::ifstream in(name);
std::cout << in.rdbuf();
```

Короче, работает. Но: - Менее **прозрачно** на уровне ядра. Каждый std::ifstream под капотом — буферы, скрытые промежуточные копии. - Тут мы хотим **обучить** низкоуровневой работе. read/write — это **системные вызовы** один-к-одному. - На крупных файлах ручной read+write часто **быстрее** — нет лишних копий.

В production-коде обычно std::ifstream достаточно. Здесь — учимся.

32.4 mywc — счётчик

wc (word count) считает в файле: - **Строки** (line count) — число символов \n. - **Слова** (word count) — последовательности непробельных символов. - **Байты** или **символы** (с -с).

utils/mywc.cpp (фрагмент):

```
struct Counts {
    long long lines = 0;
    long long words = 0;
    long long bytes = 0;
```

```

    long long chars = 0;
};

Counts count_stream(std::istream& in) {
    Counts c;
    char ch;
    bool in_word = false;
    while (in.get(ch)) {
        ++c.bytes;

        // UTF-8: считаем «не-продолжающие» байты.
        if ((static_cast<unsigned char>(ch) & 0xC0) != 0x80) {
            ++c.chars;
        }

        if (ch == '\n') ++c.lines;

        bool ws = (ch == ' ' || ch == '\t' || ch == '\n' || ch == '\r' || ch == '\f' || ch == '\v');
        if (ws) {
            in_word = false;
        } else if (!in_word) {
            in_word = true;
            ++c.words;
        }
    }
    return c;
}

```

Разбор алгоритма.

bytes — простой счётчик байтов.

chars — счётчик символов UTF-8. Помните главу 2: UTF-8 кодирует один символ в 1-4 байта. Первый байт всегда **не имеет** маски 10 в старших двух битах; продолжающие байты имеют. Маска 0xC0 берёт верхние 2 бита; сравнение с 0x80 — это битовая комбинация 10xxxxxx.

Если бит == 10 — это продолжение **существующего** символа, не считаем. Иначе — это начало нового символа (ASCII или первый байт многобайтного), считаем.

Для ASCII файла bytes == chars. Для UTF-8 с кириллицей chars будет меньше: «Привет» = 12 байт = 6 символов.

lines — простой подсчёт `\n`. Заметьте: если файл не оканчивается `\n`, последняя строка не считается. Это совместимо с POSIX `wc` — она тоже так.

words — двухсостоянная машина: `- in_word = false` — мы сейчас в пробелах. Если встречаем непробельный — переход в `in_word = true, ++words`. `- in_word = true` — мы в слове. Пробельный → выход; непробельный → остаёмся.

Пробельные символы (ws): пробел, таб, `\n`, `\r`, `\f` (form feed), `\v` (vertical tab). Чтобы соответствовать `wc`.

32.4.1 main

```
int main(int argc, char* argv[]) {
    bool show_chars = false;
    std::vector<std::string> files;
    for (int i = 1; i < argc; ++i) {
        std::string a = argv[i];
        if (a == "-c") show_chars = true;
        else files.push_back(a);
    }

    if (files.empty()) {
        Counts c = count_stream(std::cin);
        print_counts(c, "", show_chars);
        return 0;
    }

    Counts total;
    int rc = 0;
    for (const auto& f : files) {
        std::ifstream in(f);
        if (!in.is_open()) {
            std::cerr << "mywc: " << f << ": " << std::strerror(errno) << "\n";
            rc = 1;
            continue;
        }
        Counts c = count_stream(in);
        print_counts(c, f, show_chars);
        total.lines += c.lines;
        total.words += c.words;
    }
}
```

```

        total.bytes += c.bytes;
        total.chars += c.chars;
    }
    if (files.size() > 1) {
        print_counts(total, "total", show_chars);
    }
    return rc;
}

```

Парсим аргументы (-с), накапливаем тотал по всем файлам, печатаем строку «total» если файлов больше одного. Совместимо с POSIX wc.

32.4.2 Проверка

```

$ ./build/mywc Makefile src/*.cpp
   72    195   1602 Makefile
  151    509   4041 src/builtins.cpp
  117    425   3239 src/exec_runner.cpp
   72    197   1928 src/main.cpp
  239    818   7964 src/pipeline.cpp
   32     80    880 src/signals.cpp
  683   2224  19654 total

$ echo "Привет мир" | ./build/mywc
   1     2    21      # 21 байт (12 на «Привет» + 1 пробел + 6 на «мир» + 2 на «\n»)

$ echo "Привет мир" | ./build/mywc -c
   1     2    11      # 11 символов (6 + 1 + 3 + 1)

```

Работает. Совпадает с системным wc побайтно.

32.5 mygrep — фильтр по шаблону

grep (Global Regular Expression Print) ищет строки, **подходящие под шаблон**. Каждая строка проверяется через regex, и если есть совпадение — печатается.

Опции в нашей версии: -i — ignore case. -n — печатать номер строки. -v — invert (печатать НЕсовпадающие).

utils/mygrep.cpp (фрагмент):

```

#include <regex>

struct Options {
    bool ignore_case = false;
    bool show_line_numbers = false;
    bool invert = false;
    std::string pattern;
    std::vector<std::string> files;
};

int grep_stream(std::istream& in, const std::regex& re,
               const Options& opts, const std::string& filename) {
    std::string line;
    long long line_no = 0;
    bool multi_file = opts.files.size() > 1;
    while (std::getline(in, line)) {
        ++line_no;
        bool matched = std::regex_search(line, re);
        if (matched == opts.invert) continue;

        if (multi_file && !filename.empty()) std::cout << filename << ":";
        if (opts.show_line_numbers)         std::cout << line_no << ":";
        std::cout << line << "\n";
    }
    return 0;
}

```

Разбор.

std::regex — стандартный класс для регулярных выражений. По умолчанию использует ECMAScript-синтаксис (близкий к POSIX extended). Поддерживает: - `^foo/foos` — начало/конец строки. - `.` — любой символ. - `*` / `+` / `?` — повторения. - `[abc]` / `[a-z]` — символьные классы. - `\d` / `\w` / `\s` — цифры/слова/пробелы. - `()` — группы. - `|` — альтернативы.

std::regex_search(line, re) — ищет **первое совпадение** где угодно в строке. Возвращает **bool**.

Альтернатива — **std::regex_match**, которая требует, чтобы **вся строка** совпала. Для **grep** нужен **search** — частичное совпадение.

matched == opts.invert — компактная инвертированная логика. Если **invert=false**, пропускаем когда не совпало; если **invert=true**, пропускаем когда совпало.

32.5.1 Компиляция regex

```
auto flags = std::regex::ECMAScript;
if (opts.ignore_case) flags |= std::regex::icase;

std::regex re;
try {
    re = std::regex(opts.pattern, flags);
} catch (const std::regex_error& e) {
    std::cerr << "mygrep: bad regex '" << opts.pattern << "': " << e.what() << "\n";
    return 2;
}
```

Компиляция regex — **дорогая** операция (парсинг шаблона, построение конечного автомата). Делаем **один раз** до цикла. Каждый regex_search уже быстрый.

std::regex_error бросается при невалидном шаблоне (например, [unclosed]). Ловим и выводим ошибку.

32.5.2 Проверка

```
$ ./build/mygrep -n "include" src/main.cpp
1:#include "builtins.h"
2:#include "pipeline.h"
3:#include "signals.h"
5:#include <stdlib>
6:#include <exception>
...

$ ./build/mygrep -i "ERROR" *.cpp 2>/dev/null # case-insensitive

$ ./build/mygrep -v "^#" Makefile # строки НЕ начинающиеся с #
```

Полноценный grep, в 80 строк C++.

32.6 Полный пайплайн через myshell

Соберём всё. В нашем myshell запустим **наши** утилиты, соединённые пайпами:


```
$ ./build/myshell
myshell$ ./build/mycat Makefile | ./build/mygrep -i make | ./build/mywc -c
      2      11      90
```

Что произошло: 1. `mycat Makefile` читает `Makefile`, пишет в `stdout` (но `stdout` перенаправлен `myshell` через `pipe` в `mygrep`). 2. `mygrep -i make` читает `stdin` (из пайпа), ищет «make» (ignore case), пишет совпадения в `stdout` (опять в пайп). 3. `mywc -c` читает `stdin`, считает строки/слова/символы, печатает на терминал.

Всё работает. Три **наши** программы, соединённые **нашим** shell. От `fork+exec+pipe` в shell до системных `read/write` в утилитах — весь цикл наш.

32.7 Что мы не реализовали

Реальные `cat`, `wc`, `grep` имеют десятки опций. Мы реализовали базовые. Что не сделано:

cat: - `-n` (нумерация строк). - `-E` (показывать \$ на конце строк). - `-T` (показывать ^I для табов). - `-A` (всё это вместе).

wc: - `-l/-w/-c/-m` (только строки/слова/байты/символы соответственно). - `-L` (максимальная длина строки).

grep: - `-r` (рекурсивно по каталогам). - `-l` (только имена файлов). - `-c` (только число совпадений). - `-A/-B` (контекст N строк до/после). - `-E/-P` (extended / Perl regex). - `-F` (literal string без regex).

Все они — простые расширения. Если хотите практики — допишите.

32.8 mmap для скорости

POSIX даёт `mmap(addr, len, prot, flags, fd, off)` — **отображение файла в память**. После этого файл доступен как массив байтов, без `read/write`:

```
#include <sys/mman.h>

int fd = open("big.txt", O_RDONLY);
struct stat st;
fstat(fd, &st);
char* data = (char*)mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);

// Теперь data[0..st.st_size-1] — содержимое файла.
// Использовать как char-массив.
```

```
munmap(data, st.st_size);  
close(fd);
```

Преимущества mmap: - **Никаких read/write**. Ядро само подкачивает страницы по мере обращения (lazy). - **Меньше копий**. Данные в kernel-page-cache доступны напрямую программе. - **Удобно**. Если нужно искать паттерн или скакать по файлу — data[i] это просто индексация.

Недостатки: - **Не работает для не-seekable** (терминалов, pipe-ов). - **Нагружает виртуальное пространство**. На больших файлах (10+ GB) приходится управлять регионами. - **EOF на сетевом mount** дает SIGBUS, а не EOF — особый случай.

Для grep на больших файлах mmap может дать 2-3× ускорение. Для cat в pipe-пайплайне — не подойдёт (читатель stdin не отображается). Мы не использовали — наш read+buffer работает везде.

32.9 Архитектура «маленьких программ»

Что мы реально сделали:

[mycat] → buffer → write → [pipe в ядре] → read → [mygrep] → ... → [mywc] → терминал

Каждая утилита читает из stdin или файла, пишет в stdout. Не знает, что справа и слева. Не знает, в файле stdin или в pipe.

Это **Unix-философия**: «маленькие программы, каждая делающая одно дело». Их можно соединять.

Альтернативный подход — **монолит**: один большой бинарь со всем. Большие IDE, Microsoft Office, монорепо-фреймворки. У него тоже есть преимущества (быстрее не запускать процессы, доступ к общим структурам данных).

Но Unix-стиль выиграл в системном программировании, потому что: - **Композиция** простая. `ls -l | grep .cpp | wc -l` — три программы соединены за две клавиши. - **Тестируемость**: каждая утилита тестируется отдельно. - **Параллелизм**: ядро автоматически параллелит пайплайн. - **Стабильность**: одна утилита упала — другие живы.

Эта философия — то, что вы освоили в Части III. Знаете, как написать программу-фильтр, как соединить программы, как shell их запускает. Дальше — применимо везде, где Linux/macOS.

32.10 Главные правила главы

1. **read/write в цикле с обработкой EINTR и short.** Не предполагайте, что прочитается всё одним вызовом.
2. **Буфер 4 KB** — хороший дефолт. Меньше = больше syscall'ов; больше = расход памяти на буфер.
3. **Argv-парсинг руками** — для простых утилит сойдёт. Сложные — getopt.
4. **regex компилируется один раз** перед циклом, не на каждой итерации.
5. **std::regex_search** для grep, **std::regex_match** для целой строки.
6. **UTF-8 — байты с маской 10 в старших** — продолжающие. Не считаем как символы.
7. **Утилиты — фильтры stdin-stdout.** Не зависят от того, к чему подключены.
8. **mmap для больших файлов** — иногда быстрее read. Не работает на pipe/socket.

32.11 Маленькое упражнение

1. Соберите все: make. Запустите тесты — ./build/mycat, ./build/mywc, ./build/mygrep.
2. Прогоните **наш** пайплайн через **наш** shell:

```
myshell$ ./build/mycat src/main.cpp | ./build/mygrep -n include | ./build/mywc
```

Посчитайте сами и сравните.
3. Добавьте в mycat опцию -n (нумерация строк, как cat -n).
4. Добавьте в mywc опции -l/-w/-c/-m (показывать только запрошенное).
5. Добавьте в mygrep опцию -c (только число совпадений) и -l (только имена файлов с совпадением).
6. (Сложнее) Перепишите mycat на mmap — открыть файл, отобразить, write целиком. Замерьте скорость на 1 GB файле.
7. (Сложнее) Сделайте myhead — печатает первые N строк (по умолчанию 10). Похоже на mycat, но останавливается.
8. (Сложнее) Сделайте myreverse — печатает строки **в обратном порядке** (последняя первой). Намёк: для этого нужно прочитать **весь файл** в память, потом печатать reverse.

32.12 Часть III закрыта

Подведём итоги Части III. Глава 26: процессы, fork/exec/wait. Глава 27: pipes и dup2. Глава 28: сигналы. Глава 29: парсер с кавычками и редиректами. Глава 30: builtins и история. Глава 31: свои утилиты.

В коде `demo-shell/`: - ~800 строк shell-логики в `src/`. - ~250 строк трёх утилит в `utils/`. - ~1100 строк всего.

Что вы освоили:

- **POSIX API**: `fork/exec/wait/pipe/dup2/open/read/write/close/chdir/getcwd/setenv/unsetenv/signal/sigact`
- **Парсинг текста**: токенизация с состояниями, парсинг команд.
- **Сигналы и асинхронность**: что такое `async-signal-safe`, как обрабатывать прерывания.
- **Управление процессами**: процессное дерево, `foreground groups` (упоминание), зомби.
- **Файловые дескрипторы**: что это, наследование при `fork/exec`, переназначение через `dup2`.
- **Unix-философия**: маленькие программы как фильтры, композиция через пайпы.

В резюме — теперь вы можете написать любую CLI-утилиту на C++. Прочитать конфиг, пройти по файлам, общаться с другими программами через стандартные потоки. Это **системное программирование**.

32.13 Что дальше

Часть IV — мини-СУБД `demo-db/` (главы 32-39). Низкоуровневая работа с диском: страничный файл, B+tree-индекс, `write-ahead log`, парсер SQL. Это про **производительность и корректность**: что значит «база данных надёжна» на уровне байтов.

После — **Часть V** (TCP-чат, главы 40-47) — сети, многопоточность, реактор-цикл.

И **Часть VI** (C++17 как бонус, главы 48-51) — `optional/variant/filesystem/string_view`.

Дальше — глава 32, старт мини-СУБД.

Глава 33

Глава 32. Бинарный формат данных

Начинается **Часть IV — мини-СУБД**. Восемь глав, в которых построим маленькую, но настоящую базу данных. С файлом на диске. С B+tree-индексом. С журналом транзакций. С мини-SQL и REPL. Это самая «низкоуровневая» часть книги: байты, страницы, выравнивание, бинарные форматы.

Если в Части II (RPG) мы хранили сейв в **текстовом** виде с эскейпингом, в Части IV это **не подойдёт**: миллион записей в текстовом виде — мегабайты на диске, и парсинг каждый раз. **Бинарный формат** даёт компактность и нулевую цену чтения. Но за это надо платить: разобраться с выравниванием, endianness, packed-структурами.

В этой главе мы заложим **фундамент**: код, который читает и пишет числа и строки в бинарный буфер. Покажу, чего избегать (raw memcpu struct) и почему. Дальше — стройные страницы, B+tree, и всё остальное.

33.1 Текст vs бинарь

Сравним хранение одной записи: id=42, name="Alice".

Текстовый формат (CSV, наш сейв-формат, JSON):

```
42,Alice
```

9 байт. Чтобы прочитать id, надо: разделить по ,, конвертировать "42" в int через stoi. Конвертация — это парсинг символов один за одним, мини-state machine.

Бинарный формат:

```
2A 00 00 00 05 00 00 00 41 6C 69 63 65  
[--- id ---] [-- len ---] [---- "Alice" ----]
```

13 байт. Чтобы прочитать `id`, прочесть 4 байта, **сразу** интерпретировать как `int32_t`. Никакого парсинга. Скорость — наносекунды.

Парадоксально, бинарь иногда **больше** на маленьких числах (42 в тексте — 2 байта; в бинаре — 4). Но на больших — обратное (12345678 в тексте — 8 байт; в бинаре — 4). И главное: чтение бинаря **не парсится**. Это критично для миллионов записей в СУБД.

Минусы бинаря: - Нечитаемо глазами. - Зависит от `endianness`. - Требуется версионирования (изменишь поле — старый файл сломается). - Не редактируется текстовым редактором.

В нашей мини-СУБД — бинарный, потому что: - Скорость критична. - Мы делаем настоящий движок, а не игрушку. - Понимание бинарных форматов — нужный навык.

33.2 Что такое `struct` в памяти

Возьмём:

```
struct Row {
    std::uint32_t id;
    std::uint16_t age;
    std::uint64_t balance;
};
```

Naïve ожидание: $4 + 2 + 8 = 14$ байт. **Не так.**

Реальное размещение:

id (4 bytes)	age (2 bytes)	PADDING (2 bytes)	balance (8 bytes)	
0	4	6	8	16

Компилятор **добавил padding** в 2 байта после `age`. Размер `sizeof(Row)` = 16, не 14.

Зачем? **Выравнивание** (alignment). Каждый тип имеет требование к адресу: - `uint8_t` — любой адрес (выравнивание 1). - `uint16_t` — адрес кратный 2. - `uint32_t` — адрес кратный 4. - `uint64_t` — адрес кратный 8.

Это не прихоть компилятора. На некоторых процессорах (ARM до v8) обращение к **невыровненному** числу — это **fault** (программа падает). На x86 — работает, но **медленнее** (нужно несколько обращений к памяти).

Поэтому компилятор раскладывает поля **так, чтобы каждое было выровнено**. Если для этого надо вставить пустые байты — вставляет.

`balance` начинается с адреса 8 (кратно 8 — выравнивание `uint64_t`). После `age` на адресе 6 нужно «прыгнуть» на 8, отсюда 2 байта `padding`.

В конце структуры тоже может быть padding — чтобы при размещении в массиве каждый следующий элемент тоже был выровнен.

33.3 alignof, alignas, offsetof

C++ даёт операторы для проверки:

```
#include <cstddef>

std::cout << sizeof(Row);           // 16
std::cout << alignof(Row);          // 8 (по самому большому полю)
std::cout << offsetof(Row, id);     // 0
std::cout << offsetof(Row, age);    // 4
std::cout << offsetof(Row, balance); // 8 (НЕ 6!)
```

alignof(T) — требование к выравниванию типа. offsetof(struct, field) — смещение поля от начала структуры в байтах.

Можно **принудительно увеличить** выравнивание через alignas:

```
struct alignas(64) CacheLine {
    int data;
};

std::cout << sizeof(CacheLine); // 64 (выровнено и дополнено)
```

Используется когда вам нужны cache-line-выровненные структуры (для производительности).

Уменьшить выравнивание стандартным C++ нельзя. Но многие компиляторы дают расширения:

33.4 Packed-структуры

GCC и Clang поддерживают __attribute__((packed)):

```
struct __attribute__((packed)) RowPacked {
    std::uint32_t id;
    std::uint16_t age;
    std::uint64_t balance;
};

std::cout << sizeof(RowPacked); // 14 — без padding!
```

Все поля «упакованы» вплотную. Никакого padding.

MSVC использует `#pragma pack(push, 1) ... #pragma pack(pop)`.

Проблема packed: **обращение к полю — не-выровнено**:

```
RowPacked r;  
std::uint64_t b = r.balance; // на ARM может быть медленным или fault
```

Современные компиляторы делают «безопасную» сборку для packed-полей: разбивают чтение на несколько маленьких. Медленнее, чем обычное чтение, но работает на всех архитектурах.

Когда брать packed: когда вам нужно **точное соответствие** структуры памяти и формату файла/сети. Например, заголовок TCP-пакета определён стандартом — каждое поле строго по своему смещению.

Когда НЕ брать: для всего остального. Просто оставьте компилятору выравнивать. Padding — это копейки на структурах, но микро-секунды на доступе.

33.5 Проблема №1: raw struct dump

Соблазн: «у меня есть Row в памяти, могу же я просто записать его в файл?»

```
Row r{42, 25, 1000};  
std::ofstream out("data.bin", std::ios::binary);  
out.write(reinterpret_cast<const char*>(&r), sizeof(r));
```

И прочитать:

```
Row r;  
std::ifstream in("data.bin", std::ios::binary);  
in.read(reinterpret_cast<char*>(&r), sizeof(r));
```

Это **компилируется и работает** на той же машине. Но **никогда не делайте этого** в production. Причины:

33.5.1 1. Padding бит — это мусор

Записанная структура содержит **padding-байты**, которые компилятор не инициализирует. Они могут быть **чем угодно** — мусором из стека/кучи, остатками других данных. Возможно даже **конфиденциальной информацией** (если перед этим в этой памяти лежал пароль).

Для bit-точной сравнения двух структур в файле — нужна определённость. Padding делает её невозможной.

33.5.2 2. Размер зависит от платформы

`sizeof(long)` на Linux x86-64 = 8, на Windows x86-64 = 4. Если вы записали структуру с `long` на Linux — на Windows не прочтаете правильно.

Используйте `std::int32_t`, `std::int64_t` (из `<cstdint>`) — у них **фиксированный** размер на всех платформах. Это глава 8.

33.5.3 3. Endianness

Числа в памяти кодируются байтами. В каком **порядке**?

Little-endian: младший байт первый. `0x12345678` лежит как 78 56 34 12.

Big-endian: старший байт первый. `0x12345678` лежит как 12 34 56 78.

x86-64 (Intel/AMD) — **little-endian**. ARM64 (Apple Silicon, мобильные) — настраиваемая, обычно **little-endian**. Сетевой стандарт TCP/IP — **big-endian** (called «network byte order»).

Если вы записали `struct` на **little-endian** машине, прочитали на **big-endian** — числа развёрнуты. Будет хаос.

33.5.4 4. Алайнмент зависит от ABI

Even разные компиляторы на одной платформе могут иначе выравнивать структуры (если есть необычные типы). С GCC vs MSVC будут разные `sizeof` для одной и той же `struct`.

33.5.5 Вывод

Никогда не записывайте сырую structure через `memcpy/write` для обмена между процессами/машинами.

Что делать? Записывать поле за полем, в **фиксированном порядке, байт-за-байт**.

33.6 Правильный подход: explicit serialization

Пишем по байтам, явно. Никаких `struct-dump'ов`.

`include/binary.h`:

```
namespace db {  
  
inline void write_u32(std::vector<unsigned char>& out, std::uint32_t v) {  
    for (int i = 0; i < 4; ++i) {  
        out.push_back(static_cast<unsigned char>((v >> (i * 8)) & 0xFFu));  
    }  
}
```

```

    }
}

inline std::uint32_t read_u32(const unsigned char* p) {
    std::uint32_t v = 0;
    for (int i = 0; i < 4; ++i) {
        v |= static_cast<std::uint32_t>(p[i]) << (i * 8);
    }
    return v;
}
}

```

Разбор:

write_u32(v): пишем младший байт первым (little-endian). - $v \& 0xFF$ — младший байт. - $(v \gg 8) \& 0xFF$ — второй. - $(v \gg 16) \& 0xFF$ — третий. - $(v \gg 24) \& 0xFF$ — старший.

read_u32(p): читаем 4 байта, собираем обратно. - $p[0]$ это младший. - $p[1] \ll 8$ сдвигается на второй. - И так далее.

После этого read_u32(p) всегда даёт **то же число**, что мы записали через write_u32, на любой машине. **Эндианнес фиксирован.**

unsigned char (или uint8_t) для буферов — чтобы избежать UB при битовых операциях с signed char (где знаковый бит сложен).

Аналогично для u16/u64:

```

inline void write_u16(std::vector<unsigned char>& out, std::uint16_t v) {
    out.push_back(static_cast<unsigned char>(v & 0xFFu));
    out.push_back(static_cast<unsigned char>((v >> 8) & 0xFFu));
}

inline void write_u64(std::vector<unsigned char>& out, std::uint64_t v) {
    for (int i = 0; i < 8; ++i) {
        out.push_back(static_cast<unsigned char>((v >> (i * 8)) & 0xFFu));
    }
}

```

И строки — длина + байты:

```
inline void write_string(std::vector<unsigned char>& out, const std::string& s) {
    write_u32(out, static_cast<std::uint32_t>(s.size()));
    out.insert(out.end(), s.begin(), s.end());
}
```

Сначала 4-байтная длина, потом тело. Это **length-prefixed** формат. Лучше null-terminated (как C-строки) — длина известна заранее, не надо сканить.

33.7 Sign vs unsigned, signed numbers

Для **signed**-чисел `int32_t` есть нюанс. Они хранятся в дополнительном коде (глава 2). Если мы запишем `int32_t` через каст к `uint32_t`:

```
inline void write_i32(std::vector<unsigned char>& out, std::int32_t v) {
    write_u32(out, static_cast<std::uint32_t>(v));
}

inline std::int32_t read_i32(const unsigned char* p) {
    return static_cast<std::int32_t>(read_u32(p));
}
```

— байты сохраняются правильно. Например, `-1` в дополнительном коде = `0xFFFFFFFF`. Каст в `uint32_t` даёт `0xFFFFFFFF`. Записали 4 байта `FF FF FF FF`. Прочли — обратно `0xFFFFFFFF`, каст в `int32_t` = `-1`.

Работает **благодаря дополнительному коду**. Если бы числа кодировались иначе (вспомните прямой код из главы 2) — пришлось бы знаки обрабатывать отдельно. На современных платформах — все two's complement, дополнительный код. Безопасно.

33.8 htonl и подобные

POSIX даёт функции для **network byte order** (big-endian):

```
#include <arpa/inet.h>

std::uint32_t nbo = htonl(0x12345678); // host-to-network long
std::uint32_t hbo = ntohl(nbo);       // обратно
```

`htonl` (host-to-network long) переводит из локального порядка в сетевой. На big-endian машинах — no-op. На little-endian — переставляет байты.

Полезно, если вы делаете протокол, использующий сетевой порядок (как TCP, HTTP/2 фреймы). Для **локального** файла наша СУБД использует little-endian — её формат предназначен для одной машины.

33.9 Пример: всё вместе

src/main.cpp (демо):

```
#include "binary.h"
#include <iostream>
#include <iomanip>
#include <vector>

int main() {
    std::vector<unsigned char> buf;
    db::write_u32(buf, 0x12345678);
    db::write_u64(buf, 0xCAFEFABEDEADBEEF);
    db::write_string(buf, "hello");

    std::cout << "Сериализовано " << buf.size() << " байт:\n";
    for (std::size_t i = 0; i < buf.size(); ++i) {
        std::cout << std::hex << std::setw(2) << std::setfill('0')
                    << static_cast<int>(buf[i]) << " ";
        if ((i + 1) % 16 == 0) std::cout << "\n";
    }
    std::cout << "\n";
}
```

Вывод:

Сериализовано 21 байт:

78 56 34 12 ef be ad de be ba fe ca 05 00 00 00

68 65 6c 6c 6f

Разбор: - 78 56 34 12 — 0x12345678 в little-endian. - ef be ad de be ba fe ca — 0xCAFEFABEDEADBEEF в little-endian. - 05 00 00 00 — длина строки (5) в little-endian. - 68 65 6c 6c 6f — буквы hello в ASCII.

Hexdump читается легко, отладка очевидна.

33.10 Hexdump для отладки

Утилита `xxd` или `hexdump` показывает бинарные файлы в hex+ASCII:

```
$ xxd data.bin
00000000: 7856 3412 efbe adde beba feca 0500 0000  xV4.....
00000010: 6865 6c6c 6f                                hello
```

Слева — адреса, потом hex-байты, справа — ASCII (точка для непечатных).

Это **must-have** для отладки бинарных форматов. Когда что-то не парсится — открываете в `xxd`, смотрите глазами, ищете ошибку.

Аналог на macOS — `xxd` (тоже работает) или `hexdump -C`.

В коде на C++ можно написать свою функцию `hexdump(buf)` за 20 минут — посмотрите примеры на креп-страницах.

33.11 Versioning формата

Прежде чем разворачивать парсер, **версионите**. Самое начало файла — **magic number** и **version**:

```
mydb v1\n
[содержимое]
```

Или бинарно:

```
4D 44 42 01      ← magic "MDB" + version 1
[содержимое]
```

Парсер сначала проверяет: магия совпадает? версия моя? Если нет — отказ.

Когда выпустите v2 — добавьте поля, новый формат, версию 2. Если читаете файл с версией 1 — либо отказываете, либо умеете читать v1 через специальную функцию.

Это **критическая инфраструктура** долгоживущих форматов. У нас уже было в save-формате RPG (глава 24). В мини-СУБД — обязательно.

33.12 Что увидите в следующих главах

В этой главе — фундамент. Дальше:

- **Глава 33:** страничный файл. Файл — массив страниц по 4 KiB. Каждая страница — `vector<unsigned char>(4096)`. Page cache: LRU, чтобы часто используемые страницы лежали в памяти.
- **Глава 34:** B+tree. Структура для упорядоченного индексирования. Разбиение узлов при заполнении, слияние при опустошении.
- **Глава 35:** WAL (Write-Ahead Log). Журнал «сначала запиши что собираешься делать, потом делай». Восстановление после краха.
- **Главы 36-39:** мини-SQL, индексы, REPL, бенчмарки.

Каждая глава вносит свой слой, фундамент остаётся — байтовый формат.

33.13 Главные правила главы

1. **Никогда не сохраняйте struct через memcpu** в файл. Padding, endianness, ABI — всё против вас.
2. **Сериализуйте поле за полем.** Явный код, явный порядок байтов.
3. `std::int32_t/std::uint32_t` вместо `int/long` — фиксированный размер.
4. **Little-endian** как default для локальных форматов. Network byte order — для сети.
5. **Length-prefixed строки** — длина (4 байта) + содержимое. Не null-terminated.
6. **Magic number и version** в начале файла. Обязательно.
7. `alignof/offsetof/sizeof` — ваши друзья при дизайне. Изучите, что лежит в памяти.
8. `xxd` для отладки. Поверьте, понадобится часто.

33.14 Маленькое упражнение

1. Соберите `demo-db` и запустите. Изучите вывод. Сопоставьте hex-байты с числами.
2. Возьмите вывод программы, сохраните в файл (например, через `write` в файл). Откройте через `xxd file.bin`.
3. Напишите `write_i32` и `read_i32` для signed. Проверьте на `-1`, `INT32_MIN`, `INT32_MAX`.
4. Создайте `struct Row { uint32_t id; uint16_t age; uint64_t balance; };`. Узнайте `sizeof`, `alignof`, `offsetof` для каждого поля. Объясните padding.
5. Добавьте `__attribute__((packed))`. Узнайте `sizeof` снова. Что изменилось?
6. Запишите 1000 `Row` через **explicit serialization** (поле за полем) — узнайте размер. То же через **raw memcpu** — узнайте размер. Сколько разница?
7. (Сложнее) Напишите `hexdump(const void* data, size_t len)` — функция, печатающая hex+ASCII в стиле `xxd`.

8. (Сложнее) Реализуйте `write_varint` — переменнo-длинная кодировка чисел (как в `protobuf`). Маленькие числа — 1 байт, большие — больше. На реальной СУБД часто более компактно.

33.15 Что дальше

Глава 33 — **страничный файл и кэш**. Базы хранят данные **страницами** фиксированного размера (4 KiB обычно). Это упрощает работу с диском и менеджером памяти. В этой главе сделаем `PageManager` — компонент, который умеет читать/писать страницы файла с `in-memory` кэшем.

Дальше — B+tree (34), WAL (35), мини-SQL (36-37), REPL (38), бенчмарки (39). Часть IV — концентрат низкоуровневой работы.

Глава 34

Глава 33. Страничный файл и LRU-кэш

В прошлой главе мы научились писать и читать байты в буфер. Сейчас — следующий уровень: файл базы данных как **массив страниц** фиксированного размера, с **кэшем** в памяти. Это базовая инфраструктура **любой настоящей СУБД** — PostgreSQL, MySQL, SQLite, ClickHouse. Без неё было бы катастрофически медленно.

К концу главы у нас работает PageManager: открывает файл, разбивает на страницы по 4 KiB, читает/пишет страницы через кэш, синхронизирует на диск через fsync.

34.1 Зачем страницы

Альтернатива страницам: «база данных как набор записей произвольного размера». Каждая запись на своём смещении в файле. Чтобы добавить запись — пишем на следующее свободное место. Чтобы удалить — оставляем «дыру».

Проблемы такого подхода: 1. **Фрагментация**: после многих delete файл превращается в «швейцарский сыр». 2. **Размер записи**: некоторые большие, некоторые маленькие. Поиск свободного места — задача. 3. **Кэш**: что кэшировать? Записи разного размера сложнее держать в LRU. 4. **Диск работает блоками**: даже если читаешь 1 байт — ОС всё равно прочтёт целый блок (обычно 4 KB).

Страничный подход: - Файл делится на **страницы** одинакового размера (обычно 4 KB). - Каждая страница имеет **id** — индекс в файле. - Внутри страницы — что угодно (записи, узлы дерева, метаданные). - Все IO — целыми страницами.

Преимущества: - **Совместимость с ОС**: размер страницы СУБД = размер блока диска = размер страницы памяти. Никаких лишних чтений. - **Простой кэш**: страницы все одного размера,

легко иметь LRU. - **Атомарность**: запись 4 KB гарантированно атомарна на современных дисках (страница записывается целиком или не записывается). - **Простой алгоритм allocate**: новая страница — `page_count`, инкремент.

34.2 PAGE_SIZE = 4096

Почему именно 4 KiB?

- **Размер страницы памяти** в большинстве ОС — 4 KB. Ядро управляет памятью страницами по 4 KB. Если ваша страница совпадает — `mmap` и `virtual memory` работают эффективно.
- **Размер блока файловой системы** — обычно 4 KB (`ext4`, `APFS`, `NTFS`). Меньшая запись — всё равно округлится до блока.
- **Размер страницы SSD** — 4 KB или 16 KB. Запись меньше страницы — `write amplification`.

Некоторые СУБД используют 8 KB или 16 KB (`PostgreSQL` по умолчанию 8 KB, `MySQL InnoDB` — 16 KB). Это компромисс: больше страница = больше данных за один IO, но больше `overhead` на маленькие операции.

Мы — 4 KB. Стандарт для учебной задачи.

34.3 Адресация страниц

Файл: 0123456789...

page0	page1	page2	...
↑	↑	↑	
0	4096	8192	...

Page 0 — байты 0..4095. Page 1 — байты 4096..8191. И так далее. Адрес страницы в файле:

```
off_t offset = static_cast<off_t>(page_id) * PAGE_SIZE;
```

`page_id` имеет тип `PageId` — у нас `uint32_t`. Это поддерживает до 4 миллиардов страниц = 16 TB на 4 KB страницах. Достаточно.

Если файл «закончился» (например, попросили читать страницу 100, а в файле только 10) — это **ошибка**. У нас `read_page` проверяет `pid < page_count_`.

34.4 pread / pwrite

POSIX даёт две системные вызовы для IO по конкретному смещению:

```
ssize_t pread(int fd, void* buf, size_t count, off_t offset);
ssize_t pwrite(int fd, const void* buf, size_t count, off_t offset);
```

Их преимущество перед read/write + lseek:

1. **Атомарны** для многопоточных приложений. read/write + lseek — это две операции; между ними другой поток может изменить позицию.
2. **Не меняют файловую позицию.** После pwrite cursor где был.

Для нашей СУБД это пока не критично (мы однопоточные), но привычка pread/pwrite правильная.

Возвращаемое значение — число реально записанных/прочитанных байт. Может быть **меньше** запрошенного (short read/write). Цикл обязателен:

```
ssize_t written = 0;
while (written < PAGE_SIZE) {
    ssize_t n = pwrite(fd, buf + written, PAGE_SIZE - written, off + written);
    if (n < 0) {
        if (errno == EINTR) continue;    // сигнал
        throw PageError(...);
    }
    written += n;
}
```

EINTR (interrupted) — прерывание сигналом, повторяем. Это та же история, что в muscat (глава 31).

34.5 Кэш — зачем

Каждый pread — это **системный вызов** + потенциальное обращение к диску. Время: - Если страница в page cache ОС — ~микросекунда. - Если на SSD — ~100 микросекунд. - Если на HDD — ~10 миллисекунд.

При активной работе с одной и той же страницей миллион раз — это много. Решение — **кэш в нашей программе**: держим часто используемые страницы в памяти.

ОС тоже имеет page cache. Но: - ОС-кэш — глобальный, разделяется всеми программами. Может вытеснить наши страницы. - Наш кэш — **знает специфику нашего workflow** (B+tree часто посещает корень). - Между ОС-кэшем и пользовательским кодом есть syscall — бесплатно.

Поэтому свой кэш — не дублирование, а ускорение.

34.6 LRU как стратегия

Что вытеснять, когда кэш заполнен?

Самый частый подход — **LRU** (Least Recently Used): «выбросить ту страницу, которую дольше всего не трогали». Гипотеза: если страница не нужна давно, скорее всего и дальше не понадобится.

Альтернативы: - **FIFO**: вытесняем самую старую (по времени попадания в кэш). - **LFU**: вытесняем самую редко используемую. - **Random**: случайно.

LRU — золотая середина. Хорошо для большинства реальных workload'ов. PostgreSQL использует усовершенствованный LRU («clock sweep»), MySQL — LRU с защищённой/непривилегированной половиной. Мы — простой LRU.

34.6.1 Реализация LRU

Нужно: 1. Быстро найти страницу по id — `unordered_map`. 2. Быстро понять, что трогаем сейчас — переместить в «начало». Это `list` + переустановка указателя.

Связка `list` + `map`:

```
std::list<PageId> lru_; // фронт = недавнее
std::unordered_map<PageId, std::pair<Entry, LruIter>> cache_;
```

`cache_` мапит `page_id` в (содержимое страницы + итератор на её позицию в LRU). При обращении:

```
void PageManager::touch(PageId pid) {
    auto it = cache_.find(pid);
    if (it == cache_.end()) return;
    lru_.erase(it->second.second);
    lru_.push_front(pid);
    it->second.second = lru_.begin();
}
```

`lru_.erase(iterator)` — $O(1)$ для `list`. `push_front` — $O(1)$. Поиск в `map` — $O(1)$. Итого `touch` — **$O(1)$** , что важно для горячих путей.

При вытеснении:

```
void PageManager::evict_one_if_full() {
    if (cache_.size() < cache_size_) return;
    PageId victim = lru_.back(); // самая старая
    auto it = cache_.find(victim);
```

```
flush_entry(victim, it->second.first); // если dirty — записать
lru_.pop_back();
cache_.erase(it);
}
```

Берём с хвоста LRU (самую старую), сбрасываем на диск (если она грязная), удаляем из map.

34.7 Dirty pages

Когда мы пишем в страницу через кэш, мы **не** идём на диск сразу. Помечаем `dirty = true` и пишем только в кэш. Когда страница выкидывается из кэша **или** при `sync()` — пишем на диск.

```
void PageManager::write_page(PageId pid, const Page& page) {
    Entry& e = load_into_cache(pid);
    e.page = page;
    e.dirty = true;
}
```

Это **write-back** стратегия — пишем «когда удобно». Альтернатива — **write-through** (сразу на диск). Write-back быстрее для частых обновлений одной страницы (один диск-IO вместо ста).

Цена: если программа упадёт до flush'a — данные потеряются. Защита — **журнал** (WAL, глава 35). Каждое изменение сначала пишется в WAL, потом в страницу. После краша WAL переигрываем.

34.8 fsync — durability

Записали через `pwrite` 4 КБ на диск. Готово? **Нет**.

ОС держит **свой** буфер записи. Запись из ОС в буфер диска — мгновенно. Запись из буфера в флэш — асинхронно. Если в этот момент пропадёт питание — данные потеряются, хотя `pwrite` уже вернул.

`fsync(fd)` — заставить ОС сбросить **все** свои буферы для этого файла на физический диск. Только после `fsync` можно сказать «данные точно на диске».

`fsync` дорогой — миллисекунды, иногда десятки. Делать его на каждый `pwrite` сильно тормозит. Стратегия: **периодические fsync** или **fsync только на commit** (после транзакции).

В нашем `sync()`:

```

void PageManager::sync() {
    for (auto& kv : cache_) {
        flush_entry(kv.first, kv.second.first);
    }
    if (::fsync(fd_) < 0) {
        throw PageError(...);
    }
}

```

Сначала сбрасываем все dirty страницы через `pwrite`. Потом `fsync`. Теперь данные гарантированно на диске.

В деструкторе `PageManager` — тоже `sync`. Но **глотаем** исключения: деструктор бросать не должен. Если `sync` упал — в лучшем случае данные на диске недополнены. Решение «правильное» — отдельный явный `close()` метод, который пользователь вызывает и проверяет результат. Мы упрощаем.

`fdatasync` — оптимизированный вариант: сбрасывает данные, но не метаданные (`mtime`, `size`). Если вам нужен только `sync` содержимого — `fdatasync` чуть быстрее.

34.9 Класс PageManager

`include/page_manager.h`:

```

class PageManager {
public:
    PageManager(const std::string& path, std::size_t cache_size = 64);
    ~PageManager();

    PageManager(const PageManager&) = delete;
    PageManager& operator=(const PageManager&) = delete;

    Page read_page(PageId pid);
    void write_page(PageId pid, const Page& page);
    PageId allocate_page();
    PageId page_count() const { return page_count_; }
    void sync();

private:
    struct Entry {

```

```

    Page page;
    bool dirty = false;
};

int fd_;
PageId page_count_;
std::size_t cache_size_;

std::list<PageId> lru_;
std::unordered_map<PageId, std::pair<Entry, LruIter>> cache_;

void evict_one_if_full();
void flush_entry(PageId pid, Entry& e);
Entry& load_into_cache(PageId pid);
void touch(PageId pid);
};

```

Запрет копирования через `= delete` — `PageManager` владеет файловым дескриптором, копировать его (= иметь два владельца `fd`) приведёт к проблемам.

34.9.1 Открытие файла

```

PageManager::PageManager(const std::string& path, std::size_t cache_size)
: fd_(-1), page_count_(0), cache_size_(cache_size) {
    fd_ = ::open(path.c_str(), O_RDWR | O_CREAT, 0644);
    if (fd_ < 0) {
        throw PageError("не открыт: " + path + ": " + std::strerror(errno));
    }
    struct stat st;
    if (::fstat(fd_, &st) < 0) {
        ::close(fd_);
        throw PageError(std::string("fstat: ") + std::strerror(errno));
    }
    if (st.st_size % PAGE_SIZE != 0) {
        ::close(fd_);
        throw PageError("файл не выровнен по PAGE_SIZE");
    }
}

```

```

    page_count_ = static_cast<PageId>(st.st_size / PAGE_SIZE);
}

```

Открываем (создаём при отсутствии), читаем размер через `fstat`, проверяем, что он кратен `PAGE_SIZE` (иначе файл повреждён). Количество страниц = `size / PAGE_SIZE`.

34.9.2 Загрузка страницы

```

PageManager::Entry& PageManager::load_into_cache(PageId pid) {
    auto it = cache_.find(pid);
    if (it != cache_.end()) {
        touch(pid);
        return it->second.first;
    }

    evict_one_if_full();

    Entry e;
    e.page = make_page();
    e.dirty = false;

    off_t off = static_cast<off_t>(pid) * static_cast<off_t>(PAGE_SIZE);
    // pread в цикле с обработкой short read и EINTR
    // ...

    lru_.push_front(pid);
    auto inserted = cache_.emplace(pid, std::make_pair(std::move(e), lru_.begin()));
    return inserted.first->second.first;
}

```

Если страница есть в кэше — `touch` и вернуть. Если нет — `evict_one_if_full` (освободить место), потом загрузить с диска, потом добавить в `map+list`.

`make_page()` — функция из `include/page.h`, создаёт `vector<unsigned char>(PAGE_SIZE, 0)`.

34.9.3 Выделение новой страницы

```

PageId PageManager::allocate_page() {
    PageId new_id = page_count_;
    page_count_++;
}

```

```

Entry e;
e.page = make_page();
e.dirty = true;      // новая = dirty, чтобы записаться при sync

evict_one_if_full();
lru_.push_front(new_id);
cache_.emplace(new_id, std::make_pair(std::move(e), lru_.begin()));
return new_id;
}

```

Просто инкрементируем счётчик, добавляем пустую (нулевую) страницу в кэш, помечаем dirty. При следующем sync() или evict — страница появится в файле.

Это **lazy allocate** — файл расширяется только при flush. Альтернатива — сразу делать ftruncate(fd, new_size). Мы лениво.

34.10 Демо

src/main.cpp создаёт PageManager, выделяет 3 страницы, записывает разное, закрывает. Открывает заново, читает обратно:

```

$ ./build/mydb
=== mydb (глава 33: страничный файл) ===

Выделено страниц: 0, 1, 2 (page_count=3)
Записано и fsync'ено.

Открыли заново. page_count=3
page 0 first 18 bytes: 'Hello from page 0!'
page 1 bytes 0..4: 0 1 2 3 ...
page 2 bytes 0..4: de ad be ef

$ ls -la data/demo.db
-rw-r--r-- 1 user user 12288 ... data/demo.db

```

Размер файла 12288 = 3 × 4096. Каждая страница на своём смещении.

xxd data/demo.db:

```

00000000: 4865 6c6c 6f20 6672 6f6d 2070 6167 6520  Hello from page
00000010: 3021 0000 0000 0000 0000 0000 0000 0000  0!.....

```



```
...
00001000: 0001 0203 0405 0607 0809 0a0b 0c0d 0e0f  ....
```

```
...
00002000: dead beef 0000 0000 0000 0000 0000 0000  ....
```

Видно: - Page 0 на offset 0: Hello from page 0!. - Page 1 на offset 0x1000 (4096): 00 01 02 03
- Page 2 на offset 0x2000 (8192): DE AD BE EF.

Точно как задумано.

34.11 Cache size: сколько страниц держать

В моём демо — 2 (PageManager pm(path, 2)). Поэтому при работе с 3 страницами кэш постоянно вытесняет одну. Это специально для теста.

В реальной СУБД — гораздо больше. PostgreSQL по умолчанию ~16 тысяч страниц (shared_buffers 128 MB / 8 KB). MySQL InnoDB по умолчанию буфер 128 MB.

Чем больше кэш — тем меньше дисковых IO. Но и больше памяти.

В нашей мини-СУБД кэш — параметр конструктора. По умолчанию 64 страницы = 256 KB. Маленький, но достаточный для учебной работы.

34.12 Тонкости и неприятности

34.12.1 Page tearing

При записи 4 KB на диск может произойти **page tearing** — потеря питания на середине записи. На SSD обычно записи 4 KB атомарны на уровне controller'a, но не гарантировано на всех уровнях.

Защита — WAL (глава 35): сначала пишем full image страницы в журнал. При восстановлении после краха — берём данные из журнала.

PostgreSQL называет это «full page writes» — каждое первое изменение страницы в checkpoint пишет всю страницу в WAL. Защита от page tearing.

34.12.2 Write amplification

SSD имеет страницы 4-16 KB и блоки стираемые 128 KB - 4 MB. Любая запись 4 KB на «грязный» блок требует: 1. Прочитать весь блок (4 MB). 2. Стереть. 3. Записать обратно с изменением.

Это **write amplification** — реальная запись больше логической. Сокращает время жизни SSD.

Хорошие СУБД учитывают этот эффект — пишут большими порциями (`sequential writes`), редко делают `fsync`, используют SSD-friendly алгоритмы.

34.12.3 O_DIRECT

Линукс предоставляет флаг `O_DIRECT` при `open`, который **обходит page cache ОС**. Чтения и записи идут напрямую между программой и диском.

Зачем? Если у вас свой большой кэш — ОС-кэш только тратит память. Также убирается двойное копирование (ОС → кэш ОС → кэш программы).

Мы не используем `O_DIRECT` — это требует выровненной памяти, выровненных смещений, и осторожности. Современные NVMe настолько быстры, что `page cache` ОС редко мешает.

34.13 Главные правила главы

1. **Страницы фиксированного размера** — основа всех СУБД. 4 KB стандарт.
2. **`pread/pwrite`** вместо `read+lseek/write+lseek` — атомарно, проще.
3. **LRU-кэш** с `list + unordered_map` — $O(1)$ операции.
4. **`dirty flag`** + `write-back` — пишем на диск только при `evict/sync`.
5. **`fsync` для `durability`**. Без него запись может потеряться при крахе.
6. **`fsync` дорог** — не на каждый `write`. Группируйте.
7. **`O_RDWR` | `O_CREAT`** для открытия — открываем или создаём.
8. **Запрещайте копирование** для классов с `fd` через `= delete`.

34.14 Маленькое упражнение

1. Запустите `./build/mydb`. Изучите `data/demo.db` через `xxd`. Найдите все три страницы.
2. Увеличьте `cache_size` в `main` на 100 — увидите, что вытеснений нет (`page_count=3 < 100`).
3. Уменьшите `cache_size` до 1 — каждое обращение к новой странице будет вытеснять предыдущую. Программа всё равно должна работать.
4. Добавьте в `PageManager` метод `int hit_count()` и `int miss_count()` — счётчики попаданий и промахов в кэш. Замерьте на тысячах обращений.
5. Запустите без `sync()` в `main` (закомментируйте). Запустите снова и прочитайте — что увидите?
6. (Сложнее) Замените LRU на FIFO. Сравните `hit rate` на случайном `workflow` (1000 обращений к `random page` из 100).

7. (Сложнее) Сделайте измерение производительности: 100k случайных записей через Page-Manager, замерьте время с разными `cache_size`: 1, 16, 64, 256.
8. (Сложнее) Прочитайте `man 2 fsync` и `man 2 fdatasync`. Поменяйте `fsync` на `fdatasync` — что-то изменилось в работе или скорости?

34.15 Что дальше

Глава 34 — **B+tree**. Это структура для **упорядоченного** хранения. На каждой странице — узел дерева. При вставке узлы разбиваются, при удалении — сливаются. Поиск, диапазонные запросы — все $O(\log N)$.

Дальше — WAL (35), мини-SQL (36), индексы и план запроса (37), REPL (38), бенчмарки (39). Часть IV — самая «системная» в книге.

Глава 35

Глава 34. B+tree

Самая алгоритмически тяжёлая глава Части IV. **B+tree** — структура данных, лежащая в основе **любой** настоящей реляционной СУБД: PostgreSQL, MySQL InnoDB, SQLite, Oracle. Это упорядоченное дерево, оптимизированное под дисковое хранение.

К концу главы у нас работает B+tree поверх нашего PageManager. Можно вставлять (`int64` ключ $\rightarrow$ `int64` значение), искать, обходить в порядке возрастания. Дерево живёт в файле — открыли программу, прочитали, всё работает.

35.1 Зачем B+tree

Альтернативные структуры:

std::map (красно-чёрное дерево) — балансируемое бинарное. $O(\log N)$ операции. **Но:** каждый узел — отдельная аллокация на куче. Узлов миллионы — миллионы малых аллокаций. Кэш-промахи. Нельзя удобно сохранить на диск.

Hash table (`std::unordered_map`). $O(1)$ операции. **Но:** нет порядка. Нельзя сделать range query «все ключи от 100 до 200». Нет stability при росте.

Sorted array. Поиск $O(\log N)$. **Но:** вставка $O(N)$ — сдвигать всё. На миллион записей — нереально.

B+tree — компромисс. Дерево, **но широкое и неглубокое:** - Каждый узел вмещает много ключей (сотни на нашем 4 KB). - Высота — $\log_M(N)$, где M — fan-out. На миллион записей и fan-out 250 — высота ~ 3 . - На диске **узел = страница**. Одно IO — это переход на один уровень глубже.

При поиске миллиона ключей на 4 KB страницах: ~ 3 IO. На SSD — несколько миллисекунд. Bad: если бы хранили в `std::map` на диске — миллионы IO.

35.2 B-tree vs B+tree

Из семейства «многозначных деревьев» популярны два:

B-tree: значения лежат **во всех узлах** (внутренних и листьях). При поиске можно остановиться в середине, если нашёл.

B+tree: - **Внутренние узлы** хранят только **ключи и указатели** на детей. Никаких значений.
- **Листья** хранят **ключи и значения**. - **Листья связаны** в список (next-pointer) — для эффективного range scan.

Преимущества B+tree: - Внутренние узлы компактнее (нет значений) → больше fan-out → меньше высота. - Range queries через linked-leaf — O(K) для K результатов. - Все «найденные» данные на одной глубине — стабильное время доступа.

Минус: каждый ключ хранится **дважды** — в листе и (если разделитель) во внутренних узлах. На миллионах это лишняя память. Compromise приемлемый.

В нашей мини-СУБД — **B+tree**.

35.3 Структура узла

Каждый узел занимает **одну страницу**. Layout:

```
[0..1]  u16 type (0=Internal, 1=Leaf)
[2..3]  u16 num_keys
[4..7]  u32 next_leaf (только для Leaf; 0 если последний)
[8..15] зарезервировано
[16..]  keys: int64[num_keys]
[...]   values: int64[num_keys]    (если Leaf)
        или
        children: PageId[num_keys + 1]    (если Internal)
```

Internal node имеет num_keys ключей и num_keys + 1 детей. Ключ i — это **граница** между детьми i и i+1: - Дети < keys[0] идут в children[0]. - Дети >= keys[0] && < keys[1] идут в children[1]. - ... - Дети >= keys[last] идут в children[last + 1].

Leaf node имеет num_keys ключей и столько же значений. Плюс next_leaf — id следующего листа в порядке возрастания. Нулевой next_leaf = последний.

35.4 MAX_KEYS — порядок

Сколько ключей помещается в одну страницу?

Грубо: $4096 - 16 \text{ (header)} = 4080$ байт на тело. Для leaf: $\text{keys} + \text{values} = 16$ байт на пару. Помещается $4080 / 16 = 255$ пар. Для internal: 8 байт ключа + 4 байта PageId на «слот». Помещается ~ 340 ключей.

Большой fan-out = неглубокое дерево.

Для нашей учебной реализации я ставлю $\text{MAX_KEYS} = 4$. Почему? Чтобы splits происходили часто, на 10-20 вставках. Тогда видно, как дерево растёт.

В production надо выставить $\text{MAX_KEYS} \approx 250$. Меняется одна константа.

35.5 Поиск

Алгоритм:

1. Начинаем с корня.
2. Если узел — Leaf: бинарный поиск ключа. Найден — возвращаем значение, не найден — false.
3. Если Internal: бинарный поиск *upper_bound* по ключам. Получаем индекс ребёнка. Спускаемся туда.

```
PageId BPlusTree::find_leaf(std::int64_t key, std::vector<PageId>* path) const {
    PageId pid = root_page_id_;
    while (true) {
        NodeData n = read_node(pid);
        if (n.type == Leaf) return pid;
        if (path) path->push_back(pid);

        auto it = std::upper_bound(n.keys.begin(), n.keys.end(), key);
        std::size_t idx = static_cast<std::size_t>(it - n.keys.begin());
        pid = n.children[idx];
    }
}
```

`std::upper_bound` находит первое значение $> \text{key}$. Если все меньше — возвращает `end()`. Индекс этого position — индекс правильного ребёнка.

`path` — стек посещённых внутренних узлов. Нужен для вставки (split может потребовать вставку в parent).

`find` — это `find_leaf` + linear search в листе:

```
bool BPlusTree::find(std::int64_t key, std::int64_t& out_value) const {
    PageId leaf_pid = find_leaf(key, nullptr);
    NodeData leaf = read_node(leaf_pid);
    auto it = std::lower_bound(leaf.keys.begin(), leaf.keys.end(), key);
    if (it == leaf.keys.end() || *it != key) return false;
    out_value = leaf.values[static_cast<std::size_t>(it - leaf.keys.begin())];
    return true;
}
```

`std::lower_bound` находит первое $\geq$ `key`. Если оно равно — ключ есть, возвращаем значение. Иначе — нет.

Сложность: $O(\log_M N)$ IO + $O(\log M)$ сравнений на узел. На $N=1M$, $M=4$ это $\log_4(1M) \approx 10$ IO. С $M=250$ — ~ 2.5 IO.

35.6 Вставка

Это сложнее. Алгоритм:

1. **find_leaf** — спускаемся до листа, запоминая путь.
2. Вставляем (`key`, `value`) в лист, сортировка через `std::lower_bound`.
3. Если лист **переполнен** (`size > MAX_KEYS`) — **split**: разделяем на два.
4. Поднимаем **separator key** в `parent`.
5. Если `parent` тоже переполнен — **split parent**.
6. Если **split** добрался до корня — создаём новый корень.

```
void BPlusTree::insert(std::int64_t key, std::int64_t value) {
    std::vector<PageId> path;
    PageId leaf_pid = find_leaf(key, &path);
    NodeData leaf = read_node(leaf_pid);

    auto it = std::lower_bound(leaf.keys.begin(), leaf.keys.end(), key);
    std::size_t idx = static_cast<std::size_t>(it - leaf.keys.begin());

    if (it != leaf.keys.end() && *it == key) {
        // Replace.
        leaf.values[idx] = value;
        write_node(leaf_pid, leaf);
        return;
    }
}
```

```

leaf.keys.insert(leaf.keys.begin() + idx, key);
leaf.values.insert(leaf.values.begin() + idx, value);

if (leaf.keys.size() <= MAX_KEYS) {
    write_node(leaf_pid, leaf);
    return;
}

// Split.
std::int64_t sep = 0;
PageId right = split_leaf(leaf_pid, leaf, sep);
insert_in_parent(path, sep, right);
}

```

Замена существующего ключа — простой write. Иначе вставляем в нужную позицию.

35.6.1 Split leaf

```

PageId BPlusTree::split_leaf(PageId leaf_pid, NodeData& leaf,
                             std::int64_t& sep_key) {
    std::size_t mid = leaf.keys.size() / 2;

    NodeData right;
    right.type = Leaf;
    right.keys.assign(leaf.keys.begin() + mid, leaf.keys.end());
    right.values.assign(leaf.values.begin() + mid, leaf.values.end());
    right.next_leaf = leaf.next_leaf;

    leaf.keys.resize(mid);
    leaf.values.resize(mid);

    PageId right_pid = pm_.allocate_page();
    leaf.next_leaf = right_pid;

    write_node(leaf_pid, leaf);
    write_node(right_pid, right);

    sep_key = right.keys.front();
}

```



```
    return right_pid;
}
```

Делим пополам. **Все ключи с правой половины** идут в новый правый лист. Левый сохраняет только первую половину.

next_leaf правого узла = старый next_leaf (сохраняем цепочку). next_leaf левого = id правого (вставляем правого в цепочку).

Separator key для parent — первый ключ правого листа. В B+tree separator **копируется** в parent (не **выносится** как в B-tree).

35.6.2 Split internal

```
PageId BPlusTree::split_internal(PageId int_pid, NodeData& node,
                                std::int64_t& sep_key) {
    std::size_t mid = node.keys.size() / 2;
    sep_key = node.keys[mid];

    NodeData right;
    right.type = Internal;
    right.keys.assign(node.keys.begin() + mid + 1, node.keys.end());
    right.children.assign(node.children.begin() + mid + 1, node.children.end());

    node.keys.resize(mid);
    node.children.resize(mid + 1);

    PageId right_pid = pm_.allocate_page();
    write_node(int_pid, node);
    write_node(right_pid, right);
    return right_pid;
}
```

Для internal **средний ключ выносится в parent** (не копируется). Поэтому правый получает keys.begin() + mid + 1, не + mid. Это особенность: в B+tree leaf splits **копирует** separator, а internal splits — **выносят**.

35.6.3 Insert in parent

```

void BPlusTree::insert_in_parent(const std::vector<PageId>& path,
                                std::int64_t sep_key,
                                PageId right_child) {
    if (path.empty()) {
        // Корень разбился — создаём новый корень.
        NodeData new_root;
        new_root.type = Internal;
        new_root.keys.push_back(sep_key);
        new_root.children.push_back(root_page_id_);
        new_root.children.push_back(right_child);
        PageId new_root_pid = pm_.allocate_page();
        write_node(new_root_pid, new_root);
        root_page_id_ = new_root_pid;
        save_metadata();
        return;
    }

    PageId parent_pid = path.back();
    NodeData parent = read_node(parent_pid);

    auto it = std::upper_bound(parent.keys.begin(), parent.keys.end(), sep_key);
    std::size_t idx = static_cast<std::size_t>(it - parent.keys.begin());
    parent.keys.insert(parent.keys.begin() + idx, sep_key);
    parent.children.insert(parent.children.begin() + idx + 1, right_child);

    if (parent.keys.size() <= MAX_KEYS) {
        write_node(parent_pid, parent);
        return;
    }

    std::int64_t new_sep = 0;
    PageId new_right = split_internal(parent_pid, parent, new_sep);
    std::vector<PageId> grand_path(path.begin(), path.end() - 1);
    insert_in_parent(grand_path, new_sep, new_right);
}

```

Если path пуст — это корень разбился. Создаём новый корень с двумя детьми (старый корень слева, новый правый справа). Сохраняем root_page_id_.

Иначе берём parent из path, вставляем (sep_key, right_child) в правильную позицию. Если parent не переполнен — записываем. Иначе **рекурсивно** разбиваем parent.

Это **самая хитрая** часть. Понять идею: корень — особенный (его split создаёт новый уровень дерева); остальные узлы — обычные (split добавляет ключ в parent).

35.7 Range scan

scan обходит все пары в порядке возрастания. Используется **leaf chain**:

```
void BPlusTree::scan(std::function<void(std::int64_t, std::int64_t)> cb) const {
    PageId pid = root_page_id_;
    while (true) {
        NodeData n = read_node(pid);
        if (n.type == Leaf) break;
        pid = n.children.front();
    }
    while (pid != 0) {
        NodeData leaf = read_node(pid);
        for (std::size_t i = 0; i < leaf.keys.size(); ++i) {
            cb(leaf.keys[i], leaf.values[i]);
        }
        pid = leaf.next_leaf;
    }
}
```

1. Спускаемся в самый левый лист (всё время children.front()).
2. Идём по next_leaf до конца.

Это **O(N)** для всего набора. Для range query (от ключа A до B) можно начать с find_leaf(A) и идти, пока ключ <= B. Будет $O(\log N + K)$ — оптимально.

35.8 Удаление — кратко

Я не реализовал. Удаление — **намного** сложнее вставки:

1. Найти лист, удалить пару.
2. Если лист слишком пуст ($< \text{MAX_KEYS}/2$):
 - **Borrow** — взять ключ у соседа.
 - **Merge** — слить с соседом.
3. Если слили листы — у parent на одного ребёнка меньше.

4. Если parent теперь слишком пуст — recursive merge с его соседом.
5. Если merge добрался до корня — корень может «съёжиться».

Это десятки строк осторожного кода. SQLite не сразу научился правильному merge — было несколько раундов багфиксов в коде.

В нашей мини-СУБД insert+find хватит для демонстрации. Реальная СУБД обязательно делает и delete.

35.9 Демо

```
$ ./build/mydb  
=== mydb (глава 34: B+tree) ===
```

Вставлено 15 ключей.

Page count: 10

Root page: 9

Поиск:

`find(1)` = 100

`find(5)` = 500

`find(7)` = 700

`find(12)` = 1200

`find(99)` = не найдено

Обход по порядку:

1=100 2=200 3=300 4=400 5=500 6=600 7=700 8=800 9=900 10=1000 11=1100 12=1200 13=1300 14=1400 15=1500

15 ключей в случайном порядке (5, 1, 8, 3, ...) — после вставки scan выдаёт **отсортированный** список.

Дерево заняло 10 страниц = 40 KB. С MAX_KEYS=4 и 15 ключами получилось ~5 листьев + несколько внутренних. Корень — Page 9.

При MAX_KEYS=250 — те же 15 ключей уместились бы в один лист, дерево из одной страницы.

35.10 Дисковое дерево — что нового

Главное отличие от **в памяти**:

- **Нельзя держать всё в памяти.** Только узлы, которые сейчас обрабатываем (плюс через page cache).
- `read_node/write_node` — это IO. Минимизируем их количество.
- `pm.write_page` идёт через кэш. На реальной СУБД важно, чтобы порядок записей был **правильный**: parent после child (или WAL гарантирует консистентность).
- `pm.sync()` делает `fsync`. В нашем коде вызывается явно из `main` после `insert`.

При краше СУБД во время `split` — состояние может быть некорректным: например, новый правый лист записан, parent ещё нет. Решение — **WAL** (глава 35).

35.11 Performance estimates

Для нашего `MAX_KEYS=4` и 1M записей: - Высота = $\log_4(1M) \approx 10$. - Insert: 10 read + ~3 write (split chain). ~13 IO. - Find: 10 read.

Для `MAX_KEYS=250` и 1M: - Высота = $\log_{250}(1M) \approx 2.5$. - Insert: 3 read + 1 write среднее. Чаше без `split`. - Find: 3 read. Если кэш горячий — 0 IO.

Реальные СУБД близки ко второму. Поиск ключа в индексе — **несколько микросекунд** на SSD с горячим кэшем.

35.12 Тонкости

35.12.1 Биение по середине

При `splitting` у нас `mid = size / 2`. Это упрощение. В реальной СУБД: - Если все вставки **отсортированные** (1, 2, 3, ...) — `split` по середине даёт всегда полные левые и полупустые правые. Половина места теряется. - Оптимизация: `split` со смещением 90/10 для `append-only workload`.

PostgreSQL имеет `fillfactor` — насколько заполнять страницу. По умолчанию 90% — оставляем 10% для удобства `update`.

35.12.2 Concurrency

Наш B+tree — однопоточный. Параллельный доступ требует **locking**: - Lock-coupling (latch crabbing): держим lock на parent пока проверяем child. - Optimistic locking: пробуем читать без lock, если кто-то изменил — `retry`. - Multi-version concurrency (MVCC) — каждая транзакция видит «свой» снимок дерева.

Это материал учебной книги по СУБД, не наша книга.

35.12.3 Variable-length keys

У нас ключи `int64` — фиксированной длины. Реальные СУБД часто используют **строки переменной длины** как ключи.

Это сложнее: одна страница вмещает разное количество ключей в зависимости от их размера. Layout страницы становится динамичным. PostgreSQL делает это через **slotted pages**: массив указателей в начале + переменные данные в конце.

В нашем учебном — фиксированные ключи. Если будете расширять — изучите `slotted-page-layout`.

35.12.4 Prefix compression

Для строковых ключей в B+tree часто применяют **prefix compression**: если все ключи в листе начинаются на «hello_», хранят префикс один раз. Экономия места в разы.

Опять же — оптимизация, не для нашей учебной.

35.13 Главные правила главы

1. **B+tree** = широкое неглубокое дерево, оптимизированное под IO.
2. **Один узел = одна страница** (4 KB у нас).
3. **Большой fan-out** = меньше IO. На production: 200-500 ключей в узле.
4. **Internal: keys + children pointers**. Leaf: keys + values + next-pointer.
5. **Leaf chain** для эффективного range scan.
6. **Split при переполнении**, рекурсивно вверх. Корень может породить новый уровень.
7. **Separator key копируется при leaf split, выносится при internal split**. Особенность B+tree.
8. **Delete сложнее**; в production — обязательно, в учебном — можно опустить.

35.14 Маленькое упражнение

1. Запустите `./build/mydb`. Изучите вывод — `find` и `scan`.
2. Увеличьте `MAX_KEYS = 50`. Соберите. Запустите. Сколько pages занимает? Какая root page?
3. Уменьшите `MAX_KEYS = 2`. Запустите. Какая высота дерева на 15 ключах?
4. Добавьте проверку: после каждого `insert` — `scan` показывает ключи в правильном порядке.

5. Реализуйте `range_scan(from, to, callback)` — обход ключей в диапазоне. Подсказка: `find_leaf(from)` плюс `next_leaf` пока ключ $\leq$ `to`.
6. (Сложнее) Реализуйте `count()` — общее число записей. Через `scan` — $O(N)$. Можно ли быстрее? Подсказка: хранить счётчик в метаданных, обновлять при `insert`.
7. (Сложнее) Реализуйте **delete**. Borrow при $< \text{MAX_KEYS}/2$, merge если borrow не получился. Тестируйте на тысячах операций.
8. (Сложнее) Бенчмарк: 1M insert. Замерьте время. С `MAX_KEYS=4` и `MAX_KEYS=250` — сравните.

35.15 Что дальше

Глава 35 — **WAL (Write-Ahead Log)**. Мы видели, что при `splitting` или другой составной операции краш может оставить структуру в плохом состоянии. WAL — это **журнал намерений**: «я собираюсь сделать X». Сначала пишем в WAL, потом делаем. После краша — повторяем по WAL то, что не доделали. Это и есть основа **транзакций**.

Дальше — мини-SQL (36), индексы (37), REPL (38), бенчмарки (39).

Глава 36

Глава 35. Write-Ahead Log

Глава 33 дала нам страницы и кэш. Глава 34 — B+tree поверх них. Сейчас мы добавим **третий критический слой**: журнал, защищающий нас от потери данных при крахе. WAL (Write-Ahead Log) — это идея, лежащая в основе **транзакций** в любой настоящей СУБД.

К концу главы наш `mydb` сможет: - Вставлять записи через WAL + B+tree. - Имитировать краш (отсутствие `sync()`). - При следующем открытии **восстановиться** через WAL. - После `checkpoint` — обрезать WAL, освободив место.

36.1 Проблема, которую решает WAL

Без WAL наш цикл: 1. `pm.write_page(pid, page)` — данные в кэше памяти. 2. ... много операций ... 3. `pm.sync()` — все `dirty` страницы на диск + `fsync`.

Между шагом 1 и 3 может пропасть питание. Тогда страницы B+tree, которые мы изменили, **не дошли до диска**. На следующем запуске: - Прочитаем старые версии страниц. - B+tree может быть в **неконсистентном состоянии**: например, `split` добавил новую страницу, но `parent` ещё не записан с указателем на неё. Получится «потерянный» лист в файле, или ссылка на несуществующую страницу.

Это плохо. Особенно для серверов БД: пользователь сделал `commit`, получил «ОК», после краха данные пропали.

WAL решает обе проблемы: `durability` (данные сохранены) и `atomicity` (операция либо завершилась, либо нет).

36.2 ACID — короткое напоминание

Транзакции в СУБД характеризуются четырьмя свойствами:

- **Atomicity** — атомарность. Операция либо вся применена, либо вся откатена. Не «полусделана».
- **Consistency** — согласованность. Транзакция переводит БД из одного валидного состояния в другое.
- **Isolation** — изолированность. Одновременные транзакции не видят промежуточные состояния друг друга.
- **Durability** — устойчивость. После commit изменения сохраняются даже при крахе.

WAL отвечает за **A** и **D**.

I требует locking или MVCC — у нас однопоточно, проблемы нет.

C — на стороне приложения (constraints, integrity checks) — мы не реализуем.

36.3 Идея WAL

«Write-Ahead» = «пиши заранее». Главный принцип:

Перед изменением данных запиши в журнал намерение это сделать. Дождись fsync журнала. Только потом меняй данные.

Цикл: 1. Программа хочет: `insert(key=10, value=100)`. 2. Записать в WAL: «type=insert, key=10, value=100». 3. `fsync WAL` — журнал durable. 4. Вернуть пользователю «ОК» — операция совершена. 5. Когда-нибудь потом изменить B+tree в кэше. Возможно, через час сделать `rm.sync()`.

Что произойдёт при крахе?

До шага 3: WAL не содержит записи. Пользователь видит ошибку. Дерево не тронут. Всё консистентно.

После шага 3, до 5: WAL содержит запись. Дерево в файле — старое. При запуске **replay** проиграет запись в дерево.

После шага 5, до `rm.sync()`: WAL содержит запись, дерево в кэше изменено, но не сброшено. Краш → дерево в файле старое. Recovery replay'нет запись в дерево.

После `rm.sync()` и checkpoint: запись и в WAL, и в дереве на диске. Можно **truncate** WAL — данные уже надёжно в дереве.

WAL **никогда** не отстаёт от данных. Если запись в WAL — она проиграется в дерево при recovery, **даже если** она уже была применена раньше (важно: операция должна быть **идемпотентной**, и insert в B+tree это так — повторная вставка просто перепишет значение).

36.4 Формат записи WAL

Каждая запись — фиксированная структура:

```
[0..3]   u32 length (всегда 17 для наших insert-записей)
[4]      u8  type (1 = insert)
[5..12]  i64 key
[13..20] i64 value
[21..24] u32 crc
```

Всего 25 байт. Поля: - **length** — длина «полезной нагрузки» (type+key+value). Для будущих типов записей может быть переменной. - **type** — что за операция. 1 = insert. В будущем 2 = delete, 3 = checkpoint marker и т.д. - **key, value** — данные. - **crc** — простая хеш-сумма от type+key+value, для отлова повреждений.

36.5 Алгоритм replay

При открытии БД:

```
WAL wal(wal_path);
wal.replay([&](key, value) {
    tree.insert(key, value);
});
```

Читаем записи **с начала**. Для каждой — вызываем callback. Callback применяет в B+tree.

Если по пути встретили **повреждение** (плохой crc, неполная запись) — **останавливаемся**. Это означает: либо writer не дописал, либо физический сектор испортился. Всё **после** повреждения считается недурабельным.

Идемпотентность: повторное применение записи **не вредно**. `tree.insert(10, 100)` дважды — оба раза значение 100. Поэтому неважно, дошли ли изменения до B+tree до краша — replay применит их «как минимум один раз».

36.6 Truncation после checkpoint

Если WAL растёт без ограничений — место кончится. Поэтому периодически делаем **checkpoint**:

1. **Flush B+tree**: `pm.sync()` — все dirty страницы на диск + `fsync`.
2. После этого данные **в дереве на диске** идентичны WAL.
3. **Truncate WAL**: обнуляем файл.

После truncate, если краш — replay начинается с пустого WAL. Дерево уже на диске. Всё консистентно.

Когда делать checkpoint? Компромисс: - Часто → много fsync дерева → медленно при высокой нагрузке. - Редко → большой WAL → долгий recovery.

В production БД checkpoint делают **по таймеру** (каждые 5-30 минут) или **по размеру WAL** (когда превысил предел).

36.7 Group commit

Каждый fsync — миллисекунды. Если транзакции прилетают тысячами в секунду, **по одному fsync каждой** — узкое место.

Group commit: собираем N транзакций, записываем их **все** в WAL, делаем **один** fsync, отвечаем «ОК» всем сразу.

Эффект: время на одну транзакцию падает с миллисекунды до микросекунд (если очередь длинная).

Усложнение: нужно ждать, пока соберётся группа. Не делаем в нашем демо — однопоточно и нет очереди.

36.8 Класс WAL

include/wal.h:

```
class WAL {
public:
    explicit WAL(const std::string& path);
    ~WAL();

    WAL(const WAL&) = delete;
    WAL& operator=(const WAL&) = delete;

    void log_insert(std::int64_t key, std::int64_t value);
    void replay(std::function<void(std::int64_t, std::int64_t)> cb);
    void truncate();
    std::size_t size_on_disk() const;

private:
    std::string path_;
```

```
int fd_;
};
```

Простой API: log/replay/truncate.

0_APPEND при open гарантирует, что все write идут в конец файла, даже если есть параллельные writers (важно для будущей многопоточности).

36.8.1 log_insert

```
void WAL::log_insert(std::int64_t key, std::int64_t value) {
    unsigned char payload[PAYLOAD_BYTES];
    payload[0] = 1;
    // ... сериализация key и value в little-endian ...
    std::uint32_t crc = crc32_simple(payload, PAYLOAD_BYTES);

    unsigned char rec[RECORD_BYTES];
    // ... запись length + payload + crc ...

    if (!write_all(fd_, rec, RECORD_BYTES)) {
        throw WALError(...);
    }
    if (::fsync(fd_) < 0) {
        throw WALError(...);
    }
}
```

Сериализуем запись в буфер, делаем **один** write (атомарность от ОС), потом **fsync**.

fsync после каждой записи — самая консервативная стратегия. Гарантирует, что после возврата из log_insert данные на диске.

Альтернатива: батчинг (group commit). Записываем в буфер ОС, fsync периодически. Быстрее, но при крахе теряется хвост, не достигший fsync.

write_all — обработка short writes как в главе 27. EINTR — повтор.

36.8.2 replay

```
void WAL::replay(std::function<void(std::int64_t, std::int64_t)> cb) {
    int rfd = ::open(path_.c_str(), O_RDONLY);
```

```

// ...

unsigned char header[4];
while (read_all(rfd, header, 4)) {
    std::uint32_t len = parse_u32(header);
    if (len != PAYLOAD_BYTES) break;    // битый заголовок

    unsigned char body[PAYLOAD_BYTES];
    if (!read_all(rfd, body, PAYLOAD_BYTES)) break;

    unsigned char tail[4];
    if (!read_all(rfd, tail, 4)) break;

    std::uint32_t got_crc = parse_u32(tail);
    if (got_crc != crc32_simple(body, PAYLOAD_BYTES)) break;    // повреждение

    if (body[0] != 1) break;    // неизвестный тип

    int64_t key = parse_i64(body + 1);
    int64_t value = parse_i64(body + 9);
    cb(key, value);
}
::close(rfd);
}

```

Открываем отдельный fd для чтения (writer-fd в append-mode, не подходит для seek в начало). Читаем header, body, tail. Проверяем crc. Вызываем колбэк.

Любая ошибка — **break**, не **throw**. WAL может быть «недописан» — это нормальное состояние после краха. Просто останавливаемся в этой точке.

36.8.3 truncate

```

void WAL::truncate() {
    if (::ftruncate(fd_, 0) < 0) {
        throw WALError(...);
    }
    if (::fsync(fd_) < 0) {
        throw WALError(...);
    }
}

```

```

    }
}

```

`ftruncate(fd, 0)` обнуляет файл. `fsync` обязателен — иначе `truncate` может потеряться при крахе, и WAL вернётся к старому содержимому.

`0_APPEND` гарантирует, что новые записи будут писаться с `offset 0` после `truncate`.

36.9 Демо: имитация краха

Сценарий: вставили 5 записей через `wal.log_insert + tree.insert`, **не** сделали `pm.sync()`. Программа закрылась без `flush B+tree`.

```

{
    db::PageManager pm(db_path, 8);
    db::BPlusTree tree(pm);
    db::WAL wal(wal_path);

    for (auto& kv : inserts) {
        wal.log_insert(kv.first, kv.second);
        tree.insert(kv.first, kv.second);
    }
    // НЕ вызываем pm.sync() — имитируем краш
}

```

После выхода из блока деструктор `PageManager` пытается `sync()`, но **в нашем коде из главы 33** деструктор глотает ошибки и не гарантирует `durability` при программных ошибках. В реальной `production-БД` мы бы здесь падали без `sync`, и страницы `B+tree` остались бы в **старом** состоянии.

Открываем заново — `playback`:

```

{
    db::PageManager pm(db_path, 8);
    db::BPlusTree tree(pm);
    db::WAL wal(wal_path);

    wal.replay([&](key, value) {
        tree.insert(key, value);
    });
}

```

```
// дерево восстановлено
}
```

replay читает 5 записей из WAL, для каждой вызывает `tree.insert`. Дерево в кэше теперь актуально. На следующем `rm.sync()` запишется на диск.

После `rm.sync()` — **checkpoint**: данные надёжно в дереве, можно `wal.truncate()`.

36.9.1 Запуск

```
$ ./build/mydb
=== mydb (глава 35: WAL) ===

После вставок WAL размер: 125 байт
(5 записей × 25 байт = 125)
B+tree pages: 4

Имитируем краш — B+tree НЕ синхронизирован.

--- Восстановление ---
Реплейнуто 5 записей
Содержимое дерева после восстановления:
  10=100 20=200 30=300 40=400 50=500

После checkpoint:
  WAL размер: 0 байт
  B+tree pages durable.

--- Второе открытие (WAL пуск) ---
find(30) = 300
```

Сценарий полный: 1. Вставили 5 записей. WAL 125 байт. Дерево в кэше, не на диске. 2. Имитация краха (закрытие без `sync`). 3. Открытие → `replay` → дерево восстановлено. 4. Checkpoint: `sync` дерева + `truncate` WAL. 5. Второе открытие → WAL пуст → дерево в дереве уже всё есть.

Это надёжность production-уровня. Чуть упрощённая (мы не делаем `page checksums`, не верифицируем дерево после `recovery`), но идейно та же.

36.10 Ещё уровни сложности

Что добавила бы настоящая СУБД:

36.10.1 LSN

Log Sequence Number — порядковый номер записи в WAL. Каждая запись имеет свой LSN. Каждая страница B+tree помечается LSN последнего изменения. При recovery: для каждой страницы знаем, до какого LSN она актуальна, и replay'им только записи после.

Это **incremental recovery** — не переигрываем уже применённые операции. Огромная экономия времени.

36.10.2 Physical vs logical WAL

Наш WAL — **logical**: записи операций («insert key=X»). Recovery — переигрывание операций.

Альтернатива — **physical**: записи изменений байтов («на странице Р смещение О изменился байт X на Y»). Recovery — байт-за-байт применение.

PostgreSQL использует **physical**. MySQL InnoDB — **physiological** (что-то среднее).

Logical проще писать, но требует идемпотентности и предсказуемости (не все операции легко переиграть). Physical — точное копирование изменений, нечувствителен к деталям логики.

36.10.3 Checkpoint protocol

В production checkpoint — не просто sync + truncate. Это **алгоритм**:

1. Записать в WAL BEGIN CHECKPOINT.
2. Сбросить все dirty страницы.
3. fsync файла данных.
4. Записать в WAL END CHECKPOINT.
5. fsync WAL.
6. (Опционально) truncate WAL до точки BEGIN CHECKPOINT.

Если краш на любом шаге — recovery понимает, где остановились, и продолжает корректно.

36.10.4 Full page writes

При крахе во время записи 4 KB страницы — частичная запись (page tearing). Решение: при **первом** изменении страницы после checkpoint — записать в WAL **всю страницу**. Recovery восстанавливает страницу из WAL.

Это **full page writes** в PostgreSQL. Увеличивает размер WAL, но защищает от tearing.

36.10.5 WAL archiving

WAL хранит **все** изменения. Если копировать WAL-файлы куда-то ещё (например, на другую машину) — можно построить **репликацию** и **point-in-time recovery** (восстановиться на любой момент в прошлом).

PostgreSQL `archive_mode = on` копирует WAL на холодный хранилище. MySQL `binlog` — то же.

36.11 Главные правила главы

1. **WAL — основа durability и atomicity.** Без него СУБД не серьёзная.
2. **Write-Ahead** — пиши в журнал ДО изменения данных. Сначала намерение, потом действие.
3. **fsync WAL обязателен** перед возвратом «ОК» пользователю.
4. **Идемпотентность операций** — replay должен работать корректно после повторного применения.
5. **Crc для отлова повреждений.** При recovery — стоп на первом плохом.
6. **Checkpoint = sync дерева + truncate WAL.** Иначе WAL разрастается.
7. **Group commit** для масштаба. Объединяй fsync'и.
8. **LSN и physical WAL** — индустриальный уровень. Для нашей демо хватит logical.

36.12 Маленькое упражнение

1. Соберите. Запустите. Изучите вывод.
2. Запустите без `wal.truncate()` (закомментируйте). Запустите дважды. Что происходит? Подсказка: replay сработает дважды, но идемпотентно.
3. Откройте `data/wal_demo.wal` через `xxd`. Изучите формат — найдите `length (17)`, `key`, `value`, `crc`.
4. Сломайте WAL: измените байт в записи через `hex-editor` или `dd`. Запустите. Replay должен остановиться на этой записи.
5. Удалите `rm.sync()` после успешного replay. Сделайте `wal.truncate()`. Запустите снова. Что произойдёт? (Дерево потеряно, WAL пуст — данные исчезли.)
6. (Сложнее) Добавьте тип `2 = delete` в WAL и реализуйте `wal.log_delete(key)`. Замените на полноценный `tree.remove(key)` (см. упражнение 7 главы 34).
7. (Сложнее) Реализуйте LSN. В каждой записи WAL — порядковый номер. На каждой странице B+tree храните LSN последнего изменения. При replay пропускайте записи с `LSN <= LSN` страницы.

8. (Сложнее) Реализуйте **group commit**: накапливайте записи в буфер, fsync один раз каждые N записей или каждые M миллисекунд.

36.13 Что дальше

Глава 36 — **мини-SQL**. Парсер CREATE TABLE, INSERT, SELECT ... WHERE. Свой токенизатор, свой парсер запросов, AST, выполнение поверх нашего B+tree. После SQL — будет похоже на настоящую СУБД, только маленькую.

Дальше — индексы и план запроса (37), REPL-клиент (38), бенчмарки (39). Часть IV — длинная, насыщенная, но финал близок.

Глава 37

Глава 36. Мини-SQL

К этому моменту у нас уже есть рабочая СУБД: страничный файл, B+tree, WAL для durability. Но пользователю чтобы вставить запись, нужно писать на C++. Это не реально для применения. **SQL** — стандартный язык запросов, понятный пользователю и инструментам.

В этой главе мы напишем **мини-SQL** парсер и executor: CREATE TABLE, INSERT, SELECT ... WHERE. После этого мы сможем сказать «SELECT * FROM items WHERE value > 250» и получить таблицу. Поверх нашего B+tree, со всеми гарантиями WAL.

Это четвёртая глава с парсингом в книге (мир в RPG, shell, save-формат, теперь SQL). Каждый раз — те же принципы, но более сложный язык. После этой главы у вас будет уверенность писать парсеры самостоятельно.

37.1 Зачем SQL

В нашем коде `tree.insert(1, 100)` — это **низкоуровневый** API. Знаем тип B+tree, знаем структуру. Если бы захотели поменять движок (например, на hash) — переписали бы код пользователя.

SQL — **декларативный** язык. Вы говорите **что** вам нужно, не **как**:

```
SELECT * FROM items WHERE value > 250;
```

Это «выбери всё из items, где value больше 250». Как именно — заботит **executor**: может полным scan'ом, может через индекс, может через комбинацию. Программа меняется — SQL не трогаешь.

Это **разделение интерфейса и реализации**. Главная идея реляционных СУБД.

37.2 Грамматика мини-SQL

Что поддержим:

```
CREATE TABLE <name> (<col> INT [PRIMARY KEY], <col> INT [PRIMARY KEY], ...)
```

```
INSERT INTO <name> VALUES (<num>, <num>, ...)
```

```
SELECT <col> | * [, <col> ...] FROM <name> [WHERE <col> <op> <num>]
```

Операторы сравнения: =, !=, <, <=, >, >=.

Что **не** поддержим (для простоты): - Несколько таблиц одновременно (одна на БД). - DELETE, UPDATE, JOIN. - Сложные WHERE (AND, OR, ()). - Строковые колонки (только INT). - ORDER BY, LIMIT, GROUP BY.

Это **подмножество** реального SQL — достаточное для демонстрации архитектуры.

37.3 Архитектура

Три фазы обработки запроса:

SQL-строка

↓

[Tokenizer] — разбивает на токены (keywords, identifiers, numbers, operators)

↓

[Parser] — строит AST (Statement struct)

↓

[Executor] — выполняет, обращается к B+tree/WAL, выводит результат

Тот же стек, что в shell parser (глава 29), только на другом языке.

37.4 Tokenizer

Алгоритм похож на shell-tokenizer, но проще (нет кавычек):

```
enum class TokenType {
    kIdent,      // foo, bar — имена таблиц и колонок
    kKeyword,    // CREATE, TABLE, SELECT, ... — зарезервированные слова
    kNumber,     // 42, -100
    kStar,       // *
    kComma,
    kLParen,
    kRParen,
```

```

kOp,          // =, <, >, <=, >=, !=
kSemi,
kEnd
};

```

В цикле по символам:

```

while (i < s.size()) {
    char c = s[i];
    if (isspace(c)) { ++i; continue; }

    // Спец-символы — отдельные токены
    if (c == '(') { out.push_back({kLParen, "("}); ++i; continue; }
    // ... остальные ...

    // Операторы сравнения: 1 или 2 символа
    if (c == '=' || c == '<' || c == '>' || c == '!=') {
        std::string op(1, c);
        if (i + 1 < s.size() && s[i + 1] == '=') { op += '='; i += 2; }
        else { ++i; }
        out.push_back({kOp, op});
        continue;
    }

    // Числа: цифры, опциональный минус впереди
    if (isdigit(c) || (c == '-' && isdigit(s[i+1]))) {
        // ... сканируем до конца числа ...
        out.push_back({kNumber, ...});
        continue;
    }

    // Идентификатор или keyword
    if (isalpha(c) || c == '_') {
        // ... сканируем буквы/цифры/_ ...
        std::string text = s.substr(...);
        std::string upper = to_upper(text);
        if (is_keyword(upper)) {
            out.push_back({kKeyword, upper});
        } else {

```

```

        out.push_back({kIdent, text});
    }
    continue;
}
throw SqlError("неизвестный символ");
}
out.push_back({kEnd, ""});

```

Главные моменты:

Operators: =, <, >, ! могут быть односимвольными или с = следующим (<=, >=, !=). Смотрим следующий символ для решения.

Numbers могут начинаться с минуса. Проверяем условие отдельно. Парсинг — до первого нецифрового.

Keywords vs идентификаторы: SQL **регистронезависимый** для keywords (SELECT == select). Приводим к верхнему регистру и сверяем со списком. Если есть — keyword. Иначе — идентификатор (имя колонки/таблицы).

Идентификаторы у нас регистрозависимые (как в большинстве баз — но опять, в Oracle и PostgreSQL разные правила).

kEnd в конце — sentinel, парсер на нём останавливается без проверки границ.

37.4.1 Список keywords

```

static const std::vector<std::string> kw = {
    "CREATE", "TABLE", "INT", "PRIMARY", "KEY",
    "INSERT", "INTO", "VALUES",
    "SELECT", "FROM", "WHERE",
};

```

Минимальный набор. Реальный SQL имеет сотни keywords.

37.5 Parser

Recursive descent парсер. Для каждой правила грамматики — функция.

```

struct Parser {
    const std::vector<Token>& tokens;
    std::size_t pos = 0;

```

```

const Token& peek() const { return tokens[pos]; }
Token consume() { return tokens[pos++]; }
Token expect(Token::Type t);
Token expect_keyword(const std::string& kw);
std::int64_t parse_number();

Statement parse();
Statement parse_create();
Statement parse_insert();
Statement parse_select();
};

```

`peek()` — посмотреть следующий токен без потребления. `consume()` — потребить и продвинуть позицию. `expect(...)` — потребить, проверив тип/значение.

37.5.1 Главная функция `parse`

```

Statement parse() {
    if (peek().type != Token::kKeyword) throw SqlError("ожидался keyword");
    const std::string& kw = peek().text;
    if (kw == "CREATE") return parse_create();
    if (kw == "INSERT") return parse_insert();
    if (kw == "SELECT") return parse_select();
    throw SqlError("неизвестный statement: " + kw);
}

```

Простая диспетчеризация по первому ключевому слову.

37.5.2 `parse_create`

```

Statement parse_create() {
    Statement st;
    st.kind = Statement::kCreate;
    expect_keyword("CREATE");
    expect_keyword("TABLE");
    st.table = expect(Token::kIdent).text;
    expect(Token::kLParen);
    while (true) {

```

```

std::string col = expect(Token::kIdent).text;
expect_keyword("INT");
if (peek().type == Token::kKeyword && peek().text == "PRIMARY") {
    consume();
    expect_keyword("KEY");
}
st.columns.push_back({col, "INT"});
if (peek().type == Token::kComma) { consume(); continue; }
break;
}
expect(Token::kRParen);
if (peek().type == Token::kSemi) consume();
return st;
}

```

Линейно идём по структуре: keyword «CREATE», keyword «TABLE», идентификатор таблицы, (, колонки через запятую,), опциональный ;.

Каждая колонка: идентификатор + INT + опциональный PRIMARY KEY. PRIMARY KEY игнорируется — у нас и так первая колонка primary.

while (true) + break — типичная идиома для «обрабатываем элементы через разделитель». Прочитали один → если запятая, продолжаем; иначе выходим.

37.5.3 parse_select

```

Statement parse_select() {
    Statement st;
    st.kind = Statement::kSelect;
    expect_keyword("SELECT");
    if (peek().type == Token::kStar) {
        consume();
        st.select_columns.push_back("*");
    } else {
        while (true) {
            st.select_columns.push_back(expect(Token::kIdent).text);
            if (peek().type == Token::kComma) { consume(); continue; }
            break;
        }
    }
}

```



```

expect_keyword("FROM");
st.table = expect(Token::kIdent).text;
if (peek().type == Token::kKeyword && peek().text == "WHERE") {
    consume();
    st.has_where = true;
    st.where_col = expect(Token::kIdent).text;
    st.where_op = expect(Token::kOp).text;
    st.where_val = parse_number();
}
if (peek().type == Token::kSemi) consume();
return st;
}

```

SELECT * FROM ... или SELECT col1, col2 FROM Потом FROM и таблица. Потом опциональный WHERE.

WHERE поддерживает только <col> <op> <number>. Никаких AND/OR/скобок. Если их нужно — отдельный мини-парсер выражений (часто рекурсивный, с приоритетами).

37.6 Recursive descent в общем

Идея recursive descent: - Для каждого правила грамматики — функция. - Функция читает токены и при необходимости рекурсивно вызывает другие функции. - Возвращает соответствующий узел AST.

Это самый простой вариант парсера. Подходит для несложных грамматик. Для сложных (выражения с приоритетами, ambiguity) — есть **Pratt parsing**, **shift-reduce** (bison/yacc), **PEG** (peg.js).

Все профессиональные SQL-парсеры (PostgreSQL, MySQL, SQLite) — вручную написанные recursive descent или yacc-generated. Идея та же, что у нас, только грамматика в десятки раз больше.

37.7 AST как одна структура

Я выбрал **один Statement struct** со всеми возможными полями. Альтернатива — полиморфная иерархия (CreateStmt : Statement, InsertStmt : Statement, и так далее). Полиморфизм гибче для расширения. Один struct проще для маленькой грамматики.

```

struct Statement {
    enum Kind { kCreate, kInsert, kSelect };
    Kind kind;

    std::string table;
    std::vector<std::pair<std::string, std::string>> columns; // для CREATE
    std::vector<std::int64_t> values; // для INSERT
    std::vector<std::string> select_columns;
    bool has_where = false;
    std::string where_col;
    std::string where_op;
    std::int64_t where_val = 0;
};

```

«Лишние» поля для других kind просто пустые. На производительность это не влияет — Statement короткоживущий объект.

37.8 Executor

Берёт Statement, выполняет:

```

void Database::execute(const std::string& sql, std::ostream& out) {
    Statement st = parse_sql(sql);
    switch (st.kind) {
        case Statement::kCreate: execute_create(st, out); break;
        case Statement::kInsert: execute_insert(st, out); break;
        case Statement::kSelect: execute_select(st, out); break;
    }
}

```

37.8.1 CREATE TABLE

```

void Database::execute_create(const Statement& st, std::ostream& out) {
    if (st.columns.size() != 2) {
        throw SqlError("эта мини-СУБД поддерживает таблицы только из 2 колонок");
    }
    table_name_ = st.table;
    col_names_ = {st.columns[0].first, st.columns[1].first};
}

```

```
    out << "OK (table '" << table_name_ << "')\n";
}
```

Сохраняем имя таблицы и колонок в полях Database. **Не персистим** на диск — в этой версии схема живёт только в памяти, после рестарта забывается.

В настоящей СУБД схема хранится в системной таблице (например, `pg_class` в PostgreSQL).

37.8.2 INSERT

```
void Database::execute_insert(const Statement& st, std::ostream& out) {
    if (st.table != table_name_) {
        throw SqlError("неизвестная таблица: " + st.table);
    }
    if (st.values.size() != 2) {
        throw SqlError("ожидается 2 значения");
    }
    std::int64_t id = st.values[0];
    std::int64_t value = st.values[1];

    wal_>log_insert(id, value);    // durable
    tree_>insert(id, value);      // в кэше
    out << "1 row\n";
}
```

Берём первое значение как key (id), второе как value. Логируем в WAL (с fsync), вставляем в B+tree. Всё durable, всё чисто.

37.8.3 SELECT

Самая интересная часть:

```
void Database::execute_select(const Statement& st, std::ostream& out) {
    if (st.table != table_name_) throw SqlError(...);

    bool star = (st.select_columns.size() == 1 && st.select_columns[0] == "*");
    std::vector<std::string> cols = star ? col_names_ : st.select_columns;

    // Заголовок таблицы
    print_header(cols, out);
}
```

```

// Оптимизация: если фильтр по primary key с '=' — используем find вместо scan.
if (st.has_where && st.where_col == col_names_[0] && st.where_op == "=") {
    std::int64_t v = 0;
    if (tree_>find(st.where_val, v)) {
        print_row(cols, st.where_val, v, out);
        out << "(1 row)\n";
    } else {
        out << "(0 rows)\n";
    }
    return;
}

// Иначе — полный scan с фильтром.
int rows = 0;
tree_>scan([&](std::int64_t id, std::int64_t value) {
    if (!matches_where(st, id, value)) return;
    print_row(cols, id, value, out);
    ++rows;
});
out << "(" << rows << " rows)\n";
}

```

Главная **оптимизация** — для WHERE id = X использовать tree_>find(X) вместо полного scan. Это разница между $O(\log N)$ и $O(N)$.

Это уже **простой query planner** — выбор стратегии выполнения по форме запроса. В реальной СУБД — десятки оптимизаций (использовать ли индекс, в каком порядке join'ить таблицы, и так далее). Тема главы 37.

matches_where — функция фильтрации:

```

bool Database::matches_where(const Statement& st,
                             std::int64_t id, std::int64_t value) const {
    if (!st.has_where) return true;
    std::int64_t left = (st.where_col == col_names_[0]) ? id : value;
    const auto& op = st.where_op;
    const auto r = st.where_val;
    if (op == "=") return left == r;
    if (op == "!=") return left != r;
    if (op == "<") return left < r;
}

```

```

    if (op == "<=") return left <= r;
    if (op == ">") return left > r;
    if (op == ">=") return left >= r;
    throw SqlError("неизвестный оператор: " + op);
}

```

37.9 Демо

```

$ ./build/mydb
> CREATE TABLE items (id INT PRIMARY KEY, value INT);
OK (table 'items')

> INSERT INTO items VALUES (1, 100);
1 row

> INSERT INTO items VALUES (2, 200);
1 row

> INSERT INTO items VALUES (3, 300);
1 row

> SELECT * FROM items;
id      | value
-----+-----
1       | 100
2       | 200
3       | 300
4       | 400
5       | 500
(5 rows)

> SELECT * FROM items WHERE id = 3;
id      | value
-----+-----
3       | 300
(1 row)

```

```
> SELECT * FROM items WHERE value > 250;
id      | value
-----+-----
3       | 300
4       | 400
5       | 500
(3 rows)

> SELECT id FROM items WHERE value <= 200;
id
-----
1
2
(2 rows)
```

Полноценный SQL на нашем движке. Запросы парсятся, executor применяет правильные операции, B+tree даёт результаты, всё работает.

37.10 Архитектура «парсер-исполнитель»

Если присмотреться, наш `Database::execute` — это **компилятор и интерпретатор** в одной упаковке:

SQL-строка → токены → AST → действия на B+tree

То же делает любой интерпретируемый язык (Python, JS). Те же три фазы.

Разница: настоящий SQL-компилятор **не** интерпретирует AST напрямую. Он: 1. Парсит в AST. 2. **Анализирует семантику**: проверяет, что таблицы существуют, типы совместимы. 3. **Оптимизирует**: переписывает AST в более эффективную форму (плана). 4. **Генерирует план**: дерево физических операторов (TableScan, IndexLookup, Filter, Sort, Join, ...). 5. **Выполняет** план через volcano-model (`next()` рекурсивно).

Это полноценный **query optimizer**. У нас он минимален — одна оптимизация для `WHERE pkey = const`.

В главе 37 (индексы и план запроса) добавим больше.

37.11 Что не получится по нашей грамматике

Несколько примеров запросов, которые **не сработают**:

- `SELECT * FROM items WHERE id = 1 AND value > 100` — мы не понимаем `AND`.
- `SELECT * FROM items ORDER BY value` — нет `ORDER BY`.
- `SELECT * FROM items LIMIT 5` — нет `LIMIT`.
- `DELETE FROM items WHERE id = 1` — нет `DELETE`.
- `UPDATE items SET value = 999 WHERE id = 1` — нет `UPDATE`.
- `SELECT COUNT(*) FROM items` — нет функций.
- `SELECT a.id, b.value FROM a JOIN b ON a.id = b.id` — нет `JOIN`.

Каждое — отдельное расширение парсера и executor. Хороший выпуск для домашних упражнений.

37.12 Главные правила главы

1. **Три фазы:** `tokenize` → `parse` → `execute`. Стандарт для языковых процессоров.
2. **Recursive descent** для простых грамматик — каждое правило = функция.
3. `peek()`/`consume()`/`expect()` — типичные методы парсера.
4. **Keywords case-insensitive** в SQL. Уменьшайте/увеличивайте на этапе токенизации.
5. **AST как struct** с `kind`-полем для маленьких грамматик; полиморфизм для больших.
6. **Простейший query planner** — оптимизация частых случаев (например, `lookup` через `primary key`).
7. **Сообщения об ошибках с местом** — пользователь должен знать, где не парсится.
8. **Декларативный язык** vs низкоуровневый API — главное преимущество SQL.

37.13 Маленькое упражнение

1. Запустите. Попробуйте свои запросы. Какие работают, какие нет?
2. Добавьте оператор `<>` как синоним `!=`. Это стандартный SQL.
3. Добавьте `LIMIT n` после `WHERE` — ограничение числа строк в результате.
4. Добавьте `ORDER BY col` — наш B+tree уже даёт отсортированный обход по ключу. Для `non-primary` колонки нужно собрать все в `vector` и `std::sort`.
5. Добавьте `DELETE FROM <table> WHERE <col> = <num>`. Понадобится `tree.remove(key)` (упражнение из главы 34).
6. (Сложнее) Расширьте `WHERE` на `AND/OR` с скобками. Подсказка: грамматика выражений (`recursive descent` с приоритетами или `Pratt parsing`).
7. (Сложнее) Добавьте `UPDATE ... SET col = num WHERE ...`. Это `find` → `modify` → `insert`.

8. (Очень сложно) Поддержите строковые колонки. B+tree должен научиться хранить переменные значения; формат страницы становится «slotted page».

37.14 Что дальше

Глава 37 — **индексы и план запроса**. Сейчас у нас primary index на id (через B+tree). Добавим **secondary index** — отдельный B+tree, отображающий, например, value → id. Тогда WHERE value = X тоже сможем найти за $O(\log N)$ вместо $O(N)$.

Глава 38 — REPL-клиент (как psql или mysql shell). Будут стрелки вверх для истории, многострочный ввод, цвет.

Глава 39 — бенчмарки и итог Части IV.

Глава 38

Глава 37. Индексы и план запроса

В прошлой главе наш SQL-движок имел **одну** оптимизацию: `WHERE id = X` шёл через `tree.find()` за $O(\log N)$. Все остальные `WHERE` — full scan $O(N)$. Это значит, на миллионе записей запрос `WHERE value = 200` будет проходить миллион ключей. Медленно.

Решение — **secondary index**: ещё один B+tree, отображающий `value → id`. Тогда поиск по `value` тоже $O(\log N)$.

В этой главе добавим: - `CREATE INDEX idx_name ON table(column)` — создание secondary index. - Автоматическое обновление индекса на `INSERT`. - **Query planner** — выбор стратегии: primary lookup / secondary lookup / secondary scan / full scan. - `EXPLAIN SELECT ...` — показать план запроса без выполнения.

38.1 Primary vs secondary index

Primary index — по primary key. В нашем случае — на `id`. Структура B+tree, где `key = id`, `value = ...что-то связанное с записью` (у нас просто `value`).

Secondary index — на любую другую колонку. Отдельный B+tree, где `key = значение колонки`, `value = primary key`. Для поиска по `value`: `secondary.find(200) → id → primary.find(id) → full row`.

Это два **отдельных** B+tree, каждый со своими страницами на диске. Каждый `INSERT` в основную таблицу должен обновить **оба** дерева.

В PostgreSQL это так же: одна таблица + N индексов. У каждого индекса — свой файл. `CREATE TABLE` создаёт primary, `CREATE INDEX` добавляет secondary.

38.1.1 Цена индексов

Индексы — **двойной меч**:

Плюсы: быстрый поиск по индексированным колонкам.

Минусы: - **Запись медленнее**: каждый INSERT обновляет все индексы. - **Место на диске**: каждый индекс — отдельный B+tree. - **WAL больше**: каждое обновление логируется и для основной таблицы, и для каждого индекса.

Поэтому в production: индексируйте **только то, по чему часто фильтруете**. Избыточные индексы тормозят запись без пользы для чтения.

38.2 Структура secondary index в коде

```
struct SecondaryIndex {
    std::unique_ptr<PageManager> pm;
    std::unique_ptr<BPlusTree> tree;
    std::unique_ptr<WAL> wal;
    std::string column;
};

std::unordered_map<std::string, SecondaryIndex> indexes;
```

Каждый индекс — **полноценная мини-СУБД**: свой PageManager, свой B+tree, свой WAL. Файлы — idx_<name>.db и idx_<name>.wal в директории базы.

Когда выполняется CREATE INDEX idx_val ON items(value):

```
void Database::execute_create_index(const Statement& st, std::ostream& out) {
    const std::string& col = st.columns[0].first;

    SecondaryIndex idx;
    idx.column = col;
    idx.pm     = mk<PageManager>(dir_ + "/idx_" + st.index_name + ".db", 32);
    idx.tree   = mk<BPlusTree>(*idx.pm);
    idx.wal    = mk<WAL>(dir_ + "/idx_" + st.index_name + ".wal");
    idx.wal->replay([&idx](std::int64_t k, std::int64_t v) {
        idx.tree->insert(k, v);
    });

    // Populate from existing rows.
```

```

int populated = 0;
tree_>scan([&](std::int64_t row_id, std::int64_t row_value) {
    std::int64_t key = (col == col_names_[0]) ? row_id : row_value;
    idx.wal->log_insert(key, row_id);
    idx.tree->insert(key, row_id);
    ++populated;
});
idx.pm->sync();
idx.wal->truncate();

indexes_.emplace(col, std::move(idx));
}

```

Алгоритм: 1. Создать новый PageManager + B+tree + WAL для индекса. 2. Replay WAL индекса (на случай, если индекс существовал ранее). 3. **Index build**: пройтись по всем строкам в основной таблице, для каждой вставить (value, id) в индекс. 4. Сделать checkpoint индекса (sync + truncate WAL). 5. Зарегистрировать в indexes_.

Этот **index build** в реальной СУБД может занять часы на миллионах строк. PostgreSQL умеет CREATE INDEX CONCURRENTLY — построение индекса параллельно с чтениями/записями. У нас — простой блокирующий вариант.

38.2.1 Обновление индексов на INSERT

```

void Database::execute_insert(const Statement& st, std::ostream& out) {
    std::int64_t id = st.values[0];
    std::int64_t value = st.values[1];

    wal->log_insert(id, value);
    tree->insert(id, value);

    // Обновляем все secondary indexes.
    for (auto& kv : indexes_) {
        const std::string& col = kv.first;
        std::int64_t key = (col == col_names_[0]) ? id : value;
        kv.second.wal->log_insert(key, id);
        kv.second.tree->insert(key, id);
    }
}

```

```
out << "1 row\n";
}
```

Каждый INSERT теперь делает $(1 + N)$ WAL fsync для N индексов. На больших N это узкое место. Оптимизация в production — **group commit** (один fsync на batch операций) — упоминал в главе 35.

38.3 Уникальность ключей в индексе

Подвох: secondary index должен мапить value $\rightarrow$ id. Что если две записи имеют **одинаковый value**?

Наш B+tree поддерживает **только уникальные ключи**. При повторной вставке того же ключа — старая запись перезаписывается.

В нашем демо мы вставляем value 100, 200, 300, 400, 500 — все уникальные. Работает. Если бы были дубликаты — индекс терял бы записи.

Решения в реальных СУБД: - **Composite key**: ключ = (value, id) (пара). Все ключи уникальны. - **Posting list**: значение — список ids (для дубликатов). - **Duplicate-aware B+tree**: расширенные узлы хранят несколько values на одном key.

PostgreSQL использует composite keys для secondary indexes. SQLite — то же.

В нашей мини-СУБД для учебной цели — упрощение «уникальные значения». Реальный код требовал бы изменения B+tree или ключей.

38.4 Query planner

Когда приходит SELECT ... WHERE col op val, нужно выбрать **стратегию выполнения**:

1. **PrimaryLookup**: WHERE id = X. Один tree.find(). $O(\log N)$, 1 IO + кэш.
2. **SecondaryLookup**: WHERE col = X с индексом на col. idx.find() $\rightarrow$ tree.find(). $O(\log N)$, 2 IO.
3. **SecondaryScan**: WHERE col > X (или другой range) с индексом. Range scan secondary + fetch каждой строки.
4. **FullScan**: ничего не подходит. Перебор всей основной таблицы с фильтром.

Простейший планировщик:

```
std::string Database::plan_select(const Statement& st) const {
    if (!st.has_where) return "FullScan(primary)";
    if (st.where_col == col_names[0] && st.where_op == "=") {
```

```

        return "PrimaryLookup(id=" + std::to_string(st.where_val) + ")";
    }
    auto it = indexes_.find(st.where_col);
    if (it != indexes_.end()) {
        if (st.where_op == "=") {
            return "SecondaryLookup(" + it->first + "=" + ... + ")";
        }
        return "SecondaryScan(" + ... + ")";
    }
    return "FullScan with filter (" + ... + ")";
}

```

Возвращает **строку** — описание плана. Используется в EXPLAIN для вывода, и в execute_select для диспетчеризации.

В настоящей СУБД план — **дерево операторов**: Filter(IndexScan(idx_val, value > 250)). Каждый узел — отдельный класс. Это позволяет составные планы (join'ы, агрегации). У нас — простая строка.

38.5 Стратегии execute

```

void Database::execute_select(const Statement& st, std::ostream& out) {
    std::string plan = plan_select(st);

    if (st.is_explain) {
        out << "Plan: " << plan << "\n";
        return;
    }

    // ... печать заголовка ...

    if (!st.has_where) {
        run_full_scan(...);
    } else if (st.where_col == col_names[0] && st.where_op == "=") {
        run_primary_eq(...);
    } else if (auto it = indexes_.find(st.where_col); it != indexes_.end()) {
        run_secondary(..., it->second, ...);
    } else {

```

```

        run_full_scan(...);
    }
}

```

Те же четыре ветки, что в планировщике.

38.5.1 run_primary_eq

```

void Database::run_primary_eq(...) {
    std::int64_t v = 0;
    if (tree->find(st.where_val, v)) {
        print_row(cols, st.where_val, v, out);
    } else {
        // не найдено
    }
}

```

Один find. Самый быстрый случай.

38.5.2 run_secondary

```

void Database::run_secondary(const Statement& st, ..., const SecondaryIndex& idx, ...) {
    auto emit = [&](std::int64_t row_id) {
        std::int64_t row_value = 0;
        if (!tree->find(row_id, row_value)) return;
        print_row(cols, row_id, row_value);
    };

    if (st.where_op == "=") {
        std::int64_t row_id = 0;
        if (idx.tree->find(st.where_val, row_id)) emit(row_id);
    } else {
        // Range scan через secondary
        idx.tree->scan([&](std::int64_t key, std::int64_t row_id) {
            // Проверка предиката
            if (matches(...)) emit(row_id);
        });
    }
}

```

Для = — один find в индексе, потом find в основной таблице. Два дерева, **два** $O(\log N)$.

Для range — **scan** индекса (scan в нашем B+tree обходит весь leaf chain), для каждого подходящего — find в основной. Если индекс отсортирован по value, range scan по indexed-полю эффективен.

Реальная оптимизация: для range value > 250 начать scan не с начала индекса, а с find_leaf(250). Тогда обход — только подходящие записи. У нас scan начинается с самого левого листа — для большого диапазона неоптимально.

38.5.3 EXPLAIN

```
if (st.is_explain) {
    out << "Plan: " << plan << "\n";
    return;
}
```

Просто печатаем план, не выполняя. Это позволяет пользователю **проверить**, какой план выберет planner, перед запуском дорогого запроса.

38.6 Демо

```
> CREATE TABLE items (id INT PRIMARY KEY, value INT);
> INSERT INTO items VALUES (1, 100);
> ... 5 строк ...

> EXPLAIN SELECT * FROM items WHERE id = 3;
Plan: PrimaryLookup(id=3)

> EXPLAIN SELECT * FROM items WHERE value = 300;
Plan: FullScan with filter (value = 300)      ← пока нет индекса

> EXPLAIN SELECT * FROM items WHERE value > 250;
Plan: FullScan with filter (value > 250)

> CREATE INDEX idx_val ON items(value);
Index 'idx_val' on value (5 rows indexed)

> EXPLAIN SELECT * FROM items WHERE value = 300;
```

```

Plan: SecondaryLookup(value=300)           ← теперь через индекс!

> EXPLAIN SELECT * FROM items WHERE value > 250;
Plan: SecondaryScan(value > 250)

> SELECT * FROM items WHERE value = 300;
id      | value
-----+-----
3       | 300
(1 rows)

> INSERT INTO items VALUES (6, 600);      ← обновит индекс автоматически
> SELECT * FROM items WHERE value = 600;
6       | 600

```

Видно: **до** CREATE INDEX запросы по value идут через full scan. **После** — через index. EXPLAIN наглядно показывает разницу.

После INSERT (6, 600) индекс автоматически содержит 600 → 6. SELECT WHERE value = 600 находит через индекс.

38.7 Сложные query planners

Production-планировщики делают **гораздо больше**.

38.7.1 Cost-based optimization

Для каждого возможного плана **оценить стоимость** (cost estimation): - Сколько строк прочитается. - Сколько IO потребуется. - Сколько CPU.

Выбрать план с **минимальной** оценочной стоимостью.

PostgreSQL хранит **статистику** о данных: pg_statistic — гистограммы значений, частоту дубликатов, и так далее. Используется для оценки.

Простой пример: WHERE col = x. Сколько строк подойдёт? - Если x в гистограмме — оценка точная. - Иначе — экстраполяция.

Для индексированного поиска — $est_rows \times cost_per_index_lookup$. Для full scan — $total_rows \times cost_per_seq_scan$. Выбираем меньшее.

В нашем простом планировщике — **rule-based**: всегда использовать индекс, если есть. Это **обычно** правильно, но не всегда. Если в таблице 100 строк, full scan может быть **быстрее** index lookup из-за overhead. Но мы упрощаем.

38.7.2 Joins

Когда `SELECT ... FROM a JOIN b ON a.id = b.id`, планировщик выбирает **алгоритм join'a**:

- **Nested Loop**: для каждой строки `a` — поиск в `b`. $O(N \times M)$ без индекса.
- **Hash Join**: построить hash таблицу из меньшей, пройти по большей. $O(N+M)$.
- **Merge Join**: отсортировать обе по join-ключу, идти параллельно. $O(N+M)$ если сортировка дешёвая (например, индексы уже отсортированы).

И **порядок** join'ов в multi-table запросе — отдельная NP-сложная задача.

У нас нет joins, поэтому ничего этого нет. Тема для отдельной книги.

38.7.3 Composite indexes

Иногда полезен индекс на **две колонки**: `CREATE INDEX ON items(category, value)`. Тогда запрос `WHERE category = 'foo' AND value > 100` использует индекс эффективно.

Порядок в индексе **важен**: `(category, value)` хорош для `category = ... AND value ...`, не очень для `value = ...` (без category).

В нашей мини-СУБД таких индексов нет — поддерживаем только single-column.

38.7.4 Index-only scan

Если в запросе **только** поля, попадающие в индекс, можно **не идти в основную таблицу**. Просто scan индекса — у которого, кстати, обычно меньше страниц = быстрее.

PostgreSQL делает это для запросов вида `SELECT value FROM items WHERE value > 250` — все поля видны в indexed B+tree, основную таблицу не трогаем.

В нашей версии — всегда `tree->find(row_id)` за value. Простая реализация, можно ускорить.

38.8 Главные правила главы

1. **Primary index** = автоматически по pkey. Secondary = добавляется через CREATE INDEX.
2. **Индексы стоят** места на диске и времени на запись. Не злоупотребляйте.
3. **Каждый INSERT обновляет все индексы**. Group commit для масштаба.
4. **Query planner выбирает стратегию** на основе доступных индексов.
5. **EXPLAIN** — обязательная команда. Без неё нет диагностики производительности.

6. **Cost-based** оптимизация на больших таблицах. Rule-based — для простых.
7. **Уникальность ключей** — простое ограничение в учебной СУБД. В production — composite keys или posting lists.
8. **Index-only scan** — оптимизация для запросов, у которых все поля в индексе.

38.9 Маленькое упражнение

1. Запустите. Сравните EXPLAIN до и после CREATE INDEX.
2. Сделайте 10000 вставок (через цикл в коде). Засеките время. Создайте индекс. Засеките SELECT WHERE value = X до и после. Должно быть в сотни раз быстрее.
3. Добавьте команду DROP INDEX idx_name — удаление индекса.
4. Добавьте секцию «индексы» в EXPLAIN: Plan: SecondaryScan(idx_val: ...) с явным именем индекса.
5. (Сложнее) Поддержите дубликаты значений в secondary index через composite keys. Подсказка: ключ = value × 1_000_000 + id или pack в int64.
6. (Сложнее) Реализуйте **index-only scan**: если все select-колонки покрыты secondary index'ом — не идти в основную таблицу.
7. (Сложнее) Сделайте **cost-based planner**: оценивайте стоимость каждой стратегии и выбирайте минимальную. Учтите размер таблицы.
8. (Очень сложно) Добавьте **composite indexes**: CREATE INDEX ON items(id, value). Парсер должен принимать несколько колонок. Индекс ключует пару, упорядочивает по первой колонке.

38.10 Что дальше

Глава 38 — **REPL-клиент**. Сейчас наш main выполняет хардкод-список запросов. Сделаем интерактивный prompt: пользователь вводит SQL, видит результат, набирает следующий. Это уже **похоже на psql**.

Глава 39 — **бенчмарки и итог Части IV**. Замерим производительность нашей мини-СУБД на 1M вставках, сравним с SQLite, обсудим, что осталось «за кадром». Часть IV закрывается.

Глава 39

Глава 38. REPL-клиент

В предыдущих главах наш `main` запускал жёстко зашитый список SQL-запросов. Для настоящей СУБД это не годится — нужно интерактивное общение с пользователем. **REPL** (Read-Eval-Print Loop) — стандартный интерфейс таких программ. `psql`, `mysql`, `sqlite3` — все они REPL'ы.

В этой главе превратим `mydb` в полноценный REPL: - Приглашение `mydb>`. - Multi-line ввод (запрос на нескольких строках, до `;`). - Meta-команды (`.help`, `.quit`, `.history`, `.checkpoint`). - История в файл `~/.mydb_history`. - Комментарии `--` до конца строки.

Глава короткая — мы уже знаем все нужные инструменты из прошлых глав. Это **интеграция**.

39.1 Что такое REPL

Read — прочитать ввод пользователя
Eval — вычислить (выполнить)
Print — напечатать результат
Loop — повторить

Простая идея. Главные элементы:

1. **Prompt** — приглашение, показывающее, что система ждёт ввода. У нас `mydb>`.
2. **Reader** — чтение строки/команды.
3. **Executor** — выполнение.
4. **Output** — печать результата.
5. **Loop** — повтор до выхода.

Что отличает «настоящий» REPL от тестового цикла:

- **Multi-line input**: команда может занимать несколько строк. Continuation prompt (`...` у нас, `...` у `psql`).

- **Meta-команды:** то, что не SQL (`.quit`, `.help`).
- **История** между сессиями.
- **Раскраска и автодополнение** — приятные дополнения.

Самые мощные REPL'ы (Python, Node) могут даже **запоминать переменные** между запросами, иметь автозавершение по имени функций, выводить контекстную справку. Это огромная инфраструктура; мы делаем минимум.

39.2 Multi-line ввод

В SQL запрос завершается **точкой с запятой** `;`. Это позволяет разбивать длинные запросы:

```
mydb> SELECT *  
... FROM items  
... WHERE value > 250;
```

Алгоритм: 1. Накопить во **buffer** строки, разделённые пробелом. 2. После каждой строки проверять: есть ли в **buffer** `;`? 3. Если есть — извлечь **statement** до `;`, выполнить, остаток оставить в **buffer**. 4. Если нет — продолжить считывание.

```
std::string buffer;  
bool first_line = true;  
  
while (true) {  
    std::cout << (first_line ? "mydb> " : " ... ");  
    std::cout.flush();  
  
    std::string line;  
    if (!std::getline(std::cin, line)) break;  
    line = strip_comment(line);  
  
    std::string t = trim(line);  
    if (t.empty() && first_line) continue;  
  
    if (!buffer.empty()) buffer += " ";  
    buffer += t;  
    first_line = false;  
  
    auto semi = buffer.find(';');  
    if (semi == std::string::npos) continue;
```

```

std::string stmt = buffer.substr(0, semi + 1);
std::string rest = buffer.substr(semi + 1);

try {
    db.execute(stmt, std::cout);
} catch (const std::exception& e) {
    std::cout << "Error: " << e.what() << "\n";
}
history_add(trim(stmt));

buffer = trim(rest);
first_line = buffer.empty();
}

```

`first_line` — флаг, чтобы показывать разный prompt: главный или продолжения.

`strip_comment` срезает `---`комментарии: всё после `--` до конца строки удаляется.

После выполнения statement остаток (если что-то после `;`) кладётся обратно в `buffer`. Это позволяет:

```
mydb> SELECT * FROM items; SELECT * FROM items WHERE id = 1;
```

Сначала выполнится первый, потом второй.

39.2.1 Ограничения

Простейший поиск `;` через `find` ломается если `;` **внутри строкового литерала**: `INSERT INTO items VALUES ('hello; world')`. Реальный shell/REPL учитывает контекст (квотинг). У нас в SQL нет строк, поэтому проблема не актуальна.

В **настоящем psql** парсинг даже сложнее — например, `;` внутри функции PL/pgSQL (`CREATE FUNCTION ... AS $$ ... $$`). Они решают через **special delimiters** (`\$$`).

39.3 Meta-команды

Команды, начинающиеся с **точки** — не SQL, а команды REPL'а. `psql` использует `\` (`\q`, `\d`), `sqlite3` — `.` (`.quit`, `.tables`). Мы за `sqlite3`-стиль.

```

bool handle_meta(const std::string& cmd, db::Database& db) {
    std::string c = trim(cmd);
    if (c == ".quit" || c == ".exit") return false;
}

```

```

if (c == ".help") { show_help(); return true; }
if (c == ".history") { show_history(); return true; }
if (c == ".checkpoint") {
    db.checkpoint();
    std::cout << "(checkpoint done)\n";
    return true;
}
std::cout << "Неизвестная meta-команда: " << c << "\n";
return true;
}

```

Возвращает false — пользователь хочет выйти. true — продолжить REPL.

В главном цикле:

```

if (first_line && !t.empty() && t[0] == '.') {
    if (!handle_meta(t, db)) break;
    continue;
}

```

Meta-команда обрабатывается **только в начале**, не в продолжении (где buffer уже не пуст).

39.4 История

История запросов — стандартное удобство. Стрелка вверх (с readline) или просто список через .history.

```

std::vector<std::string> g_history;

std::string history_path() {
    const char* home = std::getenv("HOME");
    if (!home) return "./.mydb_history";
    return std::string(home) + "/.mydb_history";
}

void history_load() {
    std::ifstream in(history_path());
    if (!in.is_open()) return;
    std::string line;
    while (std::getline(in, line)) {

```

```

        if (!line.empty()) g_history.push_back(line);
    }
}

void history_save() {
    std::ofstream out(history_path());
    if (!out.is_open()) return;
    for (const auto& l : g_history) out << l << "\n";
}

void history_add(const std::string& q) {
    if (q.empty()) return;
    if (!g_history.empty() && g_history.back() == q) return;
    g_history.push_back(q);
}

```

То же, что в shell (глава 30). Загружаем при старте, сохраняем при выходе, не дублируем подряд идущие.

.history:

```

void show_history() {
    std::size_t i = 1;
    for (const auto& q : g_history) {
        std::cout << " " << i << " " << q << "\n";
        ++i;
    }
}

```

39.5 Comments

SQL-комментарии — `--` до конца строки. Удаляем перед буферизацией:

```

std::string strip_comment(const std::string& s) {
    auto pos = s.find("--");
    if (pos == std::string::npos) return s;
    return s.substr(0, pos);
}

```

Не учитывает `--` внутри строк (опять, у нас строк нет).

В SQL есть и **многострочные** `/* ... */`. Не реализуем для простоты.

39.6 Запуск

```
$ ./build/mydb
mydb REPL – таблица 'items(id INT PRIMARY KEY, value INT)' по умолчанию.
Open: data/mydb
.help для справки, .quit для выхода.

mydb> CREATE TABLE items (id INT PRIMARY KEY, value INT);
OK (table 'items')

mydb> INSERT INTO items VALUES (1, 100);
1 row

mydb> SELECT *
... FROM items;
id      | value
-----+-----
1       | 100
(1 rows)

mydb> CREATE INDEX idx ON items(value);
Index 'idx' on value (1 rows indexed)

mydb> EXPLAIN SELECT * FROM items WHERE value = 100;
Plan: SecondaryLookup(value=100)

mydb> .history
1 CREATE TABLE items (id INT PRIMARY KEY, value INT);
2 INSERT INTO items VALUES (1, 100);
3 SELECT * FROM items;
4 CREATE INDEX idx ON items(value);
5 EXPLAIN SELECT * FROM items WHERE value = 100;

mydb> .checkpoint
(checkpoint done)
```



```
mydb> .quit
$ cat ~/.mydb_history
CREATE TABLE items (id INT PRIMARY KEY, value INT);
INSERT INTO items VALUES (1, 100);
SELECT * FROM items;
CREATE INDEX idx ON items(value);
EXPLAIN SELECT * FROM items WHERE value = 100;
```

Multi-line через `SELECT *\n FROM items;` собирает запрос. История сохранена в файл.

39.7 Параметры командной строки

```
int main(int argc, char* argv[]) {
    std::string dir = "data/mydb";
    if (argc > 1) dir = argv[1];
    // ...
}
```

`./build/mydb data/customer_db` — открывает другую БД. Это полезно для нескольких баз на одной машине.

В `psql` есть много параметров: `-c "query"` для одной команды, `-f file.sql` для файла, `-h host` для удалённого подключения. Мы — минимум.

39.8 Стрелки вверх/вниз

Чтобы стрелка вверх возвращала предыдущую команду, нужно: 1. Включить **raw mode** для терминала (`tcsetattr` с отключённым `ICANON`). 2. Читать **по одному символу**. 3. Распознавать **escape-sequences**: стрелка вверх это `\033[A`, вниз — `\033[B`, влево — `\033[D`, вправо — `\033[C`. 4. Поддерживать **курсор**: писать символ, потом обновлять позицию. 5. Восстановить **canonical mode** при выходе.

Это **много кода** на сырых `termios` — мы коснёмся этой темы в `shell` (глава 30 говорила: «raw `termios`», не реализовали).

Готовое решение — библиотеки: - **GNU readline** — самая популярная. Используется `bash`, `gdb`, `psql`. LGPL. - **libedit** (BSD) — лицензионная альтернатива. - **linenoise** — крошечная (~1000 строк C), используется `sqlite`, `redis`.

Подключение `readline`:

```
#include <readline/readline.h>
#include <readline/history.h>

while (true) {
    char* line = readline(first_line ? "mydb> " : " ... ");
    if (!line) break;
    std::string sline = line;
    free(line);
    add_history(sline.c_str());
    // ... process sline ...
}
write_history(history_path().c_str());
```

И добавить `-lreadline` к линкеру.

У нас — без readline. Стрелка вверх не работает, но `.history` показывает список.

39.9 Раскраска

psql иногда подсвечивает: ключевые слова синим, строки зелёным. Это делается через **ANSI escape codes**:

```
const char* RED    = "\033[31m";
const char* GREEN  = "\033[32m";
const char* RESET  = "\033[0m";

std::cout << RED << "Error: " << RESET << e.what();
```

Терминал интерпретирует `\033[31m` (ESC + `[31m`) как «начни писать красным», `\033[0m` — «сбрось». В файле или non-terminal stdout эти sequences просто видны как байты — мусор. Поэтому опытные программы проверяют `isatty(STDOUT_FILENO)` и раскрашивают только в терминале.

Мы — без цвета, чтобы не усложнять.

39.10 Архитектура

Структура нашего REPL:

```
main loop
└─ prompt (mydb> или ... )
```

```

└─ getline
└─ strip_comment
└─ meta-command? → handle_meta → continue
└─ append to buffer
└─ ';' in buffer? →
    └─ split: stmt + rest
    └─ db.execute(stmt) → output
    └─ history_add(stmt)
    └─ buffer = rest
└─ repeat

```

on exit:

```

└─ db.checkpoint()
└─ history_save()

```

Простая, прозрачная. Те же концепции, что в shell (глава 30) — отличаются только конкретные команды.

39.11 Главные правила главы

1. **REPL = Read+Eval+Print+Loop.** Базовая модель интерактивной программы.
2. **Multi-line через накопительный buffer и ; как terminator.**
3. **Meta-команды (. или \-prefix)** отдельно от языка БД.
4. **История** в ~/.<app>_history. Не дублируем подряд.
5. **Комментарии --** удаляются до парсинга.
6. **readline для стрелочек и автодополнения.** Самим писать — сложно.
7. **Раскраска через ANSI** — только в isatty.
8. **Параметры командной строки (argv)** — для разных БД, для batch-execution.

39.12 Маленькое упражнение

1. Запустите. Поиграйтесь с командами. Попробуйте многострочные.
2. Запустите снова — увидите, что данные сохранены и .history помнит прошлые команды.
3. Добавьте meta-команду .schema — печатает текущую таблицу и колонки.
4. Добавьте .tables — список всех таблиц (у нас одна).
5. (Сложнее) Реализуйте .read file.sql — выполнить SQL из файла.

6. (Сложнее) Добавьте параметр `-c "query"` — выполнить одну команду и выйти. Полезно для скриптов.
7. (Сложнее) Подключите `libreadline`. Получите стрелки вверх/вниз, `Ctrl+R`, автодополнение по `keywords`.
8. (Очень сложно) Добавьте раскраску ключевых слов в результате `.history` через `ANSI codes`.

39.13 Что дальше

Глава 39 — **бенчмарки и финал Части IV**. Замерим производительность нашей мини-СУБД на тысячах вставок, сравним поведение с/без индекса, обсудим, что осталось «за кадром» (`delete`, `update`, `transactions`, конкурентность, расширенные SQL-фичи).

После этого Часть IV закроется, и мы перейдём к **Части V — ТСП-чат** (главы 40-47): сетевое программирование, многопоточность, реактор-цикл.

Глава 40

Глава 39. Бенчмарки и финал Части IV

Часть IV приближается к концу. У нас в `demo-db/` — почти 2000 строк C++: страничный файл с LRU-кэшем, B+tree-индекс, WAL, мини-SQL, secondary indexes, query planner, REPL. Это **настоящая** СУБД, маленькая, но полная: ACID-A и -D, индексы, persistence.

В этой главе: 1. **Замерим** производительность на разных нагрузках. 2. Подведём **итог** Части IV — что освоили. 3. Обсудим, что **не реализовали** — куда расти. 4. **Сравним** идейно с SQLite, PostgreSQL.

40.1 Бенчмарк

`utils/bench.cpp` запускает: 1. Создать таблицу. 2. **Insert N строк** через SQL. 3. **Checkpoint**. 4. **Find N строк по id** (primary key). 5. **Find 100 строк по value БЕЗ индекса** (full scan каждый раз). 6. **Create secondary index** на value. 7. **Find N строк по value через индекс**.

`make && ./build/bench 500` для 500 строк.

Запуск:

=== Бенчмарк мини-СУБД, N = 500 ===

Note: MAX_KEYS=4 в нашем B+tree (учебное), реальная СУБД 200+

Insert: 232.204 ms, 2153.28 ops/s

Checkpoint: 3.39606 ms

Find by id (B+tree): 89.4436 ms, 5590.11 ops/s

Find by value (full scan, 100 samples): 1556.61 ms, 64.242 ops/s

Build index: 230.755 ms

Find by value (index): 176.584 ms, 2831.52 ops/s

Ускорение find-by-value с индексом: 44.0758x

Числа на моём ноуте (Apple M3, debug-сборка). Что важно:

40.1.1 Insert: 2.1k ops/s

Для каждой вставки: `-wal.log_insert + fsync` — миллисекунда на NVMe. - B+tree insert (split возможен).

`fsync` доминирует. Если бы делать **group commit** (см. главу 35), один `fsync` на 100 вставок дал бы ~100× ускорение.

Реальный SQLite: ~50k inserts/s **с WAL и без fsync на каждую** (синхронный режим NORMAL). С `synchronous = FULL` (как у нас) — несколько тысяч.

PostgreSQL: ~10-20k inserts/s по умолчанию, до 100k с группированием и SSD.

40.1.2 Find by id: 5.6k ops/s

Через B+tree, $O(\log_4 N) \approx 4-5$ IO. С page cache всё в памяти, IO дешёвые. Не быстрее, потому что overhead парсинга SQL для каждого запроса — десятки микросекунд.

В SQLite prepared statement переиспользует план; запрос быстрее. У нас всё парсится заново.

40.1.3 Find by value (без индекса): 64 ops/s

Full scan на 500 строк = 500 точек проверки. $64 \text{ ops/s} \times 500 \text{ точек} = 32\text{k}$ операций сравнения в секунду. Не быстро — overhead `tree.scan()` (читать каждую leaf-страницу), плюс парсинг SQL.

40.1.4 Find by value (с индексом): 2.8k ops/s

С индексом — `idx.tree.find() + tree.find()` = 2 поиска по B+tree. Меньше IO, ближе к find by id.

Ускорение 44× на 500 строках. На миллионе было бы тысячи×.

40.1.5 Что замедляет нашу мини-СУБД

Несколько узких мест:

1. **MAX_KEYS = 4** — дерево неестественно глубокое. С 250 высота была бы 1-2.
2. **fsync на каждом INSERT** — без group commit не масштабируется.
3. **Парсинг SQL каждый раз** — нет prepared statements.

4. **Debug-сборка** — без -O2. С -O2 всё в 2-5 раз быстрее.
5. **Маленький cache (64 страницы)** — на больших данных промахи.
6. **Одинаковая сериализация в WAL и B+tree** — каждое поле проходит метаску 2-3 раза.

С исправлениями каждый из пунктов даёт улучшение. Это **обычный путь** оптимизации: профилируйте, найдите узкое место, исправьте, повторите.

40.1.6 Сравнение с release-сборкой

```
$ make MODE=release
$ ./build/bench 1000
Insert:    ~200 ms, 5000 ops/s (вместо 2153)
Find:      ~50 ms, 20000 ops/s
```

-O2 ускоряет ~2-3 раза. Inline-функции, удаление лишних копий, лучше vectorization.

40.2 Что мы построили — обзор

demo-db/ — 1800 строк C++ в 14 файлах:

```
include/
├─ binary.h      — write/read u16/u32/u64/string в LE
├─ btree.h       — B+tree
├─ database.h    — фасад
├─ page.h        — PAGE_SIZE, Page = vector<unsigned char>
├─ page_manager.h — страничный файл + LRU cache
├─ sql.h         — Statement, parse_sql
├─ util.h
└─ wal.h         — Write-Ahead Log
src/
├─ btree.cpp     — поиск, вставка, split (~240 строк)
├─ database.cpp  — execute_create/insert/select + query planner (~250 строк)
├─ main.cpp      — REPL (~120 строк)
├─ page_manager.cpp — LRU, pread/pwrite, fsync (~150 строк)
├─ sql.cpp       — токенизатор + recursive descent parser (~180 строк)
└─ wal.cpp       — append-only log + replay (~150 строк)
utils/
└─ bench.cpp     — бенчмарк (~80 строк)
```

Возможности: - **CREATE TABLE / CREATE INDEX / INSERT / SELECT / EXPLAIN**. - **Primary key индекс** автоматически, **secondary indexes** по запросу. - **Query planner** с 4 стратегиями.

- **Persistence** в файлы. - **Durability** через WAL + fsync. - **Recovery** после краха через replay. - **Checkpoint** = sync + truncate WAL. - **REPL** с multi-line, history, meta-командами.

Это архитектура реальной СУБД в миниатюре.

40.3 Что НЕ реализовали

Реальная СУБД в десятки раз больше. Из нашего списка отсутствуют:

40.3.1 Delete и Update

```
DELETE FROM items WHERE id = 1;
UPDATE items SET value = 999 WHERE id = 1;
```

DELETE в B+tree сложен — borrow или merge с соседом, рекурсивно вверх. UPDATE = find + modify in place. Парсер + executor расширяются легко; B+tree.remove — много кода.

40.3.2 Транзакции

```
BEGIN;
INSERT ...;
INSERT ...;
COMMIT;
-- или ROLLBACK;
```

Группа операций — либо все применены, либо все отменены. Реализация: - При BEGIN — отметить точку в WAL. - При COMMIT — записать COMMIT-запись. - При ROLLBACK — записать ROLLBACK + replay не применяет операции этой транзакции.

Уровни изоляции (READ COMMITTED, REPEATABLE READ, SERIALIZABLE) — отдельная тема.

40.3.3 JOIN

```
SELECT * FROM orders o JOIN customers c ON o.cust_id = c.id;
```

Несколько таблиц + связь между. Алгоритмы: nested loop, hash join, merge join. **Самая сложная** часть SQL после optimizer.

40.3.4 Aggregates


```
SELECT COUNT(*), AVG(value) FROM items;  
SELECT category, SUM(value) FROM items GROUP BY category;
```

COUNT, SUM, AVG, MIN, MAX плюс GROUP BY. Парсер + специальный executor.

40.3.5 Concurrent access

Несколько процессов/потоков одновременно. Требуется: - **Locking** на уровне строки/страницы. - **MVCC** (multi-version concurrency control) — у каждой транзакции свой snapshot. - **Deadlock detection**.

PostgreSQL, MySQL InnoDB — MVCC. SQLite — file-level locking.

40.3.6 Schema persistence

Сейчас table_name_/col_names_ живут в памяти. Запустили программу заново — потеряли. В реальной СУБД схема — в системной таблице (pg_class и т.п.) или в metadata-страницах нашего файла.

40.3.7 Replication

Master-slave: запись на master, чтение со slave. Делается через **отправку WAL** на slave + replay там. PostgreSQL streaming replication, MySQL binlog.

40.3.8 Triggers / Stored procedures

CREATE TRIGGER ..., CREATE FUNCTION Свой язык программирования внутри SQL (PL/pgSQL в PostgreSQL, T-SQL в MS SQL). **Огромная** тема.

40.3.9 Foreign keys, constraints

REFERENCES other_table(id). Проверка на каждом INSERT/UPDATE/DELETE — целостность данных.

40.3.10 Distributed

Шардинг между серверами, координатор запросов, конс-протоколы (Raft, Paxos). Эта область — **distributed databases**: CockroachDB, Spanner, Cassandra.

Каждое — отдельная книга-другая.

40.4 Сравнение идейно: наш ☒ SQLite ☒ PostgreSQL

Свойство	Наш	SQLite	PostgreSQL
Размер кода	1.8k LOC	~150k	~1.5M
Языки	C++	C	C
Storage	B+tree	B+tree	B+tree (Heap + B-tree)
Page size	4 KB	4 KB (default)	8 KB
WAL	Logical, простой	Physical, WAL mode	Physical, full
Транзакции	Нет	Да, ACID	Да, ACID + MVCC
Параллелизм	Нет	File-level lock	MVCC
SQL features	CREATE/INSERT/SELECT	SQL-92	SQL-92 + расширения
Joins	Нет	Все алгоритмы	Все алгоритмы + planner
Replication	Нет	Нет	Streaming + logical
Use case	Учебный	Embedded, mobile	Production server

SQLite — отличный пример «маленькой» СУБД, которую можно прочесть целиком за месяц. У нас её упрощённая версия.

PostgreSQL — индустриальный стандарт. Те же концепции, но в десятки раз больше деталей.

40.5 Чему мы научились

Часть IV была про:

- **Низкоуровневую работу с файлами:** pread/pwrite, fsync, mmap (упоминание).
- **Бинарные форматы:** endianness, alignment, packed structures, explicit serialization.
- **Структуры данных на диске:** B+tree поверх страниц.
- **Durability и atomicity:** WAL, recovery, checkpoint, group commit.
- **Парсинг:** tokenizer + recursive descent + AST + executor.
- **Простой query optimization:** rule-based planner, EXPLAIN.
- **Архитектуру СУБД:** страницы → tree → WAL → SQL → REPL.

И **главное:** вы теперь знаете, **как устроена БД изнутри**. Когда напишете SQL-запрос в production, вы понимаете, что движок делает. Какие операции дорогие. Зачем нужны индексы. Почему EXPLAIN показывает план. Это знание ценнее, чем тысячи запросов на собеседовании.

40.6 Главные правила всей Части IV

1. **Страницы фиксированного размера** — основа всех СУБД.
2. **B+tree** для упорядоченного индекса. Большой fan-out, мало IO.
3. **WAL** для durability — пиши сначала в журнал, fsync.
4. **Идемпотентные операции** позволяют replay при recovery.
5. **Indexes** — двойной меч: быстрое чтение, медленная запись.
6. **EXPLAIN** до запуска — поймите плохой план до отправки в production.
7. **Бенчмарк рано** — не оптимизируйте на глаз, измеряйте.
8. **Простая архитектура — основа.** Сложности добавляются по мере роста.

40.7 Маленькое упражнение

1. Соберите. Запустите бенчмарк на разных N: 100, 1000, 10000. Постройте таблицу. Где O(N), где O(log N)?
2. Запустите `make MODE=release + бенчмарк`. Сравните с `debug`.
3. Увеличьте `MAX_KEYS` в `btree.h` до 64, 256. Сравните производительность.
4. (Сложнее) Реализуйте `DELETE` и `UPDATE` на основе нашего парсера. Без `B+tree.remove` можно: для `DELETE` по `pkey` — поставить tombstone (специальный value), сканировать игнорируя; для `UPDATE` — `overwrite` (`B+tree` уже поддерживает).
5. (Сложнее) Реализуйте `COUNT(*)` — добавьте поддержку `SELECT COUNT(*) FROM ...`. Парсер должен распознать функцию.
6. (Сложнее) Замените `fsync` на каждой записи на **group commit**: накапливать N записей, `fsync` один раз. Замерьте, насколько быстрее.
7. (Очень сложно) Поддержите простые транзакции: `BEGIN/COMMIT/ROLLBACK`. При `ROLLBACK` откатить все изменения с последнего `BEGIN` через `WAL`.
8. (Очень сложно) Реализуйте `SELECT ... FROM A, B WHERE A.id = B.id` через nested-loop join.

40.8 Часть IV закрыта

Что у нас в `demo-db/`:

- 1750 строк рабочего кода C++.
- Полнофункциональная мини-СУБД.
- Бенчмарк программа.
- REPL клиент.

Что освоили концептуально:

- POSIX file IO низкоуровневый.
- B+tree алгоритм с операциями insert/find/split.
- WAL архитектура durability/recovery.
- Query parser + planner + executor.

40.9 Что дальше

Часть V — ТСП-чат (главы 40-47): сетевое программирование. Сокеты, многопоточность, atomic, реактор-цикл epoll/kqueue, собственный бинарный протокол, многокомнатный чат с историей.

Это **четвёртый и последний** большой проект книги. После него — **Часть VI**: C++17 как бонус (optional, variant, filesystem, string_view, ranges/concepts/coroutines обзорно).

Конец книги виден. Осталось ~10 глав.

Глава 41

Глава 40. BSD-сокеты с нуля

Часть V — **TCP-чат**. Восемь глав сетевого программирования: сокеты, многопоточность, atomic, реактор-цикл, бинарные протоколы, многокомнатный чат. К концу — рабочий сервер и клиент, на котором можно общаться.

Эта глава — фундамент. **BSD-сокеты** — это POSIX API для сетевой работы. Тот же на Linux, macOS, в Cygwin/MinGW на Windows. Идея проста: ОС даёт «socket» — файловый дескриптор, который можно read/write, как обычный файл. Под капотом ОС обрабатывает TCP-пакеты, АСК'и, retransmission, окна. Программа видит **поток байтов**.

Напишем простой **echo-server**: принимает соединения, отвечает каждой строкой обратно. Плюс клиент. Это «hello world» сетевого программирования.

41.1 Что такое socket

socket — это endpoint двусторонней коммуникации. Каждый сокет — это: - **Адрес** (IP + порт): откуда/куда. - **Тип**: TCP (поточный, гарантированная доставка) или UDP (пакетный, без гарантий). - **Файловый дескриптор**: программа работает с ним как с файлом — read, write, close.

Идея «всё файл» в Unix действительно глубокая. Сокет — это fd, как stdin или открытый файл. На нём работают pread/pwrite... ну, на сокетах их нет, но есть похожие recv/send.

41.2 TCP vs UDP

TCP (Transmission Control Protocol): - Поток байтов между двумя endpoints. - Гарантированная доставка, в правильном порядке. - Reconnect при разрывах. - Управление потоком (sliding window). - Прозрачно для приложения. - Чуть медленнее UDP из-за overhead.

UDP (User Datagram Protocol): - Дискретные пакеты (datagrams). - Без гарантий — может потеряться, прийти из порядка, дублироваться. - Минимальный overhead. - Используется в DNS, видеоконференциях, играх.

Для **чата** TCP — естественный выбор. Гарантия доставки важна.

41.3 API

POSIX-сокеты — функции из `<sys/socket.h>`, `<netinet/in.h>`, `<arpa/inet.h>`, `<netdb.h>`:

Функция	Что делает
<code>socket(domain, type, proto)</code>	Создать сокет, вернуть fd
<code>bind(fd, addr, len)</code>	Привязать сокет к адресу (для сервера)
<code>listen(fd, backlog)</code>	Перевести в режим прослушивания (для сервера)
<code>accept(fd, ...)</code>	Принять входящее соединение, вернуть новый fd
<code>connect(fd, addr, len)</code>	Подключиться к удалённому адресу (для клиента)
<code>send(fd, buf, n, flags)</code>	Послать данные
<code>recv(fd, buf, n, flags)</code>	Принять данные
<code>close(fd)</code>	Закрыть
<code>setsockopt(fd, ...)</code>	Настройки сокета
<code>getaddrinfo(host, port, ...)</code>	Резолв host:port в адреса

`send/recv` похожи на `write/read` (которые тоже работают на сокетах). Разница — у них есть флаги (например, `MSG_PEEK` чтобы посмотреть без потребления).

41.4 Жизненный цикл сервера

```

socket()
  ↓
setsockopt(SO_REUSEADDR)
  ↓
bind(addr:port)
  ↓
listen()
  ↓
while (true):
    accept() → новый fd для клиента
      ↓

```

```

recv/send цикл
    ↓
close(client)
    ↓
close(server)

```

Семь системных вызовов, и сервер готов.

41.5 Жизненный цикл клиента

```

getaddrinfo(host, port)
    ↓
socket()
    ↓
connect(addr)
    ↓
send/recv цикл
    ↓
close()

```

Проще — клиент только подключается и общается, не слушает.

41.6 socket()

```
int fd = ::socket(AF_INET, SOCK_STREAM, 0);
```

- **AF_INET** — IPv4. Для IPv6 — **AF_INET6**. Для Unix sockets (локальные) — **AF_UNIX**.
- **SOCK_STREAM** — TCP. Для UDP — **SOCK_DGRAM**.
- **0** — protocol auto (для **AF_INET** + **SOCK_STREAM** это TCP).

Возвращает **fd** при успехе, -1 при ошибке (с **errno**).

41.7 sockaddr_in для IPv4

Адрес в IPv4 описывается структурой:

```

struct sockaddr_in {
    sa_family_t    sin_family;    // AF_INET
    in_port_t      sin_port;      // порт в network byte order
    struct in_addr sin_addr;      // IP-адрес

```

```
char        sin_zero[8]; // padding
};
```

Заполнить для прослушивания «всех интерфейсов на порту 9001»:

```
sockaddr_in addr;
std::memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(9001);
addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

memset нулями — обязательно. Поле `sin_zero` должно быть пустым.

htons / **htonl** (host-to-network short/long) — конверсия в **network byte order** (big-endian). Сетевые протоколы исторически big-endian. Без `htons` порт 9001 на Intel прочтётся неправильно.

`INADDR_ANY` (0.0.0.0) — «любой интерфейс на этой машине». Альтернатива — `INADDR_LOOPBACK` (127.0.0.1) — только localhost.

41.7.1 sockaddr и его потомки

В API часто параметр — `sockaddr*`, не `sockaddr_in*`. Это «базовый» тип. Реально для IPv4 передаём `sockaddr_in*`, приведя через `cast`:

```
bind(fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
```

Это как полиморфизм в C: одна функция, разные структуры. Внутри `bind` смотрит на `addr.sin_family` и понимает, что это.

41.8 bind()

```
::bind(srv, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
```

Привязать сокет к адресу. После этого ассепт будет принимать соединения именно на этот IP:port.

Самая частая ошибка — **EADDRINUSE** «адрес занят». Бывает, если сервер недавно закрылся, а ОС держит порт в `TIME_WAIT` (защита от перепутанных пакетов от предыдущего соединения).

SO_REUSEADDR:

```
int yes = 1;
::setsockopt(srv, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));
```


Сказать ядру «разреши перепривязку даже в TIME_WAIT». Стандартный приём для серверов, которые часто перезапускаются (разработка, тесты).

41.9 listen()

```
::listen(srv, 16);
```

Перевести сокет в **режим прослушивания**. С этого момента ОС начинает принимать TCP handshake (SYN-SYN/ACK-ACK) от клиентов и **складывать готовые соединения** в очередь.

16 — **backlog**, длина очереди. Если приходит больше клиентов, чем мы успеваем обработать через ассепт — лишние отбрасываются (получают TCP reset). На простом сервере 16 хватает; на нагруженных — 128, 1024, 65535.

41.10 accept()

```
sockaddr_in cli;  
socklen_t cli_len = sizeof(cli);  
int c = ::accept(srv, reinterpret_cast<sockaddr*>(&cli), &cli_len);
```

Блокирующий вызов. Ждёт, пока в очереди появится готовое соединение, потом возвращает **новый fd** для общения с этим клиентом.

Параметры cli / cli_len — выходные: ОС заполняет адрес клиента (его IP/port). cli_len — сначала указывает размер буфера, после вызова — сколько реально записано.

После ассепт: - **srv** остаётся в listen-режиме для следующих клиентов. - **c** — новый fd, через него send/recv.

Один сервер, **один accept-loop**, много client-fd одновременно (если многопоточность; пока один за раз).

41.11 recv() / send()

```
char buf[1024];  
ssize_t n = ::recv(c, buf, sizeof(buf), 0);
```

Прочитать до n байтов от клиента. Возвращает: - **0** — клиент **закрыл** соединение (graceful close). Это нормальный сценарий! - **> 0** — число прочитанных байт (может быть меньше запрошенного — short read). - **-1** — ошибка (errno уточняет).

Особый `errno`: - **EINTR** — прерывание сигналом. Повторяем (как мы делали в `shell`-утилитах). - **ECONNRESET** — клиент сбросил без `graceful close` (`kill -9` на клиенте). Тоже «нормально» — клиент пропал.

```
ssize_t sent = ::send(c, buf, n, 0);
```

Отправить. Возвращает реально отправленное (может быть меньше). Нужен цикл, как для `write` в `shell`:

```
ssize_t total = 0;
while (total < n) {
    ssize_t s = ::send(c, buf + total, n - total, 0);
    if (s < 0) {
        if (errno == EINTR) continue;
        break;
    }
    total += s;
}
```

Флаги (последний параметр) обычно 0. Реже используются: - `MSG_PEEK` — посмотреть, не потребляя. - `MSG_DONTWAIT` — non-blocking даже на blocking-сокетe. - `MSG_NOSIGNAL` — на Linux: не посылать `SIGPIPE` при записи в закрытый сокет. Очень полезно для серверов.

41.12 close()

```
::close(c);
```

Закрыть. На TCP-сокетe это инициирует **graceful close** (FIN-пакет). Другая сторона получит `recv() == 0`.

Альтернатива — `shutdown(fd, SHUT_WR)` — закрыть **только запись**, оставив возможность читать ответ. Полезно для протоколов вроде HTTP, где клиент шлёт запрос, говорит «я кончил», и слушает ответ.

41.13 getaddrinfo для клиента

```
addrinfo hints;
std::memset(&hints, 0, sizeof(hints));
hints.ai_family = AF_UNSPEC;    // и IPv4, и IPv6
hints.ai_socktype = SOCK_STREAM;
```

```
addrinfo* res = nullptr;
::getaddrinfo("example.com", "9001", &hints, &res);
```

getaddrinfo делает DNS-резолв: имя → список IP-адресов. Возвращает связный список addrinfo структур.

AF_UNSPEC — «не важно, IPv4 или IPv6». ОС вернёт обе версии, мы попробуем поочерёдно.

Перебираем:

```
int fd = -1;
for (addrinfo* p = res; p; p = p->ai_next) {
    fd = ::socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (fd < 0) continue;
    if (::connect(fd, p->ai_addr, p->ai_addrlen) == 0) break;
    ::close(fd);
    fd = -1;
}
::freeaddrinfo(res);
```

::freeaddrinfo(res) обязательно — это malloc’нутый связный список.

Это **правильный** способ работы с адресами. **Не** парсить «1.2.3.4:5678» руками — getaddrinfo умеет и IPv4-литералы, и IPv6, и DNS-имена.

41.14 inet_ntop для печати

После ассепт мы получили sockaddr_in клиента. Как напечатать «1.2.3.4»?

```
char ip[INET_ADDRSTRLEN] = {0};
::inet_ntop(AF_INET, &cli.sin_addr, ip, sizeof(ip));
std::cout << ip << ":" << ntohs(cli.sin_port);
```

inet_ntop (network-to-presentation) превращает бинарный адрес в текстовый. INET_ADDRSTRLEN (16) — достаточный буфер для «255.255.255.255\0».

Для IPv6 — AF_INET6 и INET6_ADDRSTRLEN (46).

41.15 Echo-server — полная сборка

utils/echo_server.cpp:

```
int main(int argc, char* argv[]) {
    int port = (argc > 1) ? std::atoi(argv[1]) : 9001;

    int srv = ::socket(AF_INET, SOCK_STREAM, 0);

    int yes = 1;
    ::setsockopt(srv, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    ::bind(srv, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
    ::listen(srv, 16);

    std::cout << "слушаем на :" << port << "\n";

    while (true) {
        sockaddr_in cli;
        socklen_t cli_len = sizeof(cli);
        int c = ::accept(srv, reinterpret_cast<sockaddr*>(&cli), &cli_len);
        if (c < 0) { if (errno == EINTR) continue; break; }

        // recv/send loop для одного клиента
        char buf[1024];
        while (true) {
            ssize_t n = ::recv(c, buf, sizeof(buf), 0);
            if (n == 0) break;
            if (n < 0) { if (errno == EINTR) continue; break; }
            ssize_t sent = 0;
            while (sent < n) {
                ssize_t s = ::send(c, buf + sent, n - sent, 0);
                if (s < 0) { if (errno == EINTR) continue; break; }
                sent += s;
            }
        }
        ::close(c);
    }
}
```

```
    }  
    ::close(srv);  
}
```

100 строк. Это **полнофункциональный** TCP-сервер. Работает с telnet, nc, curl, любым клиентом, говорящим TCP.

41.16 Client

utils/echo_client.cpp:

```
int main(int argc, char* argv[]) {  
    const char* host = (argc > 1) ? argv[1] : "127.0.0.1";  
    const char* port = (argc > 2) ? argv[2] : "9001";  
  
    addrinfo hints; std::memset(&hints, 0, sizeof(hints));  
    hints.ai_family = AF_UNSPEC;  
    hints.ai_socktype = SOCK_STREAM;  
  
    addrinfo* res = nullptr;  
    ::getaddrinfo(host, port, &hints, &res);  
  
    int fd = -1;  
    for (addrinfo* p = res; p; p = p->ai_next) {  
        fd = ::socket(p->ai_family, p->ai_socktype, p->ai_protocol);  
        if (fd < 0) continue;  
        if (::connect(fd, p->ai_addr, p->ai_addrlen) == 0) break;  
        ::close(fd);  
        fd = -1;  
    }  
    ::freeaddrinfo(res);  
  
    std::string line;  
    while (std::getline(std::cin, line)) {  
        if (line.empty()) break;  
        line += '\n';  
        ::send(fd, line.data(), line.size(), 0);  
  
        char buf[1024];
```

```
    ssize_t n = ::recv(fd, buf, sizeof(buf), 0);
    if (n <= 0) break;
    std::cout << "<- " << std::string(buf, n);
}
::close(fd);
}
```

Пользователь вводит строку → клиент шлёт → ждёт ответ → печатает.

41.17 Запуск

Терминал 1:

```
$ ./build/echo_server 9099
слушаем на :9099
```

Терминал 2:

```
$ ./build/echo_client 127.0.0.1 9099
Подключён к 127.0.0.1:9099.
> hello
<- hello
> world
<- world
```

В терминале 1 параллельно:

```
Клиент 127.0.0.1:54032 подключён
клиент отключился
```

Можно подключиться и через `telnet localhost 9099` — будет работать.

41.18 Что наш сервер НЕ умеет

Прежде чем переходить к улучшениям — что сейчас не работает:

1. **Один клиент за раз.** Пока обслуживаем одного — другие висят в backlog (или отказываются после переполнения).
2. **Любой сигнал прерывает.** Ctrl+C на сервере убивает.
3. **Нет защиты от медленных клиентов.** Если клиент висит на recv (медленная сеть) — сервер ждёт.
4. **Никакой логики чата.** Только echo.

Решения:

- **Один-поток-на-клиента:** в асепт-loop форкаем поток для каждого клиента. Это глава 41.
- **Реактор:** один поток обрабатывает много клиентов через select/poll/epoll. Глава 43.
- **Свой протокол** для разделения сообщений. Глава 44.

41.19 SO_LINGER и другие настройки

Несколько других полезных setsockopt:

- **SO_LINGER** — что делать при close с непрочитанными данными. По умолчанию ОС асинхронно сбрасывает их. С `linger {true, N}` — close ждёт N секунд.
- **TCP_NODELAY** — отключить Nagle's algorithm. Полезно для интерактивных приложений (chat, gaming): каждый send отправляется немедленно, не накапливается.
- **SO_KEEPALIVE** — periodic keepalive-пакеты. Обнаруживают «мёртвые» соединения.
- **SO_RCVBUF** / **SO_SNDBUF** — размер буферов в ядре. Большие — для bulk-передачи; маленькие — для low-latency.

Эти настройки — на 90% можно не трогать. Дефолты ОС разумные.

41.20 TIME_WAIT и почему важен SO_REUSEADDR

Когда TCP-соединение закрывается через `close`, оно проходит несколько состояний. Последнее у инициатора `close` — **TIME_WAIT** на $2 \times \text{MSL}$ (Maximum Segment Lifetime, обычно 30-120 секунд).

Зачем: страховка от ложных пакетов. Если предыдущее соединение оставило пакет «в эфире», и новое соединение использовало бы тот же ip:port — пакет может прийти в новое соединение и испортить данные.

Для **разработчика** это значит: после Ctrl+C сервера попытка перезапустить = EADDRINUSE. Подождать минуту или поставить `SO_REUSEADDR`.

В production обычно настраивают и оставляют `SO_REUSEADDR=1`.

41.21 Главные правила главы

1. **Socket = fd.** Все правила POSIX file IO применимы.
2. **TCP для гарантированной доставки**, UDP для скорости.
3. **htons/htonl** для портов и IP. Network byte order — big-endian.

4. **SO_REUSEADDR** на серверах — обязательно для удобного перезапуска.
5. `recv() == 0` означает graceful close. `< 0` — ошибка.
6. **getaddrinfo** для DNS-резолва и IPv4/IPv6 совместимости.
7. **send/recv могут возвращать меньше запрошенного**. Цикл обязателен.
8. **MSG_NOSIGNAL** для серверов — без SIGPIPE.

41.22 Маленькое упражнение

1. Соберите. Запустите сервер. Подключитесь клиентом. Поиграйтесь.
2. Запустите параллельно два клиента (два терминала). Что произойдёт? Один работает, второй висит до отключения первого.
3. Запустите `telnet localhost 9099` вместо нашего клиента. Поработает?
4. Попробуйте `curl -X POST -d "hello" http://localhost:9099` — наш сервер не понимает HTTP, но что-то увидите. Что?
5. Сделайте сервер на IPv6: измените `AF_INET` на `AF_INET6`, `sockaddr_in` на `sockaddr_in6`. Проверьте через `telnet ::1 9099`.
6. (Сложнее) Добавьте обработку SIGINT в сервере: закрыть `srv` сокет, выйти из accept-loop корректно.
7. (Сложнее) Сделайте «лимит на сообщение»: после получения N байтов от клиента — отключайте его.
8. (Сложнее) Прочитайте `man 2 socket`, `man 2 bind`, `man 7 tcp`. Особое внимание — `tcp(7)` со списком `SO_*` опций.

41.23 Что дальше

Глава 41 — **многопоточность**: `std::thread`, `std::mutex`, `std::condition_variable`. Делаем модель «один поток на клиента» — сервер сможет обслуживать многих параллельно. Обсуждаем `deadlock`, `race conditions`, как защищать общие данные.

Дальше — `atomic` и `memory ordering` (42), реактор `select/poll/epoll/kqueue` (43), бинарный протокол (44), многокомнатный чат (45), шифрование+история (46), сборка и деплой (47).

Глава 42

Глава 41. Многопоточность — `std::thread`, `mutex`, `condition_variable`

Наш echo-server из главы 40 обслуживает **одного клиента за раз**. Пока один общается — другие висят в очереди. Это неприемлемо для чата: пользователи должны переписываться **одновременно**.

Решение — **один поток на клиента**. Server-loop принимает соединения, для каждого forks-новый `std::thread`. Все потоки работают параллельно, читают свои сокеты, общаются с общими данными (списком клиентов).

В этой главе изучим инструменты C++11 для многопоточности: `std::thread`, `std::mutex`, `std::lock_guard`, `std::condition_variable`. И сразу применим — превратим echo-server в **multi-client чат с broadcast**: что один сказал, все услышали.

42.1 Что такое поток

Поток (thread) — это исполняющаяся последовательность инструкций внутри процесса. У потоков общая память процесса, но отдельные стеки. ОС переключает процессор между потоками — на 8 ядрах могут реально выполняться 8 потоков параллельно.

Concurrent vs parallel: - **Concurrent** — много вещей идут «одновременно» с точки зрения логики. На одноядерном CPU тоже concurrent (через переключение). - **Parallel** — реально одновременно на нескольких CPU.

C++11 ввёл стандартную поддержку. До этого — POSIX `pthread_*`, Windows `CreateThread`. Сейчас — `std::thread`, переносимо.

42.2 std::thread

```
#include <thread>
#include <iostream>

void worker(int id) {
    std::cout << "thread " << id << " работает\n";
}

int main() {
    std::thread t(worker, 42);
    t.join();    // ждать завершения
}
```

std::thread t(func, args...) — создать поток, запустить func(args...) параллельно.

t.join() — заблокировать текущий поток до завершения t. Аналог waitpid для процессов.

t.detach() — «забыть» поток. Он работает в фоне, мы за ним не следим. Когда программа завершается, detached-поток тоже умирает (как daemon threads в Java).

```
std::thread t(worker, 42).detach();    // fire-and-forget
```

Это то, что мы будем использовать для chat-server: на каждого клиента — поток, который сам уберётся при отключении клиента.

Опасность detach: если поток обращается к стеку main'a, который уже разрушен — UB. У нас потоки работают со своим аргументом (fd) и глобальными данными — безопасно.

42.2.1 Что захватывает std::thread

std::thread(func, arg) **копирует** или **перемещает** аргументы в свой контекст. Если хотите передать по ссылке — std::ref(x):

```
int counter = 0;
std::thread t(increment, std::ref(counter));    // counter передан по ссылке
```

Лямбды тоже работают:

```
std::thread t1([]() { ... });
std::thread t2([n = 5]() { ... });    // capture
```

42.3 Race condition

Два потока меняют **одну переменную** — может быть гонка:

```
int counter = 0;

void inc() {
    for (int i = 0; i < 1000000; ++i) ++counter;
}

int main() {
    std::thread a(inc), b(inc);
    a.join(); b.join();
    std::cout << counter;    // ??? Ожидаемо 2000000, реально меньше
}
```

++counter это **три** машинных инструкции: read, increment, store. Между read одного потока и store другого — другой поток может тоже сделать read+inc+store. Часть инкрементов **теряется**.

Это **data race** — UB по стандарту C++. Программа может вернуть что угодно.

Защита: либо **мьютекс** (lock), либо **atomic** (специальные типы — глава 42).

42.4 std::mutex

std::mutex (mutual exclusion) — синхронизационный объект. В каждый момент его держит **не больше одного** потока. Остальные ждут.

```
#include <mutex>

int counter = 0;
std::mutex m;

void inc() {
    for (int i = 0; i < 1000000; ++i) {
        m.lock();
        ++counter;
        m.unlock();
    }
}
```

Теперь только один поток в любой момент в ++counter. Race нет.

Проблема lock/unlock вручную: если ++counter бросит исключение — unlock не выполнится, мьютекс **навсегда заложен**, другие потоки висят. Это deadlock.

42.4.1 lock_guard — RAII

Решение — std::lock_guard:

```
void inc() {
    for (int i = 0; i < 1000000; ++i) {
        std::lock_guard<std::mutex> lock(m);
        ++counter;
    }
}
```

lock_guard в конструкторе **захватывает** мьютекс, в деструкторе **отпускает**. Это **RAII** (глава 11). При любом выходе из scope — исключении, return, break — unlock гарантированно срабатывает.

В современном C++ **никогда не пишите** lock()/unlock() руками. Всегда lock_guard или unique_lock.

В C++17 — std::scoped_lock, может лочить несколько мьютексов сразу без deadlock.

42.4.2 unique_lock

std::unique_lock — более гибкий вариант:

```
std::unique_lock<std::mutex> lock(m);
// что-то делаем
lock.unlock(); // явный unlock
// что-то ещё без lock
lock.lock();   // снова взять
```

Может временно отпустить мьютекс. Нужен для condition_variable (ниже).

lock_guard проще и быстрее. Используйте его, кроме случаев когда нужна гибкость.

42.5 Atomicity vs locking

std::atomic<int> — special тип, **атомарные** операции на нём гарантированы:

```
#include <atomic>
std::atomic<int> counter{0};
```

```
void inc() {  
    for (int i = 0; i < 1000000; ++i) ++counter;  
}
```

Никакого lock_guard. ++counter под капотом — атомарная инструкция процессора (на x86 — LOCK XADD).

Atomic быстрее **mutex** для простых операций (один счётчик). Mutex нужен для **сложных** критических секций (несколько связанных переменных).

Подробнее про atomic — глава 42.

42.6 condition_variable

Иногда поток должен **ждать** условия. Например, worker ждёт, пока появится задача в очереди.

std::mutex решает «не наступайте друг другу на пятки», но **не** «дождись, пока что-то произойдёт».

std::condition_variable:

```
std::mutex m;  
std::condition_variable cv;  
std::queue<Task> tasks;  
bool stop = false;  
  
void worker() {  
    while (true) {  
        std::unique_lock<std::mutex> lock(m);  
        cv.wait(lock, [&]() { return stop || !tasks.empty(); });  
        if (stop && tasks.empty()) return;  
        Task t = std::move(tasks.front());  
        tasks.pop();  
        lock.unlock();  
        t.run();  
    }  
}  
  
void enqueue(Task t) {  
    {  
        std::lock_guard<std::mutex> lock(m);
```

```

        tasks.push(std::move(t));
    }
    cv.notify_one();
}

```

cv.wait(lock, predicate): 1. Проверяет predicate. Если true — продолжает. 2. Иначе — **отпускает мьютекс** и засыпает. 3. Кто-то вызывает cv.notify_one() — поток просыпается, **берёт мьютекс обратно**, проверяет предикат, повторяет если false.

notify_one() — разбудить **один** ждущий поток. notify_all() — все.

Это **producer-consumer pattern**. Producer вызывает enqueue, consumer — worker. Без condition_variable worker бы делал busy-wait (постоянная проверка), нагружая CPU.

42.7 Deadlock

Самая страшная проблема многопоточности. Поток А держит мьютекс X, ждёт Y. Поток В держит Y, ждёт X. **Оба висят навсегда.**

```

std::mutex mx, my;

void a() {
    std::lock_guard<std::mutex> la(mx);
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    std::lock_guard<std::mutex> lb(my);    // ждём my
}

void b() {
    std::lock_guard<std::mutex> lb(my);
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    std::lock_guard<std::mutex> la(mx);    // ждём mx
}

```

Запустите параллельно — deadlock.

42.7.1 Профилактика

1. **Всегда брать мьютексы в одном порядке.** Сначала mx, потом my. Если все потоки соблюдают — deadlock невозможен.
2. **std::scoped_lock** (C++17) — берёт несколько мьютексов **атомарно** через специальный алгоритм:

```
std::scoped_lock lock(mx, my);
```

Безопасно.

3. **std::lock(mx, my)** в C++11 — то же:

```
std::lock(mx, my);
std::lock_guard<std::mutex> la(mx, std::adopt_lock);
std::lock_guard<std::mutex> lb(my, std::adopt_lock);
```

Громоздко, в C++17 заменено `scoped_lock`.

4. **Hierarchy lock** — нумеруйте мьютексы, берите только в возрастающем порядке.

В нашем chat-server **один мьютекс** на список клиентов — deadlock невозможен.

42.8 Многопоточный chat-server

Идея проста:

main loop:

```
accept() → c
std::thread(handle_client, c).detach();
```

Каждый клиент в своём потоке.

Shared state: **список клиентов**. Защищается мьютексом.

```
struct Client {
    int fd;
    std::string name;
};

std::mutex g_mutex;
std::vector<Client> g_clients;
```

42.8.1 broadcast

При получении сообщения от одного клиента — посылаем всем остальным:

```
void broadcast(int from_fd, const std::string& msg) {
    std::lock_guard<std::mutex> lock(g_mutex);
    for (const auto& c : g_clients) {
        if (c.fd == from_fd) continue;
```

```

        ::send(c.fd, msg.data(), msg.size(), MSG_NOSIGNAL);
    }
}

```

Лочим мьютекс на время рассылки. Без лока — другой поток мог бы изменять список во время итерации (UB на vector).

MSG_NOSIGNAL — флаг для send: не посылать SIGPIPE если другой конец закрыт. Без него, если один клиент отключился, наш сервер бы получил SIGPIPE и упал.

Внимание: send блокирует, если буфер receiver'a полный (медленный клиент). В таком случае **весь broadcast** заморожен — лочка не отдаётся, другие клиенты тоже ждут.

Реальные чат-серверы решают это через **per-client queue + writer thread**, или через **non-blocking send + epoll**. У нас простая версия — работает на быстрых клиентах.

42.8.2 handle_client

```

void handle_client(int fd) {
    std::string name = "user_" + std::to_string(g_next_id++);
    register_client(fd, name);

    broadcast(fd, "* " + name + " вошёл в чат\n");

    char buf[1024];
    std::string acc;
    while (true) {
        ssize_t n = ::recv(fd, buf, sizeof(buf), 0);
        if (n == 0) break;
        if (n < 0) { if (errno == EINTR) continue; break; }
        acc.append(buf, n);

        // Разбиваем по '\n'
        std::size_t pos;
        while ((pos = acc.find('\n')) != std::string::npos) {
            std::string line = acc.substr(0, pos);
            acc.erase(0, pos + 1);
            if (line == "/quit") goto disconnect;
            broadcast(fd, "<" + name + "> " + line + "\n");
        }
    }
}

```



```

    }
disconnect:
    unregister_client(fd);
    ::close(fd);
    broadcast(-1, "*" + name + " вышел из чата\n");
}

```

Каждый поток: 1. Регистрирует своего клиента в общем списке. 2. Шлёт всем «вошёл». 3. Циклически читает, накапливает в асс (важно — TCP это поток, сообщение может прийти частями), разбивает по \n, broadcast'ит. 4. На /quit или close — unregister и «вышел».

асс — accumulator для частичных recv. Если клиент написал «hello\nworld\n», но второй recv получил "hel" и "lo\nworld\n" отдельно — мы корректно собираем сообщения.

g_next_id++ — race condition! Два клиента подключились одновременно, оба ++ — могут получить одинаковый id. Защита: атомик или мьютекс. Для учебной цели игнорируем (вероятность мала).

42.8.3 main

```

int main() {
    // ... socket/bind/listen ...
    while (true) {
        int c = ::accept(srv, ...);
        std::thread(handle_client, c).detach();
    }
}

```

Просто. Каждый ассепт → новый поток.

-pthread обязательно при сборке (Linux) — линковка с pthread library. На macOS обычно автоматом.

42.9 Запуск

Терминал 1:

```

$ ./build/threaded_chat_server 9099
Chat-server слушает на :9099
[server] user_1 присоединился
[server] <user_1> привет всем

```

```
[server] user_2 присоединился
[server] <user_2> привет user_1
```

Терминал 2 (Alice):

```
$ ./build/echo_client 127.0.0.1 9099
> <- Привет, user_1. Пишите сообщения. /quit для выхода.
привет всем
> <- * user_2 вошёл в чат
<- <user_2> привет user_1
```

Терминал 3 (Bob):

```
$ ./build/echo_client 127.0.0.1 9099
> <- Привет, user_2. Пишите сообщения. /quit для выхода.
> <- <user_1> привет всем (увидел уже отправленное Alice)
```

(Echo client лёгкое неудобство — он не отделяет отправку от приёма; реальный chat-client сделаем позже.)

42.10 Архитектура «thread-per-connection»

Что мы реализовали — **thread-per-connection** модель. Один поток обслуживает одного клиента всю его сессию.

Плюсы: - Простой код. Сериальная логика «recv → process → broadcast → recv». - Изоляция: один клиент не мешает другому (если не блокирует общий мьютекс).

Минусы: - **Память:** каждый поток имеет свой стек, обычно 8 МВ на Linux. 10 000 клиентов = 80 GB. Серьёзная нагрузка. - **CPU:** переключение контекста между потоками — микросекунды. На тысячах активных потоков — заметно. - **GIЛ-подобные проблемы:** общий мьютекс становится bottleneck.

Альтернатива — **реактор** на epoll/kqueue: **один поток** обслуживает тысячи соединений через non-blocking IO. Глава 43.

В production: - Nginx — реактор + worker-процессы. - Apache (старая модель) — thread/process per connection. Сейчас тоже event-based. - Node.js, Redis, MongoDB — реактор.

42.11 Главные правила главы

1. **std::thread** + **join()** или **detach()** — основа потоков.
2. **std::lock_guard<std::mutex>** в RAII-стиле — никогда **lock/unlock** руками.

3. `std::condition_variable` для wait/notify, не busy-wait.
4. `Atomic` для простых счётчиков, `mutex` для составных данных.
5. **Deadlock = circular wait**. Берите мьютексы в одном порядке или через `scoped_lock`.
6. `detach` для fire-and-forget. Не обращайтесь к стеку родителя.
7. `MSG_NOSIGNAL` на serverside send — против SIGPIPE.
8. **Thread-per-connection** простая модель; реактор для масштаба.

42.12 Маленькое упражнение

1. Соберите и запустите сервер. Подключите 2-3 клиента, обменяйтесь сообщениями.
2. Закомментируйте `lock_guard` в `broadcast`. Запустите. На большой нагрузке (много клиентов отправляют одновременно) — увидите краш через `vector iteration`.
3. Замените `++g_next_id` на `std::atomic<int> g_next_id{1}; g_next_id.fetch_add(1)`. Это безопаснее.
4. Реализуйте команду `/who` — клиент отправляет, получает список всех в чате.
5. Реализуйте команду `/nick <name>` — клиент меняет имя.
6. (Сложнее) Реализуйте **rooms**: команда `/join general` помещает клиента в комнату. Broadcast только в той же комнате.
7. (Сложнее) Используйте **thread pool** вместо `thread-per-connection`: фиксированное число worker-потоков и очередь задач.
8. (Очень сложно) Замените `send` в `broadcast` на **per-client queue + writer thread**. Тогда медленный клиент не блокирует broadcast.

42.13 Что дальше

Глава 42 — `std::atomic` и **memory ordering**. Что значит `relaxed/acquire/release/seq_cst`. Lock-free queue. Compare-and-swap.

Дальше — реактор (43), бинарный протокол (44), многокомнатный чат (45), шифрование+история (46), деплой (47).

Глава 43

Глава 42. `std::atomic` и `memory ordering`

В прошлой главе мы использовали `std::mutex` для защиты `shared state`. Но для **простых** операций (один счётчик, один флаг) `mutex` — `overkill`. Каждый `lock/unlock` — это `syscall` в худшем случае, переключение контекста, кэш-инвалидация.

`std::atomic` даёт **lock-free** примитивы: операции, которые **атомарны** на уровне процессора. Без мьютекса. В десятки раз быстрее на простых случаях.

Но за скорость — цена сложности. `std::atomic` приносит понятие **memory ordering** — `relaxed/acquire/release/seq_cst`. Это одна из самых сложных тем C++. В этой главе разберём её на пальцах.

43.1 Зачем `atomic` — наглядно

Запустим демо:

```
$ ./build/atomic_demo
Без защиты:      counter=1248361 (ожидалось 2000000), 3.93 ms
Mutex:           counter=2000000, 101.99 ms
atomic (seq_cst): counter=2000000, 34.85 ms
atomic (relaxed): counter=2000000, 43.99 ms
```

Два потока инкрементируют один счётчик миллион раз каждый. Ожидаемо 2 000 000.

Без защиты: получили 1 248 361 — **38% инкрементов потерялись** из-за `race condition`. Зато быстро (4 ms).

Mutex: правильный результат, 102 ms.

atomic с seq_cst: правильный результат, 35 ms. **В 3 раза быстрее mutex.**

atomic с relaxed: правильный результат, 44 ms (в этом тесте — почти как seq_cst, но на других нагрузках разница может быть в разы).

Atomic выигрывает по двум причинам: **нет syscall** (mutex иногда зовёт ОС для парковки потока) и **нет переключений контекста**. Это «спин на CPU instruction»: процессор использует особые atomic-инструкции (LOCK XADD на x86, LDXR/STXR на ARM).

43.2 std::atomic

```
#include <atomic>

std::atomic<int> counter{0};

++counter;                // атомарный инкремент
counter.fetch_add(1);      // явный
counter += 5;              // тоже атомарно
counter.store(42);         // присвоить
int v = counter.load();    // прочитать
```

Все эти операции **атомарны**: либо вся применилась, либо нет. Никакого «частично применённого» состояния, как с обычным ++.

Для каких типов работает: - **Встроенные числа** (int, long, uint64_t, ...): всегда атомарны. - **Указатели** (std::atomic<T*>): атомарны. - **Bool** (std::atomic<bool>): атомарен. - **Свои struct**: если их размер ≤ 8 байт и trivially copyable — атомарны через std::atomic. Иначе — внутри будет мьютекс.

Проверить, lock-free ли тип на вашей платформе:

```
if (std::atomic<int>::is_lock_free) {
    // на этой платформе int атомарно lock-free
}
```

43.3 compare_exchange (CAS)

Самая мощная атомарная операция — **compare-and-swap (CAS)**:

```
bool compare_exchange_strong(T& expected, T desired);
```

Логика: 1. Прочитать текущее значение атомика. 2. Если оно равно expected — заменить на desired, вернуть true. 3. Иначе — записать **текущее** значение в expected, вернуть false.

Всё это — атомарно.

Использование — типичный паттерн для lock-free алгоритмов:

```
std::atomic<int> value{0};

int expected = value.load();
int desired;
do {
    desired = compute(expected);
} while (!value.compare_exchange_weak(expected, desired));
```

«Прочитал, посчитал новое, попробовал поменять — если кто-то опередил, начну заново».

weak vs strong: compare_exchange_weak может **ложно** возвращать false (на некоторых архитектурах для производительности). В цикле — нормально. Вне цикла — используйте strong.

43.4 Memory ordering — главная страшилка

Здесь и начинается сложное.

В мире однопоточной программы всё последовательно: «инструкция А, потом В, потом С». В мире многопоточной — **не так**. Процессор может **переставить** инструкции, компилятор тоже может, кэши разных ядер видят значения по-разному.

Пример:

<i>// Thread 1</i>	<i>Thread 2</i>
data = 42;	while (!ready) {}
ready = true;	std::cout << data;

Что напечатает Thread 2? Кажется, 42. На самом деле — **может напечатать 0**. Почему?

- Компилятор может переставить присваивания в Thread 1 (если не видит зависимости):
ready = true; data = 42;.
- CPU может выполнить store'ы в другом порядке.
- Кэши могут синхронизироваться с задержкой.

Чтобы это работало, нужен **memory barrier** — гарантия: «всё, что было до этого store, видно после load на другом ядре».

В C++11 эти барьеры задаются через **memory_order**:

```
counter.fetch_add(1, std::memory_order_relaxed);
counter.store(42, std::memory_order_release);
int v = counter.load(std::memory_order_acquire);
```

43.5 Пять режимов memory_order

43.5.1 memory_order_relaxed

«Атомарность есть, но **никаких** гарантий порядка с другими операциями».

```
std::atomic<int> c{0};
c.fetch_add(1, std::memory_order_relaxed);
```

Самый быстрый. Подходит когда: - Просто счётчик, и его значение нужно только в конце. - Не используем как **сигнал** между потоками.

Пример: счётчик processed_requests. Каждый поток инкрементирует. В конце программы читаем суммарное. Никаких других операций «после inc» не нужно — relaxed ok.

43.5.2 memory_order_release / memory_order_acquire

Парная гарантия. **release** на write, **acquire** на read.

```
// Producer
data = 42;
ready.store(true, std::memory_order_release); // всё что было ДО store видно

// Consumer
while (!ready.load(std::memory_order_acquire)) {} // увидит данные ДО store
std::cout << data; // напечатает 42 (или старше; не «новее» чем release)
```

release говорит: «все мои предыдущие операции должны быть **завершены** до этого store».

acquire говорит: «все мои последующие операции должны видеть всё, что было перед matching release».

Это даёт **synchronizes-with** отношение. Самая частая пара в lock-free коде.

43.5.3 memory_order_acq_rel

Когда **одна** операция и читает, и пишет (RMW = read-modify-write): fetch_add, compare_exchange. Гарантии acquire (на read часть) + release (на write часть).

43.5.4 memory_order_seq_cst — sequentially consistent

Самый строгий. Все операции в программе **выстраиваются в единый порядок**, видимый всеми потоками. Гарантии acq_rel + общий глобальный порядок.

Это default для atomic-операций без указания (++counter = seq_cst).

Самый медленный. Но **проще рассуждать**: всё последовательно, как однопоточная программа.

В большинстве кода **seq_cst достаточно**. Используйте relaxed/acquire/release только когда **нужна производительность** и вы понимаете, что делаете.

43.6 Сравнение

Ordering	Атомарность	Sync-with	Скорость	Когда
relaxed	✓	✗	быстрее всего	Счётчики, статистика
release (только store/RMW)	✓	✓ (с acquire)	средне	Producer: «вот данные готовы»
acquire (только load/RMW)	✓	✓ (с release)	средне	Consumer: «жду данные»
acq_rel (только RMW)	✓	✓ обе стороны	средне	CAS-loops
seq_cst	✓	✓ глобально	медленнее всего	Когда не уверены — берите это

43.7 Пример с release/acquire

«Producer публикует, consumer ждёт»:

```
struct Message { int data; };
Message msg;
std::atomic<bool> ready{false};

// Producer
```



```

void producer() {
    msg.data = 42; // обычная запись
    ready.store(true, std::memory_order_release); // публикуем
}

// Consumer
void consumer() {
    while (!ready.load(std::memory_order_acquire)) {
        std::this_thread::yield();
    }
    std::cout << msg.data; // гарантированно 42
}

```

release создаёт барьер: «всё, что было до store, видно после acquire». В частности, msg.data = 42 гарантированно видно consumer’у.

Без release/acquire (например, с relaxed на обе стороны) — нет гарантии. Consumer мог бы увидеть ready == true но data == 0.

Это **паттерн публикации**: один поток подготавливает данные, другой ждёт сигнала.

43.8 Lock-free очередь SPSC

Single-Producer/Single-Consumer очередь — один поток пишет, другой читает. Lock-free реализация — кольцевой буфер с двумя индексами:

```

template <typename T, std::size_t N>
class SpscQueue {
public:
    bool push(const T& v) {
        std::size_t h = head_.load(std::memory_order_relaxed);
        std::size_t next = (h + 1) % N;
        if (next == tail_.load(std::memory_order_acquire)) {
            return false; // полна
        }
        buf_[h] = v;
        head_.store(next, std::memory_order_release);
        return true;
    }
}

```

```

bool pop(T& out) {
    std::size_t t = tail_.load(std::memory_order_relaxed);
    if (t == head_.load(std::memory_order_acquire)) {
        return false; // пуста
    }
    out = buf_[t];
    tail_.store((t + 1) % N, std::memory_order_release);
    return true;
}

private:
    T buf_[N];
    std::atomic<std::size_t> head_{0}; // producer пишет сюда
    std::atomic<std::size_t> tail_{0}; // consumer читает отсюда
};

```

Логика: - head — индекс, куда **писать** дальше. Меняет producer. - tail — индекс, **откуда** читать. Меняет consumer. - Очередь пустая когда head == tail. Полная когда (head + 1) % N == tail.

Memory ordering: - Producer пишет в buf_[h], потом publish'ит head через **release**. - Consumer читает head через **acquire**, чтобы гарантированно увидеть buf_[h]. - Симметрично для tail.

relaxed на собственных чтениях (own loads) — потому что один поток пишет одну переменную, никакой синхронизации не нужно для **своих** чтений.

Это **классическая** lock-free структура. SPSC — самый простой случай. Multi-producer/multi-consumer (MPMC) — гораздо сложнее, требует Michael-Scott queue или подобных алгоритмов.

43.9 АВА-проблема

Главная ловушка lock-free программирования. Имеем compare_exchange:

```

int expected = atom.load();
do {
    int desired = ...;
} while (!atom.compare_exchange_weak(expected, desired));

```

CAS гарантирует «изменилось ли значение с момента моего read». **Но не** «прошло ли через другие значения».

АВА: я прочитал А. Другой поток сделал А → В → А. Я делаю CAS с expected=A — успех. **Но что-то изменилось**, и моя логика может быть неверна.

Пример: lock-free stack. Я хочу порнуть. Прочитал `top = node_X`. Другой поток порнул X, освободил, выделил новый `node_X` (тот же адрес), запустил. Мой `CAS top == old_X` пройдёт, но **внутри — другой объект**.

Решение — **tag** к указателю (старшие биты как «версия»). Или **hazard pointers** / **epoch-based reclamation**.

В нашей мини-СУБД, mini-shell, chat-server — АВА не встретится. Это для глубокой lock-free работы.

43.10 std::memory_order_consume

Шестой режим, но **не используется** в практике. Стандарт оставил, но компиляторы трактуют как `acquire`. Тема для академических исследований.

43.11 Когда использовать atomic, когда mutex

Грубые правила:

atomic для: - Простых счётчиков, флагов. - Lock-free структур данных (если вам реально нужно). - Сигнализации между потоками («готов»/«не готов»).

mutex для: - Составных данных (vector, map, struct с несколькими полями). - Когда логика «прочитать X, проверить, записать Y» должна быть атомарной. - Условных переменных (std::condition_variable требует mutex).

В chat-server мы использовали mutex — список клиентов это составная структура. Правильно.

Если бы делали счётчик `total_messages_sent` — atomic было бы быстрее.

43.12 Lock-free vs wait-free

Lock-free — система всегда **прогрессирует** (хотя бы один поток успешно завершит операцию).

Wait-free — **каждый** поток гарантированно завершит операцию за конечное число шагов. Сильнее lock-free.

Blocking (mutex) — поток может ждать долго (если кто-то держит lock).

Wait-free алгоритмы редки и сложны. Lock-free — реалистичная цель для производительности.

43.13 Главные правила главы

1. **atomic** для простого (счётчики), **mutex** для сложного (составные данные).
2. **Default seq_cst** — самый строгий, проще рассуждать. Оптимизация через **relaxed/acq_rel** — только когда измеряли и нужно.
3. **release-acquire пара** — главная идиома lock-free публикации.
4. **compare_exchange_weak** в цикле, **strong** вне цикла.
5. **is_always_lock_free** проверка перед использованием на ваших типах.
6. **ABA-проблема** при работе с указателями в lock-free.
7. **Lock-free не всегда быстрее mutex** — на низкой нагрузке mutex быстрее, на высокой — atomic.
8. **Тестируйте под нагрузкой** — race condition проявляется только при contention.

43.14 Маленькое упражнение

1. Запустите `./build/atomic_demo`. Посмотрите, насколько race теряет инкрементов на вашей машине.
2. Замените в `threaded_chat_server.cpp` `int g_next_id` на `std::atomic<int>`. Используйте `g_next_id.fetch_add(1)`.
3. Напишите минимальный SPSC queue по примеру из главы. Запустите тест: producer-поток отправляет числа 1..N, consumer проверяет, что прочитал все по порядку.
4. (Сложнее) Реализуйте producer-consumer **тест на скорость**: SPSC через atomic vs через mutex. Замерьте throughput.
5. (Сложнее) Сделайте `std::atomic<bool> shutdown_flag` в server. Поток-обработчик клиента проверяет на каждой итерации и выходит. Это «cooperative cancellation».
6. (Очень сложно) Реализуйте MPMC очередь. Подсказка: Michael-Scott queue. Это десятки строк сложного кода.
7. (Очень сложно) Прочитайте «C++ Concurrency in Action» (Anthony Williams) — глава про memory ordering. Самый ясный текст по теме.
8. Используйте `std::atomic_thread_fence` — explicit fence без атомика. Когда полезно?

43.15 Что дальше

Глава 43 — **реактор**: select/poll/epoll/kqueue. Один поток, тысячи соединений. Без потока на клиента — gigascale.

Дальше — бинарный протокол (44), многокомнатный чат (45), шифрование+история (46), деплой (47). Часть V на финишной прямой.

Глава 44

Глава 43. Реактор — select/poll/epoll/kqueue

В главе 41 мы сделали чат-сервер на **thread-per-connection**. Работает, но не масштабируется: на 10 000 клиентов нужно 10 000 потоков, 80 GB памяти под стеки, deadly context switching.

В этой главе — **реактор**. Один поток обслуживает **тысячи** соединений. Принцип: вместо «блокироваться на recv одного клиента» — «опрашивать ОС: на каком из моих fd готовы данные?». Это **event-driven IO**.

Сделаем чат-сервер на poll() — кросс-платформенный. На production использовали бы epoll (Linux) или kqueue (BSD/macOS) — масштабируемые до миллионов соединений. Идея та же.

44.1 C10K problem

В конце 90-х серверы упёрлись в **C10K** — «10 000 одновременных соединений». На thread-per-connection упирались: - Память: стек 8 MB × 10k = 80 GB. - CPU: переключение контекстов раз в миллисекунду × 10k = простой.

Решение нашли через **non-blocking IO + event polling**. Один поток, десятки тысяч соединений.

Сейчас обсуждают **C10M** — 10 миллионов. Решения экзотичные: bypass kernel TCP stack (DPDK), kernel bypass entirely (XDP, eBPF), zero-copy IO.

Для обычных приложений epoll/kqueue хватает на сотни тысяч.

44.2 Blocking vs non-blocking

Blocking IO (как в главах 40, 41):

```
ssize_t n = recv(fd, buf, sizeof(buf), 0); // блокирует до данных
```

Если данных нет — поток спит. ОС просыпает его, когда придут.

Non-blocking IO:

```
fcntl(fd, F_SETFL, O_NONBLOCK); // переключаем сокет в non-blocking режим
ssize_t n = recv(fd, buf, sizeof(buf), 0); // возвращается сразу
if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
    // данных нет — пробуйте позже
}
```

recv возвращается **немедленно**. Если данные есть — возвращает. Если нет — -1 с errno == EAGAIN (или EWOULDBLOCK, синоним).

С non-blocking один поток может **попробовать** все свои сокеты по очереди. Но это **busy-wait** — CPU кипит. Нужно **спать**, пока кто-то не готов.

Для этого — select/poll/epoll/kqueue.

44.3 select() — оригинал

POSIX даёт select:

```
fd_set readfds;
FD_ZERO(&readfds);
FD_SET(srv, &readfds);
FD_SET(c1, &readfds);
FD_SET(c2, &readfds);

select(max_fd + 1, &readfds, nullptr, nullptr, nullptr);
// Заблокирует, пока хоть один fd не готов

if (FD_ISSET(srv, &readfds)) { /* accept */ }
if (FD_ISSET(c1, &readfds)) { /* recv c1 */ }
```

fd_set — битовая маска. Ограничение: max fd обычно **1024**. Это слишком мало для серверов.

Плюс — после select **нужно перебрать все fd**, проверить кто готов. O(N) на каждой итерации.

select появился в 1983 году. Сейчас почти не используется в production.

44.4 poll() — улучшенный select

```
std::vector<pollfd> pfd;
pfd.push_back({srv, POLLIN, 0});
pfd.push_back({c1, POLLIN, 0});

poll(pfd.data(), pfd.size(), -1 /*ms*/);

if (pfd[0].revents & POLLIN) { /* accept */ }
if (pfd[1].revents & POLLIN) { /* recv */ }
```

pollfd: - fd — файловый дескриптор. - events — что нас интересует (POLLIN/POLLOUT/POLLERR/...).
- revents — что **реально** произошло (заполняется poll).

Преимущества над select: - **Нет лимита 1024**. - Передаём вектор, не битовую маску — компактнее на маленьких N.

Но **те же** проблемы: - O(N) на каждой итерации — ядро перебирает все fd. - Структура с events/revents копируется в ядро каждый вызов.

poll подходит до ~1000 соединений. Дальше — медленно.

44.5 epoll (Linux) и kqueue (BSD/macOS)

Современные scalable mechanisms:

epoll (Linux):

```
int ep = epoll_create1(0);

epoll_event ev;
ev.events = EPOLLIN;
ev.data.fd = srv;
epoll_ctl(ep, EPOLL_CTL_ADD, srv, &ev);

epoll_event events[64];
int n = epoll_wait(ep, events, 64, -1);
for (int i = 0; i < n; ++i) {
    // events[i].data.fd ротон
}
```


Идея: **регистраруем** интересующие fd один раз (epoll_ctl), потом многократно вызываем epoll_wait. Ядро **запоминает** список, не перебирает каждый раз. O(1) добавление/удаление, O(K) для K готовых fd.

kqueue (BSD/macOS) — аналог:

```
int kq = kqueue();

struct kevent ev;
EV_SET(&ev, srv, EVFILT_READ, EV_ADD, 0, 0, nullptr);
kevent(kq, &ev, 1, nullptr, 0, nullptr);

struct kevent events[64];
int n = kevent(kq, nullptr, 0, events, 64, nullptr);
```

Семантически близко к epoll. Linux и BSD разошлись в API в 2002, и это разные подходы к одной задаче.

Windows IOCP (I/O Completion Ports) — ещё одна модель. Не «опрашивать готовность», а «получить завершённую операцию». Архитектурно ближе к async-IO.

Для **кросс-платформенности** обычно делают тонкую обёртку: EventLoop::add(fd, callback), который под капотом использует epoll/kqueue/IOCP.

44.6 Edge-triggered vs level-triggered

В **epoll** и **kqueue** два режима:

Level-triggered (LT) — поведение poll: «если есть данные — снова и снова сигнал». Если ваш код не успел всё прочитать, следующий poll снова покажет fd готовым.

Edge-triggered (ET) — сигнализирует **только** при изменении состояния: «было пусто, стало непусто». Если не прочитали всё за раз — epoll_wait не вернёт этот fd, пока не придёт ещё что-то.

ET **немного быстрее**, но требует **non-blocking IO** и **читать до EAGAIN**. Иначе пропустите данные.

В нашей версии — poll, который только LT. Простая модель.

44.7 Реактор-цикл

main loop:

```
готовые_fd = poll(все_fd, ...)
```

для каждого готового fd:

если listening fd:

 accept → новый client_fd

 добавить в список

иначе если client fd:

 если можно читать: recv → накопить, парсить, broadcast

 если можно писать: send из per-client send_buffer

 если ошибка: close, удалить

Это **single-threaded event loop**. Никаких мьютексов на shared state — всё в одном потоке.

44.7.1 Per-client send buffer

Когда мы делаем broadcast, нам нужно **записать** в N клиентов. Каждый send может вернуть EAGAIN (буфер ядра полный). Что делать?

В **thread-per-connection**: блокируем send, поток ждёт.

В **реакторе**: блокировать **нельзя** — это парализует весь сервер. Решение: - Кладём данные в send_buf для каждого клиента. - На следующей итерации, если POLLOUT готов, пишем оттуда.

```
struct ClientState {
    int fd;
    std::string recv_buf;
    std::string send_buf;
};

void broadcast(...) {
    for (auto& kv : clients) {
        kv.second.send_buf += msg;
    }
}

// В реактор-цикле:
for (auto& pfd : pfd) {
    short events = POLLIN;
    if (!clients[pfd.fd].send_buf.empty()) {
        events |= POLLOUT;
    }
}
```

```

    pfd.events = events;
}

// При срабатывании POLLOUT:
ssize_t w = send(fd, s.send_buf.data(), s.send_buf.size(), 0);
if (w > 0) s.send_buf.erase(0, w);

```

Главное: **POLLOUT** запрашиваем только когда есть что писать. Иначе poll будет постоянно возвращать «готов писать» (level-triggered), занимая CPU.

44.8 set_nonblocking

```

void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

```

Каждый сокет (включая listening!) переключаем в non-blocking. Тогда accept, recv, send возвращаются сразу, не блокируют.

Для accept это позволяет вызывать его в цикле, пока есть готовые соединения. Без non-blocking — после первого accept'а второй блокировал бы.

44.9 Полная реализация — reactor_chat_server.cpp

(Пропускаем boilerplate socket/bind/listen — он как в главе 40.)

```

std::unordered_map<int, ClientState> clients;
int next_id = 1;

while (true) {
    std::vector<pollfd> pfd;
    pfd.push_back({srv, POLLIN, 0});
    for (const auto& kv : clients) {
        short events = POLLIN;
        if (!kv.second.send_buf.empty()) events |= POLLOUT;
        pfd.push_back({kv.first, events, 0});
    }
}

```

```

int n = ::poll(pfds.data(), pfds.size(), -1);
if (n < 0) { if (errno == EINTR) continue; break; }

// 1) listening socket
if (pfds[0].revents & POLLIN) {
    while (true) {
        int c = ::accept(srv, nullptr, nullptr);
        if (c < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;
            if (errno == EINTR) continue;
            break;
        }
        set_nonblocking(c);
        ClientState s; s.fd = c;
        s.name = "user_" + std::to_string(next_id++);
        clients.emplace(c, std::move(s));
        broadcast(clients, c, "* " + clients[c].name + " вошёл\n");
        clients[c].send_buf += "Привет, " + clients[c].name + "\n";
    }
}

// 2) клиенты
for (size_t i = 1; i < pfds.size(); ++i) {
    int fd = pfds[i].fd;
    short re = pfds[i].revents;
    auto it = clients.find(fd);
    if (it == clients.end()) continue;
    ClientState& s = it->second;

    if (re & POLLIN) {
        char buf[1024];
        bool gone = false;
        while (true) {
            ssize_t r = ::recv(fd, buf, sizeof(buf), 0);
            if (r == 0) { gone = true; break; }
            if (r < 0) {
                if (errno == EAGAIN || errno == EWOULDBLOCK) break;
                if (errno == EINTR) continue;
            }

```

```
        gone = true; break;
    }
    s.recv_buf.append(buf, r);
}
// парсим сообщения
size_t pos;
while ((pos = s.recv_buf.find('\n')) != std::string::npos) {
    std::string line = s.recv_buf.substr(0, pos);
    s.recv_buf.erase(0, pos + 1);
    if (line == "/quit") { gone = true; break; }
    broadcast(clients, fd, "<" + s.name + "> " + line + "\n");
}
if (gone) {
    std::string bye = "* " + s.name + " вышел\n";
    ::close(fd);
    clients.erase(it);
    broadcast(clients, -1, bye);
    continue;
}
}

if ((re & POLLOUT) && !s.send_buf.empty()) {
    while (!s.send_buf.empty()) {
        ssize_t w = ::send(fd, s.send_buf.data(), s.send_buf.size(), MSG_NOSIGNAL);
        if (w < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;
            if (errno == EINTR) continue;
            ::close(fd); clients.erase(it); break;
        }
        s.send_buf.erase(0, w);
    }
}

if (re & (POLLHUP | POLLERR | POLLNVAL)) {
    ::close(fd);
    clients.erase(it);
}
```

```
    }
}
```

Ключевые моменты:

1. **pfds пересобирается каждую итерацию** — у некоторых клиентов появился `send_buf`, у других исчез.
2. **ассепт в цикле, пока EAGAIN** — на одну итерацию `poll` могло прийти несколько SYN.
3. **recv в цикле, пока EAGAIN** — то же, накапливаем всё, что пришло.
4. **send в цикле**, пока не отправили всё или EAGAIN.

44.10 Тестирование

```
$ ./build/reactor_chat_server 9096 &
$ ./build/echo_client 127.0.0.1 9096 # Терминал 1
> hello from A
<- * user_2 вошёл
<- <user_2> from B

$ ./build/echo_client 127.0.0.1 9096 # Терминал 2
> from B
<- <user_1> from A
```

Работает. Один поток обслуживает обоих. Если запустить 100 клиентов — тоже один поток.

44.11 Сравнение моделей

Модель	Поток на клиента	Память	CPU	Сложность кода
Thread-per-conn	1 поток / N клиентов	8 MB × N	context switch	простой
Reactor (poll)	1 поток / 1000 клиентов	минимум	O(N) на opt	сложнее
Reactor (epoll/kqueue)	1 поток / миллион	минимум	O(K) на opt	сложнее
Hybrid (reactor + worker pool)	1 reactor + N workers	средне	масштабируемое	сложнее всего

В production: - **Nginx**: один master + worker процессы, каждый — реактор. - **Redis**: один реактор, всё в одном потоке. - **Node.js**: один реактор (libuv).

Реактор — стандарт для C10K+ серверов.

44.12 Cross-platform wrapper

Идея: интерфейс высокого уровня, реализация — epoll/kqueue/poll под капотом.

```
class EventLoop {
public:
    void add(int fd, EventType type, Callback cb);
    void remove(int fd);
    void run();    // loop
};
```

Под Linux — epoll_ctl/epoll_wait. Под macOS — kevent. Под Windows — IOCP с другой моделью (асинхронность вместо опроса). Известные библиотеки:

- **libevent** — старая C-библиотека.
- **libuv** — Node.js core. Кросс-платформенная.
- **Boost.Asio** — C++ Boost library.
- **ASIO** (standalone).
- **liburing** (Linux io_uring — самый современный async API).

Используют **production** проекты, не пишут сами. Наш реактор — учебный.

44.13 Async/await и coroutines

Реактор-код некомфортен: вместо линейного «прочитал → обработал» — fragmented callbacks. Это **callback hell**.

С C++20 пришли **coroutines**: пишем «async-await»-стиль, под капотом — реактор. Намного читабельнее.

```
// Псевдо-C++20:
task<void> handle_client(int fd) {
    while (true) {
        std::string msg = co_await read_line(fd);
        if (msg == "/quit") break;
        co_await broadcast(msg);
    }
}
```

```
}  
}
```

C++ coroutines сложны в реализации (низкоуровневые), но дают читабельный код. В нашей C++11-базе их нет. В Части VI поговорим обзорно.

44.14 Главные правила главы

1. **Thread-per-connection не масштабируется.** Реактор для серверов.
2. **Non-blocking IO + полинг.** `O_NONBLOCK` через `fcntl`.
3. **poll** для маленьких/учебных, **epoll/kqueue** для production.
4. **Edge-triggered + non-blocking + читать до EAGAIN.** Не пропустите.
5. **Per-client send buffer.** Не блокируйте `send` в реакторе.
6. **POLLOUT** запрашиваем только когда есть что писать. Иначе CPU кипит.
7. **EAGAIN/EWOULDBLOCK — нормальная ситуация**, не ошибка.
8. **libuv/libevent/asio** для cross-platform. Не пишите сами в production.

44.15 Маленькое упражнение

1. Запустите. Подключите несколько клиентов. Убедитесь, что один поток обслуживает всех.
2. Замерьте память сервера через `ps -o rss`. С 100 подключёнными клиентами — должен быть в десятки MB, не GB.
3. (Сложнее) Замените `poll` на `epoll` (если на Linux). Сравните скорость на тысяче клиентов.
4. (Сложнее) На macOS реализуйте через `kqueue`. Изучите `EV_SET`, `EVFILT_READ`, `EVFILT_WRITE`.
5. (Сложнее) Сделайте **hybrid** модель: главный реактор принимает соединения и передаёт их в **thread pool** worker'ов. Каждый worker — свой реактор. Это nginx-стиль.
6. (Очень сложно) Подключите `libuv`. Перепишите сервер на её API. Сравните читаемость.
7. Прочитайте оригинальный пост о C10K (Dan Kegel): <http://www.kegel.com/c10k.html>. Историческая ценность.
8. Прочитайте `man 2 epoll`, `man 2 kqueue`. Сравните API.

44.16 Что дальше

Глава 44 — **свой бинарный протокол**. До сих пор клиенты слали строки, разделённые \n. Это нестабильно (что если в сообщении есть \n?). Сделаем фрейм-формат [length:4] [type:1] [payload:N]. Версионирование протокола, частичные чтения через кольцевой буфер.

Дальше — многокомнатный чат (45), шифрование+история (46), сборка и deployment (47). Финал Части V близок.

Глава 45

Глава 44. Свой бинарный протокол

До этой главы наш чат пользовался **текстовым** протоколом: «строка до `\n` — одно сообщение». Просто, но хрупко:

- Что если в сообщении есть `\n` (multi-line текст)?
- Как различать «обычное сообщение» и «join room»?
- Как версионировать? Старый клиент шлёт старый формат серверу с новым.

Решение — **бинарный фрейм-протокол**. Каждое сообщение — фрейм с **явной длиной** и **типом**. Никакой неоднозначности. Это то, что делают HTTP/2, WebSocket, gRPC, MongoDB wire protocol, и большинство production-протоколов.

В этой главе спроектируем и реализуем простой фрейм-формат. Плюс **аккумулятор** для частичных reads (TCP — поток, не пакеты!).

45.1 Зачем фрейминг

TCP даёт **поток байтов**, не сообщения. Если клиент сделал `send("hello")`, `send("world")` — сервер может получить: - За один `recv`: "helloworld". - За два `recv`: "hello" + "world". - За пять `recv`: "he" + "l" + "low" + "or" + "ld".

Как сервер узнает, где кончается одно сообщение и начинается другое? **Нужен делIMITер**.

Варианты: 1. **ДелIMITер-символ** (`\n`, `\0`). Просто, но что если он встретится в данных? Бинарные данные ломают. 2. **Длина-префикс** (length-prefixed): сначала длина, потом столько байтов. Самый чистый. 3. **Фиксированный размер** сообщений — подходит редко.

В чате нам нужны произвольные строки + разные типы сообщений. **Length-prefixed + type-byte** — стандарт.

45.2 Формат фрейма

```
[0..3]  u32 LE length  (= version + type + payload, т.е. всё после length)
[4]     u8  version
[5]     u8  type
[6..]   payload (length - 2 байт)
```

Минимальный фрейм — $\text{length}(4) + \text{version}(1) + \text{type}(1) + \text{payload}(0) = 6$ байт.

length хранится в **little-endian** (как договорились в главе 32). Все стороны должны знать порядок — мы фиксируем LE.

version — текущая версия протокола. У нас 1. Если в будущем формат поменяется — версия инкрементится. Клиент с v1 → сервер с v2 поймёт по версии: нужно отказывать или fallback.

type — что за сообщение. У нас: - 1 = Hello (клиент представляется) - 2 = Message (текст в чате) - 3 = Notify (серверное уведомление) - 4 = Join (войти в комнату) - 5 = Leave (выйти)

payload — переменной длины, до 64 KiB (MAX_PAYLOAD). Лимит защищает от plain malicious клиентов, шлющих гигабайтные «длины».

45.3 Encoder

src/protocol.cpp:

```
std::vector<std::uint8_t> encode(std::uint8_t type, const std::string& payload) {
    if (payload.size() > MAX_PAYLOAD) {
        throw std::runtime_error("payload too large");
    }
    std::vector<std::uint8_t> out;
    std::uint32_t len = static_cast<std::uint32_t>(2 + payload.size());
    out.reserve(4 + len);
    write_u32_le(out, len);
    out.push_back(PROTO_VERSION);
    out.push_back(type);
    out.insert(out.end(), payload.begin(), payload.end());
    return out;
}
```

Простой. Считаем length, пишем 4 байта LE, потом version+type+payload.

write_u32_le — как в главе 32:

```
void write_u32_le(std::vector<std::uint8_t>& out, std::uint32_t v) {
    for (int i = 0; i < 4; ++i) {
        out.push_back(static_cast<std::uint8_t>((v >> (i * 8)) & 0xFF));
    }
}
```

45.4 Decoder и аккумулятор

recv возвращает произвольное количество байт. Может быть **половина** одного фрейма. Может быть **полтора** фрейма (целый + начало второго).

Решение — **аккумулятор**: `std::vector<uint8_t>`, в который складываем всё пришедшее. На каждом recv:

1. Дописать данные в аккумулятор.
2. **В цикле** пробовать извлечь фрейм. Если получилось — обработать, продолжить цикл. Если данных не хватает — выйти и ждать ещё.

```
bool try_decode(std::vector<std::uint8_t>& acc,
               std::uint8_t& out_type,
               std::string& out_payload) {
    if (acc.size() < 4) return false;           // нет даже длины
    std::uint32_t len = read_u32_le(acc.data());
    if (len < 2 || len > MAX_PAYLOAD + 2) {
        throw std::runtime_error("invalid length");
    }
    if (acc.size() < 4 + len) return false;      // длина есть, тело ещё не пришло

    std::uint8_t version = acc[4];
    if (version != PROTO_VERSION) {
        throw std::runtime_error("unsupported version");
    }
    out_type = acc[5];
    out_payload.assign(reinterpret_cast<const char*>(acc.data() + 6), len - 2);

    acc.erase(acc.begin(), acc.begin() + 4 + len);
    return true;
}
```

Три исхода: 1. **Данных хватает на фрейм** — извлечь, потребить байты, вернуть true. 2. **Длину видим, но тела нет** — false. Ждём ещё. 3. **Длины нет (< 4 байт)** — false.

При **повреждении** (нереальная длина, плохая версия) — throw. Это означает «соединение испорчено», caller должен закрыть клиента.

45.4.1 Шаблон использования

```
char buf[1024];
ssize_t r = recv(fd, buf, sizeof(buf), 0);
acc.insert(acc.end(), buf, buf + r);

uint8_t type;
std::string payload;
try {
    while (try_decode(acc, type, payload)) {
        handle(type, payload);
    }
} catch (const std::exception& e) {
    log("протокол сломан: ", e.what());
    close(fd);
}
```

Идиома «**читай в буфер, потом извлекай все доступные фреймы**». Работает в любой архитектуре — thread-per-conn, реактор, async.

45.5 Альтернатива — кольцевой буфер

std::vector::erase с начала — O(N) (сдвигает все остальные элементы). На больших объёмах данных — медленно.

Решение — **кольцевой буфер** (ring buffer): фиксированный массив + два индекса (read_pos, write_pos). Read/write — O(1).

```
class RingBuffer {
    std::vector<uint8_t> data_;
    size_t read_pos_ = 0;
    size_t write_pos_ = 0;
    size_t size_ = 0;
};
```

Для **большого трафика** (мегабайты в секунду) — обязательно. Для нашего чата — vector достаточно.

Boost.Asio использует свой streambuf, Node.js — внутренние Buffer'ы, ниже уровень — ring buffers.

45.6 Версионирование

Что делать при **изменении** протокола?

Минорные: добавили новый тип сообщения (типа 6). Старый клиент шлёт сообщения типов 1-5 — сервер v2 их понимает. Новый клиент шлёт 6 — старый сервер падает или игнорирует.

Мажорные: поменялось значение существующего поля. Например, length стало 8 байт. Старый клиент с v1 → сервер v2 не разберёт.

Стратегии:

1. **Bump major version, отказ старым.** Грубо, но просто.
2. **Negotiation:** при handshake клиент шлёт «поддерживаю v1, v2, v3», сервер отвечает «давай v2».
3. **Forward compatibility:** новые поля **в конце**. Старый клиент игнорирует «лишние» байты.

В нашем протоколе — bump version на каждое изменение. Учебное упрощение.

45.7 Демо

utils/protocol_demo.cpp показывает round-trip с фрагментацией:

```
$ ./build/protocol_demo
=== Закодированные фреймы ===
Hello 'Alice' (11 байт):
07 00 00 00 01 01 41 6c 69 63 65
  ^^^^^^^^^^                                length=7 (LE)
              ^^                             version=1
              ^^                             type=1 (Hello)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^ "Alice" (ASCII)

Message 'Привет всем!' (28 байт):
18 00 00 00 01 02 d0 9f d1 80 d0 b8 d0 b2 d0 b5
d1 82 20 d0 b2 d1 81 d0 b5 d0 bc 21
      ^^ "П" в UTF-8 = 0xD0 0x9F (2 байта)
```

```

=== Декодирование с фрагментацией ===
[чанк 3 байт, в буфере 3]           ← length ещё не полностью пришла
[чанк 10 байт, в буфере 13]
  → Hello: 'Alice'                  ← извлечён фрейм 1
[чанк 1 байт, в буфере 3]
[чанк 7 байт, в буфере 10]
[чанк 4 байт, в буфере 14]
[чанк 27 байт, в буфере 41]
  → Message: 'Привет всем!'        ← фрейм 2 + начало 3 в одном чанке
  → Join: 'general'                ← фрейм 3 извлечён сразу
Остаток в буфере: 0 байт

```

Decoder корректно работает при произвольной фрагментации. Это критично для TCP.

45.8 Тонкости в production

45.8.1 Frame compression

Большие сообщения можно **сжимать** перед отправкой. Добавить bit в header: «payload сжат через gzip/zstd». Decoder декомпрессирует.

WebSocket делает это через `permessage-deflate` extension.

45.8.2 TLS / шифрование на уровне фрейма

Шифровать **payload** каждого фрейма отдельным ключом. Или шифровать **весь** TCP-поток через TLS (правильнее). О шифровании — глава 46.

45.8.3 Heartbeat / keepalive

Длинно-висящие соединения теряются (NAT timeout, network glitch). Решение — периодический «ping» фрейм с ответом «pong». Если не пришёл за N секунд — закрываем.

WebSocket: control frames `0x09` (ping) / `0x0A` (pong).

45.8.4 Big-endian для сетевых протоколов

Исторически сетевые протоколы используют **big-endian** (`htons/htonl`). Мы выбрали LE, потому что: - Современные процессоры (x86, ARM) — LE. - Наш клиент и сервер — на одной архитектуре. - Нет лишних `htons/ntohs` преобразований.

Если бы делали **переносимый** протокол (Java-клиент, например), стоило бы выбрать BE — стандарт всё-таки.

45.8.5 Schema evolution: protobuf, FlatBuffers, MessagePack

Для **сложного** контента (нестабильная схема, много полей) есть готовые форматы:

- **Protocol Buffers** (Google) — schema-based, optional fields, forward/backward compat.
- **FlatBuffers** — zero-copy, mmap-able.
- **MessagePack** — JSON-like, но binary.
- **Cap'n Proto** — то же.

Production-протоколы обычно **не** делают свой бинарный формат, а используют готовые. Они решают версионирование, имеют клиенты на 30 языках, проверены временем.

Наш «свой» формат — учебный. На реальном проекте — protobuf.

45.9 Демо использования в чате

Старый чат шёл строками. Новый — фреймами:

```
// Клиент: представляется
std::vector<uint8_t> hello = chat::encode(chat::kHello, "Alice");
send(fd, hello.data(), hello.size(), 0);

// Клиент: шлёт сообщение
auto msg = chat::encode(chat::kMessage, "Привет!");
send(fd, msg.data(), msg.size(), 0);

// Сервер: получает
char buf[1024];
ssize_t r = recv(fd, buf, sizeof(buf), 0);
acc.insert(acc.end(), buf, buf + r);

uint8_t type;
std::string payload;
while (chat::try_decode(acc, type, payload)) {
    if (type == chat::kHello) {
        client.name = payload;
    } else if (type == chat::kMessage) {
        broadcast("<" + client.name + "> " + payload + "\n");
    }
}
```



```
}  
}
```

Никакой неоднозначности. Сообщения с переносами строки внутри — нормально. Бинарные blob'ы — тоже.

В главе 45 мы интегрируем это с реактор-сервером для multi-room функционала.

45.10 Главные правила главы

1. **TCP = поток, не пакеты.** Делите сами через фрейминг.
2. **Length-prefixed framing** — стандартный подход. Длина в начале, тело после.
3. **Версионирование с первой версии.** Потом поздно.
4. **Лимит на размер payload.** Иначе DoS-атакой убьют память.
5. **Аккумулятор для частичных reads.** `std::vector<uint8_t> + try_decode` в цикле.
6. **Кольцевой буфер для масштаба** — $O(1)$ операции.
7. **Production protobuf/FlatBuffers/MessagePack**, не своё. Если только не нужна абсолютная компактность.
8. **TLS поверх TCP** для безопасности, не своё шифрование (см. главу 46).

45.11 Маленькое упражнение

1. Запустите `./build/protocol_demo`. Изучите hex-вывод, найдите length-байты, version, type, payload.
2. Измените `MAX_PAYLOAD` на 100, попробуйте закодировать строку 200 байт. Что произойдёт?
3. Подайте в `try_decode` намеренно битые данные: `length=0xFFFFFFFF`. Что вернёт?
4. Реализуйте `encode_binary(type, vector<uint8_t>& payload)` — для бинарных данных, не только string.
5. (Сложнее) Перепишите `reactor_chat_server` с использованием нашего framing protocol. Сервер ждёт `kHello`, потом `kMessage`'и.
6. (Сложнее) Реализуйте ping/pong: каждые 30 секунд сервер шлёт `kPing`, клиент отвечает `kPong`. Без ответа в 60 секунд — `disconnect`.
7. (Сложнее) Реализуйте `RingBuffer` класс с $O(1)$ push/pop. Замените `std::vector` accumulator на него.
8. (Очень сложно) Подключите `protobuf`. Опишите `.proto` файл для наших сообщений, сгенерируйте C++ код, замените наш `encode/try_decode`.

45.12 Что дальше

Глава 45 — **многокомнатный чат**. Используем наш framing-протокол + реактор. Room структура, `unordered_map<string, set<int>>`. Команды `/join`, `/leave`, `broadcast` только в текущую комнату.

Дальше — шифрование XOR pre-key + история на диск (46), сборка-deployment (47). Часть V на финишной прямой.

Глава 46

Глава 45. Многокомнатный чат

Реактор у нас есть (43), бинарный протокол есть (44). Соединим — добавим **комнаты**. Пользователь шлёт Hello с ником, Join с именем комнаты, потом Message'и. Сообщения видны **только участникам той же комнаты**.

Это уже похоже на настоящий Slack/Discord/IRC — на маленьком масштабе.

К концу главы у нас rooms_chat_server поверх готовой инфраструктуры. ~200 строк кода — благодаря тому, что протокол и реактор уже сделаны.

46.1 Что добавляется

Над реактор-сервером из главы 43: - **Каждый клиент имеет** name (после Hello) и room (после Join). - **Структура комнат**: unordered_map<string, set<int>> — имя комнаты → множество fd. - **Команды по типам сообщения**: - Hello → установить name. - Join → войти в комнату (выйти из предыдущей). - Leave → выйти. - Message → broadcast в текущую комнату. - **Notify** от сервера: «вы вошли», «X присоединился», и т.п.

46.2 Структуры данных

```
struct ClientState {
    int fd;
    std::string name;           // пусто, пока не Hello
    std::string room;          // пусто = не в комнате
    std::vector<uint8_t> recv_buf;
    std::vector<uint8_t> send_buf;
};
```

```
std::unordered_map<int, ClientState> clients;
std::unordered_map<std::string, std::unordered_set<int>> rooms;
```

Двойная индексация: - `clients[fd]` — что за клиент с этим `fd`. - `rooms[name]` — кто в комнате `name`.

При входе в комнату вставляем `fd` в `rooms[room]`. При выходе — удаляем.

`unordered_set<int>` для участников — $O(1)$ вставка/удаление, без дубликатов.

46.3 Команды-обработчики

`process_message` — диспетчер по типу:

```
void process_message(...) {
    switch (type) {
        case chat::kHello:
            c.name = payload;
            enqueue_frame(c, kNotify, "Привет, " + c.name);
            break;
        case chat::kJoin:
            if (c.name.empty()) {
                enqueue_frame(c, kNotify, "сначала Hello");
                break;
            }
            leave_room(c, rooms); // если был в другой
            c.room = payload;
            rooms[c.room].insert(fd);
            broadcast_room(rooms, c.room, fd, kNotify,
                          "* " + c.name + " вошёл");
            break;
        case chat::kLeave:
            leave_room(c, rooms);
            break;
        case chat::kMessage:
            if (c.room.empty()) break;
            broadcast_room(rooms, c.room, fd, kMessage,
                          "<" + c.name + "> " + payload);
            break;
```

```

    }
}

```

Состояние клиента — это маленький FSM: 1. **Just connected** — name пусто. Можно только Hello. 2. **Hello'd** — name есть, room пуст. Можно Join. 3. **In room** — name + room. Можно Message, Leave, Join (в другую).

Игнорировать «недопустимые в текущем состоянии» сообщения — корректное поведение.

46.4 leave_room — сложнее, чем кажется

```

void leave_room(...) {
    if (c.room.empty()) return;
    std::string room = c.room;
    rooms[room].erase(fd);
    if (rooms[room].empty()) rooms.erase(room);
    broadcast_room(rooms, room, fd, kNotify,
                   "* " + c.name + " вышел из " + room);
    c.room.clear();
}

```

Тонкости: 1. **Удалить fd из rooms[room]** — раньше очистки c.room. 2. **Если комната пустая** — удалить её из map. Иначе пустые комнаты копятся. 3. **Broadcast** — оповестить оставшихся. Делать **после** удаления, иначе оставивший получит «вышел» сам. 4. **Очистить c.room** — клиент больше не в этой комнате.

При **disconnect** клиента — тоже надо leave_room сначала. У нас в реактор-цикле перед close(fd):

```

if (drop) {
    leave_room(clients, rooms, fd);
    ::close(fd);
    clients.erase(it);
}

```

Иначе rooms[room] будет содержать stale fd — broadcast попытается отправить на закрытый дескриптор. С MSG_NOSIGNAL это безопасно (вернёт EPIPE), но логически мусор.

46.5 broadcast_room

```
void broadcast_room(..., const std::string& room, int from_fd,
                  uint8_t type, const std::string& payload) {
    auto it = rooms.find(room);
    if (it == rooms.end()) return;
    for (int fd : it->second) {
        if (fd == from_fd) continue;
        auto ci = clients.find(fd);
        if (ci == clients.end()) continue;
        enqueue_frame(ci->second, type, payload);
    }
}
```

Идём по `rooms[room]`, для каждого `fd` кладём фрейм в `send_buf`. Реактор-цикл потом сам отправит, когда `POLLOUT` станет готов.

Не блокируем send прямо здесь — мы в одном потоке, `send` мог бы заблокировать весь сервер.

46.6 Mutex vs shared_mutex

В **многопоточном** чате (как глава 41) надо защитить `rooms` мьютексом. Но что если **читателей** много (`broadcast` в большую комнату), а **писателей** мало (`join/leave` редко)?

C++17 даёт `std::shared_mutex`:

```
std::shared_mutex rooms_mtx;

void broadcast(...) {
    std::shared_lock<std::shared_mutex> lock(rooms_mtx); // ← shared
    // ... несколько потоков могут читать одновременно ...
}

void join_room(...) {
    std::unique_lock<std::shared_mutex> lock(rooms_mtx); // ← exclusive
    // ... только один писатель ...
}
```

`shared_lock` — много читателей одновременно. `unique_lock` — эксклюзивный доступ для записи.

Если читателей значительно больше — `shared_mutex` снимает bottleneck.

В **нашем реакторе** — однопоточно, никаких мьютексов не нужно. Преимущество реактор-подхода.

46.7 Multi-room vs one-room-per-client

В нашем варианте клиент **в одной комнате**. Это упрощение.

Реальные чаты (IRC, Discord, Slack): - Клиент может быть **во многих комнатах одновременно**.
- Сообщение в одной комнате — broadcast только этой. - Прерванное соединение — клиент остаётся в комнатах для истории (offline messages).

Для multi-room:

```
struct ClientState {  
    std::set<std::string> rooms;  
};
```

При message нужно специфицировать **target room**:

```
struct MessagePayload {  
    std::string room;  
    std::string text;  
};
```

Это уже не строка, а struct. Сериализация — через свой формат внутри payload или через protobuf.

В IRC решено иначе: команда `PRIVMSG #channel :hello` — target в самой команде. У нас оставим simple version.

46.8 Слабые места реализации

46.8.1 Race на disconnect и broadcast

Сценарий: 1. Поток A читает `recv` в клиенте X → Message. 2. `broadcast_room` итерирует `rooms[r]`, отправляет всем кроме X. 3. fd Y параллельно отключается — `close(Y)`. 4. `broadcast_room` шлёт на Y — EPIPE.

С `MSG_NOSIGNAL` — безопасно (просто ошибка). Без — SIGPIPE убивает сервер.

В реакторе (однопоточно) этой гонки нет — broadcast и close в одном потоке.

46.8.2 Очередь `send_buf` переполняется

Если клиент очень медленный (или вообще завис), `send_buf` накапливается. На частом broadcast — гигабайты в памяти на одного клиента.

Решение: - **Лимит на размер `send_buf`** (например, 1 MB). Если превышен — disconnect. - **Drop policy**: при переполнении выбрасывать старые сообщения, не новые.

В нашем коде — без лимита. Учебное упрощение.

46.8.3 Per-room mutex (если бы было multi-threading)

Если бы делали multi-threaded с разными мьютексами на каждую комнату — могли бы получить deadlock при cross-room операциях. Например, `/move` — клиент из А в В одновременно с другим из В в А.

Решение — **per-room mutex с отсортированным порядком взятия** (взять lock'и в алфавитном порядке имён комнат).

В реакторе — все в одном потоке, нет проблемы.

46.9 Полная реализация

`rooms_chat_server.cpp` объединяет: - Реактор-цикл (как в главе 43). - Framing-протокол через `chat::try_decode` (глава 44). - Управление комнатами.

```
while (true) {
    poll(...);

    // accept
    // for each client:
    //   recv → acc → try_decode loop
    //   process_message → enqueue Notify/Message в send_buf
    //   send out send_buf when POLLOUT
}
```

Всё в одном потоке. Без мьютексов. Простой код.

Сборка:

```
$ make
$ ./build/rooms_chat_server 9001
Rooms chat-server слушает на :9001
```


Чтобы протестировать, нужен **framing-aware** клиент. У нашего `echo_client` нет поддержки протокола — это упражнение читателя в конце главы.

46.10 Что в production-чатах

Slack, Discord, IRC — **миллионы** пользователей. Что выходит за рамки нашей версии:

46.10.1 Persistence

История сообщений в БД. Чтобы можно было прочитать офлайн, scroll-up, поиск. Глава 46 затронет это для нашей мини-версии.

46.10.2 Federation / distributed

Сервер не один — кластер. IRC — старые federation servers. Matrix — современный протокол с decentralized серверами. Discord — закрытый кластер.

46.10.3 Identity

Учётные записи, OAuth, регистрация. Не просто «Hello, моё имя X», а **аутентификация**.

46.10.4 Voice/video

WebRTC, peer-to-peer audio. Совсем другая инфраструктура.

46.10.5 End-to-end encryption

Сервер **не видит** содержимое сообщений (Signal). Сложная криптография: Diffie-Hellman, Double Ratchet, X3DH. Очень trustworthy для приватности.

В главе 46 коснёмся basic шифрования (XOR pre-key) — учебный пример.

46.10.6 Anti-spam, anti-abuse

Rate limiting, captcha, ban-list. Серверная инфраструктура отдельная.

46.10.7 Push notifications

Когда клиент офлайн, push на телефон. APNs/FCM.

46.10.8 Search

Elastic/Algolia для поиска по истории.

Все эти системы — отдельные сервисы вокруг chat-server. Сам chat-server в production — обычно довольно простой кусок.

46.11 Главные правила главы

1. **Двойная индексация** `clients[fd] + rooms[name]` — для эффективных операций.
2. **Удаление пустых комнат** — иначе утечка.
3. **leave_room ДО close(fd)** при `disconnect`.
4. **broadcast не блокирует** — складывает в `send_buf`, реактор отправит.
5. **shared_mutex для read-heavy** структур, если многопоточно.
6. **Простое FSM на клиенте** — `initial / hello'd / in-room`.
7. **Лимит на send_buf** — против медленных клиентов.
8. **Реактор > thread-per-conn** для chat — общая память без мьютексов.

46.12 Маленькое упражнение

1. Соберите `rooms_chat_server`. Запустите.
2. Реализуйте **framing-aware клиент**: `chat_client.cpp` который шлёт Hello/Join/Message фреймы. Подсказка — переиспользуйте `chat::encode` и `chat::try_decode`.
3. Подключите двух клиентов через свой `chat_client`. Один Join “general”, второй тоже. Обменяйтесь сообщениями.
4. Третий клиент Join “secret”. Сообщения от него видны только участникам “secret”.
5. (Сложнее) Поддержите **multi-room** для одного клиента. Команда `/msg <room> <text>`.
6. (Сложнее) Команда `/who [room]` — список участников. Отправлять как Notify с форматированным текстом.
7. (Сложнее) Команда `/list` — список существующих комнат.
8. (Очень сложно) Перепишите на multi-threaded: thread pool обрабатывает события из общей очереди (готовых fd). Реализуйте per-client lock и общий shared_mutex на rooms.

46.13 Что дальше

Глава 46 — **шифрование XOR pre-key + история на диск**. Простое шифрование сообщений (учебное — НЕ для production!) и запись истории чата в файл, чтобы новые клиенты могли её прочитать.

Глава 47 (последняя в Части V) — **сборка и деплой**: static linking, минимальный Docker образ, упаковка для дистрибуции.

После — Часть VI: C++17 как бонус. Финал книги виден.

Глава 47

Глава 46. Шифрование XOR pre-key + история на диск

В этой главе две темы: 1. **Шифрование** — пользователь не хочет, чтобы провайдер/сосед видел его сообщения. Покажем XOR pre-key (**учебный** способ; **категорически не подходит для production!**) и обсудим, что делает TLS. 2. **История на диск** — чтобы клиент после перезапуска сервера или подключения позже увидел свежие сообщения комнаты.

Это **предпоследняя** глава Части V. После — финал части (сборка/деплой) и Часть VI про C++17 как бонус.

47.1 XOR pre-key — учебный пример

Алгоритм:

```
encrypt(plain, key):  
    for i in 0..len(plain):  
        cipher[i] = plain[i] XOR key[i % len(key)]
```

XOR — побитовая операция «исключающее или». Свойство: $A \text{ XOR } B \text{ XOR } B == A$. То есть **decrypt** — **тот же алгоритм** с тем же ключом.

```
void xor_in_place(std::string& data, const std::string& key) {  
    for (std::size_t i = 0; i < data.size(); ++i) {  
        data[i] ^= key[i % key.size()];  
    }  
}
```

Если ключ длиннее plaintext'a — каждый байт XOR'ится со своим. Если короче — ключ повторяется (repeating key).

Пример:

```
plain:    Hello, this is a confidential message!
cipher:   3b 00 0f 1e 0a 58 11 46 5b 1a 16 43 ...
decrypted: Hello, this is a confidential message!
```

Encrypt → cipher выглядит как мусор. Decrypt → возвращает оригинал.

47.1.1 Почему XOR pre-key сломан

Несколько атак:

1. Известный plaintext (known-plaintext attack). Если злоумышленник знает **часть** plaintext'a (например, что сообщения начинаются с «Hello»), он восстанавливает ключ:

```
known = "Hello, "
cipher = ... (известно из перехвата)
key = known XOR cipher[:7]
```

В нашем демо:

```
Знаем начало 'Hello, '.
XOR с cipher[:7] даёт: 'secret1'
Это первые 7 байт реального ключа!
```

Имея ключ, дешифруем **остальное**.

2. Repeated key + статистика. Если ключ короче plaintext'a и повторяется, атакующий собирает много шифровок одного ключа. Через анализ частот букв (буква «о» в русском самая частая) восстанавливает ключ. Это классическая криптоанализ Виженера, известная с **1854 года**.

3. XOR-сравнение шифровок. Если две разные plaintext'a зашифрованы **одним** ключом:

```
cipher1 XOR cipher2 == plain1 XOR plain2
```

Если plain1 короче plain2 — оставшаяся часть = plain2 XOR (повтор ключа), что даёт частичную информацию.

Вывод: XOR pre-key — **не криптография**. Не используйте для реальной защиты. Это **обфускация** — спрятать данные от случайного просмотра, не от атакующего.

47.2 Что делает настоящая криптография

Современные шифры (AES, ChaCha20) обходят все XOR-атаки:

- **Замена и перестановка** битов в каждом блоке (S-boxes).
- **Несколько раундов** для diffusion (один бит plaintext влияет на много бит cipher).
- **Initialization vector (IV)** — случайные данные при каждом шифровании, чтобы один plaintext дал разные cipher'ы.
- **AEAD** (authenticated encryption) — встроенная проверка целостности (MAC).

Реализовывать **своё** шифрование в production — катастрофическая ошибка. «Don't roll your own crypto». Используйте проверенные библиотеки.

47.3 TLS — стандарт для сетевого шифрования

TLS (Transport Layer Security) — стандарт защиты TCP-соединений. HTTPS = HTTP + TLS. Тот же TLS для chat/email/database протоколов.

Что делает TLS:

1. **Handshake**: клиент и сервер договариваются о ключах через **Diffie-Hellman** (или вариант).
2. **Аутентификация сервера** через **X.509 сертификат** (Let's Encrypt и др.).
3. **Симметричное шифрование** (AES-GCM или ChaCha20-Poly1305) данных в обе стороны.
4. **MAC** на каждый фрейм — обнаружение tampering.
5. **Replay protection** — повторное использование старых пакетов не работает.

Главная фишка — **forward secrecy**: даже если ключ скомпрометируют завтра, **старые** перехваченные сообщения нельзя расшифровать. Достигается через **ephemeral Diffie-Hellman**: ключи создаются на сессию, потом уничтожаются.

47.3.1 Diffie-Hellman вкратце

Алиса и Боб хотят общий секрет, общаясь через **публичный канал** (где подслушивают).

1. Договариваются о публичных параметрах p (простое) и g (генератор).
2. Алиса: выбирает секретный a , шлёт Бобу $A = g^a \bmod p$.
3. Боб: выбирает секретный b , шлёт Алисе $B = g^b \bmod p$.
4. Алиса: вычисляет $S = B^a \bmod p$.
5. Боб: вычисляет $S = A^b \bmod p$.
6. **S одинаков** у обоих, но третья сторона его получить не может.

Это **математическое чудо**: подслушав А и В, нельзя из них восстановить S без а или b (задача дискретного логарифмирования, NP-hard).

Современные варианты — **ECDH** на эллиптических кривых (Curve25519): те же гарантии, меньше ключи, быстрее.

47.3.2 TLS в коде

Самостоятельная реализация TLS — тысячи строк сложной криптографии. Используют:

- **OpenSSL** — индустриальный стандарт, C API.
- **BoringSSL** — fork от Google.
- **mbedTLS** — для embedded.
- **wolfSSL** — небольшой коммерческий.

В C++ обёртки: **boost::asio::ssl**, **cpp-httplib** (HTTPS клиент), **Poco::Net::SecureStreamSocket**.

Подключение TLS к нашему чату — это: 1. `SSL_CTX_new` — создать контекст. 2. Загрузить сертификат сервера. 3. Заменить `recv/send` на `SSL_read/SSL_write`. 4. Сделать `handshake` при аксепт.

Не сложно, но за рамками книги. Если делаете production — изучайте OpenSSL.

47.4 История на диск

Когда клиент подключается **в существующую** комнату, он не видел сообщений до этого. Решение — **scrollback**: при `join` клиенту шлют **последние N** сообщений из истории.

Это требует **персистентности** — записывать каждое сообщение в файл.

47.4.1 Формат

Простой append-only бинарный лог (как наш WAL из главы 35):

Каждая запись:

```
[4]  length u32 LE
[4]  timestamp u32 LE (Unix epoch)
[2]  name_len u16 LE
[2]  room_len u16 LE
[2]  text_len u16 LE
[...] name (utf-8)
[...] room (utf-8)
[...] text (utf-8)
```

Похоже на наш framing protocol (глава 44), но field-based — для удобства десериализации.

47.4.2 History класс

```
class History {
public:
    explicit History(const std::string& path);
    ~History();

    void append(const HistoryRecord& r);
    std::vector<HistoryRecord> recent(const std::string& room, std::size_t n);

private:
    std::string path_;
    int fd_;
};
```

append — добавить запись + fsync (как WAL).

recent(room, n) — последние N записей для комнаты. Простая реализация: читать **весь файл**, фильтровать по room, оставлять последние n.

47.4.3 Реализация append

```
void History::append(const HistoryRecord& r) {
    std::vector<std::uint8_t> body;
    w_u32(body, r.timestamp);
    w_u16(body, r.name.size());
    w_u16(body, r.room.size());
    w_u16(body, r.text.size());
    body.insert(body.end(), r.name.begin(), r.name.end());
    body.insert(body.end(), r.room.begin(), r.room.end());
    body.insert(body.end(), r.text.begin(), r.text.end());

    std::vector<std::uint8_t> framed;
    w_u32(framed, body.size());
    framed.insert(framed.end(), body.begin(), body.end());

    write_all(fd_, framed);
}
```



```
fsync(fd_);
}
```

Сначала собрали body, обернули в [length:4] [body], записали + fsync.

O_APPEND при open — атомарность write на конец файла даже при concurrent writers.

47.4.4 Реализация recent

```
std::vector<HistoryRecord> History::recent(const std::string& room, std::size_t n) {
    std::vector<HistoryRecord> all;
    int rfd = ::open(path_.c_str(), O_RDONLY);

    std::uint8_t hdr[4];
    while (read_all(rfd, hdr, 4)) {
        std::uint32_t body_len = r_u32(hdr);
        std::vector<std::uint8_t> body(body_len);
        if (!read_all(rfd, body.data(), body_len)) break;

        HistoryRecord rec;
        // ... парсим поля ...

        if (room.empty() || rec.room == room) {
            all.push_back(std::move(rec));
        }
    }
    ::close(rfd);

    if (all.size() > n) {
        all.erase(all.begin(), all.end() - n);
    }
    return all;
}
```

Linear scan всего файла. На миллионах сообщений — медленно.

Production-решения: - **Индексация по room:** отдельные файлы или B+tree (как в нашей мини-СУБД!). - **Log rotation:** каждый день — свой файл. Recent читает только последние. - **In-memory cache:** последние N сообщений каждой комнаты в RAM. - **Базу данных** (PostgreSQL/MongoDB) вместо своего формата.

В нашем учебном — простой scan.

47.5 Демо

```
$ ./build/history_demo
=== Последние 10 в general ===
[1778530684] [general] <Alice> первое сообщение
[1778530685] [general] <Bob> ответ
[1778530687] [general] <Alice> ещё одно

=== Все записи ===
[1778530684] [general] <Alice> первое сообщение
[1778530685] [general] <Bob> ответ
[1778530686] [tech] <Carol> что-то про код
[1778530687] [general] <Alice> ещё одно
```

История сохранилась, фильтр по комнате работает.

47.6 Интеграция в chat-server

В `rooms_chat_server.cpp` добавляем:

```
chat::History history("data/chat_history.bin");

// При Message:
case chat::kMessage: {
    chat::HistoryRecord r;
    r.timestamp = std::time(nullptr);
    r.name = c.name;
    r.room = c.room;
    r.text = payload;
    history.append(r);
    broadcast_room(...);
    break;
}

// При Join:
case chat::kJoin: {
    // ... установка room ...
```

```

auto recent = history.recent(c.room, 20);
for (const auto& msg : recent) {
    enqueue_frame(c, chat::kMessage,
                  "<" + msg.name + "> " + msg.text);
}
break;
}

```

Новый клиент при входе в комнату сразу получает последние 20 сообщений. Опыт **намного** ближе к настоящему чату.

Опасность: history.append + fsync **блокирует**. На реакторе — застопорит всех клиентов на миллисекунды. Решения: - **Batch append**: накопить N сообщений, один fsync. - **Async writer thread**: основной поток отдаёт сообщение в очередь, отдельный поток пишет. - **Background flush** — fsync раз в секунду, не на каждое сообщение (потеря 1 сек при крахе).

В нашем коде — простой синхронный append. Учебное.

47.7 End-to-end encryption — обзор

Текущая модель: сервер видит сообщения **в открытом виде** (даже с TLS — TLS только защищает канал клиент↔сервер, не от самого сервера).

E2E: сервер видит только зашифрованные blob'ы, не текст. Только адресат может расшифровать.

Используется в **Signal, WhatsApp** (2014+), **iMessage, Telegram secret chats**. Алгоритмы:

- **X3DH** для handshake — обмен ключами через сервер, но без раскрытия секретов серверу.
- **Double Ratchet** для постоянной ротации ключей. Forward secrecy + future secrecy.
- **Out-of-band verification** — пользователи сверяют отпечатки ключей (QR код).

Реализовать E2E **самому** — почти невозможно сделать правильно. Используют **libsignal-protocol** или **libolm** (для Matrix).

Что **проигрывает** server-side в E2E: - Поиск по истории — нельзя (сервер не видит текста). - Multi-device — сложнее (каждое устройство — свой ключ). - Push с предпросмотром — нет полного текста в notification.

Trade-off приватность vs функциональность.

47.8 Главные правила главы

1. **Не пишите свой криптоалгоритм для production.** Используйте AES/ChaCha20.

2. **XOR pre-key — обфускация, не шифрование.**
3. **TLS для сетевой защиты** — OpenSSL/BoringSSL.
4. **Diffie-Hellman** для согласования ключей без раскрытия в канале.
5. **Forward secrecy** — даже компрометация ключа не раскрывает старые сообщения.
6. **append-only лог** для истории — durable, простой.
7. **fsync на каждое сообщение блокирует** — batch или async writer для масштаба.
8. **E2E через libsignal**, не самостоятельно. Сложно.

47.9 Маленькое упражнение

1. Запустите ./build/xor_demo. Понаблюдайте known-plaintext attack — ключ восстанавливается из 7 байт известного начала.
2. Запустите ./build/history_demo. Откройте data/chat_history.bin через xxd — изучите формат.
3. Сделайте history.append 1000 раз в цикле. Засеките время. fsync — узкое место?
4. (Сложнее) Интегрируйте History в rooms_chat_server: записывать каждое Message, при Join отправлять recent 20.
5. (Сложнее) Замените XOR на **AES-GCM** из OpenSSL. Подключите -lcrypto, прочитайте man EVP_EncryptInit_ex.
6. (Сложнее) Реализуйте простой **per-room index**: при append кроме записи в общий лог писать смещение в index_<room>.bin. Recent читает по индексу — быстрее.
7. (Очень сложно) Прочитайте Signal Protocol whitepaper. Реализуйте упрощённый Double Ratchet (без X3DH).
8. (Очень сложно) Подключите OpenSSL к chat-server: TLS handshake при accept, потом SSL_read/SSL_write вместо recv/send.

47.10 Что дальше

Глава 47 — **финал Части V**: сборка и упаковка. Static linking, минимальный Docker, что нужно для деплоя сервера в облако. Подведём итог.

После — Часть VI: C++17 как бонус (optional, variant, filesystem, string_view, ranges/concepts/coroutines обзор). Книга подходит к финишу.

Глава 48

Глава 47. Сборка и деплой

Финальная глава Части V. У нас в `demo-chat/` — 1300+ строк работающего сетевого кода. Сервер запускается локально через `./build/rooms_chat_server`. Но **production** — это другое. В этой главе:

1. Static vs dynamic linking — какой нужен для портабельности.
2. glibc vs musl — почему статический glibc проблемен.
3. Docker для C++ — минимальный образ.
4. Итоги Части V.
5. Что не реализовали.

48.1 Static vs dynamic — снова

Из главы 7: - **Dynamic linking**: бинарь маленький, требует библиотеки в системе при запуске.
- **Static linking**: все библиотеки внутри бинаря, размер большой, ничего не нужно снаружи.

Для **production-сервера** часто нужен static: - Кладёшь файл на чужую машину — работает без установки зависимостей. - Минимальные Docker-образы (без libc на runtime stage). - Меньше surprise'ов «у меня libstdc++ 9, у тебя 11, не совместимо».

Минусы: - Бинарь больше (5-10 MB вместо 100 KB). - Не обновляется безопасностью (новая уязвимость в OpenSSL — пересобрать всё, что её используют).

48.2 g++ -static — что не так с glibc

Пробуем:

```
$ g++ -static -std=c++11 main.cpp -o server
```

На большинстве Linux это даёт **warning** или **fail** на `getaddrinfo/gethostbyname`/похожие: «error: function not implemented»:

warning: Using 'getaddrinfo' in statically linked applications requires at runtime the shared libraries from the glibc version used for linking

Причина: **glibc** для некоторых функций (включая DNS-resolver) использует **NSS** (Name Service Switch). NSS подгружает плагины **динамически в рантайме**. Статически слинковать DNS-resolver технически невозможно — он должен `dlopen` свои плагины.

Эта неприятность мешает «настоящему» static-binary на glibc.

48.3 musl — альтернатива

musl libc — альтернативная C-стандартная библиотека, написанная с нуля для: - Простоты (компактный код, меньше surprise). - Корректной static linking (без NSS-issues). - Скорости и компактности (меньше memory footprint).

Дистрибутивы на musl: **Alpine Linux**, **Void Linux musl edition**.

C musl static linking **работает чисто**:

```
# На Alpine:
$ apk add build-base
$ g++ -static -static-libstdc++ -static-libgcc -std=c++11 -O2 main.cpp -o server
$ file server
server: ELF 64-bit LSB executable, statically linked, ...
$ ./server # запускается на любой Linux машине
```

Размер: ~3-5 MB для нашего chat-server. Без зависимостей.

Чтобы делать musl-сборку с macOS — кросс-компилятор сложно. Проще через Docker.

48.4 Docker для C++

Docker — стандартный способ packaging серверов. Идея: «контейнер» = весь окружение программы (бинарь + библиотеки + конфиг).

48.4.1 Многоступенчатый build

Для C++ идиоматичен **multi-stage build**:

```
# Stage 1: builder
FROM alpine:3.19 AS builder
RUN apk add --no-cache build-base make
WORKDIR /src
COPY . .
RUN make MODE=release \
    CXXFLAGS="-std=c++11 -O2 -DNDEBUG -static -static-libstdc++ \
        -static-libgcc -pthread -Iinclude -MMD -MP" \
    LDFLAGS="-static -pthread"

# Stage 2: minimal runtime
FROM scratch
COPY --from=builder /src/build/rooms_chat_server /chat-server
EXPOSE 9001
ENTRYPOINT ["/chat-server", "9001"]
```

Что происходит:

Stage 1 — Alpine с инструментами сборки. Это «жирный» образ (~500 MB с компилятором). Но мы используем его только для сборки.

Stage 2 — **scratch**, пустой образ. Туда копируется **только** наш бинарь. Никакой **libc**, никакого **shell**, никаких инструментов.

Итог: образ ~5 MB, содержит **только** наш **chat-server**. Меньше surface для атак, быстрее загрузка.

48.4.2 Сборка и запуск

```
$ docker build -t my-chat .
[+] Building 30s
...
Successfully built abc123

$ docker images
my-chat    latest    abc123    5.2MB

$ docker run -p 9001:9001 my-chat
Rooms chat-server слушает на :9001
```

`-p 9001:9001` пробрасывает порт хоста в контейнер.

С хоста (или другого контейнера) можем подключаться к `localhost:9001`. Внутри контейнера — `0.0.0.0:9001`.

48.4.3 Базовые образы для C++

Если **scratch** не подходит (нужен shell для дебага, или образ запускается с пользователя `non-root`):

- **alpine:3** (~5 MB) — минимальный Linux с `busybox`, `musl libc`.
- **distroless/cc** (Google, ~25 MB) — Debian-минимальный, нет shell, есть `libc/libstdc++`.
- **debian:slim** (~70 MB) — Debian без extras.
- **ubuntu** (~70 MB) — Ubuntu.

Для серверов выбирают **alpine** или **distroless** — баланс размера и compatibility.

48.5 Cross-compilation

С macOS собирать Linux-бинарь? Несколько подходов:

1. **Docker для сборки** — самый простой. Внутри `alpine` получаем настоящую сборку.
2. **Cross-compiler**: `aarch64-linux-musl-gcc`, `x86_64-linux-musl-gcc`. Установить через `brew install musl-cross`. Сложнее.
3. **VM на Linux** — старый школьный путь.
4. **CI/CD**: GitHub Actions, CircleCI собирает в нужном окружении.

Для production — обычно CI/CD строит и пушит в registry (Docker Hub, GHCR, GCR). Скачиваешь готовый образ на сервер, запускаешь.

48.6 Что нужно для production-депоя

Полный список того, что у нас **нет**, но в реальном проекте надо:

48.6.1 Logging

Сейчас наш сервер `std::cout << ....` В production: - **Структурированные логи**: JSON-форматные. Поисковые системы (Elastic, Splunk) парсят. - **Уровни**: DEBUG/INFO/WARN/ERROR. - **Ротация**: лог-файлы не должны расти бесконечно. - **Tracing**: distributed tracing для микросервисов (Jaeger, Zipkin).

Библиотеки: **spdlog**, **glog**, **fmt**.

48.6.2 Метрики

Что-то надо мониторить: - Активных клиентов. - Latency сообщений. - Memory usage. - Error rate.

Стандарт — **Prometheus + Grafana**. Программа экспортирует метрики по HTTP, Prometheus собирает.

C++ библиотеки: **prometheus-cpp**.

48.6.3 Конфигурация

Жёсткое кодирование `port = 9001` — плохо. Должно быть: - Через **env vars** (Docker-style): `PORT=9001`. - Через **CLI args**: `--port 9001`. - Через **config файл** (YAML/TOML/JSON).

Библиотеки: **CLI11** для argv-парсинга, **toml++** для TOML.

48.6.4 Health checks

/healthz endpoint, который возвращает 200 OK если сервер живой. Kubernetes/Docker используют для liveness/readiness probes.

48.6.5 Graceful shutdown

При SIGTERM: 1. Перестать принимать новые соединения. 2. Отправить «server shutting down» текущим. 3. Подождать, пока они отключатся (или timeout). 4. Закрыть всё, fsync history. 5. Выйти.

Без этого — резкое прерывание соединений, потеря данных.

48.6.6 Authentication / authorization

Сейчас «представляюсь, имя — Alice». Никакой проверки. В production: - **Аутентификация**: пароль, OAuth, JWT-токены. - **Avalanche-protection**: rate-limiting на login. - **Ban-list**: блок недобросовестных юзеров.

48.6.7 Database вместо файлов

Наш history — bin-файл. На миллионах сообщений медленно. Production — PostgreSQL/Redis/Cassandra. Тогда: - Индексы по комнате/времени. - Search. - Backup/replication.

48.6.8 Horizontal scaling

Один сервер не выдержит 1M+ клиентов. Решения: - **Multiple instances + load balancer** (HAProxy, nginx, AWS ALB). - **Sticky sessions** — клиент возвращается к тому же серверу. - **Inter-server communication** — Redis pub/sub для broadcast между instances.

Этой инфраструктуры может быть **больше**, чем кода самого сервера.

48.7 Часть V — итог

demo-chat/ финальный объём: ~1300 строк C++ кода + ~1300 строк глав.

Что построили: - **Echo server/client** (40) — основа socket API. - **Threaded chat** (41) — multi-client, mutex, condition_variable, deadlock. - **Atomic demo** (42) — race condition, memory ordering, lock-free. - **Reactor chat** (43) — poll-based, single-thread, scale-friendly. - **Binary protocol** (44) — framing, version, partial reads. - **Rooms chat** (45) — multi-room with rooms map, FSM на клиенте. - **XOR + History** (46) — учебное шифрование, append-only лог. - **Docker** (этот) — packaging для production.

Освоено: - **POSIX sockets**: socket/bind/listen/accept/connect/send/recv/getaddrinfo. - **htons/htonl/inet_ntop** для адресов. - **SO_REUSEADDR, MSG_NOSIGNAL, O_NONBLOCK**. - **std::thread/mutex/condition_variable** — concurrency. - **poll/select/epoll/kqueue** — event-driven IO. - **Length-prefixed framing** + аккумулятор для частичных reads. - **Peak-trip-pattern** vs thread-per-connection. - **Forward secrecy, TLS** — обзорно. - **Append-only persistence**. - **Docker multi-stage build**.

48.8 Что не реализовано

Большой список production-фич: - TLS/encryption между клиентами и сервером. - Authentication. - Configuration (env/CLI/file). - Structured logging. - Prometheus metrics. - Graceful shutdown на SIGTERM. - Database вместо файлов для истории. - Multi-server clustering. - E2E encryption. - Client SDK на нескольких языках. - Mobile push notifications. - Voice/video через WebRTC. - Reactions, threads, edit history. - Spam protection, content moderation.

Каждое — недели разработки. Production-чат типа Slack/Discord — это **сотни инженеров на годы**. У нас — концептуальный фундамент.

48.9 Что выучили концептуально

После Части V у вас понимание:

- Как устроено сетевое программирование на Unix.

- **Как масштабируются серверы** до сотен тысяч соединений.
- **Какие модели concurrency** существуют (threads, reactor, async/await).
- **Что такое framing-протоколы** и зачем они нужны.
- **Зачем TLS и Diffie-Hellman.**
- **Как упаковать сервер** в Docker.

Это базовый набор **системного программиста**. Применим к серверам игр, чатам, HTTP-серверам, message queues — везде, где сеть.

48.10 Часть V закрыта

Большая Часть V (главы 40-47, 8 глав) — самая «системная» в книге. Если все четыре проекта (RPG → shell → DB → chat) у вас собираются и работают — вы освоили C++ на уровне хорошего senior-разработчика.

48.11 Главные правила всей Части V

1. **Sockets** — это `fd`. POSIX file IO применяется.
2. **TCP** — поток, не пакеты. Framing обязателен.
3. **Один поток на клиента не масштабируется** — реактор для серьёзных нагрузок.
4. **atomic** для простого, **mutex** для сложного, **реактор лучше обоих** для серверов.
5. **MSG_NOSIGNAL** на серверах — против SIGPIPE.
6. **Length-prefixed framing** — стандарт для бинарных протоколов.
7. **«Don't roll your own crypto»** — используйте TLS и проверенные библиотеки.
8. **Docker scratch + static binary** — минимальные production-образы.

48.12 Маленькое упражнение

1. Соберите Docker-образ: `docker build -t my-chat ..` Запустите: `docker run -p 9001:9001 my-chat`.
2. Замерьте размер образа: `docker images my-chat`. Должно быть < 10 MB.
3. (Сложнее) Замените `FROM scratch` на `FROM alpine` — что изменится в размере?
4. (Сложнее) Подключите `spdlog` для логирования вместо `std::cout`. Добавьте уровни и форматирование.
5. (Сложнее) Подключите **prometheus-cpp** для экспорта метрик. Покажите счётчик «активных клиентов».

6. (Сложнее) Реализуйте graceful shutdown: SIGTERM → закрыть accept-socket → дождаться отключения активных клиентов → выход.
7. (Очень сложно) Подключите OpenSSL для TLS. Сертификат через mkcert для разработки.
8. (Очень сложно) Разверните в production: купите VPS (DigitalOcean/Hetzner), настройте systemd unit, поднимите nginx как reverse проху, подключите Let's Encrypt.

48.13 Что дальше

Часть V закрыта.

Часть VI (главы 48-51) — финальный блок, **C++17 как бонус**. До этого вся книга на C++11. В этой части — обзор того, что появилось в C++14/17 и почему это **меньше boilerplate**:

- **48**: optional, variant, any, if constexpr, structured bindings.
- **49**: std::filesystem.
- **50**: std::string_view, parallel algorithms, std::variant для tagged unions.
- **51**: Куда дальше — обзор C++20/23 (concepts, ranges, coroutines, modules) + маршрут роста.

После 51-й главы — финал книги. Осталось 4 главы.

Глава 49

Глава 48. C++17 — optional, variant, structured bindings, if constexpr

Начинается **Часть VI — C++17 как бонус**. До этого вся книга была на C++11. Четыре финальные главы — обзор того, что появилось в C++14/17, и как это упрощает код. Не отдельный проект, а **переписывание привычных паттернов** через современные средства.

Эта глава — четыре главные новинки C++17: - **std::optional<T>** — «значение или нет». - **std::variant<T1, T2, ...>** — typed union. - **Structured bindings** — `auto [a, b] = ....` - **if constexpr** — ветвление на компиляции.

Все работают на компиляторе C++17 (gcc 7+, clang 5+). Если у вас C++11-only компилятор — главу читайте как «что есть в современном C++», без запуска.

49.1 std::optional

Главная проблема старого C++: как функция возвращает «либо значение, либо ничего»?

Варианты до C++17: 1. **Special value** (-1 для int, “ ” для string). Опасно: что если -1 — реальный возраст? 2. **bool find(key, T& out)**. Out-параметр некрасивый. 3. **std::pair<T, bool>**. Лучше, но многословно. 4. **T* find()**. Указатель — но возможно nullptr-confusion. 5. **Throw exception**. Дорого для нормального «не нашли».

C++17: **std::optional<T>**.

```
#include <optional>

std::optional<int> find_age(const std::string& name) {
    if (name == "Alice") return 30;
```

```

    return std::nullopt;
}

auto a = find_age("Alice");
if (a.has_value()) {
    std::cout << *a << "\n";    // 30
}

// Сокращение через bool:
if (auto b = find_age("Charlie")) {
    std::cout << *b;
}

// Дефолт:
int age = find_age("Charlie").value_or(0);    // 0

// Бросает bad_optional_access если пусто:
int x = find_age("ghost").value();    // throw

```

Что мы получили: - **Тип сам говорит**: «может быть пусто». В сигнатуре `optional<int>`. - **Никакого спец-value**. -1 теперь не нужен как «магия». - **value_or, has_value** — удобные операции.

В `Inventory::find` из нашего RPG (глава 18) мы возвращали `const Item*` со значением `nullptr` для «не нашли». С `std::optional<Item*>` интерфейс был бы яснее. Но **указатели уже могут быть nullptr**, для них `optional` избыточен. `optional` нужен для **значимых типов** (`int`, `double`, `string`, `struct`).

49.1.1 Когда optional хорош

- Возвращаемое значение, которого может не быть.
- Параметр функции, опциональный.
- Поле в `struct`, которое опционально (`alternative` — `pointer`).

49.1.2 Когда optional плох

- Когда `null` имеет смысл «default». В JSON-парсинге `null` vs `missing field` — разные семантики; `optional` их объединяет, теряя различие.
- Когда есть **специальное значение** в домене (например, `INT_MIN` означает «уведомление»).

49.2 std::variant

std::variant<T1, T2, T3> — tagged union. **Один из** N типов одновременно.

```
#include <variant>

struct Weapon { int damage; };
struct Armor  { int defense; };
struct Potion { int heal_amount; };

using Item = std::variant<Weapon, Armor, Potion>;

Item it = Weapon{8};
```

it — это **значение** одного из трёх типов. Хранится **на стеке** (или в struct, где Item — поле).
Размер = max(sizeof Ti) + tag (~1 байт).

Доступ:

```
// 1. holds_alternative + get
if (std::holds_alternative<Weapon>(it)) {
    std::cout << std::get<Weapon>(it).damage;
}

// 2. std::visit — рекомендуется
std::visit([](const auto& v) {
    using T = std::decay_t<decltype(v)>;
    if constexpr (std::is_same_v<T, Weapon>) {
        std::cout << "Weapon, dmg=" << v.damage;
    } else if constexpr (std::is_same_v<T, Armor>) {
        std::cout << "Armor, def=" << v.defense;
    }
}, it);
```

std::visit берёт **callable** и применяет к текущему типу. Лямбда с auto& принимает любой тип; if constexpr диспатчит.

49.2.1 variant vs наследование

В нашем RPG (главы 17-18) у нас была иерархия Item → Weapon/Armor/Consumable через виртуальные функции и unique_ptr<Item>.

C variant:

```
using Item = std::variant<Weapon, Armor, Consumable>;
std::vector<Item> inventory; // значения, не указатели на куче
```

Плюсы variant: - **Значения на стеке/в vector** — нет аллокации, кэш-friendly. - **Закрытое множество типов** — компилятор знает все. Может проверять exhaustiveness visit. - **Без виртуальных функций** — меньше overhead, агрессивная инлайн.

Минусы: - **Нельзя добавить новый тип** без правок везде, где visit. Иерархия с virtual — open для extension. - **Тяжелее код visit'ов** в простых случаях.

Когда что: - Полиморфизм с **открытым набором типов** (плагины, расширения) → наследование. - **Закрытый набор**, известный заранее → variant. - **AST-узлы, JSON-значения, состояния FSM** → variant идеален.

В нашем RPG — мог быть и variant. Если бы делали с нуля на C++17 — стоило бы подумать.

49.3 std::any

std::any — «любой тип». Внутри type-erased значение.

```
#include <any>

std::any x = 42;
x = std::string("hello");
x = 3.14;

if (x.type() == typeid(double)) {
    double d = std::any_cast<double>(x);
}
```

any хранит **любой** copyable тип. Тип проверяется в runtime через RTTI.

Редко полезно. Variant лучше когда множество типов закрытое. Any — когда **открытое и неизвестно заранее** (например, конфигурация с произвольными значениями, generic event bus).

Цена: аллокация в куче для больших типов + runtime overhead на cast'ы. **Используйте редко.**

49.4 Structured bindings

Декомпозиция кортежей/пар/struct'ов в локальные имена.

До C++17:

```
std::pair<std::string, int> p = ...;
std::string name = p.first;
int age = p.second;
```

C++17:

```
auto [name, age] = p;
```

Намного короче. Особенно с tuple'ами:

```
auto [id, name, height] = std::tuple{42, "Alice", 5.7};
```

49.4.1 С map

```
std::map<std::string, int> ages = {...};

// До C++17:
for (const auto& kv : ages) {
    std::cout << kv.first << " = " << kv.second;
}

// C++17:
for (const auto& [key, value] : ages) {
    std::cout << key << " = " << value;
}
```

Намного читабельнее. **Идиома** для C++17+.

49.4.2 С insert / emplace

```
std::map<std::string, int> m;
auto [it, inserted] = m.insert({"key", 42});
if (inserted) {
    std::cout << "новая запись\n";
}
```

insert возвращает pair<iterator, bool> — теперь раскладывается в два имени.

49.4.3 Co struct

```
struct Point { int x; int y; };
Point p{10, 20};

auto [x, y] = p;           // копии
auto& [rx, ry] = p;        // ссылки
const auto& [cx, cy] = p;  // const-ссылки
```

auto& — реально alias. Меняешь rx — меняешь p.x.

49.4.4 Ограничения

- **Имена** фиксированы порядком полей, не их именами в struct. Если struct поменяется — все bindings перепишите.
- Для **tuple** / **pair** работает через `std::get<N>` — нужно правильное количество имён.
- В C++20 добавили **auto& [_, value]** для игнорирования (пока нет, в C++17 — все имена должны быть «настоящими»).

49.5 if constexpr

Ветвление **на компиляции**, не в runtime.

До C++17 — **SFINAE**:

```
template <typename T>
typename std::enable_if<std::is_integral<T>::value, void>::type
process(T x) {
    // только для integral
}

template <typename T>
typename std::enable_if<std::is_floating_point<T>::value, void>::type
process(T x) {
    // только для float
}
```

Уродливо. Каждый вариант — отдельная функция, разные сигнатуры с `enable_if`.

C++17 — **одна функция**:

```
template <typename T>
void process(const T& v) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "integer: " << v * 2;
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "float: " << std::sqrt(v);
    } else if constexpr (std::is_same_v<T, std::string>) {
        std::cout << "string size: " << v.size();
    } else {
        std::cout << "other";
    }
}
```

if constexpr **полностью убирает** «не выбранные» ветки **из** компиляции. Не «evaluates to false» — а вообще не компилируется.

Это важно: внутри ветки можно использовать **операции, недопустимые** для других типов:

```
template <typename T>
void f(const T& v) {
    if constexpr (std::is_integral_v<T>) {
        v % 2; // не работает для float, но if constexpr спасает
    }
}
```

Если бы был обычный if — `v % 2` пытался бы компилироваться для всех типов. Падало бы для float.

49.5.1 typeid и type traits

`std::is_integral_v<T>`, `std::is_floating_point_v<T>`, `std::is_same_v<T, X>` — `_v` суффиксы от C++17, удобнее `::value`:

```
// C++11
std::is_integral<T>::value

// C++17
std::is_integral_v<T>
```

Просто короче.

`std::decay_t<decltype(v)>` — убирает `const/ref/&&` с типа. В лямбда-callback'ах часто нужно.

49.6 Inline variables

Мелочь, но полезная:

```
// До C++17 — в .h:
extern const int MAX_SIZE;

// В .cpp:
const int MAX_SIZE = 100;

// C++17:
inline constexpr int MAX_SIZE = 100;
```

inline для переменной — разрешает определение в .h без multiple definition. Удобно для констант в header-only библиотеках.

49.7 std::filesystem (упоминание)

Один из больших добавлений C++17 — <filesystem>. Кросс-платформенная работа с путями, файлами, каталогами.

Это **глава 49**, отдельно. Просто упомянем, что в C++17 теперь есть нормальное API.

49.8 Когда переходить на C++17

В **новом проекте** на macOS/Linux/Windows — берите C++17 минимум, **сейчас уже C++20**. Старый код с C++11 — оставляйте; не переписывайте просто ради миграции.

В **учебной книге** мы остались на C++11 потому, что: 1. Всё работает на старых компиляторах. 2. C++17-фичи в основном **синтаксический сахар** — concepts остаются те же.

Когда **точно нужен C++17**: - std::filesystem (вместо <dirent.h> или WinAPI). - std::optional для чистых API. - std::variant для AST/state machines. - if constexpr для template-heavy код.

49.9 Где переписать наш код

В нашей книге было бы естественно использовать:

49.9.1 parser save (RPG, глава 24)

```
// Сейчас:
Player p2("Other", 50);
```

```

save::read(path, p2, w2, current);

// C variant + optional:
struct SaveData {
    std::string name;
    int hp, max_hp;
    std::string current_loc;
    std::vector<std::variant<Weapon, Armor, Consumable>> items;
};

std::optional<SaveData> read_save(const std::string& path);

```

Чище: «нет данных» через optional, типы предметов через variant.

49.9.2 B+tree (DB, глава 34)

find сейчас возвращает bool + out-param:

```
bool find(int64_t key, int64_t& out_value);
```

С optional:

```
std::optional<int64_t> find(int64_t key);
```

Использование: if (auto v = tree.find(42)) { ... }. Чище.

49.9.3 parser SQL (DB, глава 36)

Statement сейчас — struct с kind-enum и «по умолчанию пустыми» полями. С variant:

```

struct CreateTable { std::string name; std::vector<...> cols; };
struct Insert { std::string table; std::vector<int64_t> values; };
struct Select { std::string table; bool has_where; ... };

using Statement = std::variant<CreateTable, Insert, Select>;

```

Каждый тип — своя структура. visit диспатчит. Чище, типобезопаснее.

49.10 Полное демо

demo-cpp17/ содержит 4 файла: - optional_demo.cpp — основы optional. - variant_demo.cpp — Item-иерархия через variant. - structured_demo.cpp — bindings везде. - if_constexpr_demo.cpp — type-dispatched шаблон.

```
$ cd demo-cpp17
$ make
$ ./build/optional_demo
Alice: 30
Charlie не найден
Charlie age (or 0): 0
bad_optional_access поймали

$ ./build/variant_demo
=== std::visit + if constexpr ===
Weapon, dmg=8
Armor, def=4
Potion, heal=15
```

Каждый демонстрирует одну фичу автономно. Чтобы прочувствовать.

49.11 Главные правила главы

1. `std::optional<T>` для «значение или нет» вместо магических спец-значений.
2. `std::variant<...>` для закрытого множества типов вместо иерархии virtual.
3. `std::visit + if constexpr` — идиоматический визитор для variant.
4. `auto [a, b] = pair` в range-for с map — стандарт C++17+.
5. `if constexpr` убирает SFINAE-ужас для type-dispatched шаблонов.
6. `_v / _t` суффиксы type traits — удобнее `::value / ::type`.
7. `inline constexpr` для header-only констант.
8. C++17 — **реальный default** для нового кода в 2026.

49.12 Маленькое упражнение

1. Соберите и запустите все 4 demo. Изучите вывод.
2. Перепишите функцию `World::find` из главы 16 чтобы возвращала `std::optional<const Location*>` (или просто `Location*`, но через variant с типом `ErrorReason` для других ошибок).

3. Замените в `VPlusTree::find` (глава 34) сигнатуру с out-param на `std::optional<int64_t>`. Сравните читаемость.
4. (Сложнее) Перепишите `Statement` (глава 36) на `variant` из `CreateTable/Insert/Select`. `Executor` через `std::visit + if constexpr`.
5. (Сложнее) Перепишите `Item`-иерархию (главы 17-18) на `variant`. Сравните производительность (1M insert/lookup).
6. Используйте `[[nodiscard]]` (C++17) для функций, чей возвращаемый результат **не должен игнорироваться**. Например, `[[nodiscard]] std::optional<int> find(...)`.
7. Прочитайте `std::optional::and_then / transform` (C++23). `Monadic interface`.
8. Изучите `std::variant<T...>::index()` — возвращает индекс активного варианта (0-based).

49.13 Что дальше

Глава 49 — **`std::filesystem`**. Кросс-платформенный API для работы с файлами/папками. Зачем нужен, как заменить наш ручной `mkdir/open` в RPG и DB-проектах.

Дальше — `string_view + parallel algorithms` (50), куда расти после книги — обзор C++20/23 (51). Финал близок.

Глава 50

Глава 49. std::filesystem

В наших проектах мы работали с файлами и папками **руками**: - В demo-shell (главы 26-31) — POSIX-вызовы open/read/close/opendir/stat. - В demo-db — mkdir, pread/pwrite/fsync. - В demo-chat — open/read/write для history.

Всё это **POSIX**, на macOS/Linux работает. На **Windows** — другой API: CreateFile, FindFirstFile, путь с \, не /. Кросс-платформенный код приходится писать через #ifdef _WIN32.

C++17 добавил **std::filesystem** — стандартный кросс-платформенный API. В этой главе обзорно: path, операции, итерация по каталогам, разбор пути. Финальный демо собирает всё.

50.1 std::filesystem::path

Главный класс — fs::path. Представляет путь, **не** проверяет существование.

```
#include <filesystem>
namespace fs = std::filesystem;

fs::path p1 = "/Users/serik/docs/report.pdf";
fs::path p2 = "data";
fs::path p3 = p2 / "config" / "app.toml";    // оператор / для конкатенации
// p3 = "data/config/app.toml"
```

Оператор / — кросс-платформенный, использует правильный разделитель: / на Unix, \ на Windows. Никакого if (windows) ... руками.

50.1.1 Разбор пути

```
fs::path p = "/Users/alice/docs/report.pdf";

p.parent_path();    // "/Users/alice/docs"
p.filename();        // "report.pdf"
p.stem();            // "report" (filename без extension)
p.extension();       // ".pdf"
p.root_path();       // "/"
p.root_directory(); // "/"
```

Удобно для: - Найти родительский каталог. - Узнать тип файла по расширению. - Заменить расширение: `p.replace_extension(".bak")`.

Тип `fs::path` имеет операторы сравнения, `hash` — можно класть в `std::unordered_map<fs::path, T>`.

50.2 Файловые операции

```
fs::create_directory(path);    // создать одну папку
fs::create_directories(path);  // и все промежуточные (как mkdir -p)
fs::remove(path);              // удалить файл или ПУСТУЮ папку
fs::remove_all(path);          // удалить папку со всем содержимым (rm -rf!)
fs::copy_file(from, to);
fs::rename(from, to);          // также для перемещения

fs::exists(path);
fs::is_regular_file(path);
fs::is_directory(path);
fs::is_symlink(path);
fs::is_empty(path);

fs::file_size(path);           // в байтах
fs::last_write_time(path);     // время модификации
```

Никаких POSIX-вызовов, без `#ifdef`. Работает одинаково на Linux/macOS/Windows.

50.2.1 Текущая папка

```
fs::path cwd = fs::current_path();
fs::current_path("/tmp"); // изменить cwd
```

Эквивалент getcwd / chdir в POSIX.

50.2.2 Symlinks и absolute paths

```
fs::path abs = fs::absolute(p); // относительный → абсолютный
fs::path canonical = fs::canonical(p); // абсолютный без `..` и симлинков
fs::create_symlink("/tmp", "/home/link");
```

50.3 Итерация по каталогу

Два класса:

fs::directory_iterator — один уровень (как ls):

```
for (const auto& entry : fs::directory_iterator("/tmp")) {
    std::cout << entry.path() << "\n";
}
```

fs::recursive_directory_iterator — все вложенные (как find . -type f):

```
for (const auto& entry : fs::recursive_directory_iterator("/tmp")) {
    if (entry.is_regular_file()) {
        std::cout << entry.path()
                    << " " << entry.file_size() << " bytes\n";
    }
}
```

entry — это fs::directory_entry. Имеет path(), is_regular_file(), is_directory(), file_size(), и так далее. **Cached** — данные читаются один раз при обходе, не stat'ом на каждый запрос.

Это **гигантское** упрощение vs POSIX. Чтобы пройти дерево с opendir/readdir вручную, нужно стек, рекурсию, обработку errno. С recursive_directory_iterator — 3 строки.

50.4 Демо

utils/filesystem_demo.cpp:

```

namespace fs = std::filesystem;

int main() {
    fs::path base = "/tmp/fs_demo";
    fs::remove_all(base);
    fs::create_directories(base / "subdir1" / "deeper");

    std::ofstream(base / "a.txt") << "hello a";
    std::ofstream(base / "subdir1" / "b.txt") << "hello b";
    std::ofstream(base / "subdir1" / "deeper" / "c.txt") << "hello c";

    for (const auto& entry : fs::recursive_directory_iterator(base)) {
        if (entry.is_regular_file()) {
            std::cout << entry.path().string()
                      << " (" << entry.file_size() << " bytes)\n";
        }
    }

    fs::path file = base / "a.txt";
    std::cout << "exists: " << fs::exists(file) << "\n";
    std::cout << "size: " << fs::file_size(file) << "\n";

    fs::copy_file(file, base / "a_copy.txt");
    fs::rename(base / "a_copy.txt", base / "a_renamed.txt");

    fs::path p = "/Users/alice/docs/report.pdf";
    std::cout << p.parent_path() << "\n";    // /Users/alice/docs
    std::cout << p.filename() << "\n";        // report.pdf
    std::cout << p.stem() << "\n";            // report
    std::cout << p.extension() << "\n";      // .pdf

    fs::remove_all(base);
}

```

Запуск:

```

=== Все файлы в "/tmp/fs_demo" ===
/tmp/fs_demo/a.txt (7 bytes)
/tmp/fs_demo/subdir1/

```

```

/tmp/fs_demo/subdir1/deeper/
/tmp/fs_demo/subdir1/deeper/c.txt  (7 bytes)
/tmp/fs_demo/subdir1/b.txt  (7 bytes)

```

```

exists: 1
is_regular_file: 1
file_size: 7

```

```

=== Разбор "/Users/alice/docs/report.pdf" ===
parent_path: "/Users/alice/docs"
filename:    "report.pdf"
stem:       "report"
extension:   ".pdf"

```

50 строк кода — а покрывает «создать дерево, найти все файлы, разобрать пути». На POSIX это было бы 200 строк со стеками и `errno`.

50.5 Где в наших проектах

В наших главах можно было заменить:

50.5.1 demo-shell

Команда `ls` — глава 31 (`utils/mycat.cpp/mywc.cpp`):

```

// Старый, POSIX:
DIR* dir = opendir(path);
struct dirent* entry;
while ((entry = readdir(dir)) != nullptr) {
    // entry->d_name — имя файла
    // нужен ещё stat() для типа
}
closedir(dir);

// Новый, std::filesystem:
for (const auto& e : fs::directory_iterator(path)) {
    std::cout << e.path().filename().string() << "\n";
}

```

5 строк вместо 10, плюс работает на Windows.

50.5.2 demo-db

В database.cpp:

```
::mkdir(dir.c_str(), 0755); // POSIX
fs::create_directories(dir); // C++17, кросс-платформенно
```

fs::create_directories — что-то типа mkdir -p, создаёт все промежуточные. POSIX mkdir ругается, если родительского нет.

50.5.3 demo-chat

В history.h — open файла. Стандартный std::ofstream справляется, но fs::file_size(path) для проверки лимита — удобно.

50.5.4 Обход проектной директории

В RPG — для загрузки всех save-файлов из data/saves/:

```
std::vector<std::string> list_saves() {
    std::vector<std::string> result;
    for (const auto& e : fs::directory_iterator("data/saves")) {
        if (e.is_regular_file() && e.path().extension() == ".sav") {
            result.push_back(e.path().stem().string());
        }
    }
    return result;
}
```

Раньше пришлось бы opendir + readdir + проверять расширение строки руками. Сейчас — 5 строк.

50.6 Ошибки

По умолчанию операции **бросают** fs::filesystem_error при ошибке. Можно через std::error_code (без throw):

```
std::error_code ec;
fs::create_directory("/no/permission/here", ec);
if (ec) {
    std::cerr << "ошибка: " << ec.message() << "\n";
}
```

Подходит когда «попытаться, и если не вышло — продолжить». Например, при создании папки с возможностью «уже существует».

50.7 Производительность

`std::filesystem` обычно **немного медленнее** ручного POSIX: - Каждая операция оборачивается C++-классом. - Path хранит строку (alloc). - Iterator кэширует stat — выгодно, если используете несколько раз; накладно, если нет.

Для большинства приложений разница неощутима. На очень частых файловых операциях (миллион stat'ов) — может быть заметно.

50.8 Windows nuances

Кросс-платформа не идеальна. На Windows есть особенности:

- **Пути в UTF-16** (`wchar_t`). `fs::path` использует `wstring` на Windows, `string` на POSIX. Для печати в `cout` нужно конвертировать.
- **Backslash** — Windows исторически. `fs::path` нормализует.
- **Привилегии** — `create_directory` в `C:\Windows` потребует `admin`.
- **Регистр имён** — Windows case-insensitive. `Foo.txt == foo.txt`.

`fs::path::string()` возвращает `narrow string`. На Windows с не-ASCII именами может потеряться информация. Безопаснее `.u8string()` или `.wstring()`.

50.9 Fallback под C++11

Если у вас компилятор без C++17, есть несколько fallback:

1. **Boost.Filesystem** — почти то же API, работает на C++03+. `#include <boost/filesystem.hpp>`.
2. **TS version** — `<experimental/filesystem>`. Работает на GCC 5.3+ / Clang 3.9+.
3. **POSIX/WinAPI** вручную. Что мы и делали.

На современных компиляторах `<filesystem>` уже доступен — без Boost.

50.10 Главные правила главы

1. `fs::path` для путей — кросс-платформенно, `parser` встроен.
2. **Оператор /** для конкатенации, не строки.
3. `fs::create_directories` делает «`mkdir -p`».
4. `fs::recursive_directory_iterator` для обхода дерева — 3 строки.

5. `fs::remove_all` это `rm -rf` — будьте осторожны!
6. `error_code` для `non-throwing` API. Бросать по умолчанию.
7. **Windows: backslash, `wchar_t`, регистр** — не идеальная переносимость.
8. **Современный default** для нового кода.

50.11 Маленькое упражнение

1. Соберите и запустите `./build/filesystem_demo`. Изучите вывод.
2. Перепишите `Database::Database` в `demo-db`: вместо `::mkdir(dir.c_str(), 0755)` используйте `fs::create_directories(dir)`.
3. Напишите `ls-utility` через `fs::directory_iterator` за 10 строк.
4. Реализуйте `du` — рекурсивный обход и сумма `file_size`. На вашем `home directory` какая суммарная?
5. (Сложнее) Реализуйте функцию `list_saves()` (см. выше) в RPG. Добавьте команду `saves` в игровой цикл RPG для просмотра списка сохранений.
6. (Сложнее) Замените `open/read/write` в `history` (`demo-chat`, глава 46) на `std::ifstream/std::ofstream + fs::file_size`.
7. (Сложнее) Сделайте `watch-mode`: периодически проверять `last_write_time` файла, если изменился — перезагрузить.
8. Прочитайте `fs::space(path)` — свободное место на диске. Когда полезно?

50.12 Что дальше

Глава 50 — **`std::string_view`, `parallel algorithms`**. Лёгкая обёртка для строк без копий, и `std::execution::par` для параллельных алгоритмов C++17.

Глава 51 — финал: куда расти после книги. C++20 `concepts`, `ranges`, `coroutines`, `modules` — обзорно.

Финал близок.

Глава 51

Глава 50. `std::string_view` и параллельные алгоритмы

Ещё две новинки C++17, которые стоят отдельного разговора. Первая — `std::string_view` — встречается в реальном коде постоянно, и именно её не хватало нашим парсерам в RPG, shell и DB: мы рубили строки на токены и каждый раз создавали новый `std::string`, то есть новую аллокацию на кучу. Вторая — **параллельные алгоритмы** — это `std::sort`, `std::for_each` и компания, которым одним аргументом говоришь «выполняйся на всех ядрах».

С `string_view` мы разберёмся подробно, с демо и переписыванием токенизатора. С параллельными алгоритмами — обзорно: к ним есть важная оговорка про поддержку компиляторами, о которой я честно скажу ниже.

51.1 `std::string_view` — строка, которой не владеют

Вспомните, что такое `std::string`: он **владеет** буфером символов на куче. Создал строку — выделилась память; скопировал строку — выделилась ещё раз; вышел из области видимости — освободилась. Это надёжно, но не бесплатно.

Теперь представьте парсер. На входе одна большая строка:

```
std::string line = "move north quickly";
```

Мы хотим разбить её на слова. Классический способ:

```
std::vector<std::string> tokenize(const std::string& line) {
    std::vector<std::string> tokens;
    // ... режем по пробелам, каждый кусок копируем в новый std::string
```



```
tokens.push_back(line.substr(start, len)); // ← аллокация на каждый токен!
}
```

line.substr(...) возвращает **новый** std::string — отдельный буфер на куче. Три слова — три аллокации. А ведь символы уже лежат в памяти, в исходной line. Зачем их копировать?

std::string_view — это **указатель плюс длина**, и больше ничего:

```
#include <string_view>

std::string_view sv = "hello world";
// внутри: const char* ptr → 'h', std::size_t size = 11
```

Он не владеет символами, не копирует их, не аллоцирует память. Он просто **смотрит** на чужие символы. Отсюда название — «view», вид, окно.

51.1.1 Что умеет string_view

Почти весь интерфейс std::string, только для чтения:

```
std::string_view sv = "hello world";

sv.size();           // 11
sv.empty();          // false
sv[0];               // 'h'
sv.front();          // 'h'
sv.back();           // 'd'
sv.substr(0, 5);      // "hello" — это тоже string_view, НЕ копия!
sv.find("world");     // 6
sv == "hello world"; // сравнение работает
```

Главная магия — substr. У std::string он делает копию. У string_view — возвращает **другой view** на тот же буфер: просто сдвигает указатель и меняет длину. Ноль аллокаций.

Есть и две специфичные операции — подрезать концы без копий:

```
sv.remove_prefix(2); // сдвинуть начало на 2 символа вправо
sv.remove_suffix(3); // отрезать 3 символа с конца
```

Это меняет сам sv (его указатель и длину), исходные символы не трогаются.

51.1.2 Главная ловушка: view не продлевает жизнь

string_view не владеет данными — значит, символы, на которые он смотрит, **обязаны его пережить**. Иначе получите указатель на освобождённую память (мы это называли *dangling* в главе 11).

Классическая ошибка:

```
std::string_view bad() {
    std::string local = "временная строка";
    return local;          // ← возвращаем view на локальную строку!
}                          // local умирает здесь, view смотрит в никуда
```

Компилируется без ошибок, а при использовании — UB. Правило простое: **не храните string_view дольше, чем живёт строка-источник**. Как параметр функции и короткоживущая локальная переменная — идеально. Как поле класса, переживающее источник, — опасно.

Второй нюанс: string_view **не гарантирует ноль-терминатор**. Обычный C-string заканчивается '\0', и функции вроде printf("%s") или fopen на это рассчитывают. А string_view — это кусок чужого буфера, после него может стоять что угодно. Поэтому нельзя писать some_c_api(sv.data()) — передастся указатель, но без длины C-функция прочтёт за границу. Если нужен настоящий C-string, придётся скопировать: std::string(sv).c_str().

51.1.3 Параметр функции: string_view вместо const std::string&

Долгие годы каноном для «принять строку только на чтение» был const std::string&. У него есть скрытая цена:

```
void process(const std::string& s);

process("literal");          // создаётся временный std::string — аллокация!
process(some_char_buffer);  // тоже временный std::string
```

Литерал "literal" — это const char[8], не std::string. Чтобы связать его с const std::string&, компилятор молча создаёт временный std::string — с аллокацией. А string_view принимает и литерал, и std::string, и кусок буфера **без единой копии**:

```
void process(std::string_view s);    // современный канон для read-only строк

process("literal");                // ноль аллокаций — view на литерал
process(my_string);                // ноль аллокаций — view на буфер std::string
process(my_string.substr(2, 4));    // тут substr строки всё ещё копирует, осторожно
```

Правило: для строкового параметра, который вы только читаете, берите std::string_view по значению (он маленький — указатель и размер, копировать дёшево).

51.2 Демо: токенизатор без копий

utils/string_view_demo.cpp переписывает наш парсер на string_view:

```
#include <iostream>
#include <string>
#include <string_view>
#include <vector>

std::vector<std::string_view> tokenize(std::string_view line) {
    std::vector<std::string_view> tokens;
    std::size_t i = 0;
    while (i < line.size()) {
        while (i < line.size() && line[i] == ' ') ++i;    // пропустить пробелы
        if (i >= line.size()) break;
        std::size_t start = i;
        while (i < line.size() && line[i] != ' ') ++i;
        tokens.push_back(line.substr(start, i - start)); // срез, не копия
    }
    return tokens;
}

bool starts_with(std::string_view s, std::string_view prefix) {
    return s.size() >= prefix.size() &&
        s.substr(0, prefix.size()) == prefix;
}

int main() {
    std::string command = " move north quickly ";

    for (std::string_view tok : tokenize(command)) {
        std::cout << "[" << tok << "]" (len=" << tok.size() << ")\\n";
    }

    std::cout << std::boolalpha
        << starts_with("move north", "move") << "\\n"    // true
```

```

        << starts_with("go east", "move") << "\n";        // false
    }

```

Запуск:

```

[move] (len=4)
[north] (len=5)
[quickly] (len=7)

```

true

false

Сравните с версией на std::string: при трёх токенах раньше было три аллокации на кучу, теперь — **ноль**. На горячем пути парсера (миллион строк лога) это ощутимая разница. И при этом сам vector<string_view> хранит лишь пары «указатель + длина», смотрящие в исходную command.

51.2.1 Где в наших проектах

- **demo-shell** (глава 29): tokenize рубит командную строку. Сейчас он копирует каждый токен в std::string. На string_view — режет без копий, а в std::string превращает только то, что реально уйдёт в exesvr.
- **demo-db** (глава 36): мини-SQL парсер разбирает запрос на слова. То же самое — слова это срезы исходного запроса.
- **demo-rpg** (глава 23): парсер мира из world.txt бьёт строки на ключ и значение. string_view для ключа избавит от лишних строк.

Важная оговорка для всех трёх: пока живёт исходная строка-запрос, view-токены валидны. Как только функция вернула управление и строка умерла — токены протухли. Поэтому string_view тут идеален как **временный** инструмент внутри одного разбора, а не как способ хранить токены «на потом».

51.3 Параллельные алгоритмы

C++17 добавил алгоритмам из <algorithm> и <numeric> необязательный первый аргумент — **политику выполнения** (execution policy). Одним словом вы говорите «можешь параллелить»:

```

#include <algorithm>
#include <execution>
#include <vector>

```

```
std::vector<int> data = /* миллион чисел */;

std::sort(std::execution::seq, data.begin(), data.end());      // как обычно, один поток
std::sort(std::execution::par, data.begin(), data.end());      // на всех ядрах
std::sort(std::execution::par_unseq, data.begin(), data.end()); // ядра + векторизация (SIMD)
```

Три политики:

- **seq** — последовательно, как всегда. То же, что без аргумента.
- **par** — параллельно: библиотека сама раскидывает работу по потокам пула. Вы не пишете ни `std::thread`, ни мьютексов — всё внутри.
- **par_unseq** — параллельно и с разрешением переставлять/векторизовать операции (SIMD). Самая агрессивная; ваша функция-предикат не должна при этом блокироваться или менять общее состояние.

Работает не только `sort`. Параллельные перегрузки получили десятки алгоритмов: `for_each`, `transform`, `count_if`, `reduce` (параллельный аналог `accumulate`), `transform_reduce` и другие.

```
// Сумма миллиона элементов на всех ядрах:
long total = std::reduce(std::execution::par, data.begin(), data.end(), 0L);
```

Почему `reduce`, а не привычный `accumulate`? Потому что `accumulate` обязан складывать **строго слева направо** — это последовательный контракт, параллелить его нельзя. `reduce` не гарантирует порядок, поэтому может разбить диапазон на куски, сложить их на разных ядрах и объединить. Цена свободы: операция должна быть ассоциативной (для сложения чисел — да; для вычитания или конкатенации строк — порядок важен, будут сюрпризы).

51.3.1 Когда это выгодно — и когда нет

Параллелизм не бесплатен: раскидать работу по потокам и собрать результат стоит времени. Выигрыш появляется, только когда работы **много**:

- Маленький массив (сотни элементов) — `par` будет **медленнее** `seq` из-за накладных расходов на потоки.
- Большой массив (сотни тысяч, миллионы) и тяжёлая операция на каждом элементе — вот где `par` окупается, иногда в разы.

Как и везде с производительностью (глава 14): не угадывайте — **измеряйте**. Замените `seq` на `par`, прогоните бенчмарк на реальных данных, сравните.

51.3.2 Честная оговорка про поддержку

А теперь то, о чём многие tutorиалы умалчивают. Параллельные алгоритмы есть в **стандарте** C++17, но в **реализациях** — неравномерно:

- **MSVC** — поддерживает с Visual Studio 2017 15.7, из коробки.
- **GCC** — поддерживает с версии 9, но требует слинковаться с библиотекой Intel **TBB** (`-ltbb`); без неё `<execution>` либо не найдётся, либо `par` молча выполнится последовательно.
- **Apple Clang / libc++** (то, что стоит на macOS по умолчанию) — **на момент написания не предоставляет** параллельные политики: `std::execution::par` просто не существует, код не скомпилируется.

Поэтому в нашем `demo-cpp17` параллельного демо нет — на типичном `make` книги он бы не собрался, а я обещал, что весь код в книге собирается. Если хотите попробовать вживую — поставьте GCC и соберите с `-ltbb`, либо запустите под MSVC на Windows. Сам же приём — «добавь политику первым аргументом» — выучить полезно уже сейчас: переносимость подтянется, а в C++20 у `std::reduce/transform_reduce` появятся ещё и удобные `ranges`-обёртки (о `ranges` — в следующей, финальной главе).

51.4 Главные правила главы

1. **`string_view` — это указатель + длина**, без владения и без аллокации.
2. **`substr` у `view` не копирует** — возвращает другой `view` на тот же буфер.
3. **`View` не продлевает жизнь источника** — не храните его дольше, чем живёт строка.
4. **Нет гарантии нуль-терминатора** — не передавайте `sv.data()` в C-API, ждущий C-string.
5. **Read-only строковый параметр** — берите **`std::string_view` по значению**, не `const std::string&`.
6. **Параллельные алгоритмы — политика первым аргументом**: `seq` / `par` / `par_unseq`.
7. **`reduce` вместо `accumulate`** для параллельной свёртки; операция должна быть ассоциативной.
8. **`par` окупается только на больших объёмах** — на малых медленнее; всегда измеряйте.

51.5 Маленькое упражнение

1. Соберите и запустите `./build/string_view_demo`. Добавьте в `command` табы (`\t`) и научите `tokenize` считать их разделителями.
2. Напишите `split(std::string_view s, char sep)`, возвращающую `std::vector<std::string_view>` — разбиение по произвольному разделителю.
3. Реализуйте `trim(std::string_view)` — отрезает пробелы с обоих концов через `remove_prefix/remove_suffix`, без единой копии.

4. Перепишите `tokenize` из `demo-shell` (глава 29) на `string_view`, а в `std::string` превращайте только финальные аргументы для `execvp`. Замерьте, изменилось ли число аллокаций (например, через переопределённый оператор `new` со счётчиком).
5. Воспроизведите ловушку `dangling`: верните `string_view` на локальную строку, соберите с `-fsanitize=address` и посмотрите, что скажет санитайзер.
6. (Сложнее) Если у вас есть GCC с TBB: напишите бенчмарк, который сортирует вектор из 10 млн `int` с `seq` и с `par`, и сравните время. На скольких элементах `par` начинает выигрывать?
7. (Сложнее) Перепишите подсчёт суммы в бенчмарке `demo-db` (глава 39) через `std::reduce`. Объясните, почему для суммы результат совпадёт с `accumulate`, а для вычитания — нет.
8. Прочитайте про `std::transform_reduce` (C++17) — «map + reduce» за один проход. Где в наших проектах он бы пригодился?

51.6 Что дальше

Осталась одна глава — **финал**. Мы пройдёмся обзором по C++20 и C++23: концепты вместо SFINAE-уродов из главы 20, `ranges` как читаемая замена «итератор-итератор», корутины и модули. И главное — поговорим, **куда расти дальше**, когда книга закончится.

Глава 52

Глава 51. Куда дальше — обзор C++20/23 и маршрут роста

Вот и финал. Вся книга стояла на C++11 — фундаменте, который соберётся даже на старом компиляторе. Две предыдущие главы показали, что добавил C++17. Эта, последняя, — взгляд вперёд: что принесли C++20 и C++23, и, главное, **куда расти**, когда вы закроете книгу.

Сразу честно: код в этой главе — **обзорный**. Многие из C++20 уже работают в свежих GCC, Clang и MSVC, но не везде и не всё (модули и корутины до сих пор местами болезненны). Поэтому не ждите, что каждый листинг соберётся «прямо сейчас» — читайте их как «вот к чему язык пришёл и куда движется». Цель главы — чтобы вы узнали эти штуки в чужом коде и понимали, зачем они.

52.1 Концепты — конец SFINAE-ужасов

Вспомните главу 20. Чтобы шаблон принимал только подходящие типы, мы городили SFINAE — `std::enable_if`, проверки через `decltype`, нечитаемые конструкции, которые я сам назвал страшными. И сообщения об ошибках были под стать: вместо «тип не подходит» компилятор вываливал три экрана подстановок.

C++20 ввёл **концепты** (concepts) — именованные требования к типам, написанные по-человечески:

```
// C++20
#include <concepts>

template <typename T>
concept Numeric = std::integral<T> || std::floating_point<T>;
```



```

template <Numeric T>           // принимаем только числа
T square(T x) { return x * x; }

square(5);           // ок
square(3.14);        // ок
square("hi");        // ошибка компиляции — и сообщение понятное:
                      // "constraints not satisfied: 'std::string' is not Numeric"

```

Сравните с SFINAE из главы 20 — это небо и земля. Концепт `Numeric` читается как обычное условие, а ошибка прямо говорит, **что** не так. В стандартной библиотеке уже есть готовые концепты: `std::integral`, `std::floating_point`, `std::sortable`, `std::copyable` и десятки других.

Можно требовать конкретное «умение» типа через `requires`:

```

template <typename T>
concept Drawable = requires(T obj) {
    obj.draw();           // у типа должен быть метод draw()
};

```

Если бы наша иерархия `Item` в RPG (главы 17-18) была на шаблонах, концепт `requires(T it) { it.describe(); }` заменил бы виртуальный интерфейс там, где он нужен только на компиляции.

52.2 Ranges — конвейеры вместо «итератор, итератор»

Всю книгу алгоритмы STL мы вызывали через пару итераторов:

```

std::sort(v.begin(), v.end());
std::copy_if(v.begin(), v.end(), std::back_inserter(out), is_even);

```

Это работает, но многословно, и сцепить два алгоритма в цепочку нельзя — приходится заводить промежуточные контейнеры. C++20 **ranges** решает обе беды. Во-первых, алгоритм берёт контейнер целиком:

```

// C++20
#include <ranges>
#include <algorithm>

std::ranges::sort(v);           // вместо v.begin(), v.end()

```

Во-вторых — и это главное — появились **views** и оператор `|`, собирающий обработку в конвейер (если писали на JS, Python или знаете LINQ из C#/Stream из Java — узнаете идею):

```
// C++20
namespace rv = std::views;

std::vector<int> nums = {1, 2, 3, 4, 5, 6, 7, 8};

auto result = nums
    | rv::filter([](int n){ return n % 2 == 0; }) // только чётные
    | rv::transform([](int n){ return n * n; }); // возвести в квадрат
// result «лениво» даёт: 4, 16, 36, 64

for (int x : result) std::cout << x << " ";
```

Ключевое слово — **лениво** (lazy). `filter` и `transform` не создают промежуточных векторов; элементы протекают по конвейеру по одному, только когда вы их запрашиваете в цикле. Это и читабельнее цепочки `copy_if+transform`, и без лишних аллокаций.

52.3 Корутины — функции, умеющие приостанавливаться

В главе 43 мы написали реактор на `poll`: один поток жонглирует сотней соединений, вручную сохраняя состояние каждого клиента в структуре. Код получился рабочим, но громоздким — машина состояний размазана по обработчику событий.

Корутины (coroutines, C++20) — это функции, которые умеют **приостановиться** на середине, вернуть управление, а потом продолжить с того же места. Состояние между приостановками компилятор сохраняет за вас:

```
// C++20, обзорно
Task handle_client(Socket sock) {
    while (true) {
        std::string msg = co_await sock.read(); // приостановились, ждём данные
        if (msg.empty()) break;
        co_await sock.write("эхo: " + msg); // снова приостановились
    }
}
```

`co_await` — точка приостановки: «жди, пока данные придут, а пока отдай поток другим». Снаружи это выглядит как простой последовательный код — `read`, потом `write`, потом снова

— но под капотом работает тот самый реактор, только машину состояний строит компилятор, а не вы. Появились ключевые слова `co_await`, `co_yield`, `co_return`.

Честно: корутины в C++20 — **сложная** тема. Стандарт дал низкоуровневый механизм (`promise_type`, `coroutine_handle`), но не дал готовых высокоуровневых обёрток вроде `Task` из примера — их приходится либо писать самому, либо брать из библиотек. Это та область, где «легко пользоваться» пока не равно «легко устроено».

52.4 Модули — прощание с `#include`

Всю книгу мы подключали заголовки через `#include`. В главе 7 я объяснял, как это работает: препроцессор тупо **вставляет** текст заголовка в каждый `.cpp`. Отсюда болезни — медленная компиляция (один и тот же `<vector>` парсится в каждом файле заново), хрупкость порядка включений, война с `include guards`.

Модули (modules, C++20) должны это похоронить:

```
// C++20, обзорно
// файл math.ixx — объявляем модуль
export module math;

export int square(int x) { return x * x; } // export = видно снаружи

// файл main.cpp
import math;                               // не #include!

int main() { return square(5); }
```

`import math;` не вставляет текст — компилятор один раз компилирует модуль в бинарный формат и переиспользует. Быстрее, чище, без препроцессорной магии.

Почему «пока больно»: на момент написания поддержка модулей в сборочных системах (CMake и прочих) ещё сырая, флаги у GCC/Clang/MSVC разные, а большие проекты на модули переходят медленно. Механизм правильный, экосистема догоняет.

52.5 Несколько штрихов C++23

C++23 — не революция, а шлифовка C++20. Что встретите чаще всего:

- **`std::expected<T, E>`** — «значение **или** ошибка» в одном типе. Мы в главе 22 обсуждали коды ошибок против исключений; `expected` — это аккуратный третий путь, без бросков и без `out`-параметров.
- **`std::print` / `std::println`** — наконец человеческий вывод вместо `std::cout << std::println("сумма = {}", total);`. По духу это `std::format` из C++20 (тоже как `f`-строки в Python), но сразу с печатью.
- **`std::flat_map` / `std::flat_set`** — словарь и множество, хранящие данные в плоском отсортированном массиве: компактнее и дружелюбнее к кэшу, чем дерево, когда вставок мало, а чтений много.

Язык продолжает развиваться: C++26 уже на подходе. Но гнаться за каждой новинкой не нужно — фундамент, который вы освоили, никуда не денется.

52.6 Куда расти после этой книги

Вы прошли путь от «что такое бит» до собственной СУБД и сетевого чата. Это **много**. Но C++ — язык, который изучают годами. Вот честный маршрут, что делать дальше.

Закончите упражнения-«посложнее». В конце почти каждой главы есть задачи со звёздочкой, которые я нарочно не разбирал. Допилите RPG (новые предметы, бои, карта побольше), доведите `shell` до удобного (автодополнение, & для фона), научите СУБД делать JOIN двух таблиц, добавьте в чат приватные сообщения. Доделанный своими руками проект учит больше десяти прочитанных глав.

Освойте инструменты профессионала. В книге мы сознательно сидели на голом `make` и `g++`, чтобы видеть каждую стадию. В реальной работе вас ждут:

- **CMake** — стандарт сборки в индустрии. Возьмите любой наш демо-проект и перепишите его сборку на CMake.
- **Санитайзеры** — `-fsanitize=address` и `-fsanitize=undefined` ловят утечки, выходы за границы и UB на лету. Прогоните под ними RPG и DB — почти наверняка что-то всплывёт.
- **Отладчик** — `gdb` или `lldb`. Точки останова, просмотр стека, пошаговое выполнение. После `cout`-отладки это другой мир.
- **GoogleTest** — мы писали самодельный мини-фреймворк тестов (глава 25). Перенесите тесты RPG на GoogleTest и почувствуйте разницу.
- **Профайлер** — `perf` на Linux, Instruments на macOS. Найдите узкое место в бенчмарке СУБД не на глаз, а замером.

Изучите то, что мы обошли. Книга — фундамент, не энциклопедия. За её рамками остались: подробная многопоточность (`std::async`, `std::future`, lock-free структуры данных), сетевые

библиотеки (Boost.Asio), метапрограммирование шаблонов вглубь, аллокаторы, и весь C++20/23 из этой главы по-настоящему, а не обзорно.

Куда смотреть за надёжной информацией:

- **cppreference.com** — справочник, которому стоит доверять. Открывайте его на каждый незнакомый тип или функцию.
- **Стандарт и предложения (proposals)** на open-std.org — если хочется понять, *почему* язык такой.
- **Конференции** — доклады CppCon, Meeting C++ есть на YouTube бесплатно. Там объясняют не «как», а «зачем» и «как НЕ надо».
- **Чужой код** — читайте исходники хороших библиотек. Сначала тяжело, потом незаменимо.

И последнее. Главного, чему я хотел научить, нет в синтаксисе. Это привычка **понимать, что происходит под капотом**: где аллокация, кто владеет памятью, когда вызовется деструктор, что делает каждая стадия компиляции. Мы ради этого и сидели на C++11 и голом make — чтобы ничего не пряталось за магией. Если теперь, глядя на любую строку C++, вы спрашиваете себя «а что здесь происходит на самом деле?» — значит, книга своё дело сделала.

Спасибо, что дошли до конца. Идите и пишите код.